RK Rockit MPI Developer Guide

Идентификатор документа: RK-KF-YF-576

Версия выпуска: V1.7.3

Дата: 2023.04

Уровень конфиденциальности: совершенно секретно секретно внутренний документ открытый документ

Отказ от ответственности

Настоящий документ предоставляется «как есть». Rockchip Electronics Co., Ltd. («Компания») не делает никаких явных или подразумеваемых заявлений и не предоставляет гарантий относительно точности, надёжности, полноты, товарной пригодности, пригодности для определённой цели и отсутствия нарушений прав в отношении любых утверждений, сведений и материалов настоящего документа. Документ предназначен только для справочного руководства по использованию.

В связи с обновлением версии продукта или по другим причинам настоящий документ может периодически обновляться или изменяться без предварительного уведомления.

Заявление о товарных знаках

«Rockchip» и соответствующие наименования являются зарегистрированными товарными знаками Компании и принадлежат Компании.

Все прочие зарегистрированные товарные знаки и товарные знаки, которые могут упоминаться в настоящем документе, принадлежат их соответствующим владельцам.

Авторское право © 2021 Rockchip Electronics Co., Ltd.

За пределами добросовестного использования без письменного разрешения Компании никаким организациям и физическим лицам не разрешается извлекать или копировать часть либо весь документ и распространять его в любой форме.

Rockchip Electronics Co., Ltd.

Rockchip Electronics Co., Ltd.

Адрес: здание 18, зона A, Software Park, Tongpan Road, г. Фучжоу, провинция Фуцзянь

Веб-сайт: www.rock-chips.com

Телефон службы поддержки: +86-4007-700-590

Факс службы поддержки: +86-591-83951833

Адрес электронной почты службы поддержки: [fae@rock-chips.com]

Предисловие

Обзор

В настоящем документе в основном описаны API MPI и типы данных.

Версии продуктов

Название микросхемы	Версия ядра
RV1126	4.19
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V1.1	Aaron.sun	2021-12-11	Объединение всей документации MPI
V1.2	Aaron.sun	2021-12-18	Обновление всей документации MPI
V1.2.1	qingxu.zhou	2021-12-24	Объединение всей документации MPI
V1.2.2	qingxu.zhou	2022-01-06	Обновление документации MPI модуля VI
V1.3.0	qingxu.zhou	2022-01-08	Объединение всей документации MPI
V1.3.1	qingxu.zhou	2022-01-22	Обновление документации MPI модулей RGN и VENC
V1.3.2	qingxu.zhou	2022-01-27	Обновление документации MPI модулей VPSS и AVS
V1.5.0	Aaron.sun	2022-03-18	Обновление всей документации MPI
V1.5.2	Colum.jin	2022-04-30	Обновление документации MPI модулей VPSS, AVS, VENC и DUMP
V1.5.3	Colum.jin	2022-05-07	Обновление документации MPI модулей AVS и DUMP
V1.6.0	Colum.jin	2022-06-07	Добавлен GDC; обновлена документация MPI модулей RGN, SYS, VDEC, VENC, VI и VPSS
V1.7.0	Aaron.sun	2022-11-02	Обновление всей документации
V1.7.3	Aaron.sun	2023-04-27	Обновление всей документации

Общее содержание

Управление системой

Управление памятью

Видеовход

Подсистема обработки видео

Декодирование видео

Кодирование видео

Видеовыход

Подсистема видеографики

Обработка графики

Аудиоподсистема

Управление областями

Подсистема коррекции геометрических искажений

Панорамное сшивание

Описание отладочной информации DUMP

Управление системой

Предисловие

Обзор

Модуль управления системой реализует общие функциональные интерфейсы RK MPI и предоставляет системные функции, управление крупными блоками физической памяти и другие возможности.

Версии продуктов

Название микросхемы	Версия ядра
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.2.0	XLM	2021-01-19	Начальная версия
V0.3.0	XLM	2021-02-07	Добавлены указатели интерфейсов и структур, добавлены описания структур
V0.4.0	FXW	2021-03-27	Добавлен интерфейс настройки режима входного потока
V0.5.0	XLM	2021-05-08	Добавлены интерфейсы работы с общим пулом памяти
V0.6.0	XLM	2021-08-18	Добавлены иллюстрации, уточнены определения некоторых понятий
V0.7.0	XLM	2021-11-18	Добавлены запрос и преобразование глобального уникального идентификатора блока буфера
V1.0.0	XLM	2021-12-06	Добавлены отдельные примечания
V1.1.0	LXH	2022-02-21	Добавлено описание RK_MPI_MB_GetLength
V1.1.1	XLM	2022-03-17	Добавлено описание общего пула буферов памяти
V1.1.2	XLM	2022-03-19	Добавлено описание интерфейсов установки и получения размера смещения блока буфера
V1.1.3	XLM	2022-08-15	Добавлены примечания о порядке байтов форматов изображений
V1.2.0	XLM	2022-08-15	Добавлен раздел с описанием управления частотой кадров

Содержание

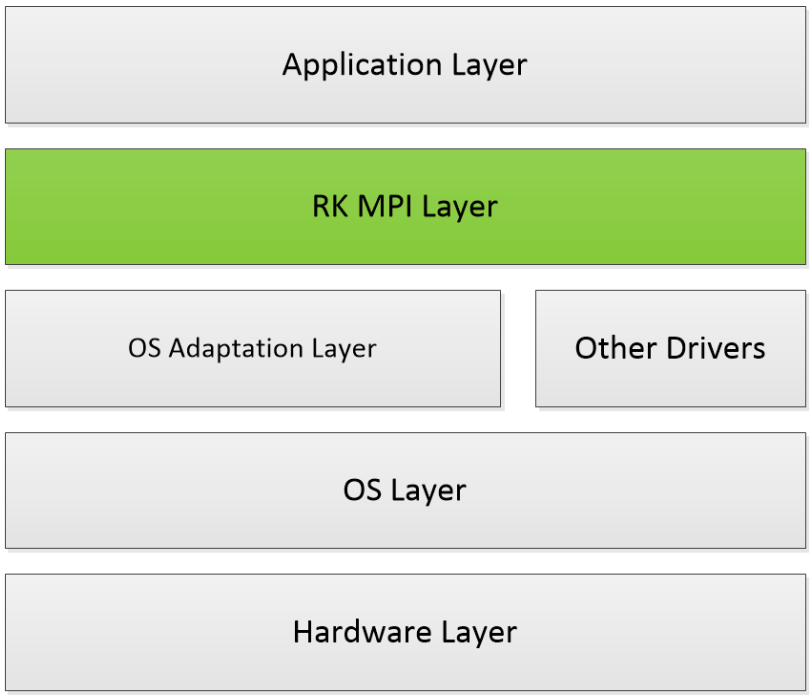
Обзор системы
Описание функций
Справочник API
Типы данных
Коды ошибок SYS

1. Обзор системы

Интерфейс обработки мультимедиа Rockchip (Rockchip Media Process Interface, сокращённо RK MPI), предоставляемый Rockchip, обеспечивает быструю разработку прикладного ПО. Платформа объединяет аппаратные ресурсы RK, скрывает от прикладного ПО сложную низкоуровневую обработку, зависящую от микросхемы, и предоставляет приложениям прямые интерфейсы для выполнения соответствующих функций. Платформа позволяет быстро разрабатывать следующие функции: захват входного видео; кодирование H.265/H.264/JPEG; декодирование H.265/H.264/JPEG; вывод и отображение видео; предварительная обработка видеоизображений (включая обрезку, масштабирование и поворот); интеллектуальная обработка; захват и вывод аудио; кодирование и декодирование аудио и т. д.

1.1 Архитектура системы

Рисунок 1-1. Схема архитектуры системы



- Уровень приложений

  Прикладная программная система, разрабатываемая пользователем на основе RK MPI и других драйверов.

- Уровень RK MPI

  Реализует функции обработки мультимедиа на основе уровня адаптации (существующих инкапсулированных интерфейсов адаптации микросхем RK). Скрывает аппаратные детали обработки и предоставляет приложениям унифицированные интерфейсы функций обработки мультимедиа.

- Уровень адаптации операционной системы

  Основан на интерфейсах драйверов, предоставляемых существующими аппаратными модулями микросхем RK.

- Уровень операционной системы

  Основан на системах Linux и Android.

- Аппаратный уровень

  Аппаратный уровень состоит из микросхемы RK и необходимых периферийных компонентов.

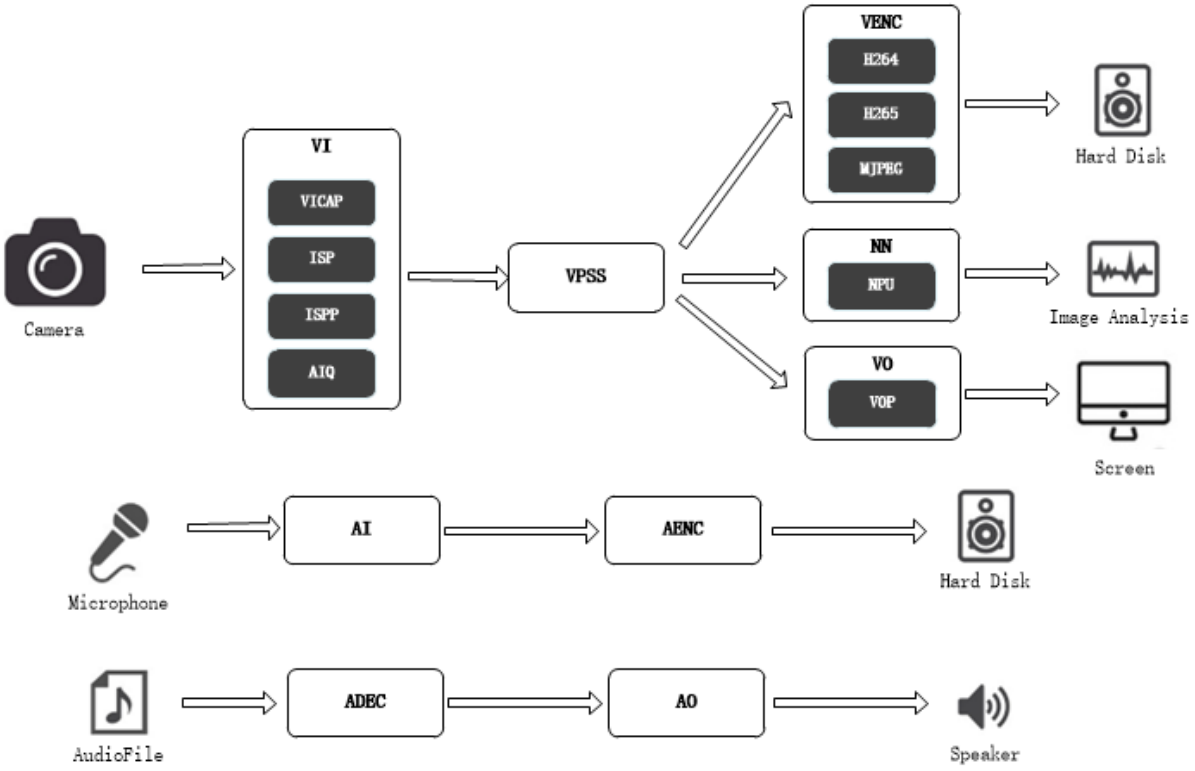
1.2 Описание модулей

Сокращение	Полное наименование	Краткое описание функций
SYS	system	Реализует инициализацию и деинициализацию системы RK MPI, управление потоками данных и платформенные функциональные интерфейсы.
MB	Media Buffer & Pool	Реализует универсальные интерфейсы памяти, управление памятью и пулами памяти.
VPSS	Video Process Sub-System	Подсистема обработки видео. Поддерживаемые функции обработки изображений включают FRC, CROP, Scale, преобразование формата пикселей, поворот на заданный угол, Cover, Overlay, Mosaic, Mirror/Flip, сжатие и распаковку и т. д.
VI	Video Input	Модуль видеовхода (VI) осуществляет сбор изображений от Sensor, HDMI IN и других источников; VI сохраняет полученные данные в указанной области памяти.
VDEC	Video Decoder	Предоставляет интерфейс аппаратного декодирования видео и реализует функции декодирования видео.
VENC	Video Encoder	Предоставляет интерфейс аппаратного кодирования видео и реализует функции кодирования видео.
VO	Video Output	Модуль активно считывает видео- и графические данные из соответствующих областей памяти и выводит их через соответствующее устройство отображения.
RGN	Region Manager	Модуль REGION предназначен для централизованного управления ресурсами областей OSD, маскирования, мозаики, рисования линий и т. д. и передаёт сведения об этих областях другим модулям.
VGS	Video Graphics System	Система видеографики, реализующая различные функции обработки изображений через интерфейсы GPU.
TDE	Two Dimensional Engine	Механизм двумерной обработки изображений, реализующий функции 2D-обработки через интерфейсы RGA.
AI	Audio Input	Модуль захвата аудио.
ADEC	Audio Decoder	Модуль декодирования аудио; на платформе микросхем RK декодирование аудио обычно выполняется программно.
AENC	Audio Encoder	Модуль кодирования аудио; на платформе микросхем RK кодирование аудио обычно выполняется программно.
AO	Audio Output	Модуль вывода аудио, выводящий аудиоданные PCM на оборудование различных звуковых карт.
AVS	Any View Stitching	Панорамное сшивание нескольких изображений и вывод результата в заданном режиме проекции.
PVS	Planar Video Stitching	Двумерное сшивание нескольких изображений.

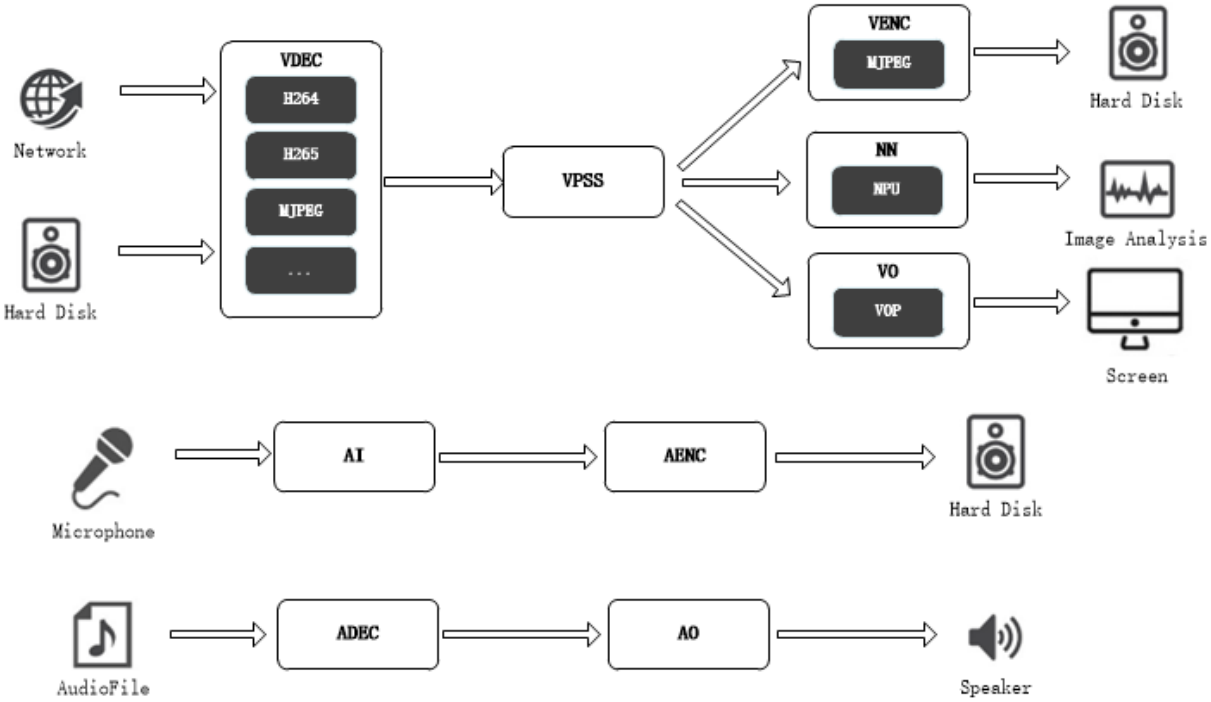
1.3 Типовые применения платформы обработки мультимедиа

Типовые изделия на базе RK MPI включают IPC, NVR, системы отображения и управления и т. д. Ниже кратко описаны рабочие процессы различных типовых изделий.

1.3.1 Типовое применение IPC



1.3.2 Типовое применение NVR



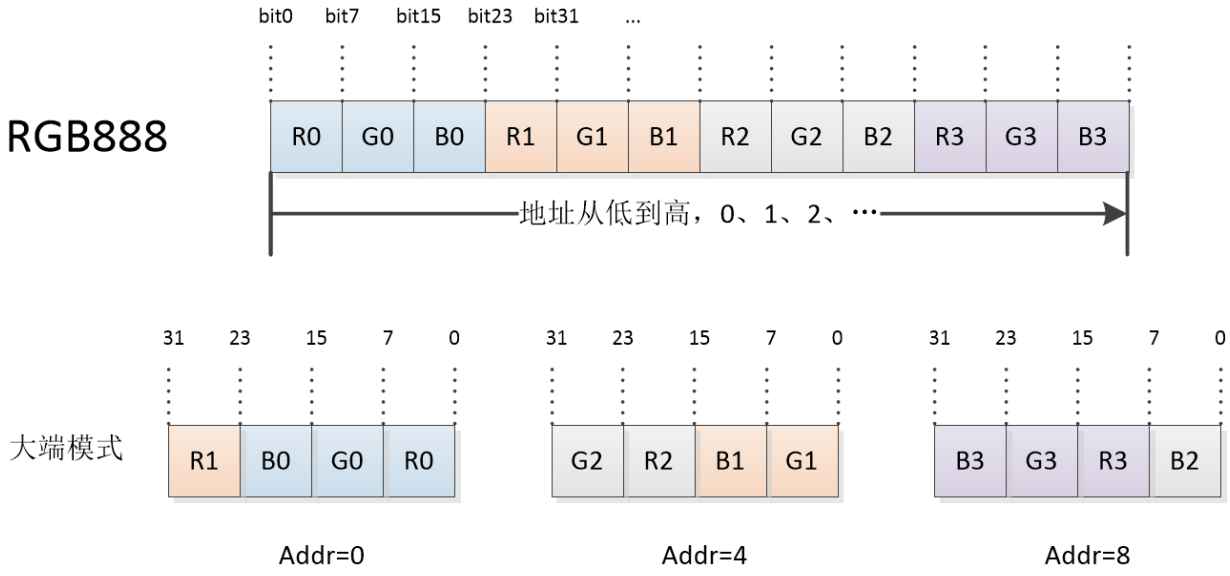
- Модуль VI захватывает видеоизображения, может выполнять их обрезку, масштабирование, зеркальное отражение и другую обработку и выводить несколько изображений с различным разрешением.
- Модуль VDEC декодирует закодированный видеопоток и передаёт разобранные данные изображения в VPSS для обработки либо непосредственно в VO для отображения. Поддерживается декодирование видеопотоков различных форматов, включая H.264/H.265/MJPEG/MPEG2/MPEG4.
- Модуль VPSS принимает изображения от VI и модуля декодирования, может выполнять обрезку, масштабирование, поворот, зеркальное отражение, наложение и другую обработку и формировать из одного источника несколько изображений с различным разрешением для кодирования, предварительного просмотра, интеллектуального анализа или снимков.
- Модуль VENC принимает изображения, захваченные VI и обработанные VPSS, может накладывать изображения OSD, заданные пользователем через модуль Region, после чего кодирует их по различным протоколам и выводит соответствующие потоки.
- Модуль VO принимает выходные изображения после обработки VPSS, может выполнять управление воспроизведением и другую обработку, после чего выводит их периферийному видеоустройству по настроенному пользователем протоколу вывода.
- Модуль AI захватывает аудиоданные, затем модуль AENC кодирует их по различным аудиопротоколам и выводит аудиопоток.
- Аудиопоток, полученный пользователем из сети или с периферийного устройства хранения, можно непосредственно передать модулю ADEC. ADEC поддерживает декодирование потоков различных аудиоформатов; после декодирования данные передаются модулю AO для воспроизведения звука.

1.4 Описание понятий

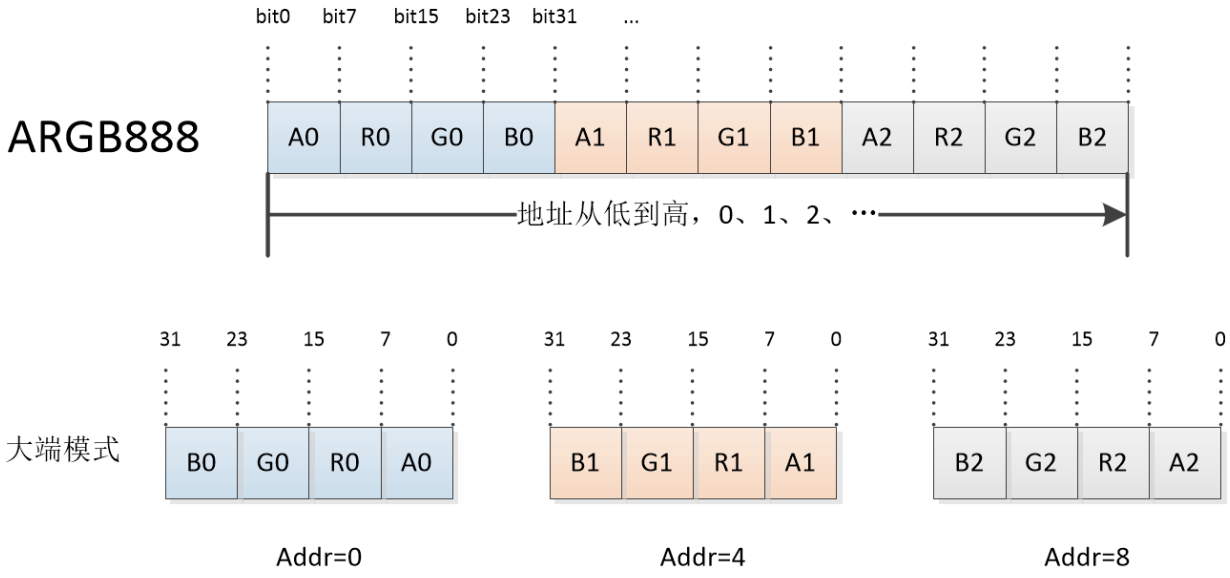
1.4.1 Расположение данных форматов пикселей

Перечисление RKMPI PIXEL_FORMAT_E определено для режима big-endian, то есть определение перечисления соответствует расположению хранимых данных в режиме big-endian. В качестве примера рассмотрено хранение изображений форматов RGB888/ARGB8888:

- Рисунок 1-2. Схема хранения изображения RGB888 (режим big-endian)



- Рисунок 1-3. Схема хранения изображения ARGB8888 (режим big-endian)



Примечание Перечисление PIXEL_FORMAT_E определено для режима big-endian, однако форматы ARGB1555 и RGB565 обрабатываются особым образом. Для унификации установлено требование: если входное изображение имеет формат ARGB1555/RGB565, данные изображения должны храниться в памяти в режиме little-endian; соответствующие значения перечисления PIXEL_FORMAT_E приведены ниже:

Перечисление	Описание хранения
RK_FMT_RGB565	Данные изображения располагаются в памяти как BGR565 little-endian, R находится в младших разрядах
RK_FMT_BGR565	Данные изображения располагаются в памяти как RGB565 little-endian, R находится в старших разрядах
RK_FMT_ARGB1555	Данные изображения располагаются в памяти как BGRA5551 little-endian, A находится в младших разрядах
RK_FMT_BGRA5551	Данные изображения располагаются в памяти как ARGB1555 little-endian, A находится в старших разрядах

Для других форматов RGB рекомендуется расположение big-endian. Если обрабатываемые данные изображения расположены в режиме little-endian, соответствующее значение PIXEL_FORMAT_E также следует задавать в обратном порядке.

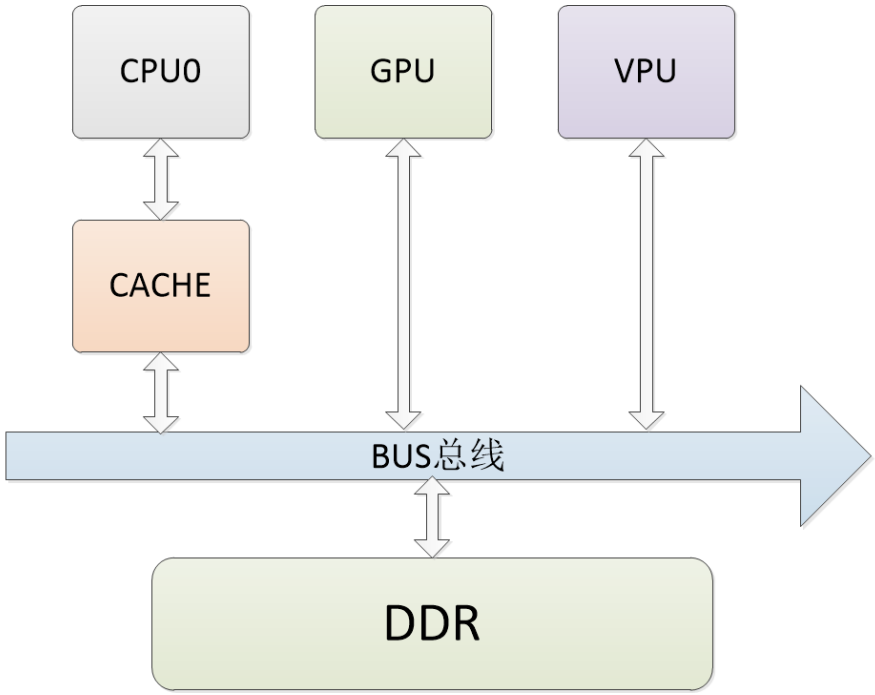
Например, если обрабатываемое изображение ARGB8888 хранится в режиме little-endian (A находится в старших разрядах), значение перечисления следует задать как RK_FMT_BGRA8888.

1.4.2 Согласованность DMA-памяти и Cache

При отладке приложений и реализации функций CPU, GPU, RKVDEC и другие аппаратные модули часто читают и записывают одни и те же данные DMA-памяти. В таком случае обычно возникает проблема согласованности cache (для DMA-памяти non-cache эта проблема отсутствует, однако скорость чтения и записи такой памяти на практике ниже, поэтому при доступе CPU она обычно не используется). При чтении и записи DMA-памяти обращение к DDR выполняется через контроллер DDR. Для ускорения доступа некоторые данные часто кэшируются в cache, причём не по одному элементу, а целыми группами. Преимущество состоит в ускорении следующего доступа, но недостаток также очевиден: изменения данных cache не сразу отражаются в DDR и наоборот. Когда GPU, RGA или другой аппаратный IP изменяет данные DDR, CPU может ещё не знать об изменении и получить из cache старые данные, что приводит к ошибкам чтения и записи.

Взаимодействие CPU и аппаратных IP, таких как GPU и RGA, с данными DDR показано на рисунке.

- **Рисунок 1-4. Взаимодействие CPU и аппаратных IP, таких как GPU и RGA, с данными DDR**



Исходя из описанных выше особенностей cache, при чтении и записи DMA-памяти необходимо соблюдать следующие правила:

- **После записи данных CPU в DMA-память, если к этой DMA-памяти требуется доступ GPU, RGA или другого аппаратного IP, сначала необходимо вызвать**

  RK_MPI_SYS_MmzFlushCache для flush cache в направлении записи, то есть перенести данные Cache CPU в DDR.
- **Перед чтением данных CPU из DMA-памяти, если до этого GPU, RGA или другой аппаратный IP изменял эту DMA-память, сначала необходимо вызвать**

  RK_MPI_SYS_MmzFlushCache для flush cache в направлении чтения, то есть признать Cache недействительным и обеспечить согласованность с данными DDR.

2. Описание функций

2.1 Пул буферов памяти

Пул буферов памяти главным образом предоставляет службам мультимедиа функции управления памятью, отвечает за выделение и освобождение памяти и позволяет полноценно использовать пул, рационально распределяя ресурсы памяти между различными модулями обработки мультимедиа.

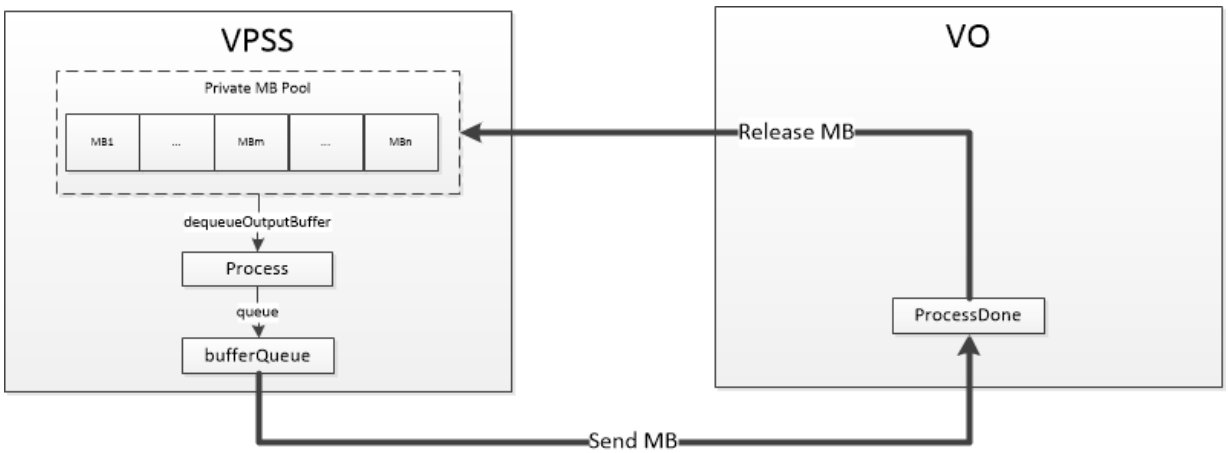
Пул буферов памяти состоит из набора блоков одинакового размера. В зависимости от задачи различаются количество запрашиваемых пулов, размер и количество блоков.

RK предоставляет четыре режима пула буферов памяти (подробности см. в MB_SOURCE_E): частный пул (MB_SOURCE_PRIVATE), общий пул (MB_SOURCE_COMMON), модульный пул (MB_SOURCE_MODULE) и пользовательский пул (MB_SOURCE_USER). Не все модули поддерживают все эти режимы. Подробности см. в описании каждого модуля; если режим в описании модуля не указан, по умолчанию считается, что модуль использует частный пул.

2.1.1 Частный пул

При использовании частного пула каждый канал модуля самостоятельно создаёт пул буферов памяти; размер и количество определяются внутри канала в соответствии с заданными и стандартными параметрами. Как показано на рисунке 2-1, канал VPSS по мере необходимости выделяет частный пул, получает из него блок памяти, выводит целевое изображение в этот блок и передаёт его VO для отображения. После завершения отображения VO освобождает блок обратно в частный пул.

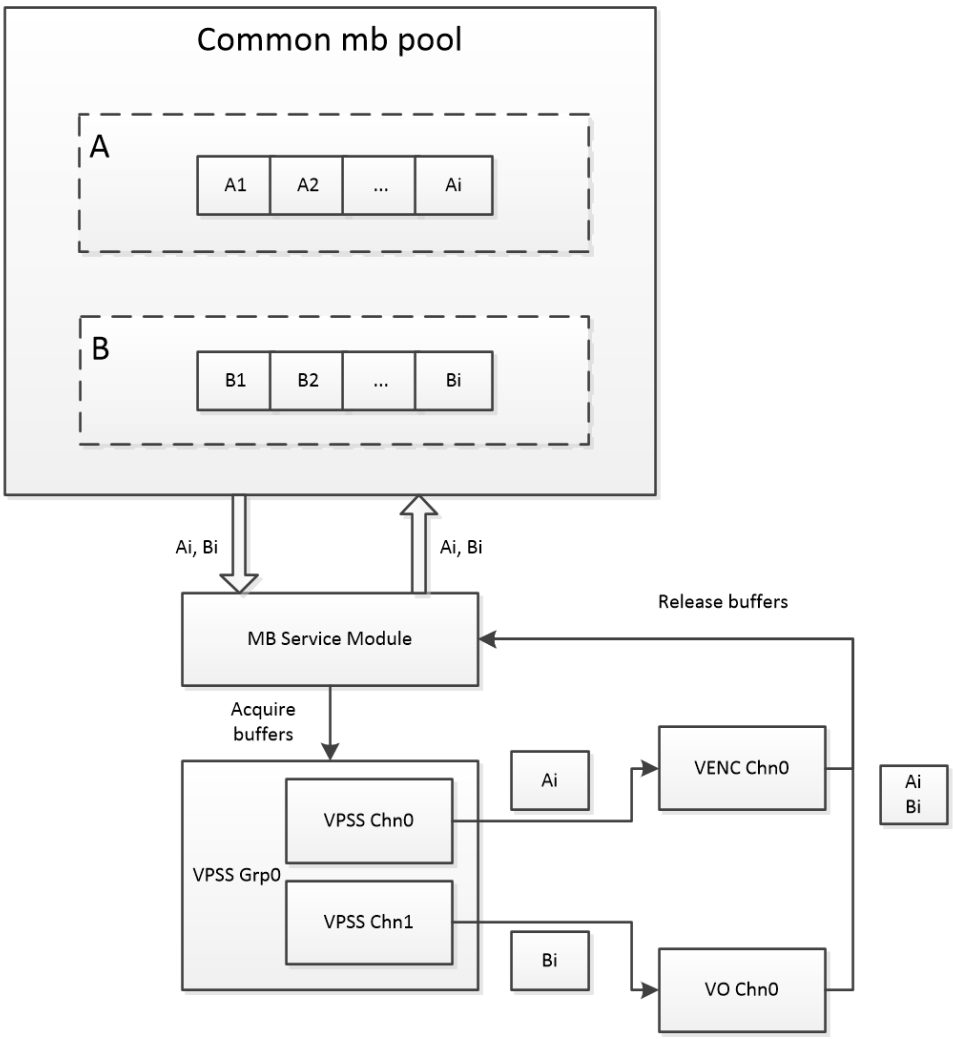
- **Рисунок 2-1. Типовая схема потока данных частного пула**



2.1.2 Общий пул

Все каналы видеовхода могут получать из общего видеопула блоки буфера для хранения захваченных изображений. Как показано на рисунке 2-2, предположим, что режим работы канала VPSS — USER: канал VPSS 0 получает из общего видеопула A блок Ai как буфер выходного изображения и передаёт его VENC, а канал VPSS 1 получает из общего видеопула B блок Bi как буфер выходного изображения и передаёт его VO. После кодирования VENC блок Ai освобождается обратно в общий видеопул, а после отображения VO блок Bi освобождается обратно в общий видеопул.

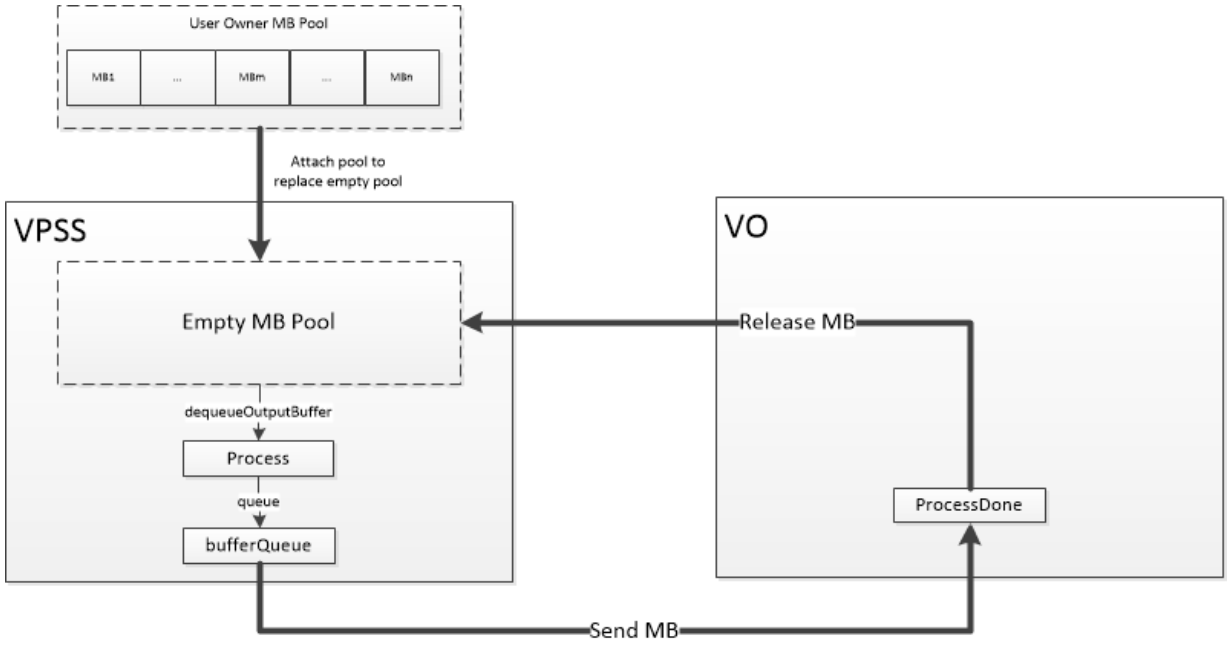
- **Рисунок 2-2. Типовая схема потока данных общего пула**



2.1.3 Пользовательский пул

Пользовательский пул буферов памяти внедряется пользователем как внешний пул через интерфейс attach pool соответствующего модуля.

- **Рисунок 2-3. Типовая схема потока данных пользовательского пула**



2.1.4 Правила вычисления размера отдельного MB в пуле памяти

Чтобы внедрённый пул памяти можно было использовать нормально, вычисление памяти пула должно соответствовать определённым правилам; непосредственно руководствуйтесь rk_mpi_cal.h.

rk_mpi_cal.h определяет интерфейсы вычисления размера памяти. Интерфейсы описаны ниже:

- Таблица 2-1. Способы вычисления размера буфера изображения и виртуальных ширины и высоты

Имя интерфейса	Назначение
RK_MPI_CAL_COMM_GetPicBufferSize	Вычисляет размер буфера универсального изображения (этот интерфейс можно использовать для модулей, для которых отдельный интерфейс вычисления размера не указан).
RK_MPI_CAL_TDE_GetPicBufferSize	Вычисляет размер буфера изображения, требуемого для вывода TDE.
RK_MPI_CAL_VGS_GetPicBufferSize	Вычисляет размер буфера изображения, требуемого для вывода VGS и VPSS.
RK_MPI_CAL_VDEC_GetPicBufferSize	Вычисляет размер буфера изображения, требуемого для вывода VDEC.
RK_MPI_CAL_VGS_GetPicVirWidth	Получает ширину выходного изображения VGS после выравнивания пикселей, в пикселях.
RK_MPI_CAL_VGS_GetPicVirHeight	Получает высоту выходного изображения VGS после выравнивания пикселей, в пикселях.
RK_MPI_CAL_VDEC_GetVirWidth	Получает ширину выходного изображения VDEC после выравнивания пикселей, в пикселях.
RK_MPI_CAL_VDEC_GetVirHeight	Получает высоту выходного изображения VDEC после выравнивания пикселей, в пикселях.
RK_MPI_CAL_COMM_GetHorStride	Преобразует виртуальную ширину (в пикселях) в Stride (в байтах).
RK_MPI_CAL_COMM_GetVirWidth	Преобразует Stride (в байтах) в виртуальную ширину (в пикселях).

2.2 Системное связывание

RK MPI предоставляет интерфейс системного связывания (RK_MPI_SYS_Bind), который устанавливает связь между источником и получателем данных путём привязки получателя к источнику (разрешается только привязка получателя данных к источнику). После связывания данные, созданные источником, автоматически отправляются получателю. В настоящее время RK MPI поддерживает следующие связи:

- Таблица 2-2. Связи, поддерживаемые RK MPI

Источник данных	Получатель данных
VI	VO
VI	VENC
VI	VPSS
VPSS	VO
VPSS	VENC
VPSS	VPSS
VDEC	VPSS
VDEC	VO
VDEC	VENC
WBC	VO
WBC	VENC
WBC	VPSS
VENC	VDEC
AI	AENC
AI	AO
ADEC	AO

2.3 Управление частотой кадров

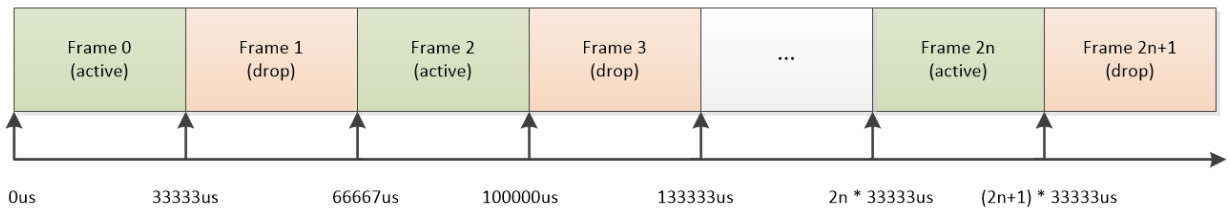
Атрибуты управления частотой кадров модуля включают входную и выходную частоты. Разрешение ввода кадров также контролируется в соответствии с входной частотой. Например, при входной частоте 30 кадров ввод одного кадра разрешается только каждые 33,33 мс.

Для удобства понимания обычно выделяются следующие режимы:

2.3.1 Режим уменьшения частоты кадров

Если входная частота больше или равна выходной, отдельные кадры равномерно отбрасываются в соответствии с заданными значениями. Например, при входной частоте 30 и выходной 15 кадров в секунду из каждых двух кадров отбрасывается один, как показано на рисунке:

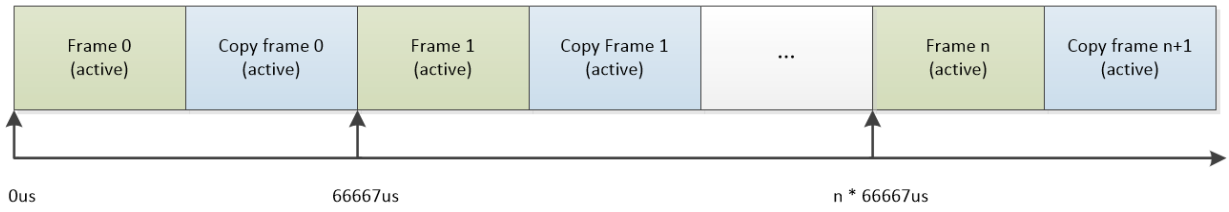
- Рисунок 2.4. Схема режима уменьшения частоты кадров



2.3.2 Режим увеличения частоты кадров

Если входная частота меньше выходной, одинаковые кадры равномерно вставляются в соответствии с заданными значениями. Например, при входной частоте 15 и выходной 30 кадров в секунду между каждыми двумя кадрами вставляется такой же кадр, как показано на рисунке:

- Рисунок 2.5. Схема режима увеличения частоты кадров



2.3.3 Перечень поддержки управления частотой кадров

- Поддержка RK356X/RK3588

Модуль	Режим увеличения	Режим уменьшения	Способ управления передачей кадров
VI	Поддерживается	Поддерживается	Блокирующий
VENC	Поддерживается	Поддерживается	Блокирующий
VPSS (группа)	Поддерживается	Поддерживается	Блокирующий
VPSS (канал)	Не поддерживается	Поддерживается	Неблокирующий
VO	Не поддерживается	Поддерживается	Неблокирующий

3. Справочник API

Управление системой реализует связывание и отмену связывания RK MPI, создание видеопулов, предоставление системных часов, управление памятью и другие функции.

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_SYS_Init: инициализирует систему RK MPI.
- RK_MPI_SYS_Exit: деинициализирует систему RK MPI.
- RK_MPI_SYS_Bind: связывает источник данных с получателем данных.
- RK_MPI_SYS_UnBind: отменяет связь источника данных с получателем данных.
- RK_MPI_SYS_MmzAlloc: выделяет память MMZ в пользовательском пространстве.
- RK_MPI_SYS_MmzAlloc_Cached: выделяет в пользовательском пространстве память MMZ с поддержкой cache.
- RK_MPI_SYS_MmzAllocEx: выделяет память MMZ в пользовательском пространстве с возможностью настройки флагов памяти, например наличия cache и физической непрерывности.
- RK_MPI_SYS_MmzFree: освобождает память MMZ в пользовательском пространстве.
- RK_MPI_SYS_MmzFlushCache: записывает содержимое cache в память и признаёт содержимое cache недействительным.
- RK_MPI_SYS_Malloc: выделяет память malloc.
- RK_MPI_SYS_Free: освобождает память, выделенную malloc.
- RK_MPI_SYS_CreateMB: создаёт блок буфера памяти.
- RK_MPI_SYS_GetCurPTS: получает текущую временную метку.
- RK_MPI_SYS_InitPTSBase: инициализирует временную метку RK MPI.
- RK_MPI_SYS_SyncPTS: синхронизирует временную метку RK MPI.
- RK_MPI_MB_CreatePool: создаёт пул буферов памяти.
- RK_MPI_MB_DestroyPool: уничтожает пул буферов памяти.
- RK_MPI_MB_GetMB: получает блок буфера.
- RK_MPI_MB_ReleaseMB: освобождает ранее полученный блок буфера.
- RK_MPI_MB_Handle2PhysAddr: получает физический адрес блока буфера.
- RK_MPI_MB_Handle2VirAddr: получает виртуальный адрес пользовательского пространства блока из пула буферов памяти.
- RK_MPI_MB_Handle2UniqueId: получает глобально уникальный системный ID блока буфера.
- RK_MPI_MB_Handle2Fd: получает в пользовательском пространстве дескриптор блока буфера.
- RK_MPI_MB_Handle2PoolId: получает ID пула, содержащего блок буфера.
- RK_MPI_MB_GetSize: получает размер памяти блока буфера.
- RK_MPI_MB_GetLength: получает фактический размер данных блока буфера.
- RK_MPI_MB_GetOffset: получает фактическое смещение памяти блока буфера.

- RK_MPI_MB_SetOffset: задаёт смещение памяти блока буфера.
- RK_MPI_MB_VirAddr2Handle: получает соответствующий блок буфера по виртуальному адресу.
- RK_MPI_MB_UniqueId2Fd: отображает глобально уникальный системный ID в fd.
- RK_MPI_MB_InquireUserCnt: запрашивает счётчик использования блока буфера.
- RK_MPI_MB_SetModPoolConfig: задаёт атрибуты общего видеопула модуля.
- RK_MPI_MB_GetModPoolConfig: получает атрибуты общего видеопула модуля.
- RK_MPI_MB_SetConfig: задаёт атрибуты общего видеопула.
- RK_MPI_MB_GetConfig: получает атрибуты общего видеопула.
- RK_MPI_MB_Init: инициализирует общий видеопул.
- RK_MPI_MB_Exit: деинициализирует общий видеопул.
- RK_MPI_MB_SetBufferStride: задаёт виртуальные ширину и высоту Stride изображения при использовании блока буфера для его хранения.

3.1 RK_MPI_SYS_Init

【Описание】

Инициализирует систему RK MPI.

【Синтаксис】

```
RK_S32 RK_MPI_SYS_Init(RK_VOID);
```

【Параметры】

Нет

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Перед запуском системы RK MPI необходимо вызвать этот интерфейс для инициализации.
- Не рекомендуется вызывать интерфейс несколько раз. Повторные вызовы всё равно возвращают успех; вызовы должны быть парными с RK_MPI_SYS_Exit.
- В настоящее время несколько процессов не могут совместно использовать ресурсы различных модулей; в нескольких процессах можно использовать только интерфейсы, относящиеся к модулю MB.

3.2 RK_MPI_SYS_Exit

【Описание】

Деинициализирует систему RK MPI.

【Синтаксис】

```
RK_S32 RK_MPI_SYS_Exit(RK_VOID);
```

【Параметры】

Нет

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Перед завершением работы системы RK MPI необходимо вызвать эту функцию.
- Повторные вызовы всё равно возвращают успех; вызовы должны быть парными с RK_MPI_SYS_Init, иначе возникает утечка.

3.3 RK_MPI_SYS_Bind

【Описание】

Интерфейс связывания источника данных с получателем данных.

【Синтаксис】

RK_S32 RK_MPI_SYS_Bind(const MPP_CHN_S *pstSrcChn, const MPP_CHN_S *pstDestChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstSrcChn	Указатель на исходный канал.	Ввод
pstDestChn	Указатель на целевой канал.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Поддерживаемые системой связи см. в разделе о системном связывании.
- Один получатель данных может быть связан только с одним источником.
- Связывание означает установление связи между источником и получателем данных. После связывания данные, созданные источником, автоматически отправляются получателю.
- Когда VI и VDEC выступают источниками данных, отправителем является канал, который передаёт данные другим модулям. Пользователь задаёт номер устройства равным 0; SDK не проверяет введённый номер устройства.
- Когда VPSS выступает получателем данных, получателем является устройство (GROUP), которое принимает данные от других модулей. Пользователь задаёт номер канала равным 0.
- Если модуль выступает отправителем данных и не поддерживает настройку глубины очереди изображений (DEPTH), после связывания невозможно получать изображения через интерфейс самого модуля. Для модулей, поддерживающих настройку глубины изображений, ручное получение изображений возможно только при глубине, отличной от 0.

3.4 RK_MPI_SYS_UnBind

【Описание】

Отменяет связь источника данных с получателем данных.

【Синтаксис】

RK_S32 RK_MPI_SYS_UnBind(const MPP_CHN_S *pstSrcChn, const MPP_CHN_S *pstDestChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstSrcChn	Указатель на исходный канал.	Ввод
pstDestChn	Указатель на целевой канал.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Если для pstDestChn не найден связанный исходный канал либо исходный канал найден, но не совпадает с pstSrcChn, возвращается ошибка.

3.5 RK_MPI_SYS_MmzAlloc

【Описание】

Выделяет память MMZ в пользовательском пространстве.

【Синтаксис】

RK_S32 RK_MPI_SYS_MmzAlloc(MB_BLK *pBlk, const RK_CHAR *pstrMmb, const RK_CHAR *pstrZone, RK_U32 u32Len);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pBlk	Указатель на блок буфера.	Вывод
pstrMmb	Указатель на строку с именем Mmb. Укажите RK_NULL.	Ввод
pstrZone	Указатель на строку с именем MMZ zone. Укажите RK_NULL.	Ввод
u32Len	Размер блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.6 RK_MPI_SYS_MmzAlloc_Cached

【Описание】

Выделяет в пользовательском пространстве память MMZ с поддержкой cache.

【Синтаксис】

RK_S32 RK_MPI_SYS_MmzAlloc_Cached(MB_BLK *pBlk, const RK_CHAR *pstrMmb, const RK_CHAR *pstrZone, RK_U32 u32Len);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pBlk	Указатель на блок буфера.	Вывод
pstrMmb	Указатель на строку с именем Mmb. Укажите RK_NULL.	Ввод
pstrZone	Указатель на строку с именем MMZ zone. Укажите RK_NULL.	Ввод
u32Len	Размер блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.7 RK_MPI_SYS_MmzAllocEx

【Описание】 Выделяет память MMZ в пользовательском пространстве с возможностью настройки флагов памяти, например наличия cache и физической непрерывности.

【Синтаксис】

RK_S32 RK_MPI_SYS_MmzAllocEx(MB_BLK *pBlk, const RK_CHAR *pstrMmb, const RK_CHAR *pstrZone, RK_U32 u32Len, RK_U32 u32HeapFlags);

【Параметры】

Имя параметра	Описание	Ввод / Тип выхода
pBlk	Указатель на блок буфера.	Вывод
pstrMmb	Указатель на строку с именем Mmb. Укажите RK_NULL.	Ввод
pstrZone	Указатель на строку с именем MMZ zone. Укажите RK_NULL.	Ввод
u32Len	Размер блока буфера.	Ввод
u32HeapFlags	Флаги памяти; одновременно можно указать несколько флагов. В настоящее время поддерживаются следующие флаги: MB_REMAP_MODE_NOCACHE: cache не поддерживается MB_REMAP_MODE_CACHED: cache поддерживается MB_DMA_TYPE_CMA: этот флаг означает запрос физически непрерывной памяти. Для физически непрерывной памяти ядро должно заранее зарезервировать специальную мультимедийную память; текущий SDK по умолчанию это не поддерживает.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.8 RK_MPI_SYS_MmzFree

【Описание】

Освобождает память MMZ в пользовательском пространстве.

【Синтаксис】

RK_S32 RK_MPI_SYS_MmzFree(MB_BLK blk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.9 RK_MPI_SYS_MmzFlushCache

【Описание】

Записывает содержимое cache в память и признаёт содержимое cache недействительным.

【Синтаксис】

RK_S32 RK_MPI_SYS_MmzFlushCache(MB_BLK blk, RK_BOOL bReadOnly);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод
bReadOnly	True: признаёт содержимое cache недействительным; эквивалентно DMA_BUF_SYNC_READ; False: записывает содержимое cache в память и признаёт cache недействительным; DMA_BUF_SYNC_RW.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.10 RK_MPI_SYS_Malloc

【Описание】

Выделяет память malloc.

【Синтаксис】

RK_S32 RK_MPI_SYS_Malloc(MB_BLK *pBlk, RK_U32 u32Len);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pBlk	Указатель на блок буфера.	Вывод
u32Len	Размер блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет.

3.11 RK_MPI_SYS_Free

【Описание】

Освобождает память, выделенную malloc.

【Синтаксис】

RK_S32 RK_MPI_SYS_Free(MB_BLK blk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.12 RK_MPI_SYS_CreateMB

【Описание】

Создаёт блок буфера памяти.

【Синтаксис】

RK_S32 RK_MPI_SYS_CreateMB(MB_BLK *pBlk, MB_EXT_CONFIG_S *pstMbExtConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pBlk	Указатель на блок буфера.	Вывод
pstMbExtConfig	Сведения о конфигурации параметров блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.13 RK_MPI_SYS_GetCurPTS

【Описание】

Получает текущую временную метку.

【Синтаксис】

RK_S32 RK_MPI_SYS_GetCurPTS(RK_U64 *pu64CurPTS);

Параметры

Имя параметра	Описание	Ввод/вывод
pu64CurPTS	Указатель на текущую временную метку. Единица: микросекунда.	Вывод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

Примечание

- Нет

3.14 RK_MPI_SYS_InitPTSBase

Описание

Инициализирует временную метку RK MPI.

Синтаксис

```
RK_S32 RK_MPI_SYS_InitPTSBase(RK_U64 u64PTSBase);
```

Параметры

Имя параметра	Описание	Ввод/вывод
u64PTSBase	База временной метки. Единица: микросекунда.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

Примечание

- Инициализация базы временной метки принудительно устанавливает текущую системную временную метку равной u64PTSBase независимо от исходной системной временной метки. Поэтому рекомендуется вызывать этот интерфейс до запуска мультимедийных служб (например, сразу после запуска операционной системы).

3.15 RK_MPI_SYS_SyncPTS

Описание

Синхронизирует временную метку RK MPI.

Синтаксис

```
RK_S32 RK_MPI_SYS_SyncPTS(RK_U64 u64PTSBase);
```

Параметры

Имя параметра	Описание	Ввод/вывод
u64PTSBase	База временной метки. Единица: микросекунда.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

Примечание

- Нет

3.16 RK_MPI_SYS_SetChnInputMode

Описание Задаёт режим входного (принимаемого) потока канала. Пользователь может по необходимости настроить режим входного (принимаемого) потока: сохранять последний кадр либо отбрасывать данные. Если пользователь включает канал модуля, но нижестоящий модуль не связан и данные канала не потребляются, нормальная ротация buffer невозможна. В этом случае режим входного потока канала можно установить в режим отбрасывания, чтобы обеспечить нормальную ротацию buffer канала.

Синтаксис

RK_S32 RK_MPI_SYS_SetChnInputMode(const MPP_CHN_S *pstChn, CHN_INPUT_MODE_E mode);

Параметры

Имя параметра	Описание	Ввод/вывод
pstChn	Указатель на канал.	Ввод
mode	Режим входного потока канала.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

Примечание

- Нет

3.17 RK_MPI_MB_CreatePool

Описание

Создаёт пул буферов памяти.

Синтаксис

MB_POOL RK_MPI_MB_CreatePool(MB_POOL_CONFIG_S *pstMbPoolCfg);

Параметры

Имя параметра	Описание	Ввод/вывод
pstMbPoolCfg	Указатель на атрибуты конфигурации пула буферов.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
Не MB_INVALID_POOLID	Допустимый ID пула.
MB_INVALID_POOLID	Не удалось создать пул; возможны недопустимые параметры или недостаток зарезервированной памяти.

【Примечание】

- Нет

3.18 RK_MPI_MB_DestroyPool

【Описание】

Уничтожает пул буферов памяти.

【Синтаксис】

RK_S32 RK_MPI_MB_DestroyPool(MB_POOL pool);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pool	ID пула буферов. Диапазон значений: [0, MB_MAX_POOLS).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- GROUP должна быть создана.

3.19 RK_MPI_MB_GetMB

【Описание】

Получает блок буфера.

【Синтаксис】

MB_BLK RK_MPI_MB_GetMB(MB_POOL pool, RK_U64 u64Size, RK_BOOL block = RK_FALSE);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pool	ID пула буферов. Диапазон значений: [0, MB_MAX_POOLS).	Ввод
u64Size	Размер блока буфера. Диапазон значений: весь диапазон типа данных, в byte.	Ввод
block	Ожидать ли возврата используемого блока буфера в пул. RK_TRUE: блокирующий режим. RK_FALSE: неблокирующий режим. По умолчанию неблокирующий.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Не MB_INVALID_HANDLE	Допустимый дескриптор блока буфера.
MB_INVALID_HANDLE	Не удалось получить блок буфера.

【Примечание】

- Нет

3.20 RK_MPI_MB_ReleaseMB

【Описание】

Освобождает ранее полученный блок буфера.

【Синтаксис】

RK_S32 RK_MPI_MB_ReleaseMB(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.21 RK_MPI_MB_Handle2PhysAddr

【Описание】

Получает физический адрес блока буфера.

【Синтаксис】

RK_U64 RK_MPI_MB_Handle2PhysAddr(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Ошибка
Не 0	Физический адрес

【Примечание】

- По умолчанию используется физически непрерывная память, получение физического адреса не поддерживается.
- Чтобы поддержать получение физического адреса, необходимо заранее настроить физически непрерывную память CMA; при создании MB/MB_POOL следует указать тип MB_DMA_TYPE_CMA. Подробности см. в FAQ по настройке.

3.22 RK_MPI_MB_Handle2VirAddr

【Описание】

Получает виртуальный адрес пользовательского пространства блока из пула буферов памяти.

【Синтаксис】

RK_VOID *RK_MPI_MB_Handle2VirAddr(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
RK_NULL	Не удалось получить виртуальный адрес.
Не RK_NULL	Допустимый виртуальный адрес.

【Примечание】

- Нет

3.23 RK_MPI_MB_Handle2UniqueId

【Описание】

Получает глобально уникальный системный ID блока буфера, который можно использовать для межпроцессных операций.

【Синтаксис】

RK_S32 RK_MPI_MB_Handle2UniqueId(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Отрицательное число	Не удалось получить глобально уникальный системный ID.
Неотрицательное число	Допустимый глобально уникальный системный ID.

【Примечание】

- Получение уникального ID поддерживается только для блоков буфера MMZ.
- Для MB_BLK, созданного через RK_MPI_SYS_CreateMB, невозможно получить глобально уникальный системный ID.
- Полученный уникальный ID можно передать другому процессу, преобразовать там в fd через RK_MPI_MB_UniqueId2Fd, затем выполнить map виртуального адреса и работать с ним. Подробности см. в следующем примере.

【Пример】

- Пример исходного процесса:

```
MB_BLK MbBlk = MB_INVALID_HANDLE;
RK_S32 s32Ret = RK_SUCCESS;
RK_S32 s32UniqueId = -1;
RK_S32 s32Size = 4 * 1024 * 1024;

s32Ret = RK_MPI_SYS_MmzAlloc_Cached(&MbBlk, RK_NULL, RK_NULL, s32Size);
if (s32Ret != RK_SUCCESS) {
    return s32Ret;
}
s32UniqueId = RK_MPI_MB_Handle2UniqueId(MbBlk);
if (s32UniqueId < 0) {
    return RK_FAILURE;
}
```

```
// send unique id cross process
s32Ret = send_info_to_dst_process(s32UniqueId, s32Size);
if (s32Ret != RK_SUCCESS) {
    return s32Ret;
}
```

- Пример целевого процесса:

```
RK_S32 s32UniqueId = -1;
RK_S32 s32DupFd = -1;
RK_VOID *pVirtAddr = RK_NULL;
RK_S32 s32Ret = RK_SUCCESS;

s32Ret = recv_info_by_src_process(&s32UniqueId, &s32Size);
if (s32Ret != RK_SUCCESS) {
    return s32Ret;
}

s32DupFd = RK_MPI_MB_UniqueId2Fd(s32UniqueId);
if (s32DupFd < 0) {
    return RK_FAILURE;
}

pVirtAddr = mmap64(NULL, s32Size,  PROT_READ | PROT_WRITE, MAP_SHARED, s32DupFd, 0);
if(pVirtAddr == RK_NULL) {
    return RK_FAILURE;
}
```

3.24 RK_MPI_MB_Handle2Fd

【Описание】

Получает в пользовательском пространстве дескриптор блока буфера.

【Синтаксис】

RK_S32 RK_MPI_MB_Handle2Fd(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Отрицательное число	Не удалось получить дескриптор блока буфера.
Неотрицательное число	Допустимый дескриптор блока буфера.

【Примечание】

- Допустимый дескриптор блока буфера можно получить только для памяти типа DMA.

3.25 RK_MPI_MB_Handle2PoolId

【Описание】

Получает ID пула, содержащего блок буфера.

【Синтаксис】

MB_POOL RK_MPI_MB_Handle2PoolId(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Не MB_INVALID_POOLID	Допустимый ID пула буферов.
MB_INVALID_POOLID	Недопустимый ID пула буферов.

【Примечание】

- Указанный блок должен быть допустимым блоком, полученным из пула буферов RK MPI, иначе получить ID пула невозможно.

3.26 RK_MPI_MB_GetSize

【Описание】

Получает размер памяти блока буфера.

【Синтаксис】

RK_U64 RK_MPI_MB_GetSize(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Любое значение	Размер памяти блока буфера.

【Примечание】

- Нет

3.27 RK_MPI_MB_GetLength

【Описание】

Получает фактический размер данных блока буфера.

【Синтаксис】

RK_U64 RK_MPI_MB_GetLength(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Любое значение	Фактический размер данных блока буфера.

【Примечание】

- Нет

3.28 RK_MPI_MB_GetOffset

【Описание】

Получает фактическое смещение памяти блока буфера.

【Синтаксис】

RK_U32 RK_MPI_MB_GetOffset(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Неотрицательное число	Фактическое смещение памяти блока буфера.

【Примечание】

- Нет

3.29 RK_MPI_MB_SetOffset

【Описание】

Задаёт смещение памяти блока буфера.

【Синтаксис】

RK_S32 RK_MPI_MB_SetOffset(MB_BLK mb, RK_U32 u32Offset);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод
u32Offset	Размер смещения памяти.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.30 RK_MPI_MB_VirAddr2Handle

【Описание】

Получает соответствующий блок буфера по виртуальному адресу.

【Синтаксис】

MB_BLK RK_MPI_MB_VirAddr2Handle(RK_VOID *pstVirAddr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstVirAddr	Виртуальный адрес.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
MB_INVALID_HANDLE	Не удалось получить ID блока буфера.
Не MB_INVALID_HANDLE	Допустимый ID блока буфера.

【Примечание】

- В настоящее время поиск блока буфера по виртуальному адресу поддерживается только для памяти DMA.

3.31 RK_MPI_MB_UniqueId2Fd

【Описание】

Получает глобально уникальный системный ID блока буфера.

【Синтаксис】

RK_S32 RK_MPI_MB_UniqueId2Fd(RK_S32 s32UniqueId);

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32UniqueId	Глобально уникальный системный ID.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Отрицательное значение	Не удалось получить дескриптор.
Неотрицательное значение	Допустимый дескриптор.

【Примечание】

- Нет

3.32 RK_MPI_MB_InquireUserCnt

【Описание】

Запрашивает счётчик использования блока буфера.

【Синтаксис】

RK_S32 RK_MPI_MB_InquireUserCnt(MB_BLK mb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Неотрицательное число	Счётчик использования блока буфера.
Отрицательное число	Недопустимый счётчик блока буфера.

【Примечание】

- Нет

3.33 RK_MPI_MB_SetModPoolConfig

【Описание】

Задаёт атрибуты общего видеопула модуля.

【Синтаксис】

RK_S32 RK_MPI_MB_SetModPoolConfig(MB_UID_E enMbUid, const MB_CONFIG_S *pstMbConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
enMbUid	ID модуля, использующего общий видеопул модуля.	Ввод
pstMbConfig	Указатель на атрибуты общего видеопула модуля.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Атрибут необходимо задать до вызова RK_MPI_SYS_Init.

3.34 RK_MPI_MB_GetModPoolConfig

【Описание】

Получает атрибуты общего видеопула модуля.

【Синтаксис】

RK_S32 RK_MPI_MB_GetModPoolConfig(MB_UID_E enMbUid, MB_CONFIG_S *pstMbConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
enMbUid	ID модуля, использующего общий видеопул модуля.	Ввод
pstMbConfig	Указатель на атрибуты общего видеопула модуля.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

3.35 RK_MPI_MB_SetConfig

【Описание】

Задаёт атрибуты общего видеопула.

【Синтаксис】

RK_S32 RK_MPI_MB_SetConfig(const MB_CONFIG_S *pstMbConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstMbConfig	Атрибуты настройки видеопула.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Атрибуты пула можно задавать только при неинициализированной системе, иначе возвращается ошибка.
- Размер каждого блока общего пула должен различаться в зависимости от текущего формата пикселей изображения и наличия сжатия. Конкретный размер выделения см. в описании правил вычисления размера отдельного MB в пуле памяти.

3.36 RK_MPI_MB_GetConfig

【Описание】

Получает атрибуты общего видеопула.

【Синтаксис】

RK_S32 RK_MPI_MB_GetConfig(MB_CONFIG_S *pstMbConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstMbConfig	Атрибуты настройки видеопула.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Перед получением атрибутов необходимо вызвать RK_MPI_MB_SetConfig и задать атрибуты общего видеопула, иначе возвращается **RK_ERR_MB_NOT_PERM**。

3.37 RK_MPI_MB_Init

【Описание】

Инициализирует общий видеопул.

【Синтаксис】

RK_S32 RK_MPI_MB_Init();

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод
u32HorStride	Виртуальная ширина изображения, в байтах.	Ввод
u32VerStride	Виртуальная высота изображения, в байтах.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Перед инициализацией пула необходимо вызвать RK_MPI_MB_SetConfig и настроить атрибуты пула, иначе возвращается RK_ERR_MB_NOT_PERM.
- Повторная инициализация разрешена и не возвращает ошибку.
- Если после успешной инициализации снова вызвать RK_MPI_MB_SetConfig, заданные параметры сохраняются, но не вступают в силу немедленно.

3.38 RK_MPI_MB_Exit

【Описание】

Деинициализирует общий видеопул.

【Синтаксис】

RK_S32 RK_MPI_MB_Exit();

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- После успешной инициализации пула необходимо вызвать RK_MPI_SYS_Exit для деинициализации системы MPP, а затем деинициализировать пул, иначе возвращается ошибка.
- Повторная деинициализация разрешена и не возвращает ошибку.
- Деинициализация не очищает ранее заданную конфигурацию пула.

3.39 RK_MPI_MB_SetBufferStride

【Описание】

Задаёт виртуальные ширину и высоту Stride изображения при использовании блока буфера для его хранения.

【Синтаксис】

RK_S32 RK_MPI_MB_SetBufferStride(MB_BLK mb, RK_U32 u32HorStride, RK_U32 u32VerStride);

【Параметры】

Имя параметра	Описание	Ввод/вывод
mb	ID блока буфера.	Ввод
u32HorStride	Виртуальная ширина изображения, в байтах.	Ввод
u32VerStride	Виртуальная высота изображения, в байтах.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок SYS.

【Примечание】

- Нет

4. Типы данных

Основные типы данных определены следующим образом:

4.1 Общие типы данных

```
typedef unsigned char           RK_UCHAR;
typedef uint8_t                 RK_U8;
typedef uint16_t                RK_U16;
typedef uint32_t                RK_U32;
typedef uint32_t                RK_UL;
typedef uintptr_t               RK_UINTPTR_T;

typedef char                    RK_CHAR;
typedef int8_t                  RK_S8;
typedef int16_t                 RK_S16;
typedef int32_t                 RK_S32;
typedef int32_t                 RK_SL;

typedef float                   RK_FLOAT;
typedef double                  RK_DOUBLE;

typedef uint64_t                RK_U64;
typedef int64_t                 RK_S64;

typedef uint32_t                RK_SIZE_T;
typedef uint32_t                RK_LENGTH_T;

typedef unsigned int            RK_HANDLE;
```

```
typedef enum {
    RK_FALSE = 0,
    RK_TRUE  = 1,
} RK_BOOL;

#ifndef NULL
    #define NULL                0L
#endif

#define RK_NULL                 0L
#define RK_SUCCESS              0
#define RK_FAILURE              (-1)

#define RK_VOID                 void
```

4.2 MOD_ID_E

【Описание】

Определяет тип перечисления ID модулей.

【Определение】

```
typedef enum rkMOD_ID_E {
    RK_ID_CMPI    = 0,
    RK_ID_MB      = 1,
    RK_ID_SYS     = 2,
    RK_ID_RGN     = 3,
    RK_ID_VENC    = 4,
    RK_ID_VDEC    = 5,
    RK_ID_VPSS    = 6,
    RK_ID_VGS     = 7,
    RK_ID_VI      = 8,
    RK_ID_VO      = 9,
    RK_ID_AI      = 10,
    RK_ID_AO      = 11,
    RK_ID_AENC    = 12,
    RK_ID_ADEC    = 13,
    RK_ID_TDE     = 14,
    RK_ID_ISP     = 15,

    RK_ID_BUTT,
} MOD_ID_E;
```

【Примечания】

Нет

4.3 MB_UID_E

【Описание】

Определяет тип перечисления ID модулей пула мультимедийной памяти.

【Определение】

```
typedef enum rkMB_UID_E {
    MB_UID_VI = 0,
    MB_UID_VO = 1,
    MB_UID_VGS = 2,
    MB_UID_VENC = 3,
    MB_UID_VDEC = 4,
    MB_UID_VPSS = 5,
    MB_UID_AI = 6,
    MB_UID_AENC = 7,
    MB_UID_ADEC = 8,
    MB_UID_AVS = 9,
    MB_UID_BUTT = 10
} MB_UID_E;
```

【Примечания】

Нет

4.4 RK_CODEC_ID_E

【Описание】

Определяет перечисление типов кодирования аудио и видео.

【Определение】

```
typedef enum rkCODEC_ID_E {
     RK_VIDEO_ID_Unused,             /**< Value when coding is N/A */
     RK_VIDEO_ID_AutoDetect,         /**< Autodetection of coding type */
     RK_VIDEO_ID_MPEG1VIDEO,
     RK_VIDEO_ID_MPEG2VIDEO,         /**< AKA: H.262 */
     RK_VIDEO_ID_H263,               /**< H.263 */
     RK_VIDEO_ID_MPEG4,              /**< MPEG-4 */
     RK_VIDEO_ID_WMV,                /**< Windows Media Video (WMV1,WMV2,WMV3)*/
     RK_VIDEO_ID_RV,                 /**< all versions of Real Video */
     RK_VIDEO_ID_AVC,                /**< H.264/AVC */
     RK_VIDEO_ID_MJPEG,              /**< Motion JPEG */
     RK_VIDEO_ID_VP8,                /**< VP8 */
     RK_VIDEO_ID_VP9,                /**< VP9 */
     RK_VIDEO_ID_HEVC,               /**< ITU H.265/HEVC */
     RK_VIDEO_ID_DolbyVision,        /**< Dolby Vision */
     RK_VIDEO_ID_ImageHEIC,          /**< HEIF image encoded with HEVC */
     RK_VIDEO_ID_VC1 = 0x01000000,   /**< Windows Media Video (WMV1,WMV2,WMV3)*/
     RK_VIDEO_ID_FLV1,               /**< Sorenson H.263 */
     RK_VIDEO_ID_DIVX3,              /**< DIVX3 */
     RK_VIDEO_ID_VP6,
     RK_VIDEO_ID_AVSPLUS,            /**< AVS+ profile=0x48 */
     RK_VIDEO_ID_AVS,                /**< AVS  profile=0x20 */
     /* *< Reserved region for introducing Khronos Standard Extensions */
     RK_VIDEO_ID_KhronosExtensions = 0x2F000000,
     /* *< Reserved region for introducing Vendor Extensions */
     RK_VIDEO_ID_VendorStartUnused = 0x3F000000,
     RK_VIDEO_ID_Max = 0x3FFFFFFF,

     RK_AUDIO_ID_Unused = 0x40000000,  /**< Placeholder value when coding is N/A  */
     RK_AUDIO_ID_AutoDetect,  /**< auto detection of audio format */
     RK_AUDIO_ID_PCM_ALAW,    /** <g711a> */
     RK_AUDIO_ID_PCM_MULAW,   /** <g711u> */
     RK_AUDIO_ID_PCM_S16LE,   /**< Any variant of PCM_S16LE coding */
     RK_AUDIO_ID_PCM_S24LE,   /**< Any variant of PCM_S24LE coding */
     RK_AUDIO_ID_PCM_S32LE,   /**< Any variant of PCM_S32LE coding */
     RK_AUDIO_ID_ADPCM_G722,         /**< Any variant of ADPCM_G722 encoded data */
     RK_AUDIO_ID_ADPCM_G726,         /**< Any variant of ADPCM_G726 encoded data */
     RK_AUDIO_ID_ADPCM_IMA_QT,       /**< Any variant of ADPCM_IMA encoded data */
     RK_AUDIO_ID_AMR_NB,      /**< Any variant of AMR_NB encoded data */
     RK_AUDIO_ID_AMR_WB,      /**< Any variant of AMR_WB encoded data */
```

```
    RK_AUDIO_ID_GSMFR,        /**< Any variant of GSM fullrate (i.e. GSM610) */
    RK_AUDIO_ID_GSMEFR,       /**< Any variant of GSM Enhanced Fullrate encoded data*/
    RK_AUDIO_ID_GSMHR,        /**< Any variant of GSM Halfrate encoded data */
    RK_AUDIO_ID_PDCFR,        /**< Any variant of PDC Fullrate encoded data */
    RK_AUDIO_ID_PDCEFR,       /**< Any variant of PDC Enhanced Fullrate encoded data */
    RK_AUDIO_ID_PDCHR,        /**< Any variant of PDC Halfrate encoded data */
    RK_AUDIO_ID_TDMAFR,       /**< Any variant of TDMA Fullrate encoded data (TIA/EIA-136-420) */
    RK_AUDIO_ID_TDMAEFR,      /**< Any variant of TDMA Enhanced Fullrate encoded data (TIA/EIA-
136-410) */
    RK_AUDIO_ID_QCELP8,       /**< Any variant of QCELP 8kbps encoded data */
    RK_AUDIO_ID_QCELP13,      /**< Any variant of QCELP 13kbps encoded data */
    RK_AUDIO_ID_EVRC,         /**< Any variant of EVRC encoded data */
    RK_AUDIO_ID_SMV,          /**< Any variant of SMV encoded data */
    RK_AUDIO_ID_G729,         /**< Any variant of G.729 encoded data */
    RK_AUDIO_ID_AAC,          /**< Any variant of AAC encoded data */
    RK_AUDIO_ID_MP3,          /**< Any variant of MP3 encoded data */
    RK_AUDIO_ID_SBC,          /**< Any variant of SBC encoded data */
    RK_AUDIO_ID_VORBIS,       /**< Any variant of VORBIS encoded data */
    RK_AUDIO_ID_WMA,          /**< Any variant of WMA encoded data */
    RK_AUDIO_ID_RA,           /**< Any variant of RA encoded data */
    RK_AUDIO_ID_MIDI,         /**< Any variant of MIDI encoded data */
    RK_AUDIO_ID_FLAC,         /**< Any variant of FLAC encoded data */
    RK_AUDIO_ID_APE = 0x50000000,
    /**< Reserved region for introducing Khronos Standard Extensions */
    RK_AUDIO_CodingKhronosExtensions = 0x6F000000,
    /**< Reserved region for introducing Vendor Extensions */
    RK_AUDIO_CodingVendorStartUnused = 0x7F000000,
    RK_AUDIO_ID_WMAV1,
    RK_AUDIO_ID_WMAV2,
    RK_AUDIO_ID_WMAPRO,
    RK_AUDIO_ID_WMALOSSLESS,
    RK_AUDIO_ID_MP1,
    RK_AUDIO_ID_MP2,
    /**< add audio bitstream Codec ID define for RT> */
    RK_AUDIO_ID_DTS,
    RK_AUDIO_ID_AC3,
    RK_AUDIO_ID_EAC3,
    RK_AUDIO_ID_DOLBY_TRUEHD,
    RK_AUDIO_ID_MLP,
    RK_AUDIO_ID_DTS_HD,
    RK_AUDIO_CodingMax = 0x7FFFFFFF,

    /* subtitle codecs */
    RK_SUB_ID_Unused = 0x80000000,    ///< A dummy ID pointing at the start of subtitle codecs.
    RK_SUB_ID_DVD,
    RK_SUB_ID_DVB,
    RK_SUB_ID_TEXT,  ///< raw UTF-8 text
    RK_SUB_ID_XSUB,
    RK_SUB_ID_SSA,
    RK_SUB_ID_MOV_TEXT,
    RK_SUB_ID_HDMV_PGS,
    RK_SUB_ID_DVB_TELETEXT,
    RK_SUB_ID_SRT,

    RK_SUB_ID_MICRODVD = 0x80000800,
    RK_SUB_ID_EIA_608,
    RK_SUB_ID_JACOSUB,
    RK_SUB_ID_SAMI,
    RK_SUB_ID_REALTEXT,
    RK_SUB_ID_STL,
    RK_SUB_ID_SUBVIEWER1,
    RK_SUB_ID_SUBVIEWER,
    RK_SUB_ID_SUBRIP,
    RK_SUB_ID_WEBVTT,
    RK_SUB_ID_MPL2,
    RK_SUB_ID_VPLAYER,
    RK_SUB_ID_PJS,
```

```
    RK_SUB_ID_ASS,
    RK_SUB_ID_HDMV_TEXT,
    RK_SUB_CodingMax
} RK_CODEC_ID_E;
```

【Примечания】

Нет

4.5 ROTATION_E

【Описание】

Определяет перечисление углов поворота.

【Определение】

```
typedef enum rkROTATION_E {
    ROTATION_0        = 0,
    ROTATION_90       = 1,
    ROTATION_180      = 2,
    ROTATION_270      = 3,
    ROTATION_BUTT
} ROTATION_E;
```

【Примечания】

Нет

4.6 POINT_S

【Описание】

Определяет структуру координат.

【Определение】

```
typedef struct rkPOINT_S {
    RK_S32 s32X;
    RK_S32 s32Y;
} POINT_S;
```

【Примечания】

Нет

4.7 RECT_S

【Описание】

Определяет структуру сведений о прямоугольной области.

【Определение】

```
typedef struct rkRECT_S {
    RK_S32 s32X;
    RK_S32 s32Y;
    RK_U32 u32Width;
    RK_U32 u32Height;
} RECT_S;
```

【Примечания】

Нет

4.8 CROP_INFO_S

【Описание】

Определяет структуру атрибутов CROP.

【Определение】

```
typedef struct rkCROP_INFO_S {
    RK_BOOL bEnable;
    RECT_S  stRect;
} CROP_INFO_S;
```

【Примечания】

Нет

4.9 ROTATION_EX_S

【Описание】

Определяет атрибуты поворота на произвольный угол.

【Определение】

```
typedef struct rkROTATION_EX_S {
    /* RW;Range: [0, 2];Rotation mode*/
    ROTATION_VIEW_TYPE_E enViewType;
    /* RW;Range: [0,360];Rotation Angle:[0,360]*/
    RK_U32               u32Angle;
    /*
     * RW;Range: [-511, 511];Horizontal offset of the image
     * distortion center relative to image center.
     */
    RK_S32               s32CenterXOffset;
    /*
     * RW;Range: [-511, 511];Vertical offset of the image
     * distortion center relative to image center.
     */
    RK_S32               s32CenterYOffset;
    /* RW;Dest size of any angle rotation*/
    SIZE_S               stDestSize;
} ROTATION_EX_S;
```

【Примечания】

Нет

4.10 VIDEO_PROC_DEV_TYPE_E

【Описание】

Определяет перечисление типов аппаратных устройств.

【Определение】

```
typedef enum rkVIDEO_PROC_DEV_TYPE_E {
    VIDEO_PROC_DEV_GPU = 0x0,       /* GPU device */
    VIDEO_PROC_DEV_RGA = 0x1,       /* RGA device */

    VIDEO_PROC_DEV_BUTT
} VIDEO_PROC_DEV_TYPE_E;
```

【Члены】

Имя члена	Описание
VIDEO_PROC_DEV_GPU	GPU
VIDEO_PROC_DEV_RGA	RGA

Примечания

- Модуль VPSS поддерживает настройку GPU/RGA в качестве аппаратного устройства, реализующего обработку видеоизображений. Подробности см. в разделе «Подсистема обработки видео» и описаниях соответствующих интерфейсов RK_MPI_VPSS_SetVProcDev и RK_MPI_VPSS_GetVProcDev.

4.11 MB_SOURCE_E

Описание

Определяет выбор источника MB.

Определение

```
typedef enum rkMB_SOURCE_E {
    MB_SOURCE_COMMON  = 0,
    MB_SOURCE_MODULE  = 1,
    MB_SOURCE_PRIVATE = 2,
    MB_SOURCE_USER    = 3,
    MB_SOURCE_BUTT
} MB_SOURCE_E;
```

Члены

Имя члена	Описание
MB_SOURCE_COMMON	Общий пул.
MB_SOURCE_MODULE	Общий пул модуля.
MB_SOURCE_USER	Пользовательский пул.
MB_SOURCE_PRIVATE	Частный пул.

Примечания

- Подробное описание см. в разделе о пуле буферов памяти.

4.12 MPP_CHN_S

Описание

Определяет структуру канала устройства модуля.

Определение

```
typedef struct rkMPP_CHN_S {
    MOD_ID_E    enModId;
    RK_S32      s32DevId;
    RK_S32      s32ChnId;
} MPP_CHN_S;
```

Члены

Имя члена	Описание
enModId	Номер модуля.
s32DevId	Номер устройства.
s32ChnId	Номер канала.

Примечания

Нет

4.13 CHN_INPUT_MODE_E

Описание

Определяет режим входного потока канала.

Определение

```
typedef enum rkCHN_INPUT_MODE_E {
    CHN_INPUT_MODE_NORMAL,          /* CHN receive all packet */
    CHN_INPUT_MODE_REMAIN_NEWEST,   /* CHN remain newest packet */
    CHN_INPUT_MODE_DROP_ALWAYS,     /* CHN drop all packet */
    CHN_INPUT_MODE_BUTT
} CHN_INPUT_MODE_E;
```

Примечания

Нет

4.14 MB_POOL

Описание

Определяет дескриптор пула памяти MB.

Определение

```
typedef RK_U32 MB_POOL;
```

Примечания

Нет

4.15 MB_BLK

Описание

Определяет дескриптор памяти MB.

Определение

```
typedef void * MB_BLK;
```

Примечания

Нет

4.16 MB_MAX_POOLS

Описание

Определяет максимальное количество пулов памяти MB.

Определение

```
#define MB_MAX_POOLS            512
```

【Примечания】

Нет

4.17 MB_POOL_CONFIG_S

【Описание】

Определяет структуру атрибутов пула буферов памяти.

【Определение】

```
typedef struct rkMB_POOL_CONFIG_S {
    RK_U64 u64MBSize;
    RK_U32 u32MBCnt;
    MB_REMAP_MODE_E enRemapMode;
    MB_ALLOC_TYPE_E enAllocType;
    MB_DMA_TYPE_E enDmaType;
    RK_BOOL bPreAlloc;
} MB_POOL_CONFIG_S;
```

【Члены】

Имя члена	Описание
u64MBSize	Размер блока буфера, в Byte.
u32MBCnt	Количество блоков в каждом пуле. Диапазон значений: (0, 10240]
enRemapMode	Режим отображения виртуального адреса пространства ядра.
enAllocType	Тип выделяемой памяти.
enDmaType	Тип выделяемой физической DMA-памяти, например настройка физической непрерывности.
bPreAlloc	Выделять ли блоки буфера при создании пула.

【Примечания】

Нет

4.18 MB_REMAP_MODE_E

【Описание】

Определяет режим отображения виртуального адреса пространства ядра MB.

【Определение】

```
typedef enum rkMB_REMAP_MODE_E {
    MB_REMAP_MODE_NONE = 0, /* no remap */
    MB_REMAP_MODE_NOCACHE = 1 << 8, /* no cache remap */
    MB_REMAP_MODE_CACHED = 1 << 9, /* cache remap, if you use this mode, you should flush cache
by yourself */
    MB_REMAP_MODE_BUTT
} MB_REMAP_MODE_E;
```

【Члены】

Имя члена	Описание
MB_REMAP_MODE_NOCACHE	Виртуальный адрес пространства ядра, отображённый MB с атрибутом nocache.
MB_REMAP_MODE_CACHED	Виртуальный адрес пространства ядра, отображённый MB с атрибутом cached.

【Примечания】

Нет

4.19 MB_DMA_TYPE_E

【Описание】

Определяет тип выделяемой физической памяти.

【Определение】

```
typedef enum rkMB_DMA_TYPE_E {
    MB_DMA_TYPE_NONE = 0, /* Physically Non-Continuous memory default */
    MB_DMA_TYPE_CMA = 1 << 12, /* Physically Continuous memory */
    MB_DMA_TYPE_BUTT
} MB_DMA_TYPE_E;
```

【Члены】

Имя члена	Описание
MB_DMA_TYPE_NONE	Выделенная физическая память имеет физически непрерывные адреса.
MB_DMA_TYPE_CMA	Выделенная физическая память имеет непрерывные физические адреса.

【Примечания】

Нет

4.20 MB_ALLOC_TYPE_E

【Описание】

Определяет тип памяти, выделяемой MB.

【Определение】

```
typedef enum rkMB_ALLOC_TYPE {
    MB_ALLOC_TYPE_UNUSED = -1,
    MB_ALLOC_TYPE_DMA = 0,
    MB_ALLOC_TYPE_MALLOC,
    MB_ALLOC_TYPE_MAX,
} MB_ALLOC_TYPE_E;
```

【Члены】

Имя члена	Описание
MB_ALLOC_TYPE_DMA	Выделяет DMA-память, доступную пространству ядра.
MB_ALLOC_TYPE_MALLOC	Выделяет память malloc.

【Примечания】

Нет

4.21 MB_EXT_CONFIG_S

【Описание】

Определяет сведения о выделении внешней памяти MB.

【Определение】

```
typedef struct _rkMB_EXT_CONFIG_S {
    RK_U8              *pu8VirAddr;
    RK_U64              u64PhyAddr;
    RK_S32              s32Fd;
    RK_U64              u64Size;
    RK_MPI_MB_FREE_CB   pFreeCB;
    RK_VOID            *pOpaque;
} MB_EXT_CONFIG_S;
```

【Члены】

Имя члена	Описание
pu8VirAddr	Внешне выделенный виртуальный адрес.
u64PhyAddr	Внешне выделенный физический адрес.
s32Fd	Дескриптор внешне выделенной памяти.
u64Size	Размер внешне выделенной памяти.
pFreeCB	Функция обратного вызова для освобождения выделенной памяти.
pOpaque	Контекст, необходимый обратному вызову освобождения выделенной памяти.

【Примечания】

- Перед использованием этой структуры рекомендуется обнулить её с помощью memset.

4.22 MB_CONFIG_S

【Описание】

Определяет структуру атрибутов пула мультимедийной памяти.

【Определение】

```
typedef struct rkMB_CONFIG_S {
    RK_U32 u32MaxPoolCnt;
    MB_POOL_CONFIG_S astCommPool[MB_MAX_COMM_POOLS];
} MB_CONFIG_S;
```

【Члены】

Имя члена	Описание
u32MaxPoolCnt	Количество пулов буферов, которое может содержать вся система. Статический атрибут. Зарезервированный атрибут, в настоящее время не действует.
astCommPool	Структура атрибутов общего пула. Статический атрибут.

【Примечания】

- Если u64MBSize равен 0 или u32MBCnt равен 0, соответствующий пул буферов не создаётся.
- Рекомендуется сначала обнулить всю структуру с помощью memset, а затем присвоить необходимые значения, иначе случайные значения невозможно полностью отфильтровать внутри.

5. Коды ошибок SYS

5.1 Коды ошибок управления системой SYS

Коды ошибок API управления системой SYS приведены ниже:

Код ошибки	Макроопределение	Описание
0xA0028006	RK_ERR_SYS_NULL_PTR	Ошибка нулевого указателя
0xA0028010	RK_ERR_SYS_NOTREADY	Атрибуты управления системой не настроены
0xA0028009	RK_ERR_SYS_NOT_PERM	Операция не разрешена
0xA002800C	RK_ERR_SYS_NOMEM	Не удалось выделить память, например из-за недостатка системной памяти
0xA0028003	RK_ERR_SYS_ILLEGAL_PARAM	Недопустимая настройка параметра
0xA0028012	RK_ERR_SYS_BUSY	Система занята
0xA0028008	RK_ERR_SYS_NOT_SUPPORT	Функция не поддерживается

5.2 Коды ошибок пула буферов памяти SYS

Код ошибки	Макроопределение	Описание
0xA0018003	RK_ERR_MB_ILLEGAL_PARAM	Недопустимая настройка параметра
0xA0018005	RK_ERR_MB_UNEXIST	Пул буферов памяти не существует
0xA0018006	RK_ERR_MB_NULL_PTR	Нулевой указатель параметра
0xA0018009	RK_ERR_MB_NOT_PERM	Операция не разрешена
0xA001800C	RK_ERR_MB_NOMEM	Не удалось выделить память
0xA001800D	RK_ERR_MB_NOBUF	Не удалось выделить буфер
0xA0018010	RK_ERR_MB_NOTREADY	Атрибуты управления системой не настроены
0xA0018012	RK_ERR_MB_BUSY	Система занята
0xA0018013	RK_ERR_MB_SIZE_NOT_ENOUGH	Недостаточный размер блока MB
0xA0018040	RK_ERR_MB_2MPOOLS	Создано слишком много пулов буферов

Управление памятью

Предисловие

Обзор

Модуль MMZ реализует управление памятью и другие функции.

Версии продуктов

Название микросхемы	Версия ядра
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	ZDD	2021-05-27	Начальная версия

Содержание

Справочник API
Типы данных

1. Справочник API

Функциональный модуль MMZ предоставляет пользователю следующие API:

- RK_MPI_MMZ_Alloc: выделяет пользовательский буфер.
- RK_MPI_MMZ_Free: освобождает пользовательский буфер.
- RK_MPI_MMZ_Handle2PhysAddr: получает физический адрес пользовательского буфера.
- RK_MPI_MMZ_Handle2VirAddr: получает виртуальный адрес пользовательского буфера.
- RK_MPI_MMZ_Handle2Fd: получает fd пользовательского буфера.
- RK_MPI_MMZ_GetSize: получает размер пользовательского буфера.
- RK_MPI_MMZ_Fd2Handle: находит соответствующий пользовательский буфер по fd.
- RK_MPI_MMZ_VirAddr2Handle: находит соответствующий пользовательский буфер по виртуальному адресу.
- RK_MPI_MMZ_PhyAddr2Handle: находит соответствующий пользовательский буфер по физическому адресу.
- RK_MPI_MMZ_IsCacheable: проверяет, является ли пользовательский буфер cache-буфером.
- RK_MPI_MMZ_FlushCacheStart: записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается перед доступом CPU. Если offset и length равны 0, выполняется full sync, иначе partial sync.
- RK_MPI_MMZ_FlushCacheEnd: записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается после завершения доступа CPU. Если offset и length равны 0, выполняется full sync, иначе partial sync.
- RK_MPI_MMZ_FlushCacheVaddrStart: записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается перед доступом CPU. Задаются виртуальный адрес и длина синхронизируемой памяти; поддерживается только partial sync.
- RK_MPI_MMZ_FlushCacheVaddrEnd: записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается после завершения доступа CPU. Задаются виртуальный адрес и длина синхронизируемой памяти; поддерживается только partial sync.
- RK_MPI_MMZ_FlushCachePaddrStart: записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается перед доступом CPU. Задаются физический адрес и длина синхронизируемой памяти; поддерживается только partial sync.
- RK_MPI_MMZ_FlushCachePaddrEnd: записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается после завершения доступа CPU. Задаются физический адрес и длина синхронизируемой памяти; поддерживается только partial sync.

1.1 RK_MPI_MMZ_Alloc

【Описание】

Выделяет пользовательский буфер.

【Синтаксис】

RK_S32 RK_MPI_MMZ_Alloc(MB_BLK *pBlk, RK_U32 u32Length, RK_U32 u32Flags);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pBlk	Указатель на блок буфера.	Вывод
u32Length	Размер блока буфера.	Ввод
u32Flags	Флаги памяти. Одновременно или по отдельности можно указать флаг поддержки Cache и флаг физической непрерывности памяти. В настоящее время поддерживаются следующие флаги: RK_MMZ_ALLOC_CACHEABLE: Cache поддерживается RK_MMZ_ALLOC_UNCACHEABLE: Cache не поддерживается RK_MMZ_ALLOC_TYPE_IOMMU: этот флаг означает запрос физически непрерывной памяти RK_MMZ_ALLOC_TYPE_CMA: этот флаг означает запрос физически непрерывной памяти	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Отрицательное значение	Ошибка

【Примечание】

- Нет.

1.2 RK_MPI_MMZ_Free

【Описание】

Освобождает пользовательский буфер.

【Синтаксис】

RK_S32 RK_MPI_MMZ_Free(MB_BLK blk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Отрицательное значение	Ошибка

【Примечание】

- Нет.

1.3 RK_MPI_MMZ_Handle2PhysAddr

【Описание】

Получает физический адрес пользовательского буфера.

【Синтаксис】

RK_U64 RK_MPI_MMZ_Handle2PhysAddr(MB_BLK blk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Больше 0	Допустимый физический адрес
0	Ошибка

【Примечание】

- Нет.

1.4 RK_MPI_MMZ_Handle2VirAddr

【Описание】

Получает виртуальный адрес пользовательского буфера.

【Синтаксис】

RK_VOID *RK_MPI_MMZ_Handle2VirAddr(MB_BLK blk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Не RK_NULL	Допустимый виртуальный адрес
RK_NULL	Не удалось получить виртуальный адрес

【Примечание】

- Нет

1.5 RK_MPI_MMZ_Handle2Fd

【Описание】

Получает fd пользовательского буфера.

【Синтаксис】

RK_S32 RK_MPI_MMZ_Handle2Fd(MB_BLK blk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Неотрицательное значение	Допустимый fd
Отрицательное число	Ошибка

【Примечание】

- Нет

1.6 RK_MPI_MMZ_GetSize

【Описание】

Получает размер пользовательского буфера.

【Синтаксис】

RK_U64 RK_MPI_MMZ_GetSize(MB_BLK blk);

Параметры

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
Неотрицательное значение	Размер блока буфера
Отрицательное число	Ошибка

Примечание

- Нет

1.7 RK_MPI_MMZ_Fd2Handle

Описание

Находит соответствующий пользовательский буфер по fd.

Синтаксис

MB_BLK RK_MPI_MMZ_Fd2Handle(RK_S32 u32Fd);

Параметры

Имя параметра	Описание	Ввод/вывод
u32Fd	Значение fd.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
Не пусто	ID блока буфера
RK_NULL	Ошибка

Примечание

- Нет

1.8 RK_MPI_MMZ_VirAddr2Handle

Описание

Находит соответствующий пользовательский буфер по виртуальному адресу.

Синтаксис

MB_BLK RK_MPI_MMZ_VirAddr2Handle(RK_VOID *pVirAddr);

Параметры

Имя параметра	Описание	Ввод/вывод
pVirAddr	Виртуальный адрес.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
Не пусто	ID блока буфера
RK_NULL	Ошибка

1.9 RK_MPI_MMZ_PhyAddr2Handle

【Описание】

Находит соответствующий пользовательский буфер по физическому адресу.

【Синтаксис】

MB_BLK RK_MPI_MMZ_PhyAddr2Handle(RK_U64 u64phyAddr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
u64phyAddr	Физический адрес.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Не пусто	ID блока буфера
RK_NULL	Ошибка

1.10 RK_MPI_MMZ_IsCacheable

【Описание】

Проверяет, является ли пользовательский буфер cache-буфером.

【Синтаксис】

RK_S32 RK_MPI_MMZ_IsCacheable(MB_BLK blk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
1	Текущий буфер является cache-буфером
0	Текущий буфер не является cache-буфером
Отрицательное значение	Ошибка запроса

1.11 RK_MPI_MMZ_FlushCacheStart

【Описание】 Записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается перед доступом CPU. Если offset и length равны 0, выполняется full sync, иначе partial sync. Операция начинается start и завершается end; между ними CPU работает с данной памятью.

【Синтаксис】

RK_S32 RK_MPI_MMZ_FlushCacheStart(MB_BLK blk, RK_U32 u32Offset, RK_U32 u32Length, RK_U32 u32Flags);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод
u32Offset	Смещение синхронизируемой памяти.	Ввод
u32Length	Длина синхронизируемого cache.	Ввод
u32Flags	Флаг чтения/записи; значение flag должно совпадать в start и end: RK_MMZ_SYNC_READONLY: код между start и end только читает указанную память с помощью CPU RK_MMZ_SYNC_WRITEONLY: код между start и end только записывает указанную память с помощью CPU RK_MMZ_SYNC_RW: код между start и end читает и записывает указанную память с помощью CPU	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Отрицательное значение	Ошибка

1.12 RK_MPI_MMZ_FlushCacheEnd

【Описание】 Записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается после завершения доступа CPU. Если offset и length равны 0, выполняется full sync, иначе partial sync. Операция начинается start и завершается end; между ними CPU работает с данной памятью.

【Синтаксис】

RK_S32 RK_MPI_MMZ_FlushCacheEnd(MB_BLK blk, RK_U32 u32Offset, RK_U32 u32Length, RK_U32 u32Flags);

【Параметры】

Имя параметра	Описание	Ввод/вывод
blk	ID блока буфера.	Ввод
u32Offset	Смещение синхронизируемой памяти.	Ввод
u32Length	Длина синхронизируемой памяти.	Ввод
u32Flags	Флаг чтения/записи; значение flag должно совпадать в start и end: RK_MMZ_SYNC_READONLY: код между start и end только читает указанную память с помощью CPU RK_MMZ_SYNC_WRITEONLY: код между start и end только записывает указанную память с помощью CPU RK_MMZ_SYNC_RW: код между start и end читает и записывает указанную память с помощью CPU	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Отрицательное значение	Ошибка

1.13 RK_MPI_MMZ_FlushCacheVaddrStart

Описание Записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается перед доступом CPU. Задаются виртуальный адрес и длина синхронизируемой памяти; поддерживается только partial sync. Операция начинается start и завершается end; между ними CPU работает с данной памятью.

Синтаксис

RK_S32 RK_MPI_MMZ_FlushCacheVaddrStart(RK_VOID *pVirAddr, RK_U32 u32Length, RK_U32 u32Flags);

Параметры

Имя параметра	Описание	Ввод/вывод
pVirAddr	Виртуальный адрес синхронизируемой памяти.	Ввод
u32Length	Длина синхронизируемой памяти.	Ввод
u32Flags	Флаг чтения/записи; значение flag должно совпадать в start и end: RK_MMZ_SYNC_READONLY: код между start и end только читает указанную память с помощью CPU RK_MMZ_SYNC_WRITEONLY: код между start и end только записывает указанную память с помощью CPU RK_MMZ_SYNC_RW: код между start и end читает и записывает указанную память с помощью CPU	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно
Отрицательное значение	Ошибка

1.14 RK_MPI_MMZ_FlushCacheVaddrEnd

Описание Записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается после завершения доступа CPU. Задаются виртуальный адрес и длина синхронизируемой памяти; поддерживается только partial sync. Операция начинается start и завершается end; между ними CPU работает с данной памятью.

Синтаксис

RK_S32 RK_MPI_MMZ_FlushCacheVaddrEnd(RK_VOID *pVirAddr, RK_U32 u32Length, RK_U32 u32Flags);

Параметры

Имя параметра	Описание	Ввод/вывод
pVirAddr	Виртуальный адрес синхронизируемой памяти.	Ввод
u32Length	Длина синхронизируемой памяти.	Ввод
u32Flags	Флаг чтения/записи; значение flag должно совпадать в start и end: RK_MMZ_SYNC_READONLY: код между start и end только читает указанную память с помощью CPU RK_MMZ_SYNC_WRITEONLY: код между start и end только записывает указанную память с помощью CPU RK_MMZ_SYNC_RW: код между start и end читает и записывает указанную память с помощью CPU	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно
Отрицательное значение	Ошибка

1.15 RK_MPI_MMZ_FlushCachePaddrStart

Описание Записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается перед доступом CPU. Задаются физический адрес и длина синхронизируемой памяти; поддерживается только partial sync. Операция начинается start и завершается end; между ними CPU работает с данной памятью.

Синтаксис

RK_S32 RK_MPI_MMZ_FlushCachePaddrStart(RK_U64 u64phyAddr, RK_U32 u32Length, RK_U32 u32Flags);

Параметры

Имя параметра	Описание	Ввод/вывод
u64phyAddr	Физический адрес синхронизируемой памяти.	Ввод
u32Length	Длина синхронизируемой памяти.	Ввод
u32Flags	Флаг чтения/записи; значение flag должно совпадать в start и end: RK_MMZ_SYNC_READONLY: код между start и end только читает указанную память с помощью CPU RK_MMZ_SYNC_WRITEONLY: код между start и end только записывает указанную память с помощью CPU RK_MMZ_SYNC_RW: код между start и end читает и записывает указанную память с помощью CPU	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно
Отрицательное значение	Ошибка

1.16 RK_MPI_MMZ_FlushCachePaddrEnd

Описание Записывает содержимое cache в память и признаёт содержимое cache недействительным; вызывается после завершения доступа CPU. Задаются физический адрес и длина синхронизируемой памяти; поддерживается только partial sync. Операция начинается start и завершается end; между ними CPU работает с данной памятью.

Синтаксис

RK_S32 RK_MPI_MMZ_FlushCachePaddrEnd(RK_U64 u64phyAddr, RK_U32 u32Length, RK_U32 u32Flags);

Параметры

Имя параметра	Описание	Ввод/вывод
u64phyAddr	Физический адрес синхронизируемой памяти.	Ввод
u32Length	Длина синхронизируемой памяти.	Ввод
u32Flags	Флаг чтения/записи; значение flag должно совпадать в start и end: RK_MMZ_SYNC_READONLY: код между start и end только читает указанную память с помощью CPU RK_MMZ_SYNC_WRITEONLY: код между start и end только записывает указанную память с помощью CPU RK_MMZ_SYNC_RW: код между start и end читает и записывает указанную память с помощью CPU	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно
Отрицательное значение	Ошибка

Основные типы данных определены следующим образом:

2.1 RK_MMZ_ALLOC_TYPE_IOMMU

【Описание】

Выделяемая физическая память имеет физически непрерывные адреса.

【Определение】

```
#define RK_MMZ_ALLOC_TYPE_IOMMU     0x00000000
```

2.2 RK_MMZ_ALLOC_TYPE_CMA

【Описание】

Выделяемая физическая память имеет непрерывные физические адреса.

【Определение】

```
#define RK_MMZ_ALLOC_TYPE_CMA       0x00000001
```

2.3 RK_MMZ_ALLOC_CACHEABLE

【Описание】

Отображает виртуальный адрес пользовательского пространства с атрибутом cached; пользователь должен самостоятельно выполнять синхронизацию cache.

【Определение】

```
#define RK_MMZ_ALLOC_CACHEABLE      0x00000000
```

2.4 RK_MMZ_ALLOC_UNCACHEABLE

【Описание】

Отображает виртуальный адрес пользовательского пространства с атрибутом non-cached.

【Определение】

```
#define RK_MMZ_ALLOC_UNCACHEABLE    0x00000010
```

2.5 RK_MMZ_SYNC_READONLY

【Описание】

Операция чтения памяти.

【Определение】

```
#define RK_MMZ_SYNC_READONLY        0x00000000
```

2.6 RK_MMZ_SYNC_WRITEONLY

【Описание】

Операция записи памяти.

【Определение】

```
#define RK_MMZ_SYNC_WRITEONLY       0x00000001
```

2.7 RK_MMZ_SYNC_RW

【Описание】

Операции чтения и записи памяти.

【Определение】

```
#define RK_MMZ_SYNC_RW              0x00000002
```

Видеовход

Предисловие

Обзор

В настоящем документе в основном описаны API и типы данных VI.

Версии продуктов

Название микросхемы	Версия ядра
RV1109/RV1126	4.19
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	LXH	2021-03-30	Начальная версия
V0.2.0	LXH	2021-08-03	Добавлено описание функции замораживания видео
V0.3.0	LXH	2021-09-07	Добавлено описание функций
V0.3.1	LXH	2021-09-27	Изменён формат документа
V0.4.0	LXH	2021-10-26	Уточнены определения структур
V0.5.0	LXH	2021-12-03	Изменено описание depth
V0.6.0	LXH	2021-12-08	Добавлены сведения о RK3588
V0.7.0	MLC	2021-12-13	Изменено количество каналов RK356X/RK3588 и блок-схема обработки RK356X
V0.8.0	MLC	2021-12-17	Изменена блок-схема аппаратной обработки VI RK3588
V0.9.0	MLC	2021-12-24	Добавлены описания атрибутов параметров соответствующих интерфейсов
V1.0.0	MLC	2021-12-28	Добавлены описания атрибутов параметров соответствующих интерфейсов
V1.1.0	MLC	2022-02-28	Добавлены описания интерфейсов API и структур
V1.1.1	MLC	2022-03-08	Добавлены описания атрибутов параметров соответствующих интерфейсов
V1.2.0	MLC	2022-03-19	Уточнены коды ошибок VI
V1.3.0	LXH	2022-04-25	Добавлены описания проверки состояния канала и интерфейсов EDID
V1.4.0	LXH	2022-10-08	Добавлены описания настройки codec и интерфейсов получения stream
V1.5.0	LXH	2022-10-31	Добавлены описания интерфейсов обратного вызова событий

Содержание

Основные понятия
Важные понятия
Описание функций
Пример
Справочник API
Типы данных
Коды ошибок VI

1. Основные понятия

Модуль видеовхода (VI) принимает видеоданные через MIPI Rx (включая интерфейсы MIPI и LVDS), BT.1120, BT.656, BT.601, DC и другие интерфейсы. VI сохраняет полученные данные в указанной области памяти, реализуя сбор видеоданных.

2. Важные понятия

- 2.0.1 Устройство DEV

Устройство видеовхода поддерживает несколько типов временных последовательностей и отвечает за их разбор.

- 2.0.2 Pipeline PIPE

PIPE видеовхода связан с задней частью устройства и отвечает за дополнительную обработку разобранных устройством данных. (В настоящее время функция не реализована; используется сквозной режим, достаточно указать тот же id, что у dev.)

- 2.0.3 Канал CHANNEL

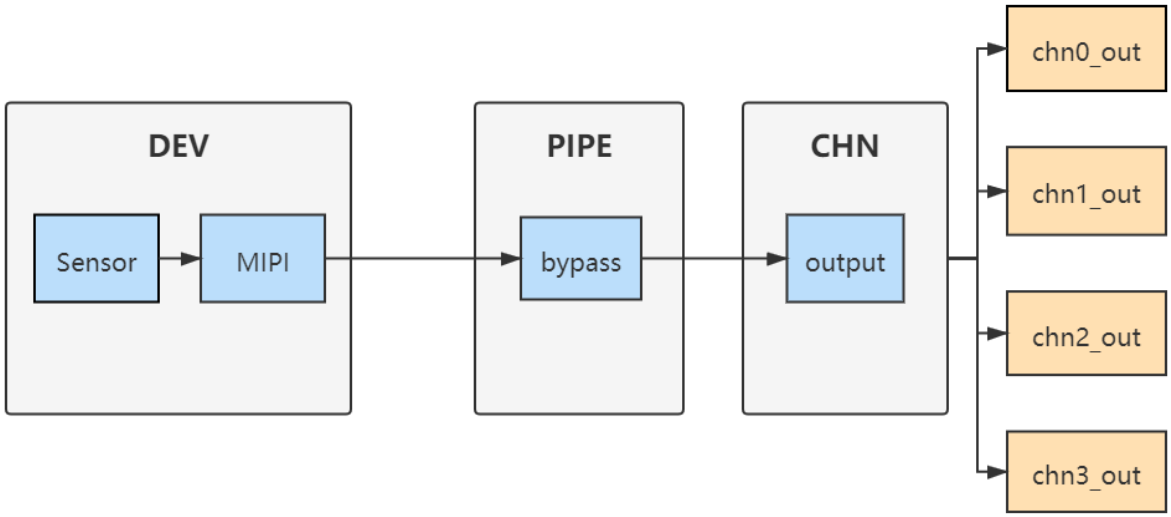
Последний уровень получения данных видеовхода; например, если dev является isp, имеется четыре выходных channel.

3. Описание функций

3.1 Функциональная блок-схема

На программном уровне VI разделён на три части, как показано на рисунке 1-1.

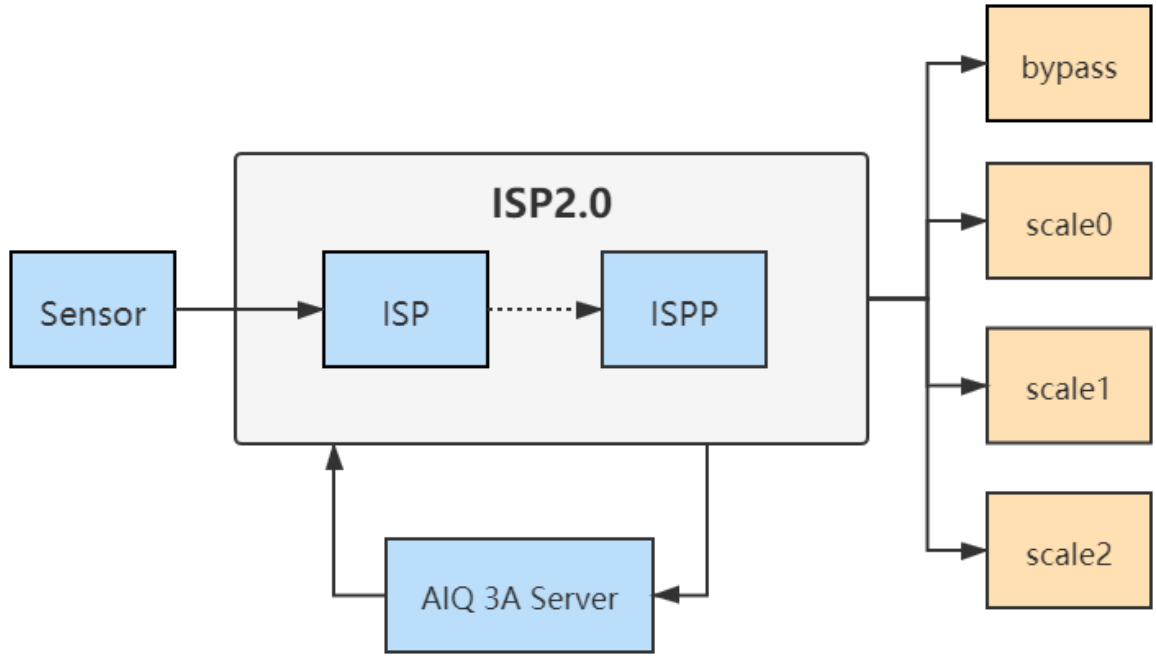
3.1.1 Рисунок 1-1. Блок-схема обработки VI



На программном уровне VI разделён на три уровня: входное устройство (dev), входной pipe (пока не реализован; используется сквозное соединение dev с chn) и выходной канал (channel). Количество устройств, PIPE и каналов различается для разных микросхем, как показано ниже.

Микросхема	dev	pipe	channel
RV1126	3	3	4
RK356X	3	3	2
RK3588	8	8	3

3.1.2 Рисунок 1-2. Блок-схема аппаратной обработки VI RV1126



Примечание

- Если при использовании VI значение aEntityName не задано (см. определение структуры VI_ISP_OPT_S), по умолчанию chn0_out соответствует bypass, chn1_out — scale0, chn2_out — scale1, chn3_out — scale2.

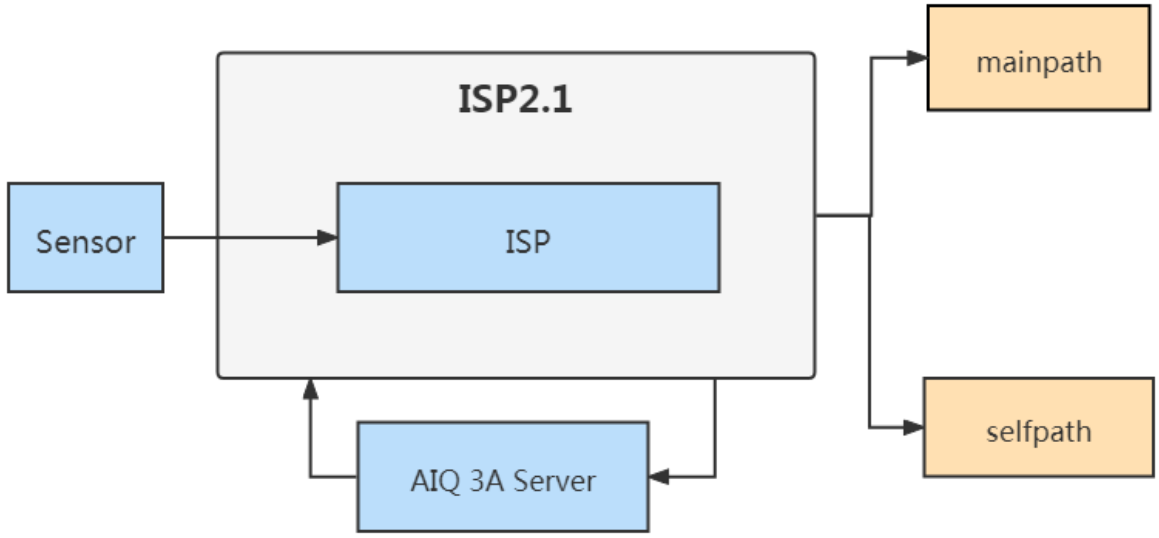
Ограничения выходного разрешения каждого канала приведены в таблице ниже:

aEntityName	name	max output	support output fmt
rkispp_bypass **или /dev/video18**	bypass	По умолчанию выводится максимальное разрешение isp, настройка разрешения и масштабирование не поддерживаются.	NV12/NV16/YUYV/UYVY/FBC0/FBC2
rkispp_scale0 **или /dev/video19**	scale0	Максимальное выходное разрешение совпадает с bypass. Масштабирование nv16: max width: 2080; nv12: max width: 3264; максимальный коэффициент масштабирования — 8.	NV12/NV16/YUYV/UYVY
rkispp_scale1 **или /dev/video20**	scale1	max width: 1280, максимальный коэффициент масштабирования — 8	NV12/NV16/YUYV/UYVY
rkispp_scale2 **или /dev/video21**	scale2	max width: 1280, максимальный коэффициент масштабирования — 8	NV12/NV16/YUYV/UYVY

Примечание

- Номер x узла /dev/videox, соответствующего aEntityName, не фиксирован; соответствующий узел можно определить с помощью media-ctl.
- С помощью команды v4l2-ctl можно проверить соответствующий узел и возможность нормального вывода разрешения; список поддержки VI совпадает со списком v4l2.
- Дополнительные сведения об isp см. в «Rockchip_Development_Guide_ISP2x_CN_v1.6.3.pdf» и «Rockchip_Instruction_Linux_Application_ISP20_CH.pdf».

3.1.3 Рисунок 1-3. Блок-схема аппаратной обработки VI RK356X



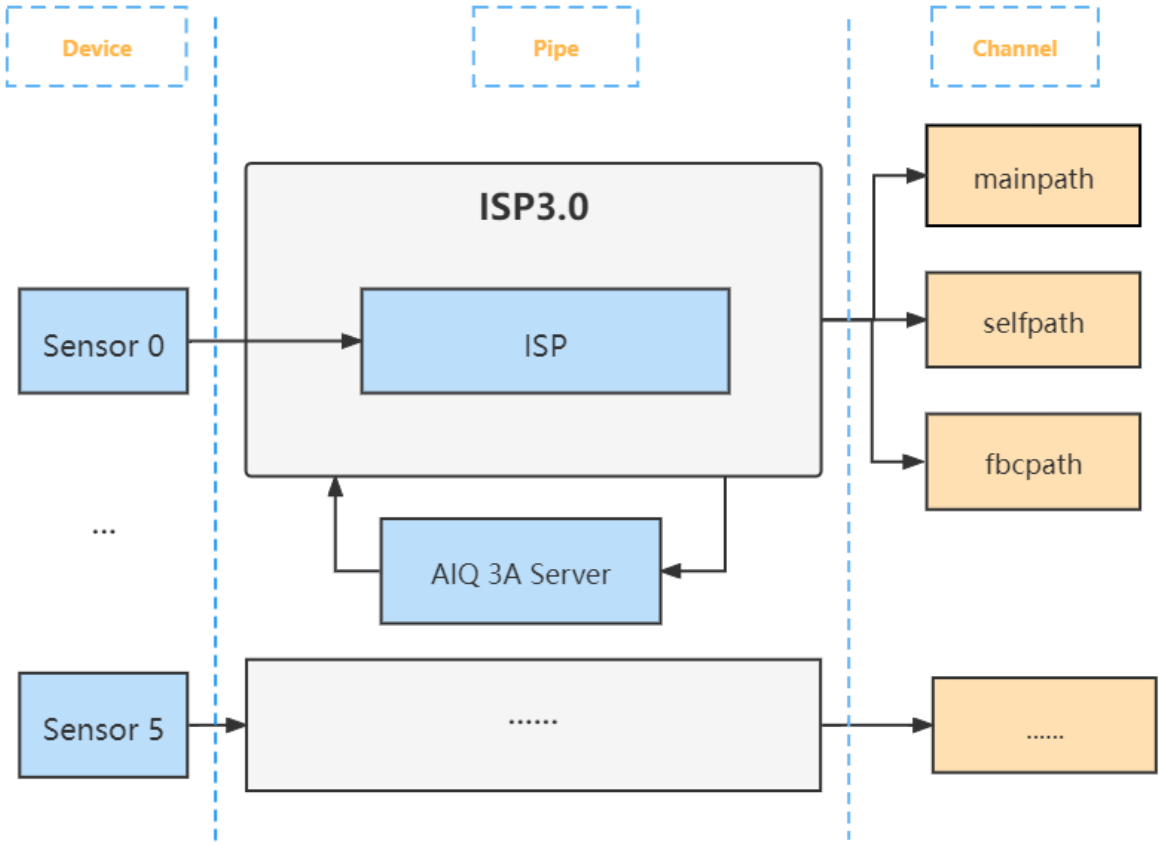
Примечание

- Если при использовании VI значение aEntityName не задано (см. определение структуры VI_ISP_OPT_S), по умолчанию chn0_out соответствует mainpath, chn1_out — selfpath。
- Номер x узла /dev/videox, соответствующего aEntityName, не фиксирован; соответствующий узел можно определить с помощью media-ctl.

Ограничения выходного разрешения каждого канала приведены в таблице ниже:

aEntityName	name	max output	support output fmt
/dev/video0	rkisp_mainpath	4096*2304, максимальный коэффициент масштабирования — 8	NV12/NV16/YUYV/UYVY
/dev/video1	rkisp_selfpath	1920*1080, максимальный коэффициент масштабирования — 8	NV12/NV16/YUYV/UYVY

3.1.4 Рисунок 1-4. Блок-схема аппаратной обработки VI RK3588



Примечание

- Если при использовании VI значение aEntityName не задано (см. определение структуры VI_ISP_OPT_S), по умолчанию chn0_out соответствует mainpath, chn1_out — selfpath, chn2_out соответствует fbcpath. При использовании нескольких isp рекомендуется использовать стандартные определения dev/pipe/channel.
- Номер x узла /dev/videox, соответствующего aEntityName, не фиксирован; соответствующий узел можно определить с помощью media-ctl.

Ограничения выходного разрешения каждого канала приведены в таблице ниже:

aEntityName	name	max output	support output fmt
/dev/video32	rkisp_mainpath	4672*3504, максимальный коэффициент масштабирования — 8	NV12/NV16/YUYV/UYVY
/dev/video33	rkisp_selfpath	1920*1080, максимальный коэффициент масштабирования — 8	NV12/NV16/YUYV/UYVY
/dev/video34	rkisp_fbcpath	Вывод fbc, максимальное разрешение sensor	AFBC

Примечание

- Указанные выше ограничения разрешения относятся к одному isp. RK3588 поддерживает комбинированный вывод двух isp; в этом случае максимальное разрешение rkisp_mainpath достигает 8064x6048.
- Номер device однозначно соответствует значению свойства camera-module-index подключения sensor в конфигурации dts.
- rkisp_fbcpath поддерживает только вывод в сжатом формате; другие форматы не поддерживаются.
- Дополнительные сведения об isp см. в разделе isp документации RK3588 datasheet.

4. Пример

```
TEST_VI_CTX_S *ctx;
ctx = reinterpret_cast<TEST_VI_CTX_S *>(malloc(sizeof(TEST_VI_CTX_S)));
memset(ctx, 0, sizeof(TEST_VI_CTX_S));

ctx->width = 1920;
ctx->height = 1080;
ctx->devId = 0;
ctx->pipeId = ctx->devId;
```

```
ctx->channelId = 1;
ctx->loopCountSet = 100;

//0. get dev config status
s32Ret = RK_MPI_VI_GetDevAttr(ctx->devId, &ctx->stDevAttr);
if (s32Ret == RK_ERR_VI_NOT_CONFIG) {
    //0-1.config dev
    s32Ret = RK_MPI_VI_SetDevAttr(ctx->devId, &ctx->stDevAttr);
    if (s32Ret != RK_SUCCESS) {
        RK_LOGE("RK_MPI_VI_SetDevAttr %x", s32Ret);
        goto __FAILED1;
    }
} else {
    RK_LOGE("RK_MPI_VI_SetDevAttr already");
}
//1.get dev enable status
s32Ret = RK_MPI_VI_GetDevIsEnable(ctx->devId);
if (s32Ret != RK_SUCCESS) {
    //1-2.enable dev
    s32Ret = RK_MPI_VI_EnableDev(ctx->devId);
    if (s32Ret != RK_SUCCESS) {
        RK_LOGE("RK_MPI_VI_EnableDev %x", s32Ret);
        goto __FAILED1;
    }
    //1-3.bind dev/pipe
    ctx->stBindPipe.u32Num = ctx->pipeId;
    ctx->stBindPipe.PipeId[0] = ctx->pipeId;
    s32Ret = RK_MPI_VI_SetDevBindPipe(ctx->devId, &ctx->stBindPipe);
    if (s32Ret != RK_SUCCESS) {
        RK_LOGE("RK_MPI_VI_SetDevBindPipe %x", s32Ret);
        goto __FAILED2;
    }
} else {
    RK_LOGE("RK_MPI_VI_EnableDev already");
}
//2.config channel
ctx->stChnAttr.stSize.u32Width = ctx->width;
ctx->stChnAttr.stSize.u32Height = ctx->height;
s32Ret = RK_MPI_VI_SetChnAttr(ctx->pipeId, ctx->channelId, &ctx->stChnAttr);
if (s32Ret != RK_SUCCESS) {
    RK_LOGE("RK_MPI_VI_SetChnAttr %x", s32Ret);
    goto __FAILED2;
}
//3.enable channel
s32Ret = RK_MPI_VI_EnableChn(ctx->pipeId, ctx->channelId);
if (s32Ret != RK_SUCCESS) {
    RK_LOGE("RK_MPI_VI_EnableChn %x", s32Ret);
    goto __FAILED2;
}
//4.save debug file
if (ctx->stDebugFile.bCfg) {
    s32Ret = RK_MPI_VI_ChnSaveFile(ctx->pipeId, ctx->channelId, &ctx->stDebugFile);
    RK_LOGE("RK_MPI_VI_ChnSaveFile %x", s32Ret);
}
while (loopCount < ctx->loopCountSet) {
//5.get the frame
    s32Ret = RK_MPI_VI_GetChnFrame(ctx->pipeId, ctx->channelId, &ctx->stViFrame, waitTime);
    if (s32Ret == RK_SUCCESS) {
        void *data = RK_MPI_MB_Handle2VirAddr(ctx->stViFrame.pMbBlk);
//6.get the channel status
        s32Ret = RK_MPI_VI_QueryChnStatus(ctx->pipeId, ctx->channelId, &ctx->stChnStatus);
//7.release the frame
        s32Ret = RK_MPI_VI_ReleaseChnFrame(ctx->pipeId, ctx->channelId, &ctx->stViFrame);
        if (s32Ret != RK_SUCCESS) {
            RK_LOGE("RK_MPI_VI_ReleaseChnFrame fail %x", s32Ret);
        }
        loopCount ++;
```

```
    } else {
        RK_LOGE("RK_MPI_VI_GetChnFrame timeout %x", s32Ret);
    }
    usleep(10*1000);
}

//8. disable one chn
s32Ret = RK_MPI_VI_DisableChn(ctx->pipeId, ctx->channelId);
RK_LOGE("RK_MPI_VI_DisableChn %x", s32Ret);
//9.disable dev(will diabled all chn)
__FAILED2:
s32Ret = RK_MPI_VI_DisableDev(ctx->devId);
RK_LOGE("RK_MPI_VI_DisableDev %x", s32Ret);
```

Подробный тестовый DEMO см. в поставляемом файле test_mpi_vi.cpp.

5. Справочник API

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_VI_SetDevAttr: задаёт атрибуты устройства VI.
- RK_MPI_VI_GetDevAttr: получает атрибуты устройства VI.
- RK_MPI_VI_EnableDev: включает устройство VI.
- RK_MPI_VI_DisableDev: отключает устройство VI.
- RK_MPI_VI_GetDevIsEnable: получает состояние включения устройства VI.
- RK_MPI_VI_SetDevBindPipe: связывает dev с pipe.
- RK_MPI_VI_GetDevBindPipe: получает атрибуты pipe, связанного с dev.
- RK_MPI_VI_SetChnAttr: задаёт атрибуты канала VI.
- RK_MPI_VI_GetChnAttr: получает атрибуты канала VI.
- RK_MPI_VI_EnableChn: включает канал VI.
- RK_MPI_VI_DisableChn: отключает канал VI.
- RK_MPI_VI_PauseChn: приостанавливает поток данных указанного канала VI.
- RK_MPI_VI_ResumeChn: возобновляет поток данных указанного канала VI.
- RK_MPI_VI_GetChnFrame: получает один кадр данных канала VI.
- RK_MPI_VI_ReleaseChnFrame: освобождает один кадр данных канала VI.
- RK_MPI_VI_ChnSaveFile: сохраняет данные канала VI.
- RK_MPI_VI_QueryChnStatus: получает состояние канала VI.
- RK_MPI_VI_GetChnFd: получает дескриптор файла устройства, соответствующего каналу VI.
- RK_MPI_VI_CloseChnFd: закрывает дескриптор файла устройства, соответствующего каналу VI.
- RK_MPI_VI_SetChnFreeze: задаёт замораживание видеовыхода канала VI.
- RK_MPI_VI_GetChnFreeze: получает состояние замораживания видеовыхода канала VI.
- RK_MPI_VI_SetUserPic: задаёт использование пользовательского изображения каналом VI.
- RK_MPI_VI_EnableUserPic: включает функцию использования пользовательского изображения.
- RK_MPI_VI_DisableUserPic: отключает функцию использования пользовательского изображения.
- RK_MPI_VI_GetChnConnectInfo: получает сведения о подключении канала VI.
- RK_MPI_VI_SetChnEdid: задаёт сведения EDID канала VI.
- RK_MPI_VI_GetChnEdid: получает сведения EDID канала VI.
- RK_MPI_VI_SetChnStreamCodec: задаёт stream codec канала VI.
- RK_MPI_VI_GetChnStreamCodec: получает stream codec канала VI.
- RK_MPI_VI_GetChnStream: получает stream канала VI.
- RK_MPI_VI_ReleaseChnStream: освобождает stream канала VI.
- RK_MPI_VI_RegChnEventCallBack: регистрирует интерфейс обратного вызова событий канала VI.

5.1 RK_MPI_VI_SetDevAttr

【Описание】

Задаёт параметры устройства VI.

【Синтаксис】

RK_S32 RK_MPI_VI_SetDevAttr(VI_DEV ViDev, const VI_DEV_ATTR_S *pstDevAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViDev	Номер устройства VI. Диапазон значений: [0, VI_MAX_DEV_NUM).	Ввод
pstDevAttr	Атрибуты устройства VI. Статический атрибут.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Перед настройкой атрибутов устройства в одном процессе необходимо проверить, были ли атрибуты VI уже заданы. В одном процессе атрибуты можно задать только один раз; для повторной настройки сначала необходимо отключить устройство, а затем задать атрибуты заново.

5.2 RK_MPI_VI_GetDevAttr

【Описание】

Получает атрибуты устройства VI.

【Синтаксис】

RK_S32 RK_MPI_VI_GetDevAttr(VI_DEV ViDev, VI_DEV_ATTR_S *pstDevAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViDev	Номер устройства VI. Диапазон значений: [0, VI_MAX_DEV_NUM).	Ввод
pstDevAttr	Атрибуты устройства VI.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Возвращаются атрибуты предыдущей конфигурации.
- Если атрибуты никогда не настраивались, возвращается ошибка «атрибуты не настроены».

5.3 RK_MPI_VI_EnableDev

【Описание】

Включает устройство VI.

【Синтаксис】

RK_S32 RK_MPI_VI_EnableDev(VI_DEV ViDev);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViDev	Номер устройства VI. Диапазон значений: [0, VI_MAX_DEV_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Перед включением необходимо настроить атрибуты устройства VI, иначе возвращается ошибка «атрибуты не настроены».
- Если устройство VI уже включено, повторное включение возвращает ошибку «используется».

5.4 RK_MPI_VI_DisableDev

【Описание】

Отключает устройство VI.

【Синтаксис】

RK_S32 RK_MPI_VI_DisableDev(VI_DEV ViDev);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViDev	Номер устройства VI. Диапазон значений: [0, VI_MAX_DEV_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Если устройство VI не включено, возвращается код ошибки VI «не включено».
- Отключение устройства VI отключает все каналы VI устройства. Чтобы закрыть только один канал, вызовите функцию закрытия канала.

5.5 RK_MPI_VI_SetDevBindPipe

【Описание】

Задаёт связь устройства VI с PIPE.

【Синтаксис】

RK_S32 RK_MPI_VI_SetDevBindPipe(VI_DEV ViDev, const VI_DEV_BIND_PIPE_S *pstDevBindPipe);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViDev	Номер устройства VI. Диапазон значений: [0, VI_MAX_DEV_NUM).	Ввод
pstDevBindPipe	Указатель на структуру сведений PIPE, связанного с Dev. Статический атрибут.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Перед связыванием PIPE необходимо включить устройство VI.
- Динамическое связывание не поддерживается.

5.6 RK_MPI_VI_GetDevBindPipe

【Описание】

Получает связь устройства VI с PIPE.

【Синтаксис】

RK_S32 RK_MPI_VI_GetDevBindPipe(VI_DEV ViDev, VI_DEV_BIND_PIPE_S *pstDevBindPipe);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViDev	Номер устройства VI. Диапазон значений: [0, VI_MAX_DEV_NUM).	Ввод
pstDevBindPipe	Указатель на структуру сведений PIPE, связанного с Dev.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Перед использованием этого интерфейса необходимо настроить атрибуты DEV, включить устройство и связать устройство с PIPE, иначе возвращается ошибка.

5.7 RK_MPI_VI_GetDevIsEnable

【Описание】

Получает состояние включения устройства.

【Синтаксис】

RK_S32 RK_MPI_VI_GetDevIsEnable(VI_DEV ViDev);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViDev	Номер устройства VI. Диапазон значений: [0, VI_MAX_DEV_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Включено.
Не 0	Не включено.

5.8 RK_MPI_VI_SetChnAttr

【Описание】

Задаёт атрибуты канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_SetChnAttr(VI_PIPE ViPipe, VI_CHN ViChn, const VI_CHN_ATTR_S *pstChnAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstChnAttr	Указатель на структуру атрибутов канала VI. Статический атрибут.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

Ограничения конфигурации атрибутов канала приведены ниже:

- Размер целевого изображения stSize:

  обязателен; размер должен находиться в поддерживаемом VI диапазоне и не превышать ограничение коэффициента масштабирования VI.

- Тип buff канала enBufType:

если тип собираемых данных — внешне выделенная память, задайте VI_ALLOC_BUF_TYPE_INTERNAL. Для внутренне выделенной памяти задайте

VI_ALLOC_BUF_TYPE_INTERNAL。

- Параметры сбора данных stIspOpt необходимо настроить, если собираются входные или сквозные данные isp:
  1. aEntityName: необходимо настроить, если тип данных — сквозной isp. Например: /dev/video0
  2. stMaxSize: обязательно; поддерживаемое разрешение bypass-тракта сбора данных isp.
- PIPE должен быть связан с устройством, иначе возвращается ошибка.

5.9 RK_MPI_VI_GetChnAttr

【Описание】

Получает атрибуты канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_GetChnAttr(VI_PIPE ViPipe, VI_CHN ViChn, VI_CHN_ATTR_S *pstChnAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Перед включением канала VI необходимо включить устройство VI, которому он принадлежит, иначе возвращается код ошибки VI «устройство не запущено».
- Перед получением атрибутов канала их необходимо задать.

5.10 RK_MPI_VI_EnableChn

【Описание】

Включает канал VI.

【Синтаксис】

RK_S32 RK_MPI_VI_EnableChn(VI_PIPE ViPipe, VI_CHN ViChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Перед включением канала VI необходимо включить устройство VI, которому он принадлежит, иначе возвращается код ошибки VI «устройство не запущено».

5.11 RK_MPI_VI_DisableChn

【Описание】

Отключает канал VI.

【Синтаксис】

RK_S32 RK_MPI_VI_DisableChn(VI_PIPE ViPipe, VI_CHN VIChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.

5.12 RK_MPI_VI_PauseChn

【Описание】

Приостанавливает поток данных указанного канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_PauseChn(VI_PIPE ViPipe, VI_CHN VIChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.

5.13 RK_MPI_VI_ResumeChn

【Описание】

Возобновляет поток данных указанного канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_ResumeChn(VI_PIPE ViPipe, VI_CHN VIChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.

5.14 RK_MPI_VI_GetChnFrame

【Описание】

Получает захваченное изображение из канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_GetChnFrame(VI_PIPE ViPipe, VI_CHN ViChn, VIDEO_FRAME_INFO_S *pstFrameInfo, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstFrameInfo	Указатель на структуру сведений о кадре выходного изображения.	Ввод
s32MilliSec	Время ожидания получения данных: -1 — блокирующий режим; 0 — неблокирующий режим; >0 — блокирование на s32MilliSec миллисекунд; при превышении времени возвращается ошибка.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.
- Значение s32MilliSec должно быть не меньше -1. При -1 данные передаются в блокирующем режиме, при 0 — в неблокирующем; при значении больше 0 после блокирования на s32MilliSec миллисекунд возвращается ошибка превышения времени ожидания.
- Полученные сведения буфера поступают из MediaBuffer, используемого внутри VI, поэтому после использования необходимо вызвать RK_MPI_VI_ReleaseChnFrame и освободить память.
- Если VI связан с другим устройством через RK_MPI_SYS_Bind, необходимо задать u32Depth структуры VI_CHN_ATTR_S больше 0, иначе этот интерфейс не получит данные.

5.15 RK_MPI_VI_ReleaseChnFrame

【Описание】

Освобождает один кадр изображения, полученный из канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_ReleaseChnFrame(VI_PIPE ViPipe, VI_CHN ViChn, const VIDEO_FRAME_INFO_S *pstFrameInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstFrameInfo	Указатель на структуру сведений о кадре выходного изображения.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.

- Этот интерфейс должен использоваться парно с RK_MPI_VI_GetChnFrame.

5.16 RK_MPI_VI_ChnSaveFile

【Описание】

Сохраняет данные канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_ChnSaveFile(VI_PIPE ViPipe, VI_CHN ViChn, VI_SAVE_FILE_INFO_S *pstSaveFileInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstSaveFileInfo Указатель на структуру сведений о сохранении данных канала VI.		

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.
- Если VI связан с другим устройством через RK_MPI_SYS_Bind, этот интерфейс не сможет сохранить данные.

5.17 RK_MPI_VI_QueryChnStatus

【Описание】

Запрашивает состояние канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_QueryChnStatus(VI_PIPE ViPipe, VI_CHN ViChn, VI_CHN_STATUS_S *pstChnStatus);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstChnStatus	Указатель на состояние канала VI.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.
- Если VI связан с другим устройством через RK_MPI_SYS_Bind, этот интерфейс не сможет получить статистику частоты кадров, количества потерянных кадров и т. д.

5.18 RK_MPI_VI_GetChnFd

【Описание】

Получает дескриптор файла устройства, соответствующего каналу VI. Возвращаемое значение можно использовать с select/poll для запроса состояния данных соответствующего канала.

【Синтаксис】

RK_S32 RK_MPI_VI_GetChnFd((VI_PIPE ViPipe, VI_CHN ViChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
Неотрицательное число	Успешно; дескриптор файла устройства канала VI.
Отрицательное число	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Для закрытия дескриптора файла необходимо вызвать RK_MPI_VI_CloseChnFd.
- Рекомендуется получать fd до включения канала.
- Не рекомендуется часто открывать и закрывать fd без остановки потока данных.

5.19 RK_MPI_VI_CloseChnFd

【Описание】

Закрывает дескриптор файла устройства, соответствующего каналу VI.

【Синтаксис】

RK_S32 RK_MPI_VI_CloseChnFd(VI_PIPE ViPipe, VI_CHN ViChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Нет

5.20 RK_MPI_VI_SetChnFreeze

【Описание】

Задаёт замораживание видеовыхода канала VI. Выходное изображение YUV модуля VI остаётся неизменным, а частота выходных кадров совпадает с частотой sensor.

【Синтаксис】

RK_S32 RK_MPI_VI_SetChnFreeze(VI_PIPE ViPipe, VI_CHN ViChn, RK_BOOL bFreeze);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
bFreeze	Включать ли функцию замораживания видеовыхода. RK_TRUE: включить RK_FALSE: не включать Динамический атрибут.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Нет

5.21 RK_MPI_VI_GetChnFreeze

【Описание】

Получает состояние включения замораживания видеовыхода канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_GetChnFreeze(VI_PIPE ViPipe, VI_CHN ViChn, RK_BOOL *pbFreeze);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pbFreeze	Включать ли функцию замораживания видеовыхода. RK_TRUE: включить RK_FALSE: не включать	Тип выхода

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

Примечание

- Нет

5.22 RK_MPI_VI_SetUserPic

Описание

Задаёт использование пользовательского изображения каналом VI.

Синтаксис

RK_S32 RK_MPI_VI_SetUserPic(VI_PIPE ViPipe, VI_CHN ViChn, const VI_USERPIC_ATTR_S*pstUsrPic);

Параметры

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstUsrPic	Указатель на структуру сведений о пользовательском изображении.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

Примечание

- Пользовательское изображение, заданное этим интерфейсом, действительно для всех PIPE.
- Интерфейс поддерживает два режима пользовательского изображения: изображение YUV и одноцветный фон.
- Задаёт пользовательское изображение VI, заменяя выходной видеокадр канала VI указанными пользователем данными изображения YUV или одноцветным фоном, а не видеоданными codec. Интерфейс необходимо использовать совместно с RK_MPI_VI_EnableUserPic и RK_MPI_VI_DisableUserPic.
- После настройки пользовательского изображения можно вызвать RK_MPI_VI_EnableUserPic и включить вставку пользовательского изображения для указанного VI PIPE. Тогда канал VI PIPE выводит настроенные данные изображения YUV или одноцветный фон. Обычно это используется для вывода изображения NoVideo каналом VI PIPE при потере видеосигнала.

  Ширина и высота настроенного пользовательского изображения обычно должны совпадать с размером VI PIPE. Если размеры не совпадают, VI автоматически масштабирует изображение до размера VI PIPE.
- В настоящее время частота пользовательского изображения фиксирована и равна 30 кадрам.

- При включённой вставке пользовательского изображения модуль VI непосредственно использует физический адрес из сведений о кадре настроенного изображения, поэтому после настройки пользовательского изображения его блок видеобуфера нельзя освобождать или уничтожать, пока не подтверждено прекращение использования. Сведения изображения можно изменить, повторно вызвав этот интерфейс с другим VideoBuffer.

- Сведения о кадре YUV, передаваемые в режиме изображения, должны представлять реальный допустимый VB.
- Для данных YUV в режиме изображения поддерживается формат пикселей RK_FMT_YUV420SP.
- enCompressMode данных YUV в режиме изображения должен быть несжатым: COMPRESS_MODE_NONE.
- Ширина и высота данных YUV в режиме изображения должны быть выровнены по 2.

5.23 RK_MPI_VI_EnableUserPic

【Описание】

Включает функцию использования пользовательского изображения.

【Синтаксис】

RK_S32 RK_MPI_VI_EnableUserPic((VI_PIPE ViPipe, VI_CHN ViChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Перед включением вставки пользовательского изображения необходимо задать сведения о его кадре.
- После включения вставки пользовательского изображения канал текущего VI PIPE выводит настроенные данные кадра пользовательского изображения.
- PIPE должен быть создан.
- Интерфейс можно вызывать повторно.

5.24 RK_MPI_VI_DisableUserPic

【Описание】

Отключает функцию использования пользовательского изображения.

【Синтаксис】

RK_S32 RK_MPI_VI_DisableUserPic((VI_PIPE ViPipe, VI_CHN ViChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Когда каналу VI больше не требуется выводить пользовательское изображение, следует вызвать этот интерфейс для восстановления вывода исходных видеоданных AD.
- Интерфейс можно вызывать повторно.
- PIPE должен быть создан.

5.25 RK_MPI_VI_GetChnConnectInfo

【Описание】

Получает сведения о подключении канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_GetChnConnectInfo((VI_PIPE ViPipe, VI_CHN ViChn, VI_CONNECT_INFO_S *pstConnectInfo)

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstConnectInfo	Указатель на структуру сведений о подключении канала VI.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- В настоящее время запрос состояния connect поддерживается только hdmi rx RK3588; во всех остальных случаях возвращается VI_CONNECT_STATE_UNKNOWN.
- После подключения или отключения hdmi rx можно повторно запросить входную частоту кадров.
- Нормальное получение поддерживается только после вызова RK_MPI_VI_SetChnAttr, иначе возвращается RK_ERR_VI_NOT_CONFIG.
- Интерфейс можно вызывать повторно.
- PIPE должен быть создан.

5.26 RK_MPI_VI_SetChnEdid

【Описание】

Задаёт сведения EDID канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_SetChnEdid((VI_PIPE ViPipe, VI_CHN ViChn, const VI_EDID_S *pstEdid)

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstEdid	Указатель на структуру сведений EDID канала VI.	Вывод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

Примечание

- Настройка сведений EDID поддерживается только hdmi rx.
- Нормальное получение поддерживается только после вызова RK_MPI_VI_SetChnAttr, иначе возвращается RK_ERR_VI_NOT_CONFIG.
- Интерфейс можно вызывать повторно.
- PIPE должен быть создан.

Пример

```
RK_U8 test_edid[2][128] = {
    {
        0x00, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x00, 0x49, 0x70, 0x88, 0x35, 0x01, 0x00, 0x00,
0x00,
        0x2d, 0x1f, 0x01, 0x03, 0x80, 0x78, 0x44, 0x78, 0x0a, 0xcf, 0x74, 0xa3, 0x57, 0x4c, 0xb0,
0x23,
        0x09, 0x48, 0x4c, 0x21, 0x08, 0x00, 0x61, 0x40, 0x01, 0x01, 0x81, 0x00, 0x95, 0x00, 0xa9,
0xc0,
        0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x08, 0xe8, 0x00, 0x30, 0xf2, 0x70, 0x5a, 0x80, 0xb0,
0x58,
        0x8a, 0x00, 0xc4, 0x8e, 0x21, 0x00, 0x00, 0x1e, 0x02, 0x3a, 0x80, 0x18, 0x71, 0x38, 0x2d,
0x40,
        0x58, 0x2c, 0x45, 0x00, 0xb9, 0xa8, 0x42, 0x00, 0x00, 0x1e, 0x00, 0x00, 0x00, 0xfc, 0x00,
0x52,
        0x4b, 0x2d, 0x55, 0x48, 0x44, 0x0a, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x00, 0x00, 0x00,
0xfd,
        0x00, 0x3b, 0x46, 0x1f, 0x8c, 0x3c, 0x00, 0x0a, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x01,
0xa3
    },
    {
        0x02, 0x03, 0x39, 0xd2, 0x51, 0x07, 0x16, 0x14, 0x05, 0x01, 0x03, 0x12, 0x13, 0x84, 0x22,
0x1f,
        0x90, 0x5d, 0x5e, 0x5f, 0x60, 0x61, 0x23, 0x09, 0x07, 0x07, 0x83, 0x01, 0x00, 0x00, 0x66,
0x03,
        0x0c, 0x00, 0x30, 0x00, 0x10, 0x67, 0xd8, 0x5d, 0xc4, 0x01, 0x78, 0xc8, 0x07, 0xe3, 0x05,
0x03,
        0x01, 0xe4, 0x0f, 0x00, 0xf0, 0x01, 0xe2, 0x00, 0xff, 0x08, 0xe8, 0x00, 0x30, 0xf2, 0x70,
0x5a,
        0x80, 0xb0, 0x58, 0x8a, 0x00, 0xc4, 0x8e, 0x21, 0x00, 0x00, 0x1e, 0x02, 0x3a, 0x80, 0x18,
0x71,
        0x38, 0x2d, 0x40, 0x58, 0x2c, 0x45, 0x00, 0xb9, 0xa8, 0x42, 0x00, 0x00, 0x9e, 0x00, 0x00,
0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0xd1
    }
};
VI_EDID_S stEdid;
```

```
memset(&stEdid, 0, sizeof(VI_EDID_S));
stEdid.u32Blocks = 2;
stEdid.pu8Edid = (RK_U8 *)test_edid;
s32Ret = RK_MPI_VI_SetChnEdid(ctx->pipeId, ctx->channelId, &stEdid);
```

5.27 RK_MPI_VI_GetChnEdid

【Описание】

Получает сведения EDID канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_GetChnEdid((VI_PIPE ViPipe, VI_CHN ViChn, VI_EDID_S *pstEdid)

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstEdid	Указатель на структуру сведений EDID канала VI.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- Запрос сведений EDID поддерживается только hdmi rx.
- Для pu8Edid в pstEdid необходимо выделить память в соответствии с запрашиваемым количеством u32Blocks; размер каждого block равен 128 байтам, общий размер памяти равен u32Blocks*128。
- Нормальное получение поддерживается только после вызова RK_MPI_VI_SetChnAttr, иначе возвращается RK_ERR_VI_NOT_CONFIG.
- Интерфейс можно вызывать повторно.
- PIPE должен быть создан.

【Пример】

```
VI_EDID_S stEdid;
memset(&stEdid, 0, sizeof(VI_EDID_S));
stEdid.u32Blocks = 3;
stEdid.pu8Edid = (RK_U8 *)calloc(128 * stEdid.u32Blocks, 1);
s32Ret = RK_MPI_VI_GetChnEdid(ctx->pipeId, ctx->channelId, &stEdid);
free(stEdid.pu8Edid);
```

5.28 RK_MPI_VI_SetChnStreamCodec

【Описание】

Задаёт stream codec канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_SetChnStreamCodec(VI_PIPE ViPipe, VI_CHN VIChn, const RK_CODEC_ID_E enCodecId);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
enCodecId	Формат потока stream codec. Диапазон значений: RK_VIDEO_ID_Unused: не задавать RK_VIDEO_ID_MJPEG: поток MJPEG RK_VIDEO_ID_AVC: поток H264 RK_VIDEO_ID_HEVC: поток H265	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.
- Необходимо задать после RK_MPI_VI_SetChnAttr и до RK_MPI_VI_EnableChn.
- Эта функция предназначена для задания формата закодированного потока, получаемого от устройства UVC.

5.29 RK_MPI_VI_GetChnStreamCodec

【Описание】

Возобновляет поток данных указанного канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_GetChnStreamCodec(VI_PIPE ViPipe, VI_CHN VIChn, RK_CODEC_ID_E *penCodecId);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
penCodecId	Формат потока stream codec.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.

5.30 RK_MPI_VI_GetChnStream

【Описание】

Получает захваченный закодированный поток из канала VI.

【Синтаксис】

RK_S32 RK_MPI_VI_GetChnStream(VI_PIPE ViPipe, VI_CHN ViChn, VI_STREAM_S *pstStream, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstStream	Указатель на структуру сведений о кадре выходного потока.	Вывод
s32MilliSec	Время ожидания получения данных: -1 — блокирующий режим; 0 — неблокирующий режим; >0 — блокирование на s32MilliSec миллисекунд; при превышении времени возвращается ошибка.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

【Примечание】

- PIPE должен быть связан с устройством, иначе возвращается ошибка.
- Перед вызовом этого интерфейса для получения данных потока через RK_MPI_VI_SetChnStreamCodec необходимо задать enCodecId одним из форматов RK_VIDEO_ID_MJPEG, RK_VIDEO_ID_AVC или RK_VIDEO_ID_HEVC. Не все настройки UVC поддерживают получение этих трёх форматов; поддерживаемые форматы и разрешения определяются атрибутами устройства UVC. Если задан формат, не поддерживаемый UVC, возвращаются данные стандартного формата UVC. Если задано разрешение, не поддерживаемое UVC, возвращаются данные ближайшего поддерживаемого разрешения. Команда v4l2-ctrl позволяет запросить форматы и разрешения соответствующего узла. Например, для /dev/video0: v4l2-ctl -d /dev/video0 --list-formats-

  ext。
- Значение s32MilliSec должно быть не меньше -1. При -1 данные передаются в блокирующем режиме, при 0 — в неблокирующем; при значении больше 0 после блокирования на s32MilliSec миллисекунд возвращается ошибка превышения времени ожидания.
- После использования полученного потока необходимо вызвать RK_MPI_VI_ReleaseChnStream и освободить его память.
- Если VI связан с другим устройством через RK_MPI_SYS_Bind, необходимо задать u32Depth структуры VI_CHN_ATTR_S больше 0, иначе этот интерфейс не получит данные.

5.31 RK_MPI_VI_ReleaseChnStream

【Описание】

Освобождает память закодированного потока, захваченного каналом VI.

【Синтаксис】

RK_S32 RK_MPI_VI_ReleaseChnStream(VI_PIPE ViPipe, VI_CHN ViChn, const VI_STREAM_S *pstStream);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstStream	Указатель на структуру сведений о кадре потока.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

Примечание

- PIPE должен быть связан с устройством, иначе возвращается ошибка.
- Этот интерфейс должен использоваться парно с RK_MPI_VI_GetChnStream.

5.32 RK_MPI_VI_RegChnEventCallback

Описание

Регистрирует интерфейс обратного вызова событий канала VI.

Синтаксис

RK_S32 RK_MPI_VI_RegChnEventCallback(VI_PIPE ViPipe, VI_CHN ViChn, const VI_EVENT_CALL_BACK_S *pstCallback);

Параметры

Имя параметра	Описание	Ввод/вывод
ViPipe	Номер PIPE VI. Диапазон значений: [0, VI_MAX_PIPE_NUM).	Ввод
ViChn	Номер канала VI. Диапазон значений: [0, VI_MAX_CHN_NUM).	Ввод
pstCallback	Указатель на структуру функции обратного вызова.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки VI.

Примечание

- PIPE должен быть связан с устройством, иначе возвращается ошибка.
- Интерфейс реализует обратные вызовы события подключения/отключения hdmirx (VI_CHANGE_EVENT_CONNECT_CHANGE) и события переключения входного источника (VI_CHANGE_EVENT_SOURCE_CHANGE). После получения обратного вызова пользователь определяет тип события по u32Event и вызывает RK_MPI_VI_GetChnConnectInfo, чтобы получить текущее состояние подключения, разрешение источника и сведения о формате.
- После получения события переключения входного источника необходимо выполнить сброс либо уничтожить и заново создать VI, иначе поток данных поступать не будет. Если разрешение и формат источника совпадают с ранее заданными, для восстановления потока достаточно вызвать RK_MPI_VI_PauseChn и RK_MPI_VI_ResumeChn. Если разрешение или формат источника отличаются от прежних, VI необходимо уничтожить и создать заново.
- Не рекомендуется выполнять длительные операции в функции обратного вызова; блокирующие операции запрещены, иначе это повлияет на последующие уведомления о событиях.

Пример

```
RK_S32 s32Ret = RK_FAILURE;
VI_EVENT_CALL_BACK_S stCallbackFunc;
stCallbackFunc.pfnCallback = test_vi_event_call_back;
stCallbackFunc.pPrivateData = ctx;
s32Ret = RK_MPI_VI_RegChnEventCallBack(ctx->pipeId, ctx->channelId, &stCallbackFunc);
...
// Реализация функции обратного вызова
static void test_vi_event_call_back(RK_VOID *pPrivateData, VI_CB_INFO_S *info) {
    TEST_VI_CTX_S *ctx = (TEST_VI_CTX_S *)pPrivateData;
    if (info) {
        if (info->u32Event & VI_CHANGE_EVENT_CONNECT_CHANGE)
            RK_LOGI("event connect change");
        if (info->u32Event & VI_CHANGE_EVENT_SOURCE_CHANGE) {
            RK_LOGI("event source change");
            test_vi_event_source_change(ctx);
        }
    }
}
```

6. Типы данных

6.1 VI_DEV

【Описание】

Определяет дескриптор устройства VI.

【Определение】

```
typedef RK_S32 VI_DEV;
```

6.2 VI_PIPE

【Описание】

Определяет PIPE VI.

【Определение】

```
typedef RK_S32 VI_PIPE;
```

6.3 VI_CHN

【Описание】

Определяет канал VI.

【Определение】

```
typedef RK_S32 VI_CHN;
```

6.4 VI_MAX_DEV_NUM

【Описание】

Определяет максимальное количество устройств.

【Определение】

```
#define VI_MAX_DEV_NUM  3
```

6.5 VI_MAX_PIPE_NUM

【Описание】

Определяет максимальное количество PIPE.

【Определение】

```
#define VI_MAX_PIPE_NUM  VI_MAX_DEV_NUM
```

6.6 VI_MAX_CHN_NUM

【Описание】

Определяет общее количество физических и расширенных каналов VI.

【Определение】

```
#define VI_MAX_CHN_NUM (VI_MAX_PHY_CHN_NUM + VI_MAX_EXT_CHN_NUM)
```

6.7 VI_DEV_ATTR_S

【Описание】

Определяет структуру атрибутов устройства VI. (Временно не используется; можно передать неинициализированную структуру данных.)

【Определение】

```
/* The attributes of a VI device */
typedef struct rkVI_DEV_ATTR_S {
    /* RW;Interface mode */
    VI_INTF_MODE_E      enIntfMode;
    /* RW;Work mode */
    VI_WORK_MODE_E      enWorkMode;
    /* The below members must be configured in BT.601 mode or DC mode and are invalid in other
modes */
     /* RW;Input data sequence (only the YUV format is supported) */
    VI_YUV_DATA_SEQ_E   enDataSeq;
    /* RW;RGB: CSC-709 or CSC-601, PT YUV444 disable; YUV: default yuv CSC coef PT YUV444 enable.
*/
    VI_DATA_TYPE_E      enInputDataType;
    /* RW;Input max size */
    SIZE_S              stMaxSize;
    /* RW;Data rate of Device */
    DATA_RATE_E         enDataRate;
} VI_DEV_ATTR_S;
```

【Члены】

Имя члена	Описание
enIntfMode	Режим интерфейса. (Временно не используется; можно не задавать.)
enWorkMode	Составной режим работы с 1, 2 или 4 трактами. (Временно не используется; можно не задавать.)
enDataSeq	Порядок входных данных (поддерживается только формат yuv). Если enIntfMode равен VI_MODE_BT656 или VI_MODE_BT601, диапазон значений — [VI_DATA_SEQ_UYVY, VI_DATA_SEQ_YVYU]. Если enIntfMod равен VI_MODE_BT1120_STANDARD или VI_MODE_MIPI_YUV420_NORMAL, VI_MODE_MIPI_YUV420_LEGACY, VI_MODE_MIPI_YUV422, значение должно быть VI_DATA_SEQ_VUVU либо VI_DATA_SEQ_UVUV. (Временно не используется; можно не задавать.)
enInputDataType	Тип входных данных: вход Sensor обычно RGB, вход AD обычно YUV. (Временно не используется; можно не задавать.)
stMaxSize	Максимальные ширина и высота захватываемого изображения. (Временно не используется; можно не задавать.)
enDataRate	Скорость устройства. (Временно не используется; можно не задавать.)

6.8 VI_DEV_BIND_PIPE_S

【Описание】

Определяет связь VI DEV с PIPE.

【Определение】

```
/* Information of pipe binded to device */
typedef struct rkVI_DEV_BIND_PIPE_S {
    RK_U32  u32Num;                                 /* RW;Range [1,VI_MAX_PHY_PIPE_NUM] */
    VI_PIPE PipeId[VI_MAX_PHY_PIPE_NUM];            /* RW;Array of pipe ID */
} VI_DEV_BIND_PIPE_S;
```

【Члены】

Имя члена	Описание
u32Num	Количество PIPE, связанных с данным VI Dev. Диапазон значений: [1, VI_MAX_PIPE_NUM].
PipeId	Номер PIPE, связанного с данным VI Dev.

【Примечания】

- В настоящее время для PIPE достаточно передать то же значение, что и для DEV.

  ctx->stBindPipe.u32Num = ctx->devId;
  ctx->stBindPipe.PipeId[0] = ctx->devId;

6.9 VI_CHN_ATTR_S

【Описание】

Определяет атрибуты канала VI.

【Определение】

```
/* The attributes of channel */
typedef struct rkVI_CHN_ATTR_S {
    SIZE_S              stSize;             /* RW;Channel out put size */
    PIXEL_FORMAT_E      enPixelFormat;      /* RW;Pixel format */
    DYNAMIC_RANGE_E     enDynamicRange;     /* RW;Dynamic Range */
```

```
    VIDEO_FORMAT_E      enVideoFormat;      /* RW;Video format */
    COMPRESS_MODE_E     enCompressMode;     /* RW;256B Segment compress or no compress.*/
    RK_BOOL             bMirror;            /* RW;Mirror enable */
    RK_BOOL             bFlip;              /* RW;Flip enable */
    RK_U32              u32Depth;           /* RW;Range [0,8];Depth */
    FRAME_RATE_CTRL_S   stFrameRate;        /* RW;Frame rate */
    VI_BUF_TYPE_E       enBufType;          /* RW;Channel buf opt */
    VI_ISP_OPT_S        stIspOpt;           /* RW;ISP opt */
} VI_CHN_ATTR_S;
```

Члены

Имя члена	Описание
stSize	Ширина и высота изображения, получаемого каналом.
enPixelFormat	Формат пикселей целевого изображения. Подробности см. в определении PIXEL_FORMAT_E в rk_comm_video.h.
enDynamicRange	Динамический диапазон целевого изображения. (Временно не используется; можно не задавать.)
enVideoFormat	Формат видеоданных целевого изображения. (Временно не используется; можно не задавать.)
enCompressMode	Формат сжатия целевого изображения.
bMirror	Переключатель Mirror. RK_FALSE: не включать; RK_TRUE: включить. (Временно не используется; можно не задавать.)
bFlip	Переключатель Flip. RK_FALSE: не включать; RK_TRUE: включить. (Временно не используется; можно не задавать.)
u32Depth	Глубина очереди получения изображений пользователем. Диапазон значений: [0, 8].
stFrameRate	Управление частотой кадров. Диапазон исходной частоты: (0, максимальная входная частота] и -1. Диапазон целевой частоты: [-1, исходная частота]. Если исходная частота равна -1, целевая также должна быть -1 (управление частотой не выполняется); в остальных случаях целевая частота не должна превышать исходную.
enAllocBufType	Тип выделения памяти изображения. VI_ALLOC_BUF_TYPE_INTERNAL: внутреннее выделение. VI_ALLOC_BUF_TYPE_EXTERNAL: внешнее выделение. Если изображение получается способом mmap, необходимо задать внешнее выделение.
stIspOpt	Параметры получения изображения, обработанного isp или проходящего сквозным путём.

Примечания

- Рекомендации по настройке u32Depth:
  - Если после VI не связан другой модуль, рекомендуется задавать u32Depth равным u32BufCount структуры VI_ISP_OPT_S. Если задать 0, получить выходной buf невозможно.
  - Если после VI связан другой модуль:
    - если требуется получать изображения через RK_MPI_VI_GetChnFrame, рекомендуется задавать u32Depth как минимум на 2 меньше u32BufCount структуры VI_ISP_OPT_S. Например, при u32BufCount=3 задайте u32Depth=1. Слишком большое u32Depth может нарушить нормальный вывод данных или привести к недостаточной частоте кадров нижестоящего модуля.
    - Если получать изображения через RK_MPI_VI_GetChnFrame не требуется, рекомендуется задать u32Depth=0.

6.10 SIZE_S

Описание

Определяет структуру сведений о размере изображения.

Определение

```
typedef struct rkSIZE_S {
    RK_U32 u32Width;
    RK_U32 u32Height;
} SIZE_S;
```

【Члены】

Имя члена	Описание
u32Width	Ширина.
u32Height	Высота.

6.11 COMPRESS_MODE_E

【Описание】

Определяет структуру формата сжатия данных.

【Определение】

```
typedef enum rkCOMPRESS_MODE_E {
    COMPRESS_MODE_NONE = 0,    /* No compress */
    COMPRESS_AFBC_16x16,
    COMPRESS_MODE_BUTT
} COMPRESS_MODE_E;
```

【Члены】

Имя члена	Описание
COMPRESS_MODE_NONE	Без сжатия
COMPRESS_AFBC_16x16	Сжатие AFBC
COMPRESS_MODE_BUTT	Нет

6.12 VI_ALLOC_BUF_TYPE_E

【Описание】

Определяет тип выделения памяти изображения VI.

【Определение】

```
typedef enum rkVI_ALLOC_BUF_TYPE_E {
    VI_ALLOC_BUF_TYPE_INTERNAL,
    VI_ALLOC_BUF_TYPE_EXTERNAL
} VI_ALLOC_BUF_TYPE_E;
```

【Члены】

Имя члена	Описание
VI_ALLOC_BUF_TYPE_INTERNA	Выделение внутри VI
VI_ALLOC_BUF_TYPE_EXTERNAL	Выделение вне VI

6.13 VI_ISP_OPT_S

【Описание】

Параметры получения изображения, обработанного isp или проходящего сквозным путём.

【Определение】

```
typedef struct rkVI_ISP_OPT_S {
    RK_U32                u32BufCount;        /* RW;ISP buf count */
    RK_U32                u32BufSize;         /* R;ISP buf size */
    VI_V4L2_CAPTURE_TYPE  enCaptureType;      /* RW;ISP capture type */
    VI_V4L2_MEMORY_TYPE   enMemoryType;       /* RW;ISP buf memory type */
    RK_CHAR               aEntityName[MAX_VI_ENTITY_NAME_LEN];       /* RW;ISP capture entity
name*/
    RK_BOOL               bNoUseLibV4L2;      /* RW;Is use libv4l2 */
    SIZE_S                stMaxSize;          /* RW;ISP bypass resolution */
} VI_ISP_OPT_S;
```

【Члены】

Имя члена	Описание
u32BufCount	Общее количество блоков буфера выходного канала.
u32BufSize	Размер буфера, выделяемого для каждого блока.
enCaptureType	Тип записи V4L2 для получения изображений.
enMemoryType	Тип памяти V4L2 для получения изображений.
aEntityName	Имя устройства канала.
bNoUseLibV4L2	Использовать ли libV4L2 (в настоящее время поддерживается только включённое состояние).
stMaxSize	Настройка размера изображения, получаемого каналом.

【Примечание】

- На платформе RK3588 из-за необходимости аппаратного буферирования u32BufCount должен содержать не менее 3 buffer, чтобы получать количество кадров, соответствующее требуемой частоте.
- На платформе RV1126 буферирование уже выполняется нижележащим драйвером, поэтому для u32BufCount достаточно меньшего значения; рекомендуется не менее 2.

6.14 VI_V4L2_CAPTURE_TYPE

【Описание】

Тип записи V4L2 для получения изображений.

【Определение】

```
typedef enum rkVI_V4L2_CAPTURE_TYPE {
    VI_V4L2_CAPTURE_TYPE_VIDEO_CAPTURE        = 1,
    VI_V4L2_CAPTURE_TYPE_VBI_CAPTURE          = 4,
    VI_V4L2_CAPTURE_TYPE_SLICED_VBI_CAPTURE   = 6,
    VI_V4L2_CAPTURE_TYPE_VIDEO_CAPTURE_MPLANE = 9,
    VI_V4L2_CAPTURE_TYPE_SDR_CAPTURE          = 11,
    VI_V4L2_CAPTURE_TYPE_META_CAPTURE         = 13,
    /* Deprecated, do not use */
    VI_V4L2_CAPTURE_TYPE_PRIVATE              = 0x80,
} VI_V4L2_CAPTURE_TYPE;
```

【Примечания】

- Совпадает с настройкой V4L2_CAPTURE_TYPE_VIDEO_CAPTURE.

6.15 VI_V4L2_MEMORY_TYPE

【Описание】

Тип памяти V4L2 для получения изображений.

【Определение】

```
typedef enum rkVI_V4L2_MEMORY_TYPE {
    VI_V4L2_MEMORY_TYPE_MMAP             = 1,
    VI_V4L2_MEMORY_TYPE_USERPTR          = 2,
    VI_V4L2_MEMORY_TYPE_OVERLAY          = 3,
    VI_V4L2_MEMORY_TYPE_DMABUF           = 4,
} VI_V4L2_MEMORY_TYPE;
```

【Члены】

Имя члена	Описание
VI_V4L2_MEMORY_TYPE_MMAP	Тип памяти MMAP.
VI_V4L2_MEMORY_TYPE_USERPTR	Тип памяти USERPTR.
VI_V4L2_MEMORY_TYPE_OVERLAY	Тип памяти OVERLAY.
VI_V4L2_MEMORY_TYPE_DMABUF	Тип памяти, выделенной DMA.

6.16 VI_CHN_STATUS_S

【Описание】

Структура состояния канала VI.

【Определение】

```
typedef struct rkVI_CHN_STATUS_S {
    RK_BOOL bEnable;                    /* RO;Whether this channel is enabled */
    RK_U32  u32FrameRate;               /* RO;Current frame rate */
    RK_U32  u32CurFrameID;              /* RO;Current frame id */
    RK_U32  u32InputLostFrame;          /* RO;Input lost frame count */
    RK_U32  u32OutputLostFrame;         /* RO;Output lost frame count */
    RK_U32  u32VbFail;                  /* RO;Video buffer malloc failure */
    SIZE_S  stSize;                     /* RO;Channel output size */
} VI_CHN_STATUS_S;
```

【Члены】

Имя члена	Описание
bEnable	Включён ли текущий канал. 0: не включён; 1: включён.
u32FrameRate	Текущая выходная частота кадров канала.
u32CurFrameID	Номер текущего кадра изображения канала.
u32InputLostFrame	Количество потерянных входных кадров текущего канала.
u32OutputLostFrame	Количество потерянных выходных кадров текущего канала.
u32VbFail	Количество неудачных попыток получения изображения текущим каналом.
stSize	Размер изображения текущего канала.

6.17 VI_SAVE_FILE_INFO_S

【Описание】

Структура сведений о сохранении данных канала VI.

【Определение】

```
typedef struct rkVI_SAVE_FILE_INFO_S {
    RK_BOOL     bCfg;
    RK_CHAR     aFilePath[MAX_VI_FILE_PATH_LEN];
    RK_CHAR     aFileName[MAX_VI_FILE_NAME_LEN];
    RK_U32      u32FileSize;  /*in KB*/
} VI_SAVE_FILE_INFO_S;
```

【Члены】

Имя члена	Описание
bCfg	Сохранять ли выходные данные канала. 0: не сохранять; 1: сохранять.
aFilePath	Путь к файлу сохранения выходных данных канала.
aFileName	Имя файла сохранения выходных данных канала.
u32FileSize	Размер файла сохранения выходных данных канала, в kB.

6.18 VI_USERPIC_BGC_S

【Описание】

Сведения о пользовательском изображении в режиме одноцветного фона.

【Определение】

```
typedef struct rkVI_USERPIC_BGC_S {
    RK_U32 u32BgColor;
} VI_USERPIC_BGC_S;
```

【Члены】

Имя члена	Описание
u32BgColor	Данные заполнения, соответствующие значению цвета RGB. Диапазон: [0, 0xFFFFFF]. Компонент R занимает биты 16–23, G — 8–15, B — 0–7.

6.19 VI_USERPIC_MODE_E

【Описание】

Тип пользовательского изображения.

【Определение】

```
typedef enum rk_VI_USERPIC_MODE_E {
    VI_USERPIC_MODE_PIC = 0,
    VI_USERPIC_MODE_BGC,
    VI_USERPIC_MODE_BUTT,
} VI_USERPIC_MODE_E;
```

【Члены】

Имя члена	Описание
VI_USERPIC_MODE_PIC	Режим изображения YUV
VI_USERPIC_MODE_BGC	Режим одноцветного фонового изображения.

【Примечания】

Нет

6.20 VI_USERPIC_ATTR_S

【Описание】

Сведения о пользовательском изображении.

【Определение】

```
typedef struct rkVI_USERPIC_ATTR_S {
    VI_USERPIC_MODE_E enUsrPicMode;
    union {
        VIDEO_FRAME_INFO_S stUsrPicFrm;
        VI_USERPIC_BGC_S stUsrPicBg;
    } unUsrPic;
} VI_USERPIC_ATTR_S;
```

【Члены】

Имя члена	Описание
enUsrPicMode	Режим пользовательского изображения
stUsrPicFrm	Сведения о пользовательском изображении в режиме YUV
stUsrPicBg	Сведения о пользовательском изображении в режиме одноцветного фона. Диапазон значений: [0x0, 0xFFFFFF]. Компонент R занимает биты 0–7, G — 8–15, B — 16–23.

【Примечания】

Нет

6.21 VI_CONNECT_INFO_S

【Описание】

Сведения о подключении канала.

【Определение】

```
typedef struct rkVI_CONNECT_INFO_S {
    RK_U32 u32Width;
    RK_U32 u32Height;
    RK_FLOAT f32FrameRate;
    PIXEL_FORMAT_E enPixFmt;
    VI_CONNECT_STATE_E enConnect;
} VI_CONNECT_INFO_S;
```

【Члены】

Имя члена	Описание
u32Width	Ширина входного изображения канала
u32Height	Высота входного изображения канала
f32FrameRate	Входная частота кадров канала
enPixFmt	Формат входного изображения канала
enConnect	Состояние подключения канала

Примечания

Нет

6.22 VI_CONNECT_STATE_E

Описание

Состояние подключения канала.

Определение

```
typedef enum rkVI_CONNECT_STATE_E {
    VI_CONNECT_STATE_UNKNOWN = 0,
    VI_CONNECT_STATE_CONNECT,
    VI_CONNECT_STATE_DISCONNECT,
    VI_CONNECT_STATE_BUTT
} VI_CONNECT_STATE_E;
```

Члены

Имя члена	Описание
VI_CONNECT_STATE_UNKNOWN	Неизвестно
VI_CONNECT_STATE_CONNECT	Подключено
VI_CONNECT_STATE_DISCONNECT	Не подключено
VI_CONNECT_STATE_BUTT	Максимальное значение

Примечания

Нет

6.23 VI_EDID_S

Описание

Структура сведений EDID канала.

Определение

```
typedef struct rkVI_EDID_S {
    RK_U32 u32Pad;
    RK_U32 u32StartBlock;
    RK_U32 u32Blocks;
    RK_U32 au32Reserved[5];
    RK_U8 *pu8Edid;
} VI_EDID_S;
```

Члены

Имя члена	Описание
u32Pad	Значение pad; зарезервированный параметр, задайте 0.
u32StartBlock	Начальный block EDID. При настройке EDID можно задать только 0; при получении EDID диапазон значений: [0, u32Blocks).
u32Blocks	Количество block EDID. При настройке означает количество block EDID, которое требуется задать; при получении означает количество block EDID канала.
au32Reserved	Зарезервированный параметр, задайте 0.
pu8Edid	Значение EDID. Каждый block EDID содержит 128 байт: начальный адрес первого block — pu8Edid[0], второго — pu8Edid[128] и так далее.

Примечания

Нет

6.24 VI_STREAM_S

Описание

Структура закодированного stream.

Определение

```
typedef struct rkVI_STREAM_S {
    MB_BLK  pMbBlk;
    RK_U32  u32Len;
    RK_U32  u32Seq;
    RK_U64  u64PTS;
} VI_STREAM_S;
```

Члены

Имя члена	Описание
pMbBlk	Дескриптор памяти потока.
u32Len	Длина потока.
u32Seq	Порядковый номер потока.
u64PTS	Временная метка потока. Единица: us.

Примечания

Нет

6.25 VI_EVENT_CALL_BACK_S

Описание

Структура обратного вызова событий.

Определение

```
typedef struct rkVI_EVENT_CALL_BACK_S {
    RK_VI_EventCallBack pfnCallback;
    RK_VOID            *pPrivateData;
} VI_EVENT_CALL_BACK_S;
```

Члены

Имя члена	Описание
pfnCallback	Указатель на функцию обратного вызова событий.
pPrivateData	Указатель на частные данные.

Примечания

Нет

6.26 RK_VI_EventCallBack

Описание

Функция обратного вызова событий.

Определение

```
typedef void (*RK_VI_EventCallback)(RK_VOID *pPrivateData, VI_CB_INFO_S *pstInfo);
```

Примечания

Нет

6.27 VI_CB_INFO_S

Описание

Структура сведений о функции обратного вызова событий.

Определение

```
typedef struct rkVI_CB_INFO_S {
    RK_U32 u32Event;
} VI_CB_INFO_S;
```

Члены

Имя члена	Описание
u32Event	Тип события.

Примечания

Нет

6.28 VI_EVENT_E

Описание

Структура типа события.

Определение

```
typedef enum rkVI_EVENT_E {
    VI_EVENT_CONNECT_CHANGE = 1 << 0,
    VI_EVENT_SOURCE_CHANGE = 1 << 1,
} VI_EVENT_E;
```

Члены

Имя члена	Описание
VI_EVENT_CONNECT_CHANGE	Событие подключения/отключения
VI_EVENT_SOURCE_CHANGE	Событие переключения входного источника

7. Коды ошибок VI

Коды ошибок API видеовхода VI приведены ниже.

Код ошибки	Макроопределение	Описание
0xA0088001	RK_ERR_VI_INVALID_DEVID	Недопустимый номер устройства VI
0xA0088002	RK_ERR_VI_INVALID_CHNID	Недопустимый номер канала VI
0xA0088003	RK_ERR_VI_INVALID_PARA	Недопустимые параметры VI
0xA0088004	RK_ERR_VI_EXIST	Канал VI уже существует
0xA0088005	RK_ERR_VI_UNEXIST	Объект канала не существует
0xA0088006	RK_ERR_VI_INVALID_NULL_PTR	Ошибка нулевого указателя VI
0xA0088007	RK_ERR_VI_NOT_CONFIG	Параметры VI не настроены
0xA0088008	RK_ERR_VI_NOT_SUPPORT	Операция не разрешена
0xA0088009	RK_ERR_VI_NOT_PERM	Нет разрешения на операцию
0xA008800A	RK_ERR_VI_INVALID_PIPEID	Недопустимый номер PIPE VI
0xA008800C	RK_ERR_VI_NOMEM	Нет памяти
0xA008800E	RK_ERR_VI_BUF_EMPTY	Входной буфер VI пуст
0xA008800F	RK_ERR_VI_BUF_FULL	Входной буфер VI заполнен
0xA0088010	RK_ERR_VI_SYS_NOTREADY	Система не готова
0xA0088012	RK_ERR_VI_BUSY	Устройство VI занято

Подсистема обработки видео

Предисловие

Обзор

VPSS (Video Process Sub-System) — подсистема обработки видео. Поддерживаемые функции обработки изображений включают CROP, Scale, преобразование формата пикселей, поворот на фиксированный угол, Cover/Coverex, Mirror/Flip, сжатие и распаковку и т. д.

Версии продуктов

Название микросхемы	Версия ядра
RV1109/RV1126	4.19
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	XLM	2020-12-16	Начальная версия
V0.2.0	XLM	2021-01-23	Уточнены определения структур данных
V0.3.0	XLM	2021-02-05	Исправлены отдельные ошибки, добавлены переходы к функциям и структурам
V0.3.1	XLM	2021-06-11	Исправлены отдельные ошибки, добавлены пояснения некоторых интерфейсов
V0.4.0	XLM	2021-07-27	Добавлены иллюстрации и перечень поддерживаемых VPSS форматов
V0.4.1	JKM	2021-09-29	Исправлены отдельные ошибки Добавлена поддержка RV1109/RV1126 и пояснения отдельных параметров
V0.4.2	XLM	2021-11-29	Добавлено описание функции соотношения сторон
V1.0.0	XLM	2021-12-06	Добавлены подробные описания и сведения о различиях микросхем
V1.0.1	XLM	2021-12-14	Исправлено описание наложения угла поворота и зеркального отражения/переворота для RK356X/RK3588
V1.0.2	ZQX	2022-02-18	Добавлен интерфейс группового поворота
V1.0.3	XLM	2022-06-16	Добавлено описание целевой частоты кадров 0 при управлении частотой
V1.0.4	XLM	2022-07-27	Исправлено описание глубины канала depth

Содержание

Описание функций
Справочник API
Типы данных
Коды ошибок VPSS

1. Описание функций

1.1 Основные понятия

1.1.1 GROUP

VPSS предоставляет пользователю понятие группы (GROUP). Максимальное количество см. в определении VPSS_MAX_GRP_NUM. Все GROUP разделяют аппаратное устройство по времени; аппаратное обеспечение последовательно обрабатывает задачи, отправленные каждой группой.

1.1.2 CHANNEL

Каналы группы VPSS. Предоставляется несколько каналов, каждый из которых поддерживает масштабирование, обрезку и другие функции. Изображение обрезается и масштабируется до заданного пользователем целевого разрешения.

1.1.3 CROP

Обрезка бывает двух типов: групповая и канальная.

- При групповой обрезке VPSS обрезает входное изображение.
- При канальной обрезке VPSS с помощью аппаратного устройства обрезает выходное изображение каждого канала.

1.1.4 Преобразование формата пикселей

Поддерживается преобразование форматов входных и выходных изображений, например NV12->RGB565.

1.1.5 Scale

Масштабирование уменьшает или увеличивает изображение. Для группы по горизонтали и вертикали поддерживается увеличение и уменьшение до 32 раз; для канала — также до 32 раз.

1.1.6 Mirror/Flip

Mirror — горизонтальное зеркальное отражение, Flip — вертикальный переворот. Сочетание Mirror+Flip позволяет выполнить поворот на 180°.

1.1.7 FRC (управление частотой кадров)

Управление частотой кадров бывает двух типов: групповое и канальное.

- Групповое управление частотой кадров контролирует приём входных изображений каждой Group.
- Канальное управление частотой кадров контролирует обработку изображений каждого канала.

1.1.8 Cover

Область маскирования видео: VGS вызывается для заполнения выходного изображения канала VPSS одноцветным блоком.

1.1.9 Overlay

Область наложения видео: VGS вызывается для наложения bitmap на выходное изображение канала VPSS.

1.1.10 Mosaic

Мозаика: VGS вызывается для заполнения выходного изображения канала VPSS мозаичными блоками.

1.1.11 Настройка канала RGN

На микросхемах RK356X/RK3588 и VPSS Group, и каналы поддерживают Overlay/Cover/Mosaic/Line и другие функции. При RGN AttachChn, если параметр MPP_CHN_S- >s32ChnId задан в диапазоне [0, VPSS_MAX_CHN_NUM), изображение накладывается на указанный канал; если он равен VPSS_MAX_CHN_NUM, изображение накладывается на Group.

1.1.12 Поворот на фиксированный угол

Поддерживается поворот на фиксированные углы 0°, 90°, 180° и 270°.

1.1.13 Сжатие

Поддерживается сжатие выходного изображения канала, включая AFBC.

1.1.14 Распаковка

Входные сжатые изображения распаковываются; поддерживается распаковка AFBC.

1.1.15 Aspect Ratio

Соотношение сторон. Задаёт отношение ширины к высоте выходного изображения относительно входного.

1.1.16 Режим работы канала

Режимы работы канала VPSS делятся на два основных типа и переключаются изменением VPSS_CHN_MODE_E:

- Режим USER (VPSS_CHN_MODE_USER): задаётся, когда пользователю требуется вручную получать обработанные выходные изображения канала; создаёт дополнительную нагрузку.
- Режим PAST (VPSS_CHN_MODE_PASSTHROUGH): используется при связывании с VO; обработка выполняется совместно внутри VO и не создаёт дополнительной нагрузки. Рекомендуется для предварительного просмотра и воспроизведения.

1.1.17 Тип аппаратного устройства

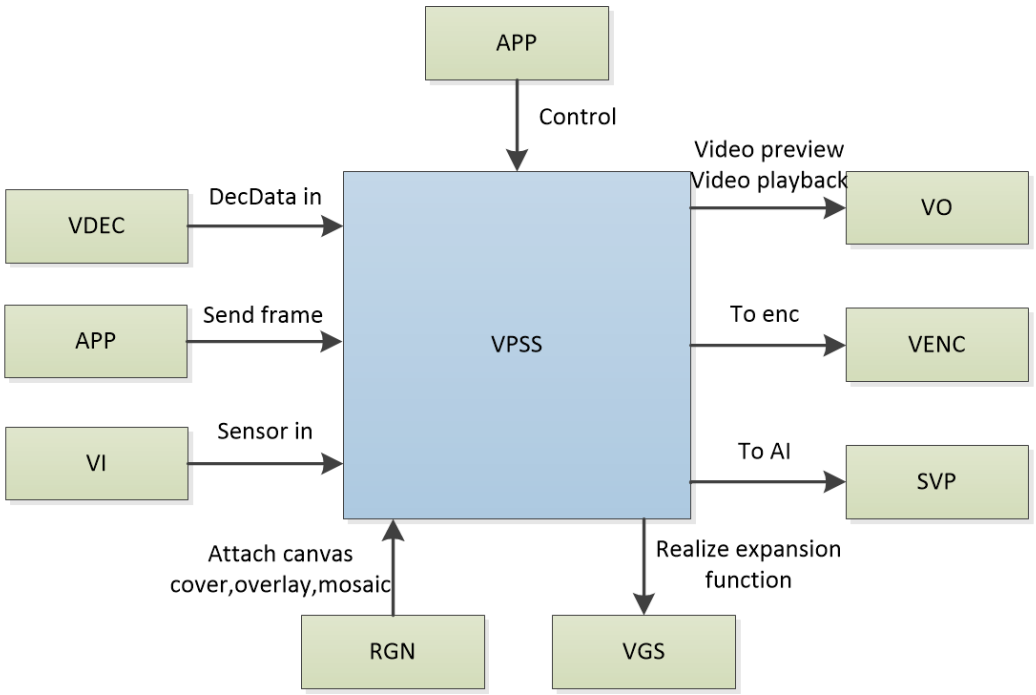
VPSS может использовать аппаратные устройства двух типов. Тип VIDEO_PROC_DEV_TYPE_E и аппаратное устройство переключаются через RK_MPI_VPSS_SetVProcDev. По умолчанию различия в зависимости от микросхемы приведены ниже:

Название микросхемы	Тип аппаратного устройства
RV1109/RV1126	RGA
RK356X	GPU
RK3588	GPU

1.2 Способы применения

Положение VPSS в системе показано на рисунке 1-1.

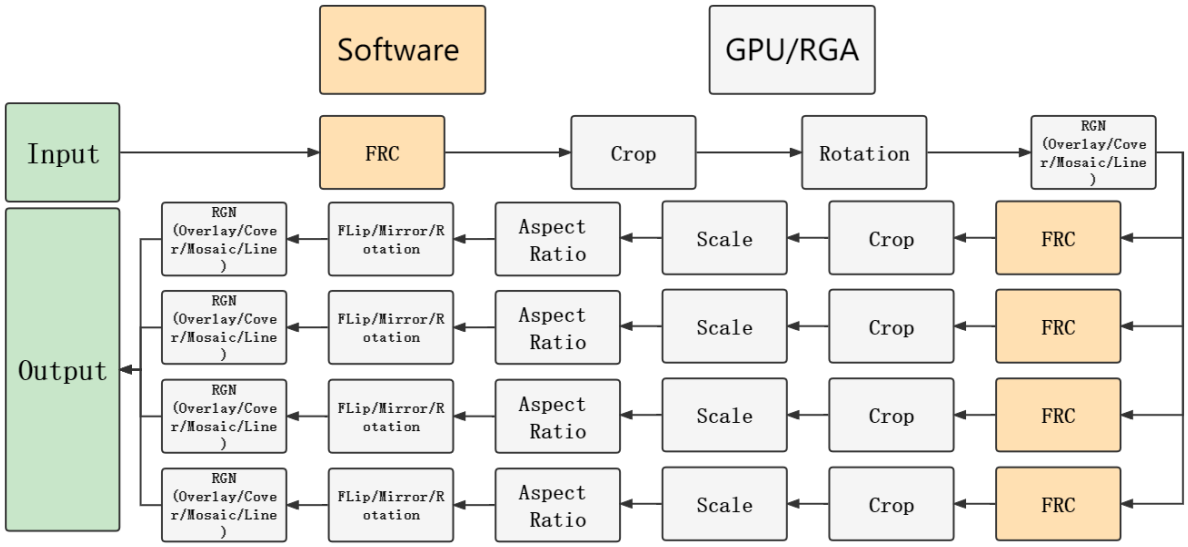
1.2.1 Рисунок 1-1. Контекстные связи VPSS



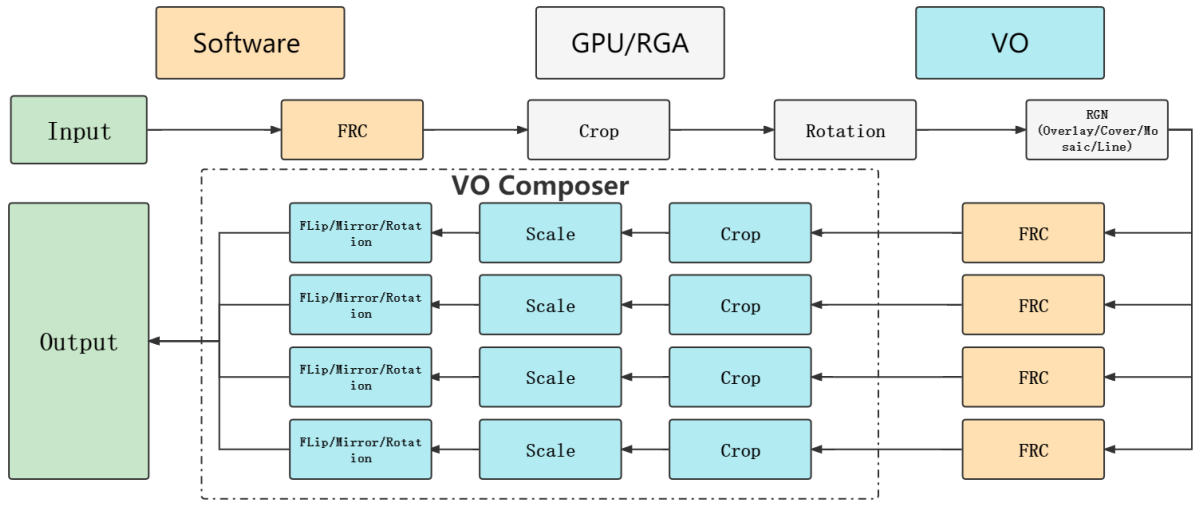
Через интерфейс связывания модуля SYS VPSS можно связать с VDEC/VI и VO/VENC/SVP: первые являются источниками входных данных VPSS, вторые — получателями VPSS. Пользователь управляет Group через интерфейсы VPSS MPI. Каждая Group может быть связана только с одним источником. Каналы Group имеют два режима работы: USER и PASSTHOUGH, между которыми можно динамически переключаться. Режим USER главным образом используется без связывания для ручного получения кадров из канала в сценариях алгоритмов AI, пользовательского анализа и обработки данных. Режим PASSTHOUGH главным образом применяется для управления воспроизведением в сценариях предварительного просмотра и воспроизведения, например для цифрового увеличения. В обоих режимах каждый канал может быть связан с несколькими получателями.

1.3 Процесс обработки микросхемы

1.3.1 Схема потока данных VPSS RK356X/RK3588 в режиме USER



1.3.2 Схема потока данных VPSS RK356X/RK3588 в режиме PAST



Когда канал VPSS работает в режиме PAST, большинство операций обработки данных (например, в пунктирной рамке на рисунке) выполняется так: VPSS передаёт параметры обработки модулю VO, а VO выполняет фактическую обработку за один проход, экономя аппаратные ресурсы.

Примечания

- В режиме PAST, если канал связан и автоматически передаёт кадры нижестоящему модулю, наложение RGN Attach на канал не действует.

1.4 Характеристики ввода и вывода

1.4.1 Форматы выходного изображения и способы выравнивания

VPSS использует выравнивание по пикселям. Подробности поддержки для каждой микросхемы приведены ниже:

1.4.1.1 Перечень поддерживаемых входных/выходных изображений RK356X/RK3588

Формат данных	Выравнивание ширины, пиксели	Выравнивание высоты, пиксели	Разрядность пикселя (bit)
RK_FMT_YUV420SP RK_FMT_YUV420SP_VU	16	1	8
RK_FMT_YUV420P	64	1	8
RK_FMT_YUV422SP	16	1	16
RK_FMT_YUV422_YUYV	32	1	16
RK_FMT_YUV400SP	64	1	8
RK_FMT_YUV420SP_10BIT	32	1	10
RK_FMT_RGB565 RK_FMT_BGR565 RK_FMT_RGBA5551 RK_FMT_BGRA5551	32	1	16
RK_FMT_RGB888 RK_FMT_BGR888	64	1	24
RK_FMT_BGRA8888 RK_FMT_RGBA8888	16	1	32

1.4.1.2 Перечень поддерживаемых выходных изображений RV1109/RV1126

Формат данных	Выравнивание ширины, пиксели	Выравнивание высоты, пиксели	Разрядность пикселя (bit)
RK_FMT_YUV420SP RK_FMT_YUV420SP_VU RK_FMT_YUV420P	4	2	8
RK_FMT_YUV422SP RK_FMT_YUV422P	4	2	8
RK_FMT_YUV400SP	4	2	8
RK_FMT_YUV420SP_10BIT RK_FMT_YUV422SP_10BIT	16	2	10
RK_FMT_YUV422_YUYV	4	2	8
RK_FMT_RGB888 RK_FMT_BGR888	4	1	24
RK_FMT_RGB565 RK_FMT_BGR565	2	1	16
RK_FMT_RGBA5551 RK_FMT_BGRA5551	2	1	16
RK_FMT_BGRA8888 RK_FMT_RGBA8888	1	1	32
RK_FMT_BGRA4444	2	1	16

1.5 Пример

```
    RK_S32 s32Ret = RK_SUCCESS;
    VPSS_GRP VpssGrp = 0;
    VPSS_CHN VpssChn[VPSS_MAX_CHN_NUM] = { VPSS_CHN0, VPSS_CHN1, VPSS_CHN2, VPSS_CHN3 };
    VPSS_GRP_ATTR_S stGrpVpssAttr;
    VPSS_CHN_ATTR_S stVpssChnAttr;

    stGrpVpssAttr.u32MaxW = SRC_WIDTH;
    stGrpVpssAttr.u32MaxH = SRC_HEIGHT;
    stGrpVpssAttr.enPixelFormat = PIXEL_FORMAT_YUV_SEMIPLANAR_420;
    stGrpVpssAttr.enCompressMode = COMPRESS_AFBC_16x16;
    stGrpVpssAttr.stFrameRate.s32SrcFrameRate = -1;
    stGrpVpssAttr.stFrameRate.s32DstFrameRate = -1;
    s32Ret = RK_MPI_VPSS_CreateGrp(VpssGrp, &stGrpVpssAttr);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }

    stCropInfo.bEnable = RK_TRUE;
    stCropInfo.enCropCoordinate = VPSS_CROP_ABS_COOR;
    stCropInfo.stCropRect.s32X = 640;
    stCropInfo.stCropRect.s32Y = 360;
    stCropInfo.stCropRect.u32Width = 640;
    stCropInfo.stCropRect.u32Height = 360;
    s32Ret = RK_MPI_VPSS_SetGrpCrop(VpssGrp, &stCropInfo);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }
    s32Ret = RK_MPI_VPSS_GetGrpCrop(VpssGrp, &stCropInfo);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
```

```
    }
    memset(&stVpssChnAttr, 0, sizeof(VPSS_CHN_ATTR_S));
    stVpssChnAttr.enChnMode = VPSS_CHN_MODE_USER;
    stVpssChnAttr.enCompressMode = COMPRESS_MODE_NONE;
    stVpssChnAttr.enDynamicRange = DYNAMIC_RANGE_SDR8;
    stVpssChnAttr.enPixelFormat = PIXEL_FORMAT_YUV_SEMIPLANAR_420;
    stVpssChnAttr.stFrameRate.s32SrcFrameRate = -1;
    stVpssChnAttr.stFrameRate.s32DstFrameRate = -1;
    stChnCropInfo.bEnable = RK_TRUE;
    stChnCropInfo.enCropCoordinate = VPSS_CROP_RATIO_COOR;
    stChnCropInfo.stCropRect.s32X = 500;
    stChnCropInfo.stCropRect.s32Y = 500;
    stChnCropInfo.stCropRect.u32Width = 500;
    stChnCropInfo.stCropRect.u32Height = 500;
    for (RK_S32 i = 0; i < VPSS_MAX_CHN_NUM; i++) {
        stVpssChnAttr.u32Width = SRC_WIDTH / VPSS_MAX_CHN_NUM * (i + 1);
        stVpssChnAttr.u32Height = SRC_HEIGHT / VPSS_MAX_CHN_NUM * (i + 1);
        s32Ret = RK_MPI_VPSS_SetChnAttr(VpssGrp, VpssChn[i], &stVpssChnAttr);
        if (s32Ret != RK_SUCCESS) {
            return s32Ret;
        }
        s32Ret = RK_MPI_VPSS_GetChnAttr(VpssGrp, VpssChn[i], &stVpssChnAttr);
        if (s32Ret != RK_SUCCESS) {
            return s32Ret;
        }
        s32Ret = RK_MPI_VPSS_SetChnCrop(VpssGrp, VpssChn[i], &stChnCropInfo);
        if (s32Ret != RK_SUCCESS) {
            return s32Ret;
        }
        s32Ret = RK_MPI_VPSS_GetChnCrop(VpssGrp, VpssChn[i], &stChnCropInfo);
        if (s32Ret != RK_SUCCESS) {
            return s32Ret;
        }
        s32Ret = RK_MPI_VPSS_EnableChn(VpssGrp, VpssChn[i]);
        if (s32Ret != RK_SUCCESS) {
            return s32Ret;
        }
    }
    s32Ret = RK_MPI_VPSS_StartGrp(VpssGrp);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }
    s32Ret = RK_MPI_VPSS_StopGrp(VpssGrp);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }
    for (RK_S32 i = 0; i < VPSS_MAX_CHN_NUM; i++) {
        s32Ret = RK_MPI_VPSS_DisableChn(VpssGrp, VpssChn[i]);
        if (s32Ret != RK_SUCCESS) {
            return s32Ret;
        }
    }
    s32Ret = RK_MPI_VPSS_DestroyGrp(VpssGrp);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }
```

2. Справочник API

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_VPSS_CreateGrp: создаёт VPSS GROUP.
- RK_MPI_VPSS_DestroyGrp: уничтожает VPSS GROUP.

- RK_MPI_VPSS_StartGrp: включает VPSS GROUP.
- RK_MPI_VPSS_StopGrp: отключает VPSS GROUP.
- RK_MPI_VPSS_ResetGrp: сбрасывает VPSS GROUP.
- RK_MPI_VPSS_GetGrpAttr: получает атрибуты VPSS GROUP.
- RK_MPI_VPSS_SetGrpAttr: задаёт атрибуты VPSS GROUP.
- RK_MPI_VPSS_SetGrpCrop: задаёт атрибуты функции CROP VPSS GROUP.
- RK_MPI_VPSS_GetGrpCrop: получает атрибуты функции CROP VPSS GROUP.
- RK_MPI_VPSS_SetGrpRotation: задаёт атрибуты поворота изображения VPSS GROUP на фиксированный угол.
- RK_MPI_VPSS_GetGrpRotation: получает атрибуты поворота изображения VPSS GROUP на фиксированный угол.
- RK_MPI_VPSS_SendFrame: пользователь отправляет данные в VPSS GROUP.
- RK_MPI_VPSS_GetGrpFrame: пользователь получает один исходный кадр из VPSS GROUP.
- RK_MPI_VPSS_ReleaseGrpFrame: пользователь освобождает один исходный кадр.
- RK_MPI_VPSS_EnableBackupFrame: включает backup-кадр.
- RK_MPI_VPSS_DisableBackupFrame: отключает backup-кадр.
- RK_MPI_VPSS_SetChnAttr: задаёт атрибуты канала VPSS.
- RK_MPI_VPSS_GetChnAttr: получает атрибуты канала VPSS.
- RK_MPI_VPSS_EnableChn: включает канал VPSS.
- RK_MPI_VPSS_DisableChn: отключает канал VPSS.
- RK_MPI_VPSS_SetChnCrop: задаёт атрибуты функции обрезки канала VPSS.
- RK_MPI_VPSS_GetChnCrop: получает атрибуты функции обрезки канала VPSS.
- RK_MPI_VPSS_SetChnRotation: задаёт атрибуты поворота изображения канала VPSS на фиксированный угол.
- RK_MPI_VPSS_GetChnRotation: получает атрибуты поворота изображения канала VPSS на фиксированный угол.
- RK_MPI_VPSS_SetChnRotationEx: задаёт атрибуты поворота VPSS на произвольный угол.
- RK_MPI_VPSS_GetChnRotationEx: получает атрибуты поворота VPSS на произвольный угол.
- RK_MPI_VPSS_GetChnFrame: пользователь получает один кадр изображения канала.
- RK_MPI_VPSS_ReleaseChnFrame: пользователь освобождает один кадр изображения канала.
- RK_MPI_VPSS_AttachMbPool: связывает канал VPSS с определённым пулом видеобуферов MB.
- RK_MPI_VPSS_DetachMbPool: отменяет связь канала VPSS с определённым пулом видеобуферов MB.
- RK_MPI_VPSS_GetChnFd: получает дескриптор файла устройства, соответствующего каналу VPSS.
- RK_MPI_VPSS_CloseFd: закрывает файловый дескриптор устройства и канала.
- RK_MPI_VPSS_SetVProcDev: задаёт тип аппаратного устройства VPSS.
- RK_MPI_VPSS_GetVProcDev: получает тип аппаратного устройства VPSS.
- RK_MPI_VPSS_SetModParam: задаёт параметры модуля VPSS.
- RK_MPI_VPSS_GetModParam: получает параметры модуля VPSS.

2.1 RK_MPI_VPSS_CreateGrp

【Описание】

Создаёт VPSS GROUP.

【Синтаксис】

RK_S32 RK_MPI_VPSS_CreateGrp(VPSS_GRP VpssGrp, const VPSS_GRP_ATTR_S *pstGrpAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
pstGrpAttr	Указатель на атрибуты VPSS GROUP.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Повторное создание не поддерживается и возвращает RK_ERR_VPSS_NOT_PERM.

2.2 RK_MPI_VPSS_DestroyGrp

【Описание】

Уничтожает VPSS GROUP.

【Синтаксис】

RK_S32 RK_MPI_VPSS_DestroyGrp(VPSS_GRP VpssGrp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Повторное уничтожение не поддерживается и возвращает RK_ERR_VPSS_UNEXIST.
- Перед вызовом этого интерфейса необходимо вызвать RK_MPI_VPSS_StopGrp и отключить GROUP.
- При вызове интерфейс ожидает завершения текущих задач GROUP и только затем фактически уничтожает её.

2.3 RK_MPI_VPSS_StartGrp

【Описание】

Включает VPSS GROUP.

【Синтаксис】

RK_S32 RK_MPI_VPSS_StartGrp(VPSS_GRP VpssGrp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Для одной группы только первый вызов функции завершается успешно; повторные вызовы возвращают ошибку.

2.4 RK_MPI_VPSS_StopGrp

【Описание】

Отключает VPSS GROUP.

【Синтаксис】

RK_S32 RK_MPI_VPSS_StopGrp(VPSS_GRP VpssGrp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Для одной группы только первый вызов функции завершается успешно; повторные вызовы возвращают ошибку.

2.5 RK_MPI_VPSS_ResetGrp

【Описание】

Сбрасывает VPSS GROUP.

【Синтаксис】

RK_S32 RK_MPI_VPSS_ResetGrp(VPSS_GRP VpssGrp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Данные, ожидающие обработки или потребления в группе и каналах, освобождаются.

2.6 RK_MPI_VPSS_GetGrpAttr

【Описание】

Получает атрибуты VPSS GROUP.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetGrpAttr(VPSS_GRP VpssGrp, VPSS_GRP_ATTR_S *pstGrpAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
pstGrpAttr	Указатель на атрибуты VPSS GROUP.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Атрибуты GROUP должны быть допустимыми; некоторые статические атрибуты нельзя задавать динамически.

2.7 RK_MPI_VPSS_SetGrpAttr

【Описание】

Задаёт атрибуты VPSS GROUP.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SetGrpAttr(VPSS_GRP VpssGrp, const VPSS_GRP_ATTR_S *pstGrpAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
pstGrpAttr	Указатель на атрибуты VPSS GROUP.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Атрибуты GROUP должны быть допустимыми; некоторые статические атрибуты нельзя задавать динамически.

2.8 RK_MPI_VPSS_SetGrpCrop

【Описание】

Задаёт атрибуты функции CROP VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SetGrpCrop(VPSS_GRP VpssGrp, const VPSS_CROP_INFO_S *pstCropInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
pstCropInfo	Параметры функции CROP.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- При обрезке в относительном режиме координаты области должны находиться в диапазоне [0, 999], а ширина и высота — [1, 1000].
- Размер области CROP не должен быть меньше минимального размера VPSS или превышать максимальное поддерживаемое входное разрешение. Начальная точка области обрезки не может иметь отрицательные координаты; правая граница не должна превышать максимальную входную ширину VPSS, а нижняя — максимальную входную высоту VPSS.
- Если область обрезки выходит за границы изображения, её координаты сдвигаются к началу координат; приоритет отдаётся сохранению заданных ширины и высоты.
- Если ширина обрезки больше ширины входного изображения, выходная ширина корректируется до ширины входного изображения.
- Если высота обрезки больше высоты входного изображения, выходная высота корректируется до высоты входного изображения.
- При связанном VO не рекомендуется использовать этот интерфейс для цифрового увеличения. Рекомендуется задать режим канала PAST и использовать RK_MPI_VPSS_SetChnCrop для цифрового увеличения.
- Если канал находится в режиме PAST, настройка действует только при связанном VO.

2.9 RK_MPI_VPSS_GetGrpCrop

【Описание】

Получает атрибуты функции CROP VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetGrpCrop(VPSS_GRP VpssGrp, VPSS_CROP_INFO_S *pstCropInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
pstCropInfo	Параметры функции CROP.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.

2.10 RK_MPI_VPSS_SetGrpRotation

【Описание】

Задаёт атрибуты поворота изображения VPSS GROUP на фиксированный угол.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SetGrpRotation(VPSS_GRP VpssGrp, ROTATION_E enRotation);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
enRotation	Атрибут поворота.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Поддерживаются только углы 0°, 90°, 180° и 270°; произвольные углы не поддерживаются.

2.11 RK_MPI_VPSS_GetGrpRotation

【Описание】

Получает атрибуты поворота изображения VPSS GROUP на фиксированный угол.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetGrpRotation(VPSS_GRP VpssGrp, ROTATION_E *penRotation);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
penRotation	Атрибут поворота.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.

2.12 RK_MPI_VPSS_SendFrame

【Описание】

Пользователь отправляет данные в VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SendFrame(VPSS_GRP VpssGrp, VPSS_GRP_PIPE VpssGrpPipe, const VIDEO_FRAME_INFO_S *pstVideoFrame, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssGrpPipe	Номер pipeline группы VPSS. Допустимо только значение 0.	Ввод
pstVideoFrame	Сведения об отправляемом изображении.	Ввод
s32MilliSec	Если параметр ожидания s32MilliSec равен -1, интерфейс блокирующий; если 0 — неблокирующий; если больше 0 — это время ожидания в миллисекундах (ms).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- При использовании интерфейса пользователь может самостоятельно управлять частотой кадров.
- Если канал VPSS работает в режиме PAST, перед использованием интерфейса VPSS должен быть нормально запущен; рекомендуется связать его с VO, иначе изображение, полученное пользователем через интерфейс получения, будет обработано VGS. Либо канал VPSS должен работать в режиме USER, и пользователь получает изображение через интерфейс получения.

2.13 RK_MPI_VPSS_GetGrpFrame

【Описание】 Пользователь получает один исходный кадр из GROUP. Основной сценарий: при паузе или покадровом воспроизведении декодированного HD-видео два канала на уровне PIP и обычном видеослое отображают один и тот же кадр. Это реализуется совместным использованием данного интерфейса, RK_MPI_VPSS_SendFrame и других интерфейсов.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetGrpFrame(VPSS_GRP VpssGrp, VPSS_GRP_PIPE VpssGrpPipe, VIDEO_FRAME_INFO_S *pstVideoFrame);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssGrpPipe	Номер pipeline группы VPSS. Допустимо только значение 0.	Ввод
pstVideoFrame	Сведения об изображении.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Полученное изображение необходимо своевременно освобождать, иначе возникает нехватка MB или останавливается декодирование/воспроизведение. Рекомендуется использовать парно с RK_MPI_VPSS_ReleaseGrpFrame.
- Получение возможно только при включённом backup-кадре.
- Получение возможно только при связанном VO.
- Получение возможно только в режиме канала PAST.

2.14 RK_MPI_VPSS_ReleaseGrpFrame

【Описание】

Пользователь освобождает один исходный кадр.

【Синтаксис】

RK_S32 RK_MPI_VPSS_ReleaseGrpFrame(VPSS_GRP VpssGrp, VPSS_GRP_PIPE VpssGrpPipe, const VIDEO_FRAME_INFO_S *pstVideoFrame);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssGrpPipe	Номер pipeline группы VPSS. Допустимо только значение 0.	Ввод
pstVideoFrame	Сведения об изображении.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Параметр VpssGrp этого интерфейса фактически не используется и может иметь любое значение допустимого диапазона. Номер PIPE должен быть только 0.
- Интерфейс необходимо использовать парно с RK_MPI_VPSS_GetGrpFrame.

2.15 RK_MPI_VPSS_EnableBackupFrame

【Описание】

Включает Backup-кадр.

【Синтаксис】

RK_S32 RK_MPI_VPSS_EnableBackupFrame(VPSS_GRP VpssGrp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.

2.16 RK_MPI_VPSS_DisableBackupFrame

【Описание】

Отключает Backup-кадр.

【Синтаксис】

RK_S32 RK_MPI_VPSS_DisableBackupFrame(VPSS_GRP VpssGrp)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.

2.17 RK_MPI_VPSS_SetChnAttr

【Описание】

Задаёт атрибуты канала VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SetChnAttr(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, const VPSS_CHN_ATTR_S *pstChnAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
pstChnAttr	Атрибуты канала VPSS.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Допустимые значения членов и функциональные ограничения см. в описании структуры VPSS_CHN_ATTR_S.

2.18 RK_MPI_VPSS_GetChnAttr

【Описание】

Получает атрибуты канала VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetChnAttr(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, VPSS_CHN_ATTR_S *pstChnAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
pstChnAttr	Атрибуты канала VPSS.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.

2.19 RK_MPI_VPSS_EnableChn

【Описание】

Включает канал VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_EnableChn(VPSS_GRP VpssGrp, VPSS_CHN VpssChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- При многократном включении только первый вызов возвращает успех, последующие — ошибку.
- GROUP должна быть создана.

2.20 RK_MPI_VPSS_DisableChn

【Описание】

Отключает канал VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_DisableChn(VPSS_GRP VpssGrp, VPSS_CHN VpssChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- При многократном отключении только первый вызов возвращает успех, последующие — ошибку.
- GROUP должна быть создана.

2.21 RK_MPI_VPSS_SetChnCrop

【Описание】

Задаёт атрибуты функции обрезки канала VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SetChnCrop(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, const VPSS_CROP_INFO_S *pstCropInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
pstCropInfo	Параметры функции CROP.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Если область обрезки выходит за границы изображения, её координаты сдвигаются к началу координат; приоритет отдаётся сохранению заданных ширины и высоты.
- Если ширина обрезки больше ширины входного изображения, выходная ширина корректируется до ширины входного изображения.
- Если высота обрезки больше высоты входного изображения, выходная высота корректируется до высоты входного изображения.
- Остальные ограничения совпадают с RK_MPI_VPSS_SetGrpCrop.

2.22 RK_MPI_VPSS_GetChnCrop

【Описание】

Получает атрибуты функции обрезки канала VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetChnCrop(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, VPSS_CROP_INFO_S *pstCropInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
pstCropInfo	Параметры функции CROP.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.

2.23 RK_MPI_VPSS_SetChnRotation

【Описание】

Задаёт атрибуты поворота изображения канала VPSS на фиксированный угол.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SetChnRotation(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, ROTATION_E enRotation);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
enRotation	Атрибут поворота.	Ввод

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

Примечание

- GROUP должна быть создана.
- Атрибуты канала должны быть заданы.
- Поддерживаются только углы 0°, 90°, 180° и 270°; произвольные углы не поддерживаются.
- В режиме канала PAST настройка не действует.
- На RV1126 приоритет угла поворота ниже приоритета горизонтального зеркального отражения и вертикального переворота. Конкретные случаи приведены ниже:

Угол поворота	Горизонтальное отражение	Вертикальный переворот	Результат
0° 90° 270°	Включено	Отключено	Горизонтальное зеркальное отражение
180°	Включено	Отключено	Вертикальный переворот
0° 90° 270°	Отключено	Включено	Вертикальный переворот
180°	Отключено	Включено	Горизонтальное зеркальное отражение
0° 90° 180° 270°	Включено	Включено	180° 270° 0° 90°

- На RK356X/RK3588 угол поворота можно сочетать с горизонтальным зеркальным отражением и вертикальным переворотом. Конкретные случаи приведены ниже:

Угол поворота	Горизонтальное отражение	Вертикальный переворот	Результат
0° 90° 270°	Включено	Отключено	Наложение горизонтального зеркального отражения и угла поворота
180°	Включено	Отключено	Вертикальный переворот
0° 90° 270°	Отключено	Включено	Наложение вертикального переворота и угла поворота
180°	Отключено	Включено	Горизонтальное зеркальное отражение
0° 90° 180° 270°	Включено	Включено	180° 270° 0° 90°

2.24 RK_MPI_VPSS_GetChnRotation

【Описание】

Получает атрибуты поворота изображения канала VPSS на фиксированный угол.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetChnRotation(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, ROTATION_E *penRotation);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
penRotation	Атрибут поворота.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.

2.25 RK_MPI_VPSS_SetChnRotationEx

【Описание】

Задаёт атрибуты поворота VPSS на произвольный угол.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SetChnRotationEx(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, const VPSS_ROTATION_EX_ATTR_S* pstRotationExAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
pstRotationExAttr	Указатель на структуру атрибутов поворота на произвольный угол.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Атрибут можно задать только после настройки атрибутов канала.
- Этот интерфейс нельзя использовать одновременно с RK_MPI_VPSS_SetChnRotation.

2.26 RK_MPI_VPSS_GetChnRotationEx

【Описание】

Получает атрибуты поворота VPSS на произвольный угол.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetChnRotationEx(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, VPSS_ROTATION_EX_ATTR_S* pstRotationExAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
pstRotationExAttr	Указатель на структуру атрибутов поворота на произвольный угол.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.

2.27 RK_MPI_VPSS_GetChnFrame

【Описание】

Пользователь получает из канала один обработанный кадр изображения.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetChnFrame(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, VIDEO_FRAME_INFO_S *pstVideoFrame, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
pstVideoFrame	Сведения об обработанном изображении.	Вывод
s32MilliSec	Если параметр ожидания s32MilliSec равен -1, интерфейс блокирующий; если 0 — неблокирующий; если больше 0 — это время ожидания в миллисекундах (ms).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- GROUP должна быть создана.
- Изображение можно получить только при глубине очереди, отличной от 0.
- Получение изображения через этот интерфейс не влияет на связанные нижестоящие модули. Например, если нижестоящий VO связан для отображения, изображение можно получать во время отображения, а VO продолжит нормально работать.
- Если s32MilliSec равен -1, используется блокирующий режим: программа ожидает до получения изображения. Если s32MilliSec равен 0, используется неблокирующий режим. Если s32MilliSec больше 0, используется режим ожидания с тайм-аутом в миллисекундах; если за это время изображение не получено, возвращается ошибка тайм-аута.
- Если канал работает в режиме PAST, при вызове этого интерфейса для обработки изображения используется VGS.
- По умолчанию в режиме частного MB для канала USER максимальное количество выходных кадров ограничено 3.
- По умолчанию в режиме частного MB для канала PAST максимальное количество выходных кадров не ограничено.
- Рекомендуется задавать глубину очереди не более 3. Если количество выходных изображений в очереди превышает 3, VPSS прекращает работу, и другие каналы той же Group также останавливаются.

2.28 RK_MPI_VPSS_ReleaseChnFrame

【Описание】

Пользователь освобождает один кадр изображения канала.

【Синтаксис】

RK_S32 RK_MPI_VPSS_ReleaseChnFrame(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, const VIDEO_FRAME_INFO_S *pstVideoFrame);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
pstVideoFrame	Сведения об изображении.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Интерфейс необходимо использовать парно с RK_MPI_VPSS_GetChnFrame.
- Количество вызовов должно точно соответствовать количеству вызовов RK_MPI_VPSS_GetChnFrame; повторно вызывать для одного кадра запрещено.
- Кадр, полученный через RK_MPI_VPSS_GetChnFrame, необходимо своевременно освободить этим интерфейсом, иначе после достижения максимального количества выходных кадров новые кадры получить невозможно.

2.29 RK_MPI_VPSS_AttachMbPool

【Описание】

Связывает канал VPSS с определённым пулом видеобуферов MB.

【Синтаксис】

RK_S32 RK_MPI_VPSS_AttachMbPool(VPSS_GRP VpssGrp, VPSS_CHN VpssChn, MB_POOL hMbPool);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод
hMbPool	Сведения о пуле видеобуферов MB.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Группа должна быть создана.
- Действует только для канала в режиме USER; режим PAST не затрагивается.
- Пользователь должен вызвать RK_MPI_MB_CreatePool и создать пул видеобуферов MB, затем через RK_MPI_VPSS_AttachMbPool связать канал текущей группы с пулом MB с фиксированным PoolId. Несколько каналов нескольких групп могут быть связаны с одним пулом MB.
- Чтобы переключить пул MB, связанный с текущей группой, достаточно снова вызвать RK_MPI_VPSS_AttachMbPool и правильно указать новый пул MB.
- hMbPool должен быть допустимым PoolId уже созданного пула MB.
- После вызова RK_MPI_VPSS_DetachMbPool перед уничтожением созданного MB необходимо убедиться, что MB не используется модулями, связанными после VPSS. Можно сначала освободить все MB с помощью sleep или очистки буфера канала нижестоящего модуля, а затем уничтожить пул буферов MB.

- После связывания все MB, запрашиваемые каналом, получаются из этого пула MB.

2.30 RK_MPI_VPSS_DetachMbPool

【Описание】

Отменяет связь канала VPSS с определённым пулом видеобуферов MB.

【Синтаксис】

RK_S32 RK_MPI_VPSS_DetachMbPool(VPSS_GRP VpssGrp, VPSS_CHN VpssChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Группа должна быть создана.

2.31 RK_MPI_VPSS_GetChnFd

【Описание】

Получает дескриптор файла устройства, соответствующего каналу VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetChnFd(VPSS_GRP VpssGrp, VPSS_CHN VpssChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Группа должна быть создана.

2.32 RK_MPI_VPSS_CloseFd

【Описание】

Закрывает файловые дескрипторы устройства и канала.

【Синтаксис】

RK_S32 RK_MPI_VPSS_CloseFd(VPSS_GRP VpssGrp, VPSS_CHN VpssChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
VpssChn	Номер канала VPSS. Диапазон значений: [0, VPSS_MAX_CHN_NUM).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Группа должна быть создана.
- Повторные вызовы разрешены и всегда возвращают успех.

2.33 RK_MPI_VPSS_SetVProcDev

【Описание】

Задаёт тип аппаратного устройства VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SetVProcDev(VPSS_GRP VpssGrp, VIDEO_PROC_DEV_TYPE_E enVProcDev);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
enVProcDev	Тип аппаратного устройства.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Группа должна быть создана.
- Если интерфейс не вызывается, по умолчанию аппаратным устройством модуля VPSS является GPU.

2.34 RK_MPI_VPSS_GetVProcDev

【Описание】

Получает тип аппаратного устройства VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetVProcDev(VPSS_GRP VPSS_GRP VpssGrp, VIDEO_PROC_DEV_TYPE_E *enVProcDev);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VpssGrp	Номер VPSS GROUP. Диапазон значений: [0, VPSS_MAX_GRP_NUM).	Ввод
enVProcDev	Тип аппаратного устройства.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Группа должна быть создана.

2.35 RK_MPI_VPSS_SetModParam

【Описание】

Задаёт параметры модуля VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_SetModParam(const VPSS_MOD_PARAM_S *pstModParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstModParam	Параметры модуля.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Обычно вызывается до создания любых групп VPSS.
- Допускается вызов после создания групп VPSS, но для уже используемых групп параметр не вступает в силу немедленно; он применяется после освобождения и повторного создания группы.

2.36 RK_MPI_VPSS_GetModParam

【Описание】

Получает параметры модуля VPSS.

【Синтаксис】

RK_S32 RK_MPI_VPSS_GetModParam(VPSS_MOD_PARAM_S *pstModParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstModParam	Параметры модуля.	Вывод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VPSS.

【Примечание】

- Нет

3. Типы данных

Типы данных модуля VPSS определены следующим образом:

- VPSS_MAX_GRP_NUM: максимальное количество VPSS GROUP.
- VPSS_MAX_GRP_PIPE_NUM: максимальное количество PIPE в VPSS GROUP.
- VPSS_MAX_CHN_NUM: максимальное количество каналов VPSS.
- VPSS_MIN_IMAGE_WIDTH: минимальная ширина изображения VPSS.
- VPSS_MIN_IMAGE_HEIGHT: минимальная высота изображения VPSS.
- VPSS_MAX_IMAGE_WIDTH: максимальная ширина изображения VPSS.
- VPSS_MAX_IMAGE_HEIGHT: максимальная высота изображения VPSS.
- VPSS_GRP: номер группы VPSS.
- VPSS_GRP_PIPE: номер pipeline группы VPSS.
- VPSS_CHN: номер канала VPSS.
- VPSS_CROP_COORDINATE_E: режим координат начальной точки CROP.
- VPSS_CROP_INFO_S: сведения, требуемые функции CROP.
- VPSS_ROTATION_EX_ATTR_S: атрибуты поворота VPSS на произвольный угол.
- VPSS_MOD_PARAM_S: параметры настройки модуля.
- VPSS_GRP_ATTR_S: атрибуты VPSS GROUP.
- VPSS_CHN_ATTR_S: атрибуты канала VPSS.
- VPSS_CHN_MODE_E: режим работы VPSS CHN.
- ASPECT_RATIO_E: режим работы соотношения сторон VPSS.
- ASPECT_RATIO_S: параметры настройки соотношения сторон выходного изображения VPSS.

3.1 VPSS_MAX_GRP_NUM

【Описание】

Определяет максимальное количество VPSS GROUP.

【Определение】

```
#define VPSS_MAX_GRP_NUM              256
```

Примечания

- Значение может различаться для разных микросхем; конкретное определение параметра см. в rk_defines.h.

3.2 VPSS_MAX_GRP_PIPE_NUM

Описание

Определяет максимальное количество VPSS GROUP.

Определение

```
#define VPSS_MAX_GRP_PIPE_NUM          1
```

Примечания

- Можно задать только 0.

3.3 VPSS_MAX_CHN_NUM

Описание

Определяет максимальное количество каналов VPSS.

Определение

```
#define VPSS_MAX_CHN_NUM               4
```

Примечания

- Значение может различаться для разных микросхем; конкретное определение параметра см. в rk_defines.h.

3.4 VPSS_MIN_IMAGE_WIDTH

Описание

Определяет минимальную ширину изображения VPSS.

Определение

```
#define VPSS_MIN_IMAGE_WIDTH           64
```

Примечания

- Значение может различаться для разных микросхем; конкретное определение параметра см. в rk_defines.h.

3.5 VPSS_MIN_IMAGE_HEIGHT

Описание

Определяет минимальную высоту изображения VPSS.

Определение

```
#define VPSS_MIN_IMAGE_HEIGHT          64
```

Примечания

- Значение может различаться для разных микросхем; конкретное определение параметра см. в rk_defines.h.

3.6 VPSS_MAX_IMAGE_WIDTH

【Описание】

Определяет максимальную ширину изображения VPSS.

【Определение】

```
#define VPSS_MAX_IMAGE_WIDTH           8192
```

【Примечания】

- Значение может различаться для разных микросхем; конкретное определение параметра см. в rk_defines.h.

3.7 VPSS_MAX_IMAGE_HEIGHT

【Описание】

Определяет максимальную высоту изображения VPSS.

【Определение】

```
#define VPSS_MAX_IMAGE_HEIGHT          8192
```

【Примечания】

- Значение может различаться для разных микросхем; конкретное определение параметра см. в rk_defines.h.

3.8 VPSS_GRP

【Описание】

Определяет номер группы VPSS.

【Определение】

```
typedef RK_S32 VPSS_GRP;
```

【Примечания】

Нет

3.9 VPSS_GRP_PIPE

【Описание】

Определяет номер pipeline группы VPSS.

【Определение】

```
typedef RK_S32 VPSS_GRP_PIPE;
```

【Примечания】

- VPSS_GRP_PIPE может быть только 0.

3.10 VPSS_CHN

【Описание】

Определяет номер канала VPSS.

【Определение】

```
typedef RK_S32 VPSS_CHN;
```

【Примечания】

Нет

3.11 VPSS_CROP_COORDINATE_E

【Описание】

Определяет режим координат начальной точки CROP.

【Определение】

```
typedef enum rkVPSS_CROP_COORDINATE_E {
    VPSS_CROP_RATIO_COOR = 0,
    VPSS_CROP_ABS_COOR
} VPSS_CROP_COORDINATE_E;
```

【Члены】

Имя члена	Описание
VPSS_CROP_RATIO_COOR	Относительные координаты.
VPSS_CROP_ABS_COOR	Абсолютные координаты.

【Примечания】

- При относительных координатах значение начальной точки выражается отношением к ширине и высоте текущего изображения и требует преобразования. Подробности см. в VPSS_CROP_INFO_S。

3.12 VPSS_CROP_INFO_S

【Описание】

Определяет сведения, требуемые функции CROP.

【Определение】

```
typedef struct rkVPSS_CROP_INFO_S {
    RK_BOOL                bEnable;
    VPSS_CROP_COORDINATE_E enCropCoordinate;
    RECT_S                 stCropRect;
} VPSS_CROP_INFO_S;
```

【Члены】

Имя члена	Описание
bEnable	Переключатель CROP.
enCropCoordinate	Режим координат начальной точки CROP.
stCropRect	Прямоугольная область CROP.

【Примечания】

- Если enCropCoordinate равен VPSS_CROP_RATIO_COOR (режим относительных координат), члены stCropRect следует преобразовать по формулам:

  s32X = координата x начальной точки × ширина исходного изображения / 1000. Допустимый диапазон: [0, 999]. После вычисления выполняются округление и выравнивание. Та же формула применяется для координаты y.

  u32Width = ширина области × фактическая ширина изображения / 1000. Диапазон ширины области: [1, 1000]. После вычисления выполняются округление и выравнивание. Та же формула применяется для высоты области.

- Координаты, ширина и высота должны быть выровнены по 2 пикселям.

3.13 VPSS_ROTATION_EX_ATTR_S

【Описание】

Определяет атрибуты поворота VPSS на произвольный угол.

【Определение】

```
typedef struct rkVPSS_ROTATION_EX_ATTR_S {
    RK_BOOL       bEnable;
    ROTATION_EX_S stRotationEx;
} VPSS_ROTATION_EX_ATTR_S;
```

【Члены】

Имя члена	Описание
bEnable	Enable/Disable функции поворота на произвольный угол.
stRotationEx	Подробные атрибуты поворота на произвольный угол. Описание см. в разделе «Управление системой».

【Примечания】

- RK356X/RK3588 не поддерживают поворот на произвольный угол.

3.14 VPSS_MOD_PARAM_S

【Описание】

Определяет параметры настройки модуля.

【Определение】

```
typedef struct rkVPSS_MOD_PARAM_S {
    MB_SOURCE_E enVpssMBSource;
} VPSS_MOD_PARAM_S;
```

【Члены】

Имя члена	Описание
enVpssMBSource	Тип пула видеобуферов. 0: общий пул. 1: пул модуля. Зарезервировано, временно не поддерживается. 2: частный пул. 3: пользовательский пул. По умолчанию используется частный пул.

【Примечания】

- Нет

3.15 VPSS_GRP_ATTR_S

【Описание】

Определяет атрибуты VPSS GROUP.

【Определение】

```
typedef struct rkVPSS_GRP_ATTR_S {
    RK_U32                  u32MaxW;
    RK_U32                  u32MaxH;
    PIXEL_FORMAT_E          enPixelFormat;
    DYNAMIC_RANGE_E         enDynamicRange;
    FRAME_RATE_CTRL_S       stFrameRate;
    COMPRESS_MODE_E         enCompressMode;
} VPSS_GRP_ATTR_S;
```

【Члены】

Имя члена	Описание
u32MaxW	Максимальная ширина входного изображения. Статический атрибут, задаётся при создании Group, не изменяется и временно не используется.
u32MaxH	Максимальная высота входного изображения. Статический атрибут, задаётся при создании Group, не изменяется и временно не используется.
enPixelFormat	Формат пикселей входного изображения. Статический атрибут, задаётся при создании Group, не изменяется и временно не используется.
enDynamicRange	Динамический диапазон входного изображения. Статический атрибут, задаётся при создании Group, не изменяется и временно не используется.
stFrameRate	Управление частотой кадров группы. Динамический атрибут, применяется немедленно.
enCompressMode	Способ сжатия изображения, получаемого group. Статический атрибут, задаётся при создании Group и не изменяется.

【Примечания】

- Недействующие параметры задавать не требуется; проверка их недопустимых значений не выполняется.
- Для включения управления частотой исходная частота должна быть больше 0. Целевая частота может быть равна 0; в этом случае выполняется блокирующее управление частотой на основе исходной частоты, а все входные кадры отбрасываются.

3.16 VPSS_CHN_ATTR_S

【Описание】

Определяет атрибуты канала VPSS.

【Определение】

```
typedef struct rkVPSS_CHN_ATTR_S {
    VPSS_CHN_MODE_E     enChnMode;
    RK_U32              u32Width;
    RK_U32              u32Height;
    VIDEO_FORMAT_E      enVideoFormat;
    PIXEL_FORMAT_E      enPixelFormat;
    DYNAMIC_RANGE_E     enDynamicRange;
    COMPRESS_MODE_E     enCompressMode;
    FRAME_RATE_CTRL_S   stFrameRate;
    RK_BOOL             bMirror;
    RK_BOOL             bFlip;
    RK_U32              u32Depth;
    ASPECT_RATIO_S      stAspectRatio;
    RK_U32              u32FrameBufCnt;
} VPSS_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
enChnMode	Режим работы канала. Динамический атрибут.
u32Width	Ширина целевого изображения. Динамический атрибут.
u32Height	Высота целевого изображения. Динамический атрибут.
enVideoFormat	Видеоформат целевого изображения. Динамический атрибут.
enPixelFormat	Формат пикселей целевого изображения. Динамический атрибут.
enDynamicRange	Динамический диапазон целевого изображения. Динамический атрибут, временно не используется.
enCompressMode	Режим сжатия целевого изображения. Динамический атрибут.
stFrameRate	Сведения управления частотой кадров. Динамический атрибут, применяется немедленно.
bMirror	Включение горизонтального зеркального отражения. Динамический атрибут.
bFlip	Включение вертикального переворота. Динамический атрибут.
u32Depth	Длина очереди получения изображений канала пользователем. Диапазон значений: [0, 8]. Динамический атрибут.
stAspectRatio	Параметры соотношения сторон. Динамический атрибут.
u32FrameBufCnt	Максимальное количество буферов целевого изображения. Действует только при private-режиме буфера канала. Статический атрибут.

Примечания

- Если исходная и целевая частоты равны -1, управление частотой не выполняется.
- Для включения управления частотой исходная частота должна быть больше 0. Целевая может быть равна 0; в этом случае все ожидающие обработки изображения канала отбрасываются.
- Если глубина u32Depth равна 0, изображения канала не сохраняются и все отбрасываются.
- bMirror и bFlip могут действовать одновременно и вместе с параметром угла поворота канала (фиксированным или произвольным) определяют итоговое выходное изображение канала. Подробности см. в описании RK_MPI_VPSS_SetChnRotation.
- Если для динамического атрибута не указано иное, он вступает в силу через один кадр.
- Если u32FrameBufCnt меньше 3, внутренне принудительно используется 3, чтобы обеспечить минимально необходимый выходной буфер VPSS.

3.17 VPSS_CHN_MODE_E

Описание

Определяет режим работы VPSS CHN.

Определение

```
typedef enum rkVPSS_CHN_MODE_E {
    VPSS_CHN_MODE_USER        = 0,
    VPSS_CHN_MODE_AUTO        = 1,
    VPSS_CHN_MODE_PASSTHROUGH = 2
} VPSS_CHN_MODE_E;
```

Члены

Имя члена	Описание
VPSS_CHN_MODE_USER	Пользовательский режим настройки.
VPSS_CHN_MODE_AUTO	Автоматический режим. (Устаревший режим, не рекомендуется.)
VPSS_CHN_MODE_PASSTHROUGH	Сквозной режим. В этом режиме аппаратное обеспечение VPSS само не обрабатывает данные, а передаёт разобранные данные нижестоящему модулю для обработки.

Примечания

- Для режима работы канала рекомендуется USER или PAST. Режим AUTO устарел. Процессы режимов см. на схемах потока данных VPSS RK356X/RK3588 в режиме USER и в режиме PAST.
- Чтобы режим PAST действовал, канал должен быть связан с VO.

3.18 ASPECT_RATIO_E

Описание

Определяет режим работы соотношения сторон.

Определение

```
typedef enum rkASPECT_RATIO_E {
    ASPECT_RATIO_NONE   = 0,
    ASPECT_RATIO_AUTO   = 1,
    ASPECT_RATIO_MANUAL = 2,
    ASPECT_RATIO_BUTT
} ASPECT_RATIO_E;
```

Члены

Имя члена	Описание
ASPECT_RATIO_NONE	Не включать функцию соотношения сторон.
ASPECT_RATIO_AUTO	Включить автоматическую адаптацию соотношения сторон; выходное изображение сохраняет соотношение ширины и высоты входного изображения.
ASPECT_RATIO_MANUAL	Вручную настроить положение выходного изображения в области отображения.

Примечания

Подробности см. в ASPECT_RATIO_S.

Связанные типы данных и интерфейсы

ASPECT_RATIO_S

3.19 ASPECT_RATIO_S

Описание

Определяет параметры настройки соотношения сторон выходного изображения VPSS.

Определение

```
typedef struct rkASPECT_RATIO_S {
    ASPECT_RATIO_E enMode;
    RK_U32         u32BgColor;
    RECT_S         stVideoRect;
} ASPECT_RATIO_S;
```

Члены

Имя члена	Описание
enMode	Тип настройки соотношения сторон.
u32BgColor	Цвет заполнения невидеозон при включённом соотношении сторон (параметр временно не поддерживается).
stVideoRect	В режиме ASPECT_RATIO_MANUAL — положение видеозоны в области отображения. В режиме MANUAL координаты должны удовлетворять условиям: x: не меньше 0 и меньше ширины изображения канала; y: не меньше 0 и меньше высоты изображения канала; w: не меньше 64 и не больше ширины изображения канала; h: не меньше 64 и не больше высоты изображения канала; x+w не должно превышать ширину, y+h — высоту изображения канала.

Примечания

- Если выходное изображение масштабируется, оно приводится к размеру видеозоны соотношения сторон.
- Параметры соотношения сторон задаются в атрибутах канала. Они действуют только в режиме канала VPSS USER; в режиме PAST параметры задаёт VO.
- В режиме ASPECT_RATIO_MANUAL пользователь вручную задаёт начальные координаты, ширину и высоту изображения. VPSS выводит изображение в соответствии с настройками и добавляет чёрные поля. Область отображения изображения не должна превышать ширину и высоту выходного изображения.
- В режиме ASPECT_RATIO_AUTO VPSS сохраняет соотношение ширины и высоты входного изображения и автоматически добавляет чёрные поля сверху и снизу либо слева и справа.

4. Коды ошибок VPSS

Коды ошибок API подсистемы обработки видео VPSS приведены ниже:

Код ошибки	Макроопределение	Описание
0xA0068001	RK_ERR_VPSS_INVALID_DEVID	Недопустимый номер VPSS GROUP
0xA0068002	RK_ERR_VPSS_INVALID_CHNID	Недопустимый номер канала VPSS
0xA0068003	RK_ERR_VPSS_ILLEGAL_PARAM	Недопустимые параметры VPSS
0xA0068004	RK_ERR_VPSS_EXIST	VPSS GROUP уже создана
0xA0068005	RK_ERR_VPSS_UNEXIST	VPSS GROUP не создана
0xA0068006	RK_ERR_VPSS_NULL_PTR	Нулевой указатель входного параметра
0xA0068008	RK_ERR_VPSS_NOT_SUPPORT	Операция не поддерживается
0xA0068009	RK_ERR_VPSS_NOT_PERM	Операция не разрешена
0xA006800C	RK_ERR_VPSS_NOMEM	Не удалось выделить память
0xA006800D	RK_ERR_VPSS_NOBUF	Не удалось выделить пул BUF
0xA006800E	RK_ERR_VPSS_BUF_EMPTY	Очередь изображений пуста
0xA006800F	RK_ERR_VPSS_BUF_FULL	Очередь изображений заполнена
0xA0068010	RK_ERR_VPSS_NOTREADY	Система VPSS не инициализирована
0xA0068012	RK_ERR_VPSS_BUSY	Система VPSS занята
0xA0068013	RK_ERR_VPSS_SIZE_NOT_ENOUGH	Недостаточный размер блока MB

Декодирование видео

Предисловие

Обзор

Модуль VDEC предоставляет интерфейсы MPI для управления аппаратным декодером видео и реализует функции декодирования.

Версии продуктов

Название микросхемы	Версия ядра
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	YWJ	2021-01-09	Начальная версия
V0.2.0	YWJ	2021-01-23	Уточнены определения структур данных
V0.3.0	YWJ	2021-09-09	Добавлены характеристики микросхем и описания интерфейсов
V1.0.0	FXW	2021-10-02	Дополнены и изменены отдельные описания, упорядочен формат документа
V1.0.1	XLM	2022-06-20	Добавлено описание стратегии, при которой изображение не отображается при PTS=-1

Содержание

Обзор
Важные понятия
Параметры модуля
Справочник API
Типы данных
Коды ошибок VDEC

1. Обзор

Модуль VDEC предоставляет интерфейс аппаратного декодирования видео. Поддерживаемые протоколы, разрешения и производительность различаются между платформами.

Характеристики VDEC для каждой микросхемы приведены в таблице 1-1:

Микросхема	Аппаратный декодер	Макс. каналов	Поддерживаемые протоколы	Поддерживаемое разрешение	Максимальная производительность
RK3568	RKVDEC VDPU JPEGD	64	**RKVDEC：** H.264/H.265 **VDPU：** MPEG2/MPEG4 **JPEGD：** MJPEG/JPEG	**RKVDEC：** - H.264: 16x16 to 4096x2304 - H.265: 64x64 to 4096x2304 **VDPU：** - MPEG2: 48x48 to 1920x1088 - MPEG4: 48x48 to 1920x1088 **JPEGD：** - MJPEG：48x48 to 65536x65536 - JPEG：48x48 to 65536x65536	**H264：** 4096x2304@30fps **H265：** 4096x2304@60fps **MPEG2/MPEG4：** 1920x1088@60fps **MPEG4：** 1920x1088@60fps **MJPEG/JPEG：** 1920x1080@120fps
RK3588	RKVDEC VDPU JPEGD	256	**RKVDEC：** H.264/H.265 **VDPU：** MPEG2/MPEG4 **JPEGD：** MJPEG/JPEG	**RKVDEC：** - H.264: 16x16 to 65520x65520 - H.265: 64x64 to 65472x65472 **VDPU：** - MPEG2: 48x48 to 1920x1088 - MPEG4: 48x48 to 1920x1088 **JPEGD：** - MJPEG：48x48 to 65536x65536 - JPEG：48x48 to 65536x65536	**H264：** 7680x4320@30fps **H265：** 7680x4320@60fps **MPEG2/MPEG4：** 1920x1088@60fps **MPEG4：** 1920x1088@60fps **MJPEG/JPEG：** 1920x1080@280fps

2. Важные понятия

- Режим отправки потока

  Декодер поддерживает два режима отправки потока:

  - Потоковая отправка (VIDEO_MODE_STREAM) Пользователь каждый раз может отправлять декодеру поток произвольной длины; декодер самостоятельно разбивает его на кадры. Для распознавания конца текущего кадра декодеру необходимо получить поток следующего кадра, поэтому в этом режиме декодированное изображение текущего кадра невозможно вывести немедленно.

  - Покадровая отправка (VIDEO_MODE_FRAME) Пользователь каждый раз отправляет декодеру полный поток одного кадра. Декодер считает, что поток содержит полное изображение, не выполняет внутреннее разбиение на кадры и начинает декодирование. Поэтому каждый вызов интерфейса отправки обязан передавать ровно один кадр, иначе возникает ошибка декодирования. Этот режим позволяет добиться быстрого декодирования.

  > Настройка режима отправки потока:
  >
  > Режим задаётся через интерфейс RK_MPI_VDEC_CreateChn в атрибуте канала VDEC_CHN_ATTR_S > enMode.

- Способы создания пакета потока Интерфейс отправки потока всегда принимает данные в блоке памяти типа MB_BLK. Блок можно создать следующими способами:

  - На основе существующей виртуальной памяти пользователя

    Создайте блок памяти через RK_MPI_SYS_CreateMB, настроив MB_EXT_CONFIG_S. Если внешний поток после чтения или разбора уже

    находится в виртуальной памяти, блок MB можно создать непосредственно на основе её адреса. Внутри выполняется только обёртка типа структуры данных без копирования. Если эта виртуальная память циклически используется снаружи, функцию освобождения pFreeCB задавать не нужно, а поток следует отправлять в режиме копирования. Если виртуальная память выделена динамически, можно задать функцию освобождения, передать управление декодеру и после завершения использования вызвать обратный вызов для освобождения памяти.

- На основе существующего DMA-BUF пользователя Если DMA Buffer уже создан другим способом и поток хранится в нём, блок MB можно создать тем же способом, что и на основе виртуальной памяти.

  Каждый DMA Buffer имеет соответствующий дескриптор FD; при настройке MB_EXT_CONFIG_S необходимо указать этот

  FD.

- Получение из пула памяти

  Сначала создайте пул памяти через RK_MPI_MB_CreatePool, затем получите блок через RK_MPI_MB_GetMB и после использования

  верните его в пул вызовом RK_MPI_MB_ReleaseMB. При таком способе рекомендуется отправлять поток в сквозном режиме.

- Непосредственное выделение Непосредственно вызовите интерфейс модуля SYS для выделения блока памяти MB, затем скопируйте в него данные потока. Интерфейс

  RK_MPI_SYS_Malloc выделяет виртуальную память, RK_MPI_SYS_MmzAlloc — физическую. При таком способе рекомендуется

  отправлять поток в сквозном режиме.

- Режим приёма потока декодером Когда пользователь отправляет поток декодеру через интерфейс отправки, декодер принимает его одним из двух способов:

  - Режим копирования

    Данные, переданные интерфейсом отправки, копируются внутри декодера; внутренний и внешний Buffer потока больше не связаны. Пользователь может по необходимости

    самостоятельно управлять блоком памяти VDEC_STREAM_S > pMbBlk.

  - Сквозной режим

    Данные, переданные интерфейсом отправки, внутри декодера не копируются; для VDEC_STREAM_S > pMbBlk только увеличивается

    счётчик ссылок, который освобождается после обработки. Если pMbBlk является внешним блоком памяти, созданным через RK_MPI_SYS_CreateMB, при освобождении

    pMbBlk также вызывается заданный пользователем при создании pMbBlk обратный вызов pFreeCB, освобождающий память. Если pMbBlk создан через

    RK_MPI_MB_GetMB, он возвращается в пул MB для повторного использования.

  > Настройка режима приёма потока:
  >
  > При отправке через RK_MPI_VDEC_SendStream настройте параметр потока VDEC_STREAM_S > bBypassMbBlk:
  >
  > RK_FALSE включает режим копирования, RK_TRUE — сквозной режим. В обоих случаях после вызова
  >
  > RK_MPI_VDEC_SendStream необходимо вызвать RK_MPI_MB_ReleaseMB, иначе возникнет утечка памяти.

- Способ выделения памяти декодированных кадров Память декодированных кадров хранит декодированные данные изображения. Модуль VDEC поддерживает следующие способы выделения пула:

  - Частный пул (MB_SOURCE_PRIVATE) Частный пул создаётся внутри модуля VDEC для конкретного канала и используется только им. Из пула запрашиваются MB в качестве Buffer изображения

    канала. При создании канала через RK_MPI_VDEC_CreateChn пользователь может задать количество блоков частного пула u32FrameBufCnt и размер отдельного

    блока u32FrameBufSize. Вычисление размера MB сложно, поэтому рекомендуется задать u32FrameBufSize=0, чтобы внутренне вычислить необходимый размер Buffer изображения по настроенной ширине и высоте.

  - Пользовательский пул (MB_SOURCE_USER) При пользовательском пуле во время создания канала декодирования пул и Buffer декодированных изображений не создаются. Пользователь вызывает

    RK_MPI_MB_CreatePool, создаёт пул MB, а затем через RK_MPI_VDEC_AttachMbPool связывает его

    с определённым каналом декодирования.

  - Пул модуля (MB_SOURCE_MODULE)

    При создании канала из общего пула выбирается пул подходящего размера и количества. Если такого нет, создаётся новый пул, который также используется только этим каналом. После уничтожения канала пул возвращается в общий пул; если размер резидентной памяти не настроен, пул уничтожается. Пользователь может настроить размер резидентной памяти, чтобы уменьшить частые выделения и освобождения, а также общий размер памяти для ограничения потребления. Если параметры не заданы, память выделяется внутренне по мере необходимости каналам декодирования.

    - Общий объём используемой памяти можно ограничить переменной окружения: export rt_mb_max_alloc=209715200 (ограничение 200 МБ; при превышении 200 МБ выделить буфер декодирования будет невозможно). Учтите, что общий объём памяти включает память, не относящуюся к декодированию; при расчёте порогового значения необходимо учитывать её размер.

    - Общий объём резидентной памяти можно ограничить переменной окружения: export rt_mb_max_retain=104857600 (ограничение 100 МБ; резидентно сохраняется не более 100 МБ для следующего декодирования).

  > Настройка способа выделения пула памяти декодирования:
  >
  > Способ выделения пула памяти настраивается интерфейсом RK_MPI_VDEC_SetModParam через параметр модуля VDEC_MOD_PARAM_S »
  >
  > enVdecMBSource. Если параметр не задан, по умолчанию используется режим MB_SOURCE_MODULE.

- Порядок вывода декодированных изображений

  Поддерживаются два порядка вывода декодированных изображений:

  - Порядок декодирования (VIDEO_OUTPUT_ORDER_DEC) Декодированные изображения выводятся в порядке поступления входных пакетов. Изображение может быть выведено сразу после декодирования, а порядок отображения совпадает с порядком декодирования. Этот режим рекомендуется для потока без B-кадров: он позволяет быстрее получать кадры и уменьшает требуемое число блоков памяти.

- Порядок отображения (VIDEO_OUTPUT_ORDER_DISP) После декодирования изображения сортируются и выводятся согласно синтаксису соответствующего протокола. Декодированное изображение может быть выведено не сразу, поэтому порядок отображения может отличаться от порядка декодирования. Например, если поток содержит B-кадр, использующий предыдущий и следующий P-кадры (исходный порядок IPBP), перед декодированием B-кадра необходимо сначала декодировать следующий за ним P-кадр, однако P-кадр не выводится немедленно и выводится только после декодирования B-кадра. В результате порядок отображения (IPBP) отличается от порядка декодирования (IPPB).

> Настройка порядка вывода декодированных изображений:
>
> Порядок вывода настраивается интерфейсом RK_MPI_VDEC_SetChnParam через параметр канала VDEC_CHN_PARAM_S » VDEC_PARAM_VIDEO_S » enOutputOrder. Для потоков JPEG/MJPEG эта настройка не поддерживается.

- Обработка временной метки отображения При отправке потока в кадровом режиме (VIDEO_MODE_FRAME) временная метка отображения PTS декодированного изображения равна PTS, переданной пользователем через интерфейс отправки потока (RK_MPI_VDEC_SendStream). Декодер не изменяет это значение, однако порядок вывода может измениться, например при наличии B-кадров, поэтому PTS выходного изображения может не совпадать с PTS входного потока. Если пользователь передал PTS, равную -1, изображение не будет отображаться модулем видеовывода (VO). Если PTS равна 0, пользователь не выполняет управление частотой кадров, и оно выполняется модулем видеовывода VO. При отправке потока в потоковом режиме (VIDEO_MODE_STREAM) PTS выходных изображений следует всегда устанавливать в 0: пользователь не управляет частотой кадров, это выполняет модуль видеовывода VO.

- Поддержка вставки пользовательского изображения Пользователь может непосредственно передать данные изображения в модуль VDEC. Модуль VDEC не обрабатывает это изображение и в режиме привязки напрямую передаёт его следующему модулю. При возникновении ошибки на стороне источника потока пользователь может вставить таким способом информационное изображение. Например, при разрыве сетевого соединения и прекращении поступления потока пользователь может настроить вставку изображения и вывести его через модуль VO, чтобы сообщить о сетевой ошибке или отсутствии потока для декодирования. Модуль VDEC поддерживает следующие способы вставки пользовательского изображения:

  - Немедленная вставка изображения

    Модуль VDEC сначала очищает поток и изображения внутри декодера, а затем вставляет пользовательское изображение.
  - Отложенная вставка изображения

    Модуль VDEC сначала декодирует и выводит весь оставшийся внутри декодера поток, а затем вставляет пользовательское изображение.

> Порядок вставки пользовательского изображения:
>
> Сначала задайте вставляемое изображение интерфейсом RK_MPI_VDEC_SetUserPic, затем включите вставку интерфейсом RK_MPI_VDEC_EnableUserPic. Если параметр bInstant этого интерфейса равен RK_TRUE, изображение немедленно вставляется и передаётся следующему модулю; если он равен RK_FALSE, изображение вставляется с задержкой после завершения декодирования оставшихся данных канала.

3. Параметры модуля

- **VDEC_MAX_CHN_NUM**

  Максимальное число каналов, поддерживаемых модулем VDEC. В настоящее время число каналов фиксировано. В дальнейшем будет добавлен файл конфигурации, позволяющий задавать подходящее максимальное число каналов декодирования в соответствии с требованиями сценария и уменьшать объём памяти, занимаемой контекстами каналов.

4. Справочник API

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_VDEC_SetModParam: задаёт параметры модуля декодирования.
- RK_MPI_VDEC_GetModParam: получает параметры модуля декодирования.
- RK_MPI_VDEC_CreateChn: создаёт канал видеодекодирования.
- RK_MPI_VDEC_DestroyChn: уничтожает канал видеодекодирования.
- RK_MPI_VDEC_ResetChn: сбрасывает канал декодирования.
- RK_MPI_VDEC_GetChnAttr: получает атрибуты канала видеодекодирования.
- RK_MPI_VDEC_SetChnAttr: задаёт атрибуты канала видеодекодирования.
- RK_MPI_VDEC_StartRecvStream: запускает приём пользовательского потока декодером.
- RK_MPI_VDEC_StopRecvStream: останавливает приём пользовательского потока декодером.
- RK_MPI_VDEC_QueryStatus: запрашивает состояние канала декодирования.
- RK_MPI_VDEC_SendStream: отправляет данные потока в канал видеодекодирования.
- RK_MPI_VDEC_GetFrame: получает декодированное изображение из канала видеодекодирования.
- RK_MPI_VDEC_ReleaseFrame: освобождает декодированное изображение канала видеодекодирования.
- RK_MPI_VDEC_SetChnParam: задаёт параметры канала декодирования.

- RK_MPI_VDEC_GetChnParam: получает параметры канала декодирования.
- RK_MPI_VDEC_SetDisplayMode: задаёт режим отображения.
- RK_MPI_VDEC_GetDisplayMode: получает режим отображения.
- RK_MPI_VDEC_GetFd: получает дескриптор файла устройства канала видеодекодирования.
- RK_MPI_VDEC_CloseFd: закрывает дескриптор файла устройства видеодекодирования.
- RK_MPI_VDEC_AttachMbPool: привязывает канал декодирования к указанному пулу видеобуферов MB.
- RK_MPI_VDEC_DetachMbPool: отвязывает канал декодирования от указанного пула видеобуферов MB.
- RK_MPI_VDEC_SetUserPic: задаёт атрибуты пользовательского изображения.
- RK_MPI_VDEC_EnableUserPic: включает вставку пользовательского изображения.
- RK_MPI_VDEC_DisableUserPic: отключает вставку пользовательского изображения.

4.1 RK_MPI_VDEC_SetModParam

【Описание】

Задаёт параметры канала декодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_SetModParam(const VDEC_MOD_PARAM_S *pstModParam)

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstModParam	Указатель на структуру параметров модуля.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Этот интерфейс необходимо вызывать до создания любых каналов декодирования, иначе настройка не действует.
- Передаваемый pstModParam не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_GetModParam。

4.2 RK_MPI_VDEC_GetModParam

【Описание】

Получение параметров канала декодирования

【Синтаксис】

RK_S32 RK_MPI_VDEC_GetModParam(VDEC_MOD_PARAM_S *pstModParam)

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstModParam	Указатель на структуру параметров модуля.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】 Передаваемый pstModParam не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_SetModParam。

4.3 RK_MPI_VDEC_CreateChn

【Описание】

Создаёт канал видеодекодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_CreateChn(VDEC_CHN VdChn, const VDEC_CHN_ATTR_S *pstAttr)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstAttr	Указатель на атрибуты канала декодирования.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Номер канала должен быть допустимым и не превышать VDEC_MAX_CHN_NUM.
- Параметр pstAttr не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.
- Перед созданием канала видеодекодирования необходимо убедиться, что канал ещё не создан (либо уже уничтожен), иначе возвращается ошибка существующего канала.

  RK_ERR_VDEC_EXIST。
- Ширина и высота в атрибутах канала pstAttr должны быть больше 0, иначе возвращается ошибка недопустимого параметра RK_ERR_VDEC_ILLEGAL_PARAM.
- Ширина и высота в атрибутах канала pstAttr не должны превышать аппаратные возможности декодирования, иначе инициализация канала завершится ошибкой.
- Для выделения пула памяти канала декодирования поддерживаются только частный и пользовательский пулы. Порядок использования см. в разделе «Способ выделения памяти кадров декодирования» главы «Важные понятия».
- Если для памяти кадров декодирования используется частный пул, пользователь должен в соответствии с потоком задать необходимое число блоков памяти u32FrameBufCnt и размер блока u32FrameBufSize. Если указано 0, модуль VDEC использует внутренние параметры по умолчанию. Значение u32FrameBufCnt рекомендуется выбирать с учётом потока и доступной системной памяти, а u32FrameBufSize рекомендуется задавать равным 0, чтобы модуль VDEC вычислил размер Buffer по настроенным ширине и высоте и создал соответствующий частный пул MB по заданным числу и размеру блоков.
- Если для памяти кадров декодирования используется пользовательский пул, параметры атрибутов канала u32FrameBufSize и u32FrameBufCnt не действуют. Пользователь выделяет память через интерфейсы rk_mpi_mb.h, а затем привязывает её к каналу декодирования. Созданный пользователем пул должен содержать блоки MB достаточного размера и количества для текущего канала. Для разных протоколов требуются разные число и размер блоков памяти изображений MB. Для H264, H265, MPEG2 и MPEG4 каждому каналу требуется не менее «число опорных кадров + 2» блоков MB; для JPEG/MJPEG каждому каналу декодирования требуется не менее 2 блоков VB. Расчёт размера блока MB достаточно сложен; подробный способ расчёта см. в функции RK_MPI_CAL_VDEC_GetPicBufferSize файла rk_mpi_cal.h.
- Если декодируемый поток H.264 содержит B-кадры либо поток H.265 поддерживает временное предсказание векторов движения (sps_temporal_mvp_enabled_flag = 1),

  при создании канала необходимо включить поддержку временного предсказания векторов движения (bTemporalMvpEnable = 1). Модуль VDEC выделит объём памяти для хранения векторов движения MV в соответствии с числом опорных кадров. В противном случае временное предсказание можно отключить (bTemporalMvpEnable = 0), чтобы уменьшить выделение памяти. Если единая настройка не удовлетворяет ограничениям всех протоколов, определяйте необходимость этой функции отдельно для каждого протокола.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_DestroyChn。

4.4 RK_MPI_VDEC_DestroyChn

【Описание】

Уничтожает канал видеодекодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_DestroyChn(VDEC_CHN VdChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_CreateChn。

4.5 RK_MPI_VDEC_ResetChn

【Описание】

Сбрасывает канал видеодекодирования и очищает данные, кэшированные каналом.

【Синтаксис】

RK_S32 RK_MPI_VDEC_ResetChn(VDEC_CHN VdChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.

【Пример】

Нет.

【Связанные темы】

Нет.

4.6 RK_MPI_VDEC_GetChnAttr

【Описание】

Получает атрибуты канала видеодекодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_GetChnAttr(VDEC_CHN VdChn, VDEC_CHN_ATTR_S *pstAttr)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstAttr	Указатель на атрибуты канала декодирования.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Передаваемый pstAttr не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.

【Пример】

Нет.

【Связанные темы】

Нет.

4.7 RK_MPI_VDEC_SetChnAttr

【Описание】

Задаёт атрибуты канала видеодекодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_SetChnAttr(VDEC_CHN VdChn, const VDEC_CHN_ATTR_S *pstAttr)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstAttr	Указатель на атрибуты канала декодирования.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Атрибуты необходимо задавать до вызова RK_MPI_VDEC_StartRecvStream; динамическое переключение не поддерживается.
- Перед изменением атрибутов канала необходимо остановить приём потока. Новые атрибуты вступают в силу после повторного запуска приёма.
- После изменения атрибутов декодер автоматически сбрасывает канал при следующем запуске приёма потока.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_StopRecvStream。

4.8 RK_MPI_VDEC_StartRecvStream

【Описание】

Запускает приём пользовательского потока декодером.

【Синтаксис】

RK_S32 RK_MPI_VDEC_StartRecvStream(VDEC_CHN VdChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Только после запуска приёма потока вызов RK_MPI_VDEC_SendStream сможет успешно отправлять поток.
- Повторный вызов интерфейса запуска приёма потока возвращает успех.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_StopRecvStream。

4.9 RK_MPI_VDEC_StopRecvStream

【Описание】

Останавливает приём пользовательского потока декодером.

【Синтаксис】

RK_S32 RK_MPI_VDEC_StopRecvStream(VDEC_CHN VdChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- После вызова этого интерфейса RK_MPI_VDEC_SendStream возвращает ошибку.
- Повторный вызов интерфейса остановки приёма потока возвращает успех.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_StartRecvStream。

4.10 RK_MPI_VDEC_QueryStatus

【Описание】

Запрашивает состояние канала декодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_QueryStatus(VDEC_CHN VdChn, VDEC_CHN_STATUS_S *pstStatus)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstStatus	Указатель на структуру состояния канала видеодекодирования.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Передаваемый pstStatus не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.

【Пример】

Нет.

【Связанные темы】

Нет.

4.11 RK_MPI_VDEC_SendStream

【Описание】

Останавливает приём пользовательского потока декодером.

【Синтаксис】

RK_S32 RK_MPI_VDEC_SendStream(VDEC_CHN VdChn, const VDEC_STREAM_S *pstStream, RK_S32 s32MilliSec)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstStream	Указатель на данные декодируемого потока.	Вход
s32MilliSec	Флаг режима отправки потока. Диапазон значений: -1: блокирующий режим. 0: неблокирующий режим. Положительное значение: время ожидания в мс; верхнего ограничения нет, допускается динамическое изменение.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Перед отправкой потока необходимо вызвать RK_MPI_VDEC_StartRecvStream, иначе возвращается ошибка недопустимой операции. RK_ERR_VDEC_NOT_PERM。
- Поток необходимо отправлять в режиме, заданном при создании канала декодирования. В кадровом режиме за один вызов требуется передать полный кадр потока, иначе возникнет ошибка декодирования. В потоковом режиме этого ограничения нет.
- После окончания потока можно установить bEndOfStream = 1 в последнем пакете либо отправить пустой пакет с bEndOfStream = 1 (pMbBlk равен MB_INVALID_HANDLE или u32Len равен 0). Декодер обработает весь поток и выведет все изображения. Во всех остальных случаях bEndOfStream следует устанавливать в 0.
- Отправлять пустой пакет с bEndOfStream = 0 запрещено; возвращается ошибка недопустимой операции RK_ERR_VDEC_NOT_PERM.
- В кадровом режиме PTS декодированного изображения равна PTS в переданной структуре pstStream. В потоковом режиме PTS декодированного изображения равна 0.
- Значение s32MilliSec позволяет отправлять поток в блокирующем, неблокирующем режиме или с тайм-аутом.
- При блокирующей отправке вызов блокируется до успешной передачи потока.
- При неблокирующей отправке, если буфер потока заполнен, возвращается RK_ERR_VDEC_BUF_FULL. Число хранимых в буфере пакетов можно изменить параметром атрибутов канала u32StreamBufCnt.
- При отправке с тайм-аутом, если поток не удалось передать до истечения заданного времени, возвращается RK_ERR_VDEC_BUF_FULL. Получив RK_ERR_VDEC_BUF_FULL, пользователь должен повторно отправить тот же пакет, иначе возникнет ошибка декодирования. Чтобы избежать RK_ERR_VDEC_BUF_FULL, рекомендуется устанавливать тайм-аут равным четырём интервалам кадров канала. Например, при частоте 25 fps следует установить не менее 160 мс.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_StartRecvStream。

4.12 RK_MPI_VDEC_GetFrame

【Описание】

Получает декодированное изображение из канала видеодекодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_GetFrame(VDEC_CHN VdChn, VIDEO_FRAME_INFO_S *pstFrameInfo, RK_S32 s32MilliSec)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstFrameInfo	Информация о полученном декодированном изображении.	Выход
s32MilliSec	Флаг режима получения изображения. Диапазон значений: -1: блокирующий режим. 0: неблокирующий режим. Положительное значение: время ожидания в мс; верхнего ограничения нет, допускается динамическое изменение.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Передаваемый pstFrameInfo не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.
- После получения изображения через RK_MPI_VDEC_GetFrame его необходимо освободить вызовом RK_MPI_VDEC_ReleaseFrame, иначе процесс декодирования заблокируется в ожидании ресурса.
- Значение s32MilliSec позволяет получать изображение в блокирующем, неблокирующем режиме или с тайм-аутом.
- В блокирующем режиме вызов блокируется до успешного получения декодированного изображения.
- В неблокирующем режиме, если в буфере нет изображения, возвращается RK_ERR_VDEC_BUF_EMPTY.
- В режиме с тайм-аутом, если изображение не получено до истечения заданного времени, возвращается RK_ERR_VDEC_BUF_EMPTY. Рекомендуется задавать тайм-аут кратным 10 мс и больше интервала выходных кадров декодирования. Например, при частоте 25 fps следует установить 50 мс.
- Можно получить fd канала через RK_MPI_VDEC_GetFd, проверить наличие изображения в буфере через select и затем вызвать этот интерфейс для получения декодированного изображения.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_ReleaseFrame。

RK_MPI_VDEC_GetFd。

4.13 RK_MPI_VDEC_ReleaseFrame

【Описание】

Освобождает изображение канала видеодекодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_ReleaseFrame(VDEC_CHN VdChn, const VIDEO_FRAME_INFO_S *pstFrameInfo)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstFrameInfo	Указатель на информацию о декодированном изображении, полученную через RK_MPI_VDEC_GetFrame.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

Примечание

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Необходимо использовать попарно с RK_MPI_VDEC_GetFrame. Полученные данные следует освобождать сразу после использования, иначе процесс декодирования заблокируется в ожидании ресурса.
- Допускается освобождать полученные изображения канала после уничтожения канала.
- Допускается освобождать изображения не по порядку их получения.
- Освобождаемые данные должны быть получены из этого канала через RK_MPI_VDEC_GetFrame; изменять структуру с информацией о данных запрещено.

Пример

Нет.

Связанные темы

RK_MPI_VDEC_GetFrame。

4.14 RK_MPI_VDEC_SetUserPic

Описание

Задаёт атрибуты пользовательского изображения.

Синтаксис

RK_S32 RK_MPI_VDEC_SetUserPic(VDEC_CHN VdChn, const VIDEO_FRAME_INFO_S *pstUsrPic)

Параметры

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstUsrPic	Указатель на структуру атрибутов пользовательского изображения.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

Примечание

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Передаваемый pstUsrPic не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.
- Пользователь может непосредственно выделить блок памяти MMZ через системный интерфейс RK_MPI_SYS_MmzAlloc либо создать частный пул MB через RK_MPI_MB_CreatePool и получить из него блок MB для хранения пользовательского изображения.
- Формат пикселей enPixelFormat пользовательского изображения должен поддерживаться следующим модулем, например NV12 / RGB888. PTS пользовательского изображения устанавливается в 0.

- Пока пользовательское изображение включено, задавать новое изображение нельзя. Сначала необходимо вызвать RK_MPI_VDEC_DisableUserPic, иначе возвращается ошибка недопустимой операции RK_ERR_VDEC_NOT_PERM.
- После успешной настройки атрибуты пользовательского изображения не действуют до вызова RK_MPI_VDEC_EnableUserPic.
- Канал декодирования не обрабатывает пользовательское изображение, поэтому его ширина и высота не ограничены размерами канала декодирования, но ограничиваются возможностями следующего модуля.
- После успешной настройки атрибутов VDEC удерживает блок MB пользовательского изображения до уничтожения канала декодирования.
- Атрибуты пользовательского изображения можно задавать повторно. При повторной настройке VDEC сначала освобождает предыдущий блок MB, затем удерживает новый блок.
- Одни и те же атрибуты пользовательского изображения можно использовать для разных каналов. Например, для сообщения о сетевой ошибке достаточно одного изображения, заданного для всех каналов.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_EnableUserPic。

RK_MPI_VDEC_DisableUserPic。

4.15 RK_MPI_VDEC_EnableUserPic

【Описание】

Включает вставку пользовательского изображения.

【Синтаксис】

RK_S32 RK_MPI_VDEC_EnableUserPic(VDEC_CHN VdChn, RK_BOOL bInstant)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
bInstant	Включает режим пользовательского изображения. Диапазон значений: 0: отложенная вставка пользовательского изображения; 1: немедленная вставка пользовательского изображения;	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Перед включением вставки необходимо задать атрибуты пользовательского изображения, иначе возвращается RK_ERR_VDEC_NOT_PERM.
- Перед включением вставки необходимо остановить приём потока, иначе возвращается RK_ERR_VDEC_NOT_PERM.
- Если bInstant равен RK_TRUE, изображение вставляется немедленно. VDEC сначала сбрасывает канал, очищает кэшированные данные и затем вставляет пользовательское изображение. Поэтому после отключения пользовательского изображения и подачи нового потока правильное декодирование может начаться не сразу, а только после прихода следующего I-кадра.
- Если bInstant равен RK_FALSE, изображение вставляется с задержкой: VDEC ждёт, пока декодер обработает весь поток и выведет все изображения, после чего вставляет пользовательское изображение.
- После включения вставки необходимо вызвать RK_MPI_VDEC_DisableUserPic, чтобы отключить её, прежде чем задавать новые атрибуты пользовательского изображения.
- Повторное включение вставки возвращает успех, но повторно изображение не вставляется.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_SetUserPic。

RK_MPI_VDEC_DisableUserPic。

4.16 RK_MPI_VDEC_DisableUserPic

【Описание】

Отключает вставку пользовательского изображения.

【Синтаксис】

RK_S32 RK_MPI_VDEC_DisableUserPic(VDEC_CHN VdChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_EnableUserPic。

4.17 RK_MPI_VDEC_SetChnParam

【Описание】

Задаёт параметры канала декодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_SetChnParam(VDEC_CHN VdChn, const VDEC_CHN_PARAM_S *pstParam)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstParam	Параметры канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Передаваемый pstParam не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.
- Изменения параметров канала не вступают в силу динамически. Необходимо остановить приём потока и повторно запустить его; при этом канал декодирования будет сброшен.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_GetChnParam。

4.18 RK_MPI_VDEC_GetChnParam

【Описание】

Получает параметры канала декодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_GetChnParam(VDEC_CHN VdChn, VDEC_CHN_PARAM_S *pstParam)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
pstParam	Параметры канала.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Передаваемый pstParam не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_SetChnParam。

4.19 RK_MPI_VDEC_SetDisplayMode

【Описание】

Задаёт режим отображения.

【Синтаксис】

RK_S32 RK_MPI_VDEC_SetDisplayMode(VDEC_CHN VdChn, VIDEO_DISPLAY_MODE_E enDisplayMode)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
enDisplayMode	Перечисление режима отображения.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

Примечание

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Режим предварительного просмотра (VIDEO_DISPLAY_MODE_PREVIEW): высокая актуальность, при передаче данных возможна потеря кадров, что обеспечивает просмотр в реальном времени. Непосредственно привязанный к VDEC следующий модуль получает декодированные изображения в неблокирующем режиме. Например, если следующим модулем является VPSS и число кадров в памяти декодера превышает размер очереди VPSS (очередь Buffer VPSS заполнена), VPSS отбрасывает изображения от VDEC, не создавая обратного давления на декодер и сохраняя просмотр в реальном времени. Однако если число кадров в памяти декодера меньше размера очереди VPSS, даже в режиме предварительного просмотра VPSS продолжает создавать обратное давление на декодер.

- Режим воспроизведения (VIDEO_DISPLAY_MODE_PLAYBACK): более низкая актуальность, при передаче данных кадры не теряются, а задержка зависит от размеров кэш-очередей модулей тракта. Непосредственно привязанный к VDEC следующий модуль получает декодированные изображения в блокирующем режиме и может создавать обратное давление на VDEC. Например, если следующим модулем является VPSS и очередь изображений VPSS заполнена, VPSS не принимает изображения от VDEC. После неудачной отправки VDEC запускает механизм повторной передачи до успешной отправки изображения.

Пример

Нет.

Связанные темы

RK_MPI_VDEC_GetDisplayMode。

4.20 RK_MPI_VDEC_GetDisplayMode

Описание

Задаёт режим отображения.

Синтаксис

RK_S32 RK_MPI_VDEC_GetDisplayMode(VDEC_CHN VdChn, VIDEO_DISPLAY_MODE_E *penDisplayMode)

Параметры

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
penDisplayMode	Указатель на перечисление режима отображения.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

Примечание

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Передаваемый penDisplayMode не должен быть пустым, иначе возвращается ошибка пустого указателя RK_ERR_VDEC_NULL_PTR.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_SetDisplayMode。

4.21 RK_MPI_VDEC_GetFd

【Описание】

Получает дескриптор файла устройства канала видеодекодирования для проверки появления декодированного изображения.

【Синтаксис】

RK_S32 RK_MPI_VDEC_GetFd(VDEC_CHN VdChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
Положительное значение	Допустимое возвращаемое значение.
Неположительное значение	Недопустимое возвращаемое значение.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- После получения fd канала можно через select проверить наличие нового декодированного изображения и затем вызвать RK_MPI_VDEC_GetFrame, чтобы получить изображение и избежать бессмысленного опроса.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_GetFrame。

4.22 RK_MPI_VDEC_CloseFd

【Описание】

Закрывает дескриптор файла устройства видеодекодирования.

【Синтаксис】

RK_S32 RK_MPI_VDEC_CloseFd(VDEC_CHN VdChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно закрыт.
-1	Ошибка закрытия.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_GetFd。

4.23 RK_MPI_VDEC_AttachMbPool

【Описание】

Привязывает канал декодирования к указанному пулу видеобуферов MB.

【Синтаксис】

RK_S32 RK_MPI_VDEC_AttachMbPool(VDEC_CHN VdChn, MB_POOL hMbPool)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход
hMbPool	Сведения о пуле видеобуферов MB.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Параметр hMbPool должен быть созданным и действительным пулом памяти MB, иначе возвращается RK_ERR_VDEC_ILLEGAL_PARAM.
- Пользователь создаёт пул видеобуферов MB вызовом RK_MPI_MB_CreatePool, затем вызывает RK_MPI_VDEC_AttachMbPool, чтобы привязать созданный пул памяти MB к каналу декодирования. Привязывать один канал декодирования к нескольким пулам MB одновременно запрещено.
- Чтобы сменить пул MB, привязанный к текущему каналу декодирования, достаточно повторно вызвать RK_MPI_VDEC_AttachMbPool и указать новый пул памяти.
- Если для текущей памяти кадров декодирования используется не пользовательский пул (MB_SOURCE_USER), возвращается ошибка. RK_ERR_VDEC_NOT_SUPPORT。

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_DetachMbPool。

4.24 RK_MPI_VDEC_DetachMbPool

【Описание】

Отвязывает канал декодирования от указанного пула видеобуферов MB.

【Синтаксис】

RK_S32 RK_MPI_VDEC_DetachMbPool(VDEC_CHN VdChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VdChn	Номер канала видеодекодирования. Диапазон значений: [0, VDEC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VDEC.

【Примечание】

- Канал должен быть создан, иначе возвращается ошибка отсутствующего канала RK_ERR_VDEC_UNEXIST.
- Если для текущей памяти кадров декодирования используется не пользовательский пул (MB_SOURCE_USER), возвращается код ошибки VDEC.

  RK_ERR_VDEC_NOT_SUPPORT。

【Пример】

Нет.

【Связанные темы】

RK_MPI_VDEC_AttachMbPool。

5. Типы данных

Ниже приведены определения типов и структур данных, относящихся к видеодекодированию:

- VDEC_CHN_ATTR_S: определяет атрибуты канала декодирования.
- VDEC_ATTR_VIDEO_S: определяет атрибуты видеоканала видеодекодирования.
- VIDEO_MODE_E: определяет перечисление способов отправки потока.
- VDEC_CHN_STATUS_S: определяет структуру состояния канала.
- VDEC_DECODE_ERROR_S: определяет структуру информации об ошибках декодирования.
- VDEC_CHN_PARAM_S: определяет структуру расширенных параметров канала декодирования.
- VDEC_PARAM_VIDEO_S: определяет структуру расширенных параметров видеодекодирования.
- VDEC_PARAM_PICTURE_S: определяет структуру расширенных параметров декодирования изображений.
- VIDEO_DEC_MODE_E: определяет перечисление режимов декодирования.
- VIDEO_OUTPUT_ORDER_E: определяет перечисление порядка вывода декодированных изображений.
- COMPRESS_MODE_E: определяет перечисление режимов сжатия декодированных изображений.
- VDEC_PRTCL_PARAM_S: определяет структуру параметров выделения памяти, связанных с протоколом.
- VDEC_STREAM_S: определяет структуру декодируемого потока.
- VIDEO_DISPLAY_MODE_E: определяет перечисление режимов отображения.
- VDEC_CHN_POOL_S: определяет структуру пулов MB, привязанных к каналу декодирования.
- VDEC_VIDEO_MOD_PARAM_S: определяет структуру параметров модуля видеодекодирования.
- VDEC_PICTURE_MOD_PARAM_S: определяет структуру параметров модуля декодирования изображений.
- VDEC_MOD_PARAM_S: определяет структуру параметров модуля декодирования.

5.1 VDEC_MAX_CHN_NUM

【Описание】

Определяет максимальное число каналов декодирования.

【Определение】

```
#define VDEC_MAX_CHN_NUM              64
```

【Примечания】

Нет

5.2 VDEC_CHN_ATTR_S

【Описание】

Определяет атрибуты канала декодирования.

【Определение】

```
typedef struct rkVDEC_CHN_ATTR_S {
    PAYLOAD_TYPE_E enType;
    VIDEO_MODE_E enMode;
    RK_U32 u32PicWidth;
    RK_U32 u32PicHeight;
    RK_U32 u32PicVirWidth;
    RK_U32 u32PicVirHeight;
    RK_U32 u32StreamBufSize;
    RK_U32 u32FrameBufSize;
    RK_U32 u32FrameBufCnt;
    RK_U32 u32StreamBufCnt;
    union {
        VDEC_ATTR_VIDEO_S stVdecVideoAttr;
    };
} VDEC_CHN_ATTR_S;
```

【Члены】

Имя поля	Описание	Атрибут
enType	Значение перечисления протокола декодирования.	Статический атрибут.
enMode	Способ отправки потока.	Статический атрибут.
u32PicWidth	Ширина видео в пикселях.	Статический атрибут.
u32PicHeight	Высота видео в пикселях.	Статический атрибут.
u32PicVirWidth	Не поддерживается.	Статический атрибут.
u32PicVirHeight	Не поддерживается.	Статический атрибут.
u32StreamBufSize	Не поддерживается.	Статический атрибут.
u32FrameBufSize	По умолчанию задаётся 0, размер вычисляется внутри по необходимости. При декодировании 10-битного потока необходимо задать размер буфера декодирования, вычисленный RK_MPI_CAL_VDEC_GetPicBufferSize.	Статический атрибут.
u32FrameBufCnt	Число кадров в памяти декодированных изображений. Диапазон: больше 0, значение по умолчанию 6. Пользователь обязан гарантировать, что настроенного числа кадров достаточно для декодируемого потока, иначе нормальное декодирование невозможно. Для H.264/H.265 требуется «число опорных кадров + 2» кадров памяти. Для JPEG/MJPEG требуется «число отображаемых кадров + 1». Действует только в режиме PrivateMB.	Статический атрибут.
u32StreamBufCnt	Число хранимых пакетов потока. Диапазон: не менее 2, значение по умолчанию 8.	Статический атрибут.

【Примечания】

- Способ распределения числа кадров памяти:

  1. Если u32FrameBufCnt > 0, число кадров памяти равно u32FrameBufCnt. 2. Если u32FrameBufCnt == 0 && u32RefFrameNum > 0, число кадров памяти равно u32RefFrameNum + 2. 3. Если u32FrameBufCnt == 0 && u32RefFrameNum == 0, используется значение по умолчанию. Размер buffer памяти кадров автоматически вычисляется по ширине и высоте; пользователь обязан указать правильные размеры. Если u32StreamBufCnt == 0, число буферизуемых пакетов потока задаётся значением по умолчанию. Рекомендуется задавать
- u32StreamBufCnt не менее 3.
-
-

【Пример】

Нет.

【Связанные темы】

Нет.

5.3 VDEC_ATTR_VIDEO_S

【Описание】

Определяет атрибуты видеоканала видеодекодирования.

【Определение】

```
typedef struct rkVDEC_ATTR_VIDEO_S {
   RK_U32 u32RefFrameNum; /* RW, Range: [0, 16]; reference frame num. /
   RK_BOOL bTemporalMvpEnable; / RW; /
   / specifies whether temporal motion vector predictors can be used for inter prediction /
   RK_U32 u32TmvBufSize; / RW; tmv buffer size(Byte) */
} VDEC_ATTR_VIDEO_S;
```

【Члены】

Имя поля	Описание	Атри-бут
u32RefFrameNum	Число опорных кадров. Диапазон: [0, 16], в кадрах. Число опорных кадров определяет число кадров, необходимых при декодировании, и существенно влияет на использование блоков памяти MB. Задавайте подходящее значение с учётом реального сценария. - Поток IPC: рекомендуется 5. - Тестовый поток: рекомендуется 16.	Статический атри-бут.
bTemporalMvpEnable	Если поток H.264 содержит B-кадры либо поток H.265 поддерживает временное предсказание векторов движения (sps_temporal_mvp_enabled_flag = 1), при создании канала необходимо включить временное предсказание векторов движения, то есть задать bTemporalMvpEnable = 1. Модуль VDEC выделит память для хранения векторов движения MV в соответствии с числом опорных кадров. В противном случае bTemporalMvpEnable задаётся равным 0. 0。	Статический атри-бут.
u32TmvBufSize	Не поддерживается.	Статический атри-бут.

【Примечания】

Нет

【Пример】

Нет.

【Связанные темы】

Нет.

5.4 VIDEO_MODE_E

【Описание】

Определяет способ отправки потока.

【Определение】

```
typedef enum rkVIDEO_MODE_E {
   VIDEO_MODE_STREAM = 0, /* send by stream /
   VIDEO_MODE_FRAME, / send by frame  /
   VIDEO_MODE_COMPAT, / One frame supports multiple packets sending. /
   / The current frame is considered to end when bEndOfFrame is equal to RK_TRUE */
   VIDEO_MODE_BUTT
} VIDEO_MODE_E;
```

【Члены】

Имя поля	Описание	Атрибут
VIDEO_MODE_STREAM	Отправка потока в потоковом режиме. Декодирование JPEG/MJPEG этот режим не поддерживает.	Статический атрибут.
VIDEO_MODE_FRAME	Отправка потока в кадровом режиме, по одному кадру.	Статический атрибут.
VIDEO_MODE_COMPAT	Не поддерживается.	Статический атрибут.

【Примечания】

- Если для последнего пакета кадра bEndOfFrame не установлен в RK_TRUE, можно дополнительно отправить декодеру пустой пакет с bEndOfFrame = RK_TRUE, чтобы обозначить завершение передачи текущего кадра.

【Пример】

Нет.

【Связанные темы】

Нет.

5.5 VDEC_CHN_STATUS_S

【Описание】

Определяет состояние канала.

【Определение】

```
typedef struct rkVDEC_CHN_STATUS_S {
    RK_CODEC_ID_E enType; /* R; video type to be decoded */
    RK_U32 u32LeftStreamBytes; /* R; left stream bytes waiting for decode */
    RK_U32 u32LeftStreamFrames; /* R; left frames waiting for decode,only valid for VIDEO_MODE_FRAME */
    RK_U32 u32LeftPics; /* R; pics waiting for output */
    RK_BOOL bStartRecvStream; /* R; had started recv stream? */
    RK_U32 u32RecvStreamFrames; /* R; how many frames of stream has been received. valid when send by frame. */
    RK_U32 u32DecodeStreamFrames; /* R; how many frames of stream has been decoded. valid when send by frame. */
    VDEC_DECODE_ERROR_S stVdecDecErr; /* R; information about decode error */
    RK_U32 u32Width;
    RK_U32 u32Height;
} VDEC_CHN_STATUS_S;
```

【Члены】

Имя члена	Описание
enType	Значение перечисления протокола декодирования.
u32LeftStreamBytes	Число байтов в buffer потока, ожидающих декодирования, включая ещё не декодированные байты текущего обрабатываемого кадра.
u32LeftStreamFrames	Число кадров в buffer потока, ожидающих декодирования, без текущего декодируемого кадра. -1 означает недействительное значение. В потоковом режиме недействительно. При ошибках потока и отбрасывании кадров счётчик может быть неточным.
u32LeftPics	Число изображений, оставшихся в buffer изображений.
bStartRecvStream	Запущен ли приём потока декодером.
u32RecvStreamFrames	Число кадров потока, принятых buffer потока. -1 означает недействительное значение. В потоковом режиме недействительно.
u32DecodeStreamFrames	Число кадров, декодированных из buffer потока.
stVdecDecErr	Информация об ошибках декодирования.
u32Width	Ширина изображения.
u32Height	Высота изображения.

【Примечания】

Нет

【Пример】

Нет.

【Связанные темы】

Нет.

5.6 VDEC_DECODE_ERROR_S

【Описание】

Определяет структуру информации об ошибках декодирования.

【Определение】

```
typedef struct rkVDEC_DECODE_ERROR_S {
    RK_S32 s32FormatErr; /* R; format error. eg: do not support filed /
    RK_S32 s32PicSizeErrSet; / R; picture width or height is larger than chnnel width or height /
    RK_S32 s32StreamUnsprt; / R; unsupport the stream specification /
    RK_S32 s32PackErr; / R; stream package error /
    RK_S32 s32PrtclNumErrSet; / R; protocol num is not enough. eg: slice, pps, sps /
    RK_S32 s32RefErrSet; / R; refrence num is not enough /
    RK_S32 s32PicBufSizeErrSet; / R; the buffer size of picture is not enough /
    RK_S32 s32StreamSizeOver; / R; the stream size is too big and and force discard stream /
    RK_S32 s32VdecStreamNotRelease; / R; the stream not released for too long time */
} VDEC_DECODE_ERROR_S;
```

【Члены】

Имя члена	Описание
s32FormatErr	Неподдерживаемый формат: заданный fmt отсутствует в списке RK_CODEC_ID_E.
s32PicSizeErrSet	Не поддерживается.
s32StreamUnsprt	Неподдерживаемые параметры: заданные ширина или высота меньше 0.
s32PackErr	Ошибка потока.
s32PrtclNumErrSet	Не поддерживается.
s32RefErrSet	Не поддерживается.
s32PicBufSizeErrSet	Не поддерживается.
s32StreamSizeOver	Не поддерживается.
s32VdecStreamNotRelease	Не поддерживается.

【Примечания】

- Вся информация об ошибках представляется накопительными счётчиками. Например, при каждом обнаружении ошибки потока значение s32PackErr увеличивается на 1.
- После сброса канала все счётчики ошибок обнуляются.

【Пример】

Нет.

【Связанные темы】

Нет.

5.7 VDEC_CHN_PARAM_S

【Описание】

Определяет расширенные параметры канала декодирования.

【Определение】

```
typedef struct rkVDEC_CHN_PARAM_S {
    PAYLOAD_TYPE_E enType;
    RK_U32 u32DisplayFrameNum;
    union {
        VDEC_PARAM_VIDEO_S stVdecVideoParam;
        VDEC_PARAM_PICTURE_S stVdecPictureParam;
    };
} VDEC_CHN_PARAM_S;
```

【Члены】

Имя поля	Описание	Атрибут
enType	Протокол декодирования.	Статический атрибут.
u32DisplayFrameNum	Не поддерживается.	Статический атрибут.
stVdecVideoParam	Расширенные параметры декодирования видео (H.264/H.265).	Статический атрибут.
stVdecPictureParam	Расширенные параметры декодирования изображений (JPEG/MJPEG).	Статический атрибут.

【Примечания】

- Для получения декодированных данных, пригодных для обычного отображения, необходимо установить несжатый вывод: COMPRESS_MODE_NONE.

【Пример】

Нет.

【Связанные темы】

Нет

5.8 VDEC_PARAM_VIDEO_S

【Описание】

Определяет расширенные параметры видеодекодирования.

【Определение】

```
typedef struct rkVDEC_PARAM_VIDEO_S {
   RK_S32 s32ErrThreshold;
   VIDEO_DEC_MODE_E enDecMode;
   VIDEO_OUTPUT_ORDER_E enOutputOrder;
   COMPRESS_MODE_E enCompressMode;
   VIDEO_FORMAT_E enVideoFormat;
} VDEC_PARAM_VIDEO_S;
```

【Члены】

Имя поля	Описание	Атрибут
s32ErrThreshold	Не поддерживается.	Статический атрибут.
enDecMode	Не поддерживается.	Статический атрибут.
enOutputOrder	Порядок вывода декодированных изображений. Default：VIDEO_OUTPUT_ORDER_DISP	Статический атрибут.
enCompressMode	Режим сжатия декодированных изображений. Диапазон: поддерживаются только COMPRESS_MODE_NONE и COMPRESS_AFBC_16x16. Default：COMPRESS_MODE_NONE	Статический атрибут.
enVideoFormat	Не поддерживается.	Статический атрибут.

【Примечания】

- Для получения декодированных данных, пригодных для обычного отображения, необходимо установить несжатый вывод: COMPRESS_MODE_NONE.

【Пример】

Нет.

【Связанные темы】

Нет

5.9 VDEC_PARAM_PICTURE_S

【Описание】

Определяет расширенные параметры декодирования изображений.

【Определение】

```
typedef struct rkVDEC_PARAM_PICTURE_S {
   PIXEL_FORMAT_E enPixelFormat; /* RW; out put pixel format /
   RK_U32 u32Alpha; / RW, Range: [0, 255]; value 0 is transparent. /
   / [0 ,127]   is deemed to transparent when enPixelFormat is ARGB1555 or ABGR1555
    [128 ,256] is deemed to non-transparent when enPixelFormat is ARGB1555 or ABGR1555
   */
 } VDEC_PARAM_PICTURE_S;
```

【Члены】

Имя поля	Описание	Атрибут
enPixelFormat	Формат вывода декодирования JPEG (MJPEG). Диапазон: поддерживаются только следующие форматы вывода. RK_FMT_BGR565、 RK_FMT_RGB888、 RK_FMT_YUV420SP RK_FMT_YUV420SP_VU、 RK_FMT_YUV422_YUYV、 RK_FMT_YUV422_UYVY、 Default：RK_FMT_YUV420SP	Статический атрибут.
u32Alpha	Не поддерживается.	Статический атрибут.

【Примечания】

Нет

【Пример】

Нет.

【Связанные темы】

Нет

5.10 VIDEO_DEC_MODE_E

【Описание】

Определяет перечисление режимов видеодекодирования.

【Определение】

```
typedef enum rkVIDEO_DEC_MODE_E {
   VIDEO_DEC_MODE_IPB = 0,
   VIDEO_DEC_MODE_IP,
   VIDEO_DEC_MODE_I,
   VIDEO_DEC_MODE_BUTT
} VIDEO_DEC_MODE_E;
```

【Члены】

Имя поля	Описание	Атрибут
VIDEO_DEC_MODE_IPB	Режим IPB: декодируются I-, P- и B-кадры.	Статический атрибут.
VIDEO_DEC_MODE_IP	Не поддерживается.	Статический атрибут.
VIDEO_DEC_MODE_I	Не поддерживается.	Статический атрибут.

【Примечания】

Нет

【Пример】

Нет.

【Связанные темы】

Нет

5.11 VIDEO_OUTPUT_ORDER_E

【Описание】

Определяет перечисление порядка вывода видеодекодирования.

【Определение】

```
typedef enum rkVIDEO_OUTPUT_ORDER_E {
    VIDEO_OUTPUT_ORDER_DISP = 0,
    VIDEO_OUTPUT_ORDER_DEC,
    VIDEO_OUTPUT_ORDER_BUTT
} VIDEO_OUTPUT_ORDER_E;
```

【Члены】

Имя поля	Описание	Атрибут
VIDEO_OUTPUT_ORDER_DISP	Вывод в порядке отображения.	Статический атрибут.
VIDEO_OUTPUT_ORDER_DEC	Вывод в порядке декодирования.	Статический атрибут.

【Примечания】

Для декодирования потока с B-кадрами следует выбирать вывод в порядке отображения.

Для потока без B-кадров следует выбирать порядок декодирования, чтобы ускорить вывод.

【Пример】

Нет.

【Связанные темы】

Нет

5.12 COMPRESS_MODE_E

【Описание】

Определяет перечисление режимов сжатия декодированных изображений.

【Определение】

```
typedef enum rkCOMPRESS_MODE_E {
    COMPRESS_MODE_NONE = 0,   /* no compress */
    COMPRESS_AFBC_16x16,
    COMPRESS_MODE_BUTT
} COMPRESS_MODE_E;
```

【Члены】

Имя поля	Описание	Атрибут
COMPRESS_MODE_NONE	Без сжатия. Декодирование поддерживает этот режим.	Статический атрибут.
COMPRESS_AFBC_16x16	Сжатие кадров. Декодирование поддерживает этот режим.	Статический атрибут.

【Примечания】

Нет

【Пример】

Нет.

【Связанные темы】

Нет

5.13 VDEC_STREAM_S

【Описание】

Определяет структуру потока видеодекодирования.

【Определение】

```
typedef struct rkVDEC_STREAM_S {
    MB_BLK  pMbBlk;
    RK_U32  u32Len;
    RK_U64  u64PTS;
    RK_BOOL bEndOfStream;
    RK_BOOL bEndOfFrame;
    RK_BOOL bBypassMbBlk;
} VDEC_STREAM_S;
```

【Члены】

Имя члена	Описание
pMbBlk	Адрес пакета потока.
u32Len	Длина пакета потока в байтах.
u64PTS	Временная метка пакета потока в мкс.
bEndOfStream	Передан ли весь поток.
bEndOfFrame	Не поддерживается.
bBypassMbBlk	Требуется ли копирование pMbBlk.

【Примечания】

- При отправке в кадровом режиме временная метка декодированного изображения равна временной метке пакета потока.
- При отправке в потоковом режиме временная метка декодированного изображения равна 0. После отправки всего потока установите
- bEndOfStream = 1, чтобы обозначить конец файла. Тогда декодер обработает весь переданный поток и выведет все изображения. Если после отправки всего потока оставить bEndOfStream равным 0, внутри декодера может остаться невыведенным одно или более изображений, поскольку декодер должен дождаться следующего кадра потока, чтобы определить завершение текущего кадра и передать его на декодирование.

- Выделенный пользователем buffer необходимо обернуть в тип MB_BLK через RK_MPI_SYS_CreateMB. Режим копирования:
- bBypassMbBlk = RK_FALSE. После передачи пользовательских data декодер копирует их внутри; память data остаётся под управлением пользователя и может циклически использоваться повторно.

- Сквозной режим: bBypassMbBlk = RK_TRUE и задан callback освобождения user_data_callback. Этот режим рекомендуется. В нём память пользовательских data освобождает декодер, что снижает расход памяти и загрузку CPU.

【Пример】

- Сквозной режим

```
static RK_S32 user_data_callback(void *opaque) {
    if (RK_NULL != opaque) {
       free(opaque);
    }
    opaque = RK_NULL;
    return 0;
}
int main(int argc, const char **argv) {
```

```
    VDEC_STREAM_S stStream;
    MB_EXT_CONFIG_S pstMbExtConfig;
    MB_BLK buffer = RK_NULL;
    RK_S32 usersize = 1024;

    memset(&stMbExtConfig, 0, sizeof(MB_EXT_CONFIG_S));
    memset(&stStream, 0, sizeof(VDEC_STREAM_S));

    RK_S8 *userdata = (RK_S8 *)malloc(usersize);
    pstMbExtConfig.pFreeCB = user_data_callback;
    pstMbExtConfig.pOpaque = userdata;
    pstMbExtConfig.pu8VirAddr = userdata;
    pstMbExtConfig.u64Size = usersize;

    RK_MPI_SYS_CreateMB(&buffer, &pstMbExtConfig);

    pstStream.u64PTS = userpts;
    pstStream.pMbBlk = buffer;
    pstStream.u32Len = usersize;
    pstStream.bEndOfStream = RK_FALSE;
    pstStream.bBypassMbBlk = RK_TRUE;

  __RETRY:
     s32Ret = RK_MPI_VDEC_SendStream(chn, &stStream, MAX_TIME_OUT_MS);
     if (s32Ret < 0) {
         usleep(1000llu);
         goto  __RETRY;
     } else {
         RK_MPI_MB_ReleaseMB(stStream.pMbBlk);
     }
     ......
}
```

- Режим копирования

```
int main(int argc, const char **argv) {
    VDEC_STREAM_S stStream;
    MB_EXT_CONFIG_S pstMbExtConfig;
    MB_BLK buffer = RK_NULL;
    RK_S32 usersize = 1024;

    memset(&stMbExtConfig, 0, sizeof(MB_EXT_CONFIG_S));
    memset(&stStream, 0, sizeof(VDEC_STREAM_S));

    RK_S8 *userdata = (RK_S8 *)malloc(usersize);
    pstMbExtConfig.pFreeCB = RK_NULL;
    pstMbExtConfig.pOpaque = userdata;
    pstMbExtConfig.pu8VirAddr = userdata;
    pstMbExtConfig.u64Size = usersize;

    RK_MPI_SYS_CreateMB(&buffer, &pstMbExtConfig);

    pstStream.u64PTS = userpts;
    pstStream.pMbBlk = buffer;
    pstStream.u32Len = usersize;
    pstStream.bEndOfStream = RK_FALSE;
    pstStream.bBypassMbBlk = RK_FALSE;

  __RETRY:
     s32Ret = RK_MPI_VDEC_SendStream(chn, &stStream, MAX_TIME_OUT_MS);
     if (s32Ret < 0) {
         usleep(1000llu);
         goto  __RETRY;
     } else {
         RK_MPI_MB_ReleaseMB(stStream.pMbBlk);
     }
```

```
    ......
}
```

Связанные темы

Нет.

5.14 VIDEO_DISPLAY_MODE_E

Описание

Определяет перечисление режимов отображения.

Определение

```
typedef enum rkVIDEO_DISPLAY_MODE_E {
    VIDEO_DISPLAY_MODE_PREVIEW  = 0x0,
    VIDEO_DISPLAY_MODE_PLAYBACK = 0x1,

    VIDEO_DISPLAY_MODE_BUTT
} VIDEO_DISPLAY_MODE_E;
```

Члены

Имя члена	Описание
VIDEO_DISPLAY_MODE_PREVIEW	Режим предварительного просмотра.
VIDEO_DISPLAY_MODE_PLAYBACK	Режим воспроизведения.

Примечания

Нет

Пример

Нет.

Связанные темы

Нет.

5.15 VDEC_VIDEO_MOD_PARAM_S

Описание

Определяет структуру параметров модуля видеодекодирования.

Определение

```
typedef struct rkVDEC_VIDEO_MOD_PARAM_S {
    RK_U32 u32MaxPicWidth;
    RK_U32 u32MaxPicHeight;
    RK_U32 u32MaxSliceNum;
    RK_U32 u32VdhMsgNum;
    RK_U32 u32VdhBinSize;
    RK_U32 u32VdhExtMemLevel;
} VDEC_VIDEO_MOD_PARAM_S;
```

Члены

Имя члена	Описание
u32MaxPicWidth	Не поддерживается.
u32MaxPicHeight	Не поддерживается.
u32MaxSliceNum	Не поддерживается.
u32VdhMsgNum	Не поддерживается.
u32VdhBinSize	Не поддерживается.
u32VdhExtMemLevel	Не поддерживается.

Примечания

Нет

Пример

Нет.

Связанные темы

Нет.

5.16 VDEC_PICTURE_MOD_PARAM_S

Описание

Определяет структуру параметров модуля декодирования изображений.

Определение

```
typedef struct rkVDEC_PICTURE_MOD_PARAM_S {
    RK_U32 u32MaxPicWidth;
    RK_U32 u32MaxPicHeight;
    RK_BOOL bSupportProgressive;
    RK_BOOL bDynamicAllocate;
} VDEC_PICTURE_MOD_PARAM_S;
```

Члены

Имя члена	Описание
u32MaxPicWidth	Максимальная ширина, поддерживаемая декодированием JPEG/MJPEG. Не поддерживается.
u32MaxPicHeight	Максимальная высота, поддерживаемая декодированием JPEG/MJPEG. Не поддерживается.
bSupportProgressive	Не поддерживается.
bDynamicAllocate	Не поддерживается.

Примечания

Нет

Пример

Нет.

Связанные темы

Нет.

5.17 VDEC_MOD_PARAM_S

Описание

Определяет структуру параметров модуля декодирования.

【Определение】

typedef struct rkVDEC_MOD_PARAM_S {
MB_SOURCE_E enVdecMBSource; /* RW, Range: [1, 3]; frame buffer mode /
RK_U32 u32MiniBufMode; / RW, Range: [0, 1]; stream buffer mode /
RK_U32 u32ParallelMode; / RW, Range: [0, 1]; VDPU working mode */
VDEC_VIDEO_MOD_PARAM_S stVideoModParam;
VDEC_PICTURE_MOD_PARAM_S stPictureModParam;
} VDEC_MOD_PARAM_S;

【Члены】

Имя члена	Описание
enVdecMBSource	Источник MB памяти кадров декодирования. Диапазон: поддерживаются только следующие значения. MB_SOURCE_MODULE、 MB_SOURCE_PRIVATE、 MB_SOURCE_USER Default：MB_SOURCE_MODULE
u32MiniBufMode	Не поддерживается.
u32ParallelMode	Не поддерживается.
stVideoModParam	Не поддерживается.

【Примечания】

- Режим enVdecMBSource необходимо задавать до запуска любых каналов, иначе настройка не действует.
- В режиме MB_SOURCE_USER до запуска канала декодирования необходимо вызвать RK_MPI_VDEC_AttachMbPool и передать выделенный пользователем пул памяти, иначе канал не будет иметь доступного буфера декодирования.

【Пример】

Нет.

【Связанные темы】

Нет.

5.18 FRAME_FLAG_E

【Описание】

Определяет тип кадра.

【Определение】

```
typedef enum rkFRAME_FLAG_E {
    FRAME_FLAG_SNAP_FLASH  = 0x1 << 0,
    FRAME_FLAG_SNAP_CUR    = 0x1 << 1,
    FRAME_FLAG_SNAP_REF    = 0x1 << 2,
    FRAME_FLAG_SNAP_END    = 0x1 << 31,
    FRAME_FLAG_BUTT
} FRAME_FLAG_E;
```

【Члены】

Имя члена	Описание
FRAME_FLAG_SNAP_FLASH	Не поддерживается.
FRAME_FLAG_SNAP_CUR	Не поддерживается.
FRAME_FLAG_SNAP_REF	Не поддерживается.
FRAME_FLAG_SNAP_END	Последний декодированный кадр данных.

Примечания

Нет

Пример

Нет.

Связанные темы

Нет.

5.19 VIDEO_FRAME_INFO_S

Описание

Определяет структуру информации о кадре видеоизображения.

Определение

```
typedef struct rkVIDEO_FRAME_S {
    MB_BLK              pMbBlk;
    RK_U32              u32Width;
    RK_U32              u32Height;
    RK_U32              u32VirWidth;
    RK_U32              u32VirHeight;
    VIDEO_FIELD_E       enField;
    PIXEL_FORMAT_E      enPixelFormat;
    VIDEO_FORMAT_E      enVideoFormat;
    COMPRESS_MODE_E     enCompressMode;
    DYNAMIC_RANGE_E     enDynamicRange;
    COLOR_GAMUT_E       enColorGamut;

    RK_VOID            *pVirAddr[RK_MAX_COLOR_COMPONENT];

    RK_U32              u32TimeRef;
    RK_U64              u64PTS;

    RK_U64              u64PrivateData;
    RK_U32              u32FrameFlag;     /* FRAME_FLAG_E, can be OR operation. */
} VIDEO_FRAME_S;

typedef struct rkVIDEO_FRAME_INFO_S {
    VIDEO_FRAME_S stVFrame;
} VIDEO_FRAME_INFO_S;
```

Члены

Имя члена	Описание
pMbBlk	Данные изображения.
u32Width	Фактическая ширина изображения.
u32Height	Фактическая высота изображения.
u32VirWidth	Виртуальная ширина изображения.
u32VirHeight	Виртуальная высота изображения.
enField	Режим полей кадра.
enVideoFormat	Не поддерживается.
enPixelFormat	Формат пикселей целевого изображения.
enCompressMode	Режим сжатия целевого изображения.
enDynamicRange	Не поддерживается.
enColorGamut	Не поддерживается.
pVirAddr	Адреса Y и UV изображения. Диапазон значений: Адрес Y: pVirAddr[RK_COLOR_YUV_Y_PLANE] Адрес UV: pVirAddr[RK_COLOR_YUV_UV_PLANE]
u32TimeRef	Не поддерживается.
u64PTS	Временная метка изображения.
u64PrivateData	Не поддерживается.
u32FrameFlag	Флаги текущего кадра из FRAME_FLAG_E; допускается побитовое ИЛИ.

Примечания

Нет.

Пример

- Чтобы получить данные изображения YUV, пользователь может преобразовать pMbBlk в виртуальный адрес через RK_MPI_MB_Handle2VirAddr.

```
data = RK_MPI_MB_Handle2VirAddr(sFrame.stVFrame.pMbBlk);
fwrite(data, 1, sFrame.stVFrame.u32Width * sFrame.stVFrame.u32Height * 3 /2, fp);
fflush(fp);
```

Связанные темы

Нет.

5.20 VIDEO_FIELD_E

Описание

Определяет тип поля кадра видеоизображения.

Определение

```
typedef enum rkVIDEO_FIELD_E {
    VIDEO_FIELD_TOP         = 0x1,    /* even field */
    VIDEO_FIELD_BOTTOM      = 0x2,    /* odd field */
    VIDEO_FIELD_INTERLACED  = 0x3,    /* two interlaced fields */
    VIDEO_FIELD_FRAME       = 0x4,    /* frame */

    VIDEO_FIELD_BUTT
} VIDEO_FIELD_E;
```

Члены

Имя члена	Описание
VIDEO_FIELD_TOP	Верхнее поле. Не поддерживается.
VIDEO_FIELD_BOTTOM	Нижнее поле. Не поддерживается.
VIDEO_FIELD_INTERLACED	Тип с чередованием двух полей.
VIDEO_FIELD_FRAME	Тип кадра.

Примечания

Если bDeiEn == RK_TRUE и источник чересстрочный, выполняется Deinterlace, а число кадров удваивается.

Если bDeiEn == RK_FALSE и источник чересстрочный, верхнее и нижнее поля объединяются в один выходной кадр.

Пример

Нет.

Связанные темы

VIDEO_FRAME_INFO_S

6. Коды ошибок VDEC

Ниже приведены коды ошибок API видеодекодирования VDEC:

Код ошибки	Макроопределение	Описание
0xA0058002	RK_ERR_VDEC_INVALID_CHNID	Недопустимый номер канала VDEC.
0xA0058003	RK_ERR_VDEC_ILLEGAL_PARAM	Недопустимый параметр VDEC.
0xA0058004	RK_ERR_VDEC_EXIST	Канал VDEC уже создан.
0xA0058005	RK_ERR_VDEC_UNEXIST	Канал VDEC не создан.
0xA0058006	RK_ERR_VDEC_NULL_PTR	Пустой указатель во входном параметре.
0xA0058008	RK_ERR_VDEC_NOT_SUPPORT	Операция не поддерживается.
0xA0058009	RK_ERR_VDEC_NOT_PERM	Операция не разрешена.
0xA005800C	RK_ERR_VDEC_NOMEM	Не удалось выделить память.
0xA005800D	RK_ERR_VDEC_NOBUF	Не удалось выделить пул BUF.
0xA005800E	RK_ERR_VDEC_BUF_EMPTY	Очередь изображений пуста.
0xA005800F	RK_ERR_VDEC_BUF_FULL	Очередь изображений заполнена.
0xA0058010	RK_ERR_VDEC_NOTREADY	Система VDEC не инициализирована.
0xA0058012	RK_ERR_VDEC_BUSY	Система VDEC занята.
0xA0058013	RK_ERR_VDEC_BADADDR	Ошибочный адрес.

Кодирование видео

Предисловие

Обзор

Модуль VENC — модуль видеокодирования. Он поддерживает многоканальное кодирование в реальном времени; каждый канал независим и может использовать собственные протокол и profile. Одновременно с кодированием модуль может планировать Region для наложения и маскирования содержимого изображения.

Версии продуктов

Название микросхемы	Версия ядра
RV1109/RV1126	4.19
RK356X	4.19
RK3588	5.10
RV1106/RV1103	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	XLM	2021-01-09	Первая версия.
V0.2.0	XLM	2021-01-23	Уточнены определения структур данных
V0.3.0	LXH	2021-09-24	Дополнены описания новых функций и сведения о поддерживаемых характеристиках микросхем.
V0.4.0	LXH	2021-12-06	Дополнены описания новых функций и сведения о характеристиках, поддерживаемых RK3588.
V0.5.0	LXH	2021-12-18	Дополнены определения структур данных.
V0.6.0	WZH	2021-12-24	Дополнено описание динамических параметров.
V0.7.0	WZH	2021-12-27	Дополнены описания QPMAP, максимального числа повторных кодирований и выравнивания виртуальной ширины и высоты.
V0.8.0	LXH	2022-01-05	Добавлено описание интерфейсов attach/detach MbPool.
V0.8.1	FXW	2022-03-25	Исправлены значения кодов ошибок VENC.
V0.9.0	LXH	2022-03-26	Дополнены определения структур данных и описания части функций.
V1.0.0	WZH	2022-04-06	Дополнено описание новых функций.
V1.1.0	LXH	2022-04-09	Обновлены параметры управления битрейтом.
V1.2.0	LXH	2022-04-15	Изменено описание масштабирования: допускается не задавать ширину и высоту источника.
V1.3.0	LXH	2022-05-10	Добавлено описание интерфейса разбиения slice.
V1.4.0	LXH	2022-07-04	Добавлено описание интерфейсов задания и получения параметров модуля кодирования.

Содержание

Обзор
Описание функций
Пример
Справочник API
Типы данных
Коды ошибок VENC

1. Обзор

Модуль VENC — модуль видеокодирования, главным образом поддерживающий H264, H265, JPEG и MJPEG. Он поддерживает многоканальное кодирование в реальном времени; каждый канал независим, а протокол и profile могут отличаться. Одновременно с кодированием модуль вызывает Region для наложения и маскирования содержимого изображения.

Входными источниками модуля VENC могут быть:

- **файл изображения, прочитанный пользовательским приложением и отправленный модулю кодирования;**
- **изображение, захваченное модулем видеоввода VI и переданное модулю кодирования через подсистему обработки видео VPSS;**
- **изображение, захваченное модулем видеоввода VI и напрямую переданное модулю кодирования;**
- **изображение, декодированное модулем VDEC и переданное модулю кодирования через подсистему обработки видео VPSS.**

Разные модели микросхем поддерживают разные характеристики кодирования. Поддерживаемые характеристики приведены ниже.

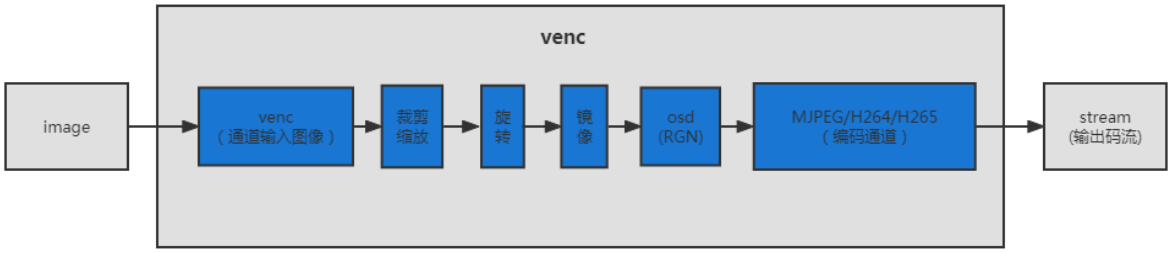
Микросхема	Поддерживаемое кодирование Категория	Поддерживаемый формат	Поддерживаемые характеристики
RV1109/RV1126	JPEG、 MJPEG	YCbCr formats: YCbCr 4:2:0 planar YCbCr 4:2:0 semi-planar YCbYCr 4:2:2 CbYCrY 4:2:2 Interleaved RGB formats: RGB444 to BGR444 RGB555 to BGR555 RGB565 to BGR565 ARGB8888 to BRGA8888 RGB101010 BRG 101010	96x96 to 8176x8176(64 million pixels) Step size 4 pixels Up to 90 million pixels per second
RV1109/RV1126	H264	ARGB, RGB, YUV422/420P/SP Arm AFBC YUV422/420	high profile encoding, up to level 5.1； 64x64 -- 4096x4096 4096x2304@30fps
RV1109/RV1126	H265	ARGB, RGB, YUV422/420P/SP Arm AFBC YUV422/420	main profile encoding, up to level 5.0; 64x64 -- 4096x4096 4096x2304@30fps
RK356x	JPEG、 MJPEG	YCbCr formats: YCbCr 4:2:0 planar YCbCr 4:2:0 semi-planar YCbYCr 4:2:2 CbYCrY 4:2:2 Interleaved RGB formats: RGB444 to BGR444 RGB555 to BGR555 RGB565 to BGR565 ARGB8888 to BRGA8888 RGB101010 BRG 101010	96x96 to 8176x8176(64 million pixels) Step size 4 pixels Up to 90 million pixels per second
RK356x	H264	ARGB, RGB, YUV422/420P/SP Arm AFBC YUV422/420	high profile encoding, up to level 5.1； 64x64 -- 1920x1080 1920x1080@60fps
RK356x	H265	ARGB, RGB, YUV422/420P/SP Arm AFBC YUV422/420	main profile encoding, up to level 5.0; 64x64 -- 1920x1080 1920x1080@60fps Примечание: поддерживается tile-кодирование.
RK3588	JPEG、 MJPEG	YCbCr formats: YCbCr 4:2:0 planar YCbCr 4:2:0 semi-planar YCbYCr 4:2:2 CbYCrY 4:2:2 Interleaved RGB formats: RGB444 to BGR444 RGB555 to BGR555 RGB565 to BGR565 ARGB8888 to BRGA8888 RGB101010 BRG 101010	96x96 to 8176x8176(64 million pixels) Step size 4 pixels Up to 90 million pixels per second 1920x1080@60fps Примечание: приведены данные для одного ядра; всего четыре ядра. Кодер JPEG/MJPEG

Микросхема	Поддерживаемое кодирование Категория	Поддерживаемый формат	Поддерживаемые характеристики
RK3588	H264	ARGB, RGB, YUV422/420P/SP Arm AFBC YUV422/420	High Profile, level 6.0 Resolution upto 16384x8192 1920x1080@240fps Примечание: приведены данные для одного ядра; всего два H264/H265-кодера.
RK3588	H265	ARGB, RGB, YUV422/420P/SP Arm AFBC YUV422/420	Main Profile, Level 6.0 High Tier Resolution upto 16384x8192 1920x1080@240fps Примечание: приведены данные для одного ядра; всего два H264/H265-кодера. Кодер; поддерживает tile-кодирование.

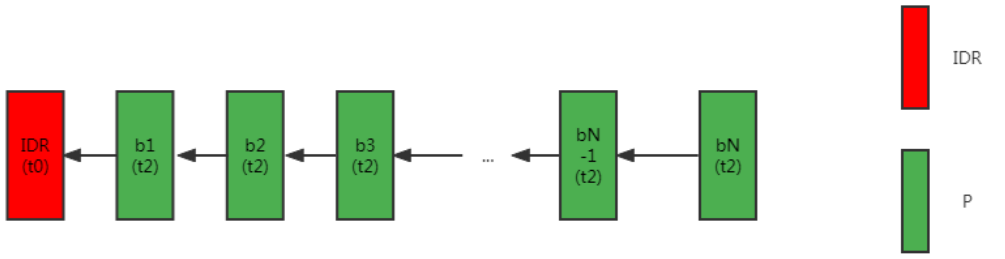
2. Описание функций

2.1 Схема потока данных кодирования

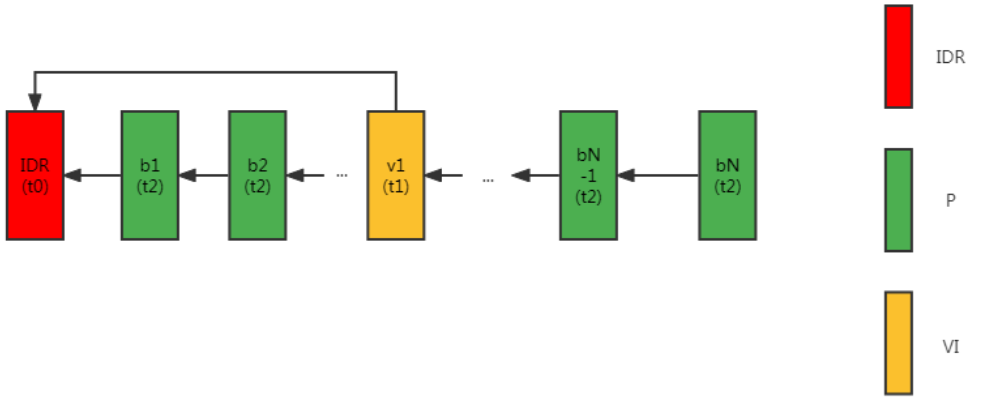
Типичный процесс кодирования включает приём входных изображений, выбор изображения, зеркальное отражение, обрезку и масштабирование, маскирование и наложение содержимого, кодирование изображения и вывод потока. Внутренняя обработка данных показана на рисунке ниже.



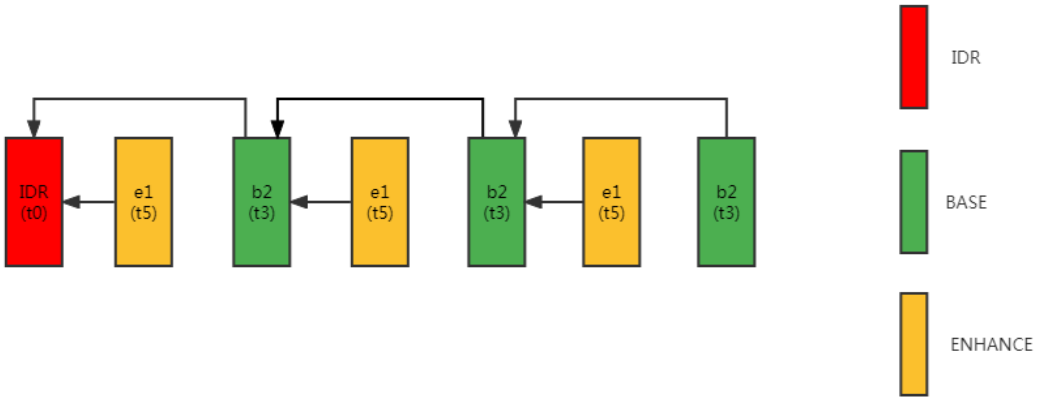
2.2 Структура GOP



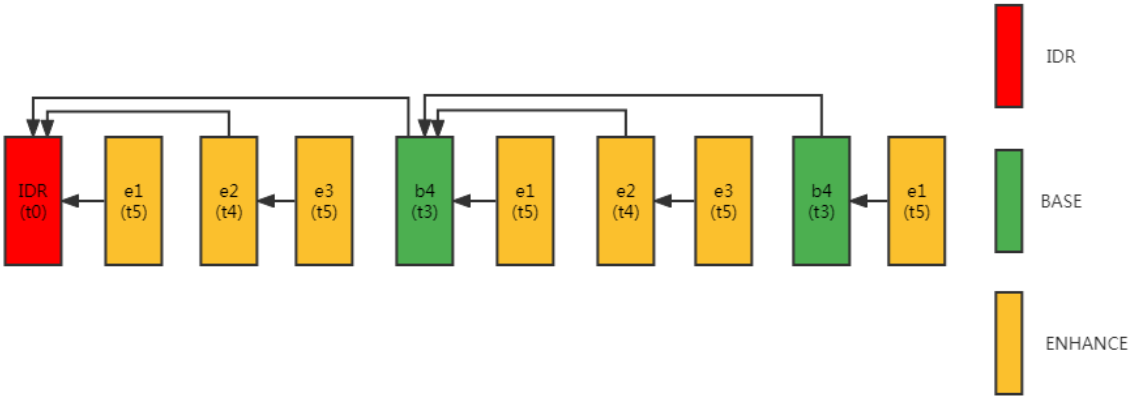
NORMALP参考关系



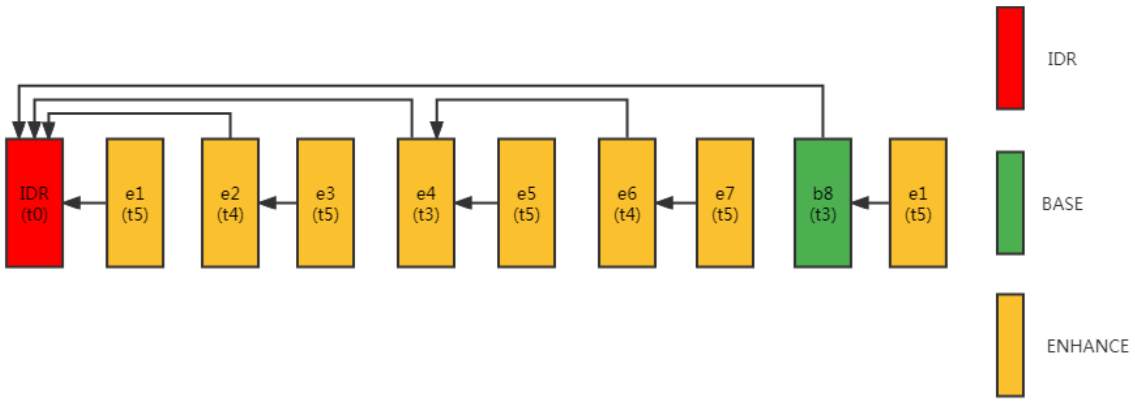
SMARTP参考关系



TSVC2参考关系



TSVC3参考关系



TSVC4参考关系

Примечание

- t0-t5 соответствуют определениям от BASE_IDRSLICE до ENHANCE_PSLICE_NOTFORREF в H264E_REF_TYPE_E.

3. Пример

```
VENC_RECV_PIC_PARAM_S pstRecvParam;
VENC_CHN_ATTR_S pstAttr;
MB_POOL vencPool

pstAttr.stVencAttr.enType = RK_VIDEO_ID_AVC;
pstAttr.stVencAttr.u32PicWidth = 720;
pstAttr.stVencAttr.u32PicHeight = 576;
pstAttr.stVencAttr.u32MaxPicWidth = 720;
pstAttr.stVencAttr.u32MaxPicHeight = 576;
pstAttr.stVencAttr.u32StreamBufCnt = 4;
pstAttr.stVencAttr.u32BufSize = 720 * 576 * 3 / 2;
pstAttr.stVencAttr.enPixelFormat = RK_FMT_YUV420SP;

RK_MPI_VENC_CreateChn(0, &gVencCtx->pstAttr);
RK_MPI_VENC_StartRecvFrame(0, &pstRecvParam);

MB_POOL_CONFIG_S pstMbPoolCfg;
memset(&pstMbPoolCfg, 0, sizeof(MB_POOL_CONFIG_S));
pstMbPoolCfg.u64MBSize = 720 * 576 * 2;
pstMbPoolCfg.u32MBCnt  = 10;
pstMbPoolCfg.enAllocType = MB_ALLOC_TYPE_DMA;

vencPool = RK_MPI_MB_CreatePool(&pstMbPoolCfg);

RK_MPI_VENC_SendFrame(0, pstFrame, -1);
RK_MPI_VENC_GetStream(0, pstStream, -1);
RK_MPI_VENC_ReleaseStream(0, pstStream);

RK_MPI_VENC_StopRecvFrame(0);
RK_MPI_VENC_DestroyChn(0);
RK_MPI_MB_DestroyPool(vencPool);
```

4. Справочник API

Модуль видеокодирования предоставляет функции создания и уничтожения каналов, сброса каналов, запуска и остановки приёма изображений, задания и получения атрибутов канала, получения и освобождения потока и т. д.

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_VENC_SetModParam: задаёт параметры модуля кодирования.
- RK_MPI_VENC_GetModParam: получает параметры модуля кодирования.
- RK_MPI_VENC_CreateChn: создаёт канал кодирования.
- RK_MPI_VENC_DestroyChn: уничтожает канал кодирования.
- RK_MPI_VENC_ResetChn: сбрасывает канал кодирования.
- RK_MPI_VENC_StartRecvFrame: запускает приём входных изображений каналом кодирования.
- RK_MPI_VENC_StopRecvFrame: останавливает приём входных изображений каналом кодирования.
- RK_MPI_VENC_QueryStatus: запрашивает состояние канала кодирования.
- RK_MPI_VENC_SetChnAttr: задаёт атрибуты канала кодирования.
- RK_MPI_VENC_GetChnAttr: получает атрибуты канала кодирования.
- RK_MPI_VENC_SendFrame: отправляет пользовательское исходное изображение на кодирование.
- RK_MPI_VENC_SendFrameEx: отправляет исходное изображение вместе с таблицей QpMap на кодирование.
- RK_MPI_VENC_GetStream: получает закодированный поток.
- RK_MPI_VENC_ReleaseStream: освобождает буфер потока.
- RK_MPI_VENC_SetJpegParam: задаёт набор параметров кодирования JPEG.
- RK_MPI_VENC_GetJpegParam: получает набор параметров кодирования JPEG.
- RK_MPI_VENC_GetRcParam: получает расширенные параметры управления битрейтом канала.
- RK_MPI_VENC_SetRcParam: задаёт расширенные параметры управления битрейтом канала.
- RK_MPI_VENC_RequestIDR: запрашивает IDR-кадр.
- RK_MPI_VENC_GetRoiAttr: получает конфигурацию кодирования области интереса канала.
- RK_MPI_VENC_SetRoiAttr: задаёт конфигурацию кодирования области интереса канала.
- RK_MPI_VENC_SetChnParam: задаёт параметры канала.
- RK_MPI_VENC_GetChnParam: получает параметры канала.
- RK_MPI_VENC_InsertUserData: вставляет пользовательские данные.
- RK_MPI_VENC_SetRcAdvParam: задаёт расширенные параметры модуля RC.
- RK_MPI_VENC_GetRcAdvParam: получает расширенные параметры модуля RC.
- RK_MPI_VENC_GetFd: получает дескриптор файла устройства канала кодирования.
- RK_MPI_VENC_CloseFd: закрывает дескриптор файла устройства канала кодирования.
- RK_MPI_VENC_SetSuperFrameStrategy: задаёт конфигурацию сверхбольших кадров кодирования.
- RK_MPI_VENC_GetSuperFrameStrategy: получает конфигурацию сверхбольших кадров кодирования.
- RK_MPI_VENC_SetFrameLostStrategy: задаёт стратегию потери кадров при превышении порога мгновенного битрейта.
- RK_MPI_VENC_GetFrameLostStrategy: получает стратегию потери кадров при превышении порога мгновенного битрейта.
- RK_MPI_VENC_SetIntraRefresh: задаёт параметры обновления I-slice в P-кадрах.
- RK_MPI_VENC_GetIntraRefresh: получает параметры обновления I-slice в P-кадрах.
- RK_MPI_VENC_SetHierarchicalQp: задаёт параметры иерархического qp.
- RK_MPI_VENC_GetHierarchicalQp: получает параметры иерархического qp.
- RK_MPI_VENC_SetChnRotation: задаёт угол поворота канала.
- RK_MPI_VENC_GetChnRotation: получает угол поворота канала.
- RK_MPI_VENC_AttachMbPool: привязывает канал кодирования к указанному пулу видеобуферов MB.
- RK_MPI_VENC_DetachMbPool: отвязывает канал кодирования от указанного пула видеобуферов MB.
- RK_MPI_VENC_SetH264IntraPred: задаёт атрибуты внутрикадрового предсказания канала H.264.
- RK_MPI_VENC_GetH264IntraPred: получает атрибуты внутрикадрового предсказания канала H.264.
- RK_MPI_VENC_SetH264Trans: задаёт атрибуты преобразования и квантования канала H.264.
- RK_MPI_VENC_GetH264Trans: получает атрибуты преобразования и квантования канала H.264.
- RK_MPI_VENC_SetH264Entropy: задаёт режим энтропийного кодирования канала H.264.
- RK_MPI_VENC_GetH264Entropy: получает режим энтропийного кодирования канала H.264.
- RK_MPI_VENC_SetH264Dblk: задаёт тип Deblocking канала H.264.
- RK_MPI_VENC_GetH264Dblk: получает тип Deblocking канала H.264.
- RK_MPI_VENC_SetH264Vui: задаёт параметры Vui канала H.264.
- RK_MPI_VENC_GetH264Vui: получает параметры Vui канала H.264.
- RK_MPI_VENC_SetH265Trans: задаёт атрибуты преобразования и квантования канала H.265.
- RK_MPI_VENC_GetH265Trans: получает атрибуты преобразования и квантования канала H.265.
- RK_MPI_VENC_SetH265Entropy: задаёт режим энтропийного кодирования канала H.265.
- RK_MPI_VENC_GetH265Entropy: получает режим энтропийного кодирования канала H.265.
- RK_MPI_VENC_SetH265Dblk: задаёт тип Deblocking канала H.265.
- RK_MPI_VENC_GetH265Dblk: получает тип Deblocking канала H.265.
- RK_MPI_VENC_SetH265Sao: задаёт атрибуты Sao канала H.265.
- RK_MPI_VENC_GetH265Sao: получает атрибуты Sao канала H.265.
- RK_MPI_VENC_SetH265PredUnit: задаёт атрибуты PU канала H.265.
- RK_MPI_VENC_GetH265PredUnit: получает атрибуты PU канала H.265.

- RK_MPI_VENC_SetH265Vui: задаёт параметры Vui канала H.265.
- RK_MPI_VENC_GetH265Vui: получает параметры Vui канала H.265.
- RK_MPI_VENC_SetRefParam: задаёт расширенные параметры пропуска кадров и опорных кадров канала H.264/H.265.
- RK_MPI_VENC_GetRefParam: получает расширенные параметры пропуска кадров и опорных кадров канала H.264/H.265.
- RK_MPI_VENC_SetMjpegParam: задаёт расширенные параметры канала MJPEG.
- RK_MPI_VENC_GetMjpegParam: получает расширенную конфигурацию канала MJPEG.
- RK_MPI_VENC_SetQpmap: задаёт MB_BLK Qpmap.
- RK_MPI_VENC_GetQpmap: получает MB_BLK Qpmap.
- RK_MPI_VENC_SetDeBreathEffect: задаёт параметры устранения эффекта дыхания.
- RK_MPI_VENC_GetDeBreathEffect: получает параметры устранения эффекта дыхания.
- RK_MPI_VENC_SetChnRefBufShareAttr: задаёт атрибуты совместного использования опорного буфера канала.
- RK_MPI_VENC_GetChnRefBufShareAttr: получает атрибуты совместного использования опорного буфера канала.
- RK_MPI_VENC_SetComboAttr: задаёт атрибуты Combo канала.
- RK_MPI_VENC_GetComboAttr: получает атрибуты Combo канала.
- RK_MPI_VENC_SetChnBufWrapAttr: задаёт атрибуты кольцевого Buf канала.
- RK_MPI_VENC_GetChnBufWrapAttr: получает атрибуты кольцевого Buf канала.
- RK_MPI_VENC_SetSliceSplit: задаёт атрибуты разбиения slice канала.
- RK_MPI_VENC_GetSliceSplit: получает атрибуты разбиения slice канала.

4.1 RK_MPI_VENC_SetModParam

【Описание】

Задаёт параметры модуля кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetModParam(const VENC_PARAM_MOD_S *pstModParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstModParam	Указатель на параметры модуля кодирования.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Этот интерфейс необходимо вызывать до создания любых каналов кодирования.
- Если pstModParam пуст, возвращается ошибка.
- Если интерфейс не вызывался, значения по умолчанию см. в описании структуры VENC_PARAM_MOD_S.

4.2 RK_MPI_VENC_GetModParam

【Описание】

Получает параметры модуля кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetModParam(VENC_PARAM_MOD_S](#VENC_PARAM_MOD_S) *pstModParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
pstModParam	Указатель на параметры модуля кодирования.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если pstModParam пуст, возвращается ошибка.

4.3 RK_MPI_VENC_CreateChn

【Описание】

Создаёт канал кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_CreateChn(VENC_CHN VeChn, const VENC_CHN_ATTR_S *pstAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstAttr	Указатель на атрибуты канала кодирования.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.4 RK_MPI_VENC_DestroyChn

【Описание】

Уничтожает канал кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_DestroyChn(VENC_CHN VeChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.5 RK_MPI_VENC_ResetChn

【Описание】

Сбрасывает канал.

【Синтаксис】

RK_S32 RK_MPI_VENC_ResetChn(VENC_CHN VeChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.6 RK_MPI_VENC_StartRecvFrame

【Описание】

Запускает приём входных изображений каналом кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_StartRecvFrame(VENC_CHN VeChn, const VENC_RECV_PIC_PARAM_S *pstRecvParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstRecvParam	Указатель на структуру параметров приёма изображений.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.7 RK_MPI_VENC_StopRecvFrame

【Описание】

Останавливает приём входных изображений каналом кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_StopRecvFrame(VENC_CHN VeChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.8 RK_MPI_VENC_QueryStatus

【Описание】

Запрашивает состояние канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_QueryStatus(VENC_CHN VeChn, VENC_CHN_STATUS_S *pstStatus);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstStatus	Указатель на состояние канала кодирования.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.9 RK_MPI_VENC_SetChnAttr

【Описание】

Задаёт атрибуты канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetChnAttr(VENC_CHN VeChn, const VENC_CHN_ATTR_S *pstChnAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstChnAttr	Указатель на атрибуты канала кодирования.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.10 RK_MPI_VENC_GetChnAttr

【Описание】

Получает атрибуты канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetChnAttr(VENC_CHN VeChn, VENC_CHN_ATTR_S *pstChnAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstChnAttr	Указатель на атрибуты канала кодирования.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.11 RK_MPI_VENC_SendFrame

【Описание】

Отправляет пользовательское исходное изображение на кодирование.

【Синтаксис】

RK_S32 RK_MPI_VENC_SendFrame(VENC_CHN VeChn, const VIDEO_FRAME_INFO_S *pstFrame, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Вход/ Тип выхода
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstFrame	Указатель на структуру информации об исходном изображении.	Вход
s32MilliSec	Если параметр тайм-аута s32MilliSec равен -1, интерфейс блокирующий; если 0 — неблокирующий; если больше 0 — время ожидания в миллисекундах (мс).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- К выравниванию ширины и высоты входных данных предъявляются специальные требования; подробности см. в VIDEO_FRAME_INFO_S.

4.12 RK_MPI_VENC_SendFrameEx

【Описание】

Поддерживает отправку исходного изображения и его таблицы QpMap на кодирование.

【Синтаксис】

RK_S32 RK_MPI_VENC_SendFrameEx(VENC_CHN VeChn, const USER_FRAME_INFO_S *pstFrame, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Вход/ Тип выхода
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstFrame	Указатель на структуру информации об исходном изображении.	Вход
s32MilliSec	Если параметр тайм-аута s32MilliSec равен -1, интерфейс блокирующий; если 0 — неблокирующий; если больше 0 — время ожидания в миллисекундах (мс).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Определение QpMap см. в RK_MPI_VENC_SetQpmap.
- Этот интерфейс поддерживается только каналами H.264 и H.265.
- В режиме RC fixqp этот интерфейс не поддерживается.

4.13 RK_MPI_VENC_GetStream

【Описание】

Получает закодированный поток.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetStream(VENC_CHN VeChn, VENC_STREAM_S *pstStream, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Вход/ Тип выхода
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstStream	Указатель на структуру потока.	Выход
s32MilliSec	Если параметр тайм-аута s32MilliSec равен -1, интерфейс блокирующий; если 0 — неблокирующий; если больше 0 — время ожидания в миллисекундах (мс).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Поток можно получать в однопакетном и многопакетном режимах:
  - В однопакетном режиме (режим по умолчанию) каждый вызов возвращает один VENC_PACK_S; данные доступны через pMbBlk и u32Len (см. пример однопакетного режима). VENC_PACK_INFO_S содержит сведения о NAL текущего кадра: для H264 — типы SPS/PPS/SEI, для H265 — VPS/SPS/PPS/SEI.
  - В многопакетном режиме (сейчас поддерживается только H264/H265) каждый вызов возвращает несколько VENC_PACK_S (не более 16). Каждый VENC_PACK_S содержит один пакет NAL либо SLICE (если включён split slice); данные отдельного NAL/SLICE доступны через pMbBlk, u32Len и u32Offset (см. пример многопакетного режима).
- Структура потока VENC_STREAM_S:
  - Указатель на сведения о пакетах потока pstPack Указывает на массив VENC_PACK_S, выделенный вызывающей стороной. В однопакетном режиме его размер должен быть не менее sizeof(VENC_PACK_S); в многопакетном — не менее N × sizeof(VENC_PACK_S), где N — число пакетов текущего кадра (сейчас необходимо выделять для максимума 16 пакетов).
  - Число пакетов потока u32PackCount На входе задаёт число элементов VENC_PACK_S в pstPack. В однопакетном режиме u32PackCount должен быть не меньше 1; в многопакетном режиме — не меньше числа пакетов текущего кадра (сейчас следует задавать максимум 16; слишком малое значение может привести к потере пакетов). После успешного вызова u32PackCount содержит фактическое число заполненных элементов pstPack.

【Пример】

- Однопакетный режим: u32OneStreamBuffer параметров модуля равен 1; режим по умолчанию.

```
    stFrame.pstPack = reinterpret_cast<VENC_PACK_S *>(malloc(sizeof(VENC_PACK_S)));
    while (!pstCtx->threadExit) {
        stFrame.u32PackCount = 1;
        s32Ret = RK_MPI_VENC_GetStream(u32Ch, &stFrame, -1);
        if (s32Ret >= 0) {
            s32StreamCnt++;
```

```
            for (RK_U32 i = 0; i < stFrame.pstPack->u32DataNum; i++) {
                RK_LOGD("get chn %d stream %d index %d type %d offset %d lenth %d", u32Ch,
s32StreamCnt, i,
                        stFrame.pstPack->stPackInfo[i].u32PackType,
                        stFrame.pstPack->stPackInfo[i].u32PackOffset,
                        stFrame.pstPack->stPackInfo[i].u32PackLength);
            }
            if (pstCtx->dstFilePath != RK_NULL) {
                pData = (char *)RK_MPI_MB_Handle2VirAddr(stFrame.pstPack->pMbBlk);
                fwrite(pData, 1, stFrame.pstPack->u32Len, fp);
                fflush(fp);
            }
            RK_MPI_VENC_ReleaseStream(u32Ch, &stFrame);
            if (stFrame.pstPack->bStreamEnd == RK_TRUE) {
                RK_LOGI("chn %d reach EOS stream", u32Ch);
                break;
            }
        } else {
             if (pstCtx->threadExit) {
                break;
             }
             usleep(10001lu);
        }
    }
```

- Многопакетный режим: u32OneStreamBuffer равен 0; сейчас поддерживается только H264/H265.

```
#define MAX_PACKET_NUM           16

stFrame.pstPack = reinterpret_cast<VENC_PACK_S *>(malloc(MAX_PACKET_NUM * sizeof(VENC_PACK_S)));
    while (!pstCtx->threadExit) {
stFrame.u32PackCount = MAX_PACKET_NUM; // входной параметр следует задавать равным 16
        s32Ret = RK_MPI_VENC_GetStream(u32Ch, &stFrame, -1);
        if (s32Ret >= 0) {
            s32StreamCnt++;
            for (RK_U32 i = 0; i < stFrame.u32PackCount; i++) {
                RK_LOGD("get chn(%d) stream(%d) packet(%d) eoi(%d) type(%d) offset(%d)
lenth(%d)",
                        u32Ch, s32StreamCnt, i,
                        stFrame.pstPack[i].bFrameEnd,
                        stFrame.pstPack[i].DataType,
                        stFrame.pstPack[i].u32Offset,
                        stFrame.pstPack[i].u32Len);
                if (pstCtx->dstFilePath != RK_NULL) {
                    pData = (char *)RK_MPI_MB_Handle2VirAddr(stFrame.pstPack[i].pMbBlk);
                    fwrite(pData + stFrame.pstPack[i].u32Offset, 1, stFrame.pstPack[i].u32Len,
fp);
                    fflush(fp);
                }
            }
            RK_MPI_VENC_ReleaseStream(u32Ch, &stFrame);
            if (stFrame.pstPack->bStreamEnd == RK_TRUE) {
                RK_LOGI("chn %d reach EOS stream", u32Ch);
                break;
            }
        } else {
             if (pstCtx->threadExit) {
                break;
             }
             usleep(10001lu);
        }
    }
```

4.14 RK_MPI_VENC_ReleaseStream

【Описание】

Освобождает буфер потока.

【Синтаксис】

RK_S32 RK_MPI_VENC_ReleaseStream(VENC_CHN VeChn, VENC_STREAM_S *pstStream);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstStream	Указатель на структуру потока.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.15 RK_MPI_VENC_SetJpegParam

【Описание】

Задаёт расширенные параметры канала JPEG.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetJpegParam(VENC_CHN VeChn, const VENC_JPEG_PARAM_S *pstJpegParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstJpegParam	Набор расширенных параметров канала JPEG.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.16 RK_MPI_VENC_GetJpegParam

【Описание】

Получает расширенные параметры канала JPEG.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetJpegParam(VENC_CHN VeChn, VENC_JPEG_PARAM_S *pstJpegParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstJpegParam	Набор расширенных параметров канала JPEG.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.17 RK_MPI_VENC_GetRcParam

【Описание】

Получает расширенные параметры управления битрейтом канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetRcParam(VENC_CHN VeChn, VENC_RC_PARAM_S* pstRcParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstRcParam	Указатель на параметры управления битрейтом канала.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.18 RK_MPI_VENC_SetRcParam

【Описание】

Задаёт расширенные параметры контроллера битрейта канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetRcParam(VENC_CHN VeChn, const VENC_RC_PARAM_S *pstRcParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstRcParam	Расширенные параметры контроллера битрейта канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.19 RK_MPI_VENC_RequestIDR

【Описание】

Запрашивает IDR-кадр.

【Синтаксис】

RK_S32 RK_MPI_VENC_RequestIDR(VENC_CHN VeChn, RK_BOOL bInstant);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
bInstant	Следует ли немедленно кодировать IDR-кадр.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Параметр bInstant временно поддерживает только RK_FLASE; при RK_TRUE возвращается RK_ERR_VENC_NOT_SUPPORT.

4.20 RK_MPI_VENC_GetRoiAttr

【Описание】

Получает расширенные атрибуты ROI канала H.264/H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetRoiAttr(VENC_CHN VeChn, VENC_ROI_ATTR_S *pstRoiAttr, RK_S32 roi_index);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstRoiAttr	Конфигурация соответствующей области ROI.	Выход
u32Index	Индекс области ROI канала H.264/H.265.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.21 RK_MPI_VENC_SetRoiAttr

【Описание】

Задаёт расширенные атрибуты ROI канала H.264/H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetRoiAttr(VENC_CHN VeChn, const VENC_ROI_ATTR_S *pstRoiAttr, RK_S32 roi_index);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstRoiAttr	Параметры области ROI.	Вход
u32Index	Индекс области ROI канала H.264/H.265.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.22 RK_MPI_VENC_GetFd

【Описание】

Получает дескриптор файла устройства канала кодирования. По возвращаемому значению можно через select/poll проверять состояние данных канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetFd(VENC_CHN VeChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успех; дескриптор файла устройства канала кодирования.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Для закрытия дескриптора вызовите RK_MPI_VENC_CloseFd.
- Рекомендуется вызывать после создания кодера, но до включения RK_MPI_VENC_StartRecvFrame.

4.23 RK_MPI_VENC_CloseFd

【Описание】

Закрывает дескриптор файла устройства канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_CloseFd(VENC_CHN VeChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Отрицательное значение	Ошибка; см. коды ошибок VENC.

【Примечание】

Нет

4.24 RK_MPI_VENC_SetChnParam

【Описание】

Задаёт параметры канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetChnParam(VENC_CHN VeChn, const VENC_CHN_PARAM_S *pstChnParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstChnParam	Параметры канала Venc.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.

4.25 RK_MPI_VENC_GetChnParam

【Описание】

Получает параметры канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetChnParam(VENC_CHN VeChn, VENC_CHN_PARAM_S *pstChnParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstChnParam	Параметры канала Venc.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.

4.26 RK_MPI_VENC_InsertUserData

【Описание】

Вставляет пользовательские данные.

【Синтаксис】

```
RK_S32 RK_S32 RK_MPI_VENC_InsertUserData(VENC_CHN VeChn, RK_U8 *pu8Data, RK_U32 u32Len);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pu8Data	Указатель на пользовательские данные.	Вход
u32Len	Длина пользовательских данных. Диапазон: (0, 1024], в байтах.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.
- Если pu8Data пуст, возвращается ошибка.
- Вставка пользовательских данных поддерживается только H.264/H.265 и MJPEG/JPEG.
- Канал H.264/H.265 одновременно выделяет не более 4 областей памяти для пользовательских данных, каждая не более 1 КБ. Если пользователь вставляет более 4 блоков либо один блок больше 1 КБ, интерфейс возвращает ошибку. Каждый блок вставляется как пакет SEI перед последним пакетом потока изображения. После кодирования и отправки блока память, использованная каналом H.264/H.265 для его кэширования, обнуляется и может хранить новые пользовательские данные.
- Канал JPEG/MJPEG одновременно выделяет не более 4 областей памяти для пользовательских данных, каждая не более 1 КБ. Если пользователь вставляет более 4 блоков либо один блок больше 1 КБ, интерфейс возвращает ошибку. Данные добавляются в поток изображения как APPsegment (0xFFE7). После их кодирования и отправки память, использованная каналом JPEG/MJPEG для кэширования этого блока, обнуляется и может хранить новые пользовательские данные.

4.27 RK_MPI_VENC_SetRcAdvParam

【Описание】

Задаёт расширенные параметры модуля RC.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetRcAdvParam(VENC_CHN VeChn, const VENC_RC_ADVPARAM_S *pstRcAdvParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstRcAdvParam	Расширенные параметры RC. Интерфейс содержит параметры, слабо связанные с алгоритмом управления битрейтом, и в будущем может расширяться.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.
- Этот интерфейс используется для задания расширенных параметров RC.
  - u32ClearStatAfterSetAttr: очищать ли статистику управления битрейтом после задания нового битрейта канала; по умолчанию 1. Сейчас поддерживается только очистка

статистики управления битрейтом.

4.28 RK_MPI_VENC_GetRcAdvParam

【Описание】

Получает расширенные параметры модуля RC.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetRcAdvParam(VENC_CHN VeChn, VENC_RC_ADVPARAM_S *pstRcAdvParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstRcAdvParam	Расширенные параметры модуля RC.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.

4.29 RK_MPI_VENC_SetSuperFrameStrategy

【Описание】

Задаёт конфигурацию сверхбольших кадров кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetSuperFrameStrategy(VENC_CHN VeChn, const VENC_SUPERFRAME_CFG_S *pstSuperFrmParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstSuperFrmParam	Параметры сверхбольших кадров кодирования.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.
- Это расширенный необязательный интерфейс; по умолчанию функция отключена.

4.30 RK_MPI_VENC_GetSuperFrameStrategy

【Описание】

Получает параметры канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetSuperFrameStrategy(VENC_CHN VeChn, VENC_SUPERFRAME_CFG_S *pstSuperFrmParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstSuperFrmParam	Параметры сверхбольших кадров кодирования.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstSuperFrmParam пуст, возвращается ошибка.

4.31 RK_MPI_VENC_SetFrameLostStrategy

【Описание】

Задаёт стратегию потери кадров при превышении порога мгновенного битрейта канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetFrameLostStrategy(VENC_CHN VeChn, const VENC_FRAMELOST_S *pstFrmLostParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstFrmLostParam	Параметры стратегии потери кадров канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.
- pstFrmLostParam определяется четырьмя основными параметрами:
  - bFrmLostOpen: переключатель потери кадров;
  - u32FrmLostBpsThr: процентный порог потери кадров;
  - enFrmLostMode: режим стратегии потери кадров;
  - u32EncFrmGaps: максимальное допустимое число последовательных потерянных кадров.

- Это расширенный необязательный интерфейс. Система имеет значения по умолчанию и по умолчанию при превышении мгновенного битрейта отбрасывает кадры.
- При превышении порога поддерживаются два способа: потеря кадра и кодирование pskip-кадра. Mjpeg поддерживает только потерю кадров.

4.32 RK_MPI_VENC_GetFrameLostStrategy

【Описание】

Получает стратегию потери кадров при превышении порога мгновенного битрейта канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetFrameLostStrategy(VENC_CHN VeChn, VENC_FRAMELOST_S *pstFrmLostParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstFrmLostParam	Параметры стратегии потери кадров канала.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstFrmLostParam пуст, возвращается ошибка.

4.33 RK_MPI_VENC_SetIntraRefresh

【Описание】

Задаёт параметры обновления I-slice в P-кадрах.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetIntraRefresh(VENC_CHN VeChn, const VENC_INTRA_REFRESH_S *pstIntraRefresh);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstIntraRefresh	Параметры обновления I-slice в P-кадрах.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.
- Это расширенный необязательный интерфейс; по умолчанию функция отключена.

4.34 RK_MPI_VENC_GetIntraRefresh

【Описание】

Получает параметры обновления I-slice в P-кадрах.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetIntraRefresh(VENC_CHN VeChn, VENC_INTRA_REFRESH_S *pstIntraRefresh);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstIntraRefresh	Параметры обновления I-slice.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstIntraRefresh пуст, возвращается ошибка.

4.35 RK_MPI_VENC_SetHierarchicalQp

【Описание】

Задаёт параметры иерархического qp.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetHierarchicalQp(VENC_CHN VeChn, const VENC_HIERARCHICAL_QP_S *pstHierarchicalQp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstHierarchicalQp	Параметры иерархического qp.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.
- Это расширенный необязательный интерфейс; по умолчанию функция отключена.
- Иерархический qp поддерживается только кодированием H.264/H.265.
- Перед вызовом рекомендуется сначала вызвать RK_MPI_VENC_GetHierarchicalQp, а затем изменить полученные параметры.
- Функция главным образом определяется следующими параметрами.

- bHierarchicalQpEn: включает иерархический qp.
- s32HierarchicalQpDelta: задаёт QpDelta P-кадров каждого слоя относительно P-кадра слоя 0. Поскольку qp P-кадра слоя 0 не корректируется, s32HierarchicalQpDelta[0] относится к слою 1, s32HierarchicalQpDelta[1] — к слою 2 и т. д.
- s32HierarchicalFrameNum: задаёт число кадров каждого слоя; s32HierarchicalFrameNum[0] относится к слою 1, s32HierarchicalFrameNum[1] — к слою 2 и т. д.

- Иерархический QP имеет два варианта: для Normal P и Smart P. Основное отличие — дополнительный виртуальный I-кадр в Smart P; его приоритет выше, чем у других P-кадров. P0, P1 и P2 обозначают P-кадры слоёв 0, 1 и 2. Схема иерархического QP приведена ниже:

Схема иерархического QP в режиме Normal P

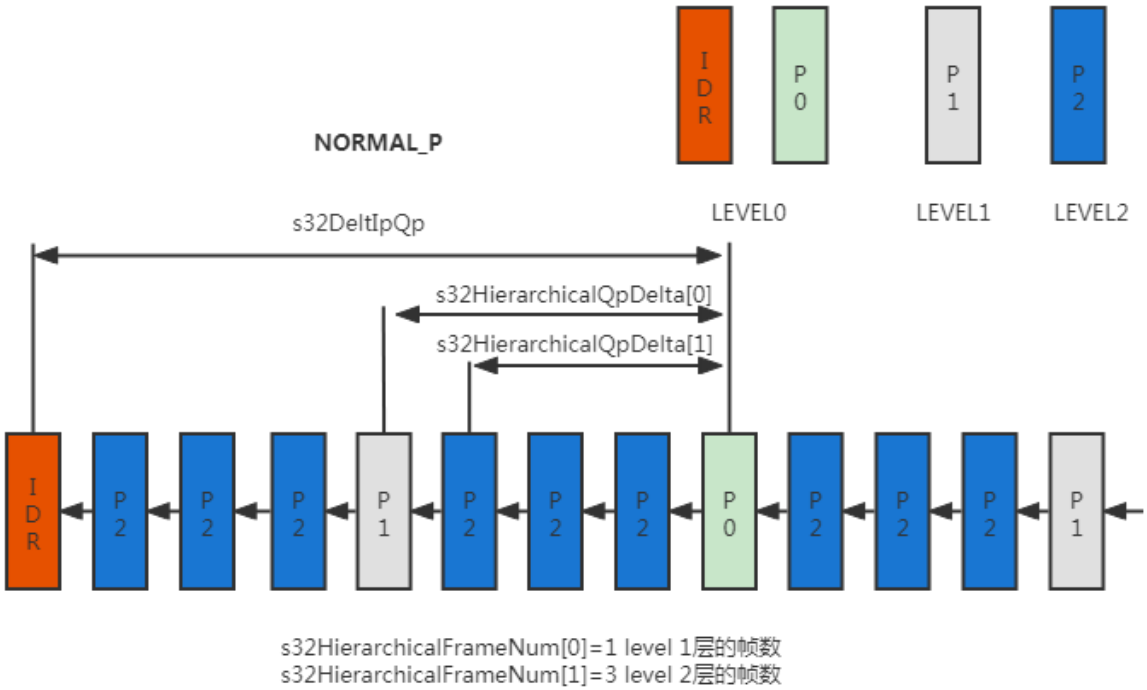
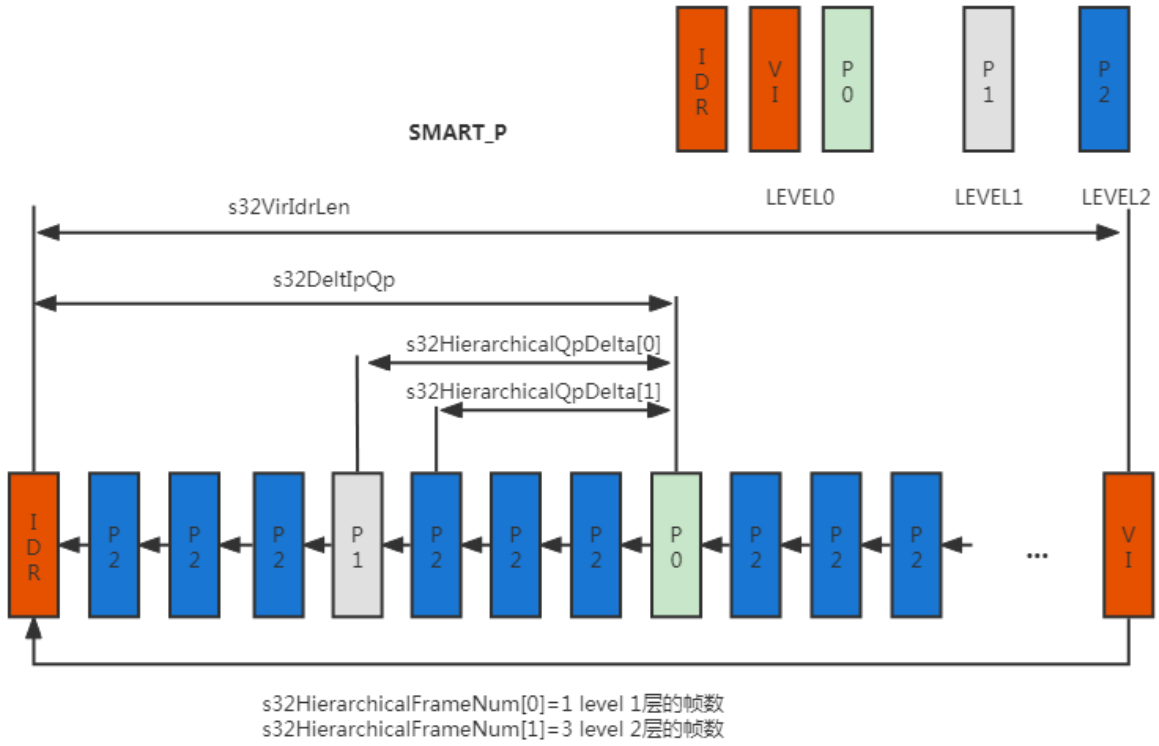


Схема иерархического QP в режиме Smart P



4.36 RK_MPI_VENC_GetHierarchicalQp

【Описание】

Получает параметры иерархического qp.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetHierarchicalQp(VENC_CHN VeChn, VENC_HIERARCHICAL_QP_S *pstHierarchicalQp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstHierarchicalQp	Параметры иерархического qp.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstHierarchicalQp пуст, возвращается ошибка.
- Получение параметров иерархического qp поддерживается только H.264/H.265.

4.37 RK_MPI_VENC_SetChnRotation

【Описание】

Задаёт угол поворота канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetChnRotation(VENC_CHN VeChn, ROTATION_E enRotation);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
enRotation	Угол поворота при кодировании.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан, возвращается ошибка.
- При кодировании jpeg/mjpeg и повороте на 90/270 ширина и высота входного изображения должны быть выровнены по 16.

4.38 RK_MPI_VENC_GetChnRotation

【Описание】

Получает угол поворота канала.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetChnRotation(VENC_CHN VeChn, ROTATION_E *enRotation);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
enRotation	Угол поворота при кодировании.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо enRotation пуст, возвращается ошибка.

4.39 RK_MPI_VENC_AttachMbPool

【Описание】

Привязывает канал кодирования к указанному пулу видеобуферов MB.

【Синтаксис】

RK_S32 RK_MPI_VENC_AttachMbPool(VENC_CHN VeChn, MB_POOL hMbPool)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
hMbPool	Сведения о пуле видеобуферов MB.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Канал должен быть создан, иначе возвращается RK_ERR_VENC_UNEXIST.
- Параметр hMbPool должен быть созданным и действительным пулом памяти MB, иначе возвращается RK_ERR_VENC_ILLEGAL_PARAM.
- Пользователь создаёт пул видеобуферов MB вызовом RK_MPI_MB_CreatePool, затем вызывает RK_MPI_VENC_AttachMbPool, чтобы привязать созданный пул памяти MB к каналу кодирования.
- Привязывать один канал кодирования к нескольким пулам MB запрещено.
- Привязывать несколько каналов кодирования к одному пулу MB запрещено.
- Этот интерфейс позволяет сократить время запуска канала кодирования.
- Интерфейс необходимо вызывать до RK_MPI_VENC_StartRecvFrame; динамический вызов во время кодирования не поддерживается.

4.40 RK_MPI_VENC_DetachMbPool

【Описание】

Отвязывает канал кодирования от указанного пула видеобуферов MB.

【Синтаксис】

RK_S32 RK_MPI_VENC_DetachMbPool(VENC_CHN VeChn)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Канал должен быть создан, иначе возвращается RK_ERR_VENC_UNEXIST.
- Интерфейс необходимо использовать после RK_MPI_VENC_StopRecvFrame; динамический вызов во время кодирования не поддерживается.

4.41 RK_MPI_VENC_SetH264IntraPred

【Описание】

Задаёт атрибуты внутрикадрового предсказания канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH264IntraPred(VENC_CHN VeChn, const VENC_H264_INTRA_PRED_S *pstH264IntraPred);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264IntraPred	Конфигурация внутрикадрового предсказания канала H.264.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264IntraPred пуст, возвращается ошибка.
- Из атрибутов внутрикадрового предсказания поддерживается только constrained_intra_pred_flag; его значение см. в спецификации H.264.
- Это расширенный необязательный интерфейс, вызывать его не рекомендуется: система имеет значение по умолчанию. По умолчанию constrained_intra_pred_flag равен 0.

  0。
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- Перед заданием рекомендуется вызвать RK_MPI_VENC_GetH264IntraPred, получить текущую конфигурацию IntraPred канала, а затем изменить её.

4.42 RK_MPI_VENC_GetH264IntraPred

【Описание】

Получает атрибуты внутрикадрового предсказания канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH264IntraPred(VENC_CHN VeChn, VENC_H264_INTRA_PRED_S *pstH264IntraPred);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264IntraPred	Конфигурация внутрикадрового предсказания канала H.264.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264IntraPred пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.43 RK_MPI_VENC_SetH264Trans

【Описание】

Задаёт атрибуты преобразования и квантования канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH264Trans(VENC_CHN VeChn, const VENC_H264_TRANS_S *pstH264Trans);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264Trans	Атрибуты преобразования и квантования канала H.264.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264Trans пуст, возвращается ошибка.
- Атрибуты преобразования и квантования в основном состоят из пяти параметров:
  - u32TransMode: атрибут преобразования макроблоков внутрикадрового/межкадрового предсказания. u32TransMode = 0 разрешает для них преобразования 4x4 и 8x8; u32TransMode = 1 разрешает только преобразование 4x4.

    - bScalingListValid: действительны ли таблицы квантования InterScalingList8x8 и IntraScalingList8x8. Устанавливать bScalingListValid в true не поддерживается. Зарезервировано, сейчас не используется.
    - InterScalingList8x8: пользовательская таблица квантования для преобразования 8x8 макроблоков межкадрового предсказания. Зарезервировано, сейчас не используется.
    - IntraScalingList8x8: пользовательская таблица квантования для преобразования 8x8 макроблоков внутрикадрового предсказания. Зарезервировано, сейчас не используется.
    - chroma_qp_index_offset: значение по умолчанию -6; подробное значение см. в спецификации H.264.
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- Перед заданием рекомендуется вызвать RK_MPI_VENC_GetH264Trans, получить текущую конфигурацию trans канала, а затем изменить её.

4.44 RK_MPI_VENC_GetH264Trans

【Описание】

Получает атрибуты преобразования и квантования канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH264Trans(VENC_CHN VeChn, VENC_H264_TRANS_S *pstH264Trans);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264Trans	Атрибуты преобразования и квантования канала H.264.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264Trans пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.45 RK_MPI_VENC_SetH264Entropy

【Описание】

Задаёт режим энтропийного кодирования канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH264Entropy(VENC_CHN VeChn, const VENC_H264_ENTROPY_S *pstH264EntropyEnc);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264EntropyEnc	Режим энтропийного кодирования канала H.264.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264EntropyEnc пуст, возвращается ошибка.
- Атрибуты энтропийного кодирования в основном состоят из двух параметров:
  - u32EntropyEncMode: способ энтропийного кодирования. 0 — cavlc, 1 — cabac. По умолчанию 1, то есть cabac.
  - Cabac_init_idc: индекс инициализации cabac; системное значение по умолчанию 0. Подробности см. в спецификации H.264.
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- Перед заданием рекомендуется вызвать RK_MPI_VENC_GetH264Entropy, получить текущую конфигурацию entropy канала, а затем изменить её.

4.46 RK_MPI_VENC_GetH264Entropy

【Описание】

Получает режим энтропийного кодирования канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH264Entropy(VENC_CHN VeChn, VENC_H264_ENTROPY_S *pstH264EntropyEnc);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264EntropyEnc	Режим энтропийного кодирования канала H.264.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264EntropyEnc пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.47 RK_MPI_VENC_SetH264Dblk

【Описание】

Задаёт тип Deblocking канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH264Dblk(VENC_CHN VeChn, const VENC_H264_DBLK_S *pstH264Dblk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264Dblk	Тип Deblocking канала H.264.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264Dblk пуст, возвращается ошибка.
- Атрибуты Deblocking в основном состоят из трёх параметров:
  - disable_deblocking_filter_idc: подробности см. в спецификации H.264.
  - slice_alpha_c0_offset_div2: подробности см. в спецификации H.264.
  - slice_beta_offset_div2: подробности см. в спецификации H.264.
- По умолчанию функция deblocking включена: disable_deblocking_filter_idc = 0, slice_alpha_c0_offset_div2 = 0, slice_beta_offset_div2 = 0。
- slice_beta_offset_div2 = 0. Чтобы отключить deblocking, установите disable_deblocking_filter_idc = 1.
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- Перед заданием рекомендуется вызвать RK_MPI_VENC_GetH264Dblk, получить текущую конфигурацию dblk канала, а затем изменить её.

4.48 RK_MPI_VENC_GetH264Dblk

【Описание】

Получает тип Deblocking канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH264Dblk(VENC_CHN VeChn, VENC_H264_DBLK_S *pstH264Dblk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264Dblk	Тип Deblocking канала H.264.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264Dblk пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.49 RK_MPI_VENC_SetH264Vui

【Описание】

Задаёт параметры Vui канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH264Vui(VENC_CHN VeChn, const VENC_H264_VUI_S*pstH264Vui);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264Vui	Параметры Vui канала H.264.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264Vui пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.50 RK_MPI_VENC_GetH264Vui

【Описание】

Получает параметры Vui канала H.264.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH264Vui(VENC_CHN VeChn, VENC_H264_VUI_S *pstH264Vui);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH264Vui	Параметры Vui канала H.264.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH264Vui пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.51 RK_MPI_VENC_SetH265Trans

【Описание】

Задаёт атрибуты преобразования и квантования канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH265Trans(VENC_CHN VeChn, const VENC_H265_TRANS_S *pstH265Trans);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Trans	Атрибуты преобразования и квантования канала H.265.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Trans пуст, возвращается ошибка.
- Атрибуты преобразования и квантования в основном состоят из следующих параметров:
  - cb_qp_offset: значение по умолчанию -6; подробности см. в описании pps_cb_qp_offset спецификации H.265.
  - cr_qp_offset: значение по умолчанию -6; подробности см. в описании pps_cr_qp_offset спецификации H.265.
  - bScalingListEnabled: действительна ли таблица квантования; при её отсутствии устанавливается 0. Зарезервировано, сейчас не используется.
  - bScalingListTu4Valid: действительны ли InterScalingList4X4 и IntraScalingList4X4; при использовании таблиц протокола по умолчанию устанавливается 0. Зарезервировано, сейчас не используется.
  - InterScalingList4X4[2][16]: InterScalingList4X4[0][16] — таблица квантования яркости межкадрового предсказания, InterScalingList4X4[1][16] — таблица квантования цветности межкадрового предсказания. Пользователь может задать таблицы через этот массив. Зарезервировано, сейчас не используется.
  - IntraScalingList4X4[2][16]: IntraScalingList4X4[0][16] — таблица квантования яркости внутрикадрового предсказания, IntraScalingList4X4[1][16] — таблица квантования цветности внутрикадрового предсказания. Пользователь может задать таблицы через этот массив.
  - IbScalingListTu8Valid: действительны ли InterScalingList8X8 и IntraScalingList8X8; при использовании таблиц протокола по умолчанию устанавливается 0. 0. Зарезервировано, сейчас не используется.
  - InterScalingList8X8[2][64]: InterScalingList8X8[0][64] — таблица квантования яркости межкадрового предсказания, InterScalingList8X8[1][64] — таблица квантования цветности межкадрового предсказания. Пользователь может задать таблицы через этот массив. Зарезервировано, сейчас не используется.
  - IntraScalingList8X8[2][64]: IntraScalingList8X8[0][64] — таблица квантования яркости внутрикадрового предсказания, IntraScalingList8X8[1][64] — таблица квантования цветности внутрикадрового предсказания. Пользователь может задать таблицы через этот массив. Зарезервировано, сейчас не используется.
  - bScalingListTu16Valid: действительны ли InterScalingList16X16 и IntraScalingList16X16; при использовании таблиц протокола по умолчанию устанавливается 0. Зарезервировано, сейчас не используется.
  - InterScalingList16X16[2][64]: InterScalingList16X16[0][64] — таблица квантования яркости межкадрового предсказания, InterScalingList16X16[1][64] — таблица квантования цветности межкадрового предсказания. Пользователь может задать таблицы через этот массив. Зарезервировано, сейчас не используется.
  - IntraScalingList16X16[2][64]: IntraScalingList16X16[0][64] — таблица квантования яркости внутрикадрового предсказания, IntraScalingList16X16[1][64] — таблица квантования цветности внутрикадрового предсказания. Пользователь может задать таблицы через этот массив. Зарезервировано, сейчас не используется.
  - bScalingListTu32Valid: действительны ли InterScalingList32X32 и IntraScalingList32X32; при использовании таблиц протокола по умолчанию устанавливается 0. Зарезервировано, сейчас не используется.
  - InterScalingList32X32[64]: таблица квантования яркости межкадрового предсказания. Пользователь может задать её через этот массив. Зарезервировано, сейчас не используется.
  - IntraScalingList32X32[64]: таблица квантования яркости внутрикадрового предсказания. Пользователь может задать её через этот массив. Зарезервировано, сейчас не используется. сейчас не используется.
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

- Перед заданием рекомендуется вызвать RK_MPI_VENC_GetH265Trans, получить текущую конфигурацию trans канала, а затем изменить её. изменить её.

4.52 RK_MPI_VENC_GetH265Trans

【Описание】

Получает атрибуты преобразования и квантования канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH265Trans(VENC_CHN VeChn, VENC_H265_TRANS_S *pstH265Trans);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Trans	Атрибуты преобразования и квантования канала H.265.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Trans пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.53 RK_MPI_VENC_SetH265Entropy

【Описание】

Задаёт атрибуты энтропийного кодирования канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH265Entropy(VENC_CHN VeChn, const VENC_H265_ENTROPY_S *pstH265Entropy);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Entropy	Атрибуты энтропийного кодирования канала H.265.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Entropy пуст, возвращается ошибка.

- Атрибуты энтропийного кодирования в основном определяются одним параметром:

  cabac_init_flag: подробности см. в спецификации H.265. Зарезервировано, сейчас не используется.

- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- Перед заданием рекомендуется вызвать RK_MPI_VENC_GetH265Entropy, получить текущую конфигурацию entropy канала, а затем изменить её. изменить её.

4.54 RK_MPI_VENC_GetH265Entropy

【Описание】

Получает атрибуты энтропийного кодирования канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH265Entropy(VENC_CHN VeChn, VENC_H265_ENTROPY_S *pstH265Entropy);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Entropy	Атрибуты энтропийного кодирования канала H.265.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Entropy пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.55 RK_MPI_VENC_SetH265Dblk

【Описание】

Задаёт атрибуты Deblocking канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH265Dblk(VENC_CHN VeChn, const VENC_H265_DBLK_S *pstH265Dblk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Dblk	Атрибуты Deblocking канала H.265.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Dblk пуст, возвращается ошибка.
- Атрибуты Deblocking в основном определяются тремя параметрами:
  - slice_deblocking_filter_disabled_flag: подробности см. в спецификации H.265.
  - slice_beta_offset_div2: подробности см. в спецификации H.265.
  - slice_tc_offset_div2: подробности см. в спецификации H.265.
- Это расширенный необязательный интерфейс; вызывать его не рекомендуется. По умолчанию функция deblocking включена. slice_deblocking_filter_disabled_flag=0，slice_tc_offset_div2=0，slice_beta_offset_div2=0。
- Чтобы отключить deblocking, установите slice_deblocking_filter_disabled_flag = 1.
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- Перед заданием рекомендуется вызвать RK_MPI_VENC_GetH265Dblk, получить текущую конфигурацию dblk канала, а затем изменить её.

4.56 RK_MPI_VENC_GetH265Dblk

【Описание】

Получает атрибуты Deblocking канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH265Dblk(VENC_CHN VeChn, VENC_H265_DBLK_S *pstH265Dblk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Dblk	Атрибуты Deblocking канала H.265.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Dblk пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.57 RK_MPI_VENC_SetH265Sao

【Описание】

Задаёт атрибуты Sao канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH265Sao(VENC_CHN VeChn, const VENC_H265_SAO_S *pstH265Sao);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Sao	Конфигурация Sao канала H.265.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Sao пуст, возвращается ошибка.
- Атрибуты Sao в основном определяются двумя параметрами:
  - slice_sao_luma_flag: применять ли фильтрацию Sao к компоненту яркости текущего slice.
  - slice_sao_chroma_flag: применять ли фильтрацию Sao к компоненту цветности текущего slice.
- Это расширенный необязательный интерфейс; вызывать его не рекомендуется. По умолчанию sao включён: slice_sao_luma_flag = 1, slice_sao_chroma_flag = 1。
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- slice_sao_chroma_flag = 1. Перед заданием рекомендуется вызвать RK_MPI_VENC_GetH265Sao, получить текущую конфигурацию sao канала, а затем изменить её.

4.58 RK_MPI_VENC_GetH265Sao

【Описание】

Получает атрибуты Sao канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH265Sao(VENC_CHN VeChn, VENC_H265_SAO_S *pstH265Sao);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Sao	Конфигурация Sao канала H.265.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Sao пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.59 RK_MPI_VENC_SetH265PredUnit

【Описание】

Задаёт атрибуты PU канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH265PredUnit(VENC_CHN VeChn, const VENC_H265_PU_S *pstPredUnit);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstPredUnit	Конфигурация PU канала H.265.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstPredUnit пуст, возвращается ошибка.
- Атрибуты PU в основном определяются двумя параметрами:
  - constrained_intra_pred_flag: подробности см. в спецификации H.265. Зарезервировано, сейчас не используется.
  - strong_intra_smoothing_enabled_flag: подробности см. в спецификации H.265.
- Это расширенный необязательный интерфейс; вызывать его не рекомендуется. По умолчанию constrained_intra_pred_flag = 0, strong_intra_smoothing_enabled_flag=1。
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- strong_intra_smoothing_enabled_flag = 1. Перед заданием рекомендуется вызвать RK_MPI_VENC_GetH265PredUnit, получить текущую конфигурацию pu канала, а затем изменить её. изменить её.

4.60 RK_MPI_VENC_GetH265PredUnit

【Описание】

Получает атрибуты PU канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH265PredUnit(VENC_CHN VeChn, VENC_H265_PU_S *pstPredUnit);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstPredUnit	Конфигурация PU канала H.265.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstPredUnit пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.61 RK_MPI_VENC_SetH265Vui

【Описание】

Задаёт параметры Vui канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetH265Vui(VENC_CHN VeChn, const VENC_H265_VUI_S*pstH265Vui);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Vui	Параметры Vui канала H.265.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Vui пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.62 RK_MPI_VENC_GetH265Vui

【Описание】

Получает параметры Vui канала H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetH265Vui(VENC_CHN VeChn, VENC_H265_VUI_S *pstH265Vui);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstH265Vui	Параметры Vui канала H.265.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstH265Vui пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.63 RK_MPI_VENC_SetRefParam

【Описание】

Задаёт расширенные параметры пропуска кадров и опорных кадров канала H.264/H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetRefParam(VENC_CHN VeChn, const VENC_REF_PARAM_S *pstRefParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstRefParam	Расширенные параметры пропуска кадров и опорных кадров канала H.264/H.265.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstRefParam пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала. При вызове во время кодирования вставляется I-кадр, после чего настройка вступает в силу.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.64 RK_MPI_VENC_GetRefParam

【Описание】

Получает расширенные параметры пропуска кадров и опорных кадров канала H.264/H.265.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetRefParam(VENC_CHN VeChn, VENC_REF_PARAM_S *pstRefParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstRefParam	Расширенные параметры пропуска кадров и опорных кадров канала H.264/H.265.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstRefParam пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.65 RK_MPI_VENC_SetMjpegParam

【Описание】

Задаёт расширенные параметры канала MJPEG.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetMjpegParam(VENC_CHN VeChn, VENC_MJPEG_PARAM_S *pstMjpegParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstMjpegParam	Набор расширенных параметров канала MJPEG.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstMjpegParam пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- Этот параметр имеет приоритет над q_factor; после вызова интерфейса q_factor не действует.
- Расширенные параметры в основном состоят из четырёх параметров:
  - u8YQt[64], u8CbQt[64], u8CrQt[64]: три массива таблиц квантования, через которые пользователь задаёт собственные таблицы.
  - u32MCUPerECS: число MCU в каждом ECS. В системном режиме u32MCUPerECS = 0, то есть все MCU текущего кадра кодируются как один ECS. Минимум u32MCUPerECS равен 0, максимум не превышает ((picwidth+15)>>4) * ((picheight+15)>>4) для yuv420sp.

    (picwidth+15)>>4 * (picheight+15)>>4 * 2(yuv422sp)、(picwidth+15)>>4 * (picheight+15)>>4 * 4（yuv444sp）。

4.66 RK_MPI_VENC_GetMjpegParam

【Описание】

Получает расширенную конфигурацию канала MJPEG.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetMjpegParam(VENC_CHN VeChn, VENC_MJPEG_PARAM_S *pstMjpegParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstMjpegParam	Расширенная конфигурация канала MJPEG.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

Примечание

- Если канал не создан либо pstMjpegParam пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.67 RK_MPI_VENC_SetQpmap

Описание

Задаёт MB_BLK Qpmap.

Синтаксис

```
RK_S32 RK_MPI_VENC_SetQpmap(VENC_CHN VeChn, const MB_BLK blk);
```

Параметры

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
blk	MB_BLK Qpmap.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

Примечание

- QPMAP имеет приоритет над ROI; если настроены оба, действует только QPMAP.
- RK_MPI_VENC_SetQpmap имеет более низкий приоритет, чем RK_MPI_VENC_SendFrameEx.
- В режиме QPMAP пользователь самостоятельно определяет стратегию управления качеством. Необходимо выделить память, отображаемую на кодируемое изображение, и значениями в ней задать qp каждого блока изображения. Qp задаётся блоками 16 * 16; каждый блок использует значение Qp соответствующего пользовательского элемента. Значения Qp всех блоков образуют таблицу Qp со следующей организацией:

- Расположение MB H.264 QPMAP

Расположение MB (16 * 16) H.264 QPMAP:

Пример: ширина изображения 9 * 16 пикселей, высота 4 * 16 пикселей.

0	1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35

Значение QPMAP каждого MB H.264 занимает 16 бит:

[15]: флаг абсолютного qp; 1 — абсолютный qp, 0 — относительный qp;

[14:8]: значение qp; абсолютный qp: [0, 51], относительный qp: [-32, 31];

[7:0]: должно быть 0x80.

Порядок размещения значений MB QPMAP из таблицы в памяти (одно значение занимает 2 байта, little-endian):

0	1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35

Требуемый объём памяти: 9 * 4 * 2 = 72 байта.

- Расположение MB H.265 QPMAP (RV1109, RV1126, RK3568, RK3588)

Расположение MB (16 * 16) и LCU (64 * 64) H.265 QPMAP:

Пример: ширина изображения 9 * 16 пикселей, высота 5 * 16 пикселей.

0	1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44

Значение QPMAP каждого MB H.265 занимает 16 бит:

[15]: флаг абсолютного qp; 1 — абсолютный qp, 0 — относительный qp;

[14:8]: значение qp; абсолютный qp: [0, 51], относительный qp: [-32, 31];

[7:0]: должно быть 0x80.

Порядок размещения значений MB QPMAP из таблицы в памяти (одно значение занимает 2 байта, little-endian):

0	1	2	3	9	10	11	12	18	19	20	21	27	28	29	30
4	5	6	7	13	14	15	16	22	23	24	25	31	32	33	34
8	x	x	x	17	x	x	x	26	x	x	x	35	x	x	x
36	37	38	39	x	x	x	x	x	x	x	x	x	x	x	x
40	41	42	43	x	x	x	x	x	x	x	x	x	x	x	x
44	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

Значения QPMAP выравниваются по 4, x обозначает произвольное значение. Требуемый объём: 16 * 6 * 2 = 192 байта.

- Расположение MB H.265 QPMAP (RV1106)

Расположение MB (16 * 16) и LCU (32 * 32) H.265 QPMAP:

Пример: ширина изображения 9 * 16 пикселей, высота 5 * 16 пикселей.

0	1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44

Значение QPMAP каждого MB H.265 занимает 16 бит:

[15]: флаг абсолютного qp; 1 — абсолютный qp, 0 — относительный qp;

[14:8]: значение qp; абсолютный qp: [0, 51], относительный qp: [-32, 31];

[7:0]: должно быть 0x80.

Порядок размещения значений MB QPMAP из таблицы в памяти (одно значение занимает 2 байта, little-endian):

0	1	9	10	2	3	11	12	4	5	13	14
6	7	15	16	8	x	17	x	18	19	27	28
20	21	29	30	22	23	31	32	24	25	33	34
26	x	35	x	36	37	x	x	38	39	x	x
40	41	x	x	42	43	x	x	44	x	x	x

Значения QPMAP выравниваются по 2, x обозначает произвольное значение. Требуемый объём: 12 * 5 * 2 = 120 байт.

4.68 RK_MPI_VENC_GetQpmap

【Описание】

Получает MB_BLK Qpmap.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetQpmap(VENC_CHN VeChn, MB_BLK *pBlk);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pBlk	MB_BLK Qpmap.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Нет

4.69 RK_MPI_VENC_SetDeBreathEffect

【Описание】

Задаёт параметры устранения эффекта дыхания.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetDeBreathEffect(VENC_CHN VeChn, const VENC_DEBREATHEFFECT_S *pstDeBreathEffect)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstDeBreathEffect	Параметры устранения эффекта дыхания.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Сейчас функция поддерживается только RK3588, RV1106 и RV1103 при обычном режиме входа VI. Онлайн-режим VI её не поддерживает.
- Функция устранения эффекта дыхания поддерживается только H264/H265.
- Канал должен быть создан, иначе возвращается код ошибки несуществующего канала VENC.
- Интерфейс задаётся после создания и до уничтожения канала. При вызове во время кодирования настройка вступает в силу со следующего кадра.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.
- При GOP = 1 включать устранение эффекта дыхания запрещено.
- Если включено обновление I-slice в P-кадрах, сначала отключите его и только затем включайте устранение эффекта дыхания, иначе возвращается ошибка недопустимой операции.
- Перед заданием рекомендуется вызвать RK_MPI_VENC_GetDeBreathEffect, получить текущие параметры, а затем изменить их.
- Функция главным образом определяется тремя параметрами:
  - bEnable: включение устранения эффекта дыхания.
  - s32Strength0: параметр регулировки интенсивности 0; не используется.
  - s32Strength1: параметр регулировки интенсивности 1, значение по умолчанию 16. Чем больше значение, тем больше I-кадр; чем меньше — тем меньше I-кадр. Если увеличение I-кадра мешает приложению, можно уменьшить s32Strength1: размер потока I-кадра снизится, но также может уменьшиться эффект подавления дыхания.

4.70 RK_MPI_VENC_GetDeBreathEffect

【Описание】

Получает параметры устранения эффекта дыхания.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetDeBreathEffect(VENC_CHN VeChn, VENC_DEBREATHEFFECT_S *pstDeBreathEffect)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstDeBreathEffect	Параметры устранения эффекта дыхания.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Сейчас функция поддерживается только RK3588, RV1106 и RV1103 при обычном режиме входа VI. Онлайн-режим VI её не поддерживает.
- Функция устранения эффекта дыхания поддерживается только H264/H265.
- Интерфейс вызывается после создания и до уничтожения канала.

4.71 RK_MPI_VENC_SetChnRefBufShareAttr

【Описание】

Задаёт атрибуты совместного использования опорного буфера канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetChnRefBufShareAttr(VENC_CHN VeChn, const VENC_CHN_REF_BUF_SHARE_S *pstVencChnRefBufShare)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstVencChnRefBufShare	Указатель на атрибуты совместного использования опорного буфера.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Поддерживается только RV1106 и RV1103.
- Поддерживается только H264 и H265.
- Включение совместного использования опорных кадров экономит память, но не поддерживает повторное кодирование сверхбольших кадров.

4.72 RK_MPI_VENC_GetChnRefBufShareAttr

【Описание】

Получает атрибуты совместного использования опорного буфера канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetChnRefBufShareAttr(VENC_CHN VeChn, VENC_CHN_REF_BUF_SHARE_S *pstVencChnRefBufShare);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstVencChnRefBufShare	Указатель на атрибуты совместного использования опорного буфера.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Поддерживается только RV1106 и RV1103.
- Поддерживается только H264 и H265.

4.73 RK_MPI_VENC_SetComboAttr

【Описание】

Задаёт атрибуты Combo канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetComboAttr(VENC_CHN VeChn, VENC_COMBO_ATTR_S *pstComboAttr)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstComboAttr	Указатель на атрибуты Combo.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Поддерживается только RV1106 и RV1103.
- Поддерживается только MJPEG и JPEG.
- После задания Combo канал кодирования не поддерживает bind. Источник входа совпадает с источником Combo-канала. Поддерживается Combo с каналом H264/H265; разрешение двух Combo-каналов должно совпадать.

4.74 RK_MPI_VENC_GetComboAttr

【Описание】

Получает атрибуты Combo канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetComboAttr(VENC_CHN VeChn, VENC_COMBO_ATTR_S *pstComboAttr)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstComboAttr	Указатель на атрибуты Combo.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Поддерживается только RV1106 и RV1103.
- Поддерживается только MJPEG и JPEG.

4.75 RK_MPI_VENC_SetChnBufWrapAttr

【Описание】

Задаёт атрибуты кольцевого Buf канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetChnBufWrapAttr(VENC_CHN VeChn, const VENC_CHN_BUF_WRAP_S *pstVencChnBufWrap)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstVencChnBufWrap	Указатель на атрибуты кольцевого Buf.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Поддерживается только RV1106 и RV1103.
- Для кольцевого режима VI и VENC должны быть настроены одинаково, а VI должен быть привязан к VENC.
- Поддерживается только один кольцевой канал.
- Число строк и размер кольцевого Buf должны совпадать с настройками VI.

4.76 RK_MPI_VENC_GetChnBufWrapAttr

【Описание】

Получает атрибуты кольцевого Buf канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetChnBufWrapAttr(VENC_CHN VeChn, VENC_CHN_BUF_WRAP_S *pstVencChnBufWrap)

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstVencChnBufWrap	Указатель на атрибуты кольцевого Buf.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Поддерживается только RV1106 и RV1103.

4.77 RK_MPI_VENC_SetSliceSplit

【Описание】

Задаёт атрибуты разбиения slice канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_SetSliceSplit(VENC_CHN VeChn, const VENC_SLICE_SPLIT_S *pstSliceSplit);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstSliceSplit	Параметры атрибутов разбиения slice канала кодирования.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstSliceSplit пуст, возвращается ошибка.
- Интерфейс задаётся после создания и до уничтожения канала кодирования.
- Рекомендуется вызывать после создания канала, но до запуска кодирования, чтобы не использовать его многократно во время кодирования.

4.78 RK_MPI_VENC_GetSliceSplit

【Описание】

Получает атрибуты разбиения slice канала кодирования.

【Синтаксис】

RK_S32 RK_MPI_VENC_GetSliceSplit(VENC_CHN VeChn, VENC_SLICE_SPLIT_S *pstRefParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
VeChn	Номер канала кодирования. Диапазон: [0, VENC_MAX_CHN_NUM).	Вход
pstSliceSplit	Параметры атрибутов разбиения slice канала кодирования.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок VENC.

【Примечание】

- Если канал не создан либо pstSliceSplit пуст, возвращается ошибка.
- Интерфейс вызывается после создания и до уничтожения канала.

5. Типы данных

5.1 VENC_MAX_CHN_NUM

【Описание】

Определяет максимальное число каналов кодирования.

【Определение】

```
#define VENC_MAX_CHN_NUM              16
```

【Примечания】

Нет

5.2 VENC_CHN_ATTR_S

【Описание】

Определяет структуру атрибутов канала кодирования.

【Определение】

```
typedef struct rkVENC_CHN_ATTR_S {
    VENC_ATTR_S     stVencAttr;
    VENC_RC_ATTR_S  stRcAttr;
    VENC_GOP_ATTR_S stGopAttr;
} VENC_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
stVencAttr	Атрибуты кодера.
stRcAttr	Атрибуты контроллера битрейта.
stGopAttr	Структура типа Gop Mode.

【Примечания】

Нет

5.3 VENC_ATTR_S

【Описание】

Определяет структуру атрибутов кодера.

【Определение】

```
typedef struct rkVENC_ATTR_S {
    RK_CODEC_ID_E enType;
    PIXEL_FORMAT_E enPixelFormat;
    MIRROR_E enMirror;
    RK_U32 u32BufSize;
    RK_U32 u32Profile;
    RK_BOOL bByFrame;
    RK_U32 u32PicWidth;
    RK_U32 u32PicHeight;
    RK_U32 u32VirWidth;
    RK_U32 u32VirHeight;
    RK_U32 u32StreamBufCnt;
    union {
        VENC_ATTR_H264_S stAttrH264e;
        VENC_ATTR_H265_S stAttrH265e;
        VENC_ATTR_MJPEG_S stAttrMjpege;
        VENC_ATTR_JPEG_S stAttrJpege;
    };
} VENC_ATTR_S;
```

【Члены】

Имя члена	Описание
enType	Тип протокола кодирования: H264, H265 и т. д.
enPixelFormat	Формат пикселей входного изображения кодирования. Подробности см. в определении PIXEL_FORMAT_E файла rk_comm_video.h.
enMirror	Зеркальное отражение выходного изображения кодирования.
u32BufSize	Размер buffer потока. Рекомендуемые значения: для H264/H265: половина размера одного изображения; для JPEG/MJPEG: произведение ширины и высоты изображения.
u32Profile	Уровень кодирования. Значения для H.264. 66：Baseline。 77：Main Profile。 100：High Profile。 Значения для H.265. 0：Main Profile。 1：Main 10 Profile。 Значения для JPEG/MJPEG. 0：Baseline
bByFrame	Получение потока по кадрам/пакетам. Диапазон: {RK_TRUE, RK_FALSE}. RK_TRUE: получение по кадрам. RK_FALSE: получение по пакетам. Сейчас не используется. По умолчанию поток получается по кадрам: I-кадр H264 содержит сведения о трёх NALU-пакетах sps/pps/idr; I-кадр H265 содержит сведения о четырёх NALU-пакетах vps/sps/pps/idr.
u32PicWidth	Ширина кодируемого изображения в пикселях.
u32PicHeight	Высота кодируемого изображения в пикселях.
u32VirWidth	Ширина аппаратного буфера кодирования в пикселях.
u32VirHeight	Высота аппаратного буфера кодирования в пикселях.
u32StreamBufCnt	Максимальное число выходных буферов кодирования.
stAttrH264e/stAttrMjpege/stAttrJpege/stAttrH265e	Атрибуты контроллера битрейта.

【Примечания】

- До RK_MPI_VENC_StartRecvFrame можно изменять все поля; после RK_MPI_VENC_StartRecvFrame изменять их нельзя.

5.4 VENC_RC_ATTR_S

【Описание】

Определяет атрибуты контроллера битрейта канала кодирования.

【Определение】

```
typedef struct rkVENC_RC_ATTR_S {
    VENC_RC_MODE_E enRcMode;
    union {
        VENC_H264_CBR_S stH264Cbr;
        VENC_H264_VBR_S stH264Vbr;
        VENC_H264_AVBR_S stH264Avbr;
        VENC_H264_FIXQP_S stH264FixQp;
```

```
        VENC_MJPEG_CBR_S stMjpegCbr;
        VENC_MJPEG_VBR_S stMjpegVbr;
        VENC_MJPEG_FIXQP_S stMjpegFixQp;

        VENC_H265_CBR_S stH265Cbr;
        VENC_H265_VBR_S stH265Vbr;
        VENC_H265_AVBR_S stH265Avbr;
        VENC_H265_FIXQP_S stH265FixQp;
    };
} VENC_RC_ATTR_S;
```

【Члены】

Имя члена	Описание
enRcMode	Режим RC.
stH264Cbr	Атрибуты режима CBR канала H.264.
stH264Vbr	Атрибуты режима VBR канала H.264.
stH264Avbr	Атрибуты режима AVBR канала H.264.
stH264FixQp	Атрибуты режима FixQp канала H.264.
stMjpegCbr	Атрибуты режима CBR канала Mjpeg.
stMjpegVbr	Атрибуты режима VBR канала Mjpeg.
stMjpegFixQp	Атрибуты режима FixQp канала Mjpeg.
stH265Cbr	Атрибуты режима CBR канала H.265.
stH265Vbr	Атрибуты режима VBR канала H.265.
stH265Avbr	Атрибуты режима AVBR канала H.265.
stH265FixQp	Атрибуты режима FixQp канала H.265.

【Примечания】

- Все поля являются динамическими атрибутами.

5.5 VENC_RC_PARAM_S

【Описание】

Определяет расширенные параметры управления битрейтом канала кодирования.

【Определение】

```
typedef struct rkVENC_RC_PARAM_S {
    RK_U32 s32FirstFrameStartQp;
    union {
        VENC_PARAM_H264_S stParamH264;
        VENC_PARAM_H265_S stParamH265;
        VENC_PARAM_MJPEG_S stParamMjpeg;
    };
} VENC_RC_PARAM_S;
```

【Члены】

Имя члена	Описание
s32FirstFrameStartQp	Значение qp первого кадра. Сейчас действует только для H264/H265; для других кодеров можно не задавать. Диапазон: [u32MinIQp, u32MaxIQp]. По умолчанию 26.
stParamH264	Управление параметрами qp канала h264.
stParamH265	Управление параметрами qp канала h265.
stParamMjpeg	Управление параметрами qp канала mjpeg.

【Примечания】

- При H264/H265 значение s32FirstFrameStartQp ограничивается параметрами qp I-кадра и должно находиться в [u32MinIQp, u32MaxIQp], иначе задание всех расширенных параметров управления битрейтом завершится ошибкой.
- Все поля являются динамическими атрибутами.

5.6 VENC_PARAM_H264_S

【Описание】

Определяет конфигурацию параметров qp канала H264.

【Определение】

```
typedef struct rkVENC_PARAM_H264_S {
    RK_U32 u32StepQp;
    RK_U32 u32MaxQp;
    RK_U32 u32MinQp;
    RK_U32 u32MaxIQp;
    RK_U32 u32MinIQp;
    RK_S32 s32DeltIpQp;
    RK_S32 s32MaxReEncodeTimes;
} VENC_PARAM_H264_S;
```

【Члены】

Имя члена	Описание
u32StepQp	Максимальный шаг qp.
u32MaxQp	Максимальный qp P-кадра. Диапазон: [1, 51]. По умолчанию 51.
u32MinQp	Минимальный qp P-кадра. Диапазон: [1, u32MaxQp]. По умолчанию 10.
u32MaxIQp	Максимальный qp I-кадра. Диапазон: [1, 51]. По умолчанию 46.
u32MinIQp	Минимальный qp I-кадра. Диапазон: [1, u32MaxIQp]. По умолчанию 24.
s32DeltIpQp	Разница между средним QP нескольких P-кадров перед I-кадром и QP I-кадра: s32DeltIpQp = average(PPPP) - I; чем больше значение, тем относительно выше качество I-кадра и заметнее эффект дыхания; чем меньше значение, тем относительно ниже качество I-кадра и слабее эффект дыхания, однако это также влияет на качество последующих P-кадров. Диапазон: [-7, 7]. По умолчанию 2.
s32MaxReEncodeTimes	Максимальное число повторных кодирований: [0, 3]. По умолчанию 1.

【Примечания】

- Параметры должны находиться в допустимых диапазонах, а минимальный qp не должен превышать максимальный, иначе задание всех параметров qp завершится ошибкой.
- Все поля являются динамическими атрибутами.

5.7 VENC_PARAM_H265_S

【Описание】

Определяет конфигурацию параметров qp канала H265.

【Определение】

```
typedef struct rkVENC_PARAM_H265_S {
    RK_U32 u32StepQp;
    RK_U32 u32MaxQp;
    RK_U32 u32MinQp;
    RK_U32 u32MaxIQp;
    RK_U32 u32MinIQp;
    RK_S32 s32DeltIpQp;
    RK_S32 s32MaxReEncodeTimes;
} VENC_PARAM_H265_S;
```

【Члены】

Имя члена	Описание
u32StepQp	Максимальный шаг qp.
u32MaxQp	Максимальный qp P-кадра. Диапазон: [1, 51]. По умолчанию 51.
u32MinQp	Минимальный qp P-кадра. Диапазон: [1, u32MaxQp]. По умолчанию 10.
u32MaxIQp	Максимальный qp I-кадра. Диапазон: [1, 51]. По умолчанию 46.
u32MinIQp	Минимальный qp I-кадра. Диапазон: [1, u32MaxIQp]. По умолчанию 24.
s32DeltIpQp	Разница между средним QP нескольких P-кадров перед I-кадром и QP I-кадра: s32DeltIpQp = average(PPPP) - I; чем больше значение, тем относительно выше качество I-кадра и заметнее эффект дыхания; чем меньше значение, тем относительно ниже качество I-кадра и слабее эффект дыхания, однако это также влияет на качество последующих P-кадров. Диапазон: [-7, 7]. По умолчанию 2.
s32MaxReEncodeTimes	Максимальное число повторных кодирований: [0, 3]. По умолчанию 1.

【Примечания】

- Параметры должны находиться в допустимых диапазонах, а минимальный qp не должен превышать максимальный, иначе задание всех параметров qp завершится ошибкой.
- Все поля являются динамическими атрибутами.

5.8 VENC_PARAM_MJPEG_S

【Описание】

Определяет конфигурацию параметров qp канала MJPEG.

【Определение】

```
typedef struct rkVENC_PARAM_MJPEG_S {
    RK_U32 u32Qfactor;
    RK_U32 u32MaxQfactor;
    RK_U32 u32MinQfactor;
} VENC_PARAM_MJPEG_S;
```

【Члены】

Имя члена	Описание
u32Qfactor	Qfactor кодирования MJPEG. Диапазон: [1, 99]. По умолчанию 70.
u32MaxQfactor	Максимальный Qfactor кодирования MJPEG. Диапазон: [u32Qfactor, 99]. По умолчанию 99.
u32MinQfactor	Минимальный Qfactor кодирования MJPEG. Диапазон: [1, u32Qfactor]. По умолчанию 30.

Примечания

- u32Qfactor совпадает с u32Qfactor в VENC_MJPEG_FIXQP_S; оба управляют Qfactor mjpeg. В режиме mjpeg fixqp рекомендуется задавать его вместе с VENC_MJPEG_FIXQP_S.
- В режимах CBR или VBR MJPEG параметры qp можно задавать диапазоном qfactor.
- Все поля являются динамическими атрибутами.

5.9 VIDEO_FRAME_INFO_S

Описание

Определяет структуру информации о кадре видеоизображения.

Определение

```
typedef struct rkVIDEO_FRAME_S {
    MB_BLK              pMbBlk;
    RK_U32              u32Width;
    RK_U32              u32Height;
    RK_U32              u32VirWidth;
    RK_U32              u32VirHeight;
    VIDEO_FIELD_E       enField;
    PIXEL_FORMAT_E      enPixelFormat;
    VIDEO_FORMAT_E      enVideoFormat;
    COMPRESS_MODE_E     enCompressMode;
    DYNAMIC_RANGE_E     enDynamicRange;
    COLOR_GAMUT_E       enColorGamut;
    RK_VOID            *pVirAddr[RK_MAX_COLOR_COMPONENT];
    RK_U32              u32TimeRef;
    RK_U64              u64PTS;
    RK_U64              u64PrivateData;
    RK_U32              u32FrameFlag;
} VIDEO_FRAME_S;

typedef struct rkVIDEO_FRAME_INFO_S {
    VIDEO_FRAME_S stVFrame;
} VIDEO_FRAME_INFO_S;
```

Члены

Имя члена	Описание
pMbBlk	Пользовательские данные.
u32Width	Ширина изображения.
u32Height	Высота изображения.
u32VirWidth	Ширина памяти изображения. Несжатые данные: JPEG/MJPEG и H264 требуют выравнивания по 16, H265 — по 64. Сжатые данные: H264/H265 (сейчас сжатые данные поддерживаются только ими) требуют выравнивания по 16.
u32VirHeight	Высота памяти изображения. Несжатые данные: JPEG/MJPEG требуют выравнивания по 16, H264/H265 — по 2. Сжатые данные: H264/H265 требуют выравнивания по 4.
enField	Режим полей кадра изображения. Сейчас не используется, можно не задавать.
enVideoFormat	Формат видеоизображения. Сейчас не используется, можно не задавать.
enPixelFormat	Формат пикселей изображения. Подробности см. в PIXEL_FORMAT_E файла rk_comm_video.h.
enCompressMode	Режим сжатия изображения. COMPRESS_MODE_NONE: без сжатия; COMPRESS_AFBC_16x16: сжатие AFBC.
enDynamicRange	Динамический диапазон изображения. Сейчас не используется, можно не задавать.
enColorGamut	Цветовой охват изображения. Сейчас не используется, можно не задавать.
pVirAddr	Виртуальный адрес данных изображения. Сейчас не используется, можно не задавать.
u32TimeRef	Порядковый номер кадра изображения. Сейчас не используется, можно не задавать.
u64PTS	Временная метка изображения в мкс. Рекомендуется задавать через clock_gettime(CLOCK_MONOTONIC, &time).
u64PrivateData	Частные данные. Сейчас не используется, можно не задавать.
u32FrameFlag	Флаги текущего кадра из RTMBFlags; допускается побитовое ИЛИ. Если специальных операций нет, рекомендуется установить 0. MB.

Примечания

- Чтобы получить данные изображения, пользователь может преобразовать pMbBlk в виртуальный адрес через RK_MPI_MB_Handle2VirAddr.

```
data = RK_MPI_MB_Handle2VirAddr(sFrame.stVFrame.pMbBlk);
fwrite(data, 1, sFrame.stVFrame.u32Width * sFrame.stVFrame.u32Height * 3 /2, fp);
fflush(fp);
```

5.10 VENC_H264_CBR_S

Описание

Определяет структуру атрибутов CBR канала H.264.

Определение

```
typedef struct rkVENC_H264_CBR_S {
    RK_U32 u32Gop;
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32BitRate;
    RK_U32 u32StatTime;
} VENC_H264_CBR_S;
```

【Члены】

Имя члена	Описание
u32Gop	Значение gop H.264. Диапазон: [1, 65536].
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32BitRate	Средний bitrate в кбит/с. Диапазон: [3, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30.
u32StatTime	Период статистики битрейта в секундах. Диапазон: [1, 60], по умолчанию 3.

【Примечания】

- При входной частоте 30 задайте u32SrcFrameRateNum = 30 и u32SrcFrameRateDen = 1.
- При выходной частоте 30 задайте fr32DstFrameRateNum = 30 и fr32DstFrameRateDen = 1.
- Если входная частота 25, а выходная 12, из 25 входных кадров кодируются 12, остальные 13 отбрасываются.
- Все поля являются динамическими атрибутами.

5.11 VENC_H264_VBR_S

【Описание】

Определяет структуру атрибутов VBR канала H.264.

【Определение】

```
typedef struct rkVENC_H264_VBR_S {
    RK_U32 u32Gop;
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32BitRate;
    RK_U32 u32MaxBitRate;
    RK_U32 u32MinBitRate;
    RK_U32 u32StatTime;
} VENC_H264_VBR_S;
```

【Члены】

Имя члена	Описание
u32Gop	Значение gop H.264. Диапазон: [1, 65536].
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32BitRate	Средний bitrate в кбит/с. Диапазон: [3, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30.
u32MaxBitRate	Максимальный bitrate в кбит/с. Диапазон: [u32BitRate, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 3 / 2.
u32MinBitRate	Минимальный bitrate в кбит/с. Диапазон: [3, u32BitRate], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 1 / 2.
u32StatTime	Период статистики битрейта в секундах. Диапазон: [1, 60], по умолчанию 3.

Примечания

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.12 VENC_H264_AVBR_S

Описание

Определяет структуру атрибутов AVBR канала H.264.

Определение

```
typedef struct rkVENC_H264_AVBR_S {
    RK_U32 u32Gop;
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32BitRate;
    RK_U32 u32MaxBitRate;
    RK_U32 u32MinBitRate;
    RK_U32 u32StatTime;
} VENC_H264_AVBR_S;
```

Члены

Имя члена	Описание
u32Gop	Значение gop H.264. Диапазон: [1, 65536].
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32BitRate	Средний bitrate в кбит/с. Диапазон: [3, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30.
u32MaxBitRate	Максимальный bitrate в кбит/с. Диапазон: [u32BitRate, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 3 / 2.
u32MinBitRate	Минимальный bitrate в кбит/с. Диапазон: [3, u32BitRate], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 1 / 2.
u32StatTime	Период статистики битрейта в секундах. Диапазон: [1, 60], по умолчанию 3.

Примечания

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.13 VENC_H264_FIXQP_S

Описание

Определяет структуру атрибутов FIXQP канала H.264.

Определение

```
typedef struct rkVENC_H264_FIXQP_S {
    RK_U32 u32Gop;
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32IQp;
    RK_U32 u32PQp;
    RK_U32 u32BQp;
} VENC_H264_FIXQP_S;
```

Члены

Имя члена	Описание
u32Gop	Значение gop H.264. Диапазон: [1, 65536].
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32IQp	Значение Qp всех макроблоков I-кадра. Диапазон: [1, 51].
u32PQp	Значение Qp всех макроблоков P-кадра. Диапазон: [1, 51].
u32BQp	Значение Qp всех макроблоков B-кадра. Диапазон: [1, 51]. Сейчас не используется.

Примечания

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.14 VENC_H265_CBR_S

Описание

Определяет структуру атрибутов CBR канала H.265.

Определение

```
typedef struct rkVENC_H265_CBR_S {
    RK_U32 u32Gop;
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32BitRate;
    RK_U32 u32StatTime;
} VENC_H265_CBR_S;
```

Члены

Имя члена	Описание
u32Gop	Значение gop H.265. Диапазон: [1, 65536].
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32BitRate	Средний bitrate в кбит/с. Диапазон: [3, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30.
u32StatTime	Период статистики битрейта в секундах. Диапазон: [1, 60], по умолчанию 3.

Примечания

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.15 VENC_H265_VBR_S

Описание

Определяет структуру атрибутов VBR канала H.265.

Определение

```
typedef struct rkVENC_H265_VBR_S {
    RK_U32 u32Gop;
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32BitRate;
    RK_U32 u32MaxBitRate;
    RK_U32 u32MinBitRate;
    RK_U32 u32StatTime;
} VENC_H265_VBR_S;
```

Члены

Имя члена	Описание
u32Gop	Значение gop H.265. Диапазон: [1, 65536].
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32BitRate	Средний bitrate в кбит/с. Диапазон: [3, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30.
u32MaxBitRate	Максимальный bitrate в кбит/с. Диапазон: [u32BitRate, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 3 / 2.
u32MinBitRate	Минимальный bitrate в кбит/с. Диапазон: [3, u32BitRate], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 1 / 2.
u32StatTime	Период статистики битрейта в секундах. Диапазон: [1, 60], по умолчанию 3.

Примечания

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.16 VENC_H265_AVBR_S

Описание

Определяет структуру атрибутов AVBR канала H.265.

Определение

```
typedef struct rkVENC_H265_AVBR_S {
    RK_U32 u32Gop;
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32BitRate;
    RK_U32 u32MaxBitRate;
    RK_U32 u32MinBitRate;
    RK_U32 u32StatTime;
} VENC_H265_AVBR_S;
```

Члены

Имя члена	Описание
u32Gop	Значение gop H.265. Диапазон: [1, 65536].
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32BitRate	Средний bitrate в кбит/с. Диапазон: [3, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30.
u32MaxBitRate	Максимальный bitrate в кбит/с. Диапазон: [u32BitRate, 200000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 3 / 2.
u32MinBitRate	Минимальный bitrate в кбит/с. Диапазон: [3, u32BitRate], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 1 / 2.
u32StatTime	Период статистики битрейта в секундах. Диапазон: [1, 60], по умолчанию 3.

Примечания

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.17 VENC_H265_FIXQP_S

Описание

Определяет структуру атрибутов FIXQP канала H.265.

Определение

```
typedef struct rkVENC_H265_FIXQP_S {
    RK_U32 u32Gop;
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32IQp;
    RK_U32 u32PQp;
    RK_U32 u32BQp;
} VENC_H265_FIXQP_S;
```

Члены

Имя члена	Описание
u32Gop	Значение gop H.265. Диапазон: [1, 65536].
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32IQp	Значение Qp всех макроблоков I-кадра. Диапазон: [1, 51].
u32PQp	Значение Qp всех макроблоков P-кадра. Диапазон: [1, 51].
u32BQp	Значение Qp всех макроблоков B-кадра. Диапазон: [1, 51]. Сейчас не используется.

Примечания

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.18 VENC_MJPEG_CBR_S

Описание

Определяет структуру атрибутов CBR канала mjpeg.

Определение

```
typedef struct rkVENC_MJPEG_CBR_S {
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32BitRate;
    RK_U32 u32StatTime;
} VENC_MJPEG_CBR_S;
```

Члены

Имя члена	Описание
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32BitRate	Средний bitrate в кбит/с. Диапазон: [5, 800000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30.
u32StatTime	Период статистики битрейта в секундах. Диапазон: [1, 60], по умолчанию 3.

Примечания

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.19 VENC_MJPEG_VBR_S

【Описание】

Определяет структуру атрибутов VBR канала mjpeg.

【Определение】

```
typedef struct rkVENC_MJPEG_VBR_S {
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32BitRate;
    RK_U32 u32MaxBitRate;
    RK_U32 u32MinBitRate;
    RK_U32 u32StatTime;
} VENC_MJPEG_VBR_S;
```

【Члены】

Имя члена	Описание
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32BitRate	Средний bitrate в кбит/с. Диапазон: [5, 800000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30.
u32MaxBitRate	Максимальный bitrate в кбит/с. Диапазон: [u32BitRate, 800000], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 3 / 2.
u32MinBitRate	Минимальный bitrate в кбит/с. Диапазон: [5, u32BitRate], по умолчанию u32VirWidth * u32VirHeight / 8 * 30 * 1 / 2.
u32StatTime	Период статистики битрейта в секундах. Диапазон: [1, 60], по умолчанию 3.

【Примечания】

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.20 VENC_MJPEG_FIXQP_S

【Описание】

Определяет структуру атрибутов FIXQP канала mjpeg.

【Определение】

```
typedef struct rkVENC_MJPEG_FIXQP_S {
    RK_U32 u32SrcFrameRateNum;
    RK_U32 u32SrcFrameRateDen;
    RK_U32 fr32DstFrameRateNum;
    RK_U32 fr32DstFrameRateDen;
    RK_U32 u32Qfactor;
} VENC_MJPEG_FIXQP_S;
```

【Члены】

Имя члена	Описание
u32SrcFrameRateNum	Числитель входной частоты кадров.
u32SrcFrameRateDen	Знаменатель входной частоты кадров.
fr32DstFrameRateNum	Числитель выходной частоты кадров.
fr32DstFrameRateDen	Знаменатель выходной частоты кадров.
u32Qfactor	Qfactor кодирования MJPEG. Диапазон: [1, 99]. По умолчанию 70.

【Примечания】

- См. примечания VENC_H264_CBR_S.
- Все поля являются динамическими атрибутами.

5.21 VENC_GOP_ATTR_S

【Описание】

Определяет структуру атрибутов GOP кодера.

【Определение】

```
typedef struct rkVENC_GOP_ATTR_S {
    VENC_GOP_MODE_E enGopMode;
    RK_S32 s32VirIdrLen;
} VENC_GOP_ATTR_S;
```

【Члены】

Имя члена	Описание
enGopMode	Тип GOP кодера.
s32VirIdrLen	Длина виртуального I-кадра. По умолчанию 0, то есть не действует.

【Примечания】

- s32VirIdrLen действует только при enGopMode = VENC_GOPMODE_SMARTP. Рекомендуется задавать половиной длины GOP либо значением, на которое GOP делится без остатка.
- Все поля являются динамическими атрибутами.

5.22 VENC_RECV_PIC_PARAM_S

【Описание】

Определяет структуру числа кадров, непрерывно принимаемых и кодируемых каналом.

【Определение】

```
typedef struct rkVENC_RECV_PIC_PARAM_S {
    RK_S32 s32RecvPicNum;
} VENC_RECV_PIC_PARAM_S;
```

【Члены】

Имя члена	Описание
s32RecvPicNum	Число кадров для непрерывного приёма и кодирования. Диапазон: [-1,0)∪(0,∞]. По умолчанию -1 — непрерывное кодирование без автоматической остановки.

【Примечания】

- Если s32RecvPicNum положителен, используется режим снимка; после кодирования заданного числа кадров вывод данных прекращается.
- Изменять поля повторно можно только после RK_MPI_VENC_StopRecvFrame.

5.23 VENC_CHN_STATUS_S

【Описание】

Определяет структуру состояния канала кодирования.

【Определение】

```
typedef struct rkVENC_CHN_STATUS_S {
    RK_U32 u32LeftPics;
    RK_U32 u32LeftStreamBytes;
    RK_U32 u32LeftStreamFrames;
    RK_U32 u32CurPacks;
    RK_U32 u32LeftRecvPics;
    RK_U32 u32LeftEncPics;
    RK_BOOL bJpegSnapEnd;
    VENC_STREAM_INFO_S stVencStrmInfo;
} VENC_CHN_STATUS_S;
```

【Члены】

Имя члена	Описание
u32LeftPics	Число изображений, ожидающих кодирования.
u32LeftStreamBytes	Число байтов, оставшихся в buffer потока.
u32LeftStreamFrames	Число кадров, оставшихся в buffer потока.
u32CurPacks	Число пакетов потока текущего кадра.
u32LeftRecvPics	Число кадров, оставшихся для приёма; действует после задания числа кадров через RK_MPI_VENC_StartRecvFrame.
u32LeftEncPics	Число кадров, оставшихся для кодирования; действует после задания числа кадров через RK_MPI_VENC_StartRecvFrame.
bJpegSnapEnd	Завершён ли процесс снимка в режиме Jpege. Сейчас не используется.
stVencStrmInfo	Характеристики потока кодера.

【Примечания】

Нет

5.24 VENC_STREAM_S

【Описание】

Определяет структуру типа потока кадра.

【Определение】

```
typedef struct rkVENC_STREAM_S {
    VENC_PACK_S ATTRIBUTE* pstPack;
    RK_U32      ATTRIBUTE u32PackCount;
    RK_U32      u32Seq;

    union {
        VENC_STREAM_INFO_H264_S   stH264Info;
        VENC_STREAM_INFO_JPEG_S   stJpegInfo;
        VENC_STREAM_INFO_H265_S   stH265Info;
        VENC_STREAM_INFO_PRORES_S stProresInfo;
    };

    union {
        VENC_STREAM_ADVANCE_INFO_H264_S   stAdvanceH264Info;
        VENC_STREAM_ADVANCE_INFO_JPEG_S   stAdvanceJpegInfo;
        VENC_STREAM_ADVANCE_INFO_H265_S   stAdvanceH265Info;
        VENC_STREAM_ADVANCE_INFO_PRORES_S stAdvanceProresInfo;
    };
} VENC_STREAM_S;
```

【Члены】

Имя члена	Описание
pstPack	Структура пакетов потока кадра.
u32PackCount	Общее число пакетов одного кадра потока. Однопакетный вывод: -- на входе задаётся 1; -- после успешного вызова содержит фактически заполненное число пакетов, по умолчанию 1 (I-кадр H264/H265 содержит несколько NALU-пакетов). Это режим по умолчанию. Многопакетный вывод: -- на входе задаёт число пакетов pstPack и должно быть не меньше числа пакетов текущего кадра; сейчас следует задавать 16. Слишком малое значение может привести к ошибке получения потока, потере кадра и возврату RK_ERR_VENC_ILLEGAL_PARAM; -- после успешного вызова содержит фактически заполненное число пакетов.
u32Seq	Порядковый номер потока: номер кадра при получении по кадрам или номер пакета при получении по пакетам.
stH624Info/stJpegInfo/ stH265Info/stProresInfo	Характеристики потока.
stAdvanceH264Info/ stAdvanceJpegInfo/ stAdvanceH265Info/stAdvanceProresInfo	Расширенные характеристики потока.

【Примечания】

Нет

5.25 VENC_PACK_S

【Описание】

Определяет структуру пакета потока кадра.

【Определение】

```
typedef struct rkVENC_PACK_S {
    MB_BLK              pMbBlk;
    RK_U32              u32Len;
    RK_U64              u64PTS;
    RK_BOOL             bFrameEnd;
    RK_BOOL             bStreamEnd;
    VENC_DATA_TYPE_U    DataType;
    RK_U32              u32Offset;
    RK_U32              u32DataNum;
    VENC_PACK_INFO_S    stPackInfo[8];
} VENC_PACK_S;
```

【Члены】

Имя члена	Описание
pMbBlk	Дескриптор блока буфера пакета потока.
u32Len	Длина пакета потока; для I-кадра H264/H265 это общая длина нескольких NALU-пакетов.
u64PTS	Временная метка в мкс.
bFrameEnd	Флаг конца кадра. По умолчанию RK_TRUE. Диапазон значений: RK_TRUE: пакет является последним пакетом кадра. RK_FALSE: пакет не является последним пакетом кадра.
bStreamEnd	Флаг конца потока. Диапазон значений: RK_TRUE: пакет является последним пакетом потока; кодер выводит поток последнего кодируемого кадра. RK_FALSE: пакет не является последним пакетом потока.
DataType	Тип потока; поддерживаются пакеты H.264/JPEG/H.265.
u32Offset	Смещение действительных данных в блоке буфера пакета. Сейчас один packet содержит один кадр, по умолчанию 0.
u32DataNum	Число пакетов других типов в данных текущего пакета (тип текущего пакета задаёт DataType), то есть число NALU-пакетов, кроме Islice/Pslice. Сейчас действует только для H264/H265. Для JPEG/MJPEG по умолчанию 1.
stPackInfo	Сведения о пакетах других типов, содержащихся в данных текущего пакета. Сейчас действует только для H264/H265.

【Примечания】

Нет

5.26 VENC_DATA_TYPE_U

【Описание】

Определяет тип результата потока.

【Определение】

```
typedef union rkVENC_DATA_TYPE_U {
    H264E_NALU_TYPE_E    enH264EType;
    JPEGE_PACK_TYPE_E    enJPEGEType;
    H265E_NALU_TYPE_E    enH265EType;
    PRORES_PACK_TYPE_E   enPRORESType;
} VENC_DATA_TYPE_U;
```

【Члены】

Имя члена	Описание
enH264EType	Тип пакета потока H.264.
enJPEGEType	Тип пакета потока JPEG.
enH265EType	Тип пакета потока H.265.
enPRORESType	Тип пакета потока PRORES.

【Примечания】

Нет

5.27 VENC_PACK_INFO_S

【Описание】

Определяет структуру данных пакетов других типов, содержащихся в данных текущего пакета.

【Определение】

```
typedef struct rkVENC_PACK_INFO_S {
    VENC_DATA_TYPE_U  u32PackType;
    RK_U32 u32PackOffset;
    RK_U32 u32PackLength;
} VENC_PACK_INFO_S;
```

【Члены】

Имя члена	Описание
u32PackType	Тип другого пакета, содержащегося в данных текущего пакета.
u32PackOffset	Смещение данных другого пакета в данных текущего пакета.
u32PackLength	Размер данных другого пакета в данных текущего пакета.

【Примечания】

Нет

5.28 H264E_NALU_TYPE_E

【Описание】

Определяет типы NALU потока H.264.

【Определение】

```
typedef enum rkH264E_NALU_TYPE_E {
    H264E_NALU_BSLICE = 0,
    H264E_NALU_PSLICE = 1,
    H264E_NALU_ISLICE = 2,
    H264E_NALU_IDRSLICE = 5,
    H264E_NALU_SEI    = 6,
    H264E_NALU_SPS    = 7,
    H264E_NALU_PPS    = 8,
    H264E_NALU_BUTT
} H264E_NALU_TYPE_E;
```

【Члены】

Имя члена	Описание
H264E_NALU_BSLICE	Тип BSLICE.
H264E_NALU_PSLICE	Тип PSLICE.
H264E_NALU_ISLICE	Тип ISLICE, тип кадра P.
H264E_NALU_IDRSLICE	Тип ISLICE, тип кадра IDR.
H264E_NALU_SEI	Тип SEI.
H264E_NALU_SPS	Тип SPS.
H264E_NALU_PPS	Тип PPS.
H264E_NALU_BUTT	Максимальное значение.

Примечания

Нет

5.29 H265E_NALU_TYPE_E

Описание

Определяет типы NALU потока H.265.

Определение

```
typedef enum rkH265E_NALU_TYPE_E {
    H265E_NALU_BSLICE = 0,
    H265E_NALU_PSLICE = 1,
    H265E_NALU_ISLICE = 2,
    H265E_NALU_IDRSLICE = 19,
    H265E_NALU_VPS    = 32,
    H265E_NALU_SPS    = 33,
    H265E_NALU_PPS    = 34,
    H265E_NALU_SEI    = 39,
    H265E_NALU_BUTT
} H265E_NALU_TYPE_E;
```

Члены

Имя члена	Описание
H265E_NALU_BSLICE	Тип BSLICE.
H265E_NALU_PSLICE	Тип PSLICE.
H265E_NALU_ISLICE	Тип ISLICE, тип кадра P.
H265E_NALU_IDRSLICE	Тип ISLICE, тип кадра IDR.
H265E_NALU_VPS	Тип VPS.
H265E_NALU_SPS	Тип SPS.
H265E_NALU_PPS	Тип PPS.
H265E_NALU_SEI	Тип SEI.
H265E_NALU_BUTT	Максимальное значение.

Примечания

Нет

5.30 H264E_REF_TYPE_E

【Описание】

Определяет типы кадров и атрибуты опорных кадров потока с пропуском кадров H.264.

【Определение】

```
typedef enum rkH264E_REF_TYPE_E {
    BASE_IDRSLICE = 0,
    BASE_PSLICE_REFTOIDR,
    BASE_PSLICE_REFBYBASE,
    BASE_PSLICE_REFBYENHANCE,
    ENHANCE_PSLICE_REFBYENHANCE,
    ENHANCE_PSLICE_NOTFORREF,
    ENHANCE_PSLICE_BUTT
} H264E_REF_TYPE_E;
```

【Члены】

Имя члена	Описание
BASE_IDRSLICE	IDR-кадр базового слоя.
BASE_PSLICE_REFTOIDR	P-кадр базового слоя, используемый как опорный другими кадрами базового слоя и ссылающийся только на IDR-кадр.
BASE_PSLICE_REFBYBASE	P-кадр базового слоя, используемый как опорный другими кадрами базового слоя.
BASE_PSLICE_REFBYENHANCE	P-кадр базового слоя, используемый как опорный кадрами слоя enhance.
ENHANCE_PSLICE_REFBYENHANCE	P-кадр слоя enhance, используемый как опорный другими кадрами слоя enhance.
ENHANCE_PSLICE_NOTFORREF	P-кадр слоя enhance, не используемый как опорный.
ENHANCE_PSLICE_BUTT	Максимальное значение.

【Примечания】

Нет

5.31 H265E_REF_TYPE_E

【Описание】

Определяет типы кадров и атрибуты опорных кадров потока с пропуском кадров H.265.

【Определение】

```
typedef enum rkH264E_REF_TYPE_E H265E_REF_TYPE_E;
```

【Члены】

См. H264E_REF_TYPE_E.

【Примечания】

Нет

5.32 JPEGE_PACK_TYPE_E

【Описание】

Определяет типы PACK потока JPEG.

【Определение】

```
typedef enum rkJPEGE_PACK_TYPE_E {
    JPEGE_PACK_ECS = 5,
    JPEGE_PACK_APP = 6,
    JPEGE_PACK_VDO = 7,
    JPEGE_PACK_PIC = 8,
    JPEGE_PACK_DCF = 9,
    JPEGE_PACK_DCF_PIC = 10,
    JPEGE_PACK_BUTT
} JPEGE_PACK_TYPE_E;
```

【Члены】

Имя члена	Описание
JPEGE_PACK_ECS	Тип ECS.
JPEGE_PACK_APP	Тип APP.
JPEGE_PACK_VDO	Тип VDO.
JPEGE_PACK_PIC	Тип PIC.
JPEGE_PACK_DCF	Тип DCF ECS.
JPEGE_PACK_DCF_PIC	Тип ECS.
JPEGE_PACK_BUTT	Максимальное значение.

【Примечания】

- Все приведённые выше параметры сейчас не используются.

5.33 PRORES_PACK_TYPE_E

【Описание】

Определяет типы PACK потока PRORES.

【Определение】

```
typedef enum rkPRORES_PACK_TYPE_E {
    PRORES_PACK_PIC = 1,
    PRORES_PACK_BUTT
} PRORES_PACK_TYPE_E;
```

【Члены】

Имя члена	Описание
PRORES_PACK_PIC	Пакет данных PRORES.
PRORES_PACK_BUTT	Максимальное значение.

【Примечания】

- Все приведённые выше параметры сейчас не используются.

5.34 VENC_STREAM_INFO_H264_S

【Описание】

Определяет характеристики потока H.264.

【Определение】

```
typedef struct rkVENC_PACK_S {
    RK_U32                  u32PicBytesNum;
    RK_U32                  u32Inter16x16MbNum;
    RK_U32                  u32Inter8x8MbNum;
    RK_U32                  u32Intra16MbNum;
    RK_U32                  u32Intra8MbNum;
    RK_U32                  u32Intra4MbNum;
    H264E_REF_TYPE_E        enRefType;
    RK_U32                  u32UpdateAttrCnt;
    RK_U32                  u32StartQp;
    RK_U32                  u32MeanQp;
    RK_BOOL                 bPSkip;
} VENC_PACK_S;
```

【Члены】

Имя члена	Описание
u32PicBytesNum	Число байтов (BYTE) закодированного текущего кадра. Сейчас не используется.
u32Inter16x16MbNum	Число макроблоков текущего кадра, закодированных с предсказанием Inter16x16. Сейчас не используется.
u32Inter8x8MbNum	Число макроблоков текущего кадра, закодированных с предсказанием Inter8x8. Сейчас не используется.
u32Intra16MbNum	Число макроблоков текущего кадра, закодированных с предсказанием Intra16. Сейчас не используется.
u32Intra8MbNum	Число макроблоков текущего кадра, закодированных с предсказанием Intra8. Сейчас не используется.
u32Intra4MbNum	Число макроблоков текущего кадра, закодированных с предсказанием Intra4. Сейчас не используется.
enRefType	Тип кодируемого кадра при расширенном пропуске кадров и опорных кадрах.
u32UpdateAttrCnt	Число заданий атрибутов или параметров канала, включая параметры RC. Сейчас не используется.
u32StartQp	Значение startqp текущего кодируемого кадра. Сейчас не используется.
u32MeanQp	Среднее значение QP текущего кодируемого кадра. Сейчас не используется.
bPSkip	Является ли текущий кадр pskip-кадром. Сейчас не используется.

【Примечания】

Нет

5.35 VENC_STREAM_INFO_H265_S

【Описание】

Определяет характеристики потока H.265.

【Определение】

```
typedef struct rkVENC_PACK_S {
    RK_U32                  u32PicBytesNum;
    RK_U32                  u32Inter64x64CuNum;
    RK_U32                  u32Inter32x32CuNum;
    RK_U32                  u32Inter16x16CuNum;
    RK_U32                  u32Inter8x8CuNum;
    RK_U32                  u32Intra32x32CuNum;
    RK_U32                  u32Intra16x16CuNum;
    RK_U32                  u32Intra8x8CuNum;
    RK_U32                  u32Intra4x4CuNum;
    H265E_REF_TYPE_E        enRefType;
    RK_U32                  u32UpdateAttrCnt;
    RK_U32                  u32StartQp;
    RK_U32                  u32MeanQp;
    RK_BOOL                 bPSkip;
} VENC_PACK_S;
```

【Члены】

Имя члена	Описание
u32PicBytesNum	Число байтов (BYTE) закодированного текущего кадра. Сейчас не используется.
u32Inter64x64CuNum	Число блоков CU текущего кадра с предсказанием Inter64x64. Сейчас не используется.
u32Inter32x32CuNum	Число блоков CU текущего кадра с предсказанием Inter32x32. Сейчас не используется.
u32Inter16x16CuNum	Число блоков CU текущего кадра с предсказанием Inter16x16. Сейчас не используется.
u32Inter8x8CuNum	Число блоков CU текущего кадра с предсказанием Inter8x8. Сейчас не используется.
u32Intra32x32CuNum	Число блоков CU текущего кадра с предсказанием Intra32x32. Сейчас не используется.
u32Intra16x16CuNum	Число блоков CU текущего кадра с предсказанием Intra16x16. Сейчас не используется.
u32Intra8x8CuNum	Число блоков CU текущего кадра с предсказанием Intra8x8. Сейчас не используется.
u32Intra4x4CuNum	Число блоков CU текущего кадра с предсказанием Intra4x4. Сейчас не используется.
enRefType	Тип кодируемого кадра при расширенном пропуске кадров и опорных кадрах.
u32UpdateAttrCnt	Число заданий атрибутов или параметров канала, включая параметры RC. Сейчас не используется.
u32StartQp	Значение startqp текущего кодируемого кадра. Сейчас не используется.
u32MeanQp	Среднее значение QP текущего кодируемого кадра. Сейчас не используется.
bPSkip	Является ли текущий кадр pskip-кадром. Сейчас не используется.

Примечания

Нет

5.36 VENC_STREAM_INFO_JPEG_S

Описание

Определяет характеристики потока JPEG/MJPEG.

Определение

```
typedef struct rkVENC_STREAM_INFO_JPEG_S {
    RK_U32 u32PicBytesNum;
    RK_U32 u32UpdateAttrCnt;
    RK_U32 u32Qfactor;
} VENC_STREAM_INFO_JPEG_S;
```

Члены

Имя члена	Описание
u32PicBytesNum	Размер одного кадра потока JPEG в байтах. Сейчас не используется.
u32UpdateAttrCnt	Число заданий атрибутов или параметров канала, включая параметры RC. Сейчас не используется.
u32Qfactor	Qfactor текущего кодируемого кадра. Сейчас не используется.

【Примечания】

Нет

5.37 VENC_STREAM_INFO_PRORES_S

【Описание】

Определяет характеристики потока PRORES.

【Определение】

```
typedef struct rkVENC_STREAM_INFO_PRORES_S {
    RK_U32 u32PicBytesNum;
    RK_U32 u32UpdateAttrCnt;
} VENC_STREAM_INFO_PRORES_S;
```

【Члены】

Имя члена	Описание
u32PicBytesNum	Число байтов текущего кодируемого кадра.
u32UpdateAttrCnt	Число заданий атрибутов или параметров канала.

【Примечания】

Нет

5.38 VENC_STREAM_ADVANCE_INFO_H264_S

【Описание】

Определяет расширенные характеристики потока H.264.

【Определение】

```
typedef struct rkVENC_STREAM_ADVANCE_INFO_H264_S {
    RK_U32             u32ResidualBitNum;
    RK_U32             u32HeadBitNum;
    RK_U32             u32MadiVal;
    RK_U32             u32MadpVal;
    RK_DOUBLE          dPSNRVal;
    RK_U32             u32MseLcuCnt;
    RK_U32             u32MseSum;
    VENC_SSE_INFO_S    stSSEInfo[8];
    RK_U32             u32QpHstgrm[VENC_QP_SGRM_NUM];
    RK_U32             u32MoveScene16x16Num;
    RK_U32             u32MoveSceneBits;
} VENC_STREAM_ADVANCE_INFO_H264_S;
```

【Члены】

Имя члена	Описание
u32ResidualBitNum	Число битов остаточных данных текущего кодируемого кадра. Сейчас не используется.
u32HeadBitNum	Число битов заголовка текущего кодируемого кадра. Сейчас не используется.
u32MadiVal	Значение Madi пространственной текстурной сложности текущего кадра. Сейчас не используется.
u32MadpVal	Значение Madp временной сложности движения текущего кадра. Сейчас не используется.
dPSNRVal	Значение PSNR текущего кодируемого кадра. Сейчас не используется.
u32MseLcuCnt	Число LCU текущего кодируемого кадра. Сейчас не используется.
u32MseSum	Значение MSE текущего кодируемого кадра. Сейчас не используется.
stSSEInfo[8]	Значения SSE восьми областей текущего кодируемого кадра. Сейчас не используется.
u32QpHstgrm	Гистограмма Qp текущего кодируемого кадра. Сейчас не используется.
u32MoveScene16x16Num	Число блоков 16x16, распознанных как передний план; требуется включить управление битрейтом макроблоков переднего плана. Сейчас не используется.
u32MoveSceneBits	Число битов кодирования областей, распознанных как передний план; требуется включить управление битрейтом макроблоков переднего плана. Сейчас не используется.

【Примечания】

Нет

5.39 VENC_STREAM_ADVANCE_INFO_H265_S

【Описание】

Определяет расширенные характеристики потока H.265.

【Определение】

```
typedef struct rkVENC_STREAM_ADVANCE_INFO_H265_S {
    RK_U32              u32ResidualBitNum;
    RK_U32              u32HeadBitNum;
    RK_U32              u32MadiVal;
    RK_U32              u32MadpVal;
    RK_DOUBLE           dPSNRVal;
    RK_U32              u32MseLcuCnt;
    RK_U32              u32MseSum;
    VENC_SSE_INFO_S     stSSEInfo[8];
    RK_U32              u32QpHstgrm[VENC_QP_SGRM_NUM];
    RK_U32              u32MoveScene32x32Num;
    RK_U32              u32MoveSceneBits;
} VENC_STREAM_ADVANCE_INFO_H265_S;
```

【Члены】

Имя члена	Описание
u32ResidualBitNum	Число битов остаточных данных текущего кодируемого кадра. Сейчас не используется.
u32HeadBitNum	Число битов заголовка текущего кодируемого кадра. Сейчас не используется.
u32MadiVal	Значение Madi пространственной текстурной сложности текущего кадра. Сейчас не используется.
u32MadpVal	Значение Madp временной сложности движения текущего кадра. Сейчас не используется.
dPSNRVal	Значение PSNR текущего кодируемого кадра. Сейчас не используется.
u32MseLcuCnt	Число LCU текущего кодируемого кадра. Сейчас не используется.
u32MseSum	Значение MSE текущего кодируемого кадра. Сейчас не используется.
stSSEInfo[8]	Значения SSE восьми областей текущего кодируемого кадра. Сейчас не используется.
u32QpHstgrm	Гистограмма Qp текущего кодируемого кадра. Сейчас не используется.
u32MoveScene32x32Num	Число блоков 32x32, распознанных как передний план; требуется включить управление битрейтом макроблоков переднего плана. Сейчас не используется.
u32MoveSceneBits	Число битов кодирования областей, распознанных как передний план; требуется включить управление битрейтом макроблоков переднего плана. Сейчас не используется.

【Примечания】

Нет

5.40 VENC_STREAM_ADVANCE_INFO_JPEG_S

【Описание】

Определяет расширенные характеристики потока JPEG/MJPEG.

【Определение】

```
typedef struct rkVENC_STREAM_ADVANCE_INFO_H264_S {
    // RK_U32 u32Reserved;
} VENC_STREAM_ADVANCE_INFO_H264_S;
```

【Члены】

Нет

【Примечания】

Нет

5.41 VENC_STREAM_ADVANCE_INFO_PRORES_S

【Описание】

Определяет расширенные характеристики потока PRORES.

【Определение】

```
typedef struct rkVENC_STREAM_ADVANCE_INFO_PRORES_S {
    // RK_U32 u32Reserved;
} VENC_STREAM_ADVANCE_INFO_PRORES_S;
```

【Члены】

Нет

【Примечания】

Нет

5.42 VENC_SSE_INFO_S

【Описание】

Определяет сведения SSE протоколов H264/H265.

【Определение】

```
typedef struct rkVENC_SSE_INFO_S {
    RK_BOOL bSSEEn;
    RK_U32  u32SSEVal;
} VENC_SSE_INFO_S;
```

【Члены】

Имя члена	Описание
bSSEEn	Включение регионального SSE. Сейчас не используется.
u32SSEVal	Значение регионального SSE. Сейчас не используется.

【Примечания】

- Все поля являются динамическими атрибутами.

5.43 VENC_JPEG_PARAM_S

【Описание】

Определяет структуру расширенных параметров канала JPEG.

【Определение】

```
typedef struct rkVENC_JPEG_PARAM_S {
    RK_U32 u32Qfactor;
    RK_U8  u8YQt[64];
    RK_U8  u8CbQt[64];
    RK_U8  u8CrQt[64];
    RK_U32 u32MCUPerECS;
} VENC_JPEG_PARAM_S;
```

Члены

Имя члена	Описание
u32Qfactor	Подробности см. в RFC2435. Системное значение по умолчанию 70. Диапазон: [1, 99].
u8YQt	Таблица квантования Y. Диапазон: [1, 255]. Сейчас не используется, можно не задавать.
u8CbQt	Таблица квантования Cb. Диапазон: [1, 255]. Сейчас не используется, можно не задавать.
u8CrQt	Таблица квантования Cr. Диапазон: [1, 255]. Сейчас не используется, можно не задавать.
u32MCUPerECS	Число MCU в каждом ECS. По умолчанию 0 — ECS не разделяется. Сейчас не используется, можно не задавать.

Примечания

- Все поля являются динамическими атрибутами.

5.44 VENC_ROI_ATTR_S

Описание

Определяет сведения об области интереса кодирования.

Определение

```
typedef struct rkVENC_ROI_ATTR_S {
    RK_U32  u32Index;
    RK_BOOL bEnable;
    RK_BOOL bAbsQp;
    RK_S32  s32Qp;
    RK_BOOL bIntra;
    RECT_S  stRect;
} VENC_ROI_ATTR_S;
```

Члены

Имя члена	Описание
u32Index	Индекс области ROI. Поддерживаемый диапазон [0, 7]; другие значения не поддерживаются.
bEnable	Включена ли область ROI.
bAbsQp	Режим QP области ROI. RK_FALSE: относительный QP. RK_TURE: абсолютный QP.
s32Qp	Значение QP. Если bAbsQp задаёт относительный QP, s32Qp — смещение относительно qp кадра, диапазон [-51, 51]; если bAbsQp задаёт абсолютный QP, s32Qp — значение QP макроблока, диапазон [1, 51].
bIntra	Принудительно ли кодировать область ROI как I-блок. RK_FALSE: не включать. RK_TURE: принудительно кодировать как I-блок.
RECT_S	Область ROI. s32X, s32Y, u32Width и u32Height должны быть выровнены по 16.

Примечания

- Действует только для H264/H265.
- Итоговое значение s32Qp должно соблюдать диапазоны qp из VENC_RC_PARAM_S: для I-кадра qp области ограничивается [u32MinIQp, u32MaxIQp], для P-кадра — [u32MinQp, u32MaxQp].
- Все поля являются динамическими атрибутами.

5.45 VENC_CHN_PARAM_S

【Описание】

Определяет структуру параметров канала Venc.

【Определение】

```
typedef struct rkVENC_CHN_PARAM_S {
    RK_BOOL bColor2Grey;
    RK_U32  u32Priority;
    RK_U32  u32MaxStrmCnt;
    RK_U32  u32PollWakeUpFrmCnt;
    VENC_CROP_INFO_S stCropCfg;
    VENC_FRAME_RATE_S stFrameRate;
} VENC_CHN_PARAM_S;
```

【Члены】

Имя члена	Описание
bColor2Grey	Включает или выключает преобразование цвета канала в серый. Сейчас не используется, можно не задавать.
u32Priority	Приоритет канала кодирования. Сейчас не используется, можно не задавать.
u32MaxStrmCnt	Максимальное число кадров буфера потока. Сейчас не используется, можно не задавать.
u32PollWakeUpFrmCnt	При получении потока с тайм-аутом или блокировкой интерфейс пробуждается после кодирования указанного числа кадров u32PollWakeUpFrmCnt. Сейчас не используется, можно не задавать.
stCropCfg	Параметры обрезки Crop канала, включая масштабирование. Подробности см. в определении VENC_CROP_INFO_S.
stFrameRate	Параметры управления частотой кадров канала.

【Примечания】

- Все поля являются динамическими атрибутами.

5.46 VENC_FRAME_RATE_S

【Описание】

Определяет параметры управления частотой кадров канала.

【Определение】

```
typedef struct rkVENC_FRAME_RATE_S {
    RK_BOOL bEnable;
    RK_S32 s32SrcFrmRateNum;
    RK_S32 s32SrcFrmRateDen;
    RK_S32 s32DstFrmRateNum;
    RK_S32 s32DstFrmRateDen;
} VENC_FRAME_RATE_S;
```

【Члены】

Имя члена	Описание
bEnable	Включена ли настройка частоты кадров. RK_TRUE: включена. RK_FALSE: выключена.
s32SrcFrmRateNum	Числитель входной частоты кадров.
s32SrcFrmRateDen	Знаменатель входной частоты кадров.
s32DstFrmRateNum	Числитель выходной частоты кадров.
s32DstFrmRateDen	Знаменатель выходной частоты кадров.

【Примечания】

- Все поля являются динамическими атрибутами.

5.47 VENC_CROP_INFO_S

【Описание】

Определяет параметры обрезки Crop канала, включая scale.

【Определение】

```
typedef struct rkVENC_CROP_INFO_S {
    VENC_CROP_TYPE_E enCropType;
    RECT_S  stCropRect;
    VENC_SCALE_RECT_S stScaleRect;
} VENC_CROP_INFO_S;
```

【Члены】

Имя члена	Описание
enCropType	VENC_CROP_NONE: выключено. VENC_CROP_ONLY: включена обрезка, управляется stCropRect. VENC_CROP_SCALE: включены обрезка и масштабирование, управляются stScaleRect. VENC_CROP_BUTT
stCropRect	Область обрезки. stCropRect.s32X: координата X начала обрезки, выравнивание по 2 пикселям. stCropRect.s32Y: координата Y начала обрезки. stCropRect.u32Width: ширина обрезанного изображения, выравнивание по 2 пикселям. stCropRect.u32Height: высота обрезанного изображения, выравнивание по 2 пикселям.
stScaleRect	Управление обрезкой и масштабированием. RECT_S stSrc: исходная область обрезки и масштабирования. stSrc.s32X: координата X начала исходной области, выравнивание по 2 пикселям. stSrc.s32Y: координата Y начала исходной области, выравнивание по 2 пикселям. stSrc.u32Width: ширина исходной области, выравнивание по 2 пикселям. Можно задать 0; см. примечания. stSrc.u32Height: высота исходной области, выравнивание по 2 пикселям. Можно задать 0; см. примечания. RECT_S stDst: целевая область обрезки и масштабирования. stDst.s32X: координата X начала целевой области, выравнивание по 2 пикселям. stDst.s32Y: координата Y начала целевой области, выравнивание по 2 пикселям. stDst.u32Width: ширина целевого изображения, выравнивание по 2 пикселям. stDst.u32Height: высота целевого изображения, выравнивание по 2 пикселям.

【Примечания】

- Размер исходного изображения определяется видеокадром, поступающим в канал кодирования.
- Обрезка и обрезка с масштабированием определяются соответственно stCropRect и stScaleRect.

- Коэффициент масштабирования RK3588 не должен превышать 8, RV1109/RV1126/RK356X — 16.
- Рекомендуется выравнивать выходное разрешение по 16.
- Если обрезка с масштабированием используется вместе с поворотом или зеркальным отражением, порядок обработки: обрезка/масштабирование > поворот > зеркало.
- Если требуется только масштабирование, включите VENC_CROP_SCALE и задайте stSrc.u32Width = 0 и stSrc.u32Height = 0; изображение канала будет масштабировано до разрешения stDst.
- Все поля являются динамическими атрибутами.

Пример

- Обрезка: из входного изображения вырезается область 100x200 с началом в X=10, Y=20.

```
VENC_CHN_PARAM_S stParam;
RK_MPI_VENC_GetChnParam(u32Ch, &stParam);
stParam.stCropCfg.enCropType = VENC_CROP_ONLY;
stParam.stCropCfg.stCropRect.s32X = 10;
stParam.stCropCfg.stCropRect.s32Y = 20;
stParam.stCropCfg.stCropRect.u32Height = 100;
stParam.stCropCfg.stCropRect.u32Width = 200;
RK_MPI_VENC_SetChnParam(u32Ch, &stParam);
```

- Масштабирование: входное изображение масштабируется до 640x360, сведения об исходной области задаются равными 0.

```
VENC_CHN_PARAM_S stParam;
RK_MPI_VENC_GetChnParam(u32Ch, &stParam);
stParam.stCropCfg.enCropType = VENC_CROP_SCALE;
stParam.stCropCfg.stScaleRect.stSrc.s32X = 0;
stParam.stCropCfg.stScaleRect.stSrc.s32Y = 0;
stParam.stCropCfg.stScaleRect.stSrc.u32Width = 0;
stParam.stCropCfg.stScaleRect.stSrc.u32Height = 0;
stParam.stCropCfg.stScaleRect.stDst.s32X = 0;
stParam.stCropCfg.stScaleRect.stDst.s32Y = 0;
stParam.stCropCfg.stScaleRect.stDst.u32Width = 640;
stParam.stCropCfg.stScaleRect.stDst.u32Height = 360;
```

- Обрезка с масштабированием: из входного изображения вырезается область 1280x720 с началом X=10, Y=20, затем масштабируется до 640x360 и выводится.

```
VENC_CHN_PARAM_S stParam;
RK_MPI_VENC_GetChnParam(u32Ch, &stParam);
stParam.stCropCfg.enCropType = VENC_CROP_SCALE;
stParam.stCropCfg.stScaleRect.stSrc.s32X = 10;
stParam.stCropCfg.stScaleRect.stSrc.s32Y = 20;
stParam.stCropCfg.stScaleRect.stSrc.u32Width = 1280;
stParam.stCropCfg.stScaleRect.stSrc.u32Height = 720;
stParam.stCropCfg.stScaleRect.stDst.s32X = 0;
stParam.stCropCfg.stScaleRect.stDst.s32Y = 0;
stParam.stCropCfg.stScaleRect.stDst.u32Width = 640;
stParam.stCropCfg.stScaleRect.stDst.u32Height = 360;
```

5.48 VENC_GOP_MODE_E

Описание

Определяет типы GOP H.264/H.265.

Определение

```
typedef enum rkVENC_GOP_MODE_E {
    VENC_GOPMODE_INIT = 0,
    VENC_GOPMODE_NORMALP,
    VENC_GOPMODE_TSVC2,
    VENC_GOPMODE_TSVC3,
    VENC_GOPMODE_TSVC4,
    VENC_GOPMODE_SMARTP,
    VENC_GOPMODE_BUTT
} VENC_GOP_MODE_E;
```

【Члены】

Имя члена	Описание
VENC_GOPMODE_INIT	Не задано. Системное значение по умолчанию NORMALP.
VENC_GOPMODE_NORMALP	Кодирование P-кадров с одним опорным кадром.
VENC_GOPMODE_TSVC2	TSVC2。 TSVC-2 предоставляет два слоя кодирования; частота кадров может изменяться между 1/2 и полной.
VENC_GOPMODE_TSVC3	TSVC3。 TSVC-3 предоставляет три слоя; частота может быть 1/4, 1/2, 3/4 или полной.
VENC_GOPMODE_TSVC4	TSVC4。 TSVC-4 предоставляет четыре слоя; частота может быть 1/8, 1/4, 3/8, 1/2, 5/8, 3/4, 7/8 или полной.
VENC_GOPMODE_SMARTP	Кодирование интеллектуальных P-кадров.
VENC_GOPMODE_BUTT	Максимальное значение.

【Примечания】

Нет

5.49 VENC_RC_ADVPARAM_S

【Описание】

Определяет расширенные параметры модуля RC, содержащие функции, не относящиеся непосредственно к алгоритму управления потоком; в будущих версиях структура может расширяться.

【Определение】

```
typedef struct rkVENC_RC_ADVPARAM_S {
    RK_U32 u32ClearStatAfterSetAttr;
} VENC_RC_ADVPARAM_S;
```

【Члены】

Имя члена	Описание
u32ClearStatAfterSetAttr	Очищать ли статистику управления битрейтом после задания нового битрейта канала. По умолчанию 1. 0: не очищать статистику частоты кадров и битрейта RC; 1: очищать статистику частоты кадров и битрейта RC; Примечание: значение по умолчанию 1; сейчас поддерживается только 1.

【Примечания】

- Все поля являются динамическими атрибутами.

5.50 VENC_SUPERFRAME_CFG_S

【Описание】

Параметры стратегии обработки сверхбольших кадров.

【Определение】

```
typedef struct rkVENC_SUPERFRAME_CFG_S {
    VENC_SUPERFRM_MODE_E enSuperFrmMode;
    RK_U32 u32SuperIFrmBitsThr;
    RK_U32 u32SuperPFrmBitsThr;
    RK_U32 u32SuperBFrmBitsThr;
    VENC_RC_PRIORITY_E enRcPriority;
} VENC_SUPERFRAME_CFG_S;
```

【Члены】

Имя члена	Описание
enSuperFrmMode	Режим обработки сверхбольшого кадра, по умолчанию SUPERFRM_NONE.
u32SuperIFrmBitsThr	Порог сверхбольшого I-кадра в битах. Диапазон: не меньше 0.
u32SuperPFrmBitsThr	Порог сверхбольшого P-кадра в битах. Диапазон: не меньше 0.
u32SuperBFrmBitsThr	Порог сверхбольшого B-кадра в битах. Диапазон: не меньше 0.
enRcPriority	Приоритет управления битрейтом, по умолчанию VENC_RC_PRIORITY_BITRATE_FIRST.

【Примечания】

- Все поля являются динамическими атрибутами.

5.51 VENC_SUPERFRM_MODE_E

【Описание】

Определяет режим обработки сверхбольших кадров в управлении битрейтом.

【Определение】

```
typedef enum rkRC_SUPERFRM_MODE_E {
    SUPERFRM_NONE = 0,
    SUPERFRM_DISCARD,
    SUPERFRM_REENCODE,
    SUPERFRM_BUTT
} VENC_SUPERFRM_MODE_E;
```

【Члены】

Имя члена	Описание
SUPERFRM_NONE	Без специальной стратегии.
SUPERFRM_DISCARD	Отбрасывать сверхбольшой кадр.
SUPERFRM_REENCODE	Повторно кодировать сверхбольшой кадр.
SUPERFRM_BUTT	Максимальное значение.

【Примечания】

Нет

5.52 VENC_RC_PRIORITY_E

【Описание】

Определяет перечисление приоритетов повторного кодирования сверхбольшого кадра.

【Определение】

```
typedef enum rkVENC_RC_PRIORITY_E {
    VENC_RC_PRIORITY_BITRATE_FIRST = 1,
    VENC_RC_PRIORITY_FRAMEBITS_FIRST,
    VENC_RC_PRIORITY_BUTT,
} VENC_RC_PRIORITY_E;
```

【Члены】

Имя члена	Описание
VENC_RC_PRIORITY_BITRATE_FIRST	Приоритет целевого битрейта.
VENC_RC_PRIORITY_FRAMEBITS_FIRST	Приоритет порога сверхбольшого кадра.
VENC_RC_PRIORITY_BUTT	Максимальное значение.

【Примечания】

Приоритет действует только при повторном кодировании сверхбольшого кадра.

5.53 VENC_FRAMELOST_S

【Описание】

Параметры стратегии потери кадров при превышении порога мгновенного битрейта.

【Определение】

```
typedef struct rkVENC_FRAMELOST_S {
    RK_BOOL bFrmLostOpen;
    RK_U32 u32FrmLostBpsThr;
    VENC_FRAMELOST_MODE_E enFrmLostMode;
    RK_U32 u32EncFrmGaps;
} VENC_FRAMELOST_S;
```

【Члены】

Имя члена	Описание
bFrmLostOpen	Переключатель потери кадров. RK_TRUE: включено; RK_FALSE: выключено. По умолчанию выключено.
u32FrmLostBpsThr	Порог потери кадров в процентах. Значение битрейта потери кадров = bps_max * (100 + u32FrmLostBpsThr) / (float)100. Диапазон: >= 0.
enFrmLostMode	Режим стратегии потери кадров. FRMLOST_NORMAL: при превышении порога мгновенного битрейта кадр обычно отбрасывается; FRMLOST_PSKIP: при превышении порога кодируется pskip-кадр.
u32EncFrmGaps	Максимальное число последовательных потерянных кадров. 0 — отбрасывать все. Рекомендуется 1 или 2.

【Примечания】

- Все поля являются динамическими атрибутами.

5.54 VENC_INTRA_REFRESH_S

【Описание】

Параметры управления обновлением I-slice в P-кадрах.

【Определение】

```
typedef struct rkVENC_INTRA_REFRESH_S {
    RK_BOOL                     bRefreshEnable;
    VENC_INTRA_REFRESH_MODE_E   enIntraRefreshMode;
    RK_U32                      u32RefreshNum;
    RK_U32                      u32ReqIQp;
} VENC_INTRA_REFRESH_S;
```

【Члены】

Имя члена	Описание
bRefreshEnable	Включено ли обновление I-slice. RK_TRUE: включено; RK_FALSE: выключено. По умолчанию выключено.
enIntraRefreshMode	Режим обновления I-макроблоков: по строкам или по столбцам. INTRA_REFRESH_ROW: по строкам; INTRA_REFRESH_COLUMN: по столбцам.
u32RefreshNum	Число строк или столбцов I-макроблоков, обновляемых за раз, определяет скорость обновления и равномерность потока. Чем больше число строк или столбцов, тем быстрее обновление, но менее равномерен поток; чем меньше, тем медленнее обновление, но равномернее поток.
u32ReqIQp	Значение QP I-кадра. В режиме внутрикадрового обновления может потребоваться вставка IDR-кадра; этот QP управляет качеством вставленного IDR. Чем выше качество, тем больше IDR; чем ниже качество, тем меньше IDR. Диапазон: [1, 51]. Настройка этого параметра сейчас не поддерживается.

【Примечания】

- Все поля являются динамическими атрибутами.

5.55 VENC_HIERARCHICAL_QP_S

【Описание】

Структура параметров иерархического QP.

【Определение】

```
typedef struct rkVENC_HIERARCHICAL_QP_S {
    RK_BOOL     bHierarchicalQpEn;
    RK_S32      s32HierarchicalQpDelta[4];
    RK_S32      s32HierarchicalFrameNum[4];
} VENC_HIERARCHICAL_QP_S;
```

【Члены】

Имя члена	Описание
bHierarchicalQpEn	Включён ли иерархический QP. RK_TRUE: включено; RK_FALSE: выключено. По умолчанию выключено.
s32HierarchicalQpDelta	Разница QpDelta между Qp P-кадра слоя 0 и Qp каждого слоя: QpDelta = Qp(P0) - Qp(текущего слоя). Положительное значение означает, что qp текущего слоя меньше qp P-кадра слоя 0, качество P-кадра слоя выше, а объём данных больше. Диапазон: [-10, 10]. По умолчанию 0.
s32HierarchicalFrameNum	Число P-кадров каждого слоя. Диапазон: [0, 5]. По умолчанию 0.

Примечания

- Все поля являются динамическими атрибутами.

5.56 VENC_H264_INTRA_PRED_S

Описание

Определяет структуру внутрикадрового предсказания канала H.264.

Определение

```
typedef struct rkVENC_H264_INTRA_PRED_S {
    RK_U32     constrained_intra_pred_flag;
} VENC_H264_INTRA_PRED_S
```

Члены

Имя члена	Описание
constrained_intra_pred_flag	По умолчанию 0. Диапазон: 0 или 1.

Примечания

- Подробное значение приведённых параметров см. в спецификации H.264.
- Все поля являются динамическими атрибутами.

5.57 VENC_H264_TRANS_S

Описание

Определяет структуру преобразования и квантования канала H.264.

Определение

```
typedef struct rkVENC_H264_TRANS_S {
    RK_U32     u32TransMode;
    RK_BOOL    bScalingListValid;
    RK_U8      InterScalingList8X8[64];
    RK_U8      IntraScalingList8X8[64];
    RK_S32     chroma_qp_index_offset;
} VENC_H264_TRANS_S;
```

Члены

Имя члена	Описание
u32TransMode	Режим преобразования внутрикадрового и межкадрового предсказания: 0: поддерживаются преобразования 4x4 и 8x8; поддерживаются high profile и svc-t. 1: преобразование 4x4. Системное значение по умолчанию 0. Динамический атрибут.
bScalingListValid	Флаг действительности InterScalingList8x8 и IntraScalingList8x8; имеет смысл только для high profile и svc-t. Диапазон: 0 или 1. 0 — недействительно, 1 — действительно. Поддерживается только 0.
InterScalingList8X8[64]	Таблица квантования межкадрового предсказания 8x8. В high profile и svc-t пользователь может задать собственную таблицу. Зарезервировано, сейчас не используется. Диапазон: [1, 255].
IntraScalingList8X8[64]	Таблица квантования внутрикадрового предсказания 8x8. В high profile и svc-t пользователь может задать собственную таблицу. Зарезервировано, сейчас не используется. Диапазон: [1, 255].
chroma_qp_index_offset	Подробности см. в спецификации H.264. По умолчанию -6. Диапазон: [-12, 12]. Динамический атрибут.

Примечания

- Подробное значение приведённых параметров см. в спецификации H.264.
- Таблицы квантования не поддерживаются, поэтому bScalingListValid, InterScalingList8X8[64] и IntraScalingList8X8[64] задать нельзя.

5.58 VENC_H264_ENTROPY_S

Описание

Определяет структуру энтропийного кодирования канала H.264.

Определение

```
typedef struct rkVENC_H264_ENTROPY_S {
    RK_U32 u32EntropyEncMode;
    RK_U32 cabac_init_idc;
} VENC_H264_ENTROPY_S;
```

Члены

Имя члена	Описание
u32EntropyEncMode	Режим энтропийного кодирования: 0 — cavlc, 1 — cabac. По умолчанию 1.
cabac_init_idc	Диапазон: [0, 2], по умолчанию 0; подробности см. в спецификации H.264.

Примечания

- Все поля являются динамическими атрибутами.

5.59 VENC_H264_DBLK_S

Описание

Определяет структуру Dblk канала H.264.

Определение

```
typedef struct rkVENC_H264_DBLK_S {
    RK_U32 disable_deblocking_filter_idc;
    RK_S32 slice_alpha_c0_offset_div2;
    RK_S32 slice_beta_offset_div2;
} VENC_H264_DBLK_S;
```

【Члены】

Имя члена	Описание
disable_deblocking_filter_idc	Диапазон: [0, 2], по умолчанию 0; подробности см. в спецификации H.264.
slice_alpha_c0_offset_div2	Диапазон: [-6, 6], по умолчанию 0; подробности см. в спецификации H.264.
slice_beta_offset_div2	Диапазон: [-6, 6], по умолчанию 0; подробности см. в спецификации H.264.

【Примечания】

- Все поля являются динамическими атрибутами.

5.60 VENC_H264_VUI_S

【Описание】

Определяет структуру Vui канала H.264.

【Определение】

```
typedef struct rkVENC_H264_VUI_S {
    VENC_VUI_ASPECT_RATIO_S           stVuiAspectRatio;
    VENC_VUI_H264_TIME_INFO_S         stVuiTimeInfo;
    VENC_VUI_VIDEO_SIGNAL_S           stVuiVideoSignal;
    VENC_VUI_BITSTREAM_RESTRIC_S      stVuiBitstreamRestric;
} VENC_H264_VUI_S;
```

【Члены】

Имя члена	Описание
VENC_VUI_ASPECT_RATIO_S	Подробности см. в спецификации H.264. Настройка не поддерживается.
VENC_VUI_H264_TIME_INFO_S	Подробности см. в спецификации H.264. Настройка не поддерживается.
VENC_VUI_VIDEO_SIGNAL_S	Подробности см. в спецификации H.264. Поддерживается часть параметров.
VENC_VUI_BITSTREAM_RESTRIC_S	Подробности см. в спецификации H.264. Настройка не поддерживается.

【Примечания】

- Все поля являются динамическими атрибутами.

5.61 VENC_VUI_ASPECT_RATIO_S

【Описание】

Определяет структуру AspectRatio в Vui канала H.264/H.265.

【Определение】

```
typedef struct rkVENC_VUI_ASPECT_RATIO_S {
    RK_U8  aspect_ratio_info_present_flag;
    RK_U8  aspect_ratio_idc;
    RK_U8  overscan_info_present_flag;
    RK_U8  overscan_appropriate_flag;
    RK_U16 sar_width;
    RK_U16 sar_height;
} VENC_VUI_ASPECT_RATIO_S;
```

【Члены】

Имя члена	Описание
aspect_ratio_info_present_flag	Подробности см. в спецификации H.264/H.265. По умолчанию 0. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.
aspect_ratio_idc	Подробности см. в спецификации H.264/H.265. По умолчанию 0. Диапазон: [0, 255], 17-254 зарезервированы. Настройка этого параметра сейчас не поддерживается.
overscan_info_present_flag	Подробности см. в спецификации H.264/H.265. По умолчанию 0. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.
overscan_appropriate_flag	Подробности см. в спецификации H.264/H.265. По умолчанию 0. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.
sar_width	sar_width Подробности см. в спецификации H.264/H.265. По умолчанию 0. Диапазон: (0, 65535]; взаимно просто с sar_height. Настройка этого параметра сейчас не поддерживается.
sar_height	sar_width Подробности см. в спецификации H.264/H.265. По умолчанию 0. Диапазон: (0, 65535]; взаимно просто с sar_height Взаимно просто с sar_width. Настройка этого параметра сейчас не поддерживается.

【Примечания】

- Все поля являются динамическими атрибутами.

5.62 VENC_VUI_H264_TIME_INFO_S

【Описание】

Определяет структуру Time_Info в Vui канала H.264.

【Определение】

```
typedef struct rkVENC_H264_VUI_TIME_INFO_S {
    RK_U8  timing_info_present_flag;
    RK_U8  fixed_frame_rate_flag;
    RK_U32 num_units_in_tick;
    RK_U32 time_scale;
} VENC_VUI_H264_TIME_INFO_S;
```

【Члены】

Имя члена	Описание
timing_info_present_flag	Подробности см. в спецификации H.264. По умолчанию 1. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.
fixed_frame_rate_flag	Подробности см. в спецификации H.264. По умолчанию 1. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.
num_units_in_tick	Подробности см. в спецификации H.264. По умолчанию 1. Диапазон: больше 0. Настройка этого параметра сейчас не поддерживается.
time_scale	Подробности см. в спецификации H.264. По умолчанию 60. Диапазон: больше 0. Настройка этого параметра сейчас не поддерживается.

Примечания

- Все поля являются динамическими атрибутами.

5.63 VENC_VUI_VIDEO_SIGNAL_S

Описание

Определяет структуру Video_Signal в Vui канала H.264/H.265.

Определение

```
typedef struct rkVENC_VUI_VIDEO_SIGNAL_S {
    RK_U8  video_signal_type_present_flag;
    RK_U8  video_format;
    RK_U8  video_full_range_flag;
    RK_U8  colour_description_present_flag;
    RK_U8  colour_primaries;
    RK_U8  transfer_characteristics;
    RK_U8  matrix_coefficients;
} VENC_VUI_VIDEO_SIGNAL_S;
```

Члены

Имя члена	Описание
video_signal_type_present_flag	Подробности см. в спецификации H.264/H.265. По умолчанию 1. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.
video_format	video_format Подробности см. в спецификации H.264/H.265. По умолчанию 5. Диапазон: H.264 [0, 7], H.265 согласно спецификации. [0, 5]。 Настройка этого параметра сейчас не поддерживается.
video_full_range_flag	0：limit color range； 1：full color range。 video_full_range_flag По умолчанию 1. Диапазон: 0 или 1.
colour_description_present_flag	colour_description_present_flag Подробности см. в спецификации H.264/H.265. По умолчанию 0. Только для чтения. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.
colour_primaries	colour_primaries/transfer_characteristics/matrix_coefficients Подробности см. в спецификации H.264/H.265. По умолчанию 0. Только для чтения; заполняется внутри по фактической информации изображения. Диапазон: [0, 255]. Настройка этого параметра сейчас не поддерживается.
transfer_characteristics	colour_primaries/transfer_characteristics/matrix_coefficients Подробности см. в спецификации H.264/H.265. По умолчанию 0. Только для чтения; заполняется внутри по фактической информации изображения. Диапазон: [0, 255]. Настройка этого параметра сейчас не поддерживается.
matrix_coefficients	colour_primaries/transfer_characteristics/matrix_coefficients Подробности см. в спецификации H.264/H.265. По умолчанию 0. Только для чтения; заполняется внутри по фактической информации изображения. Диапазон: [0, 255]. Настройка этого параметра сейчас не поддерживается.

【Примечания】

- Все поля являются динамическими атрибутами.

5.64 VENC_VUI_BITSTREAM_RESTRIC_S

【Описание】

Определяет структуру Bitstream_Restriction в Vui канала H.264/H.265.

【Определение】

```
typedef struct rkVENC_VUI_BITSTREAM_RESTRIC_S {
    RK_U8  bitstream_restriction_flag;
} VENC_VUI_BITSTREAM_RESTRIC_S;
```

【Члены】

Имя члена	Описание
bitstream_restriction_flag	Подробности см. в спецификации H.264/H.265. По умолчанию 1. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.

【Примечания】

- Все поля являются динамическими атрибутами.

5.65 VENC_H265_TRANS_S

【Описание】

Определяет структуру преобразования и квантования канала H.265.

【Определение】

```
typedef struct rkVENC_H265_TRANS_S {
    RK_S32  cb_qp_offset;
    RK_S32  cr_qp_offset;
    RK_BOOL bScalingListEnabled;
    RK_BOOL bScalingListTu4Valid;
    RK_U8   InterScalingList4X4[2][16];
    RK_U8   IntraScalingList4X4[2][16];
    RK_BOOL bScalingListTu8Valid;
    RK_U8   InterScalingList8X8[2][64];
    RK_U8   IntraScalingList8X8[2][64];
    RK_BOOL bScalingListTu16Valid;
    RK_U8   InterScalingList16X16[2][64];
    RK_U8   IntraScalingList16X16[2][64];
    RK_BOOL bScalingListTu32Valid;
    RK_U8   InterScalingList32X32[64];
    RK_U8   IntraScalingList32X32[64];
} VENC_H265_TRANS_S;
```

【Члены】

Имя члена	Описание
cb_qp_offset	По умолчанию -6, диапазон: [-12, 12].
cr_qp_offset	По умолчанию -6, диапазон: [-12, 12].
bScalingListEnabled	По умолчанию 0. Зарезервировано, сейчас не используется.
bScalingListTu4Valid	По умолчанию 0. Зарезервировано, сейчас не используется.
`InterScalingList4X4[2][16]`	По умолчанию 0. Зарезервировано, сейчас не используется.
`IntraScalingList4X4[2][16]`	По умолчанию 0. Зарезервировано, сейчас не используется.
bScalingListTu8Valid	По умолчанию 0. Зарезервировано, сейчас не используется.
InterScalingList8X8[2	По умолчанию 0. Зарезервировано, сейчас не используется.
`IntraScalingList8X8[2`][64]	По умолчанию 0. Зарезервировано, сейчас не используется.
bScalingListTu16Valid	По умолчанию 0. Зарезервировано, сейчас не используется.
`InterScalingList16X16[2][64]`	По умолчанию 0. Зарезервировано, сейчас не используется.
`IntraScalingList16X16[2`][64]	По умолчанию 0. Зарезервировано, сейчас не используется.
bScalingListTu32Valid	По умолчанию 0. Зарезервировано, сейчас не используется.
InterScalingList32X32[64]	По умолчанию 0. Зарезервировано, сейчас не используется.
IntraScalingList32X32[64]	По умолчанию 0. Зарезервировано, сейчас не используется.

【Примечания】

- Подробное значение параметров см. в спецификации H.265.
- Все поля являются динамическими атрибутами.

5.66 VENC_H265_ENTROPY_S

【Описание】

Определяет структуру энтропийного кодирования канала H.265.

【Определение】

```
typedef struct rkVENC_H265_ENTROPY_S {
    RK_U32 cabac_init_flag;
} VENC_H265_ENTROPY_S;
```

【Члены】

Имя члена	Описание
cabac_init_flag	По умолчанию 0, диапазон: 0 или 1. Зарезервировано, сейчас не используется.

【Примечания】

- Все поля являются динамическими атрибутами.

5.67 VENC_H265_DBLK_S

【Описание】

Определяет структуру Deblocking канала H.265.

【Определение】

```
typedef struct rkVENC_H265_DBLK_S {
    RK_U32 slice_deblocking_filter_disabled_flag;
    RK_S32 slice_beta_offset_div2;
    RK_S32 slice_tc_offset_div2;
} VENC_H265_DBLK_S;
```

【Члены】

Имя члена	Описание
slice_deblocking_filter_disabled_flag	По умолчанию 0. Диапазон: 0 или 1.
slice_beta_offset_div2	По умолчанию 0. Диапазон: [-6, 6].
slice_tc_offset_div2	По умолчанию 0. Диапазон: [-6, 6].

【Примечания】

- Подробное значение параметров см. в спецификации H.265.
- Все поля являются динамическими атрибутами.

5.68 VENC_H265_SAO_S

【Описание】

Определяет структуру Sao канала H.265.

【Определение】

```
typedef struct rkVENC_H265_SAO_S {
    RK_U32  slice_sao_luma_flag;
    RK_U32  slice_sao_chroma_flag;
} VENC_H265_SAO_S;
```

【Члены】

Имя члена	Описание
slice_sao_luma_flag	По умолчанию 1. Диапазон: 0 или 1.
slice_sao_chroma_flag	По умолчанию 1. Диапазон: 0 или 1.

【Примечания】

- Подробное значение параметров см. в спецификации H.265.
- Все поля являются динамическими атрибутами.

5.69 VENC_H265_PU_S

【Описание】

Определяет структуру PU канала H.265.

【Определение】

```
typedef struct rkVENC_H265_PU_S {
    RK_U32    constrained_intra_pred_flag;
    RK_U32    strong_intra_smoothing_enabled_flag;
} VENC_H265_PU_S;
```

【Члены】

Имя члена	Описание
constrained_intra_pred_flag	По умолчанию 0. Диапазон: 0 или 1. Зарезервировано, сейчас не используется.
strong_intra_smoothing_enabled_flag	По умолчанию 1. Диапазон: 0 или 1.

【Примечания】

- Подробное значение параметров см. в спецификации H.265.
- Все поля являются динамическими атрибутами.

5.70 VENC_H265_VUI_S

【Описание】

Определяет структуру Vui канала H.265.

【Определение】

```
typedef struct rkVENC_H265_VUI_S {
    VENC_VUI_ASPECT_RATIO_S           stVuiAspectRatio;
    VENC_VUI_H265_TIME_INFO_S         stVuiTimeInfo;
    VENC_VUI_VIDEO_SIGNAL_S           stVuiVideoSignal;
    VENC_VUI_BITSTREAM_RESTRIC_S      stVuiBitstreamRestric;
} VENC_H265_VUI_S;
```

【Члены】

Имя члена	Описание
VENC_VUI_ASPECT_RATIO_S	Подробности см. в спецификации H.265. Настройка не поддерживается.
VENC_VUI_H265_TIME_INFO_S	Подробности см. в спецификации H.265. Настройка не поддерживается.
VENC_VUI_VIDEO_SIGNAL_S	Подробности см. в спецификации H.265. Поддерживается часть параметров.
VENC_VUI_BITSTREAM_RESTRIC_S	Подробности см. в спецификации H.265. Настройка не поддерживается.

【Примечания】

- Все поля являются динамическими атрибутами.

5.71 VENC_VUI_H265_TIME_INFO_S

【Описание】

Определяет структуру Time_Info в Vui канала H.265.

【Определение】

```
typedef struct rkVENC_VUI_H265_TIME_INFO_S {
    RK_U32 timing_info_present_flag;
    RK_U32 num_units_in_tick;
    RK_U32 time_scale;
    RK_U32 num_ticks_poc_diff_one_minus1;
} VENC_VUI_H265_TIME_INFO_S;
```

【Члены】

Имя члена	Описание
timing_info_present_flag	Подробности см. в спецификации H.265. По умолчанию 1. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.
fixed_frame_rate_flag	Подробности см. в спецификации H.265. По умолчанию 1. Диапазон: 0 или 1. Настройка этого параметра сейчас не поддерживается.
num_units_in_tick	Подробности см. в спецификации H.265. По умолчанию 1. Диапазон: больше 0. Настройка этого параметра сейчас не поддерживается.
time_scale	Подробности см. в спецификации H.265. По умолчанию 60. Диапазон: больше 0. Настройка этого параметра сейчас не поддерживается.

【Примечания】

- Все поля являются динамическими атрибутами.

5.72 VENC_MJPEG_PARAM_S

【Описание】

Определяет структуру расширенных параметров канала MJPEG.

【Определение】

```
typedef struct rkVENC_MJPEG_PARAM_S {
    RK_U8 u8YQt[64];
    RK_U8 u8CbQt[64];
    RK_U8 u8CrQt[64];
    RK_U32 u32MCUPerECS;
} VENC_MJPEG_PARAM_S;
```

Члены

Имя члена	Описание
u8YQt	Таблица квантования Y. Диапазон: [1, 255].
u8CbQt	Таблица квантования Cb. Диапазон: [1, 255].
u8CrQt	Таблица квантования Cr. Диапазон: [1, 255].
u32MCUPerECS	Число MCU в каждом ECS. По умолчанию 0, то есть ECS не разделяется. Минимальное значение 0, максимальное зависит от входного формата: для yuv420sp: ((picwidth+15)>>4) * ((picheight+15)>>4); для yuv422sp: ((picwidth+15)>>4) * ((picheight+15)>>4) * 2; для yuv444sp: ((picwidth+15)>>4) * ((picheight+15)>>4) * 4.

Примечания

- Все поля являются динамическими атрибутами.

5.73 VENC_REF_PARAM_S

Описание

Определяет расширенные параметры пропуска кадров и опорных кадров H.264/H.265.

Определение

```
typedef struct rkVENC_REF_PARAM_S {
    RK_U32       u32Base;
    RK_U32       u32Enhance;
    RK_BOOL      bEnablePred;
} VENC_REF_PARAM_S;
```

Члены

Имя члена	Описание
u32Base	Период базового слоя. Диапазон: (0, +□).
u32Enhance	Период слоя enhance. Диапазон: [0, 255].
bEnablePred	Используются ли кадры базового слоя как опорные другими кадрами базового слоя. Если bEnablePred = RK_FALSE, это эквивалентно бесконечному u32Base, и все кадры базового слоя ссылаются на IDR-кадр.

Примечания

- Все поля являются динамическими атрибутами.

5.74 ROTATION_E

Описание

Определяет перечисление углов поворота.

Определение

```
typedef enum rkROTATION_E {
    ROTATION_0        = 0,
    ROTATION_90       = 1,
    ROTATION_180      = 2,
    ROTATION_270      = 3,
    ROTATION_BUTT
} ROTATION_E;
```

Члены

Имя члена	Описание
ROTATION_0	Без поворота.
ROTATION_90	Поворот на 90 градусов.
ROTATION_180	Поворот на 180 градусов.
ROTATION_270	Поворот на 270 градусов.
ROTATION_BUTT	Максимальное значение.

Примечания

- При одновременном повороте и зеркальном отражении сначала выполняется поворот.

5.75 MIRROR_E

Описание

Определяет перечисление зеркального отражения.

Определение

```
typedef enum rkROTATION_E {
    MIRROR_NONE,
    MIRROR_HORIZONTAL,
    MIRROR_VERTICAL,
    MIRROR_BOTH,
    MIRROR_BUTT
} ROTATION_E;
```

Члены

Имя члена	Описание
MIRROR_NONE	Без зеркального отражения.
MIRROR_HORIZONTAL	Горизонтальное отражение.
MIRROR_VERTICAL	Вертикальное отражение.
MIRROR_BOTH	Горизонтальное и вертикальное отражение, эквивалентно повороту на 180 градусов.
MIRROR_BUTT	Максимальное значение.

Примечания

- При одновременном повороте и зеркальном отражении сначала выполняется поворот.

5.76 USER_FRAME_INFO_S

【Описание】

Структура информации об изображении, отправленном пользователем.

【Определение】

```
typedef struct rkUSER_FRAME_INFO_S {
    VIDEO_FRAME_INFO_S stUserFrame;
    USER_RC_INFO_S     stUserRcInfo;
} USER_FRAME_INFO_S;
```

【Члены】

Имя члена	Описание
stUserFrame	Структура информации о кадре, отправленном пользователем.
stUserRcInfo	Структура пользовательской информации управления битрейтом.

【Примечания】

Нет.

5.77 USER_RC_INFO_S

【Описание】

Структура пользовательской информации управления битрейтом.

【Определение】

```
typedef struct rkUSER_RC_INFO_S {
    RK_BOOL bQpMapValid;
    RK_BOOL bSkipWeightValid;
    RK_U32  u32BlkStartQp;
    MB_BLK  pMbBlkQpMap;
    MB_BLK  pMbBlkSkipWeight;
    VENC_FRAME_TYPE_E enFrameType;
} USER_RC_INFO_S;
```

【Члены】

Имя члена	Описание
bQpMapValid	Действует ли таблица Qp в режиме QPMAP.
bSkipWeightValid	Действует ли таблица SkipWeight в режиме QPMAP. Сейчас не используется.
u32BlkStartQp	Qp первого блока 16*16 в режиме QPMAP. Сейчас не используется. Диапазон: [0, 51].
pMbBlkQpMap	MB_BLK таблицы Qp в режиме QPMAP.
pMbBlkSkipWeight	MB_BLK таблицы SkipWeight в режиме QPMAP. Сейчас не используется.
enFrameType	Тип кодирования текущего кадра. Сейчас не используется.

【Примечания】

Нет.

5.78 VENC_DEBREATHEFFECT_S

【Описание】

Структура параметров устранения эффекта дыхания.

【Определение】

```
typedef struct rkVENC_DEBREATHEFFECT_S {
    RK_BOOL   bEnable;
    RK_S32    s32Strength0;
    RK_S32    s32Strength1;
} VENC_DEBREATHEFFECT_S;
```

【Члены】

Имя члена	Описание
bEnable	Включено ли устранение эффекта дыхания. RK_TRUE: включено; RK_FALSE: выключено.
s32Strength0	Параметр интенсивности устранения эффекта дыхания 0. Не используется.
s32Strength1	s32Strength1 Параметр интенсивности устранения эффекта дыхания 1. Чем больше значение, тем больше I-кадр; чем меньше — тем меньше I-кадр. Диапазон: [0, 35]. По умолчанию 16.

5.79 VENC_CHN_REF_BUF_SHARE_S

【Описание】

Структура параметров совместного использования опорных кадров.

【Определение】

```
typedef struct rkVENC_CHN_REF_BUF_SHARE_S {
    RK_BOOL bEnable;
} VENC_CHN_REF_BUF_SHARE_S;
```

【Члены】

Имя члена	Описание
bEnable	Включено ли совместное использование опорных кадров. RK_TRUE: включено; RK_FLASE: выключено.

5.80 VENC_COMBO_ATTR_S

【Описание】

Структура параметров атрибутов Combo.

【Определение】

```
typedef struct rkVENC_COMBO_ATTR_S {
    RK_BOOL bEnable;
    RK_S32  s32ChnId;
} VENC_COMBO_ATTR_S;
```

【Члены】

Имя члена	Описание
bEnable	Включён ли Combo. RK_TRUE: включено; RK_FLASE: выключен.
s32ChnId	Канал-источник данных Combo.

5.81 VENC_CHN_BUF_WRAP_S

【Описание】

Структура параметров кольцевого Buf.

【Определение】

```
typedef struct rkVENC_CHN_BUF_WRAP_S {
    RK_BOOL bEnable;
    RK_U32  u32BufLine;
    RK_U32  u32WrapBufferSize;
} VENC_CHN_BUF_WRAP_S;
```

【Члены】

Имя члена	Описание
bEnable	Включён ли кольцевой Buf. RK_TRUE: включено; RK_FALSE: выключено.
u32BufLine	Число строк кольцевого Buf.
u32WrapBufferSize	Размер кольцевого Buf.

5.82 VENC_SLICE_SPLIT_S

【Описание】

Структура разбиения SLICE канала кодирования.

【Определение】

```
typedef struct rkVENC_SLICE_SPLIT_S {
    RK_BOOL bSplitEnable;
    RK_U32  u32SplitMode;
    RK_U32  u32SplitSize;
} VENC_SLICE_SPLIT_S;
```

【Члены】

Имя члена	Описание
bSplitEnable	Включено ли разбиение slice. RK_TRUE: включено; RK_FALSE: выключено. По умолчанию выключено.
u32SplitMode	Режим разбиения slice. 0: по числу байтов; 1: по числу MB или LCU.
u32SplitSize	Размер разбиения slice. При split_mode = 0 — число байтов каждого slice; при split_mode = 1 — число MB или LCU каждого slice.

Примечания

- H264 поддерживает не более 500 slice, H265 — не более 50. Оставшаяся область после достижения максимума кодируется как один slice.

  slice。
- JPEG/MJPEG поддерживает разбиение только по макроблокам. Размер макроблока зависит от YUV: 420 — 16x16, 422 — 16x8, 444 — 8x8.
- Размер MB кодирования H264 — 16x16.
- Размер LCU H265: для rv1106/rv1103 — 32x32, для остальных микросхем — 64x64.

5.83 VENC_PARAM_MOD_S

Описание

Структура параметров модулей кодирования.

Определение

```
typedef struct rkVENC_MODPARAM_S {
    VENC_MODTYPE_E enVencModType;
    VENC_MOD_VENC_S  stVencModParam;
    VENC_MOD_H264E_S stH264eModParam;
    VENC_MOD_H265E_S stH265eModParam;
    VENC_MOD_JPEGE_S stJpegeModParam;
    VENC_MOD_RC_S    stRcModParam;
} VENC_PARAM_MOD_S;
```

Члены

Имя члена	Описание
enVencModType	Тип параметров модуля для задания или получения.
stVencModParam/ stH264eModParam/ stH265eModParam/ stJpegeModParam/stRcModParam	Структуры параметров отдельных модулей кодирования.

Примечания

- Нет

5.84 VENC_MODTYPE_E

Описание

Типы параметров модулей кодирования.

Определение

```
typedef enum rkVENC_MODTYPE_E {
    MODTYPE_VENC = 1,
    MODTYPE_H264E,
    MODTYPE_H265E,
    MODTYPE_JPEGE,
    MODTYPE_RC,
    MODTYPE_BUTT
} VENC_MODTYPE_E;
```

【Члены】

Имя члена	Описание
MODTYPE_VENC	Тип параметров модуля VENC.
MODTYPE_H264E	Тип параметров модуля H264.
MODTYPE_H265E	Тип параметров модуля H265.
MODTYPE_JPEGE	Тип параметров модуля JPEG/MJPEG.
MODTYPE_RC	Тип параметров модуля RC.
MODTYPE_BUTT	Максимальное значение.

【Примечания】

- Нет

5.85 VENC_MOD_VENC_S

【Описание】

Структура параметров модуля VENC.

【Определение】

```
typedef struct rkVENC_MOD_VENC_S {
    RK_U32 u32VencBufferCache;
    RK_U32 u32FrameBufRecycle;
} VENC_MOD_VENC_S;
```

【Члены】

Имя члена	Описание
u32VencBufferCache	Поддерживает ли получение потока режим cache. 0: выключить Cache буфера потока. 1: включить Cache буфера потока. По умолчанию 0. Сейчас не используется.
u32FrameBufRecycle	Освобождается ли память кадров повторно. 0: выключить повторное использование памяти кадров кодирования. 1: включить повторное использование памяти кадров кодирования. По умолчанию 0. Сейчас не используется.

【Примечания】

- Нет

5.86 VENC_MOD_H264E_S

【Описание】

Структура параметров модуля H264.

【Определение】

```
typedef struct rkVENC_MOD_H264E_S {
    RK_U32          u32OneStreamBuffer;
    RK_U32          u32H264eMiniBufMode;
    RK_U32          u32H264ePowerSaveEn;
    MB_SOURCE_E     enH264eMBSource;
    RK_BOOL         bQpHstgrmEn;
} VENC_MOD_H264E_S;
```

【Члены】

Имя члена	Описание
u32OneStreamBuffer	Режим конфигурации кадра кодированного потока. 0: многопакетный режим. 1: однопакетный режим. По умолчанию 1.
u32H264eMiniBufMode	Режим конфигурации buffer кодированного потока. 0: buffer потока выделяется по разрешению. 1: нижний предел buffer потока 32 КБ; пользователь обязан обеспечить подходящий размер. По умолчанию 0. Сейчас не используется.
u32H264ePowerSaveEn	Режим низкого энергопотребления. 0: выключен. 1: включён низкопотребляющий режим. 2: включён режим крайне низкого энергопотребления. По умолчанию 0. Сейчас не используется.
enH264eMBSource	Способ выделения памяти опорных и реконструированных кадров. По умолчанию 0. Сейчас не используется.
bQpHstgrmEn	Управление выводом гистограммы Qp. По умолчанию 0. Сейчас не используется.

【Примечания】

- Нет

5.87 VENC_MOD_H265E_S

【Описание】

Структура параметров модуля H265.

【Определение】

```
typedef struct rkVENC_MOD_H265E_S {
    RK_U32          u32OneStreamBuffer;
    RK_U32          u32H265eMiniBufMode;
    RK_U32          u32H265ePowerSaveEn;
    MB_SOURCE_E     enH265eMBSource;
    RK_BOOL         bQpHstgrmEn;
} VENC_MOD_H265E_S;
```

【Члены】

Имя члена	Описание
u32OneStreamBuffer	Режим конфигурации кадра кодированного потока. 0: многопакетный режим. 1: однопакетный режим. По умолчанию 1.
u32H265eMiniBufMode	Режим конфигурации buffer кодированного потока. 0: buffer потока выделяется по разрешению. 1: нижний предел buffer потока 32 КБ; пользователь обязан обеспечить подходящий размер. По умолчанию 0. Сейчас не используется.
u32H265ePowerSaveEn	Режим низкого энергопотребления. 0: выключен. 1: включён низкопотребляющий режим. 2: включён режим крайне низкого энергопотребления. По умолчанию 0. Сейчас не используется.
enH265eMBSource	Способ выделения памяти опорных и реконструированных кадров. По умолчанию 0. Сейчас не используется.
bQpHstgrmEn	Управление выводом гистограммы Qp. По умолчанию 0. Сейчас не используется.

【Примечания】

- Нет

5.88 VENC_MOD_JPEGE_S

【Описание】

Структура параметров модуля JPEG/MJPEG.

【Определение】

```
typedef struct rkVENC_MOD_JPEGE_S {
    RK_U32  u32OneStreamBuffer;
    RK_U32  u32JpegeMiniBufMode;
    RK_U32  u32JpegClearStreamBuf;
} VENC_MOD_JPEGE_S;
```

【Члены】

Имя члена	Описание
u32OneStreamBuffer	Режим конфигурации кадра кодированного потока. 0: многопакетный режим. 1: однопакетный режим. По умолчанию 1. JPEG/MJPEG сейчас поддерживает только однопакетный режим.
u32JpegeMiniBufMode	Режим конфигурации buffer кодированного потока. 0: buffer потока выделяется по разрешению. 1: нижний предел buffer потока 32 КБ; пользователь обязан обеспечить подходящий размер. По умолчанию 0. Сейчас не используется.
u32JpegClearStreamBuf	Очищать ли buffer потока при задании атрибутов канала JPEG. По умолчанию 0. Сейчас не используется.

【Примечания】

- Нет

5.89 VENC_MOD_RC_S

【Описание】

Структура параметров модуля RC.

【Определение】

```
typedef struct rkVENC_MOD_RC_S {
    RK_U32  u32ClrStatAfterSetBr;
} VENC_MOD_RC_S;
```

【Члены】

Имя члена	Описание
u32ClrStatAfterSetBr	Очищать ли статистику RC при задании битрейта канала. По умолчанию 0. Сейчас не используется.

【Примечания】

- Нет

6. Коды ошибок VENC

Ниже приведены коды ошибок API видеокодирования VENC:

Код ошибки	Макроопределение	Описание
0xA0048002	RK_ERR_VENC_INVALID_CHNID	ID канала вне допустимого диапазона.
0xA0048003	RK_ERR_VENC_ILLEGAL_PARAM	Параметр вне допустимого диапазона.
0xA0048004	RK_ERR_VENC_EXIST	Попытка создать или запросить уже существующие устройство, канал или ресурс.
0xA0048005	RK_ERR_VENC_UNEXIST	Попытка использовать или уничтожить несуществующие устройство, канал или ресурс.
0xA0048006	RK_ERR_VENC_NULL_PTR	Пустой указатель в параметрах функции.
0xA0048007	RK_ERR_VENC_NOT_CONFIG	Не настроено перед использованием.
0xA0048008	RK_ERR_VENC_NOT_SUPPORT	Неподдерживаемый параметр или функция.
0xA0048009	RK_ERR_VENC_NOT_PERM	Операция не разрешена, например изменение статического параметра.
0xA004800C	RK_ERR_VENC_NOMEM	Не удалось выделить память, например недостаточно системной памяти.
0xA004800D	RK_ERR_VENC_NOBUF	Не удалось выделить буфер, например запрошен слишком большой буфер данных.
0xA004800E	RK_ERR_VENC_BUF_EMPTY	В буфере нет данных.
0xA004800F	RK_ERR_VENC_BUF_FULL	Буфер заполнен данными.
0xA0048010	RK_ERR_VENC_SYS_NOTREADY	Система не инициализирована или соответствующий модуль не загружен.
0xA0048012	RK_ERR_VENC_BUSY	Система VENC занята.

Видеовыход

Предисловие

Обзор

Документ описывает модуль видеовывода VO для RK356X/RK3588 и соответствующие API.

Версии продуктов

Название микросхемы	Версия ядра
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	XXH AYJ	2020-01-09	Первая версия.
V0.2.0	ZY	2020-03-10	Добавлено описание возвращаемых значений.
V0.2.1	XXH	2021-04-25	Добавлены примеры.
V0.2.2	XXH	2021-08-03	Добавлено описание отрисовки UI.
V0.2.3	ZY	2021-12-06	Добавлено описание VOP RK3588.
V0.2.4	SCH	2021-12-24	Добавлено описание части параметров атрибутов.
V0.2.5	ZY	2022-01-05	Добавлена схема связей выходных интерфейсов VOP и примечания.
V0.2.6	ZY	2022-04-20	Добавлено описание ID слоёв и атрибутов виртуальных слоёв.
V0.2.7	MLC	2023-01-09	Добавлен атрибут максимальной длины очереди VO.
V0.2.8	SCH	2023-03-22	Добавлено описание настройки параметров низкой задержки VO.

Содержание

Обзор
Архитектура системы VOP
Функции модуля
Таблица ресурсов VO
Справочник API
Типы данных
Коды ошибок VO
Описание отрисовки UI
Виртуальный слой

1. Обзор

Модуль видеовывода VO является оболочкой функций видеовывода.

2. Архитектура системы VOP

Таблица 1. Производительность вывода VOP

Микросхема	ID устройства вывода	Максимальное разрешение
RK3568	0	4096x2160
	1	2048x1536
	2	1920x1080
RK3588	0	4096x2160(*)
	1	4096x2160(*)
	2	4096x2160
	3	2048x1536

Примечания:

(*) при выводе 8K на RK3588 выходы 0 и 1 необходимо объединить в одно устройство вывода 0.

Рисунок 1. Связь выходных интерфейсов VO RK3568 и VoDev

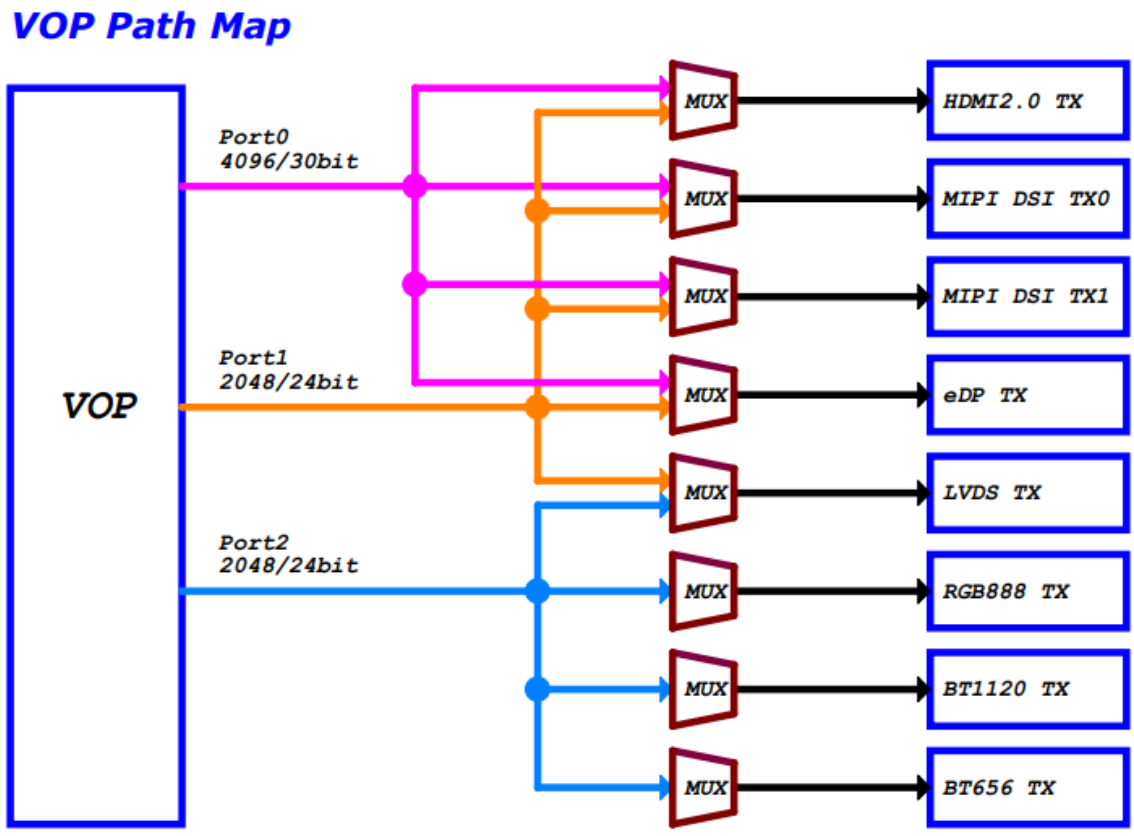
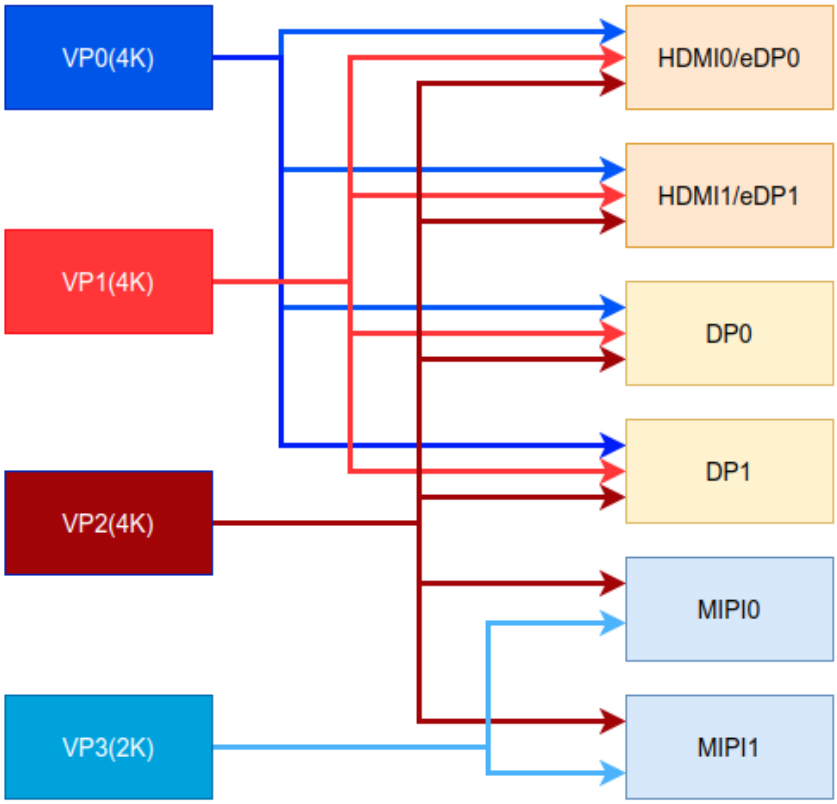


Рисунок 2. Связь выходных интерфейсов VO RK3588 и VoDev



Примечания

Если к одному VoDev RK3588 привязано несколько интерфейсов, формат цвета на всех выходах должен совпадать. Например, HDMI0 и DP0 одновременно выводят RGB888.

Таблица 2. Возможности слоёв

Микросхема	Имя слоя	ID слоя	Масштабирование	Сжатый формат	Формат YUV	Формат RGB
RK3568	Cluster0	0	Поддерживается	Поддерживается (*)	Поддерживается	Поддерживается
	Cluster1 2 Поддерживается Поддерживается (*) Поддерживается Поддерживается					
	Esmart0 4 Поддерживается Не поддерживается Поддерживается Поддерживается					
	Esmart1 5 Поддерживается Не поддерживается Поддерживается Поддерживается					
	Smart0 6 Не поддерживается Не поддерживается Не поддерживается Поддерживается					
	Smart1 7 Не поддерживается Не поддерживается Не поддерживается Поддерживается					
RK3588	Cluster0	0	Поддерживается	Поддерживается	Поддерживается (*)	Поддерживается
	Cluster1 1 Поддерживается Поддерживается Поддерживается (*) Поддерживается					
	Cluster2 2 Поддерживается Поддерживается Поддерживается (*) Поддерживается					
	Cluster3 3 Поддерживается Поддерживается Поддерживается (*) Поддерживается					
	Esmart0 4 Поддерживается Не поддерживается Поддерживается Поддерживается					
	Esmart1 5 Поддерживается Не поддерживается Поддерживается Поддерживается					
	Esmart2 6 Поддерживается Не поддерживается Поддерживается Поддерживается					
	Esmart3 7 Поддерживается Не поддерживается Поддерживается Поддерживается					

Примечания

- (*) поддерживаются только сжатые данные; несжатые не поддерживаются.
- ID слоёв см. в VO_LAYER_NAME_RK356X_E, VO_LAYER_NAME_E и VO_VIR_LAYER_NAME_E.

3. Функции модуля

Модуль видеовывода VO управляет видеовыводом, поддерживает несколько VOP и отображение нескольких слоёв. Он включает устройства и каналы вывода, отправляет видео или данные UI в выходные каналы и выполняет другие функции.

4. Таблица ресурсов VO

Имя модуля	Количество	Описание
VoChn	VO_MAX_CHN_NUM	Номер канала видеовывода.
VoDev	VO_MAX_DEV_NUM	Номер устройства вывода, диапазон [0, VO_MAX_DEV_NUM - 1].
VoLayer	VO_MAX_LAYER_NUM	VOP RK356x содержит 2 слоя cluster, 2 esmart и 2 smart.
VoWbc	VO_MAX_WBC_NUM	Устройство обратной записи. Микросхема поддерживает один тракт, поэтому VoWbc имеет единственное значение 0.

5. Справочник API

5.1 Операции с устройствами

5.1.1 RK_MPI_VO_SetPubAttr

【Описание】

Задаёт общие атрибуты устройства видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetPubAttr(VO_DEV VoDev, const VO_PUB_ATTR_S *pstPubAttr)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoDev	Номер устройства вывода.	Вход
pstPubAttr	Указатель на структуру общих атрибутов устройства видеовывода.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_NULL_PTR	pstPubAttr — пустой указатель.
RK_ERR_VO_NOT_SUPPORT	enIntfSync отсутствует в списке поддерживаемых.
RK_ERR_VO_BUSY	Не удалось задать разрешение.

【Пример】

См. пример RK_MPI_VO_Enable.

Связанные темы

RK_MPI_VO_GetPubAttr

5.1.2 RK_MPI_VO_GetPubAttr

Описание

Получает общие атрибуты устройства видеовывода.

Синтаксис

```
RK_S32 RK_MPI_VO_GetPubAttr(VO_DEV VoDev, VO_PUB_ATTR_S *pstPubAttr)
```

Параметры

Имя параметра	Описание	Вход/выход
VoDev	Номер устройства вывода.	Вход
pstPubAttr	Указатель на структуру общих атрибутов устройства видеовывода.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_NULL_PTR	pstPubAttr — пустой указатель.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы VO.

Пример

См. пример RK_MPI_VO_Enable.

Связанные темы

RK_MPI_VO_SetPubAttr

5.1.3 RK_MPI_VO_Enable

Описание

Включает устройство видеовывода.

Синтаксис

```
RK_S32 RK_MPI_VO_Enable(VO_DEV VoDev)
```

Параметры

Имя параметра	Описание	Вход/выход
VoDev	Номер устройства вывода.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы VO.
RK_ERR_VO_NOT_SUPPORT	Тип интерфейса вывода не поддерживается.
RK_ERR_VO_BUSY	Не удалось создать или включить устройство вывода.

【Пример】

```
    VO_PUB_ATTR_S VoPubAttr;
    VO_VIDEO_LAYER_ATTR_S stLayerAttr;
    RK_U32 s32Ret;

    VO_LAYER VoLayer = RK356X_VOP_LAYER_CLUSTER_0;
    VO_DEV VoDev = RK356X_VO_DEV_HD0;

    memset(&VoPubAttr, 0, sizeof(VO_PUB_ATTR_S));
    s32Ret = RK_MPI_VO_GetPubAttr(VoDev, &VoPubAttr);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }

    VoPubAttr.enIntfType = VO_INTF_HDMI;
    VoPubAttr.enIntfSync = VO_OUTPUT_1080P60;

    s32Ret = RK_MPI_VO_SetPubAttr(VoDev, &VoPubAttr);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }
    s32Ret = RK_MPI_VO_Enable(VoDev);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }

    s32Ret = RK_MPI_VO_Disable(VoDev);
    if (s32Ret != RK_SUCCESS)
        return s32Ret;

    s32Ret = RK_MPI_VO_CloseFd();
    if (s32Ret != RK_SUCCESS)
        return s32Ret;
```

【Связанные темы】

RK_MPI_VO_Disable

5.1.4 RK_MPI_VO_Disable

【Описание】

Выключает устройство видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_Disable(VO_DEV VoDev)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoDev	Номер устройства вывода.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.

【Пример】

```
    RK_S32 s32Ret = RK_SUCCESS;
    VO_DEV VoDev = RK356X_VO_DEV_HD0;

    s32Ret = RK_MPI_VO_Disable(VoDev);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;

    return s32Ret;
```

5.1.5 RK_MPI_VO_GetPostProcessParam

【Описание】

Получает параметры выходного изображения устройства.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetPostProcessParam(VO_DEV VoDev, VO_CSC_S *pstParam)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoDev	Номер устройства вывода.	Вход
pstParam	Указатель на структуру параметров выходного изображения.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_NULL_PTR	pstParam — пустой указатель.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы VO.

5.1.6 RK_MPI_VO_SetPostProcessParam

【Описание】

Задаёт параметры выходного изображения устройства.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetPostProcessParam(VO_DEV VoDev, VO_CSC_S *pstParam)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoDev	Номер устройства вывода.	Вход
pstParam	Указатель на структуру параметров выходного изображения.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_NULL_PTR	pstParam — пустой указатель.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы VO.
RK_ERR_VO_BUSY	Ошибка настройки.

5.1.7 RK_S32 RK_MPI_VO_Get_Edid

【Описание】

Получает данные EDID устройства отображения.

【Синтаксис】

```
RK_S32 RK_MPI_VO_Get_Edid(RK_U32 enIntfType, RK_U32 u32Id, VO_EDID_S *pstEdidData)
```

【Параметры】

Имя параметра	Описание	Вход/выход
enIntfType	Тип интерфейса вывода.	Вход
u32Id	Номер устройства интерфейса этого типа, например HDMI0 или HDMI1.	Вход
pstEdidData	Данные EDID.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый enIntfType.
RK_ERR_VO_NULL_PTR	pstEdidData — пустой указатель.
RK_ERR_VO_DEV_NOT_ENABLE	Устройство вывода, соответствующее VoDev, не включено.

【Примечания】

u32Id — ID интерфейса одного типа внутри микросхемы. Например, если встроено два HDMI, допустимы 0 и 1.

5.1.8 RK_S32 RK_MPI_VO_GetSinkCapability

【Описание】

Получает состояние подключения и возможности устройства отображения.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetSinkCapability(RK_U32 enIntfType, RK_U32 u32Id, VO_SINK_CAPABILITY_S
*pstSinkCap)
```

【Параметры】

Имя параметра	Описание	Вход/выход
enIntfType	Тип интерфейса вывода.	Вход
u32Id	Номер устройства интерфейса этого типа, например HDMI0 или HDMI1.	Вход
pstSinkCap	Состояние устройства отображения.	Выход

[Возвращаемое значение]

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый enIntfType.
RK_ERR_VO_NULL_PTR	pstSinkCap — пустой указатель.
RK_ERR_VO_DEV_NOT_ENABLE	Устройство вывода, соответствующее VoDev, не включено.

【Примечания】

- u32Id — ID интерфейса одного типа внутри микросхемы. Например, если встроено два HDMI, допустимы 0 и 1.

5.1.9 RK_MPI_VO_RegCallbackFunc

【Описание】

Регистрирует callback горячего подключения устройства отображения.

【Синтаксис】

```
RK_S32 RK_MPI_VO_RegCallbackFunc(RK_U32 enIntfType, RK_U32 u32Id, RK_VO_CALLBACK_FUNC_S
*pstCallbackFunc)
```

【Параметры】

Имя параметра	Описание	Вход/выход
enIntfType	Тип интерфейса вывода.	Вход
u32Id	Номер устройства интерфейса этого типа, например HDMI0 или HDMI1.	Вход
pstCallbackFunc	Callback горячего подключения.	Вход

[Возвращаемое значение]

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый enIntfType.
RK_ERR_VO_NULL_PTR	pstCallbackFunc — пустой указатель.
RK_ERR_VO_DEV_NOT_ENABLE	Устройство вывода, соответствующее VoDev, не включено.

【Примечания】

- Поддерживаются только интерфейсы с горячим подключением, например HDMI и EDP.
- Устройство отображения должно быть включено.
- u32Id — ID интерфейса одного типа внутри микросхемы. Например, если встроено два HDMI, допустимы 0 и 1.

5.1.10 RK_MPI_VO_UnRegCallbackFunc

【Описание】

Регистрирует callback горячего подключения устройства отображения.

【Синтаксис】

```
RK_S32 RK_MPI_VO_UnRegCallbackFunc(RK_U32 enIntfType, RK_U32 u32Id, RK_VO_CALLBACK_FUNC_S
*pstCallbackFunc)
```

【Параметры】

Имя параметра	Описание	Вход/выход
enIntfType	Тип интерфейса вывода.	Вход
u32Id	Номер устройства интерфейса этого типа, например HDMI0 или HDMI1.	Вход
pstCallbackFunc	Callback горячего подключения.	Вход

[Возвращаемое значение]

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый enIntfType.
RK_ERR_VO_NULL_PTR	pstCallbackFunc — пустой указатель.
RK_ERR_VO_DEV_NOT_ENABLE	Устройство вывода, соответствующее VoDev, не включено.
RK_ERR_VO_ILLEGAL_PARAM	pstCallbackFunc не зарегистрирован.

【Примечания】

- Поддерживаются только интерфейсы с горячим подключением, например HDMI и EDP.
- Устройство отображения должно быть включено.
- u32Id — ID интерфейса одного типа внутри микросхемы. Например, если встроено два HDMI, допустимы 0 и 1.

5.1.11 RK_MPI_VO_RegVsyncCallbackFunc

【Описание】

Регистрирует callback кадрового прерывания устройства отображения.

【Синтаксис】

```
RK_S32 RK_MPI_VO_RegVsyncCallbackFunc(VO_DEV VoDev, RK_VO_VSYNC_CALLBACK_FUNC_S *pstCallbackFunc)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoDev	Номер устройства интерфейса вывода.	Вход
pstCallbackFunc	Callback прерывания кадра.	Вход

[Возвращаемое значение]

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_NULL_PTR	pstCallbackFunc — пустой указатель.
RK_ERR_VO_DEV_NOT_ENABLE	Устройство вывода, соответствующее VoDev, не включено.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы VO.
RK_ERR_VO_ILLEGAL_PARAM	Недействительный callback.

Примечания

- Устройство отображения должно быть включено.

5.1.12 RK_MPI_VO_UnRegVsyncCallbackFunc

Описание

Отменяет регистрацию callback кадрового прерывания устройства отображения.

Синтаксис

```
RK_S32 RK_MPI_VO_UnRegCallbackFunc(VO_DEV VoDev, RK_VO_VSYNC_CALLBACK_FUNC_S *pstCallbackFunc)
```

Параметры

Имя параметра	Описание	Вход/выход
VoDev	Номер устройства интерфейса вывода.	Вход
pstCallbackFunc	Callback прерывания кадра.	Вход

[Возвращаемое значение]

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_NULL_PTR	pstCallbackFunc — пустой указатель.
RK_ERR_VO_DEV_NOT_ENABLE	Устройство вывода, соответствующее VoDev, не включено.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы VO.
RK_ERR_VO_ILLEGAL_PARAM	Недействительный callback.

Примечания

- Устройство отображения должно быть включено.

5.1.13 RK_MPI_VO_GetHdmiParam

Описание

Получает атрибуты HDMI интерфейса отображения.

Синтаксис

```
RK_S32 RK_MPI_VO_GetHdmiParam(RK_U32 enIntfType, RK_U32 u32Id, VO_HDMI_PARAM_S *pstHDMIParam)
```

【Параметры】

Имя параметра	Описание	Вход/выход
enIntfType	Тип интерфейса вывода.	Вход
u32Id	Номер устройства интерфейса этого типа, например HDMI0 или HDMI1.	Вход
pstHDMIParam	Указатель на структуру атрибутов HDMI.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый enIntfType.
RK_ERR_VO_NULL_PTR	pstHDMIParam — пустой указатель.
RK_ERR_VO_SYS_NOTREADY	Система не готова.

【Примечания】

- u32Id — ID интерфейса одного типа внутри микросхемы. Например, если встроено два HDMI, допустимы 0 и 1.

5.1.14 RK_MPI_VO_SetHdmiParam

【Описание】

Задаёт атрибуты HDMI интерфейса отображения.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetHdmiParam(RK_U32 enIntfType, RK_U32 u32Id, const VO_HDMI_PARAM_S
*pstHDMIParam)
```

【Параметры】

Имя параметра	Описание	Вход/выход
enIntfType	Тип интерфейса вывода.	Вход
u32Id	Номер устройства интерфейса этого типа, например HDMI0 или HDMI1.	Вход
pstHDMIParam	Указатель на структуру атрибутов HDMI.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый enIntfType.
RK_ERR_VO_NULL_PTR	pstHDMIParam — пустой указатель.
RK_ERR_VO_SYS_NOTREADY	Система не готова.

【Примечания】

- u32Id — ID интерфейса одного типа внутри микросхемы. Например, если встроено два HDMI, допустимы 0 и 1.

【Пример】

```
VO_HDMI_PARAM_S stHDMIParam;

RK_MPI_VO_GetHdmiParam(VO_INTF_HDMI, 0, &stHDMIParam);

stHDMIParam.enColorFmt = VO_HDMI_COLOR_FORMAT_RGB;
stHDMIParam.enQuantRange = VO_HDMI_QUANT_RANGE_FULL;
stHDMIParam.enHdmiMode = VO_HDMI_MODE_HDMI;
RK_MPI_VO_SetHdmiParam(VO_INTF_HDMI, 0, &stHDMIParam);
```

5.1.15 RK_MPI_VO_SyncDevs

【Описание】

Синхронизирует кадровые прерывания разных VoDev.

【Синтаксис】

```
RK_MPI_VO_SyncDevs(RK_U32 u32Vodevs)
```

【Параметры】

Имя параметра	Описание	Вход/выход
u32Vodevs	Каждый бит обозначает ID устройства вывода, которое необходимо синхронизировать.	Вход

[Возвращаемое значение]

Возвращаемое значение	Описание
0	Успешно
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_BUSY	Не удалось создать или включить устройство вывода.

【Примечания】

- Необходимо вызывать после RK_MPI_Enable для всех синхронизируемых VoDev.

【Пример】

```
....
RK_MPI_Enable(0)
....
RK_MPI_Enable(2);
....
// Синхронизация VoDev0 и VoDev2
u32TimeOut = 10;
u32Devs = BIT(0) | BIT(2);
while (timeout--) {
  u32Ret = RK_MPI_VO_SyncDevs(u32Devs);
  if (u32Ret == RK_ERR_VO_NOT_SUPPORT)
    return;
  else if (u32Ret == 0)
    break;
  else
    usleep(100000);
}
```

5.1.16 RK_MPI_VO_CloseFd

【Описание】

Закрывает все Fd, занятые модулем видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_CloseFd(RK_VOID)
```

【Параметры】

Нет

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.

5.2 Настройка Framebuffer

5.2.1 RK_MPI_VO_CreateGraphicsFrameBuffer

【Описание】

Создаёт Framebuffer графического слоя.

【Синтаксис】

```
RK_S32 RK_MPI_VO_CreateGraphicsFrameBuffer(int Width, int Hight, RK_U32 Formant, RK_VOID **fd)
```

【Параметры】

Имя параметра	Описание	Вход/выход
Width	Ширина графики.	Вход.
Hight	Высота графики.	Вход.
Formant	Тип формата пикселей.	Вход.
fd	Указатель типа void*.	Вход.

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Не удалось выделить buffer.
Не 0	Длина выделенного buffer.

【Пример】

```
    RK_U32  u32ImgeWidth;
    RK_U32  u32ImageHeight;
    RK_VOID *pMblk = RK_NULL;
    VO_FRAME_INFO_S stFrameInfo;
    RK_U32 u32BuffSize;

    u32ImgeWidth = 1920;
    u32ImageHeight = 1080;
    u32BuffSize = RK_MPI_VO_CreateGraphicsFrameBuffer(u32ImgeWidth, u32ImageHeight,
                  RK_FMT_YUV420SP, &pMblk);
    if (u32BuffSize == 0) {
```

```
        return RK_FAILURE;
    }
    /*do something to fill Graphic FrameBuffer */
    RK_MPI_VO_GetFrameInfo(*pMblk, &stFrameInfo);
    ...
    RK_MPI_VO_DestroyGraphicsFrameBuffer(pMblk);
```

【Связанные темы】

RK_MPI_VO_DestroyGraphicsFrameBuffer

5.2.2 RK_MPI_VO_DestroyGraphicsFrameBuffer

【Описание】

Уничтожает Framebuffer графического слоя.

【Синтаксис】

```
RK_S32 RK_MPI_VO_DestroyGraphicsFrameBuffer(RK_VOID* fd)
```

【Параметры】

Имя параметра	Описание	Вход/выход
fd	Указатель типа void.	Вход.

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка.

【Связанные темы】

RK_MPI_VO_CreateGraphicsFrameBuffer

5.2.3 RK_MPI_VO_GetGraphicsFrameBuffer

【Описание】

Получает виртуальный адрес созданного Framebuffer.

【Синтаксис】

```
RK_VOID * RK_MPI_VO_GetGraphicsFrameBuffer(RK_VOID *fd)
```

【Параметры】

Имя параметра	Описание	Вход/выход.
fd	Указатель типа void.	Вход.

【Возвращаемое значение】

Возвращаемое значение	Описание
NULL	Не удалось получить адрес.
Не ноль	Виртуальный адрес Frame Buffer.

5.2.4 RK_MPI_VO_SetGFxMode

【Описание】

Задаёт режим создания графического buffer.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetGFxMode(VO_GFX_MODE_E u32Mode)
```

【Параметры】

Имя параметра	Описание	Вход/выход
u32Mode	Режим создания графического buffer.	Вход.

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_ILLEGAL_PARAM	Ошибка параметра Mode.

【Примечания】

- Эту функцию необходимо вызывать до любых операций VO.

【Пример】

См. пример RK_MPI_VO_GetGfxFrameBuffer.

【Связанные темы】

RK_MPI_VO_GetGfxFrameBuffer

5.2.5 RK_MPI_VO_GetGfxFrameBuffer

【Описание】 Получает предварительно созданный buffer указанного графического канала. API создаёт один графический buffer и подходит для OSD, текста, рамок и других сценариев отрисовки, не требующих нескольких UI-buffer.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetGfxFrameBuffer(VO_LAYER VoLayer, VO_CHN VoChn, VO_FRAME_INFO_S *pstFrame)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер графического канала, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstFrame	Указатель VO_FRAME_INFO_S.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstFrame — пустой указатель.
RK_ERR_VO_NOT_PERMIT	Для VoLayer не задан режим предварительного создания.
RK_ERR_VO_NOT_SUPPORT	enPixelFormat в pstFrame не поддерживается.
RK_ERR_VO_BUSY	Для канала слоя VoChn buffer уже выделен.

Примечания

- Предварительно создавать графический buffer необходимо до операций VO; режим подходит для объединения Video- и UI-слоя.
- Если формат данных графического канала BGRA5551/RGBA5551, необходимо заполнить u32FgAlpha и u32BgAlpha.
- При использовании предварительно созданного графического buffer на видеослое UI-графический слой включать не требуется.
- Способ с одним UI-buffer через RK_MPI_VO_GetGfxFrameBuffer лучше снижает пропускную нагрузку VOP, чем сочетание Video- и UI-слоёв.

Пример

```
    VO_PUB_ATTR_S VoPubAttr;
    VO_VIDEO_LAYER_ATTR_S stLayerAttr;
    RK_U32 s32Ret;
    RK_U32 u32DispWidth = 1920;
    RK_U32 u32DispHeight = 1080;
    RK_U32 u32ImageWidth = 1920;
    RK_U32 u32ImageHeight = 1080;

    memset(&VoPubAttr, 0, sizeof(VO_PUB_ATTR_S));
    memset(&stLayerAttr, 0, sizeof(VO_VIDEO_LAYER_ATTR_S));

    stLayerAttr.enPixFormat                 = RK_FMT_YUV420SP;
    stLayerAttr.stDispRect.s32X             = 0;
    stLayerAttr.stDispRect.s32Y             = 0;
    stLayerAttr.stDispRect.u32Width         = u32DispWidth;
    stLayerAttr.stDispRect.u32Height        = u32DispHeight;
    stLayerAttr.stImageSize.u32Width        = u32ImageWidth;
    stLayerAttr.stImageSize.u32Height       = u32ImageHeight;

    VO_LAYER VoLayer = RK356X_VOP_LAYER_CLUSTER0;
    VO_DEV VoDev = RK356X_VO_DEV_HD0;
    VoPubAttr.enIntfType = VO_INTF_HDMI;
    VoPubAttr.enIntfSync = VO_OUTPUT_1080P60;

    VO_FRAME_INFO_S stVFrame;
    RK_MPI_VO_SetGFxMode(VO_MODE_GFX_PRE_CREATED);
    /* Get 1st GFX buffer */
    stVFrame.enPixelFormat = RK_FMT_BGRA5551;
    stVFrame.u32FgAlpha = 128;
    stVFrame.u32BgAlpha = 0;
    stVFrame.u32Width = 1920;
    stVFrame.u32Height = 1080;
    RK_MPI_VO_GetGfxFrameBuffer(VoLayer, 127, &stVFrame);
    /* Draw 1st GFX */
    memset(stVFrame.pData, 0xff, stVFrame.u32Size);

    /* Get 2nd GFX buffer */
    stVFrame.u32FgAlpha = 128;
    stVFrame.u32BgAlpha = 0;
    RK_MPI_VO_GetGfxFrameBuffer(VoLayer, 126, &stVFrame);

    /* Draw 2nd GFX */
```

```
    memset(stVFrame.pData, 0, stVFrame.u32Size);

    s32Ret = RK_MPI_VO_SetPubAttr(VoDev, &VoPubAttr);
    if (s32Ret != RK_SUCCESS) {
        return RK_FAILURE;
    }
    s32Ret = RK_MPI_VO_Enable(VoDev);
    if (s32Ret != RK_SUCCESS) {
        return RK_FAILURE;
    }
    s32Ret = RK_MPI_VO_SetLayerAttr(VoLayer, &stLayerAttr);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;
    s32Ret = RK_MPI_VO_EnableLayer(VoLayer);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;
    s32Ret = RK_MPI_VO_DisableLayer(VoLayer);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;
```

Связанные темы

RK_MPI_VO_ReleaseGfxFrameBuffer

5.2.6 RK_MPI_VO_ReleaseGfxFrameBuffer

Описание

Освобождает предварительно созданный buffer указанного графического канала.

Синтаксис

```
RK_S32 RK_MPI_VO_ReleaseGfxFrameBuffer(VO_LAYER VoLayer, VO_CHN VoChn)
```

Параметры

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер графического канала, диапазон [0, VO_MAX_CHN_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoDev.
RK_ERR_VO_NULL_PTR	pstFrame — пустой указатель.
RK_ERR_VO_NOT_PERMIT	Для VoLayer не задан режим предварительного создания.

5.3 Операции со слоями

Каждый VoDev поддерживает четыре типа слоёв: видео, графика, курсор и виртуальный. Видео- и графические слои поддерживают несколько каналов; каждый слой можно гибко привязать к заданному устройству вывода.

Привязки RK3568 по умолчанию приведены ниже.

Устройство вывода	Видеослой	Графический слой	Слой курсора	Виртуальный слой
Dev0	Cluster0	Esmart0	Smart0	Нет
Dev1	Custer1	Esmart1	Smart1	Нет
Dev2	Нет	Нет	Нет	Нет

Привязки RK3588 по умолчанию приведены ниже.

Устройство вывода	Видеослой	Графический слой	Слой курсора	Виртуальный слой
Dev0	Cluster0	Esmart0	Esmart3	Нет
Dev1	Custer1	Esmart1	Нет	Нет
Dev2	Custer2	Esmart2	Нет	Нет
Dev3	Custer3	Нет	Нет	Нет

5.3.1 RK_MPI_VO_ClearLayersBinding

【Описание】

Удаляет все привязки слоёв.

【Синтаксис】

```
RK_S32 RK_MPI_VO_ClearLayersBinding(RK_VOID)
```

【Параметры】

Нет

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_NOT_SUPPORT	Устройство не поддерживается.

【Примечания】

При вызове все слои и устройства отображения должны быть выключены.

5.3.2 RK_MPI_VO_BindLayer

【Описание】

Задаёт связь слоя и устройства вывода, привязывая слой к указанному устройству VOP.

【Синтаксис】

```
RK_S32 RK_MPI_VO_BindLayer(VO_LAYER VoLayer, VO_DEV VoDev, VO_LAYER_MODE_E Mode)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер привязываемого слоя.	Вход
VoDev	Номер устройства VOP для привязки.	Вход
Mode	Тип слоя.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы VO.
RK_ERR_VO_DEV_HAS_BINDED	VoLayer уже привязан к другому устройству вывода.
RK_ERR_VO_ILLEGAL_PARAM	Ошибка параметра Mode.

【Примечания】

- Если слой уже привязан к устройству вывода, перед новой привязкой его необходимо отвязать от прежнего устройства.
- Один VoDev поддерживает только один слой курсора, один видеослой и один графический слой.
- Привязываемое устройство отображения должно быть выключено.
- Тип привязываемого виртуального слоя должен быть VO_LAYER_MODE_VIRTUAL.
- К одному VoDev можно привязать несколько виртуальных слоёв.

【Пример】

См. RK_MPI_VO_EnableLayer.

【Связанные темы】

RK_MPI_VO_UnBindLayer

5.3.3 RK_MPI_VO_UnBindLayer

【Описание】

Отвязывает слой от устройства вывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_UnBindLayer(VO_LAYER VoLayer, VO_DEV VoDev)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя для отвязки.	Вход
VoDev	Номер устройства VOP для отвязки.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_DEVID	Недопустимый VoDev.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы VO.

【Примечания】

- Привязанное устройство вывода VoDev должно быть выключено.

【Пример】

См. RK_MPI_VO_EnableLayer.

Связанные темы

RK_MPI_VO_BindLayer

5.3.4 RK_MPI_VO_SetLayerAttr

Описание

Задаёт атрибуты слоя.

Синтаксис

```
RK_S32 RK_MPI_VO_SetLayerAttr(VO_LAYER VoLayer,const VO_VIDEO_LAYER_ATTR_S *pstLayerVideoAttr)
```

Параметры

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя.	Вход
pstLayerVideoAttr	Указатель на структуру атрибутов видеослоя.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_NULL_PTR	pstLayerVideoAttr — пустой указатель.
RK_ERR_VO_ILLEGAL_PARAM	Формат цвета в pstLayerVideoAttr не поддерживается.

Примечания

- Частота отображения в атрибутах слоя не должна превышать частоту обновления привязанного устройства.
- Для производительности рекомендуется задавать частоту слоя не выше 30 кадров/с.
- Если видеослой относится к Cluster, рекомендуется COMPRESS_AFBC_16x16 для enCompressMode, чтобы снизить системную пропускную нагрузку.
- Атрибут bBypassFrame структуры VO_VIDEO_LAYER_ATTR_S действует в режиме объединения GPU/RGA, только если включён один видеоканал.

Пример

См. пример RK_MPI_VO_EnableLayer.

5.3.5 RK_MPI_VO_GetLayerAttr

Описание

Получает атрибуты слоя.

Синтаксис

```
RK_S32 RK_MPI_VO_GetLayerAttr(VO_LAYER VoLayer,VO_VIDEO_LAYER_ATTR_S *pstLayerAttr)
```

Параметры

Имя параметра	Описание	Вход/выход
VoLaye	Номер слоя.	Вход
pstLayerAttr	Указатель на структуру атрибутов видеослоя.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_NULL_PTR	pstLayerAttr — пустой указатель.

5.3.6 RK_MPI_VO_SetLayerSpliceMode

【Описание】

Задаёт способ объединения слоя.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetLayerSpliceMode(VO_LAYER VoLayer, VO_SPLICE_MODE_E enSpliceMode);
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя.	Вход.
enSpliceMode	Способ объединения слоя.	Вход.

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый enSpliceMode.

【Примечания】

- По умолчанию слой объединяется через GPU.
- Способ объединения необходимо задать до включения слоя RK_MPI_VO_Enable.
- В режиме объединения RGA слой должен поддерживать формат без AFBC; использовать Cluster-слой нельзя.

5.3.7 RK_MPI_VO_GetLayerSpliceMode

【Описание】

Получает способ объединения слоя.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetLayerSpliceMode(VO_LAYER VoLayer, VO_SPLICE_MODE_E *enSpliceMode)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя.	Вход.
enSpliceMode	Способ объединения слоя.	Выход.

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_NULL_PTR	enSpliceMode — пустой указатель.

5.3.8 RK_MPI_VO_EnableLayer

【Описание】

Включает слой.

【Синтаксис】

```
RK_S32 RK_MPI_VO_EnableLayer(VO_LAYER VoLayer)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_BUSY	Не удалось создать службы VO.
RK_ERR_VO_LAYER_NOT_BINDED	VoLayer не привязан к VoDev.
RK_ERR_VO_DEV_NOT_ENABLE	VoDev, привязанный к VoLayer, не включён.
RK_ERR_VO_NO_MEM	Недостаточно системной памяти.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы слоя.

【Примечания】

- Перед включением слоя необходимо вызвать RK_MPI_VO_BindLayer.
- Устройство вывода, привязанное к слою, необходимо сначала включить RK_MPI_VO_Enable.

【Пример】

```
    VO_PUB_ATTR_S VoPubAttr;
    VO_VIDEO_LAYER_ATTR_S stLayerAttr;
    RK_U32 s32Ret;
    RK_U32 VoDev, VoLayer;

    memset(&VoPubAttr, 0, sizeof(VO_PUB_ATTR_S));
    memset(&stLayerAttr, 0, sizeof(VO_VIDEO_LAYER_ATTR_S));

    VoPubAttr.enIntfType = VO_INTF_HDMI;
    VoPubAttr.enIntfSync = VO_OUTPUT_1080P50;
    VoDev = RK356X_VO_DEV_HD0;
    VoLayer = RK356X_VOP_LAYER_CLUSTER0;

    RK_MPI_VO_BindLayer(VoLayer, VoDev, VO_LAYER_MODE_VIDEO);
    s32Ret = RK_MPI_VO_SetPubAttr(VoDev, &VoPubAttr);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
```

```
    }
    s32Ret = RK_MPI_VO_Enable(VoDev);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }
    stLayerAttr.stDispRect.s32X = 0;
    stLayerAttr.stDispRect.s32Y = 0;
    stLayerAttr.stDispRect.u32Width = 1920;
    stLayerAttr.stDispRect.u32Height = 1080;
    stLayerAttr.stImageSize.u32Width = 1920;
    stLayerAttr.stImageSize.u32Height = 1080;
    stLayerAttr.u32DispFrmRt = 25;
    stLayerAttr.enPixFormat = RK_FMT_YUV420SP;
    stLayerAttr.enCompressMode = COMPRESS_AFBC_16x16;
    s32Ret = RK_MPI_VO_SetLayerAttr(VoLayer, &stLayerAttr);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }
    s32Ret = RK_MPI_VO_EnableLayer(VoLayer);
    if (s32Ret != RK_SUCCESS) {
        return s32Ret;
    }
    s32Ret = RK_MPI_VO_DisableLayer(VoLayer);
    if (s32Ret != RK_SUCCESS)
        return s32Ret;
```

5.3.9 RK_MPI_VO_DisableLayer

【Описание】

Выключает слой.

【Синтаксис】

```
RK_S32 RK_MPI_VO_DisableLayer(VO_LAYER VoLayer)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_WBC_NOT_DISABLE	Обратная запись слоя не выключена.

5.3.10 RK_MPI_VO_SetLayerDispBufLen

【Описание】

Задаёт длину буфера отображения слоя.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetLayerDispBufLen(VO_LAYER VoLayer, RK_U32 u32BufLen)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя.	Вход
u32BufLen	Длина буфера отображения, [3, 15].	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_DEV_HAS_ENABLED	Слой уже включён.
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый u32BufLen.

5.3.11 RK_MPI_VO_GetLayerDispBufLen

【Описание】

Получает длину буфера отображения слоя.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetLayerDispBufLen(VO_LAYER VoLayer, RK_U32 *pu32BufLen)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoDev	Номер слоя.	Вход
pu32BufLen	Указатель u32.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_NULL_PTR	pu32BufLen — пустой указатель.

5.3.12 RK_MPI_VO_GetLayerFrame

【Описание】

Получает выходной кадр изображения слоя.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetLayerFrame(VO_LAYER VoLayer, VIDEO_FRAME_INFO_S *pstVFrame, RK_S32
s32MilliSec)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя.	Вход
pstVFrame	Указатель на структуру данных полученного изображения экрана.	Выход
s32MilliSec	Параметр тайм-аута, сейчас по умолчанию 0.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_NULL_PTR	pstVFrame — пустой указатель.
RK_ERR_VO_LAYER_NOT_ENABLE	Слой не включён.

【Примечания】

Поддерживается получение данных только видеослоя и виртуального слоя.

5.3.13 RK_MPI_VO_ReleaseLayerFrame

【Описание】

Освобождает выходной кадр изображения слоя.

【Синтаксис】

```
RK_S32 RK_MPI_VO_ReleaseLayerFrame(VO_LAYER VoLayer, VIDEO_FRAME_INFO_S *pstVFrame)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя.	Вход
pstVFrame	Указатель на структуру освобождаемого изображения экрана.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_NULL_PTR	pstVFrame — пустой указатель.

5.3.14 RK_MPI_VO_SendLayerFrame

【Описание】

Отправляет изображение в указанный выходной канал графического слоя для отображения.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SendLayerFrame(VO_LAYER VoLayer, VIDEO_FRAME_INFO_S *pstVFrame)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер графического слоя.	Вход
pstVFrame	Указатель на структуру отправляемого изображения экрана.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_NULL_PTR	pstVFrame — пустой указатель.
RK_ERR_VO_LAYER_NOT_ENABLE	Слой не включён.
RK_ERR_VO_ILLEGAL_PARAM	pstVFrame не соответствует требованиям.

【Примечания】

- Функция напрямую передаёт изображение слою для отображения; формат данных определяется возможностями слоя.

5.3.15 RK_MPI_VO_SetCursorPostion

【Описание】

Обновляет координаты слоя курсора.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetCursorPostion(VO_LAYER VoLayer, const RK_U32 x, const RK_U32 y)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя курсора.	Вход
x	Координата по оси X.	Вход
y	Координата по оси Y.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer либо слой не является слоем курсора.
RK_ERR_VO_NULL_PTR	Данные слоя пусты.
RK_ERR_VO_LAYER_NOT_ENABLE	Слой не включён.

【Примечания】

- Сначала необходимо отправить первый кадр через RK_MPI_VO_SendLayerFrame; начиная со второго кадра координаты обновляются этой функцией.

5.3.16 RK_MPI_VO_SetCursorLogicalRect

【Описание】

Задаёт область перемещения курсора.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetCursorLogicalRect(VO_LAYER VoLayer, const RK_U32 maxWidth, const RK_U32
maxHeight)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер слоя курсора.	Вход
maxWidth	Правая граница курсора.	Вход
maxHeight	Нижняя граница курсора.	Вход

【Примечания】

- Используется, когда диапазон перемещения курсора приложения не совпадает с реальной областью отображения. Например, при дисплее 4K, если UI и курсор работают в области 1080P, вызовите этот интерфейс. Если maxWidth/maxHeight равны 0, восстанавливается масштаб 1:1.

【Пример】

```
    VO_PUB_ATTR_S            stVoPubAttr;
    VO_VIDEO_LAYER_ATTR_S    stLayerAttr;
    VO_CHN_ATTR_S            stChnAttr;
    RK_U32                   x=100,y=100;
    RK_U32                   u32BuffSize;
    RK_S32                   s32Ret = RK_SUCCESS;
    RK_S32                   VoDev = RK356X_VO_DEV_HD1;
    VO_LAYER                 VoLayer = RK356X_VOP_LAYER_SMART_0;
    VIDEO_FRAME_INFO_S       stVFrame;
    void* pmlk;

    RK_MPI_VO_BindLayer(VoLayer, VoDev, VO_LAYER_MODE_CURSOR);

    /* Enable VO Device , set 4k resolution*/
    stVoPubAttr.enIntfType = VO_INTF_HDMI;
    stVoPubAttr.enIntfSync = VO_OUTPUT_3840x2160_30;
    RK_MPI_VO_SetPubAttr(VoDev, &stVoPubAttr);

    u32BuffSize = RK_MPI_VO_CreateGraphicsFrameBuffer(64, 64, RK_FMT_BGRA8888, &pMblk);

    memset(&stLayerAttr, 0, sizeof(VO_VIDEO_LAYER_ATTR_S));
    stLayerAttr.enPixFormat = RK_FMT_BGRA8888;
    stLayerAttr.stDispRect.s32X = 0;
    stLayerAttr.stDispRect.s32Y = 0;
    stLayerAttr.stDispRect.u32Width = 64;
    stLayerAttr.stDispRect.u32Height = 64;
    stLayerAttr.stImageSize.u32Width = 64;
    stLayerAttr.stImageSize.u32Height = 64;
    s32Ret = RK_MPI_VO_SetLayerAttr(VoLayer, &stLayerAttr);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;

    /*set cursor range*/
    RK_MPI_VO_SetCursorLogicalRect(VoLyaer, 1920, 1080);

    s32Ret = RK_MPI_VO_EnableLayer(VoLayer);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;
```

```
    stVFrame.stVFrame.pMbBlk = pmlk;

// Настройка buffer курсора
    RK_MPI_VO_SendLayerFrame(VoLayer, &stVFrame);
// Обновление координат курсора
    for (int i=0;i<10;i++) {
        RK_MPI_VO_SetCursorPostion(VoLayer, x, y);
        x+=10;
        y+=10;
    }
```

5.4 Операции с каналами

5.4.1 RK_MPI_VO_EnableChn

【Описание】

Включает указанный канал видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_EnableChn (VO_LAYER VoLayer, VO_CHN VoChn)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NO_MEM	Недостаточно системной памяти.

【Пример】

```
    VO_VIDEO_LAYER_ATTR_S    stLayerAttr;
    VO_CHN_ATTR_S            stChnAttr;
    RK_U32                   i;
    RK_S32                   s32Ret = RK_SUCCESS;
    RK_U32                   u32Layers = 2;
    VO_LAYER                 VoLayer = 4;

    memset(&stLayerAttr, 0, sizeof(VO_VIDEO_LAYER_ATTR_S));
    stLayerAttr.enPixFormat = RK_FMT_BGRA8888;
    stLayerAttr.stDispRect.s32X = 0;
    stLayerAttr.stDispRect.s32Y = 0;
    stLayerAttr.stImageSize.u32Width = 1920;
    stLayerAttr.stImageSize.u32Height = 1080;
    stLayerAttr.u32DispFrmRt = 25;
    s32Ret = RK_MPI_VO_SetLayerAttr(VoLayer, &stLayerAttr);
    if (s32Ret != RK_SUCCESS)
```

```
        return RK_FAILURE;

    s32Ret = RK_MPI_VO_EnableLayer(VoLayer);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;

    s32Ret = RK_MPI_VO_GetLayerAttr(VoLayer, &stLayerAttr);
    if (s32Ret != RK_SUCCESS) {
        RK_LOGE("[%s] Get Layer Attr failed\n", __func__);
        return RK_FAILURE;
    }

    for (i = 0; i < u32Layers; i++) {
        stChnAttr.stRect.s32X = 0;
        stChnAttr.stRect.s32Y = 0;
        stChnAttr.stRect.u32Width = stLayerAttr.stImageSize.u32Width;
        stChnAttr.stRect.u32Height = stLayerAttr.stImageSize.u32Height;
        stChnAttr.u32Priority = i;
        if (i == 0) {
            stChnAttr.u32FgAlpha = 128;
            stChnAttr.u32BgAlpha = 0;
        } else {
            stChnAttr.u32FgAlpha = 0;
            stChnAttr.u32BgAlpha = 128;
        }

        s32Ret = RK_MPI_VO_SetChnAttr(VoLayer, i, &stChnAttr);
        if (s32Ret != RK_SUCCESS) {
            RK_LOGE("[%s] set chn Attr failed\n", __func__);
            return RK_FAILURE;
        }

        s32Ret = RK_MPI_VO_EnableChn(VoLayer, i);
        if (s32Ret != RK_SUCCESS) {
            RK_LOGE("[%s] Enalbe chn failed\n", __func__);
            return RK_FAILURE;
        }
    }
```

【Связанные темы】

RK_MPI_DisableChn

5.4.2 RK_MPI_VO_DisableChn

【Описание】

Выключает указанный канал видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_DisableChn(VO_LAYER VoLayer, VO_CHN VoChn)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.

【Пример】

```
    RK_U32      i, u32Windows;
    VO_LAYER    VoLayer = 0;
    RK_S32      s32Ret = RK_SUCCESS;
    RK_S32      u32Windows = 4;

    for (i = 0; i < u32Windows; i++) {
        RK_MPI_VO_ClearChnBuffer(VoLayer, i, RK_TRUE);
        s32Ret = RK_MPI_VO_DisableChn(VoLayer, i);
        if (s32Ret != RK_SUCCESS)
            return RK_FAILURE;
    }
```

【Связанные темы】

RK_MPI_VO_EnableChn

5.4.3 RK_MPI_VO_SetChnAttr

【Описание】

Задаёт атрибуты указанного канала видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetChnAttr(VO_LAYER VoLayer, VO_CHN VoChn, const VO_CHN_ATTR_S *pstChnAttr)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstAttr	Указатель на атрибуты видеоканала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstChnAttr — пустой указатель.
RK_ERR_VO_ILLEGAL_PARAM	Параметры pstChnAttr недопустимы.

【Пример】

См. пример RK_MPI_VO_EnableChn.

【Связанные темы】

RK_MPI_VO_GetChnAttr

5.4.4 RK_MPI_VO_GetChnAttr

【Описание】

Получает атрибуты указанного канала видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetChnAttr(VO_LAYER VoLayer, VO_CHN VoChn, VO_CHN_ATTR_S *pstAttr)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstAttr	Указатель на атрибуты видеоканала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstAttr — пустой указатель.

【Пример】

См. пример RK_MPI_VO_EnableChn.

【Связанные темы】

RK_MPI_VO_SetChnAttr

5.4.5 RK_MPI_VO_SetChnParam

【Описание】

Задаёт параметры указанного канала видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetChnParam(VO_LAYER VoLayer, VO_CHN VoChn, const VO_CHN_PARAM_S *pstChnParam)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstChnParam	Указатель на параметры видеоканала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstChnParam — пустой указатель.

【Пример】

```
    VO_CHN_PARAM_S           stChnParam;
    VO_VIDEO_LAYER_ATTR_S    stLayerAttr;

    VO_LAYER    VoLayer = RK356X_VOP_LAYER_CLUSTER_0;
    VO_CHN      VoChn =  0;
    RK_S32      s32Ret = RK_SUCCESS;;

    s32Ret = RK_MPI_VO_GetLayerAttr(VoLayer, &stLayerAttr);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;

    s32Ret = RK_MPI_VO_GetChnParam(VoLayer, VoChn, &stChnParam);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;

    stChnParam.stAspectRatio.enMode = ASPECT_RATIO_MANUAL;
    u32Width = stLayerAttr.stDispRect.u32Width;
    u32Height = stLayerAttr.stDispRect.u32Height;
    stChnParam.stAspectRatio.stVideoRect.s32X = 0;
    stChnParam.stAspectRatio.stVideoRect.s32Y = 0;
    stChnParam.stAspectRatio.stVideoRect.u32Width = u32Width;
    stChnParam.stAspectRatio.stVideoRect.u32Height = u32Height;
    s32Ret = RK_MPI_VO_SetChnParam(VoLayer, VoChn, &stChnParam);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;
```

5.4.6 RK_MPI_VO_GetChnParam

【Описание】

Получает параметры указанного канала видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetChnParam(VO_LAYER VoLayer, VO_CHN VoChn, VO_CHN_ATTR_S *pstChnParam)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstChnParam	Указатель на параметры видеоканала.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstChnParam — пустой указатель.

【Пример】

См. пример RK_MPI_VO_SetChnParam.

【Связанные темы】

RK_MPI_VO_SetChnParam

5.4.7 RK_MPI_SetChnDispPos

【Описание】

Задаёт положение отображения указанного канала.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetChnDispPos(VO_LAYER VoLayer, VO_CHN VoChn, const POINT_S *pstDispPos)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstDispPos	Положение отображения канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstDispPos — пустой указатель.

5.4.8 RK_MPI_VO_GetChnDispPos

【Описание】

Получает положение отображения указанного канала.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetChnDispPos(VO_LAYER VoLayer, VO_CHN VoChn, POINT_S *pstDispPos)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstDispPos	Указатель на атрибут положения отображения канала.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstDispPos — пустой указатель.

5.4.9 RK_MPI_VO_GetChnFrame

【Описание】

Получает кадр канала.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetChnFrame(VO_LAYER VoLayer, VO_CHN VoChn, VIDEO_FRAME_INFO_S *pstFrame, RK_S32 s32MilliSec)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstVFrame	Указатель на данные видео.	Вход
s32MilliSec	Тайм-аут в миллисекундах.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstFrame — пустой указатель.

5.4.10 RK_MPI_VO_ReleaseChnFrame

【Описание】

Освобождает данные изображения выходного канала.

【Синтаксис】

```
RK_S32 RK_MPI_VO_ReleaseChnFrame(VO_LAYER VoLayer, VO_CHN VoChn, const VIDEO_FRAME_INFO_S *pstFrame)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstVFrame	Указатель на структуру освобождаемого изображения выходного канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstFrame — пустой указатель.

5.4.11 RK_MPI_VO_SendFrame

【Описание】

Отправляет видеоизображение в указанный выходной канал для отображения.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SendFrame(VO_LAYER VoLayer, VO_CHN VoChn, VIDEO_FRAME_INFO_S *pstVFrame, RK_S32 s32MilliSec)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstVFrame	Указатель на данные видео.	Вход
s32MilliSec	Тайм-аут в миллисекундах.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstFrame — пустой указатель.
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый pstFrame.

【Пример】

```
    VIDEO_FRAME_INFO_S    *pstVFrame;
    RK_VOID               *pMblk;
    VO_LAYER              VoVideoLayer;
    VO_CHN                VoChn;
    RK_S32                ret = RT_OK;
```

```
    pstVFrame = (VIDEO_FRAME_INFO_S *)(malloc(sizeof(VIDEO_FRAME_INFO_S)));
    VoVideoLayer = RK356X_VOP_LAYER_CLUSTER_0;
    VoChn = 0;
    /*fill pMbBlk*/
    pstVFrame->stVFrame.pMbBlk = pMblk;
    do {
        ret = RK_MPI_VO_SendFrame(VoVideoLayer, VoChn, pstVFrame, -1);
        if (ret == RT_OK) {
            break;
        } else {
            RK_MPI_VO_DestroyGraphicsFrameBuffer(pMblk);
            free(pstVFrame);
            break;
        }
    } while (1);
```

5.4.12 RK_MPI_VO_SetChnFrameRate

【Описание】

Задаёт частоту отображения указанного канала.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetChnFrameRate(VO_LAYER VoLayer, VO_CHN VoChn, RK_S32 s32ChnFrmRate)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
s32VoFramerate	Частота отображения видеоканала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.

【Пример】

```
    VO_LAYER    VoLayer = RK356X_VOP_LAYER_CLUSTER_0;
    VO_CHN      VoChn = 0;
    RK_S32      s32ChnFrmRate = 30;

    s32Ret = RK_MPI_VO_SetChnFrameRate(VoLayer, VoChn, s32ChnFrmRate);
    if (s32Ret != RK_VO_OK) {
        printf("Set channel %d frame rate failed with errno %#x!\n", VoChn,s32Ret);
        return RK_FAILURE;
    }
```

【Связанные темы】

RK_MPI_VO_GetChnFrameRate

5.4.13 RK_MPI_VO_GetChnFrameRate

【Описание】

Получает частоту отображения указанного канала.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetChnFrameRate(VO_LAYER VoLayer, VO_CHN VoChn, RK_S32 *ps32ChnFrmRate)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
ps32VoFramerate	Частота отображения видеоканала.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	ps32ChnFrmRate — пустой указатель.

【Связанные темы】

RK_MPI_VO_SetChnFrameRate

5.4.14 RK_MPI_VO_PauseChn

【Описание】

Приостанавливает указанный канал видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_PauseChn (VO_LAYER VoLayer, VO_CHN VoChn)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.

【Пример】

```
    RK_S32 i;
    VO_LAYER VoLayer = 0;
    RK_U32 u32Screen0Chn = 16;
    while (1) {
        for (i = 0; i < u32Screen0Chn; i++) {
            RK_MPI_VO_PauseChn(VoLayer, i);
        }
        usleep(1000llu * 2000);
        for (i = 0; i < u32Screen0Chn; i++) {
            RK_MPI_VO_ResumeChn(VoLayer, i);
        }
    }
```

【Связанные темы】

RK_MPI_VO_ResumeChn

5.4.15 RK_MPI_VO_ResumeChn

【Описание】

Возобновляет указанный канал видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_ResumeChn(VO_LAYER VoLayer, VO_CHN VoChn)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.

【Пример】

См. пример RK_MPI_VO_PauseChn.

5.4.16 RK_MPI_VO_StepChn

【Описание】

Воспроизводит один кадр указанного канала видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_StepChn(VO_LAYER VoLayer, VO_CHN VoChn)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.

【Пример】

```
    RK_S32 i;
    VO_LAYER VoLayer = 0;
    RK_U32 u32ScreenChn = 16;
    RK_S32 step_frame_num = 50;

    while (1) {
        for (i = 0; i < ctx->u32ScreenChn; i++) {
            for (RK_S32 j = 0; j < step_frame_num; j++) {
                RK_MPI_VO_StepChn(VoLayer, i);
            }
        }
        usleep(1000llu * 2000);
        for (i = 0; i < ctx->u32Screen0Chn; i++) {
            RK_MPI_VO_ResumeChn(VoLayer, i);
        }
    }
```

【Связанные темы】

RK_MPI_VO_ResumeChn

5.4.17 RK_MPI_VO_RefreshChn

【Описание】

Обновляет указанный канал видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_RefreshChn(VO_LAYER VoLayer, VO_CHN VoChn)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.

【Примечания】

- Функция сейчас не поддерживается.

5.4.18 RK_MPI_VO_ShowChn

【Описание】

Показывает указанный канал.

【Синтаксис】

```
RK_S32 RK_MPI_VO_ShowChn (VO_LAYER VoLayer, VO_CHN VoChn)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.

【Пример】

См. пример RK_MPI_VO_HideChn.

【Связанные темы】

RK_MPI_VO_HideChn

5.4.19 RK_MPI_VO_HideChn

【Описание】

Скрывает указанный канал.

【Синтаксис】

```
RK_S32 RK_MPI_VO_HideChn(VO_LAYER VoLayer, VO_CHN VoChn)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.

【Пример】

```
    /* hide,show Chn 4 */
    VO_LAYER VoLayer;
    VoLayer = 0;
    RK_MPI_VO_HideChn(VoLayer, 4);
    usleep(1000llu * 5000);
    RK_MPI_VO_ShowChn(VoLayer, 4);
```

【Связанные темы】

RK_MPI_VO_ShowChn

5.4.20 RK_MPI_VO_GetChnPts

【Описание】

Получает временную метку изображения, отображаемого указанным каналом.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetChnPts(VO_LAYER VoLayer, VO_CHN VoChn,RK_U64 *pu64ChnPts)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pu64VoChnPts	Указатель на временную метку канала.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pu64ChnPts — пустой указатель.
RK_ERR_VO_LAYER_NOT_ENABLE	VoLayer не включён.
RK_ERR_VO_CHN_NOT_ENABLE	VoChn не включён.

5.4.21 RK_MPI_VO_QueryChnStat

【Описание】

Запрашивает состояние канала видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_QueryChnStat(VO_LAYER VoLayer, VO_CHN VoChn, VO_QUERY_STATUS_S *pstStatus)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstStatus	Указатель на структуру состояния канала.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstStatus — пустой указатель.
RK_ERR_VO_LAYER_NOT_ENABLE	VoLayer не включён.
RK_ERR_VO_CHN_NOT_ENABLE	VoChn не включён.

5.4.22 RK_MPI_VO_ClearChnBuffer

【Описание】

Очищает данные буфера указанного выходного канала.

【Синтаксис】

```
RK_S32 RK_MPI_VO_ClearChnBuffer(VO_LAYER VoLayer, VO_CHN VoChn,RK_BOOL bClrAll)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
bClrAll	Очищать ли все данные в buffer канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.

【Пример】

См. пример RK_MPI_VO_DisableChn.

5.4.23 RK_MPI_VO_SetChnBorder

【Описание】

Задаёт атрибуты рамки канала.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetChnBorder(VO_LAYER VoLayer, VO_CHN VoChn,const VO_BORDER_S *pstBorder)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstBorder	Указатель на атрибуты рамки канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstBorder — пустой указатель.

【Пример】

```
    VO_BORDER_S    border;
    RK_U32         s32Ret;
    VO_LAYER       VoLayer = RK356X_VOP_LAYER_ESMART_0;
    VO_CHN         VoChn = 0;

    memset(&border, 0, sizeof(VO_BORDER_S));
    /* set border */
    border.bBorderEn = RK_TRUE;
    /* Navy blue #000080 */
    /* Midnight Blue #191970 */
    border.stBorder.u32Color = 0x191970;
    border.stBorder.u32LeftWidth = 2;
    border.stBorder.u32RightWidth = 2;
    border.stBorder.u32TopWidth = 2;
    border.stBorder.u32BottomWidth = 2;
    s32Ret = RK_MPI_VO_SetChnBorder(VoLayer, VoChn, &border);
    if (s32Ret != RK_SUCCESS) {
        printf("Set channel %d Border failed with errno %#x!\n", VoChn,s32Ret);
    }
```

5.4.24 RK_MPI_VO_GetChnBorder

【Описание】

Получает атрибуты рамки канала.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetChnBorder(VO_LAYER VoLayer, VO_CHN VoChn, VO_BORDER_S *pstBorder)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pstBorder	Указатель на атрибуты рамки канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pstBorder — пустой указатель.

【Связанные темы】

RK_MPI_VO_SetChnBorder

5.4.25 RK_MPI_VO_SetChnRecvThreshold

【Описание】

Задаёт порог отображения канала. Основной сценарий — каскад PCIV: при передаче изображений с ведомой микросхемы на ведущую для предварительного просмотра интервалы поступления могут быть неравномерными. Число ожидающих отображения кадров может превысить порог по умолчанию, вызывая потерю кадров и рывки; тогда порог следует увеличить. Для стабильной низкой задержки порог следует уменьшить.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetChnRecvThreshold(VO_LAYER VoLayer, VO_CHN VoChn, RK_U32 u32Threshold)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
u32Threshold	Порог отображения канала видеовывода.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_ILLEGAL_PARAM	Недопустимый u32Threshold.

【Примечание】

- Диапазон порога [1, 4], по умолчанию 3. Это означает три кадра, ожидающих отображения: очередь канала принимает до двух кадров, плюс один кадр отображается или готовится к отображению.
- Чем меньше порог, тем ниже задержка предварительного просмотра, и наоборот. В обычном случае увеличивать порог не рекомендуется.
- Порог можно изменять динамически. Для возврата к значению по умолчанию пользователь должен вызвать этот интерфейс и задать его явно.

【Связанные темы】

RK_MPI_VO_GetChnRecvThreshold

5.4.26 RK_MPI_VO_GetChnRecvThreshold

【Описание】

Получает порог отображения канала видеовывода.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetChnRecvThreshold(VO_LAYER VoLayer, VO_CHN VoChn, RK_U32 *pu32Threshold)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoLayer	Номер видеослоя вывода.	Вход
VoChn	Номер канала видеовывода, диапазон [0, VO_MAX_CHN_NUM).	Вход
pu32Threshold	Порог отображения канала видеовывода.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_LAYERID	Недопустимый VoLayer.
RK_ERR_VO_INVALID_CHNID	Недопустимый VoChn.
RK_ERR_VO_NULL_PTR	pu32Threshold — пустой указатель.

【Связанные темы】

RK_MPI_VO_SetChnRecvThreshold

5.5 Операции обратной записи WBC

5.5.1 RK_MPI_VO_SetWbcSource

【Описание】

Задаёт источник данных тракта обратной записи.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetWbcSource(VO_WBC VoWbc, const VO_WBC_SOURCE_S *pstWbcSource)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoWbc	Номер тракта обратной записи.	Вход
pstWbcSource	Структура источника обратной записи видео.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_WBCID	Недопустимый VoWbc.
RK_ERR_VO_NULL_PTR	pstWbcSource — пустой указатель.
RK_ERR_VO_NOT_SUPPORT	enSourceType в pstWbcSource не поддерживается.

【Пример】

См. пример RK_MPI_VO_EnableWbc.

【Связанные темы】

RK_MPI_VO_GetWbcSource

5.5.2 RK_MPI_VO_GetWbcSource

【Описание】

Получает источник тракта обратной записи.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetWbcSource(VO_WBC VoWbc, VO_WBC_SOURCE_S *pstWbcSources)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoWbc	Номер тракта обратной записи.	Вход
pstWbcSource	Структура источника обратной записи видео.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_WBCID	Недопустимый VoWbc.
RK_ERR_VO_NULL_PTR	pstWbcSources — пустой указатель.

【Пример】

См. пример RK_MPI_VO_EnableWbc.

【Связанные темы】

RK_MPI_VO_SetWbcSource

5.5.3 RK_MPI_VO_EnableWbc

【Описание】

Включает обратную запись.

【Синтаксис】

```
RK_S32 RK_MPI_VO_EnableWbc(VO_WBC VoWbc)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoWbc	Номер тракта обратной записи.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_WBCID	Недопустимый VoWbc.
RK_ERR_VO_NO_MEM	Недостаточно системной памяти.
RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы обратной записи.
RK_ERR_VO_NOT_SUPPORT	Система не поддерживает операцию.

【Примечания】

- Перед включением необходимо задать атрибуты обратной записи видео.
- Повторное включение обратной записи возвращает успех.
- Для обратной записи видео enSourceType источника задаётся VO_WBC_SOURCE_VIDEO.
- Для обратной записи устройства enSourceType задаётся VO_WBC_SOURCE_DEV.
- Обратная запись видео поддерживает Auto-bind и ручной режим. В Auto-bind WBC необходимо привязать к другому модулю, например VO или VENC, но привязка к VPSS не поддерживается.

【Пример】

```
VO_WBC_SOURCE_S  stWbcSource;
VO_WBC_ATTR_S    stWbcAttr;
VO_WBC_MODE_E    enWbcMode;
RK_U32           VoWbc;
RK_S32           s32Ret = RK_SUCCESS;
MPP_CHN_S stSrcChn, stDestChn;

VoWbc = 0;
stWbcSource.enSourceType = VO_WBC_SOURCE_VIDEO
stWbcSource.u32SourceId = RK356X_VO_DEV_HD0;
RK_MPI_VO_SetWbcSource(VoWbc, &stWbcSource);

stWbcAttr.stTargetSize.u32Width = 1920;
stWbcAttr.stTargetSize.u32Height = 1080;
stWbcAttr.enPixelFormat = RK_FMT_RGB888;
stWbcAttr.u32FrameRate = 25;
stWbcAttr.enCompressMode = COMPRESS_MODE_NONE;

s32Ret = RK_MPI_VO_SetWbcAttr(VoWbc, &stWbcAttr);
if (s32Ret != RK_SUCCESS) {
    return s32Ret;
}

s32Ret = RK_MPI_VO_EnableWbc(VoWbc);
if (s32Ret != RK_SUCCESS) {
    return s32Ret;
}

/* bind wbc to VO, i.e. */
stSrcChn.enModId    = RK_ID_WBC;
stSrcChn.s32DevId   = VoWbc;
stSrcChn.s32ChnId   = 0;

stDestChn.enModId   = RK_ID_VO;
stDestChn.s32ChnId  = 0;
stDestChn.s32DevId  = RK356X_VOP_LAYER_CLUSTER_1;

RK_MPI_SYS_Bind(&stSrcChn, &stDestChn);

/* disable wbc */
s32Ret = RK_MPI_VO_DisableWbc(VoWbc);
if (s32Ret != RK_SUCCESS) {
    return s32Ret;
}
```

【Связанные темы】

RK_MPI_VO_DisableWbc

5.5.4 RK_MPI_VO_DisableWbc

【Описание】

Выключает обратную запись.

【Синтаксис】

```
RK_S32 RK_MPI_VO_DisableWbc(VO_WBC VoWbc)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoWbc	Номер тракта обратной записи.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_WBCID	Недопустимый VoWbc.

5.5.5 RK_MPI_VO_SetWbcAttr

【Описание】

Задаёт атрибуты обратной записи видео.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetWbcAttr(VO_WBC VoWbc, const VO_WBC_ATTR_S *pstWbcAttr)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoWbc	Номер тракта обратной записи.	Вход
pstWbcAttr	Указатель на атрибуты обратной записи видео.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_WBCID	Недопустимый VoWbc.
RK_ERR_VO_NULL_PTR	pstWbcAttr — пустой указатель.
RK_ERR_VO_NOT_SUPPORT	Формат цвета enPixelFormat в pstWbcAttr не поддерживается.

【Примечания】

- Максимальное выходное разрешение обратной записи видео — 1920x1080p.
- Поддерживается увеличение, например с 720p до 1080p.
- Атрибуты обратной записи видео не поддерживают формат NV12.

【Пример】

См. пример RK_MPI_VO_EnableWbc.

【Связанные темы】

RK_MPI_VO_GetWbcAttr

5.5.6 RK_MPI_VO_GetWbcAttr

【Описание】

Получает атрибуты обратной записи видео.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetWbcAttr(VO_WBC VoWbc, VO_WBC_ATTR_S *pstWbcAttr)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoWbc	Номер тракта обратной записи.	Вход
pstWbcAttr	Указатель на атрибуты обратной записи видео.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_WBCID	Недопустимый VoWbc.
RK_ERR_VO_NULL_PTR	pstWbcAttr — пустой указатель.

【Пример】

См. пример RK_MPI_VO_EnableWbc.

【Связанные темы】

RK_MPI_VO_SetWbcAttr

5.5.7 RK_MPI_VO_GetWbcFrame

【Описание】

Получает изображение обратной записи.

【Синтаксис】

```
RK_S32 RK_MPI_VO_GetWbcFrame(VO_WBC VoWbc, VIDEO_FRAME_INFO_S *pstVFrame, RK_S32 s32MilliSec)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoWbc	Номер тракта обратной записи.	Вход
pstVFrame	Указатель на структуру полученного изображения обратной записи.	Вход
s32MilliSec	Параметр тайм-аута, сейчас 0.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
RK_ERR_VO_INVALID_WBCID	Недопустимый VoWbc.
RK_ERR_VO_NULL_PTR	pstVFrame — пустой указатель.
RK_ERR_VO_WBC_NOT_CONFIG	Обратная запись VoWbc не настроена или не включена.

5.5.8 RK_MPI_VO_ReleaseWbcFrame

【Описание】

Освобождает изображение обратной записи.

【Синтаксис】

```
RK_S32 RK_MPI_VO_ReleaseWbcFrame(VO_WBC VoWbc, VIDEO_FRAME_INFO_S *pstVFrame)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoWbc	Номер тракта обратной записи.	Вход
pstVFrame	Указатель на структуру освобождаемого изображения выходного канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
RK_ERR_VO_INVALID_WBCID	Недопустимый VoWbc.
RK_ERR_VO_NULL_PTR	pstVFrame — пустой указатель.

5.5.9 RK_MPI_VO_SetWbcDepth

【Описание】

Задаёт число буферов обратной записи.

【Синтаксис】

```
RK_S32 RK_MPI_VO_SetWbcDepth(VO_WBC VoWbc, RK_S32 u32Depth)
```

【Параметры】

Имя параметра	Описание	Вход/выход
VoWbc	Номер тракта обратной записи.	Вход
u32Depth	Число буферов.	Вход

6. Типы данных

Ниже определены типы и структуры данных видеовывода:

- VO_DEV: определяет номер устройства.
- VO_LAYER: определяет номер видеослоя.
- VO_CHN: определяет номер канала видеовывода.
- VO_WBC: определяет номер тракта обратной записи.
- VO_MAX_DEV_NUM: определяет максимальное число устройств видеовывода.
- VO_MAX_LAYER_NUM: определяет максимальное число слоёв.
- VO_MAX_CHN_NUM: определяет максимальное число каналов.
- VO_MAX_WBC_NUM: определяет максимальное число устройств обратной записи.
- VO_LAYER_MODE_E: определяет тип слоя.
- VO_PUB_ATTR_S: определяет структуру атрибутов устройства видеовывода.
- VO_VIDEO_LAYER_ATTR_S: определяет структуру общих атрибутов видеовывода.

- VO_WBC_ATTR_S: определяет структуру атрибутов обратной записи видео.
- VO_WBC_SOURCE_S: определяет источник обратной записи видео.

6.1 VO_DEV

【Описание】

Определяет номер устройства вывода.

【Определение】

```
typedef RK_S32 VO_DEV;
```

6.2 VO_LAYER

【Описание】

Определяет номер слоя.

【Определение】

```
typedef RK_S32 VO_LAYER;
```

【Примечания】

Значения см. в VO_LAYER_NAME_RK356X_E, VO_LAYER_NAME_E и VO_VIR_LAYER_NAME_E.

6.3 VO_CHN

【Описание】

Определяет номер канала видеовывода.

【Определение】

```
typedef RK_S32 VO_CHN;
```

6.4 VO_WBC

【Описание】

Определяет номер тракта обратной записи.

【Определение】

```
typedef RK_S32 VO_WBC;
```

6.5 VO_MAX_DEV_NUM

【Описание】

Определяет максимальное число устройств видеовывода.

【Определение】

```
#define VO_MAX_DEV_NUM              3
```

6.6 VO_MAX_LAYER_NUM

【Описание】

Определяет максимальное число слоёв.

【Определение】

```
#define VO_MAX_LAYER_NUM              8
```

6.7 VO_MAX_CHN_NUM

【Описание】

Определяет максимальное число каналов.

【Определение】

```
#define VO_MAX_CHN_NUM              128
```

6.8 VO_MAX_WBC_NUM

【Описание】

Определяет максимальное число устройств обратной записи.

【Определение】

```
#define VO_MAX_WBC_NUM              1
```

6.9 VO_GFX_MODE_E

【Описание】

Определяет режим создания buffer слоя.

【Определение】

```
typedef enum rkVO_GFX_MODE_E {
    VO_MODE_NORMAL,
    VO_MODE_GFX_PRE_CREATED
} VO_GFX_MODE_E;
```

【Члены】

Имя члена	Описание
VO_MODE_NORMAL	Обычный режим.
VO_MODE_GFX_PRE_CREATED	Режим предварительного создания.

6.10 VO_HDMI_MODE_E

【Описание】

Определяет режим вывода HDMI.

【Определение】

```
typedef enum rkVO_HDMI_MODE_E {
    VO_HDMI_MODE_AUTO = 0,
    VO_HDMI_MODE_HDMI,
    VO_HDMI_MODE_DVI
} VO_HDMI_MODE_E;
```

Члены

Имя члена	Описание
VO_HDMI_MODE_AUTO	Автоматический выбор по EDID.
VO_HDMI_MODE_HDMI	Принудительный режим HDMI.
VO_HDMI_MODE_DVI	Принудительный режим DVI.

6.11 VO_HDMI_COLOR_FMT_E

Описание

Определяет формат цвета вывода HDMI.

Определение

```
typedef enum rkVO_HDMI_COLOR_FMT_E {
    VO_HDMI_COLOR_FORMAT_RGB = 0,
    VO_HDMI_COLOR_FORMAT_YCBCR444,
    VO_HDMI_COLOR_FORMAT_YCBCR422,
    VO_HDMI_COLOR_FORMAT_YCBCR420,
    VO_HDMI_COLOR_FORMT_AUTO,
    VO_HDMI_COLOR_FORMAT_BUTT
} VO_HDMI_COLOR_FMT_E;
```

Члены

Имя члена	Описание
VO_HDMI_COLOR_FORMAT_RGB	RGB
VO_HDMI_COLOR_FORMAT_YCBCR444	YCbCr444
VO_HDMI_COLOR_FORMAT_YCBCR422	YCbCr422
VO_HDMI_COLOR_FORMAT_YCBCR420	YCbCr420
VO_HDMI_COLOR_FORMT_AUTO	Выбор по EDID в порядке YCbCr444 -> YCbCr422 -> YCbCr420 -> RGB.

6.12 VO_HDMI_QUANT_RANGE_E

Описание

Определяет диапазон квантования вывода HDMI.

Определение

```
typedef enum rkVO_HDMI_QUANT_RANGE_E {
    VO_HDMI_QUANT_RANGE_AUTO = 0,
    VO_HDMI_QUANT_RANGE_LIMITED,
    VO_HDMI_QUANT_RANGE_FULL,
    VO_HDMI_QUANT_RANGE_BUTT
} VO_HDMI_QUANT_RANGE_E;
```

Члены

Поле	Описание
VO_HDMI_QUANT_RANGE_AUTO	Автоматически.
VO_HDMI_QUANT_RANGE_LIMITED	Ограниченный диапазон [16-235].
VO_HDMI_QUANT_RANGE_FULL	Полный диапазон [0-255].

Примечания

- Действует только при выводе в формате RGB.

6.13 VO_PUB_ATTR_S

Описание

Структура общих атрибутов видеовывода.

Определение

```
typedef struct rkVO_PUB_ATTR_S {
    RK_U32 u32BgColor;
    VO_INTF_TYPE_E enIntfType;
    VO_INTF_SYNC_E enIntfSync;
    VO_SYNC_INFO_S stSyncInfo;
} VO_PUB_ATTR_S;
```

Члены

Имя члена	Описание
u32BgColor	Не используется.
enIntfType	Тип интерфейса отображения; можно выбрать следующие значения или их комбинацию: VO_INTF_CVBS VO_INTF_YPBPR VO_INTF_VGA VO_INTF_BT656 VO_INTF_BT1120 VO_INTF_LCD VO_INTF_LVDS VO_INTF_MIPI VO_INTF_MIPI1 VO_INTF_EDP VO_INTF_EDP1 VO_INTF_HDMI VO_INTF_HDMI1 VO_INTF_DP VO_INTF_DP1
enIntfSync	Тайминги интерфейса отображения.
stSyncInfo	Сведения о таймингах интерфейса. Структура действует, если intf_sync настроен на пользовательские тайминги RK_VO_OUT_USER.

Примечания

- Если система содержит два выходных интерфейса одного типа, VO_INTF_MIPI1, VO_INTF_EDP1, VO_INTF_HDMI1 и VO_INTF_DP1 обозначают второй интерфейс.
- При привязке нескольких интерфейсов RK3588 к одному VoDev формат цвета на выходах должен совпадать, например DP и HDMI одновременно выводят RGB888.

6.14 VO_EDID_S

【Описание】

Структура данных EDID.

【Определение】

```
typedef struct rk_VO_EDID_S {
    RK_BOOL             bEdidValid;
    RK_U32              u32Edidlength;
    RK_U8               u8Edid[512];
} VO_EDID_S;
```

【Члены】

Имя члена	Описание
bEdidValid	Флаг действительности EDID.
u32Edidlength	Длина данных EDID.
u8Edid	Данные EDID.

6.15 VO_SINK_CAPABILITY_S

【Описание】

Структура атрибутов состояния устройства отображения.

【Определение】

```
typedef struct rk_VO_SINK_CAPABILITY_S {
    RK_BOOL             bConnected;
    RK_BOOL             bSupportYCbCr;
    RK_BOOL             bSupportHDMI;
} VO_SINK_CAPABILITY_S;
```

【Члены】

Имя члена	Описание
bConnected	Состояние подключения устройства: RK_TRUE — подключено, RK_FALSE — отключено.
bSupportYCbCr	Поддерживает ли устройство данные YCbCr: RK_TRUE — да, RK_FALSE — нет.
bSupportHDMI	Поддерживает ли устройство HDMI: RK_TRUE — да, RK_FALSE — нет.

6.16 RK_VO_CallBack

【Описание】

Callback события горячего подключения устройства отображения.

【Определение】

```
typedef void (*RK_VO_CallBack)(RK_VOID *pPrivateData);
```

6.17 RK_VO_CALLBACK_FUNC_S

【Описание】

Структура callback события горячего подключения устройства отображения.

【Определение】

```
typedef struct rk_VO_CALLBACK_FUNC_S {
    RK_VO_CallBack      pfnEventCallback;
    RK_VOID             *pPrivateData;
} RK_VO_CALLBACK_FUNC_S;
```

【Члены】

Имя члена	Описание
pfnEventCallback	Callback-функция.
pPrivateData	Частные данные callback-функции.

6.18 RK_VO_VsyncCallBack

【Описание】

Callback события кадрового прерывания устройства отображения.

【Определение】

```
typedef void (*RK_VO_CallBack)(RK_VOID *pPrivateData, VO_CB_INFO_S* info);
```

6.19 RK_VO_CALLBACK_FUNC_S

【Описание】

Структура callback кадрового прерывания устройства отображения.

【Определение】

```
typedef struct rk_VO_CALLBACK_FUNC_S {
    RK_VO_VsyncCallBack pfnEventCallback;
    RK_VOID             *pPrivateData;
} RK_VO_CALLBACK_FUNC_S;
```

【Члены】

Имя члена	Описание
pfnEventCallback	Callback-функция.
pPrivateData	Частные данные callback-функции.

6.20 VO_CB_INFO_S

【Описание】

Информация о событии кадрового прерывания устройства отображения.

【Определение】

```
typedef struct rk_VO_CALLBACK_FUNC_S {
    RK_U32 u32Id;
    RK_U32 u32Sec;
    RK_U32 u32Usec;
} RK_VO_CALLBACK_FUNC_S;
```

【Члены】

Имя члена	Описание
u32Id	Номер устройства отображения.
u32Sec	Текущее время прерывания в секундах.
u32Usec	Текущее время прерывания в микросекундах.

6.21 VO_HDMI_PARAM_S

【Описание】

Структура атрибутов HDMI.

【Определение】

```
typedef struct rk_VO_HDMI_PARAM_S {
    VO_HDMI_MODE_E enHdmiMode;
    VO_HDMI_COLOR_FMT_E enColorFmt;
    VO_HDMI_QUANT_RANGE_E enQuantRange;
} VO_HDMI_PARAM_S;
```

【Члены】

Имя члена	Описание
enHdmiMode	Режим вывода.
enColorFmt	Формат цвета вывода.
enQuantRange	Диапазон квантования вывода.

【Примечания】

- enQuantRange действует только если enColorFmt — формат RGB.

6.22 VO_FRAME_INFO_S

【Описание】

Определяет структуру информации о графическом кадре.

【Определение】

```
typedef struct rkVO_GFX_FRAME_INFO_S {
    RK_U32              u32Width;
    RK_U32              u32Height;
    RK_U32              u32VirWidth;
    RK_U32              u32VirHeight;
    PIXEL_FORMAT_E      enPixelFormat;
    RK_U32              u32FgAlpha;
    RK_U32              u32BgAlpha;
    RK_VOID             *pData;
    RK_U32              u32Size;
} VO_FRAME_INFO_S;
```

【Члены】

Имя члена	Описание
u32Width	Ширина изображения в пикселях.
u32Height	Высота изображения в пикселях.
u32VirWidth	Виртуальная ширина изображения в пикселях.
u32VirHeight	Виртуальная высота изображения в пикселях.
enPixelFormat	Формат пикселей.
u32FgAlpha	Значение Alpha переднего плана.
u32BgAlpha	Значение Alpha фона.
pData	Адрес памяти.
u32Size	Размер памяти.

【Примечания】

- Если формат данных графического канала BGRA5551/RGBA5551, необходимо заполнить u32FgAlpha и u32BgAlpha.

6.23 VO_LAYER_NAME_RK356X_E

【Описание】

Определяет номера слоёв RK356X.

【Определение】

```
typedef enum rk356X_VO_LAYER_NAME_E {
    RK356X_VOP_LAYER_CLUSTER0 = 0,
    RK356X_VOP_LAYER_CLUSTER1 = 2,
    RK356X_VOP_LAYER_ESMART0,
    RK356X_VOP_LAYER_ESMART1,
    RK356X_VOP_LAYER_SMART0,
    RK356X_VOP_LAYER_SMART1,
} VO_LAYER_NAME_RK356X_E;
```

【Члены】

Имя члена	Описание
RK356X_VOP_LAYER_CLUSTER0	Слой CLUSTER 0.
RK356X_VOP_LAYER_CLUSTER1	Слой CLUSTER 1.
RK356X_VOP_LAYER_ESMART0	Слой ESMART 0.
RK356X_VOP_LAYER_ESMART1	Слой ESMART 1.
RK356X_VOP_LAYER_SMART0	Слой SMART 0.
RK356X_VOP_LAYER_SMART1	Слой SMART 1.

【Примечания】

Применяется к серии RK356X.

6.24 VO_LAYER_NAME_E

【Описание】

Определяет номер слоя.

【Определение】

```
typedef enum rkVOP2_LAYER_NAME_E {
    VO_LAYER_CLUSTER0 = 0,
    VO_LAYER_CLUSTER1,
    VO_LAYER_CLUSTER2,
    VO_LAYER_CLUSTER3,
    VO_LAYER_ESMART0,
    VO_LAYER_ESMART1,
    VO_LAYER_ESMART2,
    VO_LAYER_ESMART3,
    VO_LAYER_BUTT
} VO_LAYER_NAME_E;
```

【Члены】

Имя члена	Описание
VO_LAYER_CLUSTER0	Слой CLUSTER 0.
VO_LAYER_CLUSTER1	Слой CLUSTER 1.
VO_LAYER_CLUSTER2	Слой CLUSTER 2.
VO_LAYER_CLUSTER3	Слой CLUSTER 3.
VO_LAYER_ESMART0	Слой ESMART 0.
VO_LAYER_ESMART1	Слой ESMART 1.
VO_LAYER_ESMART2	Слой ESMART 2.
VO_LAYER_ESMART3	Слой ESMART 3.

6.25 VO_VIR_LAYER_NAME_E

【Описание】

Определяет номера виртуальных слоёв.

【Определение】

```
typedef enum rkVO_VIR_LAYER_NAME_E {
    VO_LAYER_VIRTUAL0 = VO_LAYER_BUTT,
    VO_LAYER_VIRTUAL1,
    VO_LAYER_VIRTUAL2,
    VO_LAYER_VIRTUAL3,
} VO_VIR_LAYER_NAME_E;
```

【Члены】

Имя члена	Описание
VO_LAYER_VIRTUAL0	Виртуальный слой 0.
VO_LAYER_VIRTUAL1	Виртуальный слой 1.
VO_LAYER_VIRTUAL2	Виртуальный слой 2.
VO_LAYER_VIRTUAL3	Виртуальный слой 3.

6.26 VO_LAYER_MODE_E

【Описание】

Определяет типы слоёв.

【Определение】

```
typedef enum rkVO_LAYER_MODE_E {
    VO_LAYER_MODE_CURSOR = 0,
    VO_LAYER_MODE_GRAPHIC,
    VO_LAYER_MODE_VIDEO,
    VO_LAYER_MODE_VIRTUAL,
    VO_LAYER_MODE_BUTT
} VO_LAYER_MODE_E;
```

【Члены】

Имя члена	Описание
VO_LAYER_MODE_CURSOR	Слой курсора.
VO_LAYER_MODE_GRAPHIC	Графический слой.
VO_LAYER_MODE_VIDEO	Видеослой.
VO_LAYER_MODE_VIRTUAL	Виртуальный слой.

6.27 VO_SPLICE_MODE_E

【Описание】

Определяет типы объединения слоёв.

【Определение】

```
typedef enum rkVO_SPLICE_MODE_E {
    VO_SPLICE_MODE_GPU = 0,
    VO_SPLICE_MODE_RGA
} VO_SPLICE_MODE_E;
```

【Члены】

Имя члена	Описание
VO_SPLICE_MODE_GPU	Режим объединения GPU.
VO_SPLICE_MODE_RGA	Режим объединения RGA.

6.28 VO_VIDEO_LAYER_ATTR_S

【Описание】 Определяет атрибуты видеослоя.

【Определение】

```
typedef struct rkVO_VIDEO_LAYER_ATTR_S {
    RECT_S stDispRect;
    SIZE_S stImageSize;
    RK_U32 u32DispFrmRt;
    PIXEL_FORMAT_E enPixFormat;
    RK_BOOL bBypassFrame;
    RK_BOOL bLowDelay;
    COMPRESS_MODE_E enCompressMode;
    DYNAMIC_RANGE_E enDstDynamicRange;
} VO_VIDEO_LAYER_ATTR_S;
```

【Члены】

Имя члена	Описание
stDispRect	Размер области отображения.
stImageSize	Размер холста видеослоя.
u32DispFrmRt	Частота отображения кадров.
enPixFormat	Формат пикселей видеослоя.
bBypassFrame	Видеокадр без объединения напрямую передаётся видеослою.
bLowDelay	Событие отправки кадра канала 0 запускает объединение; результат немедленно выводится.
enCompressMode	Режим сжатия кадрового буфера видеослоя.
enDstDynamicRange	Не используется.

6.29 VO_CHN_ATTR_S

【Описание】

Структура атрибутов канала.

【Определение】

```
typedef struct rkVO_CHN_ATTR_S {
    RK_U32 u32Priority; /* Video out overlay pri sd */
    RECT_S stRect;      /* Rectangle of video output channel */
    RK_BOOL bDeflicker; /* Deflicker or not sd */
    RK_U32 u32FgAlpha;  /* Alpha of A = 1 in pixel format BGRA5551/RGBA5551 */
    RK_U32 u32BgAlpha;  /* Alpha of A = 0 in pixel format BGRA5551/RGBA5551 */
    RK_BOOL bEnKeyColor;/* Enable key color or not when pixel format BGRA5551/RGBA5551*/
    RK_U32 u32KeyColor; /* Key color value of pixel format BGRA5551/RGBA5551, B[0:4] G[5:9]
R[10:14] */
    MIRROR_E enMirror; /* RW, Mirror */
    ROTATION_E enRotation;    /* RW, rotation. */
    RK_U32 u32MaxChnQueue;    /* vo channel max queue length */
} VO_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
u32Priority	Приоритет канала. При перекрытии областей канал с большим приоритетом закрывает канал с меньшим.
stRect	Область отображения канала на холсте слоя.
bDeflicker	Не поддерживается.
u32FgAlpha	Для BGRA5551/RGBA5551 значение Alpha, соответствующее A=1.
u32BgAlpha	Для BGRA5551/RGBA5551 значение Alpha, соответствующее A=0.
bEnKeyColor	Включает/выключает ключевой цвет для BGRA5551/RGBA5551.
u32KeyColor	Ключевой цвет для BGRA5551/RGBA5551.
enMirror	Включает mirror.
enRotation	Включает rotation.
u32MaxChnQueue	Максимальная длина queue канала VO; если не задана, по умолчанию 10.

Примечания

- Для формата BGRA5551/RGBA5551 необходимо заполнить u32FgAlpha и u32BgAlpha.
- u32Priority — динамический атрибут, вступает в силу в следующем цикле объединения.
- stRect — динамический атрибут, вступает в силу в следующем цикле объединения.
- u32FgAlpha — динамический атрибут, вступает в силу в следующем цикле; в режиме RGA не действует.
- u32BgAlpha — динамический атрибут, вступает в силу в следующем цикле; в режиме RGA не действует.
- bEnKeyColor — динамический атрибут, вступает в силу в следующем цикле; в режиме RGA не действует.
- u32KeyColor — динамический атрибут, вступает в силу в следующем цикле; в режиме RGA не действует.
- enMirror — динамический атрибут, вступает в силу в следующем цикле объединения.
- enRotation — динамический атрибут, вступает в силу в следующем цикле объединения.

6.30 VO_CHN_PARAM_S

Описание

Структура параметров канала.

Определение

```
typedef struct rkVO_CHN_PARAM_S {
    ASPECT_RATIO_S stAspectRatio; /* RW; aspect ratio */
} VO_CHN_PARAM_S;
```

Члены

Имя члена	Описание
stAspectRatio	Атрибуты соотношения сторон изображения.

Примечания

- stAspectRatio — динамический атрибут, вступает в силу в следующем цикле; в режиме RGA не действует.

6.31 VO_BORDER_S

Описание

Структура атрибутов рамки.

Определение

```
typedef struct rkVO_BORDER_S {
    RK_BOOL bBorderEn; /* RW; Do frame or not */
    BORDER_S stBorder; /* RW; frame's top, bottom, left, right width and color */
} VO_BORDER_S;
```

【Члены】

Имя члена	Описание
bBorderEn	Включает рамку.
stBorder	Атрибуты рамки.

【Примечания】

- bBorderEn — динамический атрибут, вступает в силу в следующем цикле; в режиме RGA не действует.
- stBorder — динамический атрибут, вступает в силу в следующем цикле; в режиме RGA не действует.

6.32 VO_WBC_ATTR_S

【Описание】

Определяет атрибуты обратной записи видео.

【Определение】

```
typedef struct rkVO_WBC_ATTR_S {
    SIZE_S stTargetSize;
    PIXEL_FORMAT_E enPixelFormat;
    RK_U32 u32FrameRate;
    DYNAMIC_RANGE_E enDynamicRange;
    COMPRESS_MODE_E enCompressMode;
} VO_WBC_ATTR_S;
```

【Члены】

Имя члена	Описание
stTargetSize	Размер изображения обратной записи.
enPixelFormat	Формат данных обратной записи видео.
u32FrameRate	Не используется.
enDynamicRange	Не используется.
enCompressMode	Режим сжатия обратной записи видео.

6.33 VO_WBC_SOURCE_TYPE_E

【Описание】

Определяет типы обратной записи видео.

【Определение】

```
typedef enum rkVO_WBC_SOURCE_TYPE_E {
    VO_WBC_SOURCE_DEV = 0x0, /* WBC source is device */
    VO_WBC_SOURCE_VIDEO = 0x1, /* WBC source is video layer */
    VO_WBC_SOURCE_GRAPHIC = 0x2, /* WBC source is graphic layer. Warning: not supported */
    VO_WBC_SOURCE_VIRTUAL = 0x3, /* WBC source is virtual layer */
    VO_WBC_SOURCE_BUTT
} VO_WBC_SOURCE_TYPE_E;
```

【Члены】

Имя члена	Описание
VO_WBC_SOURCE_DEV	Источник обратной записи — устройство.
VO_WBC_SOURCE_VIDEO	Источник обратной записи — видеослой.
VO_WBC_SOURCE_GRAPHIC	Источник обратной записи — графический слой; сейчас не поддерживается.
VO_WBC_SOURCE_VIRTUAL	Источник обратной записи — виртуальный слой.

6.34 VO_WBC_SOURCE_S

【Описание】

Определяет источник обратной записи видео.

【Определение】

```
typedef struct rkVO_WBC_SOURCE_S {
    VO_WBC_SOURCE_TYPE_E enSourceType;
    RK_U32 u32SourceId;
} VO_WBC_SOURCE_S;
```

【Члены】

Имя члена	Описание
enSourceType	Перечисление типа источника обратной записи видео.
u32SourceId	Номер источника обратной записи видео.

【Примечания】

- Если enSourceType = VO_WBC_SOURCE_VIRTUAL, u32SourceId — ID виртуального слоя.
- Если enSourceType = VO_WBC_SOURCE_DEV/VO_WBC_SOURCE_VIDEO/VO_WBC_SOURCE_GRAPHIC, u32SourceId — ID VoDev.

7. Коды ошибок VO

Ниже приведены коды ошибок API видеовывода VO:

Код ошибки	Макроопределение	Описание
0xA0098012	RK_ERR_VO_BUSY	Ошибка создания службы VO, включения вывода, задания разрешения и т. д.
0xA009800C	RK_ERR_VO_NO_MEM	Недостаточно системной памяти.
0xA0098006	RK_ERR_VO_NULL_PTR	Пустой указатель в параметрах функции.
0xA0098010	RK_ERR_VO_SYS_NOTREADY	Не удалось создать службы VO.
0xA0098001	RK_ERR_VO_INVALID_DEVID	ID устройства вне допустимого диапазона.
0xA0098002	RK_ERR_VO_INVALID_CHNID	ID канала вне допустимого диапазона.
0xA0098003	RK_ERR_VO_ILLEGAL_PARAM	Параметр вне допустимого диапазона.
0xA0098008	RK_ERR_VO_NOT_SUPPORT	Входной параметр не поддерживается VO.
0xA0098009	RK_ERR_VO_NOT_PERMIT	Операция не разрешена.
0xA0098058	RK_ERR_VO_INVALID_WBCID	Номер WBC вне диапазона.
0xa0098059	RK_ERR_VO_INVALID_LAYERID	Номер видеослоя вне диапазона.
0xA0098041	RK_ERR_VO_DEV_NOT_ENABLE	Соответствующее устройство не включено.
0xA0098042	RK_ERR_VO_DEV_HAS_ENABLED	Соответствующее устройство уже включено.
0xA0098043	RK_ERR_VO_DEV_HAS_BINDED	Устройство уже привязано к другому устройству вывода.
0xA0098044	RK_ERR_VO_DEV_NOT_BINDED	Устройство вывода не привязано.
0xA0098045	RK_ERR_VO_LAYER_NOT_ENABLE	Слой не включён.
0xA0098047	RK_ERR_VO_LAYER_NOT_CONFIG	Слой не настроен.
0xA009805b	RK_ERR_VO_LAYER_NOT_BINDED	Слой не привязан.
0xA0098055	RK_ERR_VO_WBC_NOT_DISABLE	WBC не выключен.
0xA0098056	RK_ERR_VO_WBC_NOT_CONFIG	Атрибуты WBC не заданы.
0xA0098049	RK_ERR_VO_CHN_NOT_ENABLE	Канал не включён.

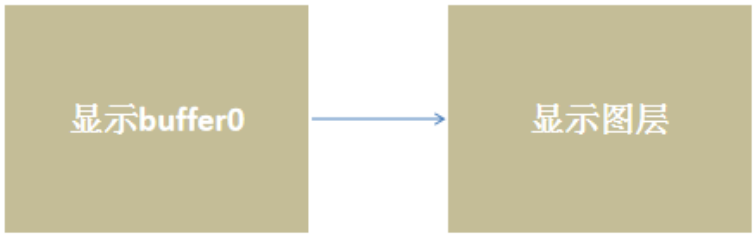
8. Описание отрисовки UI

Текущие API поддерживают следующие сценарии отрисовки UI.

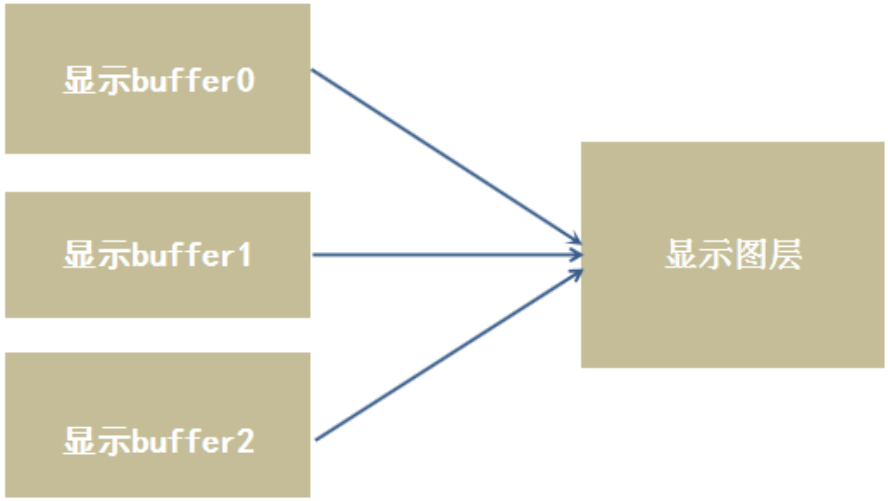
Способ отрисовки UI	Слой VOP	Режим buffer	Способ выделения buffer	GPU участвует	Способ отправки VO	Форматы
1	UI-слой	Один buffer	RK_MPI_VO_CreateGraphicsFrameBuffer	Нет	RK_MPI_VO_SendLayerFrame	ARGB8888
2	UI-слой	Несколько buffer	RK_MPI_VO_CreateGraphicsFrameBuffer	Нет	RK_MPI_VO_SendLayerFrame	ARGB8888
3	UI-слой	Один buffer	RK_MPI_VO_GetGfxFrameBuffer	Да	Функция отправки кадров не нужна	ARGB1555/ARGB8888
4	UI-слой	Несколько buffer	RK_MPI_VO_CreateGraphicsFrameBuffer	Да	RK_MPI_VO_SendFrame	ARGB1555/ARGB8888
5	Видеослой	Один buffer	RK_MPI_VO_GetGfxFrameBuffer	Да	Функция отправки кадров не нужна	ARGB1555/ARGB8888
6	Видеослой	Несколько buffer	RK_MPI_VO_CreateGraphicsFrameBuffer	Да	RK_MPI_VO_SendFrame	ARGB1555/ARGB8888

Примечания

- Схема режима с одним buffer приведена ниже.



- Схема режима с несколькими buffer приведена ниже.



8.0.1 【Пример способа 1】

```
VO_PUB_ATTR_S            stVoPubAttr;
VO_VIDEO_LAYER_ATTR_S    stLayerAttr;
VIDEO_FRAME_INFO_S       stVFrame;
VO_LAYER VoLayer;
VO_DEV VoDev;
RK_VOID *pMblk = RK_NULL;
RK_U32 s32Ret = RK_SUCCESS;
RK_U32 u32BuffSize;

/* alloc single buffer, and fill pMbk for UI data */
u32BuffSize = RK_MPI_VO_CreateGraphicsFrameBuffer(1920, 1080, RK_FMT_ARGB8888, &pMblk);
/* store data to stVFrame.stVFrame.pMbBlk */
...

/* Bind Layer */
VoLayer = RK356X_VOP_LAYER_ESMART_0;
VoDev == RK356X_VO_DEV_HD0;

RK_MPI_VO_BindLayer(VoLayer, RK356X_VO_DEV_HD0, VO_LAYER_MODE_GRAPHIC);

stVoPubAttr.enIntfType = VO_INTF_HDMI;
stVoPubAttr.enIntfSync = VO_OUTPUT_1080P60;

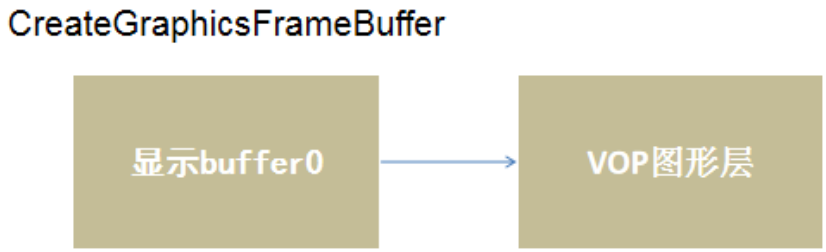
/* Enable VO Device */
RK_MPI_VO_SetPubAttr(VoDev, &stVoPubAttr);
RK_MPI_VO_Enable(VoDev);
```

```
    /* Enable Layer */
    stLayerAttr.enPixFormat                 = RK_FMT_ARGB8888;
    stLayerAttr.stDispRect.s32X             = 0;
    stLayerAttr.stDispRect.s32Y             = 0;
    tLayerAttr.stDispRect.u32Width          = 1920;
    stLayerAttr.stDispRect.u32Height        = 1080;
    stLayerAttr.stImageSize.u32Width        = 1920;
    stLayerAttr.stImageSize.u32Height       = 1080;
    /* Set Layer */
    RK_MPI_VO_SetLayerAttr(VoLayer, &stLayerAttr);
    RK_MPI_VO_EnableLayer(VoLayer);

    stVFrame.stVFrame.pMbBlk = pMblk;
    RK_MPI_VO_SendLayerFrame(VoLayer, &stVFrame);
```

Примечания

- В способе 1 один buffer внешне выделяется через RK_MPI_VO_CreateGraphicsFrameBuffer и передаётся UI-слою для отображения, как показано ниже.



8.0.2 Пример способа 2

```
    VO_PUB_ATTR_S            stVoPubAttr;
    VO_VIDEO_LAYER_ATTR_S    stLayerAttr;
    VIDEO_FRAME_INFO_S       stVFrame;
    VO_LAYER VoLayer;
    VO_DEV VoDev;
    RK_VOID *pMblk = RK_NULL;
    RK_VOID *pMblk_480P = RK_NULL;
    RK_U32 s32Ret = RK_SUCCESS;
    RK_U32 u32BuffSize;
    RK_U32 count;

    /* alloc muti buffer, and fill pMbk for UI data */
    u32BuffSize = RK_MPI_VO_CreateGraphicsFrameBuffer(1920, 1080, RK_FMT_ARGB8888, &pMblk);
    u32BuffSize = RK_MPI_VO_CreateGraphicsFrameBuffer(1920, 1080, RK_FMT_ARGB8888, &pMblk_480P);
    /* store data to stVFrame.stVFrame.pMbBlk */
    ...

    /* Bind Layer */
    VoLayer = RK356X_VOP_LAYER_ESMART_0;
    VoDev == RK356X_VO_DEV_HD0;

    RK_MPI_VO_BindLayer(VoLayer, RK356X_VO_DEV_HD0, VO_LAYER_MODE_GRAPHIC);

    stVoPubAttr.enIntfType = VO_INTF_HDMI;
    stVoPubAttr.enIntfSync = VO_OUTPUT_1080P60;

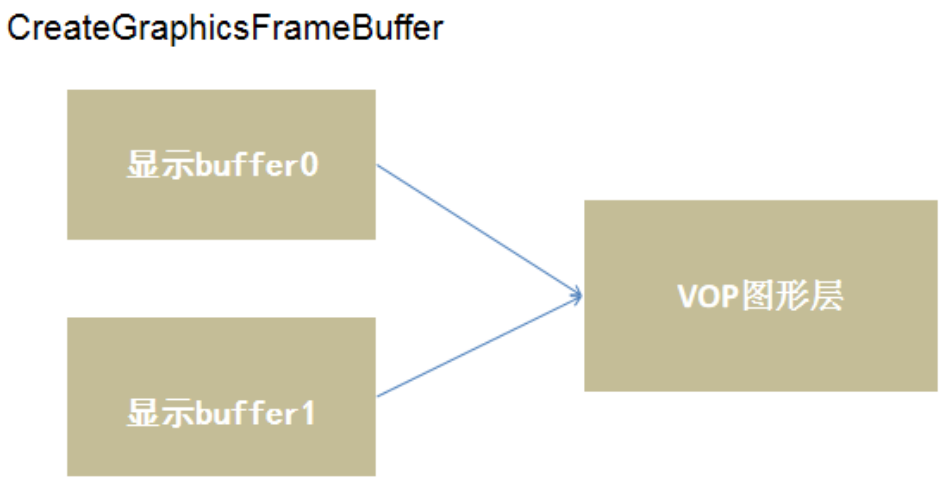
    /* Enable VO Device */
    RK_MPI_VO_SetPubAttr(VoDev, &stVoPubAttr);
    RK_MPI_VO_Enable(VoDev);
    /* Enable Layer */
```

```
    stLayerAttr.enPixFormat                 = RK_FMT_ARGB8888;
    stLayerAttr.stDispRect.s32X             = 0;
    stLayerAttr.stDispRect.s32Y             = 0;
    tLayerAttr.stDispRect.u32Width          = 1920;
    stLayerAttr.stDispRect.u32Height        = 1080;
    stLayerAttr.stImageSize.u32Width        = 1920;
    stLayerAttr.stImageSize.u32Height       = 1080;
    /* Set Layer */
    RK_MPI_VO_SetLayerAttr(VoLayer, &stLayerAttr);
    RK_MPI_VO_EnableLayer(VoLayer);

     while (1) {
        count++;
        if (count % 2) {
            stVFrame.stVFrame.pMbBlk = pMblk;
            RK_MPI_VO_SendLayerFrame(VoLayer, &stVFrame);
        } else {
            stVFrame.stVFrame.pMbBlk = pMblk_480P;
            RK_MPI_VO_SendLayerFrame(VoLayer, &stVFrame);
        }
    }
```

Примечания

- В этом примере через RK_MPI_VO_CreateGraphicsFrameBuffer внешне выделяются два buffer, которые напрямую передаются графическому UI-слою для отображения, как показано ниже.



8.0.3 Пример способа 3

```
    VO_PUB_ATTR_S         stVoPubAttr;
    VO_FRAME_INFO_S       stVFrame;
    VO_LAYER VoLayer;
    VO_VIDEO_LAYER_ATTR_S    stLayerAttr;
    RK_U32 s32Ret = RK_SUCCESS;

    stLayerAttr.enPixFormat                 = RK_FMT_ARGB8888;
    stLayerAttr.stDispRect.s32X             = 0;
    stLayerAttr.stDispRect.s32Y             = 0;
    stLayerAttr.stDispRect.u32Width         = 1920;
    stLayerAttr.stDispRect.u32Height        = 1080t;
    stLayerAttr.stImageSize.u32Width        = 1920;
    stLayerAttr.stImageSize.u32Height       = 1080;
    VoDev = RK356X_VO_DEV_HD0;
    VoLayer = RK356X_VOP_LAYER_ESMART_0; /* UI layer esmart0 */
    RK_MPI_VO_SetGFxMode(VO_MODE_GFX_PRE_CREATED);
    stVFrame.enPixelFormat = RK_FMT_ARGB8888;
    stVFrame.u32Width = 1920;
```

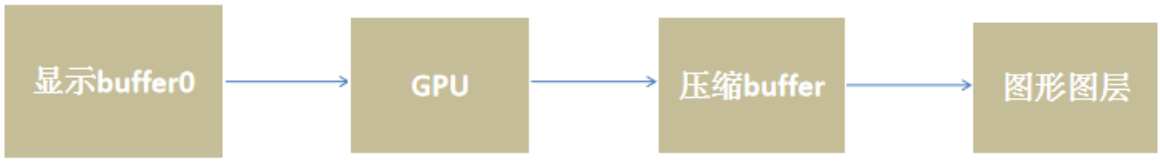
```
    stVFrame.u32Height = 1080;
    RK_MPI_VO_GetGfxFrameBuffer(VoLayer, 0, &stVFrame);
    /* fill stVFrame */
    memset(stVFrame.pData, 0xff, stVFrame.u32Size);
    stVoPubAttr.enIntfSync = VO_OUTPUT_1080P60;
    stVoPubAttr.enIntfType = VO_INTF_HDMI;
    RK_MPI_VO_SetPubAttr(VoDev, &pstPubAttr);
    RK_MPI_VO_Enable(VoDev);

    s32Ret = RK_MPI_VO_SetLayerAttr(VoLayer, &stLayerAttr);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;
    s32Ret = RK_MPI_VO_EnableLayer(VoLayer);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;
```

Примечания

- После выделения одного buffer и заполнения данных UI-изображение передаётся UI-слою при включении VO.
- В способе 3 один buffer проходит GPU, объединяется внутри и напрямую выводится слоем без функции отправки кадров, как показано ниже.

RK_MPI_VO_GetGfxFrameBuffer



Пример способа 4

```
    MPI_CTX_S *ctx = (MPI_CTX_S *) (pArgs);
    RK_S32 s32Ret = 0;

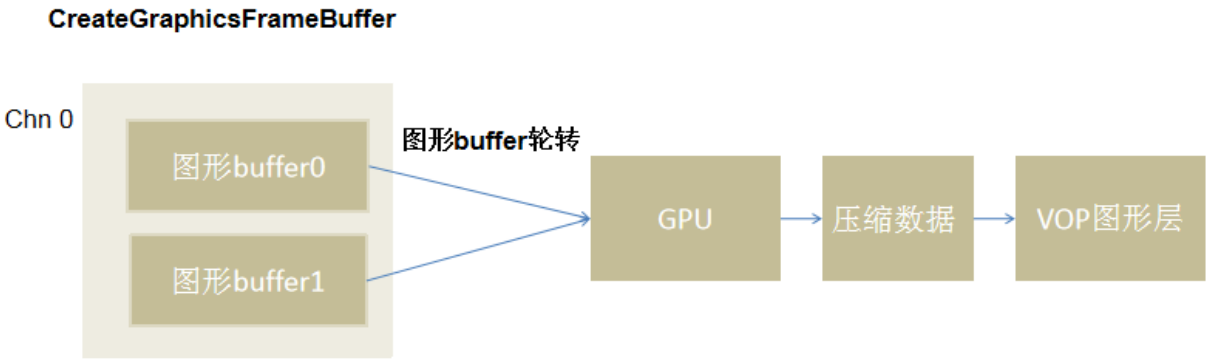
    VIDEO_FRAME_INFO_S *pstVFrame;
    RK_VOID *pMblk_G0, *pMblk_G1;
    RK_U32 count;
    ctx->stVoCfg.u32UIVoLayer = RK356X_VOP_LAYER_ESMART_0;
    /* CreateGraphicsFrameBuffer and fill pMblk_G0/pMblk_G0 */
    Sample_VO_CreateGFXData(1920, 1080, RK_FMT_BGRA5551, 0x1F, &pMblk_G0);
    Sample_VO_CreateGFXData(1920, 1080, RK_FMT_BGRA5551, 0x1F << 10, &pMblk_G1);

    pstVFrame = (VIDEO_FRAME_INFO_S *)(malloc(sizeof(VIDEO_FRAME_INFO_S)));

    while (1) {
        for (RK_U32 i = 0; i < ctx->stVoCfg.u32UIChn; i++) {
            if (i == 0)
                pMblk = pMblk_G0;
            else if (i == 1)
                pMblk = pMblk_G1;
            else
                continue;
            pstVFrame->stVFrame.pMbBlk = pMblk;
            RK_MPI_VO_SendFrame(ctx->stVoCfg.u32UIVoLayer, i, pstVFrame, 0);
            usleep(33000llu);
        }
    }
```

Примечания

- В этом примере UI использует отдельный UI-слой и два buffer отображения. RK_MPI_VO_SendFrame вращает графические buffer и поочерёдно передаёт их графическому слою, как показано ниже.
- Если при отрисовке UI формат данных BGRA5551/RGBA5551, необходимо заполнить u32FgAlpha и u32BgAlpha.



8.0.4 【Пример способа 5】

```
VO_PUB_ATTR_S VoPubAttr;
VO_VIDEO_LAYER_ATTR_S stLayerAttr;
RK_U32 s32Ret;
RK_U32 u32DispWidth = 1920;
RK_U32 u32DispHeight = 1080;
RK_U32 u32ImageWidth = 1920;
RK_U32 u32ImageHeight = 1080;

memset(&VoPubAttr, 0, sizeof(VO_PUB_ATTR_S));
memset(&stLayerAttr, 0, sizeof(VO_VIDEO_LAYER_ATTR_S));

stLayerAttr.enPixFormat                 = RK_FMT_ARGB8888;
stLayerAttr.stDispRect.s32X             = 0;
stLayerAttr.stDispRect.s32Y             = 0;
stLayerAttr.stDispRect.u32Width         = u32DispWidth;
stLayerAttr.stDispRect.u32Height        = u32DispHeight;
stLayerAttr.stImageSize.u32Width        = u32ImageWidth;
stLayerAttr.stImageSize.u32Height       = u32ImageHeight;

VO_LAYER VoLayer = RK356X_VOP_LAYER_CLUSTER0;
VO_DEV VoDev = RK356X_VO_DEV_HD0;
VoPubAttr.enIntfType = VO_INTF_HDMI;
VoPubAttr.enIntfSync = VO_OUTPUT_1080P60;

VO_FRAME_INFO_S stVFrame;
RK_MPI_VO_SetGFxMode(VO_MODE_GFX_PRE_CREATED);
stVFrame.enPixelFormat = RK_FMT_BGRA5551;
stVFrame.u32FgAlpha = 128;
stVFrame.u32BgAlpha = 0;
stVFrame.u32Width = 1920;
stVFrame.u32Height = 1080;
/* Get single GFX buffer */
RK_MPI_VO_GetGfxFrameBuffer(VoLayer, 126, &stVFrame);

/*Set Chn 126 to 4K */
VO_CHN_ATTR_S stChnAttr;
RK_MPI_VO_GetChnAttr(VoLayer, 126, &stChnAttr);
stChnAttr.stRect.u32Width = 3840;
stChnAttr.stRect.u32Height = 2160;
stChnAttr.bEnKeyColor = RK_TRUE; // Enable Key color fuction
stChnAttr.u32KeyColor = 0x8001;  // Set Key color value
/* Zoom to 4K */
RK_MPI_VO_SetChnAttr(VoLayer, 126, &stChnAttr);
memset(stVFrame.pData, 0, stVFrame.u32Size);

s32Ret = RK_MPI_VO_SetPubAttr(VoDev, &VoPubAttr);
if (s32Ret != RK_SUCCESS) {
    return RK_FAILURE;
}
```

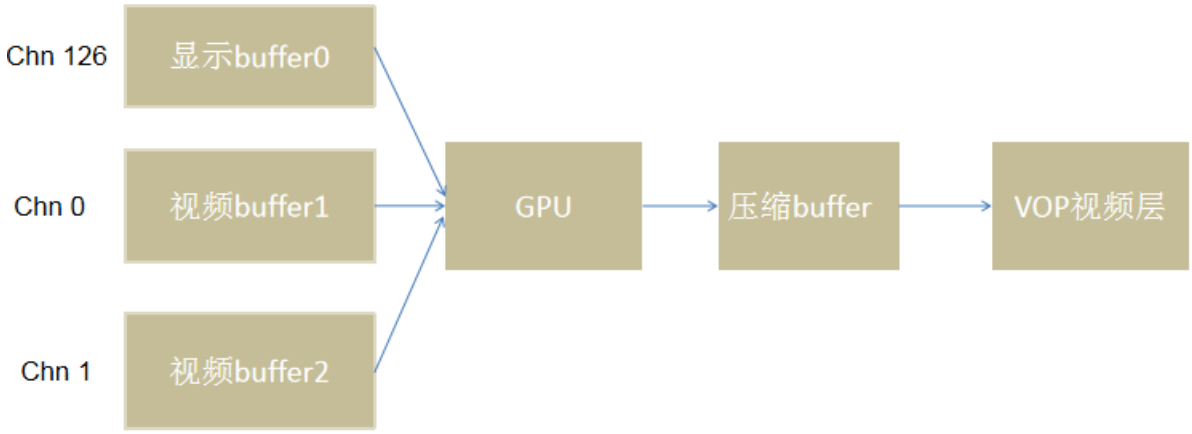
```
    s32Ret = RK_MPI_VO_Enable(VoDev);
    if (s32Ret != RK_SUCCESS) {
        return RK_FAILURE;
    }
    s32Ret = RK_MPI_VO_SetLayerAttr(VoLayer, &stLayerAttr);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;
    s32Ret = RK_MPI_VO_EnableLayer(VoLayer);
    if (s32Ret != RK_SUCCESS)
        return RK_FAILURE;
```

Связанные темы

RK_MPI_VO_GetGfxFrameBuffer

Примечания

- В способе 5 UI рисуется на видеослое: канал 126 выделяет один buffer отображения, в котором изображение UI объединяется с видео и выводится видеослоем; отдельного графического UI-слоя нет. Объединённый buffer передаётся видеослою VOP.



Примечания

- При предварительном создании UI-buffer рекомендуется использовать старшие номера каналов, например VO_MAX_CHN_NUM-1 и VO_MAX_CHN_NUM-2.
- Если разрешение UI и отображения совпадает 1:1, например исходный OSD и вывод имеют 1080P, задайте ширину и высоту VO_FRAME_INFO_S равными 1080P.
- Если разрешение UI и отображения не 1:1, например исходный OSD 1080P должен выводиться в 4K, после getGfxFrameBuffer вызовите RK_MPI_VO_SetChnAttr и задайте выходные ширину и высоту 4K; VOP увеличит OSD до 4K.

8.0.5 Пример способа 6

```
    MPI_CTX_S *ctx = (MPI_CTX_S *) (pArgs);
    RK_S32 s32Ret = 0;

    VIDEO_FRAME_INFO_S *pstVFrame;
    RK_VOID *pMblk_G0, *pMblk_G1;
    RK_U32 count;
    ctx->stVoCfg.u32VideoVoLayer = RK356X_VOP_LAYER_CLUSTER_0;
    /* CreateGraphicsFrameBuffer and fill pMblk_G0/pMblk_G0 */
    Sample_VO_CreateGFXData(1920, 1080, RK_FMT_BGRA5551, 0x1F, &pMblk_G0);
    Sample_VO_CreateGFXData(1920, 1080, RK_FMT_BGRA5551, 0x1F << 10, &pMblk_G1);

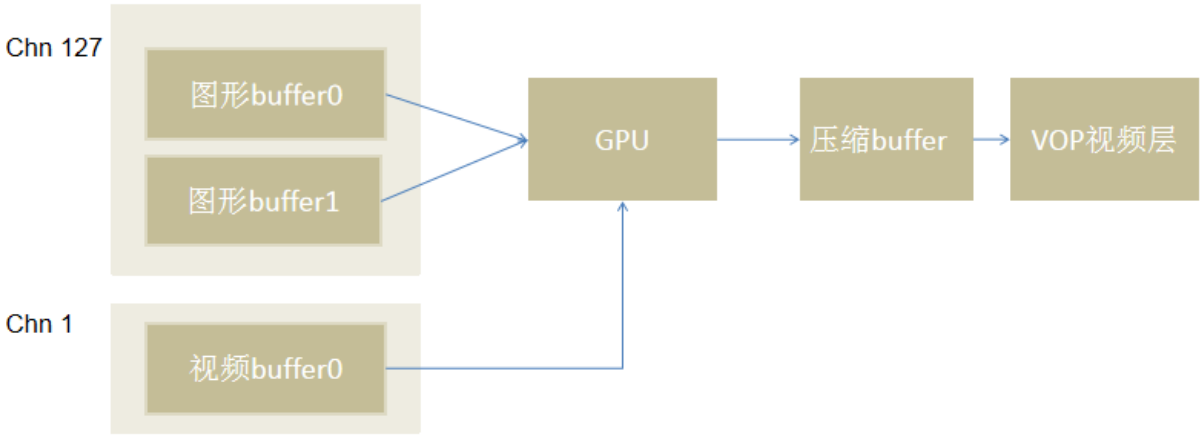
    pstVFrame = (VIDEO_FRAME_INFO_S *)(malloc(sizeof(VIDEO_FRAME_INFO_S)));

    while (1) {
        for (RK_U32 i = 0; i < ctx->stVoCfg.u32UIChn; i++) {
            if (i == 0)
                pMblk = pMblk_G0;
            else if (i == 1)
                pMblk = pMblk_G1;
```

```
            else
                continue;
            pstVFrame->stVFrame.pMbBlk = pMblk;
            RK_MPI_VO_SendFrame(ctx->stVoCfg.u32VideoVoLayer, i, pstVFrame, 0);
            usleep(33000llu);
        }
    }
```

Примечания

- В примере два выделенных buffer отображения после заполнения передаются видеослою VOP через RK_MPI_VO_SendFrame.



Примечания

- Главное отличие способов 5 и 6 — использование памяти: один buffer требует меньше памяти. На RK3568 в сценарии UI + видео рекомендуется способ 5, существенно снижающий пропускную нагрузку VOP.

9. Виртуальный слой

Виртуальный слой используется для объединения изображений без вывода на экран.

- VO поддерживает четыре виртуальных слоя, см. VO_VIR_LAYER_NAME_E.
- Для включения виртуального слоя его необходимо привязать к VoDev.
- К одному VoDev можно привязать несколько виртуальных слоёв.
- Кадр, объединённый виртуальным слоем, можно получить через RK_MPI_VO_GetLayerFrame; после использования его необходимо освободить через RK_MPI_VO_ReleaseLayerFrame.
- В качестве источника обратной записи можно выбрать виртуальный слой; тогда u32SourceId структуры VO_WBC_SOURCE_S задаётся равным ID виртуального слоя.

9.0.6 Справочный код

```
stVoPubAttr.enIntfType = VO_INTF_HDMI;
stVoPubAttr.enIntfSync = VO_OUTPUT_1080P50;
s32Ret = Sample_VO_StartDev(VoDev, &stVoPubAttr);
if (s32Ret) {
    Sample_Print("Sample_VO_StartDev s32Ret %x\n", s32Ret);
    return s32Ret;
}
VoLayer = VO_LAYER_VIRTUAL0;
s32Ret = RK_MPI_VO_BindLayer(VoLayer, VoDev, VO_LAYER_MODE_VIRTUAL);
if (s32Ret) {
    Sample_Print("RK_MPI_VO_BindLayer s32Ret %x\n", s32Ret);
    return s32Ret;
}

/* Enable Layer */
stLayerAttr.enPixFormat             = RK_FMT_BGR888;
```

```
stLayerAttr.stDispRect.s32X         = 0;
stLayerAttr.stDispRect.s32Y         = 0;
stLayerAttr.stDispRect.u32Width     = u32DispWidth;
stLayerAttr.stDispRect.u32Height    = u32DispHeight;
stLayerAttr.stImageSize.u32Width    = u32DispWidth;
stLayerAttr.stImageSize.u32Height   = u32DispHeight;
stLayerAttr.u32DispFrmRt            = 25;
Sample_VO_StartLayer(VoLayer, &stLayerAttr);

/* Get/Release Virtual Layer Frame */
s32Ret = RK_MPI_VO_GetLayerFrame(VoLayer, &stVideoFrame, 100);
if (s32Ret == 0) {
   RK_MPI_VO_ReleaseLayerFrame(VoLayer, &stVideoFrame);
} else {
   Sample_Print("GetLayer Frame Faild %x\n", s32Ret);
}

/* Enable Wbc */
stWbcSource.enSourceType = VO_WBC_SOURCE_VIRTUAL;
stWbcSource.u32SourceId = VO_LAYER_VIRTUAL0;
RK_MPI_VO_SetWbcSource(VoWbc, &stWbcSource);

stWbcAttr.enPixelFormat = RK_FMT_RGB888;
stWbcAttr.stTargetSize.u32Width = 1280;
stWbcAttr.stTargetSize.u32Height = 720;
stWbcAttr.u32FrameRate = u32Fps;
stWbcAttr.enCompressMode = COMPRESS_MODE_NONE;
RK_MPI_VO_SetWbcAttr(VoWbc, &stWbcAttr);

RK_MPI_VO_SetWbcDepth(VoWbc, 8);

s32Ret = RK_MPI_VO_EnableWbc(VoWbc);
if (s32Ret) {
    Sample_Print("RK_MPI_VO_EnableWbc s32Ret %x\n", s32Ret);
    return s32Ret;
}
```

Подсистема видеографики

Предисловие

Обзор

VGS (Video Graphics Sub-System) — видеографическая подсистема для масштабирования, поворота, наложения OSD и COVER, рисования линий и других операций над входными изображениями.

Версии продуктов

Название микросхемы	Версия ядра
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	HXM	2021-01-04	Первая версия.
V0.2.0	HXM	2021-01-23	Дополнены определения структур, добавлен интерфейс мозаики.
V0.2.1	HXM	2021-04-28	Добавлен интерфейс Crop.
V1.0.0	HXM	2021-10-08	Добавлено описание характеристик микросхем и упорядочено оформление.

Содержание

Основные понятия
Описание функций
Аппаратные характеристики
Справочник API
Типы данных
Коды ошибок VGS

1. Основные понятия

1.1 task

Выполняет над одним изображением одну или несколько операций, например OSD, масштабирование или поворот. Максимальное число task VGS по умолчанию 200.

1.2 job

VGS управляет task через job. Один job может содержать несколько task; VGS гарантирует, что они единым пакетом передаются аппаратуре в порядке добавления.

Максимальное число job VGS по умолчанию 100.

1.3 HANDLE

Дескриптор задачи, идентифицирующий job.

2. Описание функций

Функции VGS: масштабирование, вырезание, поворот, OSD, cover, линии и mosaic.

2.1 Масштабирование

VGS масштабирует изображение с увеличением или уменьшением ширины и высоты до 32 раз. Поддерживается масштабирование только компонента Y.

2.2 Поворот

VGS поддерживает поворот изображения на 0, 90, 180 и 270 градусов.

2.3 Наложение OSD

VGS поддерживает наложение битовой карты на изображение и пакетное наложение OSD.

При OSD инверсия цвета входной битовой карты не поддерживается.

При OSD входное и выходное изображения используют одну область памяти.

2.4 Наложение COVER

VGS поддерживает маскирование изображения прямоугольником или произвольным четырёхугольником.

Прямоугольный cover может быть только заполненным; произвольный четырёхугольник — заполненным или контурным.

При cover входное и выходное изображения используют одну область памяти.

2.5 Рисование линий

VGS поддерживает рисование линий на изображении, включая пакетную обработку.

При рисовании линий входное и выходное изображения используют одну область памяти.

2.6 Наложение mosaic

VGS поддерживает мозаику изображения с разными blksize и пакетную обработку mosaic.

При mosaic входное и выходное изображения используют одну область памяти.

3. Аппаратные характеристики

3.1 Аппаратные характеристики VGS RK356X

Формат данных	Выравнивание ширины, пиксели	Выравнивание высоты, пиксели	Битов на пиксель
RK_FMT_YUV420SP RK_FMT_YUV420SP_VU	16	1	8
RK_FMT_YUV420P	64	1	8
RK_FMT_YUV422SP RK_FMT_YUV422_YUYV	32	1	16
RK_FMT_YUV400SP	64	1	8
RK_FMT_YUV420SP_10BIT	32	1	10
RK_FMT_RGB565 RK_FMT_BGR565	32	1	16
RK_FMT_RGBA5551 RK_FMT_BGRA5551	32	1	16
RK_FMT_RGB888 RK_FMT_BGR888	64	1	24
RK_FMT_BGRA8888 RK_FMT_RGBA8888	16	1	32

Примечание

- VGS поддерживает масштабирование изображений размером от 64x64 до 8192x8192.

4. Справочник API

Модуль предоставляет следующие MPI:

- RK_MPI_VGS_BeginJob: запускает job.
- RK_MPI_VGS_AddScaleTask: добавляет task масштабирования в запущенный job.
- RK_MPI_VGS_AddCropTask: добавляет task обрезки в запущенный job.
- RK_MPI_VGS_AddDrawLineTask: добавляет task рисования линии в запущенный job.
- RK_MPI_VGS_AddCoverTask: добавляет task COVER в запущенный job.
- RK_MPI_VGS_AddOsdTask: добавляет task OSD в запущенный job.

- RK_MPI_VGS_AddRotationTask: добавляет task поворота в запущенный job.
- RK_MPI_VGS_AddMosaicTask: добавляет task мозаики в запущенный job.
- RK_MPI_VGS_AddDrawLineTaskArray: добавляет task пакетного рисования линий в запущенный job.
- RK_MPI_VGS_AddCoverTaskArray: добавляет task пакетного COVER в запущенный job.
- RK_MPI_VGS_AddOsdTaskArray: добавляет task пакетного OSD в запущенный job.
- RK_MPI_VGS_AddMosaicTaskArray: добавляет task пакетной мозаики в запущенный job.

4.1 RK_MPI_VGS_BeginJob

【Описание】

Запускает job.

【Синтаксис】

RK_S32 RK_MPI_VGS_BeginJob(VGS_HANDLE *phHandle);

【Параметры】

Имя параметра	Описание	Вход/выход
phHandle	Возвращаемый handle job.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Одновременно можно запустить несколько job, но HANDLE из phHandle разрешается использовать только после успешного возврата RK_MPI_VGS_BeginJob.
- phHandle не должен быть пустым или недействительным указателем.

【Пример】

```
RK_S32 s32Ret = RK_SUCCESS;
VGS_HANDLE hHandle;
VGS_TASK_ATTR_S stTask;
s32Ret = RK_MPI_VGS_BeginJob(&hHandle);
if (s32Ret != Rk_SUCCESS)
{
    VGS_ERROR_PROCESS(s32Ret);
}
s32Ret =RK_MPI_VGS_AddScaleTask(hHandle, &stTask, VGS_SCLCOEF_NORMAL);
if (s32Ret != RK_SUCCESS)
{
    RK_MPI_VGS_CancelJob(hHandle);
    VGS_ERROR_PROCESS(s32Ret);
}
s32Ret = RK_MPI_VGS_EndJob(hHandle);
if (s32Ret != RK_SUCCESS)
{
    RK_MPI_VGS_CancelJob(hHandle);
    VGS_ERROR_PROCESS(s32Ret);
}
```

4.2 RK_MPI_VGS_EndJob

【Описание】

Отправляет job на выполнение.

【Синтаксис】

RK_S32 RK_MPI_VGS_EndJob(VGS_HANDLE hHandle);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.

4.3 RK_MPI_VGS_CancelJob

【Описание】

Отменяет job.

【Синтаксис】

RK_S32 RK_MPI_VGS_CancelJob(VGS_HANDLE hHandle);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- hHandle должен обозначать запущенный job.

4.4 RK_MPI_VGS_AddScaleTask

【Описание】

Добавляет task масштабирования в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddScaleTask(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, VGS_SCLCOEF_MODE_E enScaleCoefMode);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
enScaleCoefMode	Режим коэффициентов масштабирования; сейчас не поддерживается.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- Выравнивание ширины и высоты входного изображения см. в требованиях форматов раздела аппаратных характеристик. Если ширина
- или высота входа/выхода при масштабировании не соответствует требованиям VGS, job завершается ошибкой.
- При масштабировании входное и выходное изображения могут находиться в разных областях памяти.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.5 RK_MPI_VGS_AddCropTask

【Описание】

Добавляет task обрезки в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddCropTask(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, const VGS_CROP_INFO_S *pstVgsCrop)

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
pstVgsCrop	Область обрезки.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job нельзя будет использовать повторно. этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- Выравнивание ширины и высоты входного изображения см. в требованиях форматов раздела аппаратных характеристик.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.6 RK_MPI_VGS_AddDrawLineTask

【Описание】

Добавляет task рисования линии в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddDrawLineTask(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, const VGS_DRAW_LINE_S *pstVgsDrawLine);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
pstVgsDrawLine	Указатель на конфигурацию рисования линий VGS.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job нельзя будет использовать повторно. этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- При рисовании линии вход и выход используют один buffer.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.7 RK_MPI_VGS_AddCoverTask

【Описание】

Добавляет task COVER в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddCoverTask(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, const VGS_ADD_COVER_S *pstVgsAddCover)

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
pstVgsAddCover	Указатель на конфигурацию COVER VGS.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- При COVER вход и выход используют один buffer.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.8 RK_MPI_VGS_AddOsdTask

【Описание】

Добавляет task OSD в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddOsdTask(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, const VGS_ADD_OSD_S *pstVgsAddOsd)

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
pstVgsAddOsd	Указатель на конфигурацию OSD VGS.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.

- При OSD вход и выход используют один buffer.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.9 RK_MPI_VGS_AddRotationTask

【Описание】

Добавляет task поворота в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddRotationTask(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, ROTATION_E enRotationAngle)

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
enRotationAngle	Угол поворота.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.10 RK_MPI_VGS_AddMosaicTask

【Описание】

Добавляет task мозаики в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddMosaicTask(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, const VGS_MOSAIC_S *pstVgsMosaic)

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
pstVgsMosaic	Структура конфигурации Mosaic.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- При mosaic входное и выходное изображения используют один buffer.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.11 RK_MPI_VGS_AddDrawLineTaskArray

【Описание】

Добавляет task пакетного рисования линий в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddDrawLineTaskArray(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, const VGS_DRAW_LINE_S astVgsDrawLine[], RK_U32 u32ArraySize);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
astVgsDrawLine	Массив структур конфигурации рисования линий VGS.	Вход
u32ArraySize	Число линий VGS, диапазон [1, 100].	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- При пакетном рисовании линий входное и выходное изображения используют один buffer.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.12 RK_MPI_VGS_AddCoverTaskArray

【Описание】

Добавляет task пакетного COVER в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddCoverTaskArray(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, const VGS_ADD_COVER_S astVgsAddCover[], RK_U32 u32ArraySize)

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
astVgsAddCover	Массив структур конфигурации COVER VGS.	Вход
u32ArraySize	Число Cover VGS, диапазон [1, 100].	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- При пакетном COVER входное и выходное изображения используют один buffer.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.13 RK_MPI_VGS_AddOsdTaskArray

【Описание】

Добавляет task пакетного OSD в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddOsdTaskArray(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, const VGS_ADD_OSD_S astVgsAddOsd[], RK_U32 u32ArraySize)

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
astVgsAddOsd	Массив структур конфигурации OSD VGS.	Вход
u32ArraySize	Число OSD VGS, диапазон [1, 100].	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- При пакетном OSD входное и выходное изображения используют один buffer.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

4.14 RK_MPI_VGS_AddMosaicTaskArray

【Описание】

Добавляет task пакетной мозаики в запущенный job.

【Синтаксис】

RK_S32 RK_MPI_VGS_AddMosaicTaskArray(VGS_HANDLE hHandle, const VGS_TASK_ATTR_S *pstTask, const VGS_MOSAIC_S astVgsMosaic[], RK_U32 u32ArraySize)

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE запущенного job.	Вход
pstTask	Указатель на атрибуты task VGS.	Вход
astVgsMosaic	Массив структур конфигурации mosaic VGS.	Вход
u32ArraySize	Число mosaic, диапазон [1, 100].	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок VGS.

【Примечание】

- Если интерфейс возвращает ошибку, необходимо вызвать RK_MPI_VGS_CancelJob и отменить job hHandle, иначе этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- При пакетной mosaic входное и выходное изображения используют один buffer.

【Пример】

- См. пример RK_MPI_VGS_BeginJob.

5. Типы данных

Ниже определены типы данных модуля VGS:

- VGS_HANDLE: определяет дескриптор job VGS.
- VGS_TASK_ATTR_S: определяет атрибуты task VGS.
- VGS_CROP_COORDINATE_E: определяет режим начальных координат обрезки VGS.
- VGS_CROP_INFO_S: определяет конфигурацию обрезки VGS.
- VGS_DRAW_LINE_S: определяет конфигурацию рисования линий VGS.
- VGS_COVER_TYPE_E: определяет тип COVER VGS.
- VGS_ADD_COVER_S: определяет конфигурацию COVER VGS.
- VGS_MOSAIC_S: определяет конфигурацию MOSAIC VGS.
- VGS_COLOR_REVERT_MODE_E: определяет режим инверсии цвета OSD VGS.
- VGS_OSD_REVERT_S: определяет конфигурацию инверсии OSD VGS.
- VGS_ADD_OSD_S: определяет конфигурацию OSD VGS.
- VGS_SCLCOEF_MODE_E: определяет режим коэффициентов масштабирования VGS.
- VGS_MOSAIC_BLK_SIZE_E: определяет размер блока mosaic VGS.

5.1 VGS_HANDLE

【Описание】

Определяет дескриптор job VGS.

【Определение】

typedef RK_S32 VGS_HANDLE

【Примечания】

Нет.

【Связанные типы данных и интерфейсы】

Нет.

5.2 VGS_TASK_ATTR_S

【Описание】

Определяет атрибуты task VGS.

【Определение】

```
typedef struct rkVGS_TASK_ATTR_S
{
    VIDEO_FRAME_INFO_S stImgIn;
    VIDEO_FRAME_INFO_S stImgOut;
    RK_U64 au64privateData[4];
    RK_U32 reserved;
}VGS_TASK_ATTR_S;
```

【Члены】

Имя параметра	Описание
stImgIn	Атрибуты входного изображения.
stImgOut	Атрибуты выходного изображения.
au64privateData	Частные данные task; VGS их не использует и не изменяет.
reserved	Зарезервировано.

5.3 VGS_CROP_COORDINATE_E

【Описание】

Определяет режим начальных координат обрезки VGS.

【Определение】

```
typedef enum rkVGS_CROP_COORDINATE_E {
    VGS_CROP_RATIO_COOR = 0,    /* Ratio coordinate. */
    VGS_CROP_ABS_COOR           /* Absolute coordinate. */
} VGS_CROP_COORDINATE_E;
```

【Члены】

Имя параметра	Описание
VGS_CROP_RATIO_COOR	Относительные координаты.
VGS_CROP_ABS_COOR	Абсолютные координаты.

【Примечания】 При относительных координатах начальная точка задаётся долей ширины и высоты текущего изображения и при использовании должна быть преобразована; см. VGS_CROP_INFO_S.

5.4 VGS_CROP_INFO_S

【Описание】

Определяет конфигурацию обрезки VGS.

【Определение】

```
typedef struct rkVGS_CROP_INFO_S {
    VGS_CROP_COORDINATE_E   enCropCoordinate;
    RECT_S                  stCropRect;
} VGS_CROP_INFO_S;
```

【Члены】

Имя параметра	Описание
enCropCoordinate	Режим координат начальной точки CROP.
stCropRect	Прямоугольная область CROP.

【Примечания】

- Если enCropCoordinate равен VPSS_CROP_RATIO_COOR (режим относительных координат), члены

  stCropRect следует преобразовать по формулам:

  s32X = координата x начальной точки × ширина исходного изображения / 1000. Допустимый диапазон: [0, 999]. После вычисления выполняются округление и выравнивание. Та же формула применяется для координаты y.

  u32Width = ширина области × фактическая ширина изображения / 1000. Диапазон ширины области: [1, 1000]. После вычисления выполняются округление и выравнивание. Та же формула применяется для высоты области.

5.5 VGS_DRAW_LINE_S

【Описание】

Определяет конфигурацию рисования линий VGS.

【Определение】

```
typedef struct rkVGS_DRAW_LINE_S {
    POINT_S                     stStartPoint;
    POINT_S                     stEndPoint;
    RK_U32                      u32Thick;
    RK_U32                      u32Color;
} VGS_DRAW_LINE_S;
```

【Члены】

Имя параметра	Описание
stStartPoint	Координаты начала линии.
stEndPoint	Координаты конца линии.
u32Thick	Толщина линии.
u32Color	Цвет линии в RGBA8888, диапазон [0x0, 0xFFFFFFF].

5.6 VGS_COVER_TYPE_E

【Описание】

Определяет типы COVER VGS.

【Определение】

```
typedef enum rkVGS_COVER_TYPE_E {
    COVER_RECT = 0,
    COVER_QUAD_RANGLE,
    COVER_BUTT
} VGS_COVER_TYPE_E;
```

【Члены】

Имя параметра	Описание
COVER_RECT	Прямоугольный COVER.
COVER_QUAD_RANGLE	COVER произвольным четырёхугольником.

5.7 VGS_ADD_COVER_S

【Описание】

Определяет конфигурацию COVER VGS.

【Определение】

```
typedef struct rkVGS_ADD_COVER_S {
    VGS_COVER_TYPE_E                enCoverType;
    union {
        RECT_S                      stDstRect;
        VGS_QUADRANGLE_COVER_S      stQuadRangle;
    };
    RK_U32                          u32Color;
} VGS_ADD_COVER_S;
```

【Члены】

Имя параметра	Описание
enCoverType	Тип COVER.
stDstRect	Положение, ширина и высота прямоугольного COVER.
stQuadRangle	Конфигурация COVER произвольным четырёхугольником: координаты четырёх точек и толщина контура.
u32Color	Цвет COVER в RGBA8888, диапазон [0x0, 0xFFFFFFF].

5.8 VGS_COLOR_REVERT_MODE_E

【Описание】

Определяет режим инверсии цвета OSD VGS.

【Определение】

```
ctypedef enum rkVGS_COLOR_REVERT_MODE_E {
    VGS_COLOR_REVERT_NONE = 0,
    VGS_COLOR_REVERT_RGB,
    VGS_COLOR_REVERT_ALPHA,
    VGS_COLOR_REVERT_BOTH,
    VGS_COLOR_REVERT_BUTT
} VGS_COLOR_REVERT_MODE_E;
```

【Члены】

Имя параметра	Описание
VGS_COLOR_REVERT_NONE	Без инверсии.
VGS_COLOR_REVERT_RGB	Инвертировать только RGB.
VGS_COLOR_REVERT_ALPHA	Инвертировать только alpha.
VGS_COLOR_REVERT_BOTH	Инвертировать RGB и alpha.

5.9 VGS_OSD_REVERT_S

【Описание】

Определяет конфигурацию инверсии OSD VGS.

【Определение】

```
typedef struct rkVGS_OSD_REVERT_S {
    RECT_S                      stSrcRect;
    VGS_COLOR_REVERT_MODE_E     enColorRevertMode;
} VGS_OSD_REVERT_S;
```

【Члены】

Имя параметра	Описание
stSrcRect	Начальные координаты, ширина и высота области инверсии OSD. Положение и размеры выравниваются по 2.
enColorRevertMode	Режим инверсии цвета OSD.

5.10 VGS_ADD_OSD_S

【Описание】

Определяет конфигурацию OSD VGS.

【Определение】

```
typedef struct rkVGS_ADD_OSD_S {
    MB_BLK                  pMbBlk;
    RECT_S                  stRect;
    PIXEL_FORMAT_E          enPixelFmt;
    RK_U32                  u32FgAlpha;
    RK_U32                  u32BgAlpha;
} VGS_ADD_OSD_S;
```

【Члены】

Имя параметра	Описание
stRect	Начальные координаты, ширина и высота OSD.
enPixelFmt	Формат пикселей OSD.
pMbBlk	Физический адрес изображения OSD.
u32FgAlpha	Значение alpha переднего плана OSD для RGBA551 или BGRA5551.
u32BgAlpha	Значение alpha фона OSD для RGBA551 или BGRA5551.

5.11 VGS_MOSAIC_BLK_SIZE_E

【Описание】

Перечисление размеров блока mosaic.

【Определение】

```
typedef enum rkVGS_MOSAIC_BLK_SIZE_E {
    RK_MOSAIC_BLK_SIZE_8  = 8,
    RK_MOSAIC_BLK_SIZE_16 = 16,
    RK_MOSAIC_BLK_SIZE_32 = 32,
    RK_MOSAIC_BLK_SIZE_64 = 64,
    RK_MOSAIC_BLK_SIZE_BUT
} VGS_MOSAIC_BLK_SIZE_E;
```

【Члены】

Имя параметра	Описание
RK_MOSAIC_BLK_SIZE_8	Блок Mosaic 8x8.
RK_MOSAIC_BLK_SIZE_16	Блок Mosaic 16x16.
RK_MOSAIC_BLK_SIZE_32	Блок Mosaic 32x32.
RK_MOSAIC_BLK_SIZE_64	Блок Mosaic 64x64.

5.12 VGS_MOSAIC_S

【Описание】

Определяет конфигурацию mosaic VGS.

【Определение】

```
typedef struct rkVGS_MOSAIC_S {
    VGS_MOSAIC_BLK_SIZE_E enBlkSize;
    RECT_S stDstRect;
} VGS_MOSAIC_S;
```

【Члены】

Имя параметра	Описание
enBlkSize	Размер блока mosaic.
stDstRect	Координаты прямоугольника.

6. Коды ошибок VGS

Ниже приведены коды ошибок API VGS.

Код ошибки	Макроопределение	Описание
0xA007800E	RK_ERR_VGS_BUF_EMPTY	Исчерпаны узлы job, task или node VGS.
0xA0078003	RK_ERR_VGS_ILLEGAL_PARAM	Недопустимый параметр VGS.
0xA0078006	RK_ERR_VGS_NULL_PTR	Пустой указатель во входном параметре.
0xA0078008	RK_ERR_VGS_NOT_SUPPORT	Операция не поддерживается.
0xA0078009	RK_ERR_VGS_NOT_PERMITTED	Операция не разрешена.
0xA007800D	RK_ERR_VGS_NOBUF	Не удалось выделить память.
0xA0078006	RK_ERR_VGS_NULL_PTR	Пустой указатель во входном параметре.
0xA0078010	RK_ERR_VGS_SYS_NOTREADY	Система не инициализирована.
0xA007800F	RK_ERR_VGS_BUF_FULL	Нет свободных BUF.

Обработка графики

Предисловие

Обзор

Документ описывает API и типы данных TDE.

Версии продуктов

Название микросхемы	Версия ядра
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	ZDD	2021-01-04	Первая версия.
V0.2.0	XLM	2021-01-21	Дополнено описание типов данных.
V1.0.0	HXM	2021-10-08	Добавлено описание характеристик микросхем и упорядочено оформление.
V1.1.0	HXM	2021-12-06	Добавлено описание характеристик RK3588.
V1.1.1	HXM	2022-01-05	Дополнено описание характеристик RK356X/RK3588.

Содержание

Основные понятия
Аппаратные характеристики
Справочник API
Типы данных
Коды ошибок TDE

1. Основные понятия

TDE (Two Dimensional Engine) использует аппаратный RGA для быстрой графической обработки: копирования битовых карт, заливки цветом, поворота, масштабирования, преобразования форматов, alpha-наложения и ColorKey.

1.1 Пример

```
RK_S32 s32Ret          = RK_SUCCESS;
FILE  *file            = RK_NULL;
RK_S32 u32JobTestTime  = 1;
RK_S32 u32TaskTestTime = 2;
void  *pSrcData        = RK_NULL;
RK_U32 u32taskCount    = 0;
MB_BLK srcBlk          = RK_NULL;
RK_U32 u32ImgSize      = COLOR_WIDTH * COLOR_HEIGHT * 3 / 2;
TDE_HANDLE hHandle     = 0;
TDE_SURFACE_S pstDst[TDE_MAX_TASK_NUM];
TDE_RECT_S    pstDstRect[TDE_MAX_TASK_NUM];

RK_TDE_Open();
hHandle = RK_TDE_BeginJob();
if (RK_ERR_TDE_INVALID_HANDLE == hHandle) {
    RK_LOGE("start job fail");
    return RK_FAILURE;
}
TDE_SURFACE_S pstSrc;
TDE_RECT_S    pstSrcRect;
test_tde_quick_resize_task(&pstSrc,
                           &pstSrcRect,
                           &pstDst[u32TaskIndex],
                           &pstDstRect[u32TaskIndex],
                           srcBlk, file, u32ImgSize);
s32Ret = RK_TDE_QuickResize(hHandle, &pstSrc, &pstSrcRect,
                            &pstDst[u32TaskIndex], &pstDstRect[u32TaskIndex]);
if (s32Ret != RK_SUCCESS) {
    RK_TDE_CancelJob(hHandle);
    return RK_FAILURE;
}
s32Ret = RK_TDE_EndJob(hHandle, RK_FALSE, RK_TRUE, 10);
if (s32Ret != RK_SUCCESS) {
    RK_TDE_CancelJob(hHandle);
    return RK_FAILURE;
}

RK_TDE_WaitForDone(hHandle);
RK_TDE_CancelJob(hHandle);
RK_TDE_Close();
RK_MPI_SYS_Free(srcBlk);
```

2. Аппаратные характеристики

2.1 Аппаратные характеристики TDE RK356X

Формат данных	Выравнивание ширины, пиксели	Выравнивание высоты, пиксели	Битов на пиксель
RK_FMT_YUV420SP RK_FMT_YUV420SP_VU	4	2	8
RK_FMT_YUV420p	4	2	8
RK_FMT_YUV422SP RK_FMT_YUV422_YUYV	4	2	8
RK_FMT_YUV400SP	4	2	8
RK_FMT_YUV420SP_10BIT RK_FMT_YUV422SP_10BIT	16	2	10
RK_FMT_RGB565 RK_FMT_BGR565	2	1	16
RK_FMT_RGBA5551 RK_FMT_BGRA5551	32	1	16
RK_FMT_RGB888 RK_FMT_BGR888	4	1	24
RK_FMT_BGRA8888 RK_FMT_RGBA8888	1	1	32

Примечание

- Выравнивание области TDE зависит от формата: для RGB ширина, высота и смещение не требуют выравнивания; для YUV они выравниваются по 2 пикселям.
- Перед операциями TDE убедитесь, что RK_TDE_Open() успешно вызван, иначе аппаратура недоступна.
- Кроме быстрого копирования, TDE сейчас не поддерживает сжатые входные/выходные форматы. Используйте несжатое изображение, не AFBC и не другой сжатый формат.
- TDE поддерживает масштабирование изображений размером от 64x64 до 8192x8192.

2.2 Аппаратные характеристики TDE RK3588

Формат данных	Выравнивание ширины	Выравнивание высоты	Битов на пиксель	Сжатый формат
RK_FMT_YUV420SP RK_FMT_YUV420SP_VU	16	2	8	Поддерживается
RK_FMT_YUV420p	16	2	8	Не поддерживается
RK_FMT_YUV422SP	16	2	8	Поддерживается
RK_FMT_YUV422_YUYV	16	2	8	Не поддерживается
RK_FMT_YUV400SP	16	2	8	Не поддерживается
RK_FMT_YUV420SP_10BIT RK_FMT_YUV422SP_10BIT	64	2	10	Поддерживается
RK_FMT_RGB565	8	1	16	Поддерживается
RK_FMT_BGR565	8	1	16	Не поддерживается
RK_FMT_RGB888	16	1	24	Поддерживается
RK_FMT_BGR888	16	1	24	Не поддерживается
RK_FMT_RGBA8888	4	1	32	Не поддерживается
RK_FMT_BGRA8888	4	1	32	Поддерживается
RK_FMT_RGBA5551 RK_FMT_BGRA5551	32	1	16	Поддерживается

Примечание

- Выравнивание области TDE совпадает с RK3568.
- Если вход и выход сжаты, коэффициент масштабирования ограничен 8.
- Для сжатых входа и выхода поддерживается разрешение от 128x128 до 8176x8176.

3. Справочник API

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_TDE_Open: открывает устройство TDE.
- RK_TDE_Close: закрывает устройство TDE.
- RK_TDE_BeginJob: создаёт задачу TDE.
- RK_TDE_EndJob: отправляет созданную задачу TDE.
- RK_TDE_CancelJob: отменяет задачу TDE.
- RK_TDE_WaitForDone: ожидает завершения указанной задачи.
- RK_TDE_WaitAllDone: ожидает завершения всех задач TDE.
- RK_TDE_QuickCopy: добавляет быстрое копирование в указанную задачу.
- RK_TDE_QuickResize: добавляет масштабирование растровой карты.
- RK_TDE_Bitblit: добавляет перенос растровой карты с дополнительными функциями.
- RK_TDE_QuickFill: добавляет быструю заливку.
- RK_TDE_Rotate: добавляет поворот растровой карты.
- RK_TDE_BitmapMaskBlend: добавляет перенос растровой карты Mask Blend. Маска используется для наложения переднего плана на фон.

3.1 RK_TDE_Open

【Описание】

Открывает устройство TDE.

【Синтаксис】

```
RK_S32 RK_TDE_Open(RK_VOID);
```

【Параметры】

Нет

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- Перед использованием TDE необходимо инициализировать устройство через Open.
- Повторный вызов не поддерживается и возвращает ошибку.

3.2 RK_TDE_Close

【Описание】

Закрывает устройство TDE.

【Синтаксис】

```
RK_VOID RK_TDE_Close(RK_VOID);
```

【Параметры】

Нет

【Возвращаемое значение】

Нет

【Примечание】

- RK_TDE_Open должен быть успешно вызван.

3.3 RK_TDE_BeginJob

【Описание】

Создаёт одну задачу TDE.

【Синтаксис】

```
TDE_HANDLE RK_TDE_BeginJob(RK_VOID);
```

【Параметры】

Нет

【Возвращаемое значение】

Возвращаемое значение	Описание
Дескриптор	Успех.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Проверяйте возвращаемое значение, чтобы получить действительный дескриптор задачи.
- TDE поддерживает не более 128 задач.

3.4 RK_TDE_EndJob

【Описание】 Отправляет созданную задачу TDE. Можно выбрать блокирующий или неблокирующий режим; для блокирующего задаётся тайм-аут.

- Блокирующий режим

  Функция возвращается только после завершения всех операций job TDE либо истечения тайм-аута.

- Неблокирующий режим

  Функция возвращается немедленно, не ожидая завершения job TDE.

【Синтаксис】

RK_S32 RK_TDE_EndJob(TDE_HANDLE s32Handle, RK_BOOL bSync, RK_BOOL bBlock, RK_U32 u32TimeOut);

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32Handle	Дескриптор задачи TDE.	Вход
bSync	Сейчас не используется.	Вход
bBlock	Флаг блокировки. RK_TRUE: блокирующий режим. RK_FALS: неблокирующий режим.	Ввод
u32TimeOut	Тайм-аут в мс.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Перед вызовом необходимо получить действительный дескриптор через RK_TDE_BeginJob.
- Если блокирующий вызов достигает тайм-аута, функция возвращается, но операция продолжает выполняться.

3.5 RK_TDE_CancelJob

【Описание】

Отменяет задачу TDE.

【Синтаксис】

RK_S32 RK_TDE_CancelJob(TDE_HANDLE s32Handle);

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32Handle	Дескриптор задачи TDE.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Необходимо получить действительный дескриптор через RK_TDE_BeginJob, иначе возвращаемое значение недействительно.
- Уже отправленную задачу отменить нельзя.
- После отмены задача недействительна: добавлять операции и отправлять её нельзя.

3.6 RK_TDE_WaitForDone

【Описание】

Ожидает завершения указанной задачи.

【Синтаксис】

RK_S32 RK_TDE_WaitForDone(TDE_HANDLE s32Handle);

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32Handle	Дескриптор задачи TDE.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Если интерфейс RK_TDE_END_Job используется в неблокирующем режиме или с тайм-аутом, необходимо вызвать этот интерфейс для ожидания завершения задачи. Этот интерфейс является блокирующим, и ожидает завершения указанной задачи.

3.7 RK_TDE_WaitAllDone

【Описание】

Ожидает завершения всех задач TDE.

【Синтаксис】

RK_S32 RK_TDE_WaitAllDone(RK_VOID);

【Параметры】

Нет

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Интерфейс блокирующий и ожидает завершения всех задач TDE.

3.8 RK_TDE_QuickCopy

【Описание】

Добавляет быстрое копирование в указанную задачу.

【Синтаксис】

RK_S32 RK_TDE_QuickCopy(TDE_HANDLE s32Handle,

const TDE_SURFACE_S *pstSrc,
const TDE_RECT_S *pstSrcRect,
const TDE_SURFACE_S *pstDst,
const TDE_RECT_S *pstDstRect);

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32Handle	Дескриптор задачи TDE.	Вход
pstSrc	Исходная битовая карта.	Вход
pstSrcRect	Область операции исходной карты.	Вход
pstDst	Целевая битовая карта.	Вход
pstDstRect	Область операции целевой карты.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Копирует область pstSrcRect карты с базовым адресом pstSrc в область pstDstRect памяти с целевым адресом pstDst.
- Сведения о карте задаёт TDE_SURFACE_S: ширина, высота, формат цвета и т. д.
- Область операции задаёт TDE_RECT_S: прямоугольник, участвующий в операции, включая положение и размер.
- Ширина и высота входных/выходных изображений должны соблюдать выравнивание соответствующего формата.

3.9 RK_TDE_QuickResize

【Описание】

Добавляет масштабирование растровой карты в задачу.

【Синтаксис】

RK_S32 RK_TDE_QuickResize(TDE_HANDLE s32Handle,
const TDE_SURFACE_S *pstSrc,
const TDE_RECT_S *pstSrcRect,
const TDE_SURFACE_S *pstDst,
const TDE_RECT_S *pstDstRect);

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32Handle	Дескриптор задачи TDE.	Вход
pstSrc	Исходная битовая карта.	Вход
pstSrcRect	Область операции исходной карты.	Вход
pstDst	Целевая битовая карта.	Вход
pstDstRect	Область операции целевой карты.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Масштабирует область pstSrcRect карты pstSrc до размера pstDstRect и копирует результат в область pstDstRect памяти с целевым адресом pstDst.
- Коэффициент масштабирования определяется отношением размеров pstSrcRect и pstDstRect.
- Сейчас коэффициент уменьшения и увеличения не ограничен.

3.10 RK_TDE_Bitblit

【Описание】

Добавляет перенос растровой карты с дополнительными функциями.

【Синтаксис】

```
RK_S32 RK_TDE_Bitblit(TDE_HANDLE s32Handle,
                      const TDE_SURFACE_S *pstBackGround,
                      const TDE_RECT_S *pstBackGroundRect,
                      const TDE_SURFACE_S *pstForeGround,
                      const TDE_RECT_S *pstForeGroundRect,
                      const TDE_SURFACE_S *pstDst,
                      const TDE_RECT_S *pstDstRect,
                      const TDE_OPT_S *pstOpt);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32Handle	Дескриптор задачи TDE.	Вход
pstBackGround	Фоновая битовая карта.	Вход
pstBackGroundRect	Область операции фоновой карты.	Вход
pstForeGround	Битовая карта переднего плана.	Вход
pstForeGroundRect	Область операции карты переднего плана.	Вход
pstDst	Целевая битовая карта.	Вход
pstDstRect	Область операции целевой карты.	Вход
pstOpt	Структура параметров операции.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Выполняет операцию над указанными областями карты переднего плана pstForeGround и фоновой карты pstBackGround (pstForeGroundRect и pstBackGroundRect), затем копирует результат в указанную область pstDstRect целевой карты pstDst. Область pstBackGroundRect фоновой карты должна совпадать с pstDstRect целевой карты.

- Поддерживаются поворот, обрезка, масштабирование, colorkey и наложение указанной области. ROP пока не поддерживается.
- Alpha-смешивание

  Есть два способа: выполнить Alpha-наложение переднего плана и фона по выбранной модели и вывести результат на фоновую карту; либо выполнить Alpha-наложение фона и переднего плана и вывести на целевую карту.

- Операция ColorKey

  ColorKey предварительно обрабатывает исходное изображение: у пикселей, соответствующих условию цветового ключа, alpha устанавливается в ноль; условием является прозрачное значение цвета. Затем исходное изображение смешивается с целевым по alpha. RK356x поддерживает ColorKey только для переднего плана.

3.11 RK_TDE_QuickFill

【Описание】

Добавляет быструю заливку в задачу.

【Синтаксис】

```
RK_S32 RK_TDE_QuickFill(TDE_HANDLE s32Handle,
                        TDE_SURFACE_S *pstDst,
                        TDE_RECT_S *pstDstRect,
                        RK_U32 u32FillData);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32Handle	Дескриптор задачи TDE.	Вход
pstDst	Целевая битовая карта.	Вход
pstDstRect	Область операции целевой карты.	Вход
u32FillData	Значение заливки.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Операция напрямую заполняет указанную область карты значением u32FillData.
- Для целевого формата ARGB1555/BGRA5551 биты u32FillData от старших к младшим: A (1 бит), R (5), G (5), B (5). Например, красный: u32FillData = 0x00007C00.
- Для других растровых форматов параметр u32FillData по разрядам от старших к младшим содержит A (8 бит), R (8 бит), G (8 бит), B (8 бит). Например, красный цвет: u32FillData = 0x00ff0000。
- Обработка входных данных в сжатом формате не поддерживается.

3.12 RK_TDE_Rotate

【Описание】

Добавляет в задание операцию поворота растрового изображения.

【Синтаксис】

```
RK_S32 RK_TDE_Rotate(TDE_HANDLE s32Handle,
                     TDE_SURFACE_S *pstSrc,
                     TDE_RECT_S *pstSrcRect,
                     TDE_SURFACE_S *pstDst,
                     TDE_RECT_S *pstDstRect,
```

ROTATION_E enRotateAngle);

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32Handle	Дескриптор задачи TDE.	Вход
pstSrc	Исходная битовая карта.	Вход
pstSrcRect	Область операции исходной карты.	Вход
pstDst	Целевая битовая карта.	Вход
pstDstRect	Область операции целевой карты.	Вход
enRotateAngle	Угол поворота.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки TDE.

【Примечание】

- RK_TDE_Open должен быть успешно вызван.
- Растровое изображение с базовым адресом pstSrc и областью размера, заданного pstSrcRect, поворачивается до размера pstDstRect; результат копируется в память с адресом назначения pstDst, при этом pstDstRect задаёт выходную область. Поддерживается поворот по часовой стрелке на 90, 180 и 270 градусов.

4. Типы данных

4.1 TDE_MAX_JOB_NUM

【Описание】

Определяет максимальное количество job.

【Определение】

```
#define TDE_MAX_JOB_NUM                         128
```

【Примечание】

- Нет

4.2 TDE_MAX_TASK_NUM

【Описание】

Определяет максимальное количество task.

【Определение】

```
#define TDE_MAX_TASK_NUM                        200
```

【Примечание】

- Нет

4.3 TDE_HANDLE

【Описание】

Определяет дескриптор job.

【Определение】

```
typedef RK_S32 TDE_HANDLE;
```

【Примечание】

- Нет

4.4 TDE_SURFACE_S

【Описание】

Определяет дескриптор job.

【Определение】

```
typedef struct rkTDE_SURFACE_S {
    MB_BLK pMbBlk;
    PIXEL_FORMAT_E enColorFmt;
    RK_U32 u32Height;
    RK_U32 u32Width;
    COMPRESS_MODE_E enComprocessMode;
} TDE_SURFACE_S;
```

【Члены】

Имя члена	Описание
pMbBlk	Дескриптор блока буфера.
enColorFmt	Тип формата изображения.
u32Height	Высота изображения.
u32Width	Ширина изображения.
enComprocessMode	Тип сжатия изображения.
bAlphaExt1555	Включение расширения Alpha для 1555. Действительно, когда формат растрового изображения — ARGB1555.
u8Alpha0	Значение Alpha0. Диапазон: [0, 255]. Действительно, когда формат растрового изображения — ARGB1555 и bAlphaExt1555 равно TRUE. В формате ARGB1555, если старший бит пикселя равен 0, это значение используется как alpha при альфа-смешивании.
u8Alpha1	Значение Alpha1. Диапазон: [0, 255]. Действительно, когда формат растрового изображения — ARGB1555 и bAlphaExt1555 равно TRUE. В формате ARGB1555, если старший бит пикселя равен 1, это значение используется как alpha при альфа-смешивании.

【Примечание】

- Входные и выходные изображения TDE хранятся в pMbBlk; для обоих необходимо заранее выделить корректные блоки буфера подходящего размера.
- RK3588 поддерживает настройку сжатого формата входного и выходного изображения.
- RK356X не поддерживает сжатые изображения; необходимо задать COMPRESS_MODE_NONE.

4.5 TDE_RECT_S

【Описание】

Атрибуты области операции TDE.

【Определение】

```
typedef struct rkTDE_RECT_S {
    RK_S32 s32Xpos;
    RK_S32 s32Ypos;
    RK_U32 u32Width;
    RK_U32 u32Height;
} TDE_RECT_S;
```

【Члены】

Имя члена	Описание
s32Xpos	Начальная горизонтальная координата области операции, в пикселях. Допустимый диапазон: [0, ширина растрового изображения).
s32Ypos	Начальная вертикальная координата области операции, в пикселях. Допустимый диапазон: [0, высота растрового изображения).
u32Width	Ширина области операции, в пикселях. Допустимый диапазон: (0, 8192].
u32Height	Высота области операции, в пикселях. Допустимый диапазон: (0, 8192].

【Примечание】

- Входные и выходные изображения TDE хранятся в pMbBlk; для обоих необходимо заранее выделить корректные блоки буфера подходящего размера.
- Область операции не должна выходить за границы растрового изображения; в противном случае возвращается соответствующий код ошибки TDE.

4.6 TDE_COLORKEY_MODE_E

【Описание】

Атрибуты режима TDE colorkey.

【Определение】

```
typedef enum rkTDE_COLORKEY_MODE_E {
    TDE_COLORKEY_MODE_NONE = 0,
    TDE_COLORKEY_MODE_FOREGROUND,
    TDE_COLORKEY_MODE_BACKGROUND,
    TDE_COLORKEY_MODE_BUTT
} TDE_COLORKEY_MODE_E;
```

【Члены】

Имя члена	Описание
TDE_COLORKEY_MODE_NONE	Операция colorkey не выполняется.
TDE_COLORKEY_MODE_FOREGROUND	Операция colorkey выполняется для растрового изображения переднего плана.
TDE_COLORKEY_MODE_BACKGROUND	Операция colorkey выполняется для фонового растрового изображения.
TDE_COLORKEY_MODE_BUTT	Недопустимый режим colorkey.

【Примечание】

- Нет

4.7 TDE_OPT_S

【Описание】

Структура атрибутов операции TDE.

【Определение】

```
typedef struct rkTDE_OPT_S {
    TDE_COLORKEY_MODE_E enColorKeyMode;
    RK_U32              unColorKeyValue;
    MIRROR_E            enMirror;
    TDE_RECT_S          stClipRect;
    RK_U32              u32GlobalAlpha;
} TDE_OPT_S;
```

【Члены】

Имя члена	Описание
enColorKeyMode	Режим colorkey.
unColorKeyValue	Заданное значение colorkey.
enMirror	Тип зеркального отражения.
stClipRect	Определение области clip.
u32GlobalAlpha	Глобальное значение alpha. Диапазон: [0, 255].

【Примечание】

- Нет

5. Коды ошибок TDE

Коды ошибок TDE API приведены ниже.

Код ошибки	Макроопределение	Описание
0xA00E8005	RK_ERR_TDE_DEV_NOT_OPEN	Устройство TDE не открыто, вызов API завершился ошибкой.
0xA00E8012	RK_ERR_TDE_DEV_OPEN_FAILED	Не удалось открыть устройство TDE.
0xA00E8006	RK_ERR_TDE_NULL_PTR	В параметрах передан нулевой указатель.
0xA00E800C	RK_ERR_TDE_NO_MEM	Недостаточно памяти для добавления операции.
0xA00E8001	RK_ERR_TDE_INVALID_HANDLE	Недопустимый дескриптор задания TDE.
0xA00E8003	RK_ERR_TDE_INVALID_PARA	Недопустимые параметры.
0xA00E803E	RK_ERR_TDE_NOT_ALIGNED	Начальный адрес таблицы Clut не выровнен по 4 байтам.
0xA00E803F	RK_ERR_TDE_MINIFICATION	Слишком большой коэффициент уменьшения.
0xA00E8040	RK_ERR_TDE_CLIP_AREA	Область операции не пересекается с областью clip; изображение не будет обновлено.
0xA00E8041	RK_ERR_TDE_JOB_TIMEOUT	Истекло время ожидания.
0xA00E8042	RK_ERR_TDE_UNSUPPORTED_OP_ERATION	Операция не поддерживается.
0xA00E8043	RK_ERR_TDE_QUERY_TIMEOUT	Указанное задание не завершилось до истечения времени ожидания.
0xA00E8044	RK_ERR_TDE_INTERRUPT	Ожидание завершения задания было прервано.

Аудиоподсистема

Предисловие

Обзор

Модуль AUDIO включает четыре подмодуля: аудиовход, аудиовыход, аудиокодирование и аудиодекодирование. Модули аудиовхода и аудиовыхода управляют аудиоинтерфейсами микросхем RK, обеспечивая ввод и вывод звука. Модули кодирования и декодирования содержат функции аудиокодеков форматов g711a, g711u, g722, g726 и других, а также поддерживают внешнюю регистрацию кодеков.

Версии продуктов

Название микросхемы	Версия ядра
RV1126/RV1109	4.19
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Версия Автор Дата		изменения	Описание изменения
V1.0.0	ZDD	2021-08-24	Первая версия.
V1.1.0	HH	2021-12-28	Добавлено описание интерфейсов регистрации и отмены регистрации кодеков.
V1.2.0	ZDD	2022-02-18	Добавлен интерфейс настройки независимого вывода канала AO.
v1.3.0	LMX	2023-02-07	Добавлено обнаружение звукового события сигнализации зуммера (BUZ) и разбития стекла (GBS), обновлены интерфейсы и описания функций AED и SED.
v1.3.1	LMX	2023-03-01	Добавлен параметр получения соответствующего уровня громкости в функции обнаружения звуковых событий (AED).
v1.4.0	Линь Минь Сюй	2023-03-03	Добавлен интерфейс ввода аудиоданных.

Содержание

Обзор

Описание функций

Справочник API

Типы данных

Коды ошибок AUDIO

Приложение A: описание API регистрации декодеров/кодеров

1. Обзор

Модуль AUDIO включает четыре подмодуля: аудиовход, аудиовыход, аудиокодирование и аудиодекодирование. Модули аудиовхода и аудиовыхода управляют аудиоинтерфейсами микросхем RK, обеспечивая ввод и вывод звука. Модули кодирования и декодирования содержат функции аудиокодеков форматов g711a, g711u, g722, g726 и других, а также поддерживают внешнюю регистрацию кодеков.

Примечание: если заказчику требуется использование патентов формата AAC, необходимо получить разрешение правообладателя и уплатить лицензионный сбор.

2. Описание функций

2.1 Аудиовход и аудиовыход

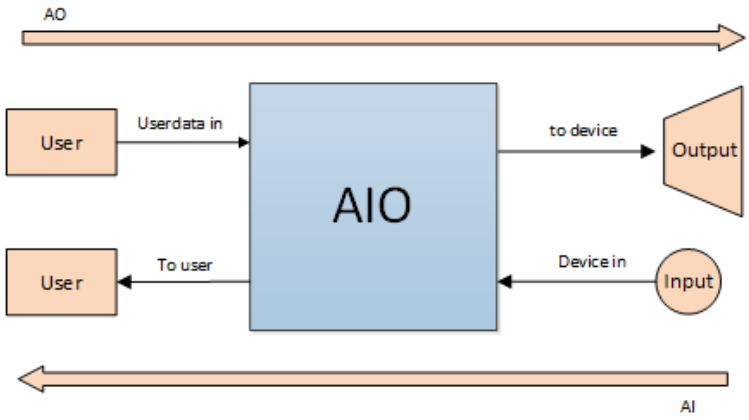
2.1.1 Интерфейсы аудиовхода и аудиовыхода

Интерфейс аудиовхода/выхода сокращённо называется AIO (Audio Input/Output). Он используется для взаимодействия с аудиосистемами Alsa (платформа Linux), TinyAlsa и AudioTrack/AudioRecord (платформа Android), обеспечивая запись и воспроизведение звука.

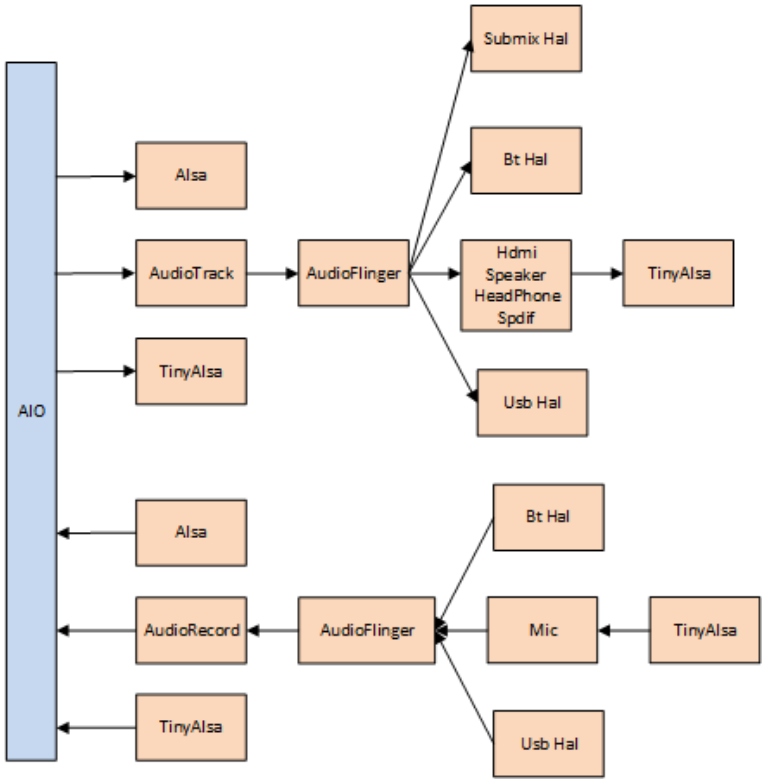
Интерфейсы AIO делятся на два типа: только вход и только выход.

- **Тип входа также называется AI (Audio Input).**
- **Тип выхода также называется AO (Audio Output).**

Интерфейс AIO показан на рисунке ниже; стрелки указывают направление потока данных.



AIO поддерживает взаимодействие с аудиосистемами платформ Linux и Android (только Android 10 и выше), обеспечивая запись и воспроизведение звука.



2.1.2 Устройства AIO

Программная абстракция функции входа аудиоинтерфейса называется устройством AI; программная абстракция функции выхода аудиоинтерфейса называется устройством AO. Для каждого входного или выходного интерфейса программа устанавливает соответствие с устройством AI или AO согласно поддерживаемой функции. Например: AI0 поддерживает только аудиовход, поэтому AI0 сопоставляется с AiDev0; AO0 поддерживает только аудиовыход, поэтому AO0 сопоставляется с AoDev0.

Номер устройства AIO — абстрактное понятие программного уровня. Обычно нумерация начинается с 0: AoDev0, AoDev1..., AiDev0, AiDev1... Как правило, одному номеру устройства AIO соответствует одна звуковая карта аудиодрайвера. Ниже приведён пример подключённых звуковых карт; фактический набор у заказчика может отличаться.

```
rk356x_box:/ $ cat proc/asound/cards
0 [rockchiphdmi   ]: rockchip_hdmi - rockchip,hdmi
                     rockchip,hdmi
1 [rockchiprk809co]: rockchip_rk809- - rockchip,rk809-codec
                     rockchip,rk809-codec
2 [ROCKCHIPSPDIF  ]: ROCKCHIP_SPDIF - ROCKCHIP,SPDIF
                     ROCKCHIP,SPDIF
```

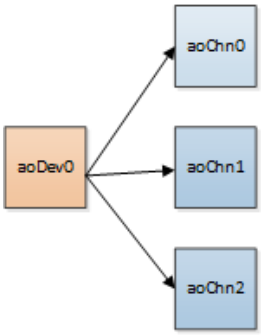
На снимке выше определены три звуковые карты. Карта с номером 0 — HDMI и поддерживает только воспроизведение; карта с номером 1 соответствует RK809 Codec и поддерживает воспроизведение и запись; карта с номером 2 — SPDIF и поддерживает только воспроизведение.

Следовательно, для изделия с такой конфигурацией можно определить три устройства AO и одно устройство AI. Например, AoDev0, AoDev1 и AoDev2 соответствуют трактам воспроизведения HDMI, RK809 и SPDIF, а AiDev0 — тракту записи RK809. Следует подчеркнуть, что номер устройства AIO не обязан однозначно соответствовать номеру звуковой карты (Audio Codec): устройство 0 (AoDev0) не обязательно выводит на звуковую карту 0; оно также может выводить на карту 1 или 2. Для удобства понимания рекомендуется прямое соответствие. Соответствие устройств AIO фактически используемым звуковым картам описано в разделе AUDIO документа FAQ.

2.1.3 Каналы AIO

Канал AIO — это входной/выходной аудиопоток устройства AIO; одному аудиопотоку соответствует один канал.

- **Для AO на одном устройстве можно создать несколько каналов (аудиопотоков). Данные нескольких каналов смешиваются в один аудиопоток и выводятся на звуковую карту. В настоящее время тракт AO поддерживает многоканальное микширование только через Alsa (Linux) и AudioFlinger (Android); TinyAlsa многоканальное микширование не поддерживает.**



- Для AI одно устройство поддерживает только один канал.

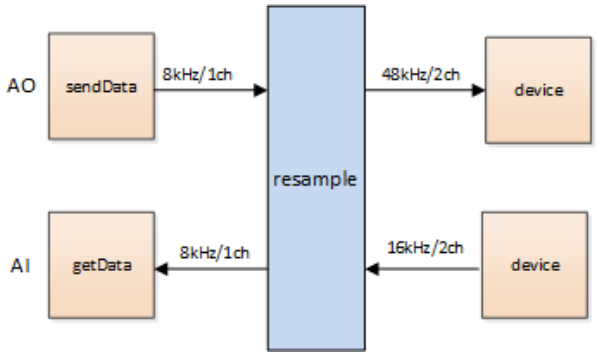
2.1.4 Передискретизация

Передискретизация — преобразование частоты дискретизации (samplerate), количества каналов (channels) и формата (format) аудиоданных. Модуль AIO поддерживает передискретизацию аудиоданных. Если включена передискретизация AI, внутри AI данные преобразуются согласно настройкам, а приложение получает их вызовом RK_MPI_AI_GetFrame

Аналогично, при включённой передискретизации AO пользователь передаёт аудиоданные через RK_MPI_AO_SendFrame, внутри AO

они преобразуются и затем воспроизводятся звуковой картой.

- Передискретизация AO главным образом преобразует входные данные в формат, поддерживаемый звуковой картой. На рисунке ниже AO получает данные (8 кГц, 1 канал), а звуковая карта поддерживает только (48 кГц, 2 канала). В этом случае можно включить передискретизацию AO и преобразовать (8 кГц, 1 канал) в поддерживаемые картой (48 кГц, 2 канала).
- Передискретизация AI главным образом преобразует данные, захваченные звуковой картой, в формат, необходимый приложению. На рисунке ниже карта записи поддерживает только (16 кГц, 2 канала), а приложению нужны данные (8 кГц, 1 канал). После включения передискретизации AI данные будут преобразованы в требуемый формат.
- Модуль AIO проверяет частоту дискретизации, количество каналов, формат и другие параметры каждого входного/выходного кадра. Если параметры входных данных совпадают с настройками преобразования, передискретизация не выполняется, и данные сразу передаются следующему этапу. Например, AO получает данные (48 кГц, 2 канала), передискретизация AO включена и её выходные параметры также заданы как (48 кГц, 2 канала). При прохождении данных (48 кГц, 2 канала) через обработку они не преобразуются, а непосредственно передаются следующему после передискретизации этапу. Поэтому даже при включённой передискретизации AO, если условия выполнены (то есть выходные параметры передискретизации совпадают с параметрами входных данных), ресурсы CPU не расходуются на преобразование. Поэтому рекомендуется всегда включать функцию передискретизации AIO.



2.2 Аудиокодирование и аудиодекодирование

2.2.1 Процесс аудиокодирования и декодирования

rockit поддерживает кодеки g711a, g711u, g722 и g726. Аудиокодирование и декодирование выполняются программно на CPU. Поддерживается интерфейс привязки модуля SYS: канал AI можно привязать к каналу AENC для записи с кодированием; канал ADEC можно привязать к каналу AO для декодирования и воспроизведения.

2.2.2 Протоколы аудиокодирования и декодирования

Поддерживаемые внутри rockit протоколы аудиокодирования и декодирования приведены в таблице ниже.

Протокол	Частота дискретизации	Количество каналов	Битрейт （kbps）	Кодовое слово (bits)	Коэффициент сжатия	Описание
g711a	8000 16000	1	64 128	8	16:8	эффективность сжатия. В Европе и других регионах преимущественно используется кодирование A-law. Преимущества: наилучшее качество речи, низкая нагрузка CPU, широкая поддержка. Недостаток: низкая
g711u	8000 16000	1	64 128	8	16:8	эффективность сжатия. В Северной Америке и Японии обычно используется кодирование μ-law. Преимущества: наилучшее качество речи, низкая нагрузка CPU, широкая поддержка. Недостаток: низкая
g726	8000	1	40 32 24 16	5 4 3 2	16:2 16:3 16:4 16:5	Для кодека g726 необходимо задавать кодовое слово (codeword), соответствующее битрейту. Кодек g726 сжимает линейные данные 128 кбит/с (PCM-данные 64 кбит/с) до 16, 24, 32 или 40 кбит/с; коэффициенты сжатия соответственно равны 8:1, 16:3, 4:1 и 16:5, а длина кодового слова — 2, 3, 4 и 5 бит. Чем выше коэффициент сжатия, тем ниже битрейт и хуже качество. Чаще всего используется 32 кбит/с, то есть кодовое слово 4 бита.
g722	16000	1	64	8	16:8	Преимущества g722 — очень низкая задержка и частота ошибок передачи битов.

2.3 Устройства аудиовхода/выхода и встроенный codec

Разным заказчикам требуются разные типы изделий и драйверы звуковых карт. Фактическая конфигурация должна быть адаптирована; настройка ведущего/ведомого режима и звуковой карты выполняется изменением dts ядра. При разработке обратитесь за консультацией к инженеру по аудиодрайверам.

3. Справочник API

3.1 Аудиовход

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_AI_SetPubAttr: задаёт атрибуты устройства AI.
- RK_MPI_AI_GetPubAttr: получает атрибуты устройства AI.
- RK_MPI_AI_Enable: включает устройство AI.
- RK_MPI_AI_Disable: отключает устройство AI.
- RK_MPI_AI_EnableChn: включает канал AI.
- RK_MPI_AI_DisableChn: отключает канал AI.
- RK_MPI_AI_EnableDataRead: включает режим чтения данных AI.
- RK_MPI_AI_DisableDataRead: отключает режим чтения данных AI.
- RK_MPI_AI_SendFrame: отправляет аудиокадр.
- RK_MPI_AI_GetFrame: получает аудиокадр.
- RK_MPI_AI_EnableReSmp: включает передискретизацию AI.
- RK_MPI_AI_DisableReSmp: отключает передискретизацию AI.
- RK_MPI_AI_ReleaseFrame: освобождает аудиокадр.
- RK_MPI_AI_SetChnParamt: получает параметры канала AI.
- RK_MPI_AI_GetChnParam: получает параметры канала AI.
- RK_MPI_AI_SetTrackMode: задаёт режим каналов.
- RK_MPI_AI_GetTrackMode: получает режим каналов.
- RK_MPI_AI_QueryFileStatus: проверяет, находится ли канал аудиовыхода в режиме сохранения в файл.
- RK_MPI_AI_SaveFile: включает сохранение аудиовыхода в файл.
- RK_MPI_AI_ClrPubAttr: очищает атрибуты устройства AI.
- RK_MPI_AI_EnableVqe: включает функции улучшения качества звука AI.
- RK_MPI_AI_DisableVqe: отключает функции улучшения качества звука AI.
- RK_MPI_AI_SetVqeAttr: задаёт атрибуты функций улучшения качества звука AI.
- RK_MPI_AI_GetVqeAttr: получает атрибуты функций улучшения качества звука AI.
- RK_MPI_AI_EnableAed: включает функции обнаружения звуковых событий AI.

- RK_MPI_AI_DisableAed: отключает функции обнаружения звуковых событий AI.
- RK_MPI_AI_SetAedAttr: задаёт атрибуты функции обнаружения звуковых событий AI.
- RK_MPI_AI_GetAedAttr: получает атрибуты функции обнаружения звуковых событий AI.
- RK_MPI_AI_GetAedResult: получает результат обнаружения звуковых событий AI.
- RK_MPI_AI_EnableBcd: включает функции обнаружения плача ребёнка AI.
- RK_MPI_AI_DisableBcd: отключает функции обнаружения плача ребёнка AI.
- RK_MPI_AI_SetBcdAttr: задаёт атрибуты функции обнаружения плача ребёнка AI.
- RK_MPI_AI_GetBcdAttr: получает атрибуты функции обнаружения плача ребёнка AI.
- RK_MPI_AI_GetBcdResult: получает результат обнаружения плача ребёнка AI.
- RK_MPI_AI_EnableBuz: включает функции обнаружения сигнала зуммера AI.
- RK_MPI_AI_DisableBuz: отключает функции обнаружения сигнала зуммера AI.
- RK_MPI_AI_SetBuzAttr: задаёт атрибуты функции обнаружения сигнала зуммера AI.
- RK_MPI_AI_GetBuzAttr: получает атрибуты функции обнаружения сигнала зуммера AI.
- RK_MPI_AI_GetBuzResult: получает результат обнаружения сигнала зуммера AI.
- RK_MPI_AI_EnableGbs: включает функции обнаружения разбития стекла AI.
- RK_MPI_AI_DisableGbs: отключает функции обнаружения разбития стекла AI.
- RK_MPI_AI_SetGbsAttr: задаёт атрибуты функции обнаружения разбития стекла AI.
- RK_MPI_AI_GetGbsAttr: получает атрибуты функции обнаружения разбития стекла AI.
- RK_MPI_AI_GetGbsResult: получает результат обнаружения разбития стекла AI.

3.2 RK_MPI_AI_SetPubAttr

【Описание】

Задаёт атрибуты устройства AI.

【Синтаксис】

RK_S32 RK_MPI_AI_SetPubAttr(AUDIO_DEV AiDevId, const AIO_ATTR_S *pstAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
pstAttr	Указатель на атрибуты устройства AI.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】 Атрибуты аудиовхода включают параметры выходных данных и входного устройства (звуковой карты): частоту дискретизации, количество каналов, разрядность, размер buffer, количество отсчётов в кадре и т. д.

- Перед настройкой атрибутов звуковой карты необходимо убедиться, что устройство AI, соответствующее AiDevId, отключено. Если оно включено, сначала отключите звуковую карту AI.
- Частота дискретизации Частота дискретизации — количество отсчётов в секунду. При настройке частоты звуковой карты необходимо убедиться, что устройство поддерживает требуемое значение.
- Количество каналов Количество каналов функции AI текущего входного устройства. Оно должно соответствовать конфигурации подключённого Audio Codec. Например, rk809-codec поддерживает запись только двух каналов.

3.3 RK_MPI_AI_GetPubAttr

【Описание】

Получает атрибуты устройства AI.

【Синтаксис】

RK_S32 RK_MPI_AI_GetPubAttr(AUDIO_DEV AiDevId, AIO_ATTR_S *pstAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
pstAttr	Указатель на атрибуты устройства AI.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Возвращаются атрибуты предыдущей конфигурации.
- Если атрибуты никогда не настраивались, возвращается ошибка «атрибуты не настроены».

3.4 RK_MPI_AI_Enable

【Описание】

Включает устройство AI.

【Синтаксис】

RK_S32 RK_MPI_AI_Enable(AUDIO_DEV AiDevId);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Перед включением необходимо настроить атрибуты устройства AI, иначе возвращается ошибка «атрибуты не настроены».
- Если устройство AI уже включено, повторный вызов возвращает ошибку «используется».

3.5 RK_MPI_AI_Disable

【Описание】

Отключает устройство AI.

【Синтаксис】

RK_S32 RK_MPI_AI_Disable(AUDIO_DEV AiDevId);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Если устройство AI не включено, возвращается код ошибки AUDIO «не включено».
- Перед отключением устройства AI необходимо отключить все его каналы AI.
- После завершения работы со звуковой картой её необходимо закрыть.

3.6 RK_MPI_AI_EnableChn

【Описание】

Включает канал AI.

【Синтаксис】

RK_S32 RK_MPI_AI_EnableChn(AUDIO_DEV AiDevId, AI_CHN AiChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Перед включением канала AI необходимо включить соответствующее устройство AI, иначе возвращается код ошибки AUDIO «устройство не запущено».

3.7 RK_MPI_AI_DisableChn

【Описание】

Отключает канал AI.

【Синтаксис】

RK_S32 RK_MPI_AI_DisableChn(AUDIO_DEV AiDevId, AI_CHN AiChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

Примечание

- После завершения работы с каналом необходимо вызвать функцию его отключения.

3.8 RK_MPI_AI_EnableDataRead

Описание

Включает режим чтения данных AI.

Синтаксис

RK_S32 RK_MPI_AI_EnableDataRead(AUDIO_DEV AiDevId, AI_CHN AiChn);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

Примечание

- Перед включением режима чтения данных AI необходимо включить соответствующее устройство AI, иначе возвращается код ошибки AUDIO «устройство не запущено».

3.9 RK_MPI_AI_DisableDataRead

Описание

Отключает режим чтения данных AI.

Синтаксис

RK_S32 RK_MPI_AI_DisableDataRead(AUDIO_DEV AiDevId, AI_CHN AiChn);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- После завершения использования режима чтения данных необходимо отключить его.

3.10 RK_MPI_AI_SendFrame

【Описание】

Отправляет аудиокадр.

【Синтаксис】

RK_S32 RK_MPI_AI_SendFrame(AUDIO_DEV AiDevId, AI_CHN AiChn, AUDIO_FRAME_S *pstFrm, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstFrm	Указатель на структуру аудиокадра.	Выход
s32MilliSec	Время ожидания получения данных: -1 означает блокирующий режим; >=0 означает неблокирующий режим и задаёт время ожидания чтения данных s32MilliSec (мс).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Если s32MilliSec равно -1, используется блокирующая отправка: интерфейс блокируется до успешной передачи данных. Значение больше либо равно 0 задаёт время ожидания отправки в неблокирующем режиме (мс). Если передача завершается в пределах указанного времени, возвращается успех; иначе возвращается ошибка тайм-аута.
- Перед отправкой аудиокадра необходимо включить соответствующий канал AI и звуковую карту.

3.11 RK_MPI_AI_GetFrame

【Описание】

Получает аудиокадр.

【Синтаксис】

RK_S32 RK_MPI_AI_GetFrame(AUDIO_DEV AiDevId, AI_CHN AiChn, AUDIO_FRAME_S *pstFrm, AEC_FRAME_S *pstAecFrm, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstFrm	Указатель на структуру аудиокадра.	Выход
pstAecFrm	Указатель на структуру опорного кадра эхоподавления.	Выход
s32MilliSec	Время ожидания получения данных: -1 означает блокирующий режим; >=0 означает неблокирующий режим и задаёт время ожидания чтения данных s32MilliSec (мс).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- В настоящее время функция эхоподавления AI не реализована; pstAecFrm следует задать равным NULL.
- Если s32MilliSec равно -1, чтение выполняется в блокирующем режиме: интерфейс блокируется до успешного получения данных либо до выхода/отключения текущего канала. Значение больше либо равно 0 задаёт время ожидания чтения в неблокирующем режиме (мс). Если за s32MilliSec миллисекунд данные не получены, возвращается ошибка тайм-аута.
- Перед получением аудиокадра необходимо включить соответствующий канал AI и звуковую карту.

3.12 RK_MPI_AI_ReleaseFrame

【Описание】

Освобождает аудиокадр.

【Синтаксис】

RK_S32 RK_MPI_AI_ReleaseFrame(AUDIO_DEV AiDevId, AI_CHN AiChn, const AUDIO_FRAME_S *pstFrm, const AEC_FRAME_S *pstAecFrm);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstFrm	Указатель на структуру аудиокадра.	Вход
pstAecFrm	Указатель на структуру опорного кадра эхоподавления.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Если освобождать опорный кадр эхоподавления не требуется, задайте pstAecFrm равным NULL.

3.13 RK_MPI_AI_SetChnParam

【Описание】

Задаёт параметры канала AI.

【Синтаксис】

RK_S32 RK_MPI_AI_SetChnParam(AUDIO_DEV AiDevId, AI_CHN AiChn, const AI_CHN_PARAM_S *pstChnParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstChnParam	Параметры аудиоканала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- В настоящее время параметры канала содержат только одно поле — количество аудиокадров, буферизуемых для получения пользователем. Значение по умолчанию: 4.

3.14 RK_MPI_AI_GetChnParam

【Описание】

Получает параметры канала AI.

【Синтаксис】

RK_S32 RK_MPI_AI_GetChnParam(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_CHN_PARAM_S *pstChnParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstChnParam	Параметры аудиоканала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Нет.

3.15 RK_MPI_AI_EnableReSmp

【Описание】

Включает передискретизацию AI.

【Синтаксис】

RK_S32 RK_MPI_AI_EnableReSmp(AUDIO_DEV AiDevId, AI_CHN AiChn, AUDIO_SAMPLE_RATE_E enOutSampleRate);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
enOutSampleRate	Выходная частота передискретизации аудио.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Этот интерфейс следует вызывать для включения передискретизации после включения канала AI.
- Допускается повторное включение передискретизации, но новая входная частота должна отличаться от ранее заданной.
- Если после отключения канал AI включается повторно и требуется передискретизация, необходимо снова вызвать этот интерфейс.
- Входная частота передискретизации AI — это частота считываемых данных.
- Настоятельно рекомендуется включать этот интерфейс: внутренний подключаемый модуль проверяет параметры и при их совпадении передаёт данные напрямую в режиме bypass.

【Пример】

Пример настройки передискретизации AI с 32 кГц записи до 8 кГц:

```
/* dev attr of ai */
aiAttr.soundCard.channels = 2;
aiAttr.soundCard.sampleRate = 32000;
aiAttr.soundCard.bitWidth = AUDIO_BIT_WIDTH_16;

aiAttr.enBitwidth = AUDIO_BIT_WIDTH_16;
aiAttr.enSamplerate = AUDIO_SAMPLE_RATE_8000;
aiAttr.enSoundmode = AUDIO_SOUND_MODE_STEREO;
aiAttr.u32FrmNum = 4;
aiAttr.u32PtNumPerFrm = 1024;
RK_MPI_AI_EnableReSmp(AiDev, AiChn, AUDIO_SAMPLE_RATE_8000);
```

3.16 RK_MPI_AI_DisableReSmp

【Описание】

Отключает передискретизацию AI.

【Синтаксис】

RK_S32 RK_MPI_AI_DisableReSmp(AUDIO_DEV AiDevId, AI_CHN AiChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовыхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Если функция передискретизации AI больше не используется, её следует отключить этим интерфейсом.

3.17 RK_MPI_AI_SetTrackMode

【Описание】

Задаёт режим каналов AI.

【Синтаксис】

RK_S32 RK_MPI_AI_SetTrackMode(AUDIO_DEV AiDevId, AUDIO_TRACK_MODE_E enTrackMode);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
enTrackMode	Режим аудиоканалов.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения устройства AI.

3.18 RK_MPI_AI_GetTrackMode

【Описание】

Получает режим каналов AI.

【Синтаксис】

RK_S32 RK_MPI_AI_GetTrackMode(AUDIO_DEV AiDevId, AUDIO_TRACK_MODE_E *penTrackMode);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
penTrackMode	Указатель на режим каналов аудиовхода.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения устройства AI.

3.19 RK_MPI_AI_ClrPubAttr

【Описание】

Очищает публичные атрибуты.

【Синтаксис】

RK_S32 RK_MPI_AI_ClrPubAttr(AUDIO_DEV AiDevId);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Перед очисткой атрибутов устройства его необходимо остановить.

3.20 RK_MPI_AI_SaveFile

【Описание】

Включает сохранение аудиовхода в файл.

【Синтаксис】

RK_S32 RK_MPI_AI_SaveFile(AUDIO_DEV AiDevId, AI_CHN AiChn, const AUDIO_SAVE_FILE_INFO_S *pstSaveFileInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовыхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstSaveFileInfo	Указатель на структуру атрибутов сохранения аудио в файл.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Этот интерфейс применяется только в непривязанном режиме AI для сохранения PCM-файла, записанного AI со звуковой карты.

3.21 RK_MPI_AI_QueryFileStatus

【Описание】

Проверяет, сохраняет ли канал аудиовхода данные в файл.

【Синтаксис】

RK_S32 RK_MPI_AI_QueryFileStatus(AUDIO_DEV AiDevId, AI_CHN AiChn, AUDIO_FILE_STATUS_S* pstFileStatus);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstFileStatus	Указатель на структуру состояния.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Этот интерфейс проверяет, находится ли канал аудиовхода в режиме сохранения в файл. После вызова пользователем RK_MPI_AI_SaveFile можно вызвать этот интерфейс и проверить, достиг ли файл заданного размера. Если поле bSaving структуры pstFileStatus равно RK_TRUE, заданный размер ещё не достигнут; если RK_FALSE — размер достигнут.

3.22 RK_MPI_AI_EnableVqe

【Описание】

Включает функции улучшения качества звука AI.

【Синтаксис】

RK_S32 RK_MPI_AI_EnableVqe(AUDIO_DEV AiDevId, AI_CHN AiChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.23 RK_MPI_AI_DisableVqe

【Описание】

Отключает функции улучшения качества звука AI.

【Синтаксис】

RK_S32 RK_MPI_AI_DisableVqe(AUDIO_DEV AiDevId, AI_CHN AiChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.24 RK_MPI_AI_SetVqeAttr

【Описание】

Задаёт атрибуты улучшения качества звука AI.

【Синтаксис】

RK_S32 RK_MPI_AI_SetVqeAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, AUDIO_DEV AoDevId, AO_CHN AoChn, const AI_VQE_CONFIG_S *pstVqeConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
AoDevId	Номер устройства AO, используемого для эхоподавления. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала AO, используемого для эхоподавления. Диапазон: [0, AO_MAX_CHN_NUM).	Вход
pstVqeConfig	Указатель на структуру атрибутов VQE.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- В поле aCfgFile структуры pstVqeConfig необходимо указать путь к JSON-файлу конфигурации AI VQE.

3.25 RK_MPI_AI_GetVqeAttr

【Описание】

Получает атрибуты улучшения качества звука AI.

【Синтаксис】

RK_S32 RK_MPI_AI_GetVqeAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_VQE_CONFIG_S *pstVqeConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstVqeConfig	Указатель на структуру атрибутов VQE.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.26 RK_MPI_AI_EnableAed

Описание

Включает функции улучшения качества звука AI.

Синтаксис

RK_S32 RK_MPI_AI_EnableAed(AUDIO_DEV AiDevId, AI_CHN AiChn);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.27 RK_MPI_AI_DisableAed

Описание

Отключает функции улучшения качества звука AI.

Синтаксис

RK_S32 RK_MPI_AI_DisableAed(AUDIO_DEV AiDevId, AI_CHN AiChn);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.28 RK_MPI_AI_SetAedAttr

【Описание】

Задаёт атрибуты улучшения качества звука AI.

【Синтаксис】

RK_S32 RK_MPI_AI_SetAedAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, const AI_AED_CONFIG_S *pstAedConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstAedConfig	Указатель на структуру атрибутов AED.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.29 RK_MPI_AI_GetAedAttr

【Описание】

Получает атрибуты улучшения качества звука AI.

【Синтаксис】

RK_S32 RK_MPI_AI_GetAedAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_AED_CONFIG_S *pstAedConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstAedConfig	Указатель на структуру атрибутов AED.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.30 RK_MPI_AI_GetAedResult

Описание

Получает атрибуты улучшения качества звука AI.

Синтаксис

RK_S32 RK_MPI_AI_GetAedResult(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_AED_RESULT_S *pstAedResult);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstAedResult	Указатель на структуру результата обнаружения события AED.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.31 RK_MPI_AI_EnableBcd

Описание

Включает функции обнаружения плача ребёнка AI.

Синтаксис

RK_S32 RK_MPI_AI_EnableBcd(AUDIO_DEV AiDevId, AI_CHN AiChn);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.32 RK_MPI_AI_DisableBcd

Описание

Отключает функции обнаружения плача ребёнка AI.

Синтаксис

RK_S32 RK_MPI_AI_DisableBcd(AUDIO_DEV AiDevId, AI_CHN AiChn);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.33 RK_MPI_AI_SetBcdAttr

Описание

Задаёт атрибуты обнаружения плача ребёнка AI.

Синтаксис

RK_S32 RK_MPI_AI_SetBcdAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, const AI_BCD_CONFIG_S *pstBcdConfig);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstBcdConfig	Указатель на структуру атрибутов BCD.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.34 RK_MPI_AI_GetBcdAttr

【Описание】

Получает атрибуты обнаружения плача ребёнка AI.

【Синтаксис】

RK_S32 RK_MPI_AI_GetBcdAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_BCD_CONFIG_S *pstBcdConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstBcdConfig	Указатель на структуру атрибутов BCD.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.35 RK_MPI_AI_GetBcdResult

【Описание】

Получает результат обнаружения плача ребёнка AI.

【Синтаксис】

RK_S32 RK_MPI_AI_GetBcdResult(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_BCD_RESULT_S *pstBcdResult);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstBcdResult	Указатель на структуру результата обнаружения события BCD.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.36 RK_MPI_AI_EnableBuz

Описание

Включает атрибуты обнаружения сигнала зуммера AI.

Синтаксис

```
RK_S32 RK_MPI_AI_EnableBuz(AUDIO_DEV AiDevId, AI_CHN AiChn);
```

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.37 RK_MPI_AI_DisableBuz

Описание

Отключает атрибуты обнаружения сигнала зуммера AI.

Синтаксис

```
RK_S32 RK_MPI_AI_DisableBuz(AUDIO_DEV AiDevId, AI_CHN AiChn);
```

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.38 RK_MPI_AI_SetBuzAttr

【Описание】

Задаёт атрибуты обнаружения сигнала зуммера AI.

【Синтаксис】

RK_S32 RK_MPI_AI_SetBuzAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, const AI_BUZ_CONFIG_S *pstBcdConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstBuzConfig	Указатель на структуру атрибутов BUZ.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.39 RK_MPI_AI_GetBuzAttr

【Описание】

Получает атрибуты обнаружения сигнала зуммера AI.

【Синтаксис】

RK_S32 RK_MPI_AI_GetBuzAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_BUZ_CONFIG_S *pstBuzConfig);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstBuzConfig	Указатель на структуру атрибутов BUZ.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.40 RK_MPI_AI_GetBuzResult

Описание

Получает результат обнаружения сигнала зуммера AI.

Синтаксис

RK_S32 RK_MPI_AI_GetBuzResult(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_BUZ_RESULT_S *pstBuzResult);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstBuzResult	Указатель на структуру результата обнаружения события BUZ.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.41 RK_MPI_AI_EnableGbs

Описание

Включает атрибуты обнаружения разбития стекла AI.

Синтаксис

RK_S32 RK_MPI_AI_EnableBcd(AUDIO_DEV AiDevId, AI_CHN AiChn);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.42 RK_MPI_AI_DisableGbs

Описание

Отключает атрибуты обнаружения разбития стекла AI.

Синтаксис

RK_S32 RK_MPI_AI_DisableGbs(AUDIO_DEV AiDevId, AI_CHN AiChn);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.43 RK_MPI_AI_SetGbsAttr

Описание

Задаёт атрибуты обнаружения разбития стекла AI.

Синтаксис

RK_S32 RK_MPI_AI_SetGbsAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, const AI_GBS_CONFIG_S *pstGbsConfig);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstGbsConfig	Указатель на структуру атрибутов GBS.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.44 RK_MPI_AI_GetGbsAttr

Описание

Получает атрибуты обнаружения разбития стекла AI.

Синтаксис

RK_S32 RK_MPI_AI_GetGbsAttr(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_GBS_CONFIG_S *pstGbsConfig);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstGbsConfig	Указатель на структуру атрибутов GBS.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.45 RK_MPI_AI_GetGbsResult

Описание

Получает результат обнаружения разбития стекла AI.

Синтаксис

RK_S32 RK_MPI_AI_GetGbsResult(AUDIO_DEV AiDevId, AI_CHN AiChn, AI_GBS_RESULT_S *pstGbsResult);

Параметры

Имя параметра	Описание	Ввод/вывод
AiDevId	Номер аудиоустройства. Диапазон: [0, AI_DEV_MAX_NUM).	Вход
AiChn	Номер канала аудиовхода. Диапазон: [0, AI_MAX_CHN_NUM).	Вход
pstGbsResult	Указатель на структуру результата обнаружения события GBS.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

3.46 Аудиовыход

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_AO_SetPubAttr: задаёт атрибуты устройства AO.
- RK_MPI_AO_GetPubAttr: получает атрибуты устройства AO.
- RK_MPI_AO_Enable: включает устройство AO.
- RK_MPI_AO_Disable: отключает устройство AO.
- RK_MPI_AO_EnableChn: включает канал AO.
- RK_MPI_AO_DisableChn: отключает канал AO.
- RK_MPI_AO_SendFrame: отправляет аудиокадр AO.
- RK_MPI_AO_EnableReSmp: включает передискретизацию AO.
- RK_MPI_AO_DisableReSmp: отключает передискретизацию AO.
- RK_MPI_AO_PauseChn: приостанавливает канал AO.
- RK_MPI_AO_ResumeChn: возобновляет канал AO.
- RK_MPI_AO_ClearChnBuf: очищает текущий буфер аудиоданных канала AO.
- RK_MPI_AO_QueryChnStat: запрашивает состояние текущего буфера аудиоданных канала AO.
- RK_MPI_AO_SetTrackMode: задаёт режим каналов устройства AO.
- RK_MPI_AO_GetTrackMode: получает режим каналов устройства AO.
- RK_MPI_AO_SetVolume: задаёт громкость устройства AO.
- RK_MPI_AO_GetVolume: получает громкость устройства AO.
- RK_MPI_AO_SetChnParams: задаёт параметры канала AO.
- RK_MPI_AO_GetChnParams: получает параметры канала AO.
- RK_MPI_AO_SetMute: задаёт состояние отключения звука устройства AO.
- RK_MPI_AO_GetMute: получает состояние отключения звука устройства AO.
- RK_MPI_AO_SaveFile: включает сохранение аудиовыхода в файл.
- RK_MPI_AO_QueryFileStatus: проверяет, сохраняет ли канал аудиовыхода данные в файл.
- RK_MPI_AO_ClrPubAttr: очищает атрибуты устройства AO.
- RK_MPI_AO_WaitEos: ожидает завершения воспроизведения указанного устройства и канала.

3.47 RK_MPI_AO_SetPubAttr

Описание

Задаёт параметры устройства AO (звуковой карты).

Синтаксис

```
RK_S32 RK_MPI_AO_SetPubAttr(AUDIO_DEV AoDevId, const AIO_ATTR_S *pstAttr);
```

Параметры

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
pstAttr	Атрибуты устройства аудиовыхода.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

Примечание

- Перед настройкой атрибутов AO необходимо убедиться, что AO отключено. Если оно включено, сначала отключите звуковую карту AO.
- Обязательно задайте поддерживаемые звуковой картой частоту дискретизации и количество каналов. Остальные параметры необязательны и при отсутствии настройки получают значения по умолчанию; см. структуру AIO_ATTR_S.
- Настройка драйвера звуковой карты на платформе Linux описана в /etc/asound.conf; в Linux на rk356x по умолчанию используется pcm.card0.

3.48 RK_MPI_AO_GetPubAttr

Описание

Получает атрибуты устройства AO.

Синтаксис

RK_S32 RK_MPI_AO_GetPubAttr(AUDIO_DEV AoDevId, AIO_ATTR_S *pstAttr);

Параметры

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
pstAttr	Атрибуты устройства аудиовыхода.	Выход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

Примечание

- Получает заданные атрибуты AO.
- Если атрибуты никогда не настраивались, возвращается ошибка «атрибуты не настроены».

3.49 RK_MPI_AO_Enable

Описание

Включает устройство AO.

Синтаксис

RK_S32 RK_MPI_AO_Enable(AUDIO_DEV AoDevId);

Параметры

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Перед включением необходимо настроить атрибуты устройства AO, иначе возвращается ошибка «атрибуты не настроены».
- Если устройство AO уже включено, повторный вызов возвращает ошибку «используется».

3.50 RK_MPI_AO_Disable

【Описание】

Отключает устройство AO.

【Синтаксис】

```
RK_S32 RK_MPI_AO_Disable(AUDIO_DEV AoDevId);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Если устройство AO не включено, возвращается код ошибки AUDIO «не включено».
- Перед отключением устройства AO необходимо отключить все его каналы AO.

3.51 RK_MPI_AO_EnableChn

【Описание】

Включает канал AO.

【Синтаксис】

RK_S32 RK_MPI_AO_EnableChn(AUDIO_DEV AoDevId, AO_CHN AoChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Перед включением канала AO необходимо включить соответствующее устройство AO, иначе возвращается код ошибки AUDIO «устройство не запущено».

3.52 RK_MPI_AO_DisableChn

【Описание】

Отключает канал AO.

【Синтаксис】

RK_S32 RK_MPI_AO_DisableChn(AUDIO_DEV AoDevId, AO_CHN AoChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- После завершения работы с каналом необходимо вызвать эту функцию для отключения канала AO.

3.53 RK_MPI_AO_SendFrame

【Описание】

Отправляет аудиокадр AO.

【Синтаксис】

RK_S32 RK_MPI_AO_SendFrame(AUDIO_DEV AoDevId, AO_CHN AoChn, const AUDIO_FRAME_S *pstData, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход
pstData	Указатель на структуру аудиокадра.	Вход
s32MilliSec	Время ожидания отправки данных: -1 означает блокирующий режим; >=0 задаёт время ожидания в неблокирующем режиме.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Этот интерфейс используется для отправки пользователем аудиокадра на выход AO. Если канал AO уже привязан системным интерфейсом RK_MPI_SYS_Bind к AI или ADEC, вызывать этот интерфейс не требуется и не рекомендуется.
- Если s32MilliSec равно -1, используется блокирующая отправка: интерфейс блокируется до успешной передачи данных. Значение больше либо равно 0 задаёт время ожидания отправки в неблокирующем режиме (мс). Если передача завершается в пределах указанного времени, возвращается успех; иначе возвращается ошибка тайм-аута.
- Перед отправкой аудиокадра на выход AO необходимо включить соответствующий канал AO и звуковую карту.
- Для выделения памяти под аудио рекомендуется по возможности использовать malloc.

3.54 RK_MPI_AO_EnableReSmp

【Описание】

Включает передискретизацию AO.

【Синтаксис】

RK_S32 RK_MPI_AO_EnableReSmp(AUDIO_DEV AoDevId, AO_CHN AoChn, AUDIO_SAMPLE_RATE_E enInSampleRate);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход
enInSampleRate	Входная частота передискретизации аудио.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Этот интерфейс следует вызывать после включения канала AO, но до его привязки.
- Допускается повторное включение передискретизации, но новая входная частота должна отличаться от ранее заданной.

- Если после отключения канал AO включается повторно и требуется передискретизация, необходимо снова вызвать этот интерфейс.
- Частота, задаваемая интерфейсом передискретизации AO, является частотой отправляемых данных.
- Даже если преобразование частоты не требуется, рекомендуется включать этот интерфейс: внутренний модуль сравнивает параметры и при их совпадении передаёт данные напрямую в режиме bypass.

【Пример】

Пример настройки передискретизации с 8 кГц до 32 кГц при декодировании ADEC и воспроизведении AO:

```
/* dev attr of ao */
AIO_ATTR_S aoAttr;
/* dev attr of ai */
aoAttr.soundCard.channels = 2;
aoAttr.soundCard.sampleRate = 32000;
aoAttr.soundCard.bitWidth = AUDIO_BIT_WIDTH_16;

aoAttr.enBitwidth = AUDIO_BIT_WIDTH_16;
aoAttr.enSamplerate = AUDIO_SAMPLE_RATE_8000;
aoAttr.enSoundmode = AUDIO_SOUND_MODE_MONO;

RK_MPI_AO_SetPubAttr(aoDevId, &aoAttr);
RK_MPI_AO_EnableReSmp(AoDev, AoChn, AUDIO_SAMPLE_RATE_8000);
```

3.55 RK_MPI_AO_DisableReSmp

【Описание】

Отключает передискретизацию AO.

【Синтаксис】

RK_S32 RK_MPI_AO_DisableReSmp(AUDIO_DEV AoDevId, AO_CHN AoChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Если функция передискретизации AO больше не используется, её следует отключить этим интерфейсом.

3.56 RK_MPI_AO_PauseChn

【Описание】

Приостанавливает канал AO.

【Синтаксис】

RK_S32 RK_MPI_AO_PauseChn(AUDIO_DEV AoDevId, AO_CHN AoChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- После приостановки канала AO аудиокадры, продолжающие поступать из привязанного канала ADEC, блокируются. Если же аудиокадры продолжает отправлять привязанный канал AI, то при наличии места они помещаются в буфер канала, а при заполненном буфере — отбрасываются.
- Нельзя приостанавливать канал AO, находящийся в отключённом состоянии.

3.57 RK_MPI_AO_ResumeChn

【Описание】

Включает передискретизацию AO.

【Синтаксис】

RK_S32 RK_MPI_AO_ResumeChn(AUDIO_DEV AoDevId, AO_CHN AoChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- После приостановки канал AO можно возобновить вызовом этого интерфейса.
- Если канал AO приостановлен или включён, интерфейс возвращает успех; в остальных состояниях возвращается ошибка.

3.58 RK_MPI_AO_ClearChnBuf

【Описание】

Очищает текущий буфер аудиоданных канала AO.

【Синтаксис】

RK_S32 RK_MPI_AO_ClearChnBuf(AUDIO_DEV AoDevId, AO_CHN AoChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AO.
- Для полного удаления всех буферизованных данных тракта декодирования и воспроизведения этот интерфейс следует использовать совместно с RK_MPI_ADEC_ClearChnBuf.

3.59 RK_MPI_AO_QueryChnStat

【Описание】

Запрашивает состояние текущего буфера аудиоданных канала AO.

【Синтаксис】

RK_S32 RK_MPI_AO_QueryChnStat(AUDIO_DEV AoDevId, AO_CHN AoChn, AO_CHN_STATE_S *pstStatus);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход
pstStatus	Указатель на структуру состояния буфера.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AO.

3.60 RK_MPI_AO_SetTrackMode

【Описание】

Задаёт режим каналов устройства AO.

【Синтаксис】

RK_S32 RK_MPI_AO_SetTrackMode(AUDIO_DEV AoDevId, AUDIO_TRACK_MODE_E enTrackMode);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
enTrackMode	Режим каналов аудиоустройства.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Определение AUDIO_TRACK_MODE_E см. в rk_comm_aio.h.
- Вызывайте этот интерфейс после успешного включения канала AO.

3.61 RK_MPI_AO_GetTrackMode

【Описание】

Получает режим каналов устройства AO.

【Синтаксис】

RK_S32 RK_MPI_AO_GetTrackMode(AUDIO_DEV AoDevId, AUDIO_TRACK_MODE_E* penTrackMode);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
penTrackMode	Указатель на режим каналов аудиоустройства.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AO.

3.62 RK_MPI_AO_SetVolume

【Описание】

Задаёт громкость устройства AO.

【Синтаксис】

RK_S32 RK_MPI_AO_SetVolume(AUDIO_DEV AoDevId, RK_S32 s32VolumeDb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
s32VolumeDb	Громкость аудиоустройства. Диапазон: [0, 100].	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AO.

3.63 RK_MPI_AO_GetVolume

【Описание】

Получает громкость устройства AO.

【Синтаксис】

```
RK_S32 RK_MPI_AO_GetVolume([AUDIO_DEV](#AUDIO_DEV) AoDevId, RK_S32 *ps32VolumeDb);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
ps32VolumeDb	Указатель на громкость аудиоустройства.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AO.

3.64 RK_MPI_AO_SetChnParams

【Описание】

Задаёт параметры канала AO.

【Синтаксис】

RK_S32 RK_MPI_AO_SetChnParams(AUDIO_DEV AoDevId, AO_CHN AoChn, const AO_CHN_PARAM_S *pstParams);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход
pstParams	Параметры канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Этот интерфейс необходимо вызывать до включения канала AO.

【Пример】

Пример настройки вывода данных канала AO 0 в левый канал:

```
AUDIO_DEV aoDevId = 0;
AO_CHN aoChn0 = 0;
AIO_ATTR_S aoAttr;
memset(&aoAttr, 0, sizeof(AIO_ATTR_S));

aoAttr.soundCard.channels = 2;
aoAttr.soundCard.sampleRate = 48000;
aoAttr.soundCard.bitWidth = AUDIO_BIT_WIDTH_16;
aoAttr.enBitwidth = AUDIO_BIT_WIDTH_16;
aoAttr.enSamplerate = AUDIO_SAMPLE_RATE_8000;
aoAttr.enSoundmode = AUDIO_SOUND_MODE_MONO;
aoAttr.u32FrmNum = 4;
aoAttr.u32PtNumPerFrm = 1024;
aoAttr.u32EXFlag = 0;
aoAttr.u32ChnCnt = 2;

RK_MPI_AO_SetPubAttr(aoDevId, &aoAttr);
RK_MPI_AO_Enable(aoDevId);

AO_CHN_PARAM_S pstParams;
memset(&pstParams, 0, sizeof(AO_CHN_PARAM_S));
pstParams.enMode = AUDIO_CHN_MODE_LEFT;
RK_MPI_AO_SetChnParams(aoDevId, aoChn0, &pstParams);

RK_MPI_AO_EnableChn(aoDevId, aoChn0);
RK_MPI_AO_EnableReSmp(aoDevId, aoChn0, AUDIO_SAMPLE_RATE_8000);
```

3.65 RK_MPI_AO_GetChnParams

【Описание】

Получает параметры канала AO.

【Синтаксис】

RK_S32 RK_MPI_AO_GetChnParams(AUDIO_DEV AoDevId, AO_CHN AoChn, AO_CHN_PARAM_S *pstParams);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход
pstParams	Параметры канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

3.66 RK_MPI_AO_SetMute

【Описание】

Задаёт состояние отключения звука устройства AO.

【Синтаксис】

RK_S32 RK_MPI_AO_SetMute(AUDIO_DEV AoDevId, RK_BOOL bEnable, const AUDIO_FADE_S *pstFade);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
bEnable	Включение отключения звука аудиоустройства. RK_TRUE: включить mute; RK_FALSE: отключить mute.	Вход
pstFade	Указатель на структуру плавного появления/затухания.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AO.
- При вызове интерфейса пользователь может выбрать, использовать ли плавное появление/затухание. Если оно не требуется, указатель на структуру следует задать равным NULL.

3.67 RK_MPI_AO_GetMute

【Описание】

Получает громкость устройства AO.

【Синтаксис】

RK_S32 RK_MPI_AO_GetVolume(AUDIO_DEV AoDevId, RK_S32 *ps32VolumeDb);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
pbEnable	Указатель на состояние отключения звука аудиоустройства.	Выход
pstFade	Указатель на структуру плавного появления/затухания.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AO.

3.68 RK_MPI_AO_SaveFile

【Описание】

Включает сохранение аудиовыхода в файл.

【Синтаксис】

RK_S32 RK_MPI_AO_SaveFile(AUDIO_DEV AoDevId, AO_CHN AoChn, AUDIO_SAVE_FILE_INFO_S* pstSaveFileInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход
pstSaveFileInfo	Указатель на структуру атрибутов сохранения аудио в файл.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AO.

3.69 RK_MPI_AO_QueryFileStatus

【Описание】

Проверяет, сохраняет ли канал аудиовыхода данные в файл.

【Синтаксис】

RK_S32 RK_MPI_AO_QueryFileStatus(AUDIO_DEV AoDevId, AO_CHN AoChn, AUDIO_FILE_STATUS_S* pstFileStatus);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход
pstFileStatus	Указатель на структуру состояния.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

Этот интерфейс используется для проверки состояния сохранения канала аудиовыхода в файл.

3.70 RK_MPI_AO_ClrPubAttr

【Описание】

Очищает атрибуты устройства AO.

【Синтаксис】

RK_S32 RK_MPI_AO_ClrPubAttr(AUDIO_DEV AoDevId);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Перед очисткой атрибутов устройства его необходимо остановить.

3.71 RK_MPI_AO_WaitEos

【Описание】

Ожидает завершения воспроизведения указанного устройства и канала.

【Синтаксис】

RK_S32 RK_MPI_AO_WaitEos(AUDIO_DEV AoDevId, AO_CHN AoChn, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AoDevId	Номер аудиоустройства. Диапазон: [0, AO_DEV_MAX_NUM).	Вход
AoChn	Номер канала аудиовыхода. Диапазон: [0, AO_MAX_CHN_NUM).	Вход
s32MilliSec	Время ожидания: -1 — блокирующий режим; >=0 — время ожидания неблокирующего режима (мс).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AO.
- После привязки ADEC к AO рекомендуется вызвать этот интерфейс в блокирующем режиме и дождаться окончания воспроизведения.
- Если s32MilliSec равно -1, ожидание завершения воспроизведения выполняется в блокирующем режиме. Значение больше либо равно 0 означает, что после ожидания s32MilliSec миллисекунд проверяется результат завершения. Если воспроизведение не завершилось за отведённое время, возвращается ошибка тайм-аута.

3.72 Аудиодекодирование

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_ADEC_CreateChn: создаёт канал аудиодекодирования.
- RK_MPI_ADEC_DestroyChn: уничтожает канал аудиодекодирования.
- RK_MPI_ADEC_SendStream: отправляет аудиопоток в канал декодирования.
- RK_MPI_ADEC_ClearChnBuf: очищает текущий буфер аудиоданных канала ADEC.
- RK_MPI_ADEC_GetFrame: получает декодированный аудиокадр.
- RK_MPI_ADEC_ReleaseFrame: освобождает декодированный аудиокадр.
- RK_MPI_ADEC_SendEndOfStream: отправляет декодеру признак окончания потока.
- RK_MPI_ADEC_QueryChnStat: запрашивает состояние текущего буфера аудиоданных канала ADEC.
- RK_MPI_ADEC_RegisterDecoder: регистрирует декодер.
- RK_MPI_ADEC_UnRegisterDecoder: отменяет регистрацию декодера.

3.73 RK_MPI_ADEC_CreateChn

【Описание】

Создаёт канал аудиодекодирования.

【Синтаксис】

RK_S32 RK_MPI_ADEC_CreateChn (ADEC_CHN AdChn, const ADEC_CHN_ATTR_S *pstAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AdChn	Номер канала. Диапазон: [0, ADEC_MAX_CHN_NUM).	Вход
pstAttr	Указатель на атрибуты канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Аудиодекодирование поддерживает протоколы RK_AUDIO_ID_PCM_ALAW, RK_AUDIO_ID_PCM_MULAW, RK_AUDIO_ID_ADPCM_G722, RK_AUDIO_ID_ADPCM_G726 и другие; см. перечисление RK_CODEC_ID_E для аудио в rk_common.h.
- Для аудиоформата g726 необходимо задать кодовое слово потока (codecwords); поддерживаемые протоколы приведены в таблице протоколов аудиокодирования и декодирования.
- В атрибутах инициализации аудиодекодирования обязательно задаются частота дискретизации потока (u32SampleRate), количество каналов (u32Channels) и codec id (enType). По умолчанию используется режим PACK. Режим STREAM поддерживается, но рекомендуется PACK.
- Этот интерфейс можно использовать только для свободного канала. Если канал уже создан, возвращается ошибка «канал уже создан».

3.74 RK_MPI_ADEC_DestroyChn

【Описание】

Уничтожает канал аудиодекодирования.

【Синтаксис】

RK_S32 RK_MPI_ADEC_DestroyChn(ADEC_CHN AdChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AdChn	Номер канала. Диапазон: [0, ADEC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Если вызвать интерфейс для несозданного канала, возвращается RK_ERR_ADEC_UNEXIST.
- Рекомендуется вызывать этот интерфейс после завершения работы с каналом.

3.75 RK_MPI_ADEC_SendStream

【Описание】

Отправляет поток в канал аудиодекодирования.

【Синтаксис】

RK_S32 RK_MPI_ADEC_SendStream (ADEC_CHN AdChn, const AUDIO_STREAM_S *pstStream, RK_BOOL bBlock);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AdChn	Номер канала. Диапазон: [0, ADEC_MAX_CHN_NUM).	Вход
pstStream	Аудиопоток.	Вход
bBlock	Признак блокировки. RK_TRUE: блокирующий режим. RK_FALSE: неблокирующий режим.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- При создании канала декодирования можно выбрать режим pack или stream:
  - режим pack используется, когда известно, что пакет потока является одним кадром, например для потока, получаемого непосредственно из сети, или при чтении из файла, когда точно известны границы кадра. Этот режим обладает более высокой эффективностью.
  - Режим stream используется, когда неизвестно, содержит ли пакет полный кадр; он менее эффективен и может увеличивать задержку.
- Перед отправкой данных необходимо убедиться, что канал создан, иначе сразу возвращается ошибка.
- Поддерживается отправка потока в блокирующем и неблокирующем режиме.
- При блокирующей отправке, если buffer для декодированных аудиокадров заполнен, вызов блокируется до тех пор, пока декодированный кадр не будет получен либо канал ADEC не будет уничтожен. Рекомендуется блокирующий режим отправки.
- Убедитесь в корректности потока, отправляемого в канал ADEC, иначе декодер может аварийно завершиться.
- Для выделения памяти под аудио рекомендуется по возможности использовать malloc.

3.76 RK_MPI_ADEC_GetFrame

【Описание】

Получает декодированный аудиокадр.

【Синтаксис】

RK_S32 RK_MPI_ADEC_GetFrame (ADEC_CHN AdChn, AUDIO_FRAME_INFO_S *pstFrmInfo, RK_BOOL bBlock);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AdChn	Номер канала. Диапазон: [0, ADEC_MAX_CHN_NUM).	Вход
pstFrmInfo	Структура данных аудиокадра.	Выход
bBlock	Получать ли данные в блокирующем режиме.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Интерфейс необходимо вызывать после создания канала ADEC.
- При получении аудиокадров этим интерфейсом рекомендуется отправлять поток в режиме pack (кадровом режиме).
- Если при получении аудиокадров этим интерфейсом поток отправляется в режиме stream, необходимо своевременно получать декодированные кадры. Если SendStream и GetFrame вызываются в блокирующем режиме, они должны выполняться в разных потоках, иначе возникнет взаимная блокировка.
- После успешного GetFrame необходимо вызвать ReleaseFrame.
- Если ADEC привязан к AO, этот интерфейс получения аудиоданных не используется. Сначала снимите привязку ADEC к AO, иначе полученные кадры будут прерывистыми.

3.77 RK_MPI_ADEC_ReleaseFrame

【Описание】

Освобождает полученный декодированный аудиокадр.

【Синтаксис】

RK_S32 RK_MPI_ADEC_ReleaseFrame (ADEC_CHN AdChn, AUDIO_FRAME_INFO_S *pstFrmInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AdChn	Номер канала. Диапазон: [0, ADEC_MAX_CHN_NUM).	Вход
pstFrmInfo	Структура данных аудиокадра.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Интерфейс необходимо вызывать после создания канала ADEC.
- Этот интерфейс должен использоваться совместно с RK_MPI_ADEC_GetFrame.

3.78 RK_MPI_ADEC_SendEndOfStream

【Описание】

Отправляет декодеру признак окончания потока.

【Синтаксис】

RK_S32 RK_MPI_ADEC_SendEndOfStream (ADEC_CHN AdChn, RK_BOOL bInstant);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AdChn	Номер канала. Диапазон: [0, ADEC_MAX_CHN_NUM).	Вход
bInstant	Следует ли немедленно очистить внутренний буфер декодера. Диапазон значений: RK_FALSE: не очищать буфер; декодирование продолжается до завершения аудиопотока. RK_TRUE: немедленно очистить внутренний буфер декодера.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Для bInstant рекомендуется использовать RK_FALSE.

3.79 RK_MPI_ADEC_QueryChnStat

【Описание】

Запрашивает состояние текущего буфера аудиоданных канала ADEC.

【Синтаксис】

RK_S32 RK_MPI_ADEC_QueryChnStat (ADEC_CHN AdChn, ADEC_CHN_STATE_S *pstBufferStatus);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AdChn	Номер канала. Диапазон: [0, ADEC_MAX_CHN_NUM).	Вход
pstBufferStatus	Указатель на структуру состояния буфера.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала ADEC.

3.80 RK_MPI_ADEC_RegisterDecoder

【Описание】

Регистрирует аудиодекодер.

【Синтаксис】

RK_S32 RK_MPI_ADEC_RegisterDecoder(RK_S32 *ps32Handle, const ADEC_DECODER_S *pstDecoder);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ps32Handle	Дескриптор регистрации.	Выход
pstDecoder	Структура атрибутов декодера.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- При регистрации и использовании декодера через этот интерфейс заказчик обязан предварительно получить разрешение правообладателя и уплатить лицензионный сбор.
- Аудиодекодер необходимо зарегистрировать до создания канала ADEC.
- Пользователь передаёт структуру атрибутов декодера, регистрирует декодер в модуле ADEC и получает дескриптор регистрации. Этот дескриптор используется для отмены регистрации декодера.
- Во внешнем коде модуля ADEC можно зарегистрировать не более 20 декодеров. Сам модуль уже регистрирует пять декодеров: LPCM, G711a, G711u, G726 и ADPCM.
- Нельзя повторно регистрировать снаружи декодер одного и того же типа. Например, если внешний декодер G711a уже зарегистрирован, ещё один внешний G711a зарегистрировать нельзя.
- Разрешается внешняя регистрация типа, уже зарегистрированного внутри ADEC. Например, модуль ADEC содержит встроенный декодер G711a, но внешний G711a также можно зарегистрировать, причём приоритет будет отдан внешнему декодеру.
- Пример см. в приложении A «Описание API регистрации декодеров/кодеров».

3.81 RK_MPI_ADEC_UnRegisterDecoder

【Описание】

Отменяет регистрацию декодера.

【Синтаксис】

```
RK_S32 RK_MPI_ADEC_UnRegisterDecoder(RK_S32 s32Handle);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
s32Handle	Дескриптор регистрации, полученный при регистрации декодера.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Если один зарегистрированный декодер используется несколькими каналами ADEC, отменить его регистрацию можно только после уничтожения всех использующих его каналов.

3.82 Аудиокодирование

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_AENC_CreateChn: создаёт канал аудиокодирования.
- RK_MPI_AENC_DestroyChn: уничтожает канал аудиокодирования.
- RK_MPI_AENC_SendFrame: отправляет аудиокадр на кодирование.
- RK_MPI_AENC_GetStream: получает закодированный аудиопоток.
- RK_MPI_AENC_ReleaseStream: освобождает закодированный аудиопоток.
- RK_MPI_AENC_SaveFile: включает сохранение данных канала до аудиокодирования в файл.
- RK_MPI_AENC_QueryFileStatus: проверяет, находится ли канал аудиокодирования в режиме сохранения в файл.
- RK_MPI_AENC_RegisterEncoder: регистрирует кодер.
- RK_MPI_AENC_UnRegisterEncoder: отменяет регистрацию кодера.

3.83 RK_MPI_AENC_CreateChn

【Описание】

Создаёт канал аудиодекодирования.

【Синтаксис】

RK_S32 RK_MPI_AENC_CreateChn(AENC_CHN AeChn, const AENC_CHN_ATTR_S *pstAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AeChn	Номер канала. Диапазон: [0, AENC_MAX_CHN_NUM).	Вход
pstAttr	Указатель на атрибуты канала.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Аудиокодирование поддерживает протоколы RK_AUDIO_ID_ADPCM_G722, RK_AUDIO_ID_ADPCM_G726, RK_AUDIO_ID_PCM_MULAW, RK_AUDIO_ID_PCM_ALAW и другие; см. перечисление RK_CODEC_ID_E для аудио в rk_common.h.
- В атрибутах инициализации аудиокодирования обязательно задаются частота дискретизации потока (u32SampleRate), количество каналов (u32Channels), разрядность (enBitwidth) и codec id (enType).
- Этот интерфейс можно использовать только для свободного канала. Если канал уже создан, возвращается ошибка «канал уже создан».

3.84 RK_MPI_AENC_DestroyChn

【Описание】

Уничтожает канал аудиокодирования.

【Синтаксис】

RK_S32 RK_MPI_AENC_DestroyChn(AENC_CHN AeChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AeChn	Номер канала. Диапазон: [0, AENC_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Если вызвать интерфейс для несозданного канала, возвращается RK_ERR_AENC_UNEXIST.
- Рекомендуется вызывать этот интерфейс после завершения работы с каналом.

3.85 RK_MPI_AENC_SendFrame

【Описание】

Отправляет аудиокадр на кодирование.

【Синтаксис】

RK_S32 RK_MPI_AENC_SendFrame(AENC_CHN AeChn, const AUDIO_FRAME_S *pstFrm, const AEC_FRAME_S *pstAecFrm, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AeChn	Номер канала. Диапазон: [0, AENC_MAX_CHN_NUM).	Вход
pstFrm	Указатель на структуру аудиокадра.	Вход
pstAecFrm	Указатель на структуру опорного кадра эхоподавления.	Вход
s32MilliSec	Время ожидания отправки данных: -1 означает блокирующий режим; >=0 задаёт время ожидания в неблокирующем режиме (мс).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- В настоящее время эхоподавление не реализовано; pstAecFrm можно задать равным NULL.
- Если s32MilliSec равно -1, отправка выполняется в блокирующем режиме: интерфейс ожидает до успешной передачи данных. Значение больше либо равно 0 означает неблокирующий режим отправки. s32MilliSec задаёт время ожидания; если данные успешно отправлены за s32MilliSec миллисекунд, возвращается успех, а если заданное время истекло и данные не отправлены — ошибка.
- Этот интерфейс применяется для явной отправки аудиокадра пользователем на кодирование. Если канал AENC уже привязан к AI системным интерфейсом RK_MPI_SYS_Bind, вызывать его не требуется и не рекомендуется.
- Перед отправкой аудиокадра на кодирование необходимо создать соответствующий канал кодирования.

3.86 RK_MPI_AENC_GetStream

【Описание】

Получает закодированный поток.

【Синтаксис】

RK_S32 RK_MPI_AENC_GetStream(AENC_CHN AeChn, AUDIO_STREAM_S *pstStream, RK_S32 s32MilliSec);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AeChn	Номер канала. Диапазон: [0, AENC_MAX_CHN_NUM).	Вход
pstStream	Структура данных аудиокадра.	Выход
s32MilliSec	Время ожидания получения данных: -1 означает блокирующий режим; >=0 задаёт время ожидания неблокирующего режима.	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Получить поток через этот интерфейс можно только после создания канала.
- Если s32MilliSec равно -1, получение выполняется в блокирующем режиме: интерфейс ожидает до успешного получения данных. Значение больше либо равно 0 означает неблокирующий режим. Если данные получены за s32MilliSec миллисекунд, возвращается успех; если заданное время истекло и данные не получены — ошибка.

3.87 RK_MPI_AENC_ReleaseStream

【Описание】

Освобождает поток, полученный из канала аудиокодирования.

【Синтаксис】

RK_S32 RK_MPI_AENC_ReleaseStream(AENC_CHN AeChn, const AUDIO_STREAM_S *pstStream);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AeChn	Номер канала. Диапазон: [0, AENC_MAX_CHN_NUM).	Вход
pstStream	Указатель на получаемый поток.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Рекомендуется освобождать поток сразу после использования; несвоевременное освобождение блокирует процесс кодирования.
- Освобождаемый поток должен быть получен из того же канала. Нельзя изменять структуру информации о потоке, иначе поток невозможно будет освободить, его buffer будет потерян и возможна авария программы.
- При освобождении потока канал должен быть создан; иначе сразу возвращается ошибка. Если канал уничтожен во время освобождения, также немедленно возвращается ошибка.

3.88 RK_MPI_AENC_SaveFile

【Описание】

Включает сохранение данных канала до аудиокодирования в файл.

【Синтаксис】

RK_S32 RK_MPI_AENC_SaveFile(AENC_CHN AeChn, const AUDIO_SAVE_FILE_INFO_S *pstSaveFileInfo);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AeChn	Номер канала. Диапазон: [0, AENC_MAX_CHN_NUM).	Вход
pstSaveFileInfo	Указатель на структуру атрибутов сохранения аудио в файл.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Вызывайте этот интерфейс после успешного включения канала AENC.

3.89 RK_MPI_AENC_QueryFileStatus

【Описание】

Проверяет, находится ли канал аудиокодирования в режиме сохранения в файл.

【Синтаксис】

RK_S32 RK_MPI_AENC_QueryFileStatus(AENC_CHN AeChn, AUDIO_FILE_STATUS_S* pstFileStatus);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AeChn	Номер канала. Диапазон: [0, AENC_MAX_CHN_NUM).	Вход
pstFileStatus	Указатель на структуру состояния.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Этот интерфейс используется для проверки состояния сохранения канала аудиовыхода в файл.

3.90 RK_MPI_AENC_RegisterEncoder

【Описание】

Регистрирует кодер.

【Синтаксис】

RK_S32 RK_MPI_AENC_RegisterEncoder(RK_S32 *ps32Handle, const AENC_ENCODER_S *pstEncoder);

【Параметры】

Имя параметра	Описание	Ввод/вывод
ps32Handle	Дескриптор регистрации.	Выход
pstEncoder	Структура атрибутов кодера.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- При регистрации и использовании кодера через этот интерфейс заказчик обязан предварительно получить разрешение правообладателя и уплатить лицензионный сбор.
- Аудиокодер необходимо зарегистрировать до создания канала AENC.
- Пользователь передаёт структуру атрибутов кодера, регистрирует кодер в модуле AENC и получает дескриптор регистрации, используемый для отмены регистрации декодера.
- кодера. Во внешнем коде модуля AENC можно зарегистрировать не более 20 кодеров. Сам модуль уже регистрирует четыре кодера: G711a, G711u, G722 и G726.
- Нельзя повторно регистрировать снаружи кодер одного и того же типа. Например, если внешний кодер G711a уже зарегистрирован, ещё один внешний G711a зарегистрировать нельзя.
- Разрешается внешняя регистрация типа, уже зарегистрированного внутри AENC. Например, модуль AENC содержит встроенный кодер G711a, но внешний G711a также можно зарегистрировать, причём приоритет будет отдан внешнему кодеру.
- Пример см. в приложении A «Описание API регистрации декодеров/кодеров».

3.91 RK_MPI_AENC_UnRegisterEncoder

【Описание】

Отменяет регистрацию кодера.

【Синтаксис】

```
RK_S32 RK_MPI_AENC_UnRegisterEncoder(RK_S32 s32Handle);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
ps32Handle	Дескриптор регистрации, полученный при регистрации кодера.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок AUDIO.

【Примечание】

- Если один зарегистрированный кодер используется несколькими каналами AENC, отменить его регистрацию можно только после уничтожения всех использующих его каналов.

4. Типы данных

4.1 Аудиовход и аудиовыход

Связанные с аудиовходом и аудиовыходом типы данных и структуры определены ниже:

- AUDIO_DEV: определяет дескриптор устройства AO.
- AO_CHN: определяет канал AO.
- AO_MAX_CHN_NUM: определяет максимальное количество каналов аудиовыхода.
- AO_DEV_MAX_NUM: определяет максимальное количество устройств аудиовыхода.
- AO_CHN_STATE_S: структура состояния буфера данных канала аудиовыхода.
- AI_CHN: определяет канал AI.
- AI_MAX_CHN_NUM: определяет максимальное количество каналов аудиовхода.
- AI_DEV_MAX_NUM: определяет максимальное количество устройств аудиовхода.
- AI_CHN_PARAM_S: определяет структуру параметров канала.
- MAX_AUDIO_FILE_PATH_LEN: максимальная длина пути к сохраняемому аудиофайлу.
- MAX_AUDIO_FILE_NAME_LEN: максимальная длина имени сохраняемого аудиофайла.
- AIO_ATTR_S: определяет структуру атрибутов устройства аудиовхода/выхода.
- AUDIO_FRAME_S: определяет структуру данных аудиокадра.
- AUDIO_SOUND_MODE_E: определяет режим аудиоканалов.
- AUDIO_BIT_WIDTH_E: определяет разрядность аудиосэмпла.
- AIO_SOUND_CARD: определяет параметры открытия звуковой карты.
- AUDIO_STREAM_S: определяет структуру аудиопотока.
- AUDIO_SAMPLE_RATE_E: определяет частоту дискретизации аудио.
- AUDIO_TRACK_MODE_E: тип режима каналов аудиоустройства.
- AUDIO_FADE_RATE_E: определяет тип скорости плавного появления/затухания устройства аудиовыхода.
- AUDIO_FADE_S: структура настройки плавного появления/затухания устройства аудиовыхода.
- AUDIO_CHN_MODE_E: тип режима аудиоканала.
- AO_CHN_PARAM_S: структура параметров канала аудиовыхода.
- AUDIO_SAVE_FILE_INFO_S: определяет структуру настроек функции сохранения аудио в файл.
- AUDIO_FILE_STATUS_S: определяет структуру состояния сохранения аудиофайла.
- AI_AED_CONFIG_S: определяет структуру атрибутов AED.
- AI_AED_RESULT_S: определяет структуру результата обнаружения события AED.
- AI_BCD_CONFIG_S: определяет структуру атрибутов BCD.
- AI_BCD_RESULT_S: определяет структуру результата обнаружения события BCD.
- AI_BUZ_CONFIG_S: определяет структуру атрибутов BUZ.
- AI_BUZ_RESULT_S: определяет структуру результата обнаружения события BUZ.
- AI_GBS_CONFIG_S: определяет структуру атрибутов GBS.
- AI_GBS_RESULT_S: определяет структуру результата обнаружения события GBS.

4.2 AUDIO_DEV

【Описание】

Определяет дескриптор устройства AO.

【Определение】

```
typedef RK_S32 AUDIO_DEV;
```

4.3 AO_CHN

【Описание】

Определяет канал AO.

【Определение】

```
typedef RK_S32 AO_CHN;
```

4.4 AO_MAX_CHN_NUM

【Описание】

Определяет максимальное количество каналов аудиовыхода.

【Определение】

```
#define AO_MAX_CHN_NUM  3
```

4.5 AO_DEV_MAX_NUM

【Описание】

Определяет максимальное количество устройств аудиовыхода.

【Определение】

```
#define AO_DEV_MAX_NUM 2
```

4.6 AO_CHN_STATE_S

【Описание】

Структура состояния буфера данных канала аудиовыхода.

【Определение】

```
typedef struct rkAO_CHN_STATE_S {
    RK_U32    u32ChnTotalNum;    /* total number of channel buffer */
    RK_U32    u32ChnFreeNum;     /* free number of channel buffer */
    RK_U32    u32ChnBusyNum;     /* busy number of channel buffer */
} AO_CHN_STATE_S;
```

【Члены】

Имя члена	Описание
u32ChnTotalNum	Общее количество блоков буфера выходного канала.
u32ChnFreeNum	Количество свободных блоков буфера.
u32ChnBusyNum	Количество занятых блоков буфера.

4.7 AI_CHN

【Описание】

Определяет канал AI.

【Определение】

```
typedef RK_S32 AI_CHN;
```

4.8 AI_MAX_CHN_NUM

【Описание】

Определяет максимальное количество каналов аудиовхода.

【Определение】

```
#define AI_MAX_CHN_NUM  1
```

4.9 AI_DEV_MAX_NUM

【Описание】

Определяет максимальное количество устройств аудиовхода.

【Определение】

```
#define AI_DEV_MAX_NUM 1
```

4.10 AI_CHN_PARAM_S

【Описание】

Определяет структуру параметров канала.

【Определение】

```
typedef struct rkAI_CHN_PARAM_S {
    RK_U32 u32UsrFrmDepth;
} AI_CHN_PARAM_S;
```

【Члены】

Имя члена	Описание
u32UsrFrmDepth	Глубина буфера аудиокадров.

4.11 MAX_AUDIO_FILE_PATH_LEN

【Описание】

Максимальная длина пути к сохраняемому аудиофайлу.

【Определение】

```
#define MAX_AUDIO_FILE_PATH_LEN 256
```

4.12 MAX_AUDIO_FILE_NAME_LEN

【Описание】

Максимальная длина имени сохраняемого аудиофайла.

【Определение】

```
#define MAX_AUDIO_FILE_NAME_LEN 256
```

4.13 AIO_ATTR_S

【Описание】

Определяет структуру атрибутов устройства аудиовхода/выхода.

【Определение】

```
typedef struct rkAIO_ATTR_S {
```

```
    // params of sound card
    AIO_SOUND_CARD      soundCard;
    // sample rate
    AUDIO_SAMPLE_RATE_E enSamplerate;
    // bitwidth
    AUDIO_BIT_WIDTH_E   enBitwidth;
    // momo or steror
    AUDIO_SOUND_MODE_E  enSoundmode;
    /* expand 8bit to 16bit,use AI_EXPAND(only valid for AI 8bit),
     * use AI_CUT(only valid for extern Codec for 24bit)
     */
    RK_U32              u32EXFlag;
    /* frame num in buf[2,MAX_AUDIO_FRAME_NUM] */
    RK_U32              u32FrmNum;
    /*
     * point num per frame (80/160/240/320/480/1024/2048)
     * (ADPCM IMA should add 1 point, AMR only support 160)
     */
    RK_U32              u32PtNumPerFrm;
    RK_U32              u32ChnCnt;      /* channle number on FS, valid value:1/2/4/8 */
    /*
     * name of sound card, if it is setted, we will
     * using it to open sound card, otherwise, use
     * the index of device to open sound card
     */
    RK_U8               u8CardName[64];
} AIO_ATTR_S;
```

Члены

Имя члена	Описание
soundCard	Параметры, задаваемые при открытии звуковой карты.
enSamplerate	Частота дискретизации аудио.
enBitwidth	Разрядность аудиосэмпла.
enSoundmode	Режим аудиоканалов.
u32EXFlag	По умолчанию задаётся 0.
u32FrmNum	Количество буферов.
u32PtNumPerFrm	Для AI: длина кадра, получаемого приложением (байт). Для AO: длина кадра, отправляемого приложением (байт).
u32ChnCnt	Поддерживаемое количество каналов.
u8CardName[64]	Имя звуковой карты.

Примечания

- soundCard: параметры открытия звуковой карты. Необходимо задать поддерживаемые драйвером частоту дискретизации, количество каналов и разрядность. Обязательный параметр.
- enSamplerate: для AI означает частоту данных, получаемых пользователем. Например, если звуковая карта открыта на 16 кГц, а читать требуется данные 8 кГц, enSamplerate задаётся как 8 кГц и вызывается RK_MPI_AI_EnableReSmp для преобразования 16 кГц в 8 кГц. Для AO означает частоту отправляемых данных. Например, пользователь отправляет данные 16 кГц, поэтому enSamplerate задаётся как 16 кГц, но звуковая карта открывается только на 48 кГц; необходимо вызвать RK_MPI_AI_EnableReSmp для преобразования 16 кГц в 48 кГц перед воспроизведением. Обязательный параметр.
- enBitwidth: принцип аналогичен enSamplerate; задаёт разрядность данных. Обязательный параметр.
- enSoundmode: принцип аналогичен enSamplerate; задаёт количество каналов данных. Обязательный параметр.
- u32FrmNum: количество кадров аудиоданных, обрабатываемых за одну передачу DMA. Если размер периода велик, за один раз обрабатывается больше данных, поэтому за единицу времени возникает меньше аппаратных прерываний, CPU получает больше времени для других задач и снижается энергопотребление. Однако это имеет существенный недостаток — увеличивается задержка обработки данных.
- u8CardName: позволяет снаружи указать открываемую звуковую карту; в коде предусмотрено значение по умолчанию.

4.14 AUDIO_FRAME_S

【Описание】

Определяет структуру данных аудиокадра.

【Определение】

```
typedef struct rkAUDIO_FRAME_S {
    MB_BLK              pMbBlk;
    AUDIO_BIT_WIDTH_E   enBitWidth;     /*audio frame bitwidth*/
    AUDIO_SOUND_MODE_E  enSoundMode;    /*audio frame momo or stereo mode*/
    RK_U64              u64TimeStamp;   /*audio frame timestamp*/
    RK_U32              u32Seq;         /*audio frame seq*/
    RK_U32              u32Len;
    /*data lenth per channel in frame, u32Len <= 0 mean eos */
    RK_BOOL             bBypassMbBlk;
    /* FALSE: copy, TRUE: MbBlock owned by internal */
} AUDIO_FRAME_S;
```

【Члены】

Имя члена	Описание
pMbBlk	Дескриптор блока буфера.
enBitWidth	Разрядность аудиосэмпла.
enSoundMode	Режим аудиоканалов.
u64TimeStamp	Временная метка аудиокадра, в мкс.
u32Seq	Порядковый номер аудиокадра.
u32Len	Длина аудиокадра, в байтах.
bBypassMbBlk	Требуется ли копировать внешний MB_BLK во внутренний. FALSE: требуется; TRUE: не требуется.

【Примечания】

- Данные AUDIO_FRAME_S хранятся в pMbBlk; для входа и выхода необходимо заранее выделить корректные блоки буфера.

4.15 AUDIO_SOUND_MODE_E

【Описание】

Определяет режим аудиоканалов.

【Определение】

```
typedef enum rkAIO_SOUND_MODE_E {
    AUDIO_SOUND_MODE_MONO   = 0,/*mono*/
    AUDIO_SOUND_MODE_STEREO = 1,/*stereo*/
    AUDIO_SOUND_MODE_BUTT
} AUDIO_SOUND_MODE_E;
```

【Члены】

Имя члена	Описание
AUDIO_SOUND_MODE_MONO	Моно.
AUDIO_SOUND_MODE_STEREO	Стерео.

4.16 AUDIO_BIT_WIDTH_E

【Описание】

Определяет разрядность аудиосэмпла.

【Определение】

```
typedef enum rkAUDIO_BIT_WIDTH_E {
    AUDIO_BIT_WIDTH_8   = 0,   /* 8bit width */
    AUDIO_BIT_WIDTH_16  = 1,   /* 16bit width*/
    AUDIO_BIT_WIDTH_24  = 2,   /* 24bit width*/
    AUDIO_BIT_WIDTH_BUTT,
} AUDIO_BIT_WIDTH_E;
```

【Члены】

Имя члена	Описание
AUDIO_BIT_WIDTH_8	Разрядность сэмпла 8 бит.
AUDIO_BIT_WIDTH_16	Разрядность сэмпла 16 бит.
AUDIO_BIT_WIDTH_24	Разрядность сэмпла 24 бита.

4.17 AIO_SOUND_CARD

【Описание】

Определяет разрядность аудиосэмпла.

【Определение】

```
typedef struct rkAIO_SOUND_CARD {
    RK_U32  channels;
    RK_U32  sampleRate;
    AUDIO_BIT_WIDTH_E bitWidth;
} AIO_SOUND_CARD;
```

【Члены】

Имя члена	Описание
channels	Количество каналов открываемой звуковой карты.
sampleRate	Частота дискретизации открываемой звуковой карты.
bitWidth	Разрядность открываемой звуковой карты.

4.18 AUDIO_STREAM_S

【Описание】

Определяет структуру аудиопотока.

【Определение】

```
typedef struct rkAUDIO_STREAM_S {
    MB_BLK pMbBlk;
    RK_U32 u32Len; /* stream lenth, by bytes */
    RK_U64 u64TimeStamp; /* frame time stamp*/
    RK_U32 u32Seq; /* frame seq,if stream is not a valid frame, u32Seq is 0*/
    RK_BOOL bBypassMbBlk; /* FALSE: copy, TRUE: MbBlock owned by internal */
} AUDIO_STREAM_S;
```

【Члены】

Имя члена	Описание
pMbBlk	Дескриптор блока буфера.
u32Len	Длина аудиопотока.
u32Seq	Порядковый номер аудиопотока.
u64TimeStamp	Временная метка аудиопотока.
bBypassMbBlk	Требуется ли копировать внешний MB_BLK во внутренний. FALSE: требуется; TRUE: не требуется.

4.19 AUDIO_SAMPLE_RATE_E

【Описание】

Определяет частоту дискретизации аудио.

【Определение】

```
typedef enum rkAUDIO_SAMPLE_RATE_E {
    AUDIO_SAMPLE_RATE_DISABLE = 0,
    AUDIO_SAMPLE_RATE_8000   = 8000,    /* 8K samplerate*/
    AUDIO_SAMPLE_RATE_12000  = 12000,   /* 12K samplerate*/
    AUDIO_SAMPLE_RATE_11025  = 11025,   /* 11.025K samplerate*/
    AUDIO_SAMPLE_RATE_16000  = 16000,   /* 16K samplerate*/
    AUDIO_SAMPLE_RATE_22050  = 22050,   /* 22.050K samplerate*/
    AUDIO_SAMPLE_RATE_24000  = 24000,   /* 24K samplerate*/
    AUDIO_SAMPLE_RATE_32000  = 32000,   /* 32K samplerate*/
    AUDIO_SAMPLE_RATE_44100  = 44100,   /* 44.1K samplerate*/
    AUDIO_SAMPLE_RATE_48000  = 48000,   /* 48K samplerate*/
    AUDIO_SAMPLE_RATE_64000  = 64000,   /* 64K samplerate*/
    AUDIO_SAMPLE_RATE_96000  = 96000,   /* 96K samplerate*/
    AUDIO_SAMPLE_RATE_BUTT,
} AUDIO_SAMPLE_RATE_E;
```

4.20 AUDIO_TRACK_MODE_E

【Описание】

Тип режима каналов аудиоустройства.

【Определение】

```
typedef enum rkAUDIO_TRACK_MODE_E {
    AUDIO_TRACK_NORMAL      = 0,
    AUDIO_TRACK_BOTH_LEFT   = 1,
    AUDIO_TRACK_BOTH_RIGHT  = 2,
    AUDIO_TRACK_EXCHANGE    = 3,
    AUDIO_TRACK_MIX         = 4,
    AUDIO_TRACK_LEFT_MUTE   = 5,
    AUDIO_TRACK_RIGHT_MUTE  = 6,
    AUDIO_TRACK_BOTH_MUTE   = 7,

    AUDIO_TRACK_BUTT
} AUDIO_TRACK_MODE_E;
```

【Члены】

Имя члена	Описание
AUDIO_TRACK_NORMAL	Обычный режим (по умолчанию), без обработки.
AUDIO_TRACK_BOTH_LEFT	Оба канала воспроизводят звук левого канала.
AUDIO_TRACK_BOTH_RIGHT	Оба канала воспроизводят звук правого канала.
AUDIO_TRACK_EXCHANGE	Данные левого и правого каналов меняются местами: левый воспроизводит правый, правый — левый.
AUDIO_TRACK_MIX	Оба канала выводят сумму левого и правого каналов (микширование).
AUDIO_TRACK_LEFT_MUTE	Левый канал отключён, правый воспроизводит исходный правый канал.
AUDIO_TRACK_RIGHT_MUTE	Правый канал отключён, левый воспроизводит исходный левый канал.
AUDIO_TRACK_BOTH_MUTE	Оба канала отключены.

4.21 AUDIO_FADE_RATE_E

【Описание】

Определяет тип скорости плавного появления/затухания устройства аудиовыхода.

【Определение】

```
typedef enum rkAUDIO_FADE_RATE_E {
    AUDIO_FADE_RATE_1   = 0,
    AUDIO_FADE_RATE_2   = 1,
    AUDIO_FADE_RATE_4   = 2,
    AUDIO_FADE_RATE_8   = 3,
    AUDIO_FADE_RATE_16  = 4,
    AUDIO_FADE_RATE_32  = 5,
    AUDIO_FADE_RATE_64  = 6,
    AUDIO_FADE_RATE_128 = 7,

    AUDIO_FADE_RATE_BUTT
} AUDIO_FADE_RATE_E;
```

【Члены】

Имя члена	Описание
AUDIO_FADE_RATE_1	Изменение после каждого 1 сэмпла.
AUDIO_FADE_RATE_2	Изменение после каждых 2 сэмплов.
AUDIO_FADE_RATE_4	Изменение после каждых 4 сэмплов.
AUDIO_FADE_RATE_8	Изменение после каждых 8 сэмплов.
AUDIO_FADE_RATE_16	Изменение после каждых 16 сэмплов.
AUDIO_FADE_RATE_32	Изменение после каждых 32 сэмплов.
AUDIO_FADE_RATE_64	Изменение после каждых 64 сэмплов.
AUDIO_FADE_RATE_128	Изменение после каждых 128 сэмплов.

4.22 AUDIO_FADE_S

【Описание】

Структура настройки плавного появления/затухания устройства аудиовыхода.

【Определение】

```
typedef struct rkAUDIO_FADE_S {
    RK_BOOL         bFade;
    AUDIO_FADE_RATE_E enFadeInRate;
    AUDIO_FADE_RATE_E enFadeOutRate;
} AUDIO_FADE_S;
```

【Члены】

Имя члена	Описание
bFade	Включение плавного появления/затухания. RK_TRUE: включить. RK_FALSE: отключить.
enFadeInRate	Скорость плавного увеличения громкости устройства аудиовыхода.
enFadeOutRate	Скорость плавного уменьшения громкости устройства аудиовыхода.

4.23 AUDIO_CHN_MODE_E

【Описание】

Тип режима аудиоканала.

【Определение】

```
typedef enum rkAUDIO_CHN_MODE_E {
    AUDIO_CHN_MODE_LEFT   = 10,
    AUDIO_CHN_MODE_RIGHT  = 11,
    AUDIO_CHN_MODE_BUTT
} AUDIO_CHN_MODE_E;
```

【Члены】

Имя члена	Описание
AUDIO_CHN_MODE_LEFT	Независимый вывод канала в левый канал.
AUDIO_CHN_MODE_RIGHT	Независимый вывод канала в правый канал.

4.24 AO_CHN_PARAM_S

【Описание】

Структура параметров канала аудиовыхода.

【Определение】

```
typedef struct rkAO_CHN_PARAM_S {
    AUDIO_CHN_MODE_E enMode;
} AO_CHN_PARAM_S;
```

【Члены】

Имя члена	Описание
enMode	Режим независимого вывода канала AO.

4.25 AUDIO_SAVE_FILE_INFO_S

【Описание】

Определяет структуру настроек функции сохранения аудио в файл.

【Определение】

```
typedef struct rkAUDIO_SAVE_FILE_INFO_S {
    RK_BOOL     bCfg;
    RK_CHAR     aFilePath[MAX_AUDIO_FILE_PATH_LEN];
    RK_CHAR     aFileName[MAX_AUDIO_FILE_NAME_LEN];
    RK_U32      u32FileSize;  /*in KB*/
} AUDIO_SAVE_FILE_INFO_S;
```

【Члены】

Имя члена	Описание
bCfg	Переключатель включения настройки.
aFilePath	Путь сохранения аудиофайла.
aFileName	Имя сохраняемого аудиофайла.
u32FileSize	Размер файла, в байтах.

4.26 AUDIO_FILE_STATUS_S

【Описание】

Определяет структуру состояния сохранения аудиофайла.

【Определение】

```
typedef struct rkAUDIO_FILE_STATUS_S {
    RK_BOOL     bSaving;
} AUDIO_FILE_STATUS_S;
```

【Члены】

Имя члена	Описание
bSaving	Находится ли файл в процессе сохранения.

4.27 AI_AED_CONFIG_S

【Описание】

Определяет структуру атрибутов AED.

【Определение】

```
typedef struct rkAI_AED_CONFIG_S {
    RK_FLOAT    fSnrDB;
    RK_FLOAT    fLsdDB;
    RK_S32      s32Policy;
} AI_AED_CONFIG_S;
```

【Члены】

Имя члена	Описание
fSnrDB	Порог отношения сигнал/шум речи; при превышении выводится 1.
fLsdDB	Порог очень громкого звука; при превышении выводится 1. Максимум 0 дБ.
s32Policy	Чувствительность алгоритма определения отношения сигнал/шум. Диапазон: [0, 2]. Чем больше значение, тем выше чувствительность и легче достигается порог. По умолчанию 1.

4.28 AI_AED_RESULT_S

【Описание】

Определяет структуру результата обнаружения события AED.

【Определение】

```
typedef struct rkAI_BCD_RESULT_S {
    RK_BOOL     bAcousticEventDetected;
    RK_BOOL     bLoudSoundDetected;
} AI_AED_RESULT_S;
```

【Члены】

Имя члена	Описание
bAcousticEventDetected	Результат обнаружения события отношения сигнал/шум речи: 1 — порог достигнут, 0 — не достигнут.
bLoudSoundDetected	Результат обнаружения очень громкого звука: 1 — порог достигнут, 0 — не достигнут.
lsdResult	Уровень громкости, полученный при обнаружении очень громкого звука, в дБ.

4.29 AI_BCD_CONFIG_S

【Описание】

Определяет структуру атрибутов BCD.

【Определение】

```
typedef struct rkAI_BCD_CONFIG_S {
    int     mFrameLen;
} AI_BCD_CONFIG_S;
```

【Члены】

Имя члена	Описание
mFrameLen	Длина подтверждения обнаружения плача в кадрах. Значение должно быть меньше 125; рекомендуется 100. Чем больше длина, тем выше задержка и вероятность пропуска; чем меньше — тем выше вероятность ложного срабатывания.

4.30 AI_BCD_RESULT_S

【Описание】

Определяет структуру результата обнаружения события BCD.

【Определение】

```
typedef struct rkAI_BCD_RESULT_S {
    RK_BOOL    bBabyCry;
} AI_BCD_RESULT_S;
```

【Члены】

Имя члена	Описание
bBabyCry	Результат обнаружения плача ребёнка: 1 — плач обнаружен, 0 — не обнаружен.

4.31 AI_BUZ_CONFIG_S

【Описание】

Определяет структуру атрибутов BUZ.

【Определение】

```
typedef struct rkAI_BUZ_CONFIG_S {
    int    mFrameLen;
} AI_BUZ_CONFIG_S;
```

【Члены】

Имя члена	Описание
mFrameLen	Длина подтверждения обнаружения сигнала зуммера в кадрах. Значение должно быть меньше 125; рекомендуется 100. Чем больше длина, тем выше задержка и вероятность пропуска; чем меньше — тем выше вероятность ложного срабатывания.

4.32 AI_BUZ_RESULT_S

【Описание】

Определяет структуру результата обнаружения события BUZ.

【Определение】

```
typedef struct rkAI_BUZ_RESULT_S {
    RK_BOOL    bBuzz;
} AI_BUZ_RESULT_S;
```

【Члены】

Имя члена	Описание
bBuzz	Результат обнаружения сигнала зуммера: 1 — сигнал обнаружен, 0 — не обнаружен.

4.33 AI_GBS_CONFIG_S

【Описание】

Определяет структуру атрибутов GBS.

【Определение】

```
typedef struct rkAI_GBS_CONFIG_S {
    int    mFrameLen;
} AI_GBS_CONFIG_S;
```

【Члены】

Имя члена	Описание
mFrameLen	Длина подтверждения обнаружения разбития стекла в кадрах. Значение должно быть меньше 125 и больше 20; рекомендуется 25–50. Чем больше длина, тем выше задержка и вероятность пропуска; чем меньше — тем выше вероятность ложного срабатывания.

4.34 AI_GBS_RESULT_S

【Описание】

Определяет структуру результата обнаружения события GBS.

【Определение】

```
typedef struct rkAI_GBS_RESULT_S {
    RK_BOOL    bGbs;
} AI_GBS_RESULT_S;
```

【Члены】

Имя члена	Описание
bGbs	Результат обнаружения разбития стекла: 1 — звук разбития обнаружен, 0 — не обнаружен.

4.35 Аудиокодирование и аудиодекодирование

Связанные с аудиокодированием и декодированием типы данных и структуры определены ниже:

- ADEC_CHN: определяет канал ADEC.
- ADEC_MAX_CHN_NUM: определяет максимальное количество каналов аудиодекодирования.
- ADEC_MODE_E: определяет режим декодирования.
- ADEC_ATTR_CODEC_S: определяет структуру атрибутов аудиодекодера.
- ADEC_CHN_ATTR_S: определяет структуру атрибутов канала декодирования.
- ADEC_DECODER_S: определяет структуру атрибутов декодера.
- AUDIO_FRAME_INFO_S: определяет структуру информации о декодированном аудиокадре.
- ADEC_CHN_STATE_S: определяет структуру состояния буфера данных канала аудиодекодирования.
- AENC_CHN: определяет канал AENC.
- AENC_MAX_CHN_NUM: определяет максимальное количество каналов аудиокодирования.
- AENC_ATTR_CODEC_S: определяет структуру атрибутов аудиокодера.
- AENC_CHN_ATTR_S: определяет структуру атрибутов канала кодирования.
- AENC_ENCODER_S: определяет структуру атрибутов кодера.
- AUDIO_STREAM_S: определяет структуру аудиопотока.
- AUDIO_ADENC_PARAM_S: определяет структуру обмена данными функций аудиокодирования/декодирования.

4.36 ADEC_CHN

【Описание】

Определяет канал ADEC.

【Определение】

```
typedef RK_S32 ADEC_CHN;
```

4.37 ADEC_MAX_CHN_NUM

【Описание】

Определяет максимальное количество каналов аудиодекодирования.

【Определение】

```
#define  ADEC_MAX_CHN_NUM  32
```

4.38 ADEC_MODE_E

【Описание】

Определяет режим декодирования.

【Определение】

```
typedef enum rkADEC_MODE_E {
    /*
     * require input is valid dec pack(a complete frame encode result),
     * e.g.the stream get from AENC is a valid dec pack, the stream know
     * actually pack len from file is also a dec pack.
     * this mode is high-performative*/
    ADEC_MODE_PACK = 0,
    /*
     * input is stream,low-performative, if you couldn't find out whether
     * a stream is vaild dec pack,you could use this mode
     */
    ADEC_MODE_STREAM,
    ADEC_MODE_BUTT
} ADEC_MODE_E;
```

【Члены】

Имя члена	Описание
ADEC_MODE_PACK	Декодирование в режиме pack.
ADEC_MODE_STREAM	Декодирование в режиме stream.

【Примечания】

- Режим pack применяется, когда пользователь уверен, что текущий пакет потока содержит результат кодирования одного кадра. Декодер сразу декодирует его; если пакет не является полным кадром, декодер завершится ошибкой. Этот режим эффективнее. Неповреждённые пакеты потока, созданные модулем AENC, можно декодировать таким способом.
- Режим stream применяется, когда пользователь не уверен, содержит ли текущий пакет полный кадр. Декодер анализирует и буферизует поток; этот режим менее эффективен и обычно используется при чтении потока из файла или неизвестных границах пакетов. Поскольку длина закодированного речевого кадра фиксирована, границы кадра в потоке обычно легко определить; рекомендуется режим pack.

4.39 AUDIO_G726_BPS

【Описание】

Определяет битрейт G726.

```
typedef enum rkAUDIO_G726_BPS_E {
    G726_BPS_16K   = 16000,     // 2bit
    G726_BPS_24K   = 24000,     // 3bit
    G726_BPS_32K   = 32000,     // 4bit
    G726_BPS_40K   = 40000,     // 5bit
    G726_BPS_BUTT,
} AUDIO_G726_BPS;
```

【Члены】

Имя члена	Описание
G726_BPS_16K	Битрейт входных/выходных данных 16000. Соответствует кодовому слову G726 2 бита.
G726_BPS_24K	Битрейт входных/выходных данных 24000. Соответствует кодовому слову G726 3 бита.
G726_BPS_32K	Битрейт входных/выходных данных 32000. Соответствует кодовому слову G726 4 бита.
G726_BPS_40K	Битрейт входных/выходных данных 40000. Соответствует кодовому слову G726 5 бит.

4.40 ADEC_ATTR_CODEC_S

【Описание】

Определяет структуру атрибутов аудиодекодера.

【Определение】

```
typedef struct rkADEC_ATTR_CODEC_S {
    RK_CODEC_ID_E  enType;
    RK_U32         u32Channels;
    RK_U32         u32SampleRate;
    RK_U32         u32Bitrate;

    RK_VOID       *pExtraData;
    RK_U32         u32ExtraDataSize;

    RK_U32         u32Resv[4];
    RK_VOID       *pstResv;
} ADEC_ATTR_CODEC_S;
```

【Члены】

Имя члена	Описание
enType	Протокол декодирования, совпадает с enType в ADEC_CHN_ATTR_S.
u32Channels	Количество каналов потока.
u32SampleRate	Частота дискретизации потока.
u32Bitrate	Битрейт аудиоданных.
pExtraData	Дополнительные данные декодера.
u32ExtraDataSize	Длина дополнительных данных декодера.
u32Resv[4]	Зарезервированные байты для расширения параметров декодера.
pstResv	Указатель на зарезервированную структуру для расширения параметров декодера.

【Примечания】

- Если полей структуры недостаточно для передачи параметров декодера, пользователь может расширить их через зарезервированные байты u32Resv и указатель pstResv.
- Для отдельных декодеров необходимо задавать поддерживаемый текущим аудиопротоколом битрейт, например для G726. При инициализации G726 необходимо согласно битрейту задать правильное кодовое слово (codeword); см. AUDIO_G726_BPS.

4.41 ADEC_CHN_ATTR_S

【Описание】

Определяет структуру атрибутов канала декодирования.

【Определение】

```
typedef struct rkADEC_CH_ATTR_S {
    RK_CODEC_ID_E     enType;
    ADEC_MODE_E       enMode;
    RK_U32            u32BufCount;
    RK_U32            u32BufSize;

    ADEC_ATTR_CODEC_S stCodecAttr;
} ADEC_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
enType	Протокол декодирования.
enMode	Режим декодирования; см. ADEC_MODE_E.
u32BufCount	Количество буферов декодирования.
u32BufSize	Размер буфера декодирования, в байтах.
stCodecAttr	Структура атрибутов декодера.

【Примечания】

- u32BufSize задаёт размер buffer декодера. При использовании зарегистрированного декодера размер pu8Outbuf в функции pfnDecodeFrm равен значению u32BufSize. Если значение слишком мало, результат одного декодирования придётся считывать несколько раз. Если значение не задано (равно 0), ADEC использует размер buffer по умолчанию 4096 байт.

【Связанные типы данных и интерфейсы】

RK_MPI_ADEC_CreateChn

4.42 ADEC_DECODER_S

【Описание】

Определяет структуру декодера.

【Определение】

```
typedef struct rkADEC_DECODER_S {
    RK_CODEC_ID_E enType;
    RK_UCHAR aszName[17];
    RK_S32 (*pfnOpenDecoder)(RK_VOID *pDecoderAttr, RK_VOID **ppDecoder);
    RK_S32 (*pfnDecodeFrm)(RK_VOID *pDecoder, RK_VOID *pDecParam);
    RK_S32 (*pfnGetFrmInfo)(RK_VOID *pDecoder, RK_VOID *pInfo);
    RK_S32 (*pfnCloseDecoder)(RK_VOID *pDecoder);
    RK_S32 (*pfnResetDecoder)(RK_VOID *pDecoder);
} ADEC_DECODER_S;
```

【Члены】

Имя члена	Описание
enType	Тип протокола декодирования.
aszName	Имя декодера.
pfnOpenDecoder	Указатель на функцию открытия декодера.
pfnDecodeFrm	Указатель на функцию декодирования.
pfnGetFrmInfo	Указатель на функцию получения информации об аудиокадре.
pfnCloseDecoder	Указатель на функцию закрытия декодера.
pfnResetDecoder	Очищает buffer и сбрасывает декодер.

【Примечания】

- Первый параметр pDecoderAttr функции pfnOpenDecoder имеет тип ADEC_ATTR_CODEC_S. Пользователь может выполнить принудительное приведение типа в соответствующей зарегистрированной функции и получить доступ к полям ADEC_ATTR_CODEC_S.
- Второй параметр pDecParam функции pfnDecodeFrm имеет тип AUDIO_ADENC_PARAM_S. Пользователь может выполнить принудительное приведение типа в соответствующей зарегистрированной функции и получить доступ к полям AUDIO_ADENC_PARAM_S.

【Связанные типы данных и интерфейсы】

RK_MPI_ADEC_RegisterDecoder

4.43 ADEC_DECODER_RESULT

【Описание】

Определяет возвращаемые значения функции декодирования зарегистрированного декодера.

【Определение】

```
typedef enum rkADEC_DECODER_RESULT {
    ADEC_DECODER_OK = RK_SUCCESS,
    ADEC_DECODER_TRY_AGAIN,
    ADEC_DECODER_ERROR,
    ADEC_DECODER_EOS,
} ADEC_DECODER_RESULT;
```

【Члены】

Имя члена	Описание
ADEC_DECODER_OK	Декодирование успешно: данные успешно переданы и декодированные данные получены.
ADEC_DECODER_TRY_AGAIN	Повторить декодирование.
ADEC_DECODER_ERROR	Ошибка декодирования.
ADEC_DECODER_EOS	Декодирован последний кадр.

Примечания

- ADEC_DECODER_TRY_AGAIN: используется, если текущие данные не удалось передать декодеру (например, внутренний буфер декодера заполнен и новые данные принять нельзя) либо после передачи потока декодированные данные не получены. Получив это значение, модуль ADEC повторно передаёт текущий поток, если выходное значение u32InLen структуры AUDIO_ADENC_PARAM_S не равно 0.
- ADEC_DECODER_ERROR: ошибка декодирования. Получив это значение, ADEC отбрасывает текущий кадр.
- ADEC_DECODER_EOS: уведомляет модуль ADEC, что декодер выдал последний кадр и текущее декодирование завершено. Получив это значение, модуль ADEC больше не вызывает функции декодера и помечает текущий аудиокадр как EOS.

4.44 ADEC_FRAME_INFO_S

Описание

Определяет информацию об аудиокадре зарегистрированного декодера.

Определение

```
typedef struct rkADEC_FRAME_INFO_S {
    RK_U32          u32SampleRate;
    RK_U32          u32Channels;
    RK_U64          u64ChnLayout;
    AUDIO_BIT_WIDTH_E enBitWidth;
    RK_U32          resv[2];
} ADEC_FRAME_INFO_S;
```

Члены

Имя поля	Описание
u32SampleRate	Частота дискретизации декодированных данных.
u32Channels	Количество каналов декодированных данных.
u64ChnLayout	Схема каналов декодированных данных.
enBitWidth	Разрядность декодированных данных.
resv[2]	Зарезервированные поля для расширения.

4.45 AUDIO_FRAME_INFO_S

Описание

Определяет структуру информации о декодированном аудиокадре.

Определение

```
typedef struct rkAUDIO_FRAME_INFO_S {
    AUDIO_FRAME_S *pstFrame;/*frame ptr*/
    RK_U32          u32Id;   /*frame id*/
} AUDIO_FRAME_INFO_S;
```

Члены

Имя члена	Описание
pstFrame	Указатель на аудиокадр.
u32Id	Индекс аудиокадра.

4.46 ADEC_CHN_STATE_S

【Описание】

Определяет структуру состояния буфера данных канала аудиодекодирования.

【Определение】

```
typedef struct rkADEC_CH_STATE_S {
    RK_BOOL bEndOfStream;       /* EOS flag */
    RK_U32 u32BufferFrmNum;     /* total number of channel buffer */
    RK_U32 u32BufferFreeNum;    /* free number of channel buffer */
    RK_U32 u32BufferBusyNum;    /* busy number of channel buffer */
} ADEC_CHN_STATE_S;
```

【Члены】

Имя члена	Описание
bEndOfStream	Состояние окончания декодируемого потока.
u32BufferFrmNum	Общее количество блоков буфера канала декодирования.
u32BufferFreeNum	Количество свободных блоков буфера.
u32BufferBusyNum	Количество занятых блоков буфера.

4.47 AENC_CHN

【Описание】

Определяет канал AENC.

【Определение】

```
typedef RK_S32 AENC_CHN;
```

4.48 AENC_MAX_CHN_NUM

【Описание】

Определяет максимальное количество каналов аудиодекодирования.

【Определение】

```
#define  AENC_MAX_CHN_NUM  32
```

4.49 AENC_ATTR_CODEC_S

【Описание】

Определяет структуру атрибутов аудиокодера.

【Определение】

```
typedef struct rkAENC_ATTR_CODEC_S {
    RK_CODEC_ID_E     enType;
    AUDIO_BIT_WIDTH_E enBitwidth;
    RK_U32            u32Channels;
    RK_U32            u32SampleRate;
    RK_U32            u32Bitrate;
    RK_U32            u32Resv[4];
    RK_VOID          *pstResv;
} AENC_ATTR_CODEC_S;
```

【Члены】

Имя члена	Описание
enType	Тип протокола кодирования, совпадает с enType в AENC_CHN_ATTR_S.
enBitwidth	Разрядность аудиосэмпла.
u32Channels	Количество аудиоканалов.
u32SampleRate	Частота дискретизации аудио.
u32Bitrate;	Выходной битрейт аудиопотока.
u32Resv[4]	Зарезервированные байты для расширения параметров кодера.
pstResv	Указатель на зарезервированную структуру для расширения параметров кодера.

【Примечания】

- Для некоторых кодеров, например G726 (см. AUDIO_G726_BPS), необходимо задавать битрейт, поддерживаемый текущим аудиопротоколом, для управления битрейтом закодированного потока.

4.50 AENC_CHN_ATTR_S

【Описание】

Определяет структуру атрибутов канала кодирования.

【Определение】

```
typedef struct rkAENC_CHN_ATTR_S {
    RK_CODEC_ID_E     enType;
    RK_U32            u32BufCount;

    AENC_ATTR_CODEC_S stCodecAttr;
} AENC_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
enType	Тип протокола кодирования.
u32BufCount	Количество буферов аудиокодирования.
stCodecAttr	Структура атрибутов кодера.

4.51 AENC_ENCODER_S

【Описание】

Определяет структуру декодера.

【Определение】

```
typedef struct rkAENC_ENCODER_S {
    RK_CODEC_ID_E enType;
    RK_U32 u32MaxFrmLen;
    RK_CHAR aszName[17];
    RK_S32 (*pfnOpenEncoder)(RK_VOID *pEncoderAttr, RK_VOID **ppEncoder);
    RK_S32 (*pfnEncodeFrm)(RK_VOID *pEncoder, RK_VOID *pParam);
    RK_S32 (*pfnCloseEncoder)(RK_VOID *pEncoder);
} AENC_ENCODER_S;
```

【Члены】

Имя члена	Описание
enType	Тип протокола кодирования.
u32MaxFrmLen	Максимальная длина потока.
aszName	Имя кодера.
pfnOpenEncoder	Указатель на функцию открытия кодера.
pfnEncodeFrm	Указатель на функцию кодирования.
pfnCloseEncoder	Указатель на функцию закрытия кодера.

【Примечания】

- Первый параметр pEncoderAttr функции pfnOpenEncoder имеет тип AENC_ATTR_CODEC_S. Пользователь может выполнить принудительное приведение типа в соответствующей зарегистрированной функции и получить доступ к полям AENC_ATTR_CODEC_S.
- Второй параметр pParam функции pfnEncodeFrm имеет тип AUDIO_ADENC_PARAM_S. Пользователь может выполнить принудительное приведение типа в соответствующей зарегистрированной функции и получить доступ к полям AUDIO_ADENC_PARAM_S.

【Связанные типы данных и интерфейсы】

RK_MPI_AENC_RegisterEncoder

4.52 AUDIO_ADENC_PARAM_S

【Описание】

Определяет структуру входных и выходных данных функций зарегистрированного декодера/кодера.

【Определение】

```
typedef struct rkAUDIO_ADENC_PARAM_S {
    RK_U8          *pu8InBuf;
    RK_U32          u32InLen;
    RK_U64          u64InTimeStamp;

    RK_U8          *pu8OutBuf;
    RK_U32          u32OutLen;
    RK_U64          u64OutTimeStamp;
} AUDIO_ADENC_PARAM_S;
```

【Члены】

Имя поля	Описание	Вход/выход
pu8InBuf	Указатель на buffer входных данных.	Вход
u32InLen	Вход: длина действительных входных данных в pu8InBuf. Выход: длина оставшихся данных во входном buffer pu8InBuf.	Вход/выход
u64InTimeStamp	Временная метка входных данных.	Вход
pu8OutBuf	Указатель на buffer выходных данных.	Вход
u32OutLen	Вход: максимальная ёмкость выходного buffer. Выход: длина действительных данных в выходном buffer.	Вход/выход
u64OutTimeStamp	Временная метка выходных данных.	Выход

4.53 AENC_ENCODER_RESULT

【Описание】

Определяет возвращаемые значения функции кодирования зарегистрированного кодера.

【Определение】

```
typedef enum rkENC_ENCODER_RESULT {
    AENC_ENCODER_OK = RK_SUCCESS,
    AENC_ENCODER_TRY_AGAIN,
    AENC_ENCODER_ERROR,
    AENC_ENCODER_EOS,
} AENC_ENCODER_RESULT;
```

【Члены】

Имя члена	Описание
AENC_ENCODER_OK	Кодирование успешно: входные данные приняты и закодированный поток получен.
AENC_ENCODER_TRY_AGAIN	Повторить кодирование.
AENC_ENCODER_ERROR	Ошибка кодирования.
AENC_ENCODER_EOS	Закодирован последний кадр.

【Примечания】

- AENC_ENCODER_TRY_AGAIN: используется, если текущий кадр не удалось передать либо после его передачи поток не получен. Получив это значение, модуль AENC повторно передаёт текущие данные, если выходное значение u32InLen структуры AUDIO_ADENC_PARAM_S не равно 0.
- AENC_ENCODER_ERROR: получив это значение, модуль AENC отбрасывает текущие входные и выходные данные.
- AENC_ENCODER_EOS: уведомляет модуль AENC, что кодер выдал последний кадр и текущее кодирование завершено. Получив это значение, модуль AENC больше не вызывает функции кодера и помечает текущий аудиокадр как EOS.

5. Коды ошибок AUDIO

5.1 Коды ошибок аудиовхода AUDIO

Коды ошибок API аудиовхода AUDIO приведены ниже.

Код ошибки	Макроопределение	Описание
0xA00A8001	RK_ERR_AI_INVALID_DEVID	Недопустимый номер устройства аудиовхода.
0xA00A8002	RK_ERR_AI_INVALID_CHNID	Недопустимый номер канала аудиовхода.
0xA00A8003	RK_ERR_AI_ILLEGAL_PARAM	Недопустимые параметры аудиовхода.
0xA00A8005	RK_ERR_AI_NOT_ENABLED	Устройство или канал аудиовхода не включён.
0xA00A8006	RK_ERR_AI_NULL_PTR	Передан нулевой указатель.
0xA00A8007	RK_ERR_AI_NOT_CONFIG	Атрибуты устройства аудиовхода не настроены.
0xA00A8008	RK_ERR_AI_NOT_SUPPORT	Операция не поддерживается.
0xA00A8009	RK_ERR_AI_NOT_PERM	Операция не разрешена.
0xA00A800C	RK_ERR_AI_NOMEM	Не удалось выделить память.
0xA00A800D	RK_ERR_AI_NOBUF	Недостаточно буферов аудиовхода.
0xA00A800E	RK_ERR_AI_BUF_EMPTY	Буфер аудиовхода пуст.
0xA00A800F	RK_ERR_AI_BUF_FULL	Буфер аудиовхода заполнен.
0xA00A8010	RK_ERR_AI_SYS_NOTREADY	Система аудиовхода не инициализирована.
0xA00A8012	RK_ERR_AI_BUSY	Система аудиовхода занята.
0xA00A8041	RK_ERR_AI_VQE_ERR	Ошибка обработки AO VQE.

5.2 Коды ошибок аудиовыхода AUDIO

Коды ошибок API аудиовыхода AUDIO приведены ниже.

Код ошибки	Макроопределение	Описание
0xA00B8001	RK_ERR_AO_INVALID_DEVID	Недопустимый номер устройства аудиовыхода.
0xA00B8002	RK_ERR_AO_INVALID_CHNID	Недопустимый номер канала аудиовыхода.
0xA00B8003	RK_ERR_AO_ILLEGAL_PARAM	Недопустимые параметры аудиовыхода.
0xA00B8005	RK_ERR_AO_NOT_ENABLED	Устройство или канал аудиовыхода не включён.
0xA00B8006	RK_ERR_AO_NULL_PTR	Передан нулевой указатель.
0xA00B8007	RK_ERR_AO_NOT_CONFIG	Атрибуты устройства аудиовыхода не настроены.
0xA00B8008	RK_ERR_AO_NOT_SUPPORT	Операция не поддерживается.
0xA00B8009	RK_ERR_AO_NOT_PERM	Операция не разрешена.
0xA00B800C	RK_ERR_AO_NOMEM	Недостаточно системной памяти.
0xA00B800D	RK_ERR_AO_NOBUF	Недостаточно буферов аудиовыхода.
0xA00B800E	RK_ERR_AO_BUF_EMPTY	Буфер аудиовыхода пуст.
0xA00B800F	RK_ERR_AO_BUF_FULL	Буфер аудиовыхода заполнен.
0xA00B8010	RK_ERR_AO_SYS_NOTREADY	Система аудиовыхода не инициализирована.
0xA00B8012	RK_ERR_AO_BUSY	Система аудиовыхода занята.
0xA00B8041	RK_ERR_AO_VQE_ERR	Ошибка обработки AO VQE.

5.3 Коды ошибок аудиодекодирования AUDIO

Коды ошибок API аудиодекодирования AUDIO приведены ниже.

Код ошибки	Макроопределение	Описание
0xA00D8001	RK_ERR_ADEC_INVALID_DEVID	Недопустимый номер устройства аудиодекодирования.
0xA00D8002	RK_ERR_ADEC_INVALID_CHNID	Недопустимый номер канала аудиодекодирования.
0xA00D8003	RK_ERR_ADEC_ILLEGAL_PARAM	Недопустимые параметры аудиодекодирования.
0xA00D8004	RK_ERR_ADEC_EXIST	Канал аудиодекодирования уже используется.
0xA00D8005	RK_ERR_ADEC_UNEXIST	Канал аудиодекодирования не создан.
0xA00D8006	RK_ERR_ADEC_NULL_PTR	Во входных параметрах передан нулевой указатель.
0xA00D8007	RK_ERR_ADEC_NOT_CONFIG	Атрибуты аудиодекодирования не настроены.
0xA00D8008	RK_ERR_ADEC_NOT_SUPPORT	Операция не поддерживается.
0xA00D8009	RK_ERR_ADEC_NOT_PERM	Операция не разрешена.
0xA00D800C	RK_ERR_ADEC_NOMEM	Не удалось выделить память для декодирования.
0xA00D800D	RK_ERR_ADEC_NOBUF	Не удалось выделить буфер канала декодирования.
0xA00D800E	RK_ERR_ADEC_BUF_EMPTY	Buffer аудиодекодирования пуст.
0xA00D800F	RK_ERR_ADEC_BUF_FULL	Буфер аудиодекодирования заполнен.
0xA00D8010	RK_ERR_ADEC_SYS_NOTREADY	Система аудиодекодирования не инициализирована.
0xA00D8040	RK_ERR_ADEC_DECODER_ERR	Ошибка данных аудиодекодирования.
0xA00D8041	RK_ERR_ADEC_BUF_LACK	Недостаточно места во входном буфере аудиодекодирования.
0xA00D8042	RK_ERR_ADEC_REGISTER_ERR	Ошибка регистрации или отмены регистрации аудиодекодера.

5.4 Коды ошибок аудиокодирования AUDIO

Коды ошибок API аудиодекодирования AUDIO приведены ниже.

Код ошибки	Макроопределение	Описание
0xA00C8001	RK_ERR_AENC_INVALID_DEVID	Недопустимый номер аудиоустройства.
0xA00C8002	RK_ERR_AENC_INVALID_CHNID	Недопустимый номер канала аудиокодирования.
0xA00C8003	RK_ERR_AENC_ILLEGAL_PARAM	Недопустимые параметры аудиокодирования.
0xA00C8004	RK_ERR_AENC_EXIST	Канал аудиокодирования уже создан.
0xA00C8005	RK_ERR_AENC_UNEXIST	Канал аудиокодирования не создан.
0xA00C8006	RK_ERR_AENC_NULL_PTR	Во входных параметрах передан нулевой указатель.
0xA00C8007	RK_ERR_AENC_NOT_CONFIG	Канал кодирования не настроен.
0xA00C8008	RK_ERR_AENC_NOT_SUPPORT	Операция не поддерживается.
0xA00C8009	RK_ERR_AENC_NOT_PERM	Операция не разрешена.
0xA00C800C	RK_ERR_AENC_NOMEM	Недостаточно системной памяти.
0xA00C800D	RK_ERR_AENC_NOBUF	Не удалось выделить буфер канала кодирования.
0xA00C800E	RK_ERR_AENC_BUF_EMPTY	Буфер канала кодирования пуст.
0xA00C800F	RK_ERR_AENC_BUF_FULL	Буфер аудиовыхода заполнен.
0xA00C8010	RK_ERR_AENC_SYS_NOTREADY	Система не инициализирована.
0xA00C8040	RK_ERR_AENC_ENCODER_ERR	Ошибка данных аудиокодирования.
0xA00C8041	RK_ERR_AENC_VQE_ERR	Ошибка обработки AENC VQE.
0xA00C8042	RK_ERR_AENC_REGISTER_ERR	Ошибка регистрации или отмены регистрации аудиокодера.

6. Приложение A: описание API регистрации декодеров/кодеров

ADEC и AENC предоставляют открытые интерфейсы регистрации для интеграции сторонних протоколов кодирования/декодирования. Пример кода находится в соответствующем каталоге rockit.

Следующие API из пакета SDK используются для регистрации и отмены регистрации кодеров и декодеров.

6.1 Регистрация декодера

RK_MPI_ADEC_RegisterDecoder: регистрирует декодер.

RK_MPI_ADEC_UnRegisterDecoder: отменяет регистрацию декодера.

ADEC_DECODER_S: определяет структуру атрибутов декодера.

Атрибуты декодера включают тип и имя декодера, указатели на функции открытия, декодирования, получения информации об аудиокадре и закрытия декодера.

- Тип декодера: SDK обозначает протоколы декодирования перечислением; при регистрации следует выбрать тип соответствующего протокола.
- Имя декодера: строка, используемая для отображения информации.
- Указатель на функцию открытия декодера:

  ```
  RK_S32 (*pfnOpenDecoder)(RK_VOID *pDecoderAttr, RK_VOID **ppDecoder);
  ```

  Функция открывает декодер. Первый параметр — атрибуты декодера ADEC_ATTR_CODEC_S. Разным реализациям декодеров могут требоваться разные параметры. Если полей ADEC_ATTR_CODEC_S недостаточно, заказчик может расширить зарезервированные u32Resv[4] или указатель pstResv для передачи необходимых параметров. При инициализации разным декодерам требуются разные данные, поэтому пользователь согласно задаче и фактическому потоку заполняет часть или все поля ADEC_ATTR_CODEC_S. Второй параметр — дескриптор декодера, возвращающий объект для управления декодером. Этот дескриптор также используется при декодировании и закрытии. Возвращаемое значение 0 означает успешное открытие, ненулевое — ошибку.

Возвращаемое значение: 0 — успех, не 0 — ошибка.

- Указатель на функцию декодирования

  RK_S32 (*pfnDecodeFrm)(RK_VOID *pEncoder, RK_VOID *pParam);

  Функция декодирует данные. Первый параметр — дескриптор, возвращённый pfnOpenDecoder при открытии декодера; второй — AUDIO_ADENC_PARAM_S. Ниже кратко описана структура AUDIO_ADENC_PARAM_S:

  pu8InBuf: начальный адрес потока для декодирования.

  u32InLen: вход — действительная длина потока для декодирования.

  Выход — длина оставшегося потока; когда все входные данные использованы, значение задаётся равным 0.

  u64InTimeStamp: временная метка потока для декодирования.

  pu8OutBuf: начальный адрес для сохранения выходного декодированного кадра.

  u32OutLen: вход — ёмкость буфера декодированных данных. Размер задаётся u32BufSize структуры ADEC_CHN_ATTR_S; если значение не задано, размер по умолчанию

  равен 4096 байт. Поскольку этот buffer хранит декодированные данные, если его недостаточно для результата одного декодирования, данные одного декодирования

  придётся копировать несколькими частями, что снижает эффективность. Поэтому при достаточной памяти рекомендуется задать u32BufSize структуры ADEC_CHN_ATTR_S

  достаточно большим.

  Выход — действительная длина декодированных данных.

  u64OutTimeStamp: временная метка декодированных данных.

  Следует подчеркнуть, что модуль ADEC поддерживает только непланарные данные. Если декодированные данные имеют planar-формат, перед возвратом в модуль ADEC необходимо преобразовать их формат.

  Возвращаемое значение: см. ADEC_DECODER_RESULT.

- Указатель на функцию получения информации об аудиокадре

  RK_S32 (*pfnGetFrmInfo)(RK_VOID *pDecoder, RK_VOID *pInfo);

  Функция получает параметры аудиокадра. Первый параметр — дескриптор декодера, возвращённый pfnOpenDecoder; второй — ADEC_FRAME_INFO_S, через который модуль ADEC получает параметры данных, декодированных pfnEncodeFrm.

  Возвращаемое значение: 0 — успех, не 0 — ошибка.

- Указатель на функцию закрытия декодера

  RK_S32 (*pfnCloseDecoder)(RK_VOID *pDecoder);

  Функция закрывает декодер. Входной параметр — дескриптор, возвращённый pfnOpenDecoder при открытии.

  Возвращаемое значение: 0 — успех, не 0 — ошибка.

- Указатель на функцию сброса декодера

  RK_S32 (*pfnResetDecoder)(RK_VOID *pDecoder);

  Функция сбрасывает декодер. Входной параметр — дескриптор, возвращённый pfnOpenDecoder. Если пользовательскому декодеру такая функция не требуется, можно предоставить пустую функцию с соответствующей сигнатурой.

  Возвращаемое значение: 0 — успех, не 0 — ошибка.

Пример

Ниже приведён код регистрации декодера G711a:

```
RK_S32 register_ext_adec(TEST_ADEC_CTX_S *params) {
    ADEC_DECODER_S adecCtx;
    memset(&adecCtx, 0, sizeof(ADEC_DECODER_S));
    // set handle to default value
    params->s32ExtCodecHandle = -1;
    adecCtx.enType = RK_AUDIO_ID_PCM_ALAW;
    snprintf(reinterpret_cast<RK_CHAR*>(adecCtx.aszName),
                              sizeof(adecCtx.aszName), "ext_g711a");

    adecCtx.pfnOpenDecoder  = RKMpiAdecFFmpegDecoderDemo::open;
    adecCtx.pfnDecodeFrm    = RKMpiAdecFFmpegDecoderDemo::decode;
    adecCtx.pfnGetFrmInfo   = RKMpiAdecFFmpegDecoderDemo::getFrameInfo;
    adecCtx.pfnCloseDecoder = RKMpiAdecFFmpegDecoderDemo::close;
```

```
    adecCtx.pfnResetDecoder = RKMpiAdecFFmpegDecoderDemo::reset;

    RK_LOGD("register_ext_adec, name = %s", adecCtx.aszName);
    RK_S32 ret = RK_MPI_ADEC_RegisterDecoder(&params->s32ExtCodecHandle, &adecCtx);
    if (ret != RK_SUCCESS) {
        RK_LOGE("adec register decoder fail, ret = 0x%x", ret);
        return RK_FAILURE;
    }

    return RK_SUCCESS;
}
```

Отмена регистрации декодера G711a.

```
RK_S32 unregister_ext_adec(TEST_ADEC_CTX_S *params) {
    if (params == NULL || params->s32ExtCodecHandle == -1) {
        return RK_SUCCESS;
    }

    RK_S32 ret = RK_MPI_ADEC_UnRegisterDecoder(params->s32ExtCodecHandle);
    if (ret != RK_SUCCESS) {
        RK_LOGE("adec unregister decoder fail, ret = 0x%x", ret);
        return RK_FAILURE;
    }

    params->s32ExtCodecHandle = -1;
    return RK_SUCCESS;
}
```

Пример использования каналом ADEC chn0 декодера G711a для потока G711a 8 кГц, 1 канал:

```
RK_S32 s32ret  = 0;
RK_S32 adecChn = 0;
ADEC_CHN_ATTR_S stChnAttr;
memset(&stChnAttr, 0, sizeof(ADEC_CHN_ATTR_S));
// init ADEC_CHN_ATTR_S
stChnAttr.enType      = RK_AUDIO_ID_PCM_ALAW;   // G711a
stChnAttr.enMode      = ADEC_MODE_PACK;
stChnAttr.u32BufCount = 4;
stChnAttr.u32BufSize  = 10*1024;  // 10K bytes
// init ADEC_ATTR_CODEC_S
stChnAttr.stCodecAttr.enType = stChnAttr.enType;
stChnAttr.stCodecAttr.u32SampleRate = 8000;
stChnAttr.stCodecAttr.u32Channels   = 1;

s32ret = RK_MPI_ADEC_CreateChn(adecChn, &stChnAttr);
if (s32ret != 0) {
    RK_LOGE("RK_MPI_ADEC_CreateChn(chn = %d) fail", adecChn);
}

return s32ret;
```

Примечания

- Разным реализациям декодеров могут требоваться разные поля структуры ADEC_ATTR_CODEC_S. Если существующих полей недостаточно, настройки можно передать через зарезервированные u32Resv и указатель pstResv.
- Поле u32BufSize структуры ADEC_CHN_ATTR_S задаёт размер выходного буфера между модулем ADEC и pfnDecodeFrm, в байтах. По умолчанию он равен 4096 байт. Если буфер меньше результата одного декодирования, pfnDecodeFrm возвращает только часть данных, поэтому во избежание потери перед следующим декодированием необходимо сначала вывести оставшиеся данные. Если возможно, рекомендуется задать u32BufSize достаточно большим. достаточно большим.

6.2 Регистрация кодера

RK_MPI_AENC_RegisterEncoder: регистрирует кодер.

RK_MPI_AENC_UnRegisterEncoder: отменяет регистрацию кодера.

AENC_ENCODER_S: структура атрибутов кодера.

Атрибуты кодера включают тип кодера, максимальную длину потока, имя и указатели на функции открытия, кодирования и закрытия кодера.

- Тип кодера

  SDK обозначает протоколы кодирования перечислением; при регистрации следует выбрать тип соответствующего протокола.

- Максимальная длина потока

  Максимальная длина закодированного потока одного кадра. Модуль AENC выделяет память согласно зарегистрированному значению. Если пользователь его не задал, используется значение по умолчанию.

  4096bytes。

- Имя кодера: строка, используемая для отображения информации.

- Указатель на функцию открытия кодера

  RK_S32 (*pfnOpenEncoder)(RK_VOID *pEncoderAttr, RK_VOID **ppEncoder);

  Первый параметр — атрибуты кодера AENC_ATTR_CODEC_S для передачи специфических параметров разных типов кодеров. При необходимости пользователь может расширить u32Resv и pstResv. Второй параметр — дескриптор кодера, возвращающий объект для управления кодером.

  Разные протоколы и реализации кодеров предъявляют разные требования к входным данным. Например, отдельный протокол может поддерживать только данные 16 кГц, 1 канал, 16 бит; некоторые протоколы — только данные определённой длины или количества кадров. Поэтому настоятельно рекомендуется за один раз передавать модулю AENC данные с частотой, количеством каналов и кадров, соответствующими требованиям кодера. Если входные данные этим требованиям не соответствуют, перед передачей в AENC их необходимо обработать, например выполнить передискретизацию.

  Возвращаемое значение: 0 — успех, не 0 — ошибка.

- Указатель на функцию кодирования

  RK_S32 (*pfnEncodeFrm)(RK_VOID *pEncoder, RK_VOID *pParam);

  Первый параметр — дескриптор, возвращённый pfnOpenEncoder; второй — AUDIO_ADENC_PARAM_S для передачи входных и выходных данных аудиокадра.

  Ниже кратко описана структура AUDIO_ADENC_PARAM_S:

  pu8InBuf: начальный адрес данных для кодирования.

  u32InLen: вход — действительная длина данных для кодирования.

  Выход — длина оставшихся данных; когда все входные данные использованы, значение задаётся равным 0.

  u64InTimeStamp: временная метка данных для кодирования.

  pu8OutBuf: начальный адрес для сохранения закодированного потока.

  u32OutLen: вход — ёмкость буфера потока; размер задаётся максимальной длиной потока.

  Выход — действительная длина закодированного потока.

  u64OutTimeStamp: временная метка закодированного потока.

  Возвращаемое значение: см. AENC_ENCODER_RESULT.

- Указатель на функцию закрытия кодера

  RK_S32 (*pfnCloseEncoder)(RK_VOID *pEncoder);

  Входной параметр — дескриптор, возвращённый pfnOpenEncoder при открытии кодера.

  Возвращаемое значение: 0 — успех, не 0 — ошибка.

Пример

Ниже приведён код регистрации кодера G711a:

```
RK_S32 register_ext_aenc(TEST_AENC_CTX_S *params) {
    AENC_ENCODER_S aencCtx;
    memset(&aencCtx, 0, sizeof(AENC_ENCODER_S));
    params->s32ExtCodecHandle = -1;
```

```
    aencCtx.enType = RK_AUDIO_ID_PCM_ALAW;
    snprintf(reinterpret_cast<RK_CHAR*>(aencCtx.aszName),
                              sizeof(aencCtx.aszName), "ext_g711a");
    aencCtx.u32MaxFrmLen     = 8192;
    aencCtx.pfnOpenEncoder   = RKMpiAEncFFmpegEncoderDemo::open;
    aencCtx.pfnEncodeFrm     = RKMpiAEncFFmpegEncoderDemo::encode;
    aencCtx.pfnCloseEncoder  = RKMpiAEncFFmpegEncoderDemo::close;

    RK_LOGD("register_ext_aenc %s", aencCtx.aszName);
    RK_S32 ret = RK_MPI_AENC_RegisterEncoder(&params->s32ExtCodecHandle, &aencCtx);
    if (ret != RK_SUCCESS) {
        RK_LOGE("aenc %s register decoder fail", aencCtx.aszName, ret);
        return RK_FAILURE;
    }

    return RK_SUCCESS;
}
```

Отмена регистрации кодера G711a.

```
RK_S32 unregister_ext_aenc(TEST_AENC_CTX_S *params) {
    if (params == NULL || params->s32ExtCodecHandle == -1) {
        return RK_SUCCESS;
    }
    RK_LOGD("unregister_ext_aenc");
    RK_S32 ret = RK_MPI_AENC_UnRegisterEncoder(params->s32ExtCodecHandle);
    if (ret != RK_SUCCESS) {
        RK_LOGE("aenc unregister decoder fail", ret);
        return RK_FAILURE;
    }

    params->s32ExtCodecHandle = -1;
    return RK_SUCCESS;
}
```

Пример использования каналом AENC chn0 кодера G711a для кодирования потока G711a с входными данными 8 кГц, 1 канал, 16 бит:

```
RK_S32 s32ret  = 0;
RK_S32 aencChn = 0;
AENC_CHN_ATTR_S stChnAttr;
memset(&stChnAttr, 0, sizeof(AENC_CHN_ATTR_S));
// init AENC_CHN_ATTR_S
stChnAttr.enType      = RK_AUDIO_ID_PCM_ALAW;   // G711a
stChnAttr.u32BufCount = 4;

// init AENC_ATTR_CODEC_S
stChnAttr.stCodecAttr.enType        = stChnAttr.enType;
stChnAttr.stCodecAttr.enBitwidth    = AUDIO_BIT_WIDTH_16;
stChnAttr.stCodecAttr.u32SampleRate = 8000;
stChnAttr.stCodecAttr.u32Channels   = 1;
s32ret = RK_MPI_AENC_CreateChn(aencChn, &stChnAttr);
if (s32ret) {
    RK_LOGE("create aenc chn %d err:0x%x\n", aencChn, s32ret);
}

return s32ret;
```

Управление областями

Предисловие

Обзор

Пользователю обычно требуется накладывать на видео OSD для отображения определённой информации, например номера канала и временной метки, а при необходимости — цветные блоки. Такие наложенные OSD и закрывающие видео цветные области совместно называются регионами. Модуль REGION предназначен для централизованного управления этими ресурсами.

Управление регионами позволяет создавать области, накладывать их на видео или закрывать участки видео. Например, в практическом приложении пользователь создаёт область и через RK_MPI_RGN_AttachToChn накладывает эту область на определённый канал, например VENC. При обработке канала OSD накладывается на видео. Одну область можно назначить нескольким каналам, например нескольким VENC, через интерфейс настройки атрибутов отображения канала; при этом для каждого канала могут быть заданы разные атрибуты отображения, такие как положение, слой и прозрачность.

Версии продуктов

Название микросхемы	Версия ядра
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	XLM	2021-02-02	Первая версия.
V0.2.0	XLM	2021-07-28	Добавлена защита QP для overlay и дополнительные пояснения.
V0.3.0	XLM	2021-07-29	Добавлен интерфейс непосредственного рисования.
V0.4.0	LXH	2021-10-02	Добавлено описание настройки холста.
V0.5.0	XLM	2021-08-09	Добавлена поддержка VPSS.
V0.6.0	LXH	2022-04-30	Добавлено описание layer для OVERLAY и COVER.
V0.9.0	ZQX	2023-02-16	Добавлено описание типа OVERLAY_EX.

Содержание

Важные понятия
Описание порядка использования
Пример
Справочник API
Типы данных
Коды ошибок RGN

1. Важные понятия

- Типы областей
  - OVERLAY: область наложения на видео, поддерживающая загрузку растрового изображения, наложение слоёв и другие функции.
    - Рисунок 1-1. Результат RGN OVERLAY



  - COVER: область закрытия видео сплошным цветным блоком.
    - Рисунок 1-2. Результат RGN COVER



  - MOSAIC: область мозаичного закрытия с настройкой размера блока.
    - Рисунок 1-3. Результат RGN MOSAIC



- LINE: расширенная область наложения линии на видео, поддерживающая настройку цвета, толщины и других атрибутов.
  - Рисунок 1-4. Результат RGN LINE



- Уровень области Уровень области определяет порядок наложения. Чем больше значение, тем выше уровень отображения. При пересечении область с большим значением перекрывает область с меньшим значением.

- Заполнение растровым изображением (действительно для OVERLAY) Заполнение растровым изображением означает копирование его содержимого в память области начиная с верхнего левого угла. Если изображение меньше области, заполняется только часть памяти, а остаток сохраняет прежние значения; если размеры равны, заполняется вся область; если изображение больше области, в область копируется только часть размером с область.

- Общие атрибуты области (RGN_ATTR_S) При создании области пользователь задаёт эти атрибуты, содержащие общие сведения о ресурсе. Например, для OVERLAY — формат пикселей и размер.

- Атрибуты отображения канала (RGN_CHN_ATTR_S) Атрибуты отображения канала определяют, как область показывается в конкретном канале. Например, для OVERLAY они включают положение, уровень, Alpha переднего плана и Alpha фона. Если поле is_show в атрибутах равно TRUE, область отображается в этом канале; иначе она существует в канале, но скрыта.

- Защита QP области

  При наложении области на видео с последующим кодированием, чтобы её чёткость не ухудшалась из-за сжатия, можно отдельно задать характеристики сжатия наложенной части, то есть параметры защиты qp. Защита qp доступна только для OVERLAY и только в каналах кодирования H.264/H.265; для других типов она недействительна.

1.1 Таблица 1-1. Модули RK356X, поддерживающие region

Тип	Поддерживаемый модуль	Диапазон номера устройства	Диапазон номера канала
OVERLAY	VENC	0	[0, VENC_MAX_CHN_NUM - 1]
	VO	[0, VO_MAX_LAYER_NUM - 1]	[0, VO_MAX_CHN_NUM - 1]
	VPSS	[0, VPSS_MAX_GRP_NUM - 1]	[0, VPSS_MAX_CHN_NUM - 1]
COVER	VENC	0	[0, VENC_MAX_CHN_NUM - 1]
	VO	[0, VO_MAX_LAYER_NUM - 1]	[0, VO_MAX_CHN_NUM - 1]
	VPSS	[0, VPSS_MAX_GRP_NUM - 1]	[0, VPSS_MAX_CHN_NUM - 1]
MOSAIC	VPSS	[0, VPSS_MAX_GRP_NUM - 1]	[0, VPSS_MAX_CHN_NUM - 1]
	VO	[0, VO_MAX_LAYER_NUM - 1]	[0, VO_MAX_CHN_NUM - 1]
LINE	VPSS	[0, VPSS_MAX_GRP_NUM - 1]	[0, VPSS_MAX_CHN_NUM - 1]

- Функции, поддерживаемые областями

  Функции, поддерживаемые различными типами областей, приведены в таблице 1-2.

1.2 Таблица 1-2. Функции region, поддерживаемые RK356X

Поддерживаемые функции	**OVERLAY**			**COVER**			**MOSAIC**		**LINE**
Поддерживаемые модуля	**VO**	**VENC**	**VPSS**	**VO**	**VENC**	**VPSS**	**VO**	**VPSS**	**VPSS**
Формат пикселей	BGRA8888、BGRA5551	BGRA8888、BGRA5551、BGRA4444	BGRA8888、BGRA5551、NV12	RGB888	RGB888	RGB888	N/A	N/A	RGB888
Уровень наложения	Поддерживается	Поддерживается	Поддерживается	Поддерживается	Поддерживается	Поддерживается	Поддерживается	Поддерживается	N/A
Заполнение изображением	Поддерживается	Поддерживается	Поддерживается	N/A	N/A	N/A	N/A	N/A	N/A
Уровень прозрачности	Поддерживается	Не поддерживается	Поддерживается	N/A	N/A	N/A	N/A	N/A	N/A
Передний план alpha, диапазон	0~255	Не поддерживается	0~255	N/A	N/A	N/A	N/A	N/A	N/A
Фон план alpha, диапазон	0~255	Не поддерживается	0~255	N/A	N/A	N/A	N/A	N/A	N/A
Инверсия цвета	Не поддерживается	Не поддерживается	Не поддерживается	N/A	N/A	N/A	N/A	N/A	N/A
QP Защита	N/A	Поддерживается	N/A	N/A	N/A	N/A	N/A	N/A	N/A

2. Описание порядка использования

2.1 Порядок использования OVERLAY

Процесс включает следующие шаги:

- Шаг 1: вызвать RK_MPI_RGN_Create, заполнить атрибуты и создать область.
- Шаг 2: вызвать RK_MPI_RGN_AttachToChn и привязать холст к указанной области канала.
- Шаг 3: вызвать RK_MPI_RGN_GetCanvasInfo и получить сведения о холсте.
- Шаг 4: записать растровые данные в память холста.
- Шаг 5: вызвать RK_MPI_RGN_UpdateCanvas и обновить холст.
- Для последующих обновлений повторять шаги 3–5.
- Шаг 6: когда область больше не нужна, вызвать RK_MPI_RGN_DetachFromChn и отвязать холст от канала.
- Шаг 7: вызвать RK_MPI_RGN_Destroy и уничтожить область.

2.2 Порядок использования COVER

Процесс включает следующие шаги:

- Шаг 1: вызвать RK_MPI_RGN_Create, заполнить атрибуты закрывающей области и создать её.
- Шаг 2: вызвать RK_MPI_RGN_AttachToChn и привязать холст к указанной области канала.
- Шаг 3: вызвать RK_MPI_RGN_GetDisplayAttr и получить сведения о закрывающей области.
- Шаг 4: изменить требуемые параметры, например положение, и вызвать RK_MPI_RGN_SetDisplayAttr для обновления.
- Для последующих обновлений повторять шаги 3–4.
- Шаг 5: когда область больше не нужна, вызвать RK_MPI_RGN_DetachFromChn и отвязать холст от канала.
- Шаг 6: вызвать RK_MPI_RGN_Destroy и уничтожить область.

3. Пример

```
coverHandle = 0;
stCoverAttr.enType = COVER_RGN;
s32Ret = RK_MPI_RGN_Create(coverHandle, &stCoverAttr);
if (RK_SUCCESS != s32Ret) {
    RK_LOGE("failed with %#x!", s32Ret);
    return RK_FAILURE;
}
stCoverChn.enModId = RK_ID_VENC;
stCoverChn.s32ChnId = 0;
stCoverChn.s32DevId = vencChn;

stCoverChnAttr.bShow = RK_TRUE;
stCoverChnAttr.enType = COVER_RGN;
stCoverChnAttr.unChnAttr.stCoverChn.enCoverType = AREA_RECT;
stCoverChnAttr.unChnAttr.stCoverChn.stRect.s32X = pstRgnCtx->stRegion.s32X;
stCoverChnAttr.unChnAttr.stCoverChn.stRect.s32Y = pstRgnCtx->stRegion.s32Y;
stCoverChnAttr.unChnAttr.stCoverChn.stRect.u32Width = pstRgnCtx->stRegion.u32Width;
stCoverChnAttr.unChnAttr.stCoverChn.stRect.u32Height = pstRgnCtx->stRegion.u32Height;
stCoverChnAttr.unChnAttr.stCoverChn.u32Color = 0xffffffff;
stCoverChnAttr.unChnAttr.stCoverChn.u32Layer = 0;
s32Ret = RK_MPI_RGN_AttachToChn(coverHandle, &stCoverChn, &stCoverChnAttr);
if (RK_SUCCESS != s32Ret) {
    RK_LOGE("failed with %#x!", s32Ret);
    goto AttachCover_failed;
}
stCoverChnAttr.unChnAttr.stCoverChn.stRect.s32X = 64;
stCoverChnAttr.unChnAttr.stCoverChn.stRect.s32Y = 64;
stCoverChnAttr.unChnAttr.stCoverChn.stRect.u32Width = 256;
```

```
stCoverChnAttr.unChnAttr.stCoverChn.stRect.u32Height = 256;
stCoverChnAttr.unChnAttr.stCoverChn.u32Color = 0x00f800;
stCoverChnAttr.unChnAttr.stCoverChn.u32Layer = 1;
s32Ret = RK_MPI_RGN_SetDisplayAttr(coverHandle, &stCoverChn, &stCoverChnAttr);
if (RK_SUCCESS != s32Ret) {
    RK_LOGE("failed with %#x!", s32Ret);
    goto exit;
}

RK_MPI_RGN_DetachFromChn(coverHandle, &stCoverChn);
RK_MPI_RGN_Destroy(coverHandle);
```

Примечания

- Нет

4. Справочник API

Модуль управления областями предоставляет функции управления ресурсами областей: создание, уничтожение, получение и настройку атрибутов, а также получение и настройку атрибутов отображения в канале.

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_RGN_Create: создаёт область.
- RK_MPI_RGN_Destroy: уничтожает область.
- RK_MPI_RGN_GetAttr: получает атрибуты области.
- RK_MPI_RGN_SetAttr: задаёт атрибуты области.
- RK_MPI_RGN_SetBitMap: задаёт растровое изображение области.
- RK_MPI_RGN_AttachToChn: накладывает область на канал.
- RK_MPI_RGN_DetachFromChn: удаляет область из канала.
- RK_MPI_RGN_SetDisplayAttr: задаёт атрибуты отображения области в канале.
- RK_MPI_RGN_GetDisplayAttr: получает атрибуты отображения области в канале.
- RK_MPI_RGN_GetCanvasInfo: получает сведения о холсте отображения области.
- RK_MPI_RGN_UpdateCanvas: обновляет сведения о холсте отображения области.

4.1 RK_MPI_RGN_Create

Описание

Создаёт область.

Синтаксис

RK_S32 RK_MPI_RGN_Create(RGN_HANDLE Handle, const RGN_ATTR_S *pstRegion);

Параметры

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Должен быть неиспользуемым номером handle. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Ввод
pstRegion	Указатель на атрибуты области.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

【Требования】

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

【Примечание】

- Дескриптор задаётся пользователем и эквивалентен идентификатору ID.
- Повторное создание не поддерживается.
- Атрибуты области должны быть допустимыми; ограничения см. в RGN_ATTR_S.
- Указатель на атрибуты области не должен быть NULL.
- При создании COVER, MOSAIC или LINE достаточно указать тип области. Остальные атрибуты, например положение и уровень, задаются при вызове RK_MPI_RGN_AttachToChn.
- При создании области интерфейс выполняет только базовую проверку параметров, например минимальной и максимальной ширины/высоты. При attach к каналу выполняется более точная проверка с учётом ограничений конкретного модуля канала, например поддерживаемых форматов пикселей.

【Связанные темы】

- RK_MPI_RGN_Destroy
- RK_MPI_RGN_GetAttr
- RK_MPI_RGN_SetAttr

4.2 RK_MPI_RGN_Destroy

【Описание】

Уничтожает область.

【Синтаксис】

RK_S32 RK_MPI_RGN_Destroy(RGN_HANDLE Handle);

【Параметры】

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

【Требования】

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

【Примечание】

- Область должна быть предварительно создана.
- Во время вызова этого интерфейса нельзя одновременно вызывать RK_MPI_RGN_SetAttr или RK_MPI_RGN_SetBitMap.

【Связанные темы】

- Нет

4.3 RK_MPI_RGN_GetAttr

【Описание】

Получает атрибуты области.

【Синтаксис】

RK_S32 RK_MPI_RGN_GetAttr(RGN_HANDLE Handle, RGN_ATTR_S *pstRegion);

【Параметры】

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход
pstRegion	Указатель на атрибуты области.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

【Требования】

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

【Примечание】

- Область должна быть предварительно создана.
- Указатель на атрибуты области не должен быть NULL.

【Связанные темы】

- Нет

4.4 RK_MPI_RGN_SetAttr

【Описание】

Задаёт атрибуты области.

【Синтаксис】

RK_S32 RK_MPI_RGN_SetAttr(RGN_HANDLE Handle, const RGN_ATTR_S *pstRegion);

【Параметры】

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход
pstRegion	Указатель на атрибуты области.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

【Требования】

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

【Примечание】

- Область должна быть предварительно создана.
- Указатель на атрибуты области не должен быть NULL.
- Во время вызова этого интерфейса нельзя одновременно вызывать RK_MPI_RGN_Destroy.
- COVER, MOSAIC и LINE этот интерфейс не поддерживают.
- Если после RK_MPI_RGN_SetBitMap вызвать этот интерфейс и задать новую область меньше прежней, прежняя область будет уничтожена; необходимо повторно вызвать RK_MPI_RGN_SetBitMap и задать растровое изображение.

【Связанные темы】

- Нет

4.5 RK_MPI_RGN_SetBitMap

【Описание】

Задаёт растровое изображение области, то есть заполняет область.

【Синтаксис】

RK_S32 RK_MPI_RGN_SetBitMap(RGN_HANDLE Handle, const BITMAP_S *pstBitmap);

【Параметры】

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход
pstBitmap	Указатель на атрибуты растрового изображения. Подробнее см. раздел «Управление системой».	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

【Требования】

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

【Примечание】

- Область должна быть предварительно создана.
- Размер растрового изображения может отличаться от размера области.
- Изображение загружается начиная с точки (0, 0) области. Если оно больше области, оно автоматически обрезается до её размера.
- Формат пикселей изображения должен совпадать с форматом области.
- Указатель на атрибуты изображения не должен быть NULL.
- Поддерживаются многократные вызовы.
- Интерфейс действителен только для OVERLAY.
- Во время вызова этого интерфейса нельзя одновременно вызывать RK_MPI_RGN_Destroy.

【Связанные темы】

- Нет

4.6 RK_MPI_RGN_AttachToChn

【Описание】

Накладывает область на канал.

【Синтаксис】

RK_S32 RK_MPI_RGN_AttachToChn(RGN_HANDLE Handle, const MPP_CHN_S *pstChn, const RGN_CHN_ATTR_S *pstChnAttr);

【Параметры】

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход
pstChn	Указатель на структуру канала. Подробнее см. раздел «Управление системой».	Вход
pstChnAttr	Указатель на атрибуты отображения области в канале.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

【Требования】

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

【Примечание】

- Область должна быть предварительно создана.
- Указатель на структуру канала не должен быть NULL.
- Указатель на атрибуты отображения области в канале не должен быть NULL.
- Повторное наложение не поддерживается.

【Связанные темы】

- Нет

4.7 RK_MPI_RGN_DetachFromChn

【Описание】

Удаляет область из канала.

【Синтаксис】

RK_S32 RK_MPI_RGN_DetachFromChn(RGN_HANDLE Handle, const MPP_CHN_S *pstChn);

【Параметры】

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход
pstChn	Указатель на структуру канала. Подробнее см. раздел «Управление системой».	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

【Требования】

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

【Примечание】

- Область должна быть предварительно создана.
- Указатель на структуру канала не должен быть NULL.
- Повторные вызовы не поддерживаются.

【Связанные темы】

- Нет

4.8 RK_MPI_RGN_SetDisplayAttr

【Описание】

Задаёт атрибуты отображения области в канале.

【Синтаксис】

RK_S32 RK_MPI_RGN_SetDisplayAttr(RGN_HANDLE Handle, const MPP_CHN_S *pstChn, const RGN_CHN_ATTR_S *pstChnAttr);

【Параметры】

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход
pstChn	Указатель на структуру канала. Подробнее см. раздел «Управление системой».	Вход
pstChnAttr	Указатель на атрибуты отображения области в канале.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

【Требования】

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

【Примечание】

- Область должна быть предварительно создана.
- Рекомендуется сначала получить атрибуты, затем изменить и задать их.
- Указатель на структуру канала не должен быть NULL.
- Указатель на атрибуты отображения области в канале не должен быть NULL.
- Область должна быть предварительно наложена на канал.

【Связанные темы】

- Нет

4.9 RK_MPI_RGN_GetDisplayAttr

【Описание】

Получает атрибуты отображения области в канале.

【Синтаксис】

RK_S32 RK_MPI_RGN_GetDisplayAttr(RGN_HANDLE Handle, const MPP_CHN_S *pstChn, RGN_CHN_ATTR_S *pstChnAttr);

【Параметры】

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход
pstChn	Указатель на структуру канала. Подробнее см. раздел «Управление системой».	Вход
pstChnAttr	Указатель на атрибуты отображения области в канале.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

【Требования】

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

【Примечание】

- Область должна быть предварительно создана.
- Указатель на структуру канала не должен быть NULL.
- Указатель на атрибуты отображения области в канале не должен быть NULL.

【Связанные темы】

- Нет

4.10 RK_MPI_RGN_GetCanvasInfo

【Описание】

Получает сведения о холсте отображения области.

【Синтаксис】

RK_S32 RK_MPI_RGN_GetCanvasInfo(RGN_HANDLE Handle, RGN_CANVAS_INFO_S *pstCanvasInfo);

【Параметры】

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход
RGN_CANVAS_INFO_S	Сведения о холсте отображения области.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

Требования

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

Примечание

- Область должна быть предварительно создана.
- Этот интерфейс аналогичен RK_MPI_RGN_SetBitMap и главным образом используется для загрузки растровых данных overlay. В отличие от RK_MPI_RGN_SetBitMap, он позволяет записывать данные непосредственно в холст, экономя копирование памяти, поэтому рекомендуется этот способ.
- Интерфейс используется совместно с RK_MPI_RGN_UpdateCanvas. После записи данных в холст вызовите RK_MPI_RGN_UpdateCanvas, чтобы фактически обновить холст в модуле, к которому выполнен Attach.
- Интерфейс можно сочетать с RK_MPI_RGN_SetBitMap, но это не рекомендуется.

Связанные темы

- Нет

4.11 RK_MPI_RGN_UpdateCanvas

Описание

Обновляет данные холста отображения области.

Синтаксис

```
RK_S32 RK_MPI_RGN_UpdateCanvas(RGN_HANDLE Handle);
```

Параметры

Имя параметра	Описание	Вход/выход
Handle	Номер дескриптора области. Диапазон: [0, RGN_MAX_HANDLE_NUM).	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; см. коды ошибок RGN.

Требования

- Заголовочные файлы: rk_comm_rgn.h, rk_mpi_rgn.h
- Библиотека: librockit.so

Примечание

- Область должна быть предварительно создана.
- Интерфейс используется совместно с RK_MPI_RGN_GetCanvasInfo и после обновления памяти холста переключает отображаемый холст.
- Перед каждым вызовом необходимо сначала получить сведения о холсте, затем обновлять; иначе изменения не вступят в силу.
- Интерфейс поддерживает только области типа OVERLAY.
- При совместном использовании с RK_MPI_RGN_GetCanvasInfo требуется синхронизация потоков: после RK_MPI_RGN_GetCanvasInfo строго в указанном порядке вызывается этот интерфейс, иначе обновление холста завершится ошибкой.

Связанные темы

- Нет

5.1 RGN_MIN_WIDTH

【Описание】

Определяет минимальную ширину области.

【Определение】

RK356X：

```
#define RGN_MIN_WIDTH                16
```

【Примечание】

- Нет

5.2 RGN_MIN_HEIGHT

【Описание】

Определяет минимальную высоту области.

【Определение】

RK356X：

```
#define RGN_MIN_HEIGHT               16
```

【Примечание】

- Нет

5.3 RGN_COVER_MIN_X

【Описание】

Определяет минимальную горизонтальную координату X закрывающей области.

【Определение】

RK356X：

```
#define RGN_COVER_MIN_X              0
```

【Примечание】

- Нет

5.4 RGN_COVER_MIN_Y

【Описание】

Определяет минимальную вертикальную координату Y закрывающей области.

【Определение】

RK356X：

```
#define RGN_COVER_MIN_Y               0
```

【Примечание】

- Нет

5.5 RGN_COVER_MAX_X

【Описание】

Определяет максимальную горизонтальную координату X закрывающей области.

【Определение】

RK356X：

```
#define RGN_COVER_MAX_X               8192
```

【Примечание】

- Нет

5.6 RGN_COVER_MAX_Y

【Описание】

Определяет минимальную вертикальную координату Y закрывающей области.

【Определение】

RK356X：

```
#define RGN_COVER_MAX_Y               8192
```

【Примечание】

- Нет

5.7 RGN_COVER_MAX_WIDTH

【Описание】

Определяет максимальную ширину закрывающей области.

【Определение】

RK356X：

```
#define RGN_COVER_MAX_WIDTH          8192
```

【Примечание】

- Нет

5.8 RGN_COVER_MAX_HEIGHT

【Описание】

Определяет максимальную высоту закрывающей области.

【Определение】

RK356X：

```
#define RGN_COVER_MAX_HEIGHT         8192
```

【Примечание】

- Нет

5.9 RGN_OVERLAY_MIN_X

【Описание】

Определяет минимальную координату X начального положения области OVERLAY.

【Определение】

RK356X：

```
#define RGN_OVERLAY_MIN_X             0
```

【Примечание】

- Нет

5.10 RGN_OVERLAY_MIN_Y

【Описание】

Определяет минимальную координату Y начального положения области OVERLAY.

【Определение】

RK356X：

```
#define RGN_OVERLAY_MIN_Y             0
```

【Примечание】

- Нет

5.11 RGN_OVERLAY_MAX_X

【Описание】

Определяет максимальную координату X начального положения области OVERLAY.

【Определение】

RK356X：

```
#define RGN_OVERLAY_MAX_X             8192
```

【Примечание】

- Нет

5.12 RGN_OVERLAY_MAX_Y

【Описание】

Определяет максимальную координату Y начального положения области OVERLAY.

【Определение】

RK356X：

```
#define RGN_OVERLAY_MAX_Y             8192
```

【Примечание】

- Нет

5.13 RGN_OVERLAY_MAX_WIDTH

【Описание】

Определяет максимальную ширину области OVERLAY.

【Определение】

RK356X：

```
#define RGN_OVERLAY_MAX_WIDTH         8192
```

【Примечание】

- Нет

5.14 RGN_OVERLAY_MAX_HEIGHT

【Описание】

Определяет максимальную высоту области OVERLAY.

【Определение】

RK356X：

```
#define RGN_OVERLAY_MAX_HEIGHT        8192
```

【Примечание】

- Нет

5.15 RGN_MOSAIC_MIN_X

【Описание】

Определяет минимальную координату X начального положения области мозаики.

【Определение】

RK356X：

```
#define RGN_MOSAIC_MIN_X              0
```

【Примечание】

- Нет

5.16 RGN_MOSAIC_MIN_Y

【Описание】

Определяет минимальную координату Y начального положения области мозаики.

【Определение】

RK356X：

```
#define RGN_MOSAIC_MIN_Y              0
```

【Примечание】

- Нет

5.17 RGN_MOSAIC_MAX_X

【Описание】

Определяет максимальную координату X начального положения области мозаики.

【Определение】

RK356X：

```
#define RGN_MOSAIC_MAX_X              8192
```

【Примечание】

- Нет

5.18 RGN_MOSAIC_MAX_Y

【Описание】

Определяет максимальную координату Y начального положения области мозаики.

【Определение】

RK356X：

```
#define RGN_MOSAIC_MAX_Y              8192
```

【Примечание】

- Нет

5.19 RGN_MOSAIC_MIN_WIDTH

【Описание】

Определяет минимальную ширину области мозаики.

【Определение】

RK356X：

```
#define RGN_MOSAIC_MIN_WIDTH          32
```

【Примечание】

- Нет

5.20 RGN_MOSAIC_MIN_HEIGHT

【Описание】

Определяет минимальную высоту области мозаики.

【Определение】

RK356X：

```
#define RGN_MOSAIC_MIN_HEIGHT          32
```

【Примечание】

- Нет

5.21 RGN_MOSAIC_MAX_WIDTH

【Описание】

Определяет максимальную ширину области мозаики.

【Определение】

RK356X：

```
#define RGN_MOSAIC_MAX_WIDTH           8192
```

【Примечание】

- Нет

5.22 RGN_MOSAIC_MAX_HEIGHT

【Описание】

Определяет максимальную высоту области мозаики.

【Определение】

RK356X：

```
#define RGN_MOSAIC_MAX_HEIGHT          8192
```

【Примечание】

- Нет

5.23 RGN_LINE_MIN_X

【Описание】

Определяет минимальную горизонтальную координату линии.

【Определение】

RK356X：

```
#define RGN_LINE_MIN_X                0
```

【Примечание】

- Нет

5.24 RGN_LINE_MIN_Y

【Описание】

Определяет минимальную вертикальную координату линии.

【Определение】

RK356X：

```
#define RGN_LINE_MIN_Y                0
```

【Примечание】

- Нет

5.25 RGN_LINE_MAX_X

【Описание】

Определяет максимальную горизонтальную координату линии.

【Определение】

RK356X：

```
#define RGN_LINE_MAX_X                8192
```

【Примечание】

- Нет

5.26 RGN_LINE_MAX_Y

【Описание】

Определяет максимальную вертикальную координату линии.

【Определение】

RK356X：

```
#define RGN_LINE_MAX_Y                8192
```

【Примечание】

- Нет

5.27 RGN_LINE_MIN_THICK

【Описание】

Определяет минимальную толщину линии.

【Определение】

RK356X：

```
#define RGN_LINE_MIN_THICK            1
```

【Примечание】

- Нет

5.28 RGN_LINE_MAX_THICK

【Описание】

Определяет максимальную толщину линии.

【Определение】

RK356X：

```
#define RGN_LINE_MAX_THICK            32
```

【Примечание】

- Нет

5.29 RGN_ALIGN

【Описание】

Определяет способ выравнивания области.

【Определение】

RK356X：

```
#define RGN_ALIGN         16
```

【Примечание】

- Нет

5.30 RGN_MAX_HANDLE_NUM

【Описание】

Определяет максимальное количество дескрипторов RGN.

【Определение】

```
#define RGN_MAX_HANDLE_NUM               128
```

【Примечание】

- Нет

5.31 RGN_MAX_BUF_NUM

【Описание】

Определяет максимальное количество холстов RGN.

【Определение】

```
#define RGN_MAX_BUF_NUM                 2
```

【Примечание】

- Нет

5.32 RGN_HANDLE

【Описание】

Определяет дескриптор области.

【Определение】

```
typedef RK_U32 RGN_HANDLE;
```

【Примечание】

- Нет

5.33 RGN_TYPE_E

【Описание】

Определяет тип области.

【Определение】

```
typedef enum rkRGN_TYPE_E {
    OVERLAY_RGN = 0,
    OVERLAY_EX_RGN,
    COVER_RGN,
    MOSAIC_RGN,
    LINE_RGN,
    RGN_BUTT
} RGN_TYPE_E;
```

【Члены】

Имя члена	Описание
OVERLAY_RGN	Область наложения на видео.
OVERLAY_EX_RGN	Область наложения на видео.
COVER_RGN	Область закрытия видео.
MOSAIC_RGN	Мозаичная область видео.
LINE_RGN	Область рисования линии на видео.

【Примечание】

- Нет

5.34 RGN_COORDINATE_E

【Описание】

Определяет тип координат.

【Определение】

```
typedef enum rkRGN_COORDINATE_E {
    RGN_ABS_COOR = 0,
    RGN_RATIO_COOR
} RGN_COORDINATE_E;
```

【Члены】

Имя члена	Описание
RGN_ABS_COOR	Абсолютные координаты.
RGN_RATIO_COOR	Относительные координаты.

【Примечание】

- По умолчанию используются абсолютные координаты.
- В настоящее время относительные координаты поддерживаются только для области COVER, наложенной на VO.

5.35 OVERLAY_ATTR_S

【Описание】

Определяет структуру атрибутов области наложения канала.

【Определение】

```
typedef struct rkOVERLAY_ATTR_S {
    PIXEL_FORMAT_E enPixelFmt;
    SIZE_S stSize;
    RK_U32 u32CanvasNum;
    RK_U32 u32ClutNum;
    RK_U32 u32Clut[RGN_CLUT_NUM];
    VIDEO_PROC_DEV_TYPE_E enVProcDev;
} OVERLAY_ATTR_S;
```

【Члены】

Имя члена	Описание
enPixelFmt	Формат пикселей. Подробнее см. раздел «Управление системой». Диапазон значений: См. таблицу 1-2.
stSize	Ширина и высота области. Диапазон значений: Если overlay привязан к каналу VENC, поддерживаются: ширина [RGN_MIN_WIDTH, RGN_OVERLAY_MAX_WIDTH], с выравниванием по 16; высота [RGN_MIN_HEIGHT, RGN_OVERLAY_MAX_HEIGHT], с выравниванием по 16.
u32CanvasNum	Количество доступных холстов области. Диапазон значений: [1, RGN_MAX_BUF_NUM] Статический атрибут.
u32ClutNum	Количество цветов палитры RGB; в настоящее время действительно только для формата BGRA8888. Диапазон: [0, 255]. 0: использовать системную палитру по умолчанию; другое значение: использовать первые u32ClutNum цветов массива u32Clut как палитру.
u32Clut	Определяет палитру цветов RGB; в настоящее время действительно только для формата BGRA8888. После выбора этой палитры venc overlay ищет цвета среди первых u32ClutNum значений. Диапазон: [0x0, 0xFFFFFFFF]. В 32-битном значении от старших разрядов к младшим расположены прозрачность (alpha, биты 24–31), красный (red, биты 16– 23), зелёный (green, биты 8–15), синий (blue, биты 0–7).
enVProcDev	Тип устройства. Поддерживаются GPU и RGA. В настоящее время используется только для OSD типа OVERLAY_EX, attach к модулю VENC.

【Примечание】

- Если u32CanvasNum меньше 1, значение принудительно устанавливается в 2.
- Если u32CanvasNum больше RGN_MAX_BUF_NUM, устанавливается RGN_MAX_BUF_NUM.
- Рекомендуемое значение u32CanvasNum — 2. При значении 1 во время обновления холст может одновременно использоваться для наложения, что приводит к рассинхронизации данных.
- u32CanvasNum определяется при первой настройке атрибутов и впоследствии не изменяется.
- Если u32ClutNum не равно 0, в массиве u32Clut необходимо определить первые u32ClutNum цветов палитры.
- Если модуль области — VENC и тип — overlay, рекомендуется настроить u32ClutNum и u32Clut и выбрать формат BGRA8888 для палитры.
- Если модуль области — VENC и тип — overlay_ex, необходимо задать тип устройства с учётом микросхемы. RGA поддерживает только ARGB8888, ABGR8888, RGBA8888 и BGRA8888 и не поддерживает динамическое изменение положения OSD.

Название микросхемы	Тип аппаратного устройства
RV1109/RV1126	RGA
RK356X	GPU 、RGA
RK3588	GPU 、RGA

5.36 OVERLAY_CHN_ATTR_S

【Описание】

Определяет атрибуты отображения области наложения в канале.

【Определение】

```
typedef struct rkOVERLAY_CHN_ATTR_S {
    POINT_S stPoint;
    RK_U32 u32FgAlpha;
    RK_U32 u32BgAlpha;
    RK_U32 u32Layer;
    OVERLAY_QP_INFO_S stQpInfo;
} OVERLAY_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
stPoint	Положение области. Диапазон значений: Горизонтальная координата X: [RGN_OVERLAY_MIN_X, RGN_OVERLAY_MAX_X]. Вертикальная координата Y: [RGN_OVERLAY_MIN_Y, RGN_OVERLAY_MAX_Y].
u32FgAlpha	Прозрачность пикселей с битом Alpha, равным 1; также называется Alpha переднего плана. Диапазон: [0, 255]. Чем меньше значение, тем выше прозрачность. Высота изображения.
u32BgAlpha	Прозрачность пикселей с битом Alpha, равным 0; также называется Alpha фона. Диапазон: [0, 255]. Чем меньше значение, тем выше прозрачность. Ширина изображения.
u32Layer	Уровень области. Диапазон: [0, 7]. Чем больше значение, тем выше уровень.
stQpInfo	Значение qp, используемое при кодировании этой области; поддерживается только VENC. Динамический атрибут.

【Примечание】

- Если память области имеет формат RRK_FMT_BGRA5551, значение Alpha расширяется. При бите Alpha 1 для смешивания используется u32FgAlpha; при бите Alpha 0 используется u32BgAlpha. VENC настройку Alpha не поддерживает. 0 означает полную прозрачность, 255 — непрозрачность.
-
- Если модуль области — VENC, OVERLAY и COVER совместно используют u32Layer, то есть суммарно не более 8 слоёв. Значения u32Layer взаимоисключающие: например, если OVERLAY использует слой 0, COVER не может использовать слой 0.
- Для VENC на каждом уровне u32Layer может находиться только одна область OVERLAY; последняя настройка того же уровня заменяет предыдущую. значением.

5.37 COVER_CHN_ATTR_S

【Описание】

Определяет атрибуты отображения закрывающей области в канале.

【Определение】

```
typedef struct rkCOVER_CHN_ATTR_S {
    RECT_S stRect;
    RK_U32 u32Color;
    RK_U32 u32Layer;
    RGN_COORDINATE_E enCoordinate;
} COVER_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
stRect	Положение, ширина и высота области. Диапазон положения: горизонтальная координата X: [RGN_COVER_MIN_X, RGN_COVER_MAX_X], с выравниванием по 16; вертикальная координата Y: [RGN_COVER_MIN_Y, RGN_COVER_MAX_Y], с выравниванием по 16. Диапазон ширины и высоты: ширина [RGN_MIN_WIDTH, RGN_COVER_MAX_WIDTH], с выравниванием по 16; высота [RGN_MIN_HEIGHT, RGN_COVER_MAX_HEIGHT], с выравниванием по 16. Динамический атрибут.
u32Color	Цвет области, заданный как RGB888.
u32Layer	Уровень области. Диапазон: [0, 7]. Динамический атрибут.
enCoordinate	Тип координат области.

【Примечание】

- Если модуль области — VENC, OVERLAY и COVER совместно используют u32Layer, то есть суммарно не более 8 слоёв. Значения u32Layer взаимоисключающие: например, если OVERLAY использует слой 0, COVER не может использовать слой 0.
- Для VENC на каждом уровне u32Layer может находиться только одна область COVER; последняя настройка того же уровня заменяет предыдущую.

5.38 MOSAIC_BLK_SIZE_E

【Описание】

Определяет размер блока типа mosaic.

【Определение】

```
typedef enum rkMOSAIC_BLK_SIZE_E {
    MOSAIC_BLK_SIZE_8 = 0,
    MOSAIC_BLK_SIZE_16,
    MOSAIC_BLK_SIZE_32,
    MOSAIC_BLK_SIZE_64,
    MOSAIC_BLK_SIZE_BUTT
} MOSAIC_BLK_SIZE_E;
```

【Члены】

Имя члена	Описание
MOSAIC_BLK_SIZE_8	Размер 8×8.
MOSAIC_BLK_SIZE_16	Размер 16×16.
MOSAIC_BLK_SIZE_32	Размер 32×32.
MOSAIC_BLK_SIZE_64	Размер 64×64.

【Примечание】

- Нет

5.39 MOSAIC_CHN_ATTR_S

【Описание】

Определяет атрибуты отображения области мозаики в канале.

【Определение】

```
typedef struct rkMOSAIC_CHN_ATTR_S {
    RECT_S stRect;
    MOSAIC_BLK_SIZE_E enBlkSize;
    RK_U32 u32Layer;
} MOSAIC_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
stRect	Область мозаики.
enBlkSize	Тип отображения мозаики.
u32Layer	Уровень области.

【Примечание】

- Нет

5.40 LINE_CHN_ATTR_S

【Описание】

Определяет атрибуты отображения области линии в канале.

【Определение】

```
typedef struct rkLINE_CHN_ATTR_S {
    RK_U32 u32Thick;
    RK_U32 u32Color;
    POINT_S stStartPoint;
    POINT_S stEndPoint;
} LINE_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
u32Thick	Толщина линии. Диапазон: [RGN_LINE_MIN_THICK, RGN_LINE_MAX_THICK].
u32Color	Цвет линии, заданный значением RGB.
stStartPoint	Начальная точка линии.
stEndPoint	Конечная точка линии.

【Примечание】

- Нет

5.41 RGN_ATTR_U

【Описание】

Определяет объединение атрибутов области.

【Определение】

```
typedef union rkRGN_ATTR_U {
    OVERLAY_ATTR_S      stOverlay;
} RGN_ATTR_U;
```

【Члены】

Имя члена	Описание
stOverlay	Атрибуты области наложения канала.

【Примечание】

- Нет

5.42 RGN_CHN_ATTR_U

【Описание】

Определяет объединение атрибутов отображения области в канале.

【Определение】

```
typedef union rkRGN_CHN_ATTR_U {
    OVERLAY_CHN_ATTR_S      stOverlayChn;
    COVER_CHN_ATTR_S        stCoverChn;
    MOSAIC_CHN_ATTR_S       stMosaicChn;
    LINE_CHN_ATTR_S         stLineChn;
} RGN_CHN_ATTR_U;
```

【Члены】

Имя члена	Описание
stOverlayChn	Атрибуты отображения области наложения в канале.
stCoverChn	Атрибуты отображения закрывающей области в канале.
stMosaicChn	Атрибуты отображения мозаики.
stLineChn	Атрибуты отображения области линии.

【Примечание】

- Нет

5.43 RGN_ATTR_S

【Описание】

Определяет структуру атрибутов области.

【Определение】

```
typedef struct rkRGN_ATTR_S {
    RGN_TYPE_E enType;
    RGN_ATTR_U unAttr;
} RGN_ATTR_S;
```

【Члены】

Имя члена	Описание
enType	Тип области.
unAttr	Атрибуты области.

【Примечание】

- Нет

5.44 RGN_CHN_ATTR_S

【Описание】

Определяет атрибуты отображения области мозаики в канале.

【Определение】

```
typedef struct rkRGN_CHN_ATTR_S {
    RK_BOOL           bShow;
    RGN_TYPE_E        enType;
    RGN_CHN_ATTR_U    unChnAttr;
} RGN_CHN_ATTR_S;
```

【Члены】

Имя члена	Описание
bShow	Отображается ли область. Допустимые значения: RK_TRUE или RK_FALSE. Динамический атрибут.
enType	Тип области. Статический атрибут.
unChnAttr	Атрибуты отображения области в канале.

【Примечание】

- Нет

5.45 OVERLAY_QP_INFO_S

【Описание】

Определяет сведения о защите QP области overlay.

【Определение】

```
typedef struct rkOVERLAY_QP_INFO {
    RK_BOOL bEnable;
    RK_BOOL bAbsQp;
    RK_BOOL bForceIntra;
    RK_S32 s32Qp;
} OVERLAY_QP_INFO_S;
```

【Члены】

Имя члена	Описание
bEnable	Включена ли защита QP области. Допустимые значения: RK_TRUE или RK_FALSE. Динамический атрибут.
bAbsQp	Тип защиты QP области. RK_TRUE: абсолютный QP. RK_FALSE: относительный QP — изменение относительно QP области без ROI. Динамический атрибут.
bForceIntra	Принудительно ли кодировать область как Intra-макроблок. Допустимые значения: RK_TRUE или RK_FALSE. Динамический атрибут.
s32Qp	Значение QP области. Диапазон при включённом абсолютном QP: [0, 51]. При относительном QP: [-51, 51]. Динамический атрибут.

【Примечание】

- Структура действует только при наложении изображения на VENC; для других модулей не применяется.

5.46 RGN_CANVAS_INFO_S

【Описание】

Определяет сведения о холсте.

【Определение】

```
typedef struct rkRGN_CANVAS_INFO_S {
    MB_BLK canvasBlk;
    RK_U64 u64VirAddr;
    SIZE_S stSize;
    RK_U32 u32VirWidth;
    RK_U32 u32VirHeight;
    PIXEL_FORMAT_E enPixelFmt;
} RGN_CANVAS_INFO_S;
```

【Члены】

Имя члена	Описание
canvasBlk	Память MB с данными холста.
u64VirAddr	Виртуальный адрес данных холста.
stSize	Размер холста.
u32VirWidth	Виртуальная ширина холста.
u32VirHeight	Виртуальная высота холста.
enPixelFmt	Формат пикселей холста.

【Примечание】

- Нет

6. Коды ошибок RGN

Коды ошибок API управления областями RGN приведены в таблице 1-3.

6.1 Таблица 1-3. Коды ошибок API управления областями RGN

Код ошибки	Макроопределение	Описание
0xA0038001	RK_ERR_RGN_INVALID_DEVID	ID устройства вне допустимого диапазона.
0xA0038002	RK_ERR_RGN_INVALID_CHNID	Недопустимый номер группы каналов или дескриптор области.
0xA0038003	RK_ERR_RGN_ILLEGAL_PARAM	Параметр вне допустимого диапазона.
0xA0038004	RK_ERR_RGN_EXIST	Повторное создание существующего устройства, канала или ресурса.
0xA0038005	RK_ERR_RGN_UNEXIST	Попытка использовать или уничтожить несуществующее устройство, канал или ресурс.
0xA0038006	RK_ERR_RGN_NULL_PTR	В параметрах функции передан нулевой указатель.
0xA0038007	RK_ERR_RGN_NOT_CONFIG	Модуль не настроен.
0xA0038008	RK_ERR_RGN_NOT_SUPPORT	Параметр или функция не поддерживается.
0xA0038009	RK_ERR_RGN_NOT_PERM	Операция не разрешена, например попытка изменить статический параметр конфигурации.
0xA003800C	RK_ERR_RGN_NOMEM	Не удалось выделить память, например недостаточно системной памяти.
0xA003800D	RK_ERR_RGN_NOBUF	Не удалось выделить буфер, например запрошен слишком большой буфер данных.
0xA003800E	RK_ERR_RGN_BUF_EMPTY	В буфере нет данных.
0xA003800F	RK_ERR_RGN_BUF_FULL	Буфер заполнен данными.
0xA0038010	RK_ERR_RGN_NOTREADY	Система не инициализирована или соответствующий модуль не загружен.
0xA0038011	RK_ERR_RGN_BADADDR	Недопустимый адрес.
0xA0038012	RK_ERR_RGN_BUSY	Система занята.

Подсистема коррекции геометрических искажений

Предисловие

Обзор

Подсистема коррекции геометрических искажений (Geometric Distortion Correction, далее GDC) выполняет коррекцию «рыбьего глаза» одного кадра, включая три режима: панорама 360°, панорама 180° и Normal, а также получение информации о цепочке точек.

Версии продуктов

Название микросхемы	Версия ядра
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	LXH	2022-01-26	Первая версия.
V0.2.0	LXH	2022-05-26	Обновлена функция получения панорамных координат по цепочке точек.

Содержание

Основные понятия
Описание функций
Аппаратные характеристики
Справочник API
Типы данных
Коды ошибок GDC

1. Основные понятия

1.1 task

Над одним изображением выполняется одна или несколько конкретных операций, например панорамная коррекция или получение цепочки точек. По умолчанию максимальное число task GDC равно 200.

1.2 job

GDC управляет task через job. Один job может содержать несколько task; GDC гарантирует их однократную передачу аппаратуре в порядке добавления в job.

По умолчанию максимальное число job GDC равно 128.

1.3 HANDLE

Дескриптор задачи, идентифицирующий job.

2. Описание функций

Функции GDC: коррекция «рыбьего глаза» и получение информации о цепочке точек.

2.1 Коррекция «рыбьего глаза»

GDC поддерживает коррекцию «рыбьего глаза» одного изображения. В настоящее время поддерживается только вывод исправленных данных через модуль VO для отображения; прямой вывод обработанных данных GDC не поддерживается.

2.2 Получение информации о цепочке точек

GDC позволяет по координатам точки на исправленном изображении получить координаты исходного изображения — для одной или нескольких точек — и координаты обработанного панорамного изображения, также для одной или нескольких точек.

3. Аппаратные характеристики

3.1 Аппаратные характеристики GDC

3.1.1 Характеристики входных и выходных изображений коррекции «рыбьего глаза»

Микросхема	Поддерживаемые форматы изображения	Ограничения изображения
RK3588	RK_FMT_YUV420SP RK_FMT_YUV420SP_VU RK_FMT_RGB888 RK_FMT_BGR888 RK_FMT_BGRA8888 RK_FMT_RGBA8888	Максимальный размер входного изображения 8192×8192, минимальный 1920×1080; максимальный размер выходного изображения 8192×8192, минимальный 640×360.

Модуль предоставляет следующие MPI:

- RK_MPI_GDC_BeginJob: запускает job.
- RK_MPI_GDC_EndJob: отправляет job; GDC начинает его обработку.
- RK_MPI_GDC_CancelJob: отменяет job.
- RK_MPI_GDC_StopJob: завершает job и освобождает ресурсы всех его task.
- RK_MPI_GDC_SetConfig: задаёт конфигурацию объектива «рыбий глаз».
- RK_MPI_GDC_AddCorrectionTask: добавляет task коррекции «рыбьего глаза» в уже запущенный job.
- RK_MPI_GDC_AddCorrectionExTask: добавляет task коррекции «рыбьего глаза» в уже открытый job, режим XY.
- RK_MPI_GDC_FisheyePosQueryDst2Src: по координате выходного исправленного изображения находит координату исходного изображения.
- RK_MPI_GDC_FisheyePosQueryDst2SrcArray: по массиву координат выходного исправленного изображения находит массив координат исходного изображения.
- RK_MPI_GDC_FisheyePosQueryDst2Pano: по координате выходного исправленного изображения находит координату панорамного изображения.
- RK_MPI_GDC_FisheyePosQueryDst2PanoArray: по массиву координат выходного исправленного изображения находит массив координат панорамного изображения.

4.1 RK_MPI_GDC_BeginJob

【Описание】

Запускает job.

【Синтаксис】

RK_S32 RK_MPI_GDC_BeginJob(GDC_HANDLE *phHandle);

【Параметры】

Имя параметра	Описание	Вход/выход
phHandle	Возвращаемый handle job.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- Одновременно можно запустить несколько job, но HANDLE, возвращаемый через phHandle, разрешается использовать только после успешного возврата RK_MPI_GDC_BeginJob.
- phHandle не должен быть пустым или недействительным указателем.

【Пример】

```
RK_S32 s32Ret = RK_SUCCESS;
GDC_HANDLE hHandle;
GDC_TASK_ATTR_S stTask;
s32Ret = RK_MPI_GDC_BeginJob(&hHandle);
if (s32Ret != RK_SUCCESS) {
    RK_LOGE("RK_MPI_GDC_BeginJob err:0x%x!", s32Ret);
    goto __FAILED;
}
s32Ret = RK_MPI_GDC_AddCorrectionTask(hHandle, &ctx->stTask, &ctx->stFisheyeAttr);
if (s32Ret != RK_SUCCESS) {
    RK_MPI_GDC_CancelJob(hHandle);
    goto __FAILED;
}
s32Ret = RK_MPI_GDC_EndJob(hHandle);
if (s32Ret != RK_SUCCESS) {
    RK_MPI_GDC_CancelJob(hHandle);
```

```
    RK_LOGE("RK_MPI_GDC_EndJob err:0x%x!", s32Ret);
    goto __FAILED;
}
RK_MPI_GDC_StopJob(hHandle);
```

4.2 RK_MPI_GDC_EndJob

【Описание】

Отправляет job на выполнение.

【Синтаксис】

RK_S32 RK_MPI_GDC_EndJob(GDC_HANDLE hHandle);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE уже запущенного job.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- Если интерфейс завершился ошибкой, необходимо вызвать RK_MPI_GDC_CancelJob и отменить job, обозначенный hHandle; иначе ресурсы job, обозначенного hHandle, этот job больше нельзя будет использовать повторно.
- не будут освобождены. Job, обозначенный hHandle, должен быть запущен.

4.3 RK_MPI_GDC_CancelJob

【Описание】

Отменяет job.

【Синтаксис】

RK_S32 RK_MPI_GDC_CancelJob(GDC_HANDLE hHandle);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE уже запущенного job.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- hHandle должен обозначать запущенный job.

4.4 RK_MPI_GDC_StopJob

【Описание】

Завершает job и освобождает ресурсы всех его task.

【Синтаксис】

RK_S32 RK_MPI_GDC_StopJob(GDC_HANDLE hHandle);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE уже запущенного job.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- hHandle должен обозначать запущенный job.
- Когда job больше не используется, вызовите эту функцию для освобождения связанных ресурсов.
- После вызова очищаются ресурсы task данного job и сведения о параметрах task в dumpsys gdc.

4.5 RK_MPI_GDC_SetConfig

【Описание】

Задаёт конфигурацию объектива «рыбий глаз».

【Синтаксис】

RK_S32 RK_MPI_GDC_SetConfig(GDC_HANDLE hHandle, const FISHEYE_JOB_CONFIG_S *pstJobConfig);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE уже запущенного job.	Вход
pstJobConfig Конфигурация объектива «рыбий глаз».		

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- Функция интерфейса пока не реализована.

4.6 RK_MPI_GDC_AddCorrectionTask

【Описание】

Добавляет task коррекции «рыбьего глаза» в уже запущенный job.

【Синтаксис】

RK_S32 RK_MPI_GDC_AddCorrectionTask(GDC_HANDLE hHandle, const GDC_TASK_ATTR_S *pstTask, FISHEYE_ATTR_S *pstFisheyeAttr);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE уже запущенного job.	Вход
pstTask	Указатель на атрибуты task GDC.	Вход
pstFisheyeAttr	Указатель на конфигурацию атрибутов области коррекции FISHEYE.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- Если интерфейс завершился ошибкой, необходимо вызвать RK_MPI_GDC_CancelJob и отменить job, обозначенный hHandle; иначе ресурсы job, обозначенного hHandle, этот job больше нельзя будет использовать повторно.
- hHandle должен обозначать запущенный job.
- Если режим коррекции области FISHEYE_VIEW_MODE_E задан как FISHEYE_NO_TRANSFORMATION, соответствующая область масштабируется согласно конфигурации входного изображения и выходной области. Для отображения масштабированного исходного изображения можно задать этот режим в параметрах области, не используя VGS или другие модули для масштабирования.
- Коррекция «рыбьего глаза» поддерживает три режима: панорама 360°, панорама 180° и normal. Панорама 360° поддерживает напольную и потолочную установку; панорама 180° — настенную; normal — напольную, потолочную и настенную.
- В настоящее время stVFrame.pMbBlk структур VIDEO_FRAME_INFO_S входных и выходных данных можно задать одним и тем же значением и затем передать VO для отображения. Не рекомендуется дополнительно масштабировать этот pMbBlk перед VO. Если требуется масштабирование исходного изображения, задайте режим коррекции области FISHEYE_VIEW_MODE_E равным FISHEYE_NO_TRANSFORMATION.

【Пример】

```
// imgin
ctx->stTask.stImgIn.stVFrame.pMbBlk = RK_MPI_MB_GetMB(ctx->inPool, ctx->u32SrcSize, RK_TRUE);
// imgout
ctx->stTask.stImgOut.stVFrame.pMbBlk = ctx->stTask.stImgIn.stVFrame.pMbBlk;
// read image
pSrcData = RK_MPI_MB_Handle2VirAddr(ctx->stTask.stImgIn.stVFrame.pMbBlk);
s32Ret = read_image(reinterpret_cast<RK_U8 *>(pSrcData), ctx->u32SrcWidth, ctx->u32SrcHeight,
ctx->u32SrcVirWidth, ctx->u32SrcVirHeight, ctx->s32SrcPixFormat, pFile);
s32Ret = RK_MPI_GDC_BeginJob(&hHandle);
if (s32Ret != RK_SUCCESS) {
    RK_LOGE("RK_MPI_GDC_BeginJob err:0x%x!", s32Ret);
    goto __FAILED;
}
s32Ret = RK_MPI_GDC_AddCorrectionTask(hHandle, &ctx->stTask, &ctx->stFisheyeAttr);
if (s32Ret != RK_SUCCESS) {
    RK_MPI_GDC_CancelJob(hHandle);
    RK_LOGE("RK_MPI_GDC_AddCorrectionTask err:0x%x!", s32Ret);
    goto __FAILED;
}
s32Ret = RK_MPI_GDC_EndJob(hHandle);
```

```
if (s32Ret != RK_SUCCESS) {
    RK_MPI_GDC_CancelJob(hHandle);
    RK_LOGE("RK_MPI_GDC_EndJob err:0x%x!", s32Ret);
    goto __FAILED;
}
// gdc out show to vo
stFrame.stVFrame = ctx->stTask.stImgOut.stVFrame;
s32Ret = RK_MPI_VO_SendFrame(ctx->s32VoLayer, ctx->s32VoChn, &stFrame, -1);
if (s32Ret != RK_SUCCESS) {
    RK_LOGE("RK_MPI_VO_SendFrame failed ret:0x%x", s32Ret);
    RK_MPI_VO_DestroyGraphicsFrameBuffer(ctx->stTask.stImgIn.stVFrame.pMbBlk);
}
```

4.7 RK_MPI_GDC_AddCorrectionExTask

【Описание】

Добавляет task коррекции «рыбьего глаза» в уже открытый job, режим XY.

【Синтаксис】

RK_S32 RK_MPI_GDC_AddCorrectionExTask(GDC_HANDLE hHandle, const FISHEYE_ATTR_EX_S *pstTask, FISHEYE_ATTR_S *pstFishEyeAttrrEx, RK_BOOL bCheckMode);

【Параметры】

Имя параметра	Описание	Вход/выход
hHandle	HANDLE уже запущенного job.	Вход
pstTask	Указатель на атрибуты task GDC.	Вход
pstFishEyeAttrrEx	Указатель на конфигурацию атрибутов области коррекции FISHEYE, режим XY.	Вход
bCheckMode	Режим проверки RK_TRUE: режим проверки чёрных полей. Пока не поддерживается. RK_FALSE: режим коррекции. В настоящее время поддерживается только он.	

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- Функция интерфейса пока не реализована.

4.8 RK_MPI_GDC_FisheyePosQueryDst2Src

【Описание】

По координате выходного исправленного изображения находит координату исходного изображения.

【Синтаксис】

RK_S32 RK_MPI_GDC_FisheyePosQueryDst2Src(const GDC_FISHEYE_POINT_QUERY_ATTR_S pstFisheyePointQueryAttr, const VIDEO_FRAME_INFO_S *pstVideoInfo, const POINT_S *pstDstPoint， POINT_S *pstSrcPoint);

【Параметры】

Имя параметра	Описание	Вход/выход
pstFisheyePointQueryAttr	Атрибуты обратного поиска координат точки коррекции «рыбьего глаза».	Вход
pstVideoInfo	Сведения о входном кадре VIDEO_FRAME_INFO_S «рыбьего глаза».	Вход
pstDstPoint	Координата на исправленном изображении, для которой требуется найти соответствие.	Вход
pstSrcPoint	Найденная координата на исходном изображении «рыбьего глаза».	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- При нескольких областях координаты каждой выходной области вычисляются отдельно; нельзя рассчитывать их по изображению после объединения.
- Возвращаемые координаты исходного изображения относятся к его исходному размеру.

【Пример】

```
RK_S32 s32Ret;
for (RK_U32 region = 1; region < ctx->stFisheyeAttr.u32RegionNum; region++) {
    GDC_FISHEYE_POINT_QUERY_ATTR_S stQuery;
    POINT_S stDstPoint, stSrcPoint;
    memset(&stQuery, 0, sizeof(sizeof(FISHEYE_ATTR_S)));
    stQuery.pstFishEyeAttr = &ctx->stFisheyeAttr;
    stQuery.u32RegionIndex = region;
    stDstPoint.s32X = 0;
    stDstPoint.s32Y = 540;
    s32Ret = RK_MPI_GDC_FisheyePosQueryDst2Src(&stQuery, &ctx->stTask.stImgIn, &stDstPoint,
&stSrcPoint);
    if (s32Ret != RK_SUCCESS) {
        RK_LOGE("RK_MPI_GDC_FisheyePosQueryDst2Src err:0x%x!", s32Ret);
    }
    RK_LOGD("region:%d dst:%d,%d src:%d,%d", region, stDstPoint.s32X, stDstPoint.s32Y,
            stSrcPoint.s32X, stSrcPoint.s32Y);
}
```

4.9 RK_MPI_GDC_FisheyePosQueryDst2SrcArray

【Описание】

По массиву координат выходного исправленного изображения находит массив координат исходного изображения.

【Синтаксис】

RK_S32 RK_MPI_GDC_FisheyePosQueryDst2SrcArray(const GDC_FISHEYE_POINT_QUERY_ATTR_S pstFisheyePointQueryAttr, const VIDEO_FRAME_INFO_S *pstVideoInfo, const RK_U32 u32PointNum, const POINT_S *pastDstPoint， POINT_S *pastSrcPoint);

【Параметры】

Имя параметра	Описание	Вход/выход
pstFisheyePointQueryAttr	Атрибуты обратного поиска координат точки коррекции «рыбьего глаза».	Вход
pstVideoInfo	Сведения о входном кадре VIDEO_FRAME_INFO_S «рыбьего глаза».	Вход
u32PointNum	Количество искомых координат.	Вход
pastDstPoint	Массив координат на исправленном изображении, для которых требуется найти соответствия.	Вход
pastSrcPoint	Массив найденных координат на исходном изображении «рыбьего глаза».	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- При нескольких областях координаты каждой выходной области вычисляются отдельно; нельзя рассчитывать их по изображению после объединения.
- Возвращаемые координаты исходного изображения относятся к его исходному размеру.

【Пример】

```
RK_S32 s32Ret;
GDC_FISHEYE_POINT_QUERY_ATTR_S stQuery;
memset(&stQuery, 0, sizeof(sizeof(FISHEYE_ATTR_S)));
stQuery.pstFishEyeAttr = &ctx->stFisheyeAttr;
stQuery.u32RegionIndex = region;
s32Ret = RK_MPI_GDC_FisheyePosQueryDst2SrcArray(&stQuery, &ctx->stTask.stImgIn, ctx->u32PointLen,
ctx->pastDstPoint, ctx->pastSrcPoint);
if (s32Ret != RK_SUCCESS) {
    RK_LOGE("RK_MPI_GDC_FisheyePosQueryDst2SrcArray err:0x%x!", s32Ret);
}
```

4.10 RK_MPI_GDC_FisheyePosQueryDst2Pano

【Описание】

По координате выходного исправленного изображения находит координату панорамного изображения.

【Синтаксис】

RK_S32 RK_MPI_GDC_FisheyePosQueryDst2Pano(const GDC_FISHEYE_POINT_QUERY_ATTR_S pstFisheyePointQueryAttr, const VIDEO_FRAME_INFO_S *pstVideoInfo, const RK_U32 u32PanoRegionIndex, const POINT_S *pstDstPoint， POINT_S *pstSrcPoint);

【Параметры】

Имя параметра	Описание	Вход/выход
pstFisheyePointQueryAttr	Атрибуты обратного поиска координат точки коррекции «рыбьего глаза».	Вход
pstVideoInfo	Сведения о входном кадре VIDEO_FRAME_INFO_S «рыбьего глаза».	Вход
u32PanoRegionIndex	Номер области панорамы; первая область имеет номер 0.	Вход
pstDstPoint	Координата на исправленном изображении, для которой требуется найти соответствие.	Вход
pstSrcPoint	Найденная координата на панорамном изображении.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- При нескольких областях координаты каждой выходной области вычисляются отдельно; нельзя рассчитывать их по изображению после объединения.
- Возвращаемые координаты панорамы относятся к размеру выходной панорамы и не зависят от размера исходного изображения.

【Пример】

```
RK_S32 s32Ret;
GDC_FISHEYE_POINT_QUERY_ATTR_S stQuery;
POINT_S stDstPoint, stPanoPoint;
memset(&stQuery, 0, sizeof(sizeof(FISHEYE_ATTR_S)));
stQuery.pstFishEyeAttr = &ctx->stFisheyeAttr;
stQuery.u32RegionIndex = region;
stDstPoint.s32X = 0;
stDstPoint.s32Y = 540;
s32Ret = RK_MPI_GDC_FisheyePosQueryDst2Pano(&stQuery, &ctx->stTask.stImgIn, 0,
                                            &stDstPoint, &stPanoPoint);
if (s32Ret != RK_SUCCESS) {
    RK_LOGE("RK_MPI_GDC_FisheyePosQueryDst2Pano err:0x%x!", s32Ret);
}
```

4.11 RK_MPI_GDC_FisheyePosQueryDst2PanoArray

【Описание】

По массиву координат выходного исправленного изображения находит массив координат панорамного изображения.

【Синтаксис】

RK_S32 RK_MPI_GDC_FisheyePosQueryDst2PanoArray(const GDC_FISHEYE_POINT_QUERY_ATTR_S pstFisheyePointQueryAttr, const VIDEO_FRAME_INFO_S *pstVideoInfo, const RK_U32 u32PanoRegionIndex, const RK_U32 u32PointNum, const POINT_S *pastDstPoint， POINT_S *pastSrcPoint);

【Параметры】

Имя параметра	Описание	Вход/выход
pstFisheyePointQueryAttr	Атрибуты обратного поиска координат точки коррекции «рыбьего глаза».	Вход
pstVideoInfo	Сведения о входном кадре VIDEO_FRAME_INFO_S «рыбьего глаза».	Вход
u32PanoRegionIndex	Номер области панорамы; первая область имеет номер 0.	Вход
u32PointNum	Количество искомых координат.	Вход
pastDstPoint	Массив координат на исправленном изображении, для которых требуется найти соответствия.	Вход
pastSrcPoint	Массив найденных координат на панорамном изображении.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно
Не 0	Ошибка; см. коды ошибок GDC.

【Примечание】

- При нескольких областях координаты каждой выходной области вычисляются отдельно; нельзя рассчитывать их по изображению после объединения.

- Возвращаемые координаты панорамы относятся к размеру выходной панорамы и не зависят от размера исходного изображения.

【Пример】

```
RK_S32 s32Ret;
GDC_FISHEYE_POINT_QUERY_ATTR_S stQuery;
memset(&stQuery, 0, sizeof(sizeof(FISHEYE_ATTR_S)));
stQuery.pstFishEyeAttr = &ctx->stFisheyeAttr;
stQuery.u32RegionIndex = region;
s32Ret = RK_MPI_GDC_FisheyePosQueryDst2PanoArray(&stQuery, &ctx->stTask.stImgIn,
                                                 pano_region, ctx->u32PointLen,
                                                 ctx->pastDstPoint, ctx->pastSrcPoint);
if (s32Ret != RK_SUCCESS) {
    RK_LOGE("RK_MPI_GDC_FisheyePosQueryDst2PanoArray err:0x%x!", s32Ret);
}
```

5. Типы данных

Связанные с модулем GDC типы данных определены ниже:

- GDC_HANDLE: определяет дескриптор job GDC.
- GDC_TASK_ATTR_S: определяет атрибуты task GDC.
- FISHEYE_LMFCOEF_NUM: количество параметров LMF объектива «рыбий глаз».
- FISHEYE_CONFIG_S: определяет конфигурацию параметров LMF объектива FISHEYE.
- FISHEYE_MOUNT_MODE_E: определяет режим установки в атрибутах FISHEYE.
- FISHEYE_VIEW_MODE_E: определяет режим коррекции в атрибутах FISHEYE.
- RECT_S: определяет атрибуты прямоугольной области.
- FISHEYE_REGION_ATTR_S: определяет конфигурацию атрибутов каждой области коррекции FISHEYE.
- FISHEYE_REGION_ATTR_EX_S: определяет конфигурацию атрибутов каждой области коррекции FISHEYE, режим XY.
- FISHEYE_MAX_REGION_NUM: определяет максимальное количество областей коррекции FISHEYE.
- FISHEYE_ATTR_S: определяет конфигурацию атрибутов FISHEYE.
- FISHEYE_ATTR_EX_S: определяет конфигурацию атрибутов FISHEYE, режим XY.
- GDC_FISYEYE_POINT_QUERY_ATTR_S: атрибуты обратного поиска координат точки коррекции «рыбьего глаза».

5.1 GDC_HANDLE

【Описание】

Определяет дескриптор job GDC.

【Определение】

typedef RK_S32 GDC_HANDLE

【Примечания】

Нет.

【Связанные типы данных и интерфейсы】

Нет.

5.2 GDC_TASK_ATTR_S

【Описание】

Определяет атрибуты task GDC.

【Определение】

```
typedef struct rkGDC_TASK_ATTR_S {
    VIDEO_FRAME_INFO_S stImgIn;
    VIDEO_FRAME_INFO_S stImgOut;
    RK_U64 au64privateData[4];
    RK_U64 u64TaskId;
} GDC_TASK_ATTR_S;
```

【Члены】

Имя параметра	Описание
stImgIn	Атрибуты входного изображения.
stImgOut	Атрибуты выходного изображения.
au64privateData	Параметры отладки Закрытые данные, связанные с task. au64privateData[0] и au64privateData[1] — шаг исправленного изображения stepX/stepY, степень двойки. Их можно использовать для отладки эффекта; внутренний код адаптивно настраивает связанные параметры. Если значение слишком мало, нагрузка системы увеличивается; если слишком велико, результат ухудшается. Рекомендуется 0.
u64TaskId	Зарезервированные параметры расширения Настройка taskId: [0, GDC_MAX_TASK_NUM-1]. Ненулевое значение явно задаёт taskId; если он уже используется, возвращается RK_ERR_GDC_BUSY. Значение 0 означает автоматический выбор свободного task системой. Рекомендуется 0.

【Примечание】

- au64privateData и u64TaskId предназначены для отладки или расширения. Если особой необходимости нет, рекомендуется задавать 0.

5.3 FISHEYE_LMFCOEF_NUM

【Описание】

Количество параметров LMF объектива «рыбий глаз».

【Определение】

```
#define FISHEYE_LMFCOEF_NUM 128
```

【Примечания】

Нет

5.4 FISHEYE_CONFIG_S

【Описание】

Определяет конфигурацию параметров LMF объектива FISHEYE.

【Определение】

```
typedef struct rkFISHEYE_CONFIG_S {
    RK_U16                  au16LMFCoef[FISHEYE_LMFCOEF_NUM];
} FISHEYE_CONFIG_S;
```

【Члены】

Имя параметра	Описание
au16LMFCoef	Параметры LMF объектива «рыбий глаз».

【Примечания】

Нет

5.5 FISHEYE_JOB_CONFIG_S

【Описание】

Определяет конфигурацию параметров LMF объектива «рыбий глаз» для task FISHEYE.

【Определение】

```
typedef struct rkFISHEYE_JOB_CONFIG_S {
    RK_U64                  u64LenMapPhyAddr;
} FISHEYE_JOB_CONFIG_S;
```

【Члены】

Имя параметра	Описание
u64LenMapPhyAddr	Физический адрес хранения коэффициентов LMF «рыбьего глаза».

【Примечания】

Нет

5.6 FISHEYE_MOUNT_MODE_E

【Описание】

Определяет режим установки в атрибутах FISHEYE.

【Определение】

```
typedef enum rkFISHEYE_MOUNT_MODE_E {
    FISHEYE_DESKTOP_MOUNT    = 0,
    FISHEYE_CEILING_MOUNT    = 1,
    FISHEYE_WALL_MOUNT       = 2,
    FISHEYE_MOUNT_MODE_BUTT
} FISHEYE_MOUNT_MODE_E;
```

【Члены】

Имя параметра	Описание
FISHEYE_DESKTOP_MOUNT	Напольная установка.
FISHEYE_CEILING_MOUNT	Потолочная установка.
FISHEYE_WALL_MOUNT	Настенная установка.
FISHEYE_MOUNT_MODE_BUTT	Максимальное значение.

5.7 FISHEYE_VIEW_MODE_E

【Описание】

Определяет режим коррекции в атрибутах FISHEYE.

【Определение】

```
typedef enum rkFISHEYE_VIEW_MODE_E {
    FISHEYE_VIEW_360_PANORAMA   = 0,
    FISHEYE_VIEW_180_PANORAMA   = 1,
    FISHEYE_VIEW_NORMAL         = 2,
    FISHEYE_NO_TRANSFORMATION   = 3,

    FISHEYE_VIEW_MODE_BUTT
} FISHEYE_VIEW_MODE_E;
```

【Члены】

Имя параметра	Описание
FISHEYE_VIEW_360_PANORAMA	Режим панорамы 360°.
FISHEYE_VIEW_180_PANORAMA	Режим панорамы 180°.
FISHEYE_VIEW_NORMAL	Режим коррекции Normal.
FISHEYE_NO_TRANSFORMATION	Без коррекции; масштабированный вывод исходного изображения согласно выходной области.
FISHEYE_VIEW_MODE_BUTT	Максимальное значение.

【Примечание】

- Режим коррекции 360° не поддерживает настенную установку; режим 180° не поддерживает потолочную и напольную установку.

5.8 RECT_S

【Описание】

Определяет атрибуты прямоугольной области.

【Определение】

```
typedef struct rkRECT_S {
    RK_S32 s32X;
    RK_S32 s32Y;
    RK_U32 u32Width;
    RK_U32 u32Height;
} RECT_S;
```

【Члены】

Имя параметра	Описание
s32X	Начальная координата X области.
s32Y	Начальная координата Y области.
u32Width	Ширина области.
u32Height	Высота области.

5.9 FISHEYE_REGION_ATTR_S

【Описание】

Определяет конфигурацию атрибутов каждой области коррекции FISHEYE.

【Определение】

```
typedef struct rkFISHEYE_REGION_ATTR_S {
    FISHEYE_VIEW_MODE_E     enViewMode;
    RK_U32                  u32InRadius;
    RK_U32                  u32OutRadius;
    RK_U32                  u32Pan;
    RK_U32                  u32Tilt;
    RK_U32                  u32HorZoom;
    RK_U32                  u32VerZoom;
    RECT_S                  stOutRect;
} FISHEYE_REGION_ATTR_S;
```

【Члены】

Имя параметра	Описание
enViewMode	Режим коррекции данной области.
u32InRadius	В режиме панорамы 360° — внутренний радиус исходного изображения, соответствующий области; в других режимах недействителен. Диапазон: [0, u32OutRadius).
u32OutRadius	В режиме панорамы 360° — внешний радиус исходного изображения, соответствующий области; в других режимах — видимый радиус области коррекции. Диапазон: [1, 3 × max(ширина входного изображения/4, высота входного изображения/4)].
u32Pan	Значение Pan параметров PTZ данной области коррекции. Диапазон: [0, 360].
u32Tilt	Значение Tilt параметров PTZ данной области коррекции. Диапазон: [0, 360].
u32HorZoom	Горизонтальное значение Zoom параметров PTZ данной области коррекции. Диапазон: [1, 4095].
u32VerZoom	Вертикальное значение Zoom параметров PTZ данной области коррекции. Диапазон: [1, 4095].
stOutRect	Выходное положение, ширина и высота данной области коррекции.

【Примечание】

- FISHEYE позволяет отдельно корректировать несколько областей одного изображения; атрибуты каждой области независимы.

5.10 FISHEYE_REGION_ATTR_EX_S

【Описание】

Определяет конфигурацию атрибутов каждой области коррекции FISHEYE, режим XY.

【Определение】

```
typedef struct rkFISHEYE_REGION_ATTR_S {
    FISHEYE_VIEW_MODE_E     enViewMode;
    RK_U32                  u32InRadius;
    RK_U32                  u32OutRadius;
    RK_U32                  u32X;
    RK_U32                  u32Y;
    RK_U32                  u32HorZoom;
    RK_U32                  u32VerZoom;
    RECT_S                  stOutRect;
} FISHEYE_REGION_ATTR_S;
```

【Члены】

Имя параметра	Описание
enViewMode	Режим коррекции данной области.
u32InRadius	В режиме панорамы 360° — внутренний радиус исходного изображения, соответствующий области; в других режимах недействителен. Диапазон: [0, u32OutRadius).
u32OutRadius	В режиме панорамы 360° — внешний радиус исходного изображения, соответствующий области; в других режимах — видимый радиус области коррекции. Диапазон: [1, 3 × max(ширина входного изображения/4, высота входного изображения/4)].
u32X	Координата X центральной точки выбранной области. Диапазон: [0, ширина входного изображения - 1].
u32Y	Координата Y центральной точки выбранной области. Диапазон: [0, высота входного изображения - 1].
u32HorZoom	Горизонтальное значение Zoom параметров PTZ данной области коррекции. Диапазон: [1, 4095].
u32VerZoom	Вертикальное значение Zoom параметров PTZ данной области коррекции. Диапазон: [1, 4095].
stOutRect	Выходное положение, ширина и высота данной области коррекции.

【Примечание】

- FISHEYE позволяет отдельно корректировать несколько областей одного изображения; атрибуты каждой области независимы.

5.11 FISHEYE_MAX_REGION_NUM

【Описание】

Определяет максимальное количество областей коррекции FISHEYE.

【Определение】

```
#define FISHEYE_MAX_REGION_NUM     9
```

5.12 FISHEYE_ATTR_S

【Описание】

Определяет конфигурацию атрибутов FISHEYE.

【Определение】

```
typedef struct rkFISHEYE_ATTR_S {
    RK_BOOL                 bEnable;
    RK_BOOL                 bLMF;
    RK_BOOL                 bBgColor;
    RK_U32                  u32BgColor;
    RK_S32                  s32HorOffset;
    RK_S32                  s32VerOffset;
    RK_U32                  u32TrapezoidCoef;
    RK_S32                  s32FanStrength;
    FISHEYE_MOUNT_MODE_E    enMountMode;
    RK_U32                  u32RegionNum;
    FISHEYE_REGION_ATTR_S   astFishEyeRegionAttr[FISHEYE_MAX_REGION_NUM];
} FISHEYE_ATTR_S;
```

【Члены】

Имя параметра	Описание
bEnable	Включение или отключение коррекции «рыбьего глаза».
bLMF	Использовать ли пользовательские параметры LMF объектива «рыбий глаз». Параметр пока не используется.
bBgColor	Добавлять ли цвет фона к выходному изображению. Параметр пока не используется.
u32BgColor	Цвет фона в формате RGB888, диапазон [0, 0xFFFFFF]. Параметр пока не используется.
s32HorOffset	Горизонтальное смещение центра объектива относительно центра SENSOR, диапазон [-511, 511], в пикселях.
s32VerOffset	Вертикальное смещение центра объектива относительно центра SENSOR, диапазон [-511, 511], в пикселях.
u32TrapezoidCoef	Коэффициент силы трапецеидальной коррекции, используемый для исправления угла наклона при настенной установке; диапазон [0, 32].
s32FanStrength	Коэффициент силы веерной коррекции; действует только в режиме 180°, диапазон [-760, 760].
enMountMode	Режим установки коррекции «рыбьего глаза».
u32RegionNum	Количество областей коррекции одного изображения; поддерживается до 9 областей.
astFishEyeRegionAttr	Независимая конфигурация атрибутов каждой области коррекции.

【Примечание】

- FISHEYE поддерживает раздельную коррекцию нескольких областей одного изображения. Режим коррекции, параметры PTZ и другие настройки каждой области независимы. Затем согласно выходному положению, ширине и высоте каждой области изображения объединяются и выводятся.

5.13 FISHEYE_ATTR_EX_S

【Описание】

Определяет конфигурацию атрибутов FISHEYE, режим XY.

【Определение】

```
typedef struct rkFISHEYE_ATTR_EX_S {
    RK_BOOL                   bEnable;
    RK_BOOL                   bLMF;
    RK_BOOL                   bBgColor;
    RK_U32                    u32BgColor;
    RK_S32                    s32HorOffset;
    RK_S32                    s32VerOffset;
    RK_U32                    u32TrapezoidCoef;
    RK_S32                    s32FanStrength;
    FISHEYE_MOUNT_MODE_E      enMountMode;
    RK_U32                    u32RegionNum;
    FISHEYE_REGION_ATTR_EX_S  astFishEyeRegionAttr[FISHEYE_MAX_REGION_NUM];
} FISHEYE_ATTR_EX_S;
```

【Члены】

Имя параметра	Описание
bEnable	Включение или отключение коррекции «рыбьего глаза».
bLMF	Использовать ли пользовательские параметры LMF объектива «рыбий глаз». Параметр пока не используется.
bBgColor	Добавлять ли цвет фона к выходному изображению. Параметр пока не используется.
u32BgColor	Цвет фона в формате RGB888, диапазон [0, 0xFFFFFF]. Параметр пока не используется.
s32HorOffset	Горизонтальное смещение центра объектива относительно центра SENSOR, диапазон [-511, 511], в пикселях.
s32VerOffset	Вертикальное смещение центра объектива относительно центра SENSOR, диапазон [-511, 511], в пикселях.
u32TrapezoidCoef	Коэффициент силы трапецеидальной коррекции, используемый для исправления угла наклона при настенной установке; диапазон [0, 32].
s32FanStrength	Коэффициент силы веерной коррекции; действует только в режиме 180°, диапазон [-760, 760].
enMountMode	Режим установки коррекции «рыбьего глаза».
u32RegionNum	Количество областей коррекции одного изображения; поддерживается до 9 областей.
astFishEyeRegionAttr	Независимая конфигурация каждой области коррекции, атрибуты режима XY.

【Примечание】

- FISHEYE поддерживает раздельную коррекцию нескольких областей одного изображения. Режим коррекции, параметры PTZ и другие настройки каждой области независимы. Затем согласно выходному положению, ширине и высоте каждой области изображения объединяются и выводятся.

5.14 GDC_FISYEYE_POINT_QUERY_ATTR_S

【Описание】

Атрибуты обратного поиска координат точки коррекции «рыбьего глаза».

【Определение】

```
typedef struct rkGDC_FISHEYE_POINT_QUERY_ATTR_S {
    RK_U32          u32RegionIndex;
    FISHEYE_ATTR_S *pstFishEyeAttr;
    RK_U16          au16LMF[FISHEYE_LMFCOEF_NUM];
} GDC_FISHEYE_POINT_QUERY_ATTR_S;
```

【Члены】

Имя параметра	Описание
u32RegionIndex	Номер области коррекции «рыбьего глаза». Диапазон: [0, FISHEYE_MAX_REGION_NUM-1].
pstFishEyeAttr	Параметры атрибутов «рыбьего глаза».
au16LMF	Коэффициенты LMF.

6. Коды ошибок GDC

Коды ошибок API GDC приведены ниже.

Код ошибки	Макроопределение	Описание
0xA0128002	RK_ERR_GDC_INVALID_CHNID	Недопустимый taskId GDC.
0xA0128003	RK_ERR_GDC_ILLEGAL_PARAM	Недопустимые параметры GDC.
0xA0128005	RK_ERR_GDC_CHN_UNEXIST	Task GDC не существует.
0xA0128006	RK_ERR_GDC_NULL_PTR	Во входных параметрах передан нулевой указатель.
0xA0128008	RK_ERR_GDC_NOT_SUPPORT	Операция не поддерживается.
0xA0128009	RK_ERR_GDC_NOT_PERMITTED	Операция не разрешена.
0xA012800D	RK_ERR_GDC_NOBUF	Не удалось выделить память.
0xA012800E	RK_ERR_GDC_BUF_EMPTY	Ресурсы job/task GDC уже использованы.
0xA012800F	RK_ERR_GDC_BUF_FULL	Свободных BUF нет.
0xA0128010	RK_ERR_GDC_SYS_NOTREADY	Система не инициализирована.
0xA0128012	RK_ERR_GDC_BUSY	Job/task GDC занят.

Панорамное сшивание

1. Введение

Обзор

AVS (Any View Stitching, панорамное объединение) объединяет несколько изображений в панораму и выводит изображение в заданном режиме проекции. Весь процесс в основном включает калибровку и вывод объединённого изображения. В этой главе описан вывод. Кроме того, AVS поддерживает горизонтальное, вертикальное и сеточное объединение без смешивания.

Версии продуктов

Название микросхемы	Версия ядра
RK3588	5.10
RV1106	5.10
RV1109/RV1126	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Версия №	Автор	Дата изменения изменения	Описание изменения
V0.1.0	JKM	2021-12-01	Первая версия.
V0.1.1	JKM	2022-03-02	Уточнены определения структур данных.
V0.1.2	JKM	2022-03-19	RK3588 поддерживает до 8 входов объединения; добавлен режим горизонтальной равноугольной цилиндрической проекции.
V0.1.3	JKM	2022-05-06	Уточнены определения структур данных.
V0.1.4	JKM	2022-09-26	Изменено и дополнено описание синхронизации последовательности кадров.
V0.1.5	JKM	2022-11-05	Изменена структура AVS_OUTPUT_ATTR_S, оптимизирована задержка вывода первого кадра на RK3588.
V0.1.6	JKM	2022-12-28	Добавлены интерфейсы RK_MPI_AVS_SetGrpRoi, RK_MPI_AVS_SetGrpRoi и новая структура. AVS_ROI_PARAM_S
V0.2.0	JKM	2022-02-25	Добавлена поддержка платформы RV1106, изменена структура AVS_INPUT_ATTR_S.
V0.2.1	JKM	2022-03-02	Добавлен интерфейс RK_MPI_AVS_GetFinalLut и структура AVS_FINAL_LUT_S.
V0.3.0	JKM	2022-03-20	Добавлена поддержка платформ RV1109/RV1126.

2. Содержание

Предисловие
Содержание
Описание функций
Справочник API
Типы данных
Коды ошибок AVS
Связанные документы

3. Описание функций

3.1 Основные понятия

3.1.1 GROUP

Группа (GROUP), далее группа. AVS предоставляет пользователю понятие GROUP. Доступно до AVS_MAX_GRP_NUM групп, которые разделяют аппаратное устройство по времени. Каждая группа AVS содержит несколько PIPE и CHANNEL.

3.1.2 PIPE

Канал ввода (PIPE), далее pipe группы AVS. Используется для входных изображений объединения. Количество PIPE равно числу входов. Пользователь может подключить их к предыдущему модулю системной привязкой или отправлять изображения непосредственно в PIPE.

3.1.3 CHANNEL

Канал группы AVS, используемый для вывода результата объединения.

3.1.4 Проекция

Панорамное пространственное изображение проецируется в плоские координаты для кодирования и передачи. AVS поддерживает пять режимов проекции: Equirectangular, Cylindrical, Rectilinear, CubeMap и Trans_Equirectangular.

3.1.5 Сжатие и распаковка

3.1.6 Сжатие

Поддерживается сжатие выходного изображения канала, включая AFBC.

3.1.6.1 Декомпрессия

Поддерживается декомпрессия входных сжатых изображений, включая декомпрессию AFBC.

3.1.6.2 Таблица поддержки сжатия и декомпрессии на различных платформах микросхем

Микросхема	Поддержка сжатия AFBC	Поддержка распаковки AFBC
RK3588	**Поддерживается**	**Поддерживается**
RV1106	**Не поддерживается**	**Не поддерживается**
RV1109/RV1126	**Не поддерживается**	**Не поддерживается**

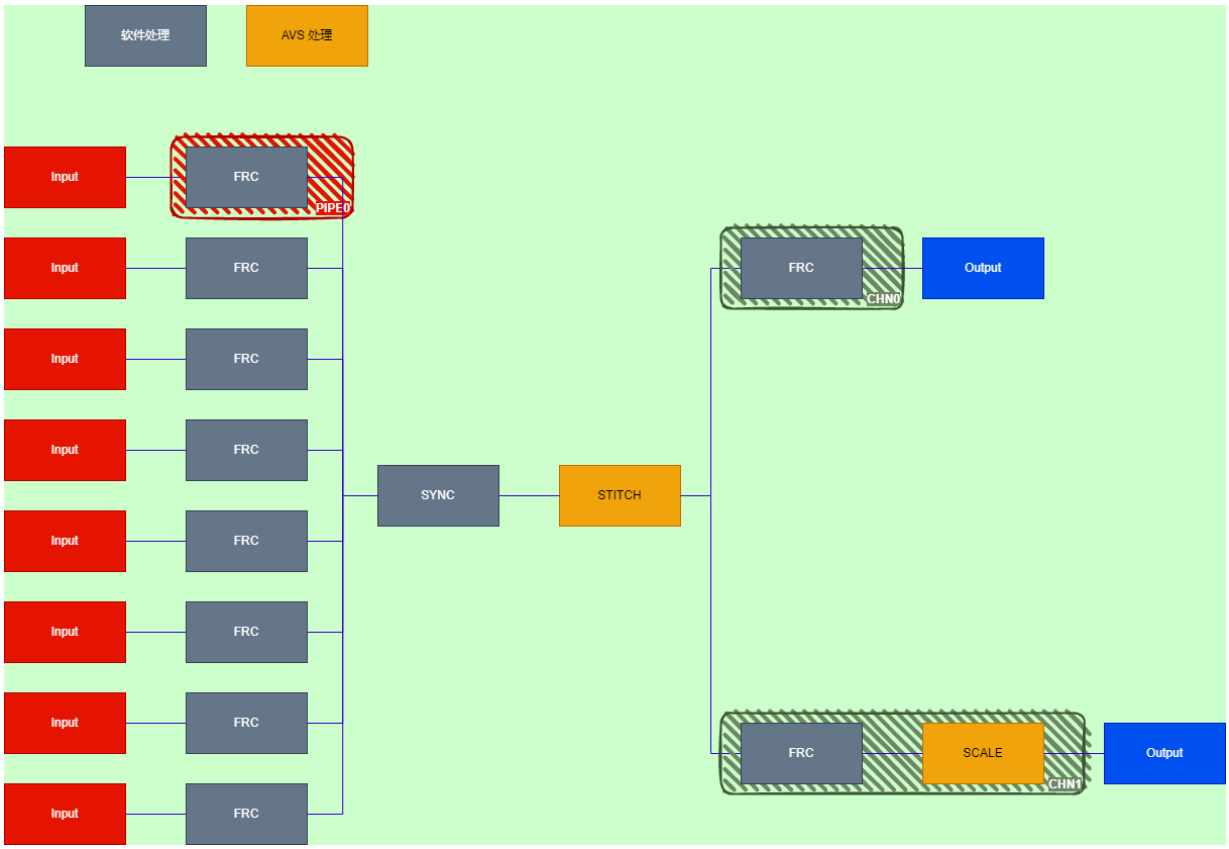
3.2 Способ применения

Через интерфейс привязки модуля SYS можно выполнять привязку к модулям VI/VPSS и VO/VENC/VPSS и т. п.; первые служат источниками входных данных AVS, а вторые — получателями AVS. Пользователь может управлять Group через интерфейсы AVS MPI. Каждый Group можно привязать к нескольким источникам входных данных (путём привязки к PIPE AVS) и к нескольким получателям (путём привязки к CHANNEL AVS; каждый CHANNEL AVS можно привязать только к одному получателю).

3.2.1 Схема потока данных AVS

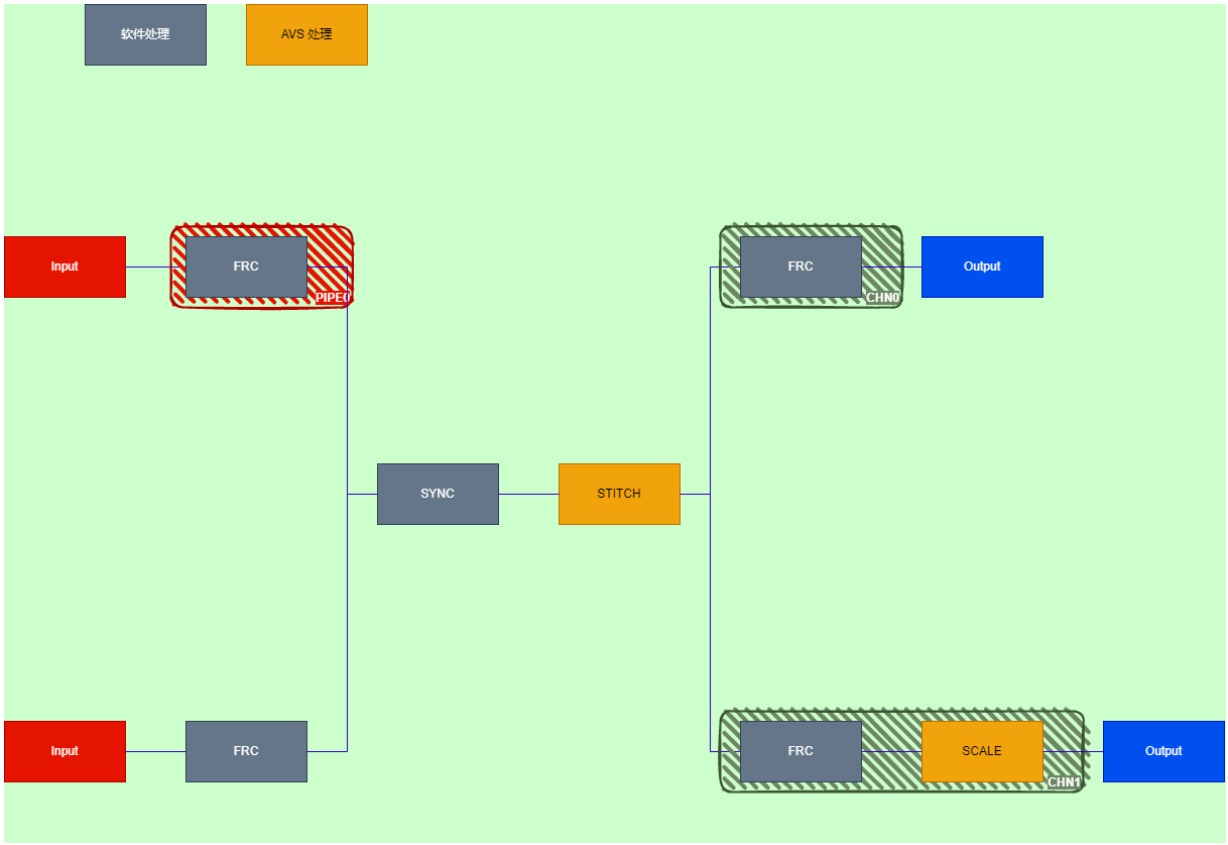
Схема потока данных AVS показана на следующем рисунке. Данные источника входного сигнала передаются через каждый PIPE в GROUP, где выполняется сшивка, после чего обработанные данные направляются в CHANNEL для получения пользователем либо передачи получателю.

Рисунок. Схема потока данных платформы RK3588



На платформе RK3588 обработка AVS выполняется аппаратным GPU.

Рисунок. Схема потока данных платформ RV1106/RV1109/RV1126



На платформах RV1106/RV1109/RV1126 обработка AVS выполняется CPU и аппаратным RGA.

3.3 Характеристики входа и выхода

3.3.0.1 Таблица поддерживаемых входных и выходных изображений платформы RK3588

Формат данных	Выравнивание ширины	Выравнивание высоты	Разрядность пикселя (bit)
RK_FMT_YUV420SP	16	2	8

3.3.0.2 Таблица поддерживаемых входных и выходных изображений платформы RV1106

Формат данных	Выравнивание ширины	Выравнивание высоты	Разрядность пикселя (bit)
RK_FMT_YUV420SP	4	2	8

3.3.0.3 Таблица поддерживаемых входных и выходных изображений платформ RV1126/RV1109

Формат данных	Выравнивание ширины	Выравнивание высоты	Разрядность пикселя (bit)
RK_FMT_YUV420SP	4	2	8

3.4 Пример

Подробный тестовый DEMO приведён в поставляемых файлах test_mpi_avs.cpp и test_comm_avs.cpp.

```
    RK_S32 s32Ret = RK_FAILURE;
    AVS_CFG_S *pstAvsCtx = reinterpret_cast<AVS_CFG_S *>(&(ctx->avsContext));

    pstAvsCtx->s32GrpId = 0;
    pstAvsCtx->s32PipeId = 0;
    pstAvsCtx->s32ChnId = 0;
    pstAvsCtx->u32AvsPipeCnt = ctx->s32StitchNum;
    pstAvsCtx->u32AvsChnCnt = 1;

    pstAvsCtx->stAvsGrpAttr.enMode = ctx->enAvsWorkMode;
    pstAvsCtx->u32OutW = 8192;
    pstAvsCtx->u32OutH = 2700;

    snprintf(pstAvsCtx->stAvsGrpAttr.stOutAttr.stCalib.pCalibFilePath,
sizeof("/usr/share/avs_calib/calib_file_pos.pto"),
             "/usr/share/avs_calib/calib_file_pos.pto");
    snprintf(pstAvsCtx->stAvsGrpAttr.stOutAttr.stCalib.pMeshAlphaPath,
sizeof("/usr/share/avs_calib/"),
             "/usr/share/avs_calib/");

    pstAvsCtx->stAvsGrpAttr.stLUT.enAccuracy                = AVS_LUT_ACCURACY_HIGH;

    pstAvsCtx->stAvsGrpAttr.u32PipeNum                      = pstAvsCtx->u32AvsPipeCnt;
    pstAvsCtx->stAvsGrpAttr.stGainAttr.enMode               = AVS_GAIN_MODE_AUTO;

    pstAvsCtx->stAvsGrpAttr.stOutAttr.enPrjMode             = AVS_PROJECTION_EQUIRECTANGULAR;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stCenter.s32X         = 4196;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stCenter.s32Y         = 2080;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stFOV.u32FOVX         = 28000;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stFOV.u32FOVY         = 9500;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stORIRotation.s32Roll  = 0;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stORIRotation.s32Pitch = 0;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stORIRotation.s32Yaw   = 0;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stRotation.s32Roll     = 0;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stRotation.s32Pitch    = 0;
    pstAvsCtx->stAvsGrpAttr.stOutAttr.stRotation.s32Yaw      = 0;

    pstAvsCtx->stAvsGrpAttr.bSyncPipe                     = ctx->bFrameSync;
    pstAvsCtx->stAvsGrpAttr.stFrameRate.s32SrcFrameRate   = -1;
    pstAvsCtx->stAvsGrpAttr.stFrameRate.s32DstFrameRate   = -1;

    pstAvsCtx->stAvsChnAttr[0].enCompressMode              = ctx->enLinkCompressMode;
    pstAvsCtx->stAvsChnAttr[0].stFrameRate.s32SrcFrameRate = -1;
    pstAvsCtx->stAvsChnAttr[0].stFrameRate.s32DstFrameRate = -1;
    pstAvsCtx->stAvsChnAttr[0].u32Depth                    = 0;
    pstAvsCtx->stAvsChnAttr[0].u32Width                    = pstAvsCtx->u32OutW;
    pstAvsCtx->stAvsChnAttr[0].u32Height                   = pstAvsCtx->u32OutH;
    pstAvsCtx->stAvsChnAttr[0].enDynamicRange              = DYNAMIC_RANGE_SDR8;

    s32Ret = RK_MPI_AVS_SetModParam(&ctx->stAvsModParam);
    if (RK_SUCCESS != s32Ret) {
        RK_LOGE("AVS set mod params failed with %#x!", s32Ret);
        goto __FAILED;
    }

    s32Ret = RK_MPI_AVS_CreateGrp(ctx->s32GrpId, &ctx->stAvsGrpAttr);
    if (RK_SUCCESS != s32Ret) {
        RK_LOGE("AVS creat grp failed with %#x!", s32Ret);
        goto __FAILED;
    }
```

```
    s32Ret = RK_MPI_AVS_SetChnAttr(ctx->s32GrpId, ctx->s32ChnId, &ctx->stAvsChnAttr[0]);
    if (RK_SUCCESS != s32Ret) {
        RK_LOGE("AVS set chn attr failed with %#x!", s32Ret);
        goto __FAILED;
    }

    s32Ret = RK_MPI_AVS_EnableChn(ctx->s32GrpId, ctx->s32ChnId);
    if (RK_SUCCESS != s32Ret) {
        RK_LOGE("AVS enable chn failed with %#x!", s32Ret);
        goto __FAILED;
    }

    s32Ret = RK_MPI_AVS_StartGrp(ctx->s32GrpId);
    if (RK_SUCCESS != s32Ret) {
        RK_LOGE("AVS start grp failed with %#x!", s32Ret);
        goto __FAILED;
    }

__FAILED:

    return s32Ret;
```

4. Справочник API

Данный функциональный модуль предоставляет пользователю следующие API:

- RK_MPI_AVS_CreateGrp: создание группы AVS.
- RK_MPI_AVS_DestroyGrp: уничтожение группы AVS.
- RK_MPI_AVS_StartGrp: запуск группы AVS.
- RK_MPI_AVS_StopGrp: остановка группы AVS.
- RK_MPI_AVS_ResetGrp: сброс группы AVS.
- RK_MPI_AVS_GetGrpAttr: получение атрибутов группы AVS.
- RK_MPI_AVS_SetGrpAttr: задание атрибутов группы AVS.
- RK_MPI_AVS_GetGrpRoi: получение области интереса группы AVS.
- RK_MPI_AVS_SetGrpRoi: задание области интереса группы AVS.
- RK_MPI_AVS_SendPipeFrame: отправка пользователем изображения в PIPE группы AVS.
- RK_MPI_AVS_SetChnAttr: задание атрибутов канала AVS.
- RK_MPI_AVS_GetChnAttr: получение атрибутов канала AVS.
- RK_MPI_AVS_EnableChn: включение канала AVS.
- RK_MPI_AVS_DisableChn: отключение канала AVS.
- RK_MPI_AVS_GetChnFrame: получение пользователем одного кадра изображения канала.
- RK_MPI_AVS_ReleaseChnFrame: освобождение пользователем одного кадра изображения канала.
- RK_MPI_AVS_GetFinalLut: получение итоговой таблицы поиска.

4.1 RK_MPI_AVS_CreateGrp

【Описание】

Создаёт группу AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_CreateGrp(AVS_GRP AVSGrp, const AVS_GRP_ATTR_S* pstGrpAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
pstGrpAttr	Атрибуты группы AVS.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Повторное создание группы не поддерживается.

4.2 RK_MPI_AVS_DestroyGrp

【Описание】

Уничтожает группу AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_StartGrp(AVS_GRP AVSGrp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.
- Перед вызовом этого интерфейса необходимо вызвать RK_MPI_AVS_StopGrp для остановки группы.
- При вызове интерфейс ожидает завершения текущих задач группы и только после этого уничтожает её.

4.3 RK_MPI_AVS_StartGrp

【Описание】

Запускает группу AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_StartGrp(AVS_GRP AVSGrp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.
- Повторный вызов функции для включения той же группы возвращает успех.

4.4 RK_MPI_AVS_StopGrp

【Описание】

Останавливает группу AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_StopGrp(AVS_GRP AVSGrp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

4.5 RK_MPI_AVS_ResetGrp

【Описание】

Сбрасывает GROUP AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_ResetGrp(AVS_GRP AVSGrp);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.

4.6 RK_MPI_AVS_GetGrpAttr

【Описание】

Получает атрибуты группы AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_GetGrpAttr(AVS_GRP AVSGrp, const AVS_GRP_ATTR_S* pstGrpAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
pstGrpAttr	Атрибуты группы AVS.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.

4.7 RK_MPI_AVS_SetGrpAttr

【Описание】

Задаёт атрибуты группы AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_SetGrpAttr(AVS_GRP AVSGrp, const AVS_GRP_ATTR_S* pstGrpAttr);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
pstGrpAttr	Атрибуты группы AVS.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.
- Атрибуты группы должны быть допустимыми. Некоторые статические атрибуты нельзя изменять динамически; подробности см. в AVS_GRP_ATTR_S.

4.8 RK_MPI_AVS_GetGrpRoi

【Описание】

Получает область интереса группы AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_GetGrpRoi(AVS_GRP AVSGrp, AVS_ROI_PARAM_S* pstRoiParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
pstRoiParam	Область интереса группы AVS.	Выход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.
- Этот интерфейс поддерживается только платформой RK3588.

4.9 RK_MPI_AVS_SetGrpRoi

【Описание】

Задаёт область интереса группы AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_SetGrpRoi(AVS_GRP AVSGrp, const AVS_GRP_ATTR_S* pstRoiParam);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
pstRoiParam	Область интереса группы AVS.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.
- Этот интерфейс поддерживается только платформой RK3588.

4.10 RK_MPI_AVS_SendPipeFrame

【Описание】

Пользователь отправляет изображение в PIPE группы AVS.

【Синтаксис】

```
RK_S32 RK_MPI_AVS_SendPipeFrame(AVS_GRP AVSGrp, AVS_PIPE AVSPipe, const
                VIDEO_FRAME_INFO_S* pstVideoFrame, RK_S32 s32MilliSec);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
AVSPipe	Номер конвейера, в который пользователь отправляет изображение. Диапазон значений: [0, AVS_PIPE_NUM).	Вход
pstVideoFrame	Сведения об отправляемом изображении. Подробное описание см. в разделе управления системой.	Вход
s32MilliSec	Параметр тайм-аута. При s32MilliSec = -1 интерфейс является блокирующим; если 0 — неблокирующий; если больше 0 — это время ожидания в миллисекундах (ms).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Рекомендуется отправлять в AVS всю группу входных BUF одновременно. Если вся группа успешно отправлена и больше не используется, входные BUF рекомендуется освободить немедленно; если возвращена ошибка, необходимо освободить всю группу BUF.

4.11 RK_MPI_AVS_SetChnAttr

【Описание】

Задаёт атрибуты канала AVS.

【Синтаксис】

```
RK_S32 RK_MPI_AVS_SetChnAttr(AVS_GRP AVSGrp，  const AVS_CHN AVSChn,
                AVS_CHN_ATTR_S* pstChnAttr);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
AVSChn	Номер канала AVS. В режиме AVS_MODE_BLEND диапазон значений [0, AVS_MAX_CHN_NUM). AVS_MODE_NOBLEND_VER、 В режимах AVS_MODE_NOBLEND_HOR и AVS_MODE_NOBLEND_QR допускается только значение 0. В режиме проекции AVS_PROJECTION_CUBE_MAP допускается только значение 0.	Ввод
pstChnAttr	Атрибуты канала AVS.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.
- Перед заданием атрибутов других каналов необходимо задать атрибуты канала 0.
- В режиме проекции AVS_PROJECTION_CUBE_MAP можно задавать только атрибуты канала 0.

4.12 RK_MPI_AVS_GetChnAttr

【Описание】

Получает атрибуты канала AVS.

【Синтаксис】

```
RK_S32 RK_MPI_AVS_GetChnAttr(AVS_GRP AVSGrp, AVS_CHN AVSChn,
                             AVS_CHN_ATTR_S* pstChnAttr);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
AVSChn	Номер канала AVS. Диапазон значений [0, AVS_MAX_CHN_NUM).	Вход
pstChnAttr	Атрибуты канала AVS.	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

4.13 RK_MPI_AVS_EnableChn

【Описание】

Включает канал AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_EnableChn(AVS_GRP AVSGrp, AVS_CHN AVSChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
AVSChn	Номер канала AVS. Диапазон значений [0, AVS_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- GROUP должна быть создана.
- Перед включением канала необходимо задать его атрибуты через интерфейс RK_MPI_AVS_SetChnAttr.
- Другие каналы можно включать только после включения канала 0.
- В режиме проекции AVS_PROJECTION_CUBE_MAP можно включить только канал 0.
- Повторное включение канала возвращает успех.

4.14 RK_MPI_AVS_DisableChn

【Описание】

Отключает канал AVS.

【Синтаксис】

RK_S32 RK_MPI_AVS_DisableChn(AVS_GRP AVSGrp, AVS_CHN AVSChn);

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
AVSChn	Номер канала AVS. Диапазон значений [0, AVS_MAX_CHN_NUM).	Вход

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.
- Канал 0 можно отключить только после отключения всех остальных каналов.
- Повторное отключение канала возвращает успех.

4.15 RK_MPI_AVS_GetChnFrame

【Описание】

Пользователь получает один кадр изображения канала.

【Синтаксис】

```
RK_S32 RK_MPI_AVS_GetChnFrame(AVS_GRP AVSGrp, AVS_CHN AVSChn,
                              VIDEO_FRAME_INFO_S* pstVideoFrame, RK_S32 s32MilliSec);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
AVSChn	Номер канала AVS. Диапазон значений [0, AVS_MAX_CHN_NUM).	Вход
pstVideoFrame	Сведения о полученном изображении. Подробное описание см. в разделе управления системой.	Вход
s32MilliSec	Параметр тайм-аута. При s32MilliSec = -1 интерфейс является блокирующим; если 0 — неблокирующий; если больше 0 — это время ожидания в миллисекундах (ms).	Ввод

【Возвращаемое значение】

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

【Примечание】

- Группа должна быть предварительно создана.
- Изображение можно получить только при ненулевой глубине очереди канала.
- Если последующий модуль привязан к AVS и этот интерфейс вызывать не требуется, рекомендуется задать ненулевую глубину очереди канала.
- Полученное изображение необходимо своевременно освобождать, иначе во внутренней обработке AVS может не хватить Buffer. Рекомендуется использовать совместно с интерфейсом RK_MPI_AVS_ReleaseChnFrame в паре.

4.16 RK_MPI_AVS_ReleaseChnFrame

【Описание】

Пользователь освобождает один кадр изображения канала.

【Синтаксис】

```
RK_S32 RK_MPI_AVS_ReleaseChnFrame(AVS_GRP AVSGrp, AVS_CHN AVSChn,
                                  const VIDEO_FRAME_INFO_S* pstVideoFrame);
```

【Параметры】

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
AVSChn	Номер канала AVS. Диапазон значений [0, AVS_MAX_CHN_NUM).	Вход
pstVideoFrame	Сведения об изображении. Подробное описание см. в разделе управления системой.	Вход

Возвращаемое значение

Возвращаемое значение	Описание
0	Успешно.
Не 0	Ошибка; значение является кодом ошибки AVS.

Примечание

- Фактически параметры AVSGrp и AVSChn этого интерфейса не используются; можно указать любые значения в допустимом диапазоне.
- Этот интерфейс должен использоваться в паре с RK_MPI_AVS_GetChnFrame.

4.17 RK_MPI_AVS_GetFinalLut

Описание

Получает итоговую таблицу поиска.

Синтаксис

```
RK_S32 RK_MPI_AVS_GetFinalLut(AVS_GRP AVSGrp, AVS_FINAL_LUT_S *pstFinalLut);
```

Параметры

Имя параметра	Описание	Ввод/вывод
AVSGrp	Номер группы AVS. Диапазон значений: [0, AVS_MAX_GRP_NUM).	Вход
pstFinalLut	Итоговая таблица поиска.	Вход

Примечание

- Группа должна быть предварительно создана.
- Перед вызовом этого интерфейса BLK в pstFinalLut необходимо заранее выделить.
- Интерфейс поддерживается платформами RV1106/RV1109/RV1126. В настоящее время поддерживается получение только данных LDCH и вызов возможен только когда AVS не работает.
- Платформа RK3588 этот интерфейс не поддерживает.

5. Типы данных

Типы данных модуля AVS определены следующим образом:

- AVS_MAX_GRP_NUM: определяет максимальное число групп AVS.
- AVS_PIPE_NUM: определяет максимальное число конвейеров AVS.
- AVS_MAX_CHN_NUM: определяет максимальное число каналов AVS.
- AVS_SPLIT_NUM: определяет число блоков AVS при сшивке 7 или 8 потоков.
- AVS_SPLIT_PIPE_NUM: определяет максимальное число сшиваемых потоков в одном блоке при сшивке 7 или 8 потоков.
- AVS_CUBE_MAP_SURFACE_NUM: определяет число граней кубической проекции AVS.
- AVS_MAX_IN_WIDTH: определяет максимальную ширину входного изображения AVS.
- AVS_MAX_IN_HEIGHT: определяет максимальную высоту входного изображения AVS.

- AVS_MIN_IN_WIDTH: определяет минимальную ширину входного изображения AVS.
- AVS_MIN_IN_HEIGHT: определяет минимальную высоту входного изображения AVS.
- AVS_MAX_OUT_WIDTH: определяет максимальную ширину выходного изображения AVS.
- AVS_MAX_OUT_HEIGHT: определяет максимальную высоту выходного изображения AVS.
- AVS_MIN_OUT_WIDTH: определяет минимальную ширину выходного изображения AVS.
- AVS_MIN_OUT_HEIGHT: определяет минимальную высоту выходного изображения AVS.
- AVS_GRP: определяет тип группы AVS.
- AVS_PIPE: определяет тип конвейера AVS.
- AVS_CHN: определяет тип канала AVS.
- AVS_MODE_E: определяет рабочий режим AVS.
- AVS_LUT_ACCURACY_E: определяет точность таблицы поиска.
- AVS_FUSE_WIDTH_E: определяет ширину полосы слияния низкочастотного слоя таблицы поиска.
- AVS_LUT_STEP_E: определяет шаг выборки таблицы поиска.
- AVS_STEP_ATTR_S: определяет атрибуты шага выборки таблицы поиска.
- AVS_LUT_S: определяет атрибуты таблицы поиска.
- AVS_CALIB_S: определяет атрибуты калибровки.
- AVS_PROJECTION_MODE_E: определяет режим проекции выходного сшитого изображения.
- AVS_GAIN_MODE_E: определяет режим компенсации усиления яркости.
- AVS_GAIN_ATTR_S: определяет атрибуты компенсации усиления яркости.
- AVS_ROTATION_S: определяет атрибуты поворота AVS.
- AVS_FOV_S: определяет поле зрения AVS.
- AVS_SPLIT_ATTR_S: определяет атрибуты разбиения при сшивке 7 или 8 потоков.
- AVS_CUBE_MAP_ATTR_S: определяет атрибуты кубической проекции.
- AVS_INPUT_ATTR_S: определяет атрибуты входа сшивки AVS.
- AVS_OUTPUT_ATTR_S: определяет атрибуты выхода сшивки AVS.
- AVS_GRP_ATTR_S: определяет атрибуты группы AVS.
- AVS_CHN_ATTR_S: определяет атрибуты канала AVS.
- AVS_ROI_PARAM_S: определяет область интереса группы AVS.
- AVS_FINAL_LUT_S: определяет атрибуты итоговой таблицы поиска.

5.1 AVS_MAX_GRP_NUM

【Описание】

Определяет максимальное число групп AVS.

【Определение】

RK3588

```
#define AVS_MAX_GRP_NUM 32
```

RV1106

```
#define AVS_MAX_GRP_NUM 4
```

RV1109/RV1126

```
#define AVS_MAX_GRP_NUM 4
```

【Примечания】

Нет.

5.2 AVS_PIPE_NUM

【Описание】

Определяет максимальное число конвейеров AVS.

【Определение】

RK3588

```
#define AVS_PIPE_NUM 8
```

RV1106

```
#define AVS_PIPE_NUM 2
```

RV1109/RV1126

```
#define AVS_PIPE_NUM 2
```

【Примечания】

Нет.

5.3 AVS_MAX_CHN_NUM

【Описание】

Определяет максимальное число каналов AVS.

【Определение】

RK3588

```
#define AVS_MAX_CHN_NUM 2
```

RV1106

```
#define AVS_MAX_CHN_NUM 2
```

RV1109/RV1126

```
#define AVS_MAX_CHN_NUM 2
```

【Примечания】

Нет.

5.4 AVS_SPLIT_NUM

【Описание】

Определяет число блоков AVS при сшивке 7 или 8 потоков.

【Определение】

```
#define AVS_SPLIT_NUM 2
```

【Примечания】

Нет.

5.5 AVS_SPLIT_PIPE_NUM

Описание

Определяет максимальное число сшиваемых потоков в одном блоке при сшивке 7 или 8 потоков.

Определение

```
#define AVS_SPLIT_PIPE_NUM 6
```

Примечания

Нет.

5.6 AVS_CUBE_MAP_SURFACE_NUM

Описание

Определяет число граней кубической проекции AVS.

Определение

```
#define AVS_CUBE_MAP_SURFACE_NUM 6
```

Примечания

Нет.

5.7 AVS_MAX_IN_WIDTH

Описание

Определяет максимальную ширину входного изображения AVS.

Определение

RK3588

```
#define AVS_MAX_IN_HEIGHT 4096
```

RV1106

```
#define AVS_MAX_IN_HEIGHT 4096
```

RV1109/RV1126

```
#define AVS_MAX_IN_HEIGHT 4096
```

Примечания

Нет.

5.8 AVS_MAX_IN_HEIGHT

Описание

Определяет максимальную высоту входного изображения AVS.

Определение

RK3588

```
#define AVS_MAX_IN_HEIGHT 4096
```

RV1106

```
#define AVS_MAX_IN_HEIGHT 4096
```

RV1109/RV1126

```
#define AVS_MAX_IN_HEIGHT 4096
```

Примечания

Нет.

5.9 AVS_MIN_IN_WIDTH

Описание

Определяет минимальную ширину входного изображения AVS.

Определение

RK3588

```
#define AVS_MIN_IN_HEIGHT 1280
```

RV1106

```
#define AVS_MIN_IN_HEIGHT 720
```

RV1109/RV1126

```
#define AVS_MIN_IN_HEIGHT 1280
```

Примечания

Нет.

5.10 AVS_MIN_IN_HEIGHT

Описание

Определяет минимальную высоту входного изображения AVS.

Определение

RK3588

```
#define AVS_MIN_IN_HEIGHT 720
```

RV1106

```
#define AVS_MIN_IN_HEIGHT 720
```

RV1109/RV1126

```
#define AVS_MIN_IN_HEIGHT 720
```

Примечания

Нет.

5.11 AVS_MAX_OUT_WIDTH

【Описание】

Определяет максимальную ширину выходного изображения AVS.

【Определение】

RK3588

```
#define AVS_MAX_OUT_HEIGHT 16383
```

RV1106

```
#define AVS_MAX_OUT_HEIGHT 4096
```

RV1109/RV1126

```
#define AVS_MAX_OUT_HEIGHT 4096
```

【Примечания】

Нет.

5.12 AVS_MAX_OUT_HEIGHT

【Описание】

Определяет максимальную высоту выходного изображения AVS.

【Определение】

RK3588

```
#define AVS_MAX_OUT_HEIGHT 10000
```

RV1106

```
#define AVS_MAX_OUT_HEIGHT 4096
```

RV1109/RV1126

```
#define AVS_MAX_OUT_HEIGHT 4096
```

【Примечания】

Нет.

5.13 AVS_MIN_OUT_WIDTH

【Описание】

Определяет минимальную ширину выходного изображения AVS.

【Определение】

RK3588

```
#define AVS_MIN_OUT_WIDTH 256
```

RV1106

```
#define AVS_MIN_OUT_WIDTH 256
```

RV1109/RV1126

```
#define AVS_MIN_OUT_WIDTH 256
```

【Примечания】

Нет.

5.14 AVS_MIN_OUT_HEIGHT

【Описание】

Определяет минимальную высоту выходного изображения AVS.

【Определение】

RK3588

```
#define AVS_MIN_OUT_HEIGHT 256
```

RV1106

```
#define AVS_MIN_OUT_HEIGHT 256
```

RV1109/RV1126

```
#define AVS_MIN_OUT_HEIGHT 256
```

【Примечания】

Нет.

5.15 AVS_GRP

【Описание】

Определяет тип группы AVS.

【Определение】

```
typedef RK_S32 AVS_GRP;
```

【Примечания】

Нет.

5.16 AVS_PIPE

【Описание】

Определяет тип PIPE AVS.

【Определение】

```
typedef RK_S32 AVS_PIPE;
```

【Примечания】

Нет.

5.17 AVS_CHN

【Описание】

Определяет тип канала AVS.

【Определение】

```
typedef RK_S32 AVS_CHN;
```

【Примечания】

Нет.

5.18 AVS_MODE_E

【Описание】

Определяет рабочий режим AVS.

【Определение】

```
typedef enum rkAVS_MODE_E {
    AVS_MODE_BLEND       = 0,
    AVS_MODE_NOBLEND_VER = 1,
    AVS_MODE_NOBLEND_HOR = 2,
    AVS_MODE_NOBLEND_QR  = 3,
    AVS_MODE_BUTT
} AVS_MODE_E;
```

【Члены】

Имя члена	Описание
AVS_MODE_BLEND	Режим сшивки со слиянием. Сшивка выполняется по LUT, в областях стыка применяется слияние.
AVS_MODE_NOBLEND_VER	Вертикальный режим сшивки без слияния. Входные изображения располагаются вертикально, слияние в местах стыка отсутствует.
AVS_MODE_NOBLEND_HOR	Горизонтальный режим сшивки без слияния. Входные изображения располагаются горизонтально, слияние в местах стыка отсутствует.
AVS_MODE_NOBLEND_QR	Матричный режим сшивки без слияния. Поддерживается сшивка только 4 изображений: две строки по два изображения; слияние в местах стыка отсутствует.

【Примечания】

- В режимах AVS_MODE_BLEND, AVS_MODE_NOBLEND_VER, AVS_MODE_NOBLEND_HOR и AVS_MODE_NOBLEND_QR динамический диапазон выходного изображения определяется входным изображением.
- В режиме AVS_MODE_BLEND ширина и высота выходного изображения определяются атрибутами канала; в режимах AVS_MODE_NOBLEND_VER, AVS_MODE_NOBLEND_HOR и AVS_MODE_NOBLEND_QR ширина и высота выходного изображения определяются входным изображением.
- В режиме AVS_MODE_NOBLEND поддерживается вывод нескольких каналов; в режимах AVS_MODE_NOBLEND_VER, AVS_MODE_NOBLEND_HOR и AVS_MODE_NOBLEND_QR поддерживается вывод только одного канала.
- В режиме AVS_MODE_NOBLEND_HOR при сжатом выходе число сшиваемых потоков не должно превышать 4.
- В режиме AVS_MODE_NOBLEND_QR поддерживается сшивка только 4 потоков.

- Максимальные и минимальные входные разрешения для каждого режима приведены в следующей таблице.

5.18.1 Таблица 2. Максимальные и минимальные входные разрешения для режимов платформы RK3588

Режим сшивки	Разрешение			
	Минимальная ширина	Максимальная ширина	Минимальная высота	Максимальная высота
AVS_MODE_BLEND	1280	AVS_MAX_IN_WIDTH	720	AVS_MAX_IN_HEIGHT
AVS_MODE_NOBLEND_VER	1280	16384	720	16384
AVS_MODE_NOBLEND_HOR	1280	16384	720	16384
AVS_MODE_NOBLEND_QR	1280	16384	720	16384

5.19 AVS_LUT_ACCURACY_E

【Описание】

Определяет точность таблицы поиска.

【Определение】

```
typedef enum rkAVS_LUT_ACCURACY_E {
    AVS_LUT_ACCURACY_HIGH   = 0,
    AVS_LUT_ACCURACY_LOW    = 1,

    AVS_LUT_ACCURACY_BUTT
} AVS_LUT_ACCURACY_E;
```

【Члены】

Имя члена	Описание
AVS_LUT_ACCURACY_HIGH	Высокая точность таблицы поиска.
AVS_LUT_ACCURACY_LOW	Низкая точность таблицы поиска.

【Примечания】

- Точность таблицы поиска должна совпадать с точностью, выбранной при калибровке, иначе AVS не сможет работать корректно.
- Высокая точность LUT повышает точность сшивки, но производительность будет ниже, чем при использовании LUT низкой точности.

5.20 AVS_PARAM_SOURCE_E

【Описание】

Определяет тип источника параметров режима слияния.

【Определение】

```
typedef enum rkAVS_PARAM_SOURCE_E {
    AVS_PARAM_SOURCE_LUT   = 0,
    AVS_PARAM_SOURCE_CALIB = 1,

    AVS_PARAM_SOURCE_MODE_BUT
} AVS_PARAM_SOURCE_E;
```

【Члены】

Имя члена	Описание
AVS_PARAM_SOURCE_LUT	Параметры поступают из таблицы поиска.
AVS_PARAM_SOURCE_CALIB	Параметры поступают из файла калибровки.

Примечания

- Платформа RK3588 поддерживает оба варианта; AVS_PARAM_SOURCE_CALIB допускает передачу файла pto.
- Платформа RV1106 поддерживает только AVS_PARAM_SOURCE_CALIB.
- Платформы RV1109/RV1126 поддерживают только AVS_PARAM_SOURCE_CALIB.

5.21 AVS_FUSE_WIDTH_E

Описание

Определяет ширину полосы слияния низкочастотного слоя таблицы поиска.

Определение

```
typedef enum rkAVS_FUSE_WIDTH_E {
    AVS_FUSE_WIDTH_HIGH      = 0,
    AVS_FUSE_WIDTH_MEDIUM    = 1,
    AVS_FUSE_WIDTH_LOW       = 2,

    AVS_FUSE_WIDTH_BUTT
} AVS_FUSE_WIDTH_E;
```

Члены

Имя члена	Описание
AVS_FUSE_WIDTH_HIGH	Ширина полосы слияния 128 пикселей.
AVS_FUSE_WIDTH_MEDIUM	Ширина полосы слияния 256 пикселей.
AVS_FUSE_WIDTH_LOW	Ширина полосы слияния 512 пикселей.

Примечания

5.22 AVS_LUT_STEP_E

Описание

Определяет шаг выборки таблицы поиска.

Определение

```
typedef enum rkAVS_LUT_STEP_E {
    AVS_LUT_STEP_HIGH    = 0,
    AVS_LUT_STEP_MEDIUM  = 1,
    AVS_LUT_STEP_LOW     = 2,
    AVS_LUT_STEP_BUTT
} AVS_LUT_STEP_E;
```

Члены

Имя члена	Описание
AVS_LUT_STEP_HIGH	Шаг выборки таблицы поиска 16 пикселей.
AVS_LUT_STEP_MEDIUM	Шаг выборки таблицы поиска 32 пикселя.
AVS_LUT_STEP_LOW	Шаг выборки таблицы поиска 64 пикселя.

Примечания

- Чем меньше шаг выборки, тем больше таблица поиска и тем больше требуется времени вычисления.

5.23 AVS_STEP_ATTR_S

Описание

Определяет атрибуты шага выборки таблицы поиска.

Определение

```
typedef struct rkAVS_LUT_STEP_ATTR_S {
    AVS_LUT_STEP_E    enStepX;
    AVS_LUT_STEP_E    enStepY;
} AVS_STEP_ATTR_S;
```

Члены

Имя члена	Описание
enStepX	Шаг выборки таблицы поиска по направлению X.
enStepY	Шаг выборки таблицы поиска по направлению Y.

Примечания

5.24 AVS_LUT_S

Описание

Определяет атрибуты таблицы поиска.

Определение

```
typedef struct rkAVS_LUT_S {
    AVS_LUT_ACCURACY_E  enAccuracy;
    AVS_FUSE_WIDTH_E    enFuseWidth;
    AVS_STEP_ATTR_S     stLutStep;
    RK_U64              u64PhyAddr[AVS_PIPE_NUM];
} AVS_LUT_S;
```

Члены

Имя члена	Описание
enAccuracy	Точность таблицы поиска; должна совпадать с конфигурацией калибровки.
enFuseWidth	Ширина полосы слияния низкочастотного слоя таблицы поиска.
stLutStep	Шаг выборки таблицы поиска.
u64PhyAddr	Физические адреса таблиц поиска для каждого потока; данные LUT получены при калибровке.

Примечания

- Пользователь должен гарантировать корректность и действительность физического адреса LUT и её содержимого, иначе возможно зависание.
- enAccuracy должен совпадать со значением, использованным при калибровке.

Связанные типы данных и интерфейсы

Атрибуты входа сшивки AVS AVS_INPUT_ATTR_S.

5.25 AVS_CALIB_S

Описание

Определяет атрибуты файла калибровки.

Определение

```
typedef struct rkAVS_CALIB_S {
    const RK_CHAR *pCalibFilePath;
    const RK_CHAR *pMeshAlphaPath;
} AVS_CALIB_S;
```

Примечания

- Пользователь должен гарантировать корректность и действительность пути к файлу калибровки и его содержимого, иначе возможно зависание.

5.26 AVS_PROJECTION_MODE_E

Описание

Определяет режим проекции выходного сшитого изображения.

Определение

```
typedef enum rkAVS_PROJECTION_MODE_E {
    AVS_PROJECTION_EQUIRECTANGULAR = 0,
    AVS_PROJECTION_RECTILINEAR     = 1,
    AVS_PROJECTION_CYLINDRICAL     = 2,
    AVS_PROJECTION_CUBE_MAP        = 3,
    AVS_PROJECTION_EQUIRECTANGULAR_TRANS = 4,

    AVS_PROJECTION_BUTT
} AVS_PROJECTION_MODE_E;
```

Члены

Имя члена	Описание
AVS_PROJECTION_EQUIRECTANGULAR	Режим эквидистантной цилиндрической проекции.
AVS_PROJECTION_RECTILINEAR	Режим прямолинейной проекции.
AVS_PROJECTION_CYLINDRICAL	Режим цилиндрической проекции.
AVS_PROJECTION_CUBE_MAP	Режим кубической проекции.
AVS_PROJECTION_EQUIRECTANGULAR_TRANS	Режим поперечной эквидистантной цилиндрической проекции.

Примечания

- Если число сшиваемых потоков превышает AVS_SPLIT_PIPE_NUM, режим прямолинейной проекции AVS_PROJECTION_RECTILINEAR и режим кубической проекции AVS_PROJECTION_CUBE_MAP。
- AVS_PROJECTION_CUBE_MAP пока не поддерживаются.
- Платформа RV1106 поддерживает только режим эквидистантной цилиндрической проекции AVS_PROJECTION_EQUIRECTANGULAR.
- Платформы RV1106/RV1126 поддерживают только режим эквидистантной цилиндрической проекции AVS_PROJECTION_EQUIRECTANGULAR.

【Связанные типы данных и интерфейсы】

Нет.

5.27 AVS_GAIN_MODE_E

【Описание】

Определяет режим компенсации усиления яркости.

【Определение】

```
typedef enum rkAVS_GAIN_MODE_E {
    AVS_GAIN_MODE_MANUAL = 0,
    AVS_GAIN_MODE_AUTO   = 1,
    AVS_GAIN_MODE_BUTT
} AVS_GAIN_MODE_E;
```

【Члены】

Имя члена	Описание
AVS_GAIN_MODE_MANUAL	Ручной режим: пользователь вручную задаёт значения компенсации усиления яркости.
AVS_GAIN_MODE_AUTO	Автоматический режим: AVS автоматически оценивает различие яркости в области стыка и выполняет компенсацию усиления.

【Примечания】

- Пока не используется; зарезервированный тип данных.

【Связанные типы данных и интерфейсы】

Атрибуты компенсации усиления яркости AVS AVS_GAIN_ATTR_S.

5.28 AVS_GAIN_ATTR_S

【Описание】

Определяет атрибуты компенсации усиления яркости.

【Определение】

```
typedef struct rkAVS_GAIN_ATTR_S {
    AVS_GAIN_MODE_E enMode;
    RK_S32          s32Coef[AVS_PIPE_NUM];
} AVS_GAIN_ATTR_S;
```

【Члены】

Имя члена	Описание
enMode	Режим усиления яркости.
s32Coef[AVS_PIPE_NUM]	Значения усиления яркости каждого потока в ручном режиме. Диапазон значений: [0, 65535]. При значении 16384 яркость не корректируется; при значении меньше 16384 изображение становится темнее, больше 16384 — светлее.

【Примечания】

- Пока не используется; зарезервированный тип данных.

5.29 AVS_ROTATION_S

【Описание】

Определяет атрибуты поворота AVS.

【Определение】

```
typedef struct rkAVS_ROTATION_S {
    RK_S32 s32Yaw;
    RK_S32 s32Pitch;
    RK_S32 s32Roll;
} AVS_ROTATION_S;
```

【Члены】

Имя члена	Описание
s32Yaw	Угол рыскания. Диапазон значений: [-18000, 18000], единица: 0,01°.
s32Pitch	Угол тангажа. Диапазон значений: [-1000, 1000], единица: 0,01°.
s32Roll	Угол крена. Диапазон значений: [-1000, 1000], единица: 0,01°.

【Примечания】

Нет.

5.30 AVS_FOV_S

【Описание】

Определяет поле зрения AVS.

【Определение】

```
typedef struct rkAVS_FOV_S {
    RK_S32 u32FOVX;
    RK_S32 u32FOVY;
} AVS_FOV_S;
```

【Члены】

Имя поля	Описание
u32FOVX	Горизонтальный угол поля зрения, единица: 0,01°. При режиме проекции AVS_PROJECTION_EQUIRECTANGULAR диапазон значений: [1000, 36000]; при режиме проекции AVS_PROJECTION_RECTILINEAR диапазон значений: [1000, 11200]; при режиме проекции AVS_PROJECTION_CYLINDRICAL диапазон значений: [1000, 36000].
u32FOVY	Вертикальный угол поля зрения, единица: 0,01°. При режиме эквидистантной цилиндрической проекции AVS_PROJECTION_EQUIRECTANGULAR диапазон значений: [1000, 18000]; при режиме прямолинейной проекции AVS_PROJECTION_RECTILINEAR диапазон значений: [1000, 11200]; при режиме AVS_PROJECTION_CYLINDRICAL диапазон значений: [1000, 11200].

【Примечания】

При режиме проекции AVS_PROJECTION_CUBE_MAP этот параметр недействителен.

Связанные типы данных и интерфейсы

Режим проекции выходного сшитого изображения AVS_PROJECTION_MODE_E.

5.31 AVS_SPLIT_ATTR_S

Описание

Определяет атрибуты разбиения при сшивке 7 или 8 потоков.

Определение

```
typedef struct rkAVS_SPLIT_ATTR_S {
    RK_U32   u32PipeNum;
    AVS_PIPE AVSPipe[AVS_SPLIT_PIPE_NUM];
} AVS_SPLIT_ATTR_S;
```

Примечания

- Пользователь должен обеспечить корректность атрибутов разбиения, иначе на выходном изображении могут появиться чёрные блоки.
- Параметр действует только когда число сшиваемых потоков превышает AVS_SPLIT_PIPE_NUM.

  Нет.

5.32 AVS_CUBE_MAP_ATTR_S

Описание

Определяет атрибуты кубической проекции.

Определение

```
typedef struct rkAVS_CUBE_MAP_ATTR_S {
    RK_BOOL bBgColor;
    RK_U32 u32BgColor;
    RK_U32 u32SurfaceLength;
    POINT_S stStartPoint[AVS_CUBE_MAP_SURFACE_NUM];
} AVS_CUBE_MAP_ATTR_S;
```

Члены

Имя члена	Описание
bBgColor	Включение фонового цвета.
u32BgColor	Атрибут фонового цвета в формате RGB888.
u32SurfaceLength	Длина стороны одной грани кубической проекции; требуется выравнивание по 4. Диапазон значений: [256, 4096].
stStartPoint[AVS_CUBE_MAP_SURFACE_NUM]	Начальная позиция каждой грани кубической проекции на выходном изображении. Не должна выходить за границы изображения; требуется выравнивание по 4.

Примечания

- Пока не поддерживается; зарезервированный тип данных.
- Включение фонового цвета увеличивает нагрузку на модуль VGS и расход VB. Если развёрнутые грани полностью заполняют выходное изображение, рекомендуется отключить фон.
- Размер развёрнутых граней не должен превышать размер выходного изображения.
- При развёртке в режиме сшивки AVS_PROJECTION_CUBE_MAP, если выход настроен как сжатый, фон необходимо включить.
- При развёртке в режиме сшивки AVS_PROJECTION_CUBE_MAP, если выход настроен как сжатый, координата X начальной точки должна быть выровнена по 256, u32SurfaceLength также должен быть выровнен по 256.

Связанные типы данных и интерфейсы

Режим проекции выходного сшитого изображения AVS_PROJECTION_MODE_E.

5.33 AVS_INPUT_ATTR_S

【Описание】

Определяет атрибуты входа сшивки AVS.

【Определение】

```
typedef struct rkAVS_INPUT_ATTR_S {
    AVS_PARAM_SOURCE_E  enParamSource;
    union {
        AVS_LUT_S       stLUT;
        AVS_CALIB_S     stCalib;
    };
    SIZE_S              stSize;
} AVS_INPUT_ATTR_S;
```

【Члены】

Имя члена	Описание
enParamSource	Источник параметров входа сшивки.
stLUT	Атрибуты таблицы поиска.
stCalib	Атрибуты калибровки.
stSize	Разрешение входного изображения.

【Примечания】

- Тип enParamSource должен соответствовать конфигурации stLut или stCalib.
- Параметр действует только когда рабочий режим группы — режим слияния AVS_MODE_BLEND.
- Ограничения параметров для разных платформ см. в типе источника параметров режима слияния AVS_PARAM_SOURCE_E.
- На платформах RV1106/RV1109/RV1126 в режиме слияния необходимо задать допустимый stSize; конкретные диапазоны см. в AVS_MIN_IN_HEIGHT, AVS_MIN_IN_WIDTH、AVS_MAX_IN_HEIGHT、AVS_MAX_IN_WIDTH。

【Связанные типы данных и интерфейсы】

Рабочий режим AVS AVS_MODE_E.
Тип источника параметров режима слияния AVS AVS_PARAM_SOURCE_E.

5.34 AVS_OUTPUT_ATTR_S

【Описание】

Определяет атрибуты выхода сшивки AVS.

【Определение】

```
typedef struct rkAVS_OUTPUT_ATTR_S {
    AVS_PROJECTION_MODE_E    enPrjMode;
    POINT_S                  stCenter;
    AVS_FOV_S                stFOV;
    AVS_ROTATION_S           stORIRotation;
    AVS_ROTATION_S           stRotation;
    AVS_SPLIT_ATTR_S         stSplitAttr[AVS_SPLIT_NUM];
    AVS_CUBE_MAP_ATTR_S      stCubeMapAttr;
    SIZE_S                   stSize;
} AVS_OUTPUT_ATTR_S;
```

Члены

Имя члена	Описание
enPrjMode	Режим проекции выходного сшитого изображения.
stCenter	Положение центра проекции на выходном изображении. Обычно задаётся равным центру выходного изображения, то есть центр проекции совпадает с центром выходного изображения.
stFOV	Поле зрения выходной области сшивки.
stORIRotation	Атрибуты начального угла поворота выходной сшивки.
stRotation	Атрибуты поворота выходной сшивки.
stSplitAttr[AVS_SPLIT_NUM]	Атрибуты разбиения при сшивке 7 или 8 потоков.
stCubeMapAttr	Атрибуты кубической проекции. Недействительный параметр; пока не поддерживается.
stSize	Разрешение MESH.

Примечания

- Центр проекции stCenter — это положение центра проекции при преобразовании трёхмерной сферической панорамы в двумерное проекционное выходное изображение. При режиме AVS_PROJECTION_CUBE_MAP stCenter не действует.
- Обычно центр проекции совпадает с центром выходного изображения, то есть поле зрения FOV выходной проекции симметрично сверху, снизу, слева и справа. Если требуется обрезать выходное изображение и показывать только часть поля зрения, можно уменьшить FOV и переместить центр проекции, выбрав интересующую область для вывода.
- Диапазон координат X и Y центра проекции: (-16384, 16384).
- Центр проекции и поле зрения FOV необходимо настраивать совместно, иначе на краях выходного изображения возможны растяжение и заполнение пикселей.
- После перемещения центра проекции область за пределами максимального FOV не отображается; выступающую часть можно обрезать уменьшением FOV.

Связанные типы данных и интерфейсы

5.35 AVS_GRP_ATTR_S

Описание

Определяет атрибуты группы AVS.

Определение

```
typedef struct rkAVS_GRP_ATTR_S {
    AVS_MODE_E           enMode;
    RK_U32               u32PipeNum;
    RK_BOOL              bSyncPipe;
    AVS_INPUT_ATTR_S     stInAttr;
    AVS_GAIN_ATTR_S      stGainAttr;
    RK_U64               u64BBoxPhyAddr[AVS_PIPE_NUM];
    AVS_OUTPUT_ATTR_S    stOutAttr;
    FRAME_RATE_CTRL_S    stFrameRate;
} AVS_GRP_ATTR_S;
```

Члены

Имя члена	Описание
enMode	Режим сшивки. Статический атрибут, задаётся при создании GROUP и не может изменяться.
u32PipeNum	Число PIPE, то есть число сшиваемых потоков. Диапазон [1, AVS_PIPE_NUM]. Статический атрибут, задаётся при создании GROUP и не может изменяться.
bSyncPipe	Синхронизация изображений разных потоков по номеру кадра SeqID. Статический атрибут, задаётся при создании GROUP и не может изменяться.
stInAttr	Атрибуты входа сшивки.
stGainAttr	Атрибуты компенсации усиления яркости. Пока не используются.
u64BBoxPhyAddr[AVS_PIPE_NUM]	Физические адреса данных Bounding Box для каждого изображения. Пока не используются.
stOutAttr	Атрибуты выхода сшивки.
stFrameRate	Атрибуты частоты кадров. Подробное описание см. в разделе управления системой. Пока не используются.

Примечания

- Если bSyncPipe включён, AVS синхронизирует входные изображения по SeqID и выбирает группу изображений с одинаковым SeqID для сшивки. Это увеличивает число MB предыдущего модуля, занимаемых AVS, и во время синхронизации приводит к пропуску кадров предыдущего модуля. При отключённом bSyncPipe AVS не отбрасывает кадры предыдущего модуля.
- При передаче MB между AVS и предыдущим модулем через связь, созданную функцией RK_MPI_SYS_Bind, синхронизация кадров SyncPipe должна быть включена. При отправке MB функцией RK_MPI_AVS_SendPipeFrame рекомендуется отключить bSyncPipe; при необходимости выравнивание кадров реализует пользователь.
- Если u32PipeNum больше 6, поддерживаются только режим эквидистантной цилиндрической проекции AVS_PROJECTION_EQUIRECTANGULAR, режим цилиндрической проекции AVS_PROJECTION_CYLINDRICAL и режим поперечной эквидистантной цилиндрической проекции AVS_PROJECTION_EQUIRECTANGULAR_TRANS.

Связанные типы данных и интерфейсы

5.36 AVS_CHN_ATTR_S

Описание

Определяет атрибуты канала AVS.

Определение

```
typedef struct rkAVS_CHN_ATTR_S {
    RK_U32            u32Width;
    RK_U32            u32Height;
    COMPRESS_MODE_E   enCompressMode;
    DYNAMIC_RANGE_E   enDynamicRange
    RK_U32            u32Depth;
    FRAME_RATE_CTRL_S stFrameRate;
    RK_U32            u32FrameBufCnt;
} AVS_CHN_ATTR_S;
```

Члены

Имя члена	Описание
u32Width	Ширина канала в пикселях; требуется выравнивание, подробности см. в разделе характеристик входа и выхода.
u32Height	Высота канала в пикселях; требуется выравнивание, подробности см. в разделе характеристик входа и выхода.
enCompressMode	Режим сжатия канала. Подробности см. в разделе «Управление системой».
enDynamicRange	Динамический диапазон канала. Подробности см. в разделе «Управление системой».
u32Depth	Глубина очереди изображений канала. Диапазон [0, 8].
stFrameRate	Атрибуты частоты кадров. Подробности см. в разделе «Управление системой».
u32FrameBufCnt	Максимальное число буферов целевого изображения. Действует только при private-режиме буфера канала.

【Примечания】

- Управление частотой кадров канала пока не поддерживается; настройка не действует.
- u32Width и u32Height действуют только при сшивке AVS_MODE_BLEND.
- Сжатие COMPRESS_AFBC_16x16 поддерживается только каналом 0; остальные каналы сжатие не поддерживают.
- Высота остальных каналов не может быть меньше 1/15 высоты канала 0 и не может превышать высоту канала 0.
- Ширина остальных каналов не может быть меньше 1/15 ширины канала 0 и не может превышать половину ширины канала 0.
- Глубина очереди изображений канала используется только при получении изображений канала. Без необходимости рекомендуется устанавливать 0, иначе увеличивается число MB, занимаемых модулем AVS.
- Настройка динамического диапазона канала 0 не действует; его выходной динамический диапазон совпадает с входным изображением.
- Если u32FrameBufCnt меньше 3,
  - на платформе RK3588 внутренне принудительно используется 3, чтобы обеспечить минимально необходимое число выходных буферов AVS.
  - На платформах RV1106/RV1109/RV1126 такая обработка отсутствует.
- На платформе RV1106 при одинаковых значениях u32FrameBufCnt и u32Depth может не оказаться доступного выходного буфера.

【Связанные типы данных и интерфейсы】

5.37 AVS_ROI_PARAM_S

【Описание】

Определяет область интереса группы AVS.

【Определение】

```
typedef struct rkAVS_ROI_PARAM_S {
    RK_BOOL      bEnable;        /* RW; Range: [0, 1]; enable. */
    RECT_S       stRect;         /* Regin of Interest */
} AVS_ROI_PARAM_S;
```

【Члены】

Имя члена	Описание
bEnable	Переключатель включения ROI.
stRect	Область интереса.

【Примечания】

- Диапазон stRect прежде всего должен удовлетворять следующим ограничениям; подробности см. в AVS_MIN_OUT_HEIGHT, AVS_MIN_OUT_WIDTH, AVS_MAX_OUT_HEIGHT、AVS_MAX_OUT_WIDTH。
- Диапазон stRect должен находиться внутри настроенного разрешения MESH, иначе на краях выходного изображения возможны растяжение и заполнение пикселей.

【Связанные типы данных и интерфейсы】

5.38 AVS_FINAL_LUT_S

【Описание】

Определяет атрибуты итоговой таблицы поиска.

【Определение】

```
typedef struct rkAVS_FINAL_LUT_S {
    MB_BLK pMeshBlk[AVS_PIPE_NUM];
    MB_BLK pAlphaBlk[AVS_PIPE_NUM];
    MB_BLK pLdchBlk[AVS_PIPE_NUM];
    MB_BLK pParamBlk[AVS_PIPE_NUM];
} AVS_FINAL_LUT_S;
```

【Члены】

Имя члена	Описание
pMeshBlk	Данные MESH.
pAlphaBlk	Данные ALPHA.
pLdchBlk	Данные LDCH.
pParamBlk	Сведения о параметрах.

【Примечания】

Нет.

6. Коды ошибок AVS

Коды ошибок API панорамной сшивки AVS приведены ниже:

Код ошибки	Макроопределение	Описание
0xA0118001	RK_ERR_AVS_INVALID_DEVID	Недопустимый номер группы AVS.
0xA0118002	RK_ERR_AVS_INVALID_CHNID	Недопустимый номер канала AVS.
0xA0118003	RK_ERR_AVS_ILLEGAL_PARAM	Недопустимая настройка параметров AVS.
0xA0118004	RK_ERR_AVS_EXIST	Группа AVS уже создана.
0xA0118005	RK_ERR_AVS_UNEXIST	Группа AVS не создана.
0xA0118006	RK_ERR_AVS_NULL_PTR	Нулевой указатель во входном параметре.
0xA0118008	RK_ERR_AVS_NOT_SUPPORT	Операция не поддерживается.
0xA0118009	RK_ERR_AVS_NOT_PERM	Операция не разрешена.
0xA011800A	RK_ERR_AVS_INVALID_PIPEID	Недопустимый номер конвейера AVS.
0xA011800C	RK_ERR_AVS_NOMEM	Не удалось выделить память.
0xA011800D	RK_ERR_AVS_NOBUF	Не удалось выделить пул BUF.
0xA011800E	RK_ERR_AVS_BUF_EMPTY	Очередь изображений пуста.
0xA011800F	RK_ERR_AVS_BUF_FULL	Очередь изображений заполнена.
0xA0118010	RK_ERR_AVS_NOTREADY	Система AVS не инициализирована.
0xA0118012	RK_ERR_AVS_BUSY	Система AVS занята.

7. Связанные документы

«Руководство по использованию Rockchip AVS tool» Rockchip_Developer_Tutorial_AVS_Tool_CN.pdf «Руководство по отладке панорамной сшивки Rockchip RK3588 AVS» Rockchip_Developer_Guide_RK3588_AVS_CN.pdf

Описание отладочной информации Dump

Предисловие

Обзор

Отладочная информация собирается отдельным приложением dumpsys, в реальном времени отражает текущее состояние системы и может использоваться для локализации и анализа проблем.

Версии продуктов

Название микросхемы	Версия ядра
RK356X	4.19
RK3588	5.10

Целевая аудитория

Настоящий документ (руководство) предназначен главным образом для следующих специалистов:

инженеров технической поддержки;

инженеров-разработчиков программного обеспечения.

История изменений

Номер версии	Автор	Дата изменения	Описание изменения
V0.1.0	HXM	2021-02-23	Первая версия.
V0.1.1	XLM	2021-02-25	Добавлена отладка модулей vpss/rgn/sys.
V0.1.2	ZDD	2021-02-25	Добавлена отладка модулей adec/ao/ai/aenc.
V0.1.3	YWJ	2021-02-27	Добавлена отладка модуля vdec.
V0.1.4	XLM	2021-02-27	Добавлена отладка модуля venc.
V0.1.5	LXH	2021-04-13	Добавлена отладка модуля vi.
V0.1.6	XLM	2021-04-26	Обновлена выводимая информация sys/vpss. Добавлен переключатель записи данных vpss.
V0.2.0	LXH	2021-09-24	Обновлена отладочная информация модуля venc.
V0.3.0	LXH	2021-11-12	Обновлена отладочная информация модуля vi.
V0.4.0	XLM	2021-11-25	Добавлена отладочная информация модуля mb.
V0.5.0	JKM	2021-12-10	Добавлена отладка модуля avs. Добавлены переключатель записи данных avs и переключатель отладки синхронизации кадров.
V1.0.0	YWJ	2022-04-04	Добавлена отладочная информация модуля vdec.
V1.1.0	LXH	2022-04-08	Добавлена отладочная информация режима vi.
V1.2.0	LXH	2022-04-11	Добавлена отладочная информация режима vi.
V1.3.0	LXH	2022-04-14	Добавлена отладочная информация модуля gdc.
V1.4.0	LXH	2022-04-25	Добавлены проверка состояния канала vi и отладочная информация EDID.
V1.5.0	LXH	2022-07-23	Обновлена отладочная информация модуля venc.

Содержание

Обзор
SYS
MB
VPSS
RGN
VGS
ADEC
AENC
AO
AI
VI
VO
TDE
VDEC
VENC
AVS
GDC

1. ALL

2. Обзор

Отладочная информация в реальном времени отражает текущее состояние системы и может использоваться для локализации и анализа проблем.

2.1 Команда запуска

dumpsys имя_модуля

2.2 Список модулей

Имя модуля	Описание
sys	Записывает текущее использование модуля SYS.
mb	Записывает использование buffer модуля MB.
vgs	Сведения о состоянии блока подсистемы обработки видеоизображений.
vpss	Записывает конфигурацию атрибутов и состояние VPSS.
rgn	Записывает сведения о текущих ресурсах областей.
adec	Записывает конфигурацию атрибутов и состояние аудиодекодирования.
aenc	Записывает конфигурацию атрибутов и состояние AENC.
ao	Записывает конфигурацию атрибутов и состояние AO.
ai	Записывает конфигурацию атрибутов и состояние AI.
tde	Записывает состояние модуля TDE.
vdec	Записывает конфигурацию атрибутов и состояние видеодекодирования.
venc	Записывает конфигурацию атрибутов и состояние видеокодирования.
vi	Записывает конфигурацию атрибутов и состояние VI.
vo	Записывает конфигурацию атрибутов и состояние VO.
avs	Записывает конфигурацию атрибутов и состояние AVS.
all	Записывает конфигурацию и состояние всех зарегистрированных модулей.

Примечание

- Поддерживается получение конфигурации атрибутов и состояния нескольких зарегистрированных модулей.

  Формат команды:

  dumpsys [mod1]-[mod2]-[mod3]-....

  Пример команды:

  dumpsys vdec-vpss-vo

- Условия выполнения команды
  - Команду dumpsys можно выполнять только после инициализации приложения вызовом RK_MPI_SYS_Init.
  - Процесс приложения должен выполняться; после выхода приложения получить результат команды dumpsys невозможно.

3. SYS

Отладочная информация

```
[root@RK356X:/userdata]# ./dumpsys sys
-------------------------------------------------------------------------------
DUMP OF SERVICE sys:
---------------------- bind relation table -----------------------------
src_mod   src_dev   src_chn   dst_mod   dst_dev   dst_chn   src_recv_cnt   dst_recv_cnt
src_recv_rate  dst_recv_rate
vdec      0         0         vo        0         0         29040          28968          25.17
       25.19
vdec      0         1         vo        0         1         28981          28967          25.00
       24.79
vdec      0         2         vo        0         2         28979          28968          25.18
       24.99
vdec      0         3         vo        0         3         28976          28965          25.40
       24.81
```

```
vdec      0         4         vo        0         4         28974          28962          24.95
      25.02
vdec      0         5         vo        0         5         28974          28960          24.99
      24.64
vdec      0         6         vo        0         6         28969          28958          24.83
      25.18
vdec      0         7         vo        0         7         28966          28955          25.43
      25.14
vdec      0         8         vo        0         8         28967          28953          24.58
      24.84
vdec      0         9         vo        0         9         28964          28950          24.95
      24.98
vdec      0         10        vo        0         10        28962          28948          24.79
      25.20
vdec      0         11        vo        0         11        28960          28946          24.66
      25.18
vdec      0         12        vo        0         12        28957          28943          24.96
      24.98
vdec      0         13        vo        0         13        28956          28941          25.00
      25.00
vdec      0         14        vo        0         14        28952          28938          25.00
      24.64
vdec      0         15        vo        0         15        28951          28936          24.98
      25.00
vdec      0         16        vo        2         0         28949          28935          25.21
      25.19
vdec      0         17        vo        2         1         28946          28932          25.00
      25.00
vdec      0         18        vo        2         2         28944          28930          24.99
      24.82
vdec      0         19        vo        2         3         28941          28927          25.21
      25.18
vdec      0         20        vpss      0         0         90396          90382          78.28
      78.70
vpss      0         0         vo        2         4         90382          28921          78.70
      25.18
vpss      0         1         vo        2         5         90382          28917          78.70
      24.94
vdec      0         21        vpss      1         0         90380          90366          78.42
      78.29
vpss      1         0         vo        2         6         90366          28916          78.29
      25.04
vpss      1         1         vo        2         7         90366          28915          78.29
      25.07

-------------------------------------------------------------------------------
END DUMP OF SERVICE sys:
```

□Анализ отладочной информации□

Записывает текущее использование модуля SYS.

□Описание параметров□

Раздел параметров	Имя параметра	Описание
bind relation table Связи привязки между каналами модулей	src_mod	Имя первого модуля в связи; данные передаются от первого модуля второму.
src_dev Номер устройства первого уровня связи; данные передаются от первого уровня второму.		
src_chn Номер канала первого уровня связи; данные передаются от первого уровня второму.		
dst_mod Имя модуля второго уровня связи; данные передаются от первого уровня второму.		
dst_dev Номер устройства второго уровня связи; данные передаются от первого уровня второму.		
dst_chn Номер канала второго уровня связи; данные передаются от первого уровня второму.		
src_recv_cnt Счётчик данных, полученных первым уровнем, обычно в кадрах.		
dst_recv_cnt Счётчик данных, отправленных первым уровнем второму, обычно в кадрах.		
src_recv_rate Частота кадров данных, получаемых первым уровнем.		
dst_recv_rate Частота кадров данных, отправляемых первым уровнем второму.		

4. MB

Отладочная информация

- Краткая информация

```
[root@RK356X:/userdata]# dumpsys mb
-------------------------------------------------------------------------------
DUMP OF SERVICE mb:
---------------------- mb total statistics -----------------------------
mb_type   total_size     total_cnt
DMA       51796992       18
MALLOC    16             1

---------------------- mb module info -----------------------------
mod            chn_cnt        total_size          total_cnt
cmpi           1              47004672            16
vpss           1              4792320             2

---------------------- mb channel info -----------------------------
owner     mod       dev_id    chn_id    total_size     total_cnt      unused_cnt
private   cmpi      0         0         47004672       16             0
private   vpss      0         0         4792320        2              2

-------------------------------------------------------------------------------
END DUMP OF SERVICE mb:
```

- Подробная информация

```
[root@RK3588:/userdata]# dumpsys mb d
-------------------------------------------------------------------------------
DUMP OF SERVICE mb:
---------------------- mb total statistics -----------------------------
mb_type   total_size     total_cnt
DMA       23596032       9
MALLOC    16             1

---------------------- mb module info -----------------------------
mod            chn_cnt        total_size          total_cnt
```

```
cmpi           1              18803712            7
vpss           1              4792320             2

---------------------- mb channel info -----------------------------
owner     mod       dev_id    chn_id    total_size     total_cnt      unused_cnt
private   cmpi      0         0         18803712       7              0
private   vpss      0         0         4792320        2              2

---------------------- mb detail info -----------------------------
owner     mod       dev_id    chn_id    blk            virt               fd        phy_addr
       size           length         refs
other     cmpi      0         0         0x27a4ffe0     0x7facb68000       13        0
        3133440        3133440        1
other     cmpi      0         0         0x27a41b30     0x7fac56e000       16        0
        3133440        3133440        1
other     cmpi      0         0         0x27a4f6e0     0x7fac271000       17        0
        3133440        3133440        1
other     cmpi      0         0         0x27a9ab30     0x7f96500000       20        0
        3133440        3133440        3
other     cmpi      0         0         0x27aa0320     0x7f96203000       21        0
        3133440        3133440        3
other     cmpi      0         0         0x27aa06a0     0x7f95f06000       22        0
        3133440        3133440        3
other     cmpi      0         0         0x27a9fc20     0x7fbb2e6000       26        0
        3072           3072           1
other     vpss      0         0         0x27a4c3c0     0x7face65000       14        0
        1658880        1658880        0
other     vpss      0         0         0x27a44460     0x7fac86b000       15        0
        3133440        3133440        0

--------------------------------------------------------------------------------
END DUMP OF SERVICE mb:
```

Анализ отладочной информации

Записывает использование buffer текущего модуля MB.

Описание параметров

Раздел параметров	Имя параметра	Описание
mb total statistics Общая статистика mb	mb_type	Тип памяти.
total_size Общий размер памяти данного типа.		
total_cnt Общее число блоков памяти данного типа.		
mb module info Статистика по модулям	mod	Тип модуля.
chn_cnt Число каналов модуля.		
total_size Общий размер памяти модуля.		
total_cnt Общее число блоков памяти модуля.		
mb channel info Статистика по каналам	owner	Тип владельца.
mod Тип модуля.		
dev_id ID устройства.		
chn_id ID канала.		
total_size Общий размер памяти канала.		
total_cnt Общее число блоков памяти канала.		
unused_cnt Число свободных блоков памяти канала.		
mb detail info Подробная статистика mb	owner	Тип владельца.
mod Тип модуля.		
dev_id ID устройства.		
chn_id ID канала.		
blk Значение BLK.		
virt Виртуальный адрес.		
fd Дескриптор.		
phy_addr Физический адрес.		
size Размер памяти.		
length Используемая длина памяти.		
refs Счётчик ссылок.		

Команды отладки

- Dump данных mb

  Сохранение данных физической памяти, соответствующей виртуальному адресу.

  Пример: сохранить в tmp данные физического адреса, соответствующего виртуальному адресу mb 0x7f8b41c000, в файл mb.bin. Виртуальный адрес можно получить из поля virt команды dumpsys mb d.

  ```
  ./dumpsys mb viraddr 0x7f8b41c000 /tmp/mb.bin
  ```

5. VPSS

Отладочная информация

```
[root@RK3588:/]# dumpsys vpss
-------------------------------------------------------------------------------
DUMP OF SERVICE vpss:
---------------------- vpss mod param ----------------------------
mb_source
PRIVATE

---------------------- vpss group attr ----------------------------
grp_id    max_w     max_h     pixel_format   dym_range src_rate  dst_rate       is_compress
rotate   vproc_dev  max_queue
0         4096      4096      image:nv21     0         -1        -1             N              0
       GPU        0

---------------------- vpss channel attr1 ----------------------------
grp_id    chn_id    mode      width     height    pixel_format   is_compress    src_rate
dst_rate  depth     align     mirror    flip      frm_cnt
0         0         USER      1920      1080      image:bgr565   Y              -1        -1
   1         16        N         N         4
0         1         USER      1920      1080      image:bgr565   Y              -1        -1
   1         16        N         N         4

---------------------- vpss channel attr2 ----------------------------
grp_id    chn_id    rotate    aspect    video_x   video_y   video_w   video_h   bg_color
0         0         0         NONE      0         0         0         0         0x0
0         1         0         NONE      0         0         0         0         0x0

---------------------- vpss group crop info ----------------------------
grp_id    crop_en   coor_type x         y         width     height
0         N         RATIO     0         0         0         0

---------------------- vpss chn crop info ----------------------------
grp_id    chn_id    crop_en   coor_type x         y         width     height
0         0         N         RATIO     0         0         0         0
0         1         N         RATIO     0         0         0         0

---------------------- vpss group pic queue ----------------------------
grp_id    delay     backup    lft_in_cnt  lft_out_cnt now_cst_us  avg_cst_us  max_cost_us run_cnt
0         0         N         0           0           0           0           0           0

---------------------- vpss group pic info ----------------------------
grp_id    width     height    vir_w     vir_h     pix_format     dyn_range compress
0         1920      1080      1920      1080      image:nv12     4707540   Y

---------------------- vpss chn work status ----------------------------
grp_id    chn_id    get_frm_cnt    get_frm_rate   rel_frm_cnt    lft_in_cnt  lft_out_cnt
now_cst_us  avg_cst_us  max_cst_us  run_cnt
0         0         234            0.00           234            0           0           816
    902         5861        234
0         1         0              0.00           0              0           1           1640
   1634        5729        167

-------------------------------------------------------------------------------
END DUMP OF SERVICE vpss:
```

Анализ отладочной информации

Записывает конфигурацию атрибутов и состояние VPSS.

Описание параметров

Раздел параметров	Имя параметра	Описание
vpss mod param Атрибуты модуля VPSS	mb_source	Источник буфера памяти канала VPSS. COMMON: общий пул буферов памяти. PRIVATE: частный пул буферов памяти. USER: пользовательский пул буферов памяти.
vpss group attr Атрибуты GRP модуля VPSS	grp_id	Номер GRP ID. Допустимый диапазон: [0, VPSS_MAX_GRP_NUM).
max_w Максимальная ширина входного изображения группы.		
max_h Максимальная высота входного изображения группы.		
pixel_format Формат пикселей входного изображения группы.		
dym_range Динамический диапазон входного изображения группы.		
src_rate Исходная частота кадров GRP.		
dst_rate Целевая частота кадров GRP.		
is_compress Режим сжатия опорного кадра. Y: сжатие; N: без сжатия.		
rotate Угол поворота. Допустимые значения: 0, 90, 180, 270.		
vproc_dev Имя выбранного устройства.		
vpss channel attr1 Атрибуты канала 1 модуля VPSS	grp_id	Номер GRP ID. Допустимый диапазон: [0, VPSS_MAX_GRP_NUM).
chn_id Номер канала ID. Допустимый диапазон: [0, VPSS_MAX_CHN_NUM).		
	mode	Режим канала. USER: режим USER; AUTO: режим AUTO.
width Целевая ширина выходного изображения канала.		
height Целевая высота выходного изображения канала.		
pixel_format Целевой формат пикселей канала.		
is_compress Режим сжатия целевого изображения канала. Y: сжатие; N: без сжатия.		
src_rate Управление частотой кадров канала: исходная частота.		
dst_rate Управление частотой кадров канала: целевая частота.		
depth Длина очереди изображений канала, доступных пользователю.		
align Выравнивание ширины и высоты выходного YUV канала.		
	mirror	Включение функции mirror. Y: включено; N: выключено.
	flip	Включение функции flip. Y: включено; N: выключено.
frm_cnt Максимальное число буферов изображений канала.		
vpss channel attr2 Атрибуты канала 2 модуля VPSS	grp_id	Номер GRP ID. Допустимый диапазон: [0, VPSS_MAX_GRP_NUM).
chn_id Номер канала ID. Допустимый диапазон: [0, VPSS_MAX_CHN_NUM).		
rotate Угол поворота. Допустимые значения: 0, 90, 180, 270.		

Раздел параметров	Имя параметра	Описание
	aspect	Режим соотношения сторон. NONE: без обработки соотношения сторон. AUTO: автоматический расчёт соотношения сторон и пропорциональное масштабирование на весь экран. MANUAL: ручной расчёт соотношения сторон; область отображения определяется заданными значениями.
video_x Начальная координата по горизонтали для соотношения сторон.		Действует только в режиме MANUAL.
video_y Начальная координата по вертикали для соотношения сторон.		Действует только в режиме MANUAL.
video_w Ширина области соотношения сторон.		Действует только в режиме MANUAL.
video_h Высота области соотношения сторон.		Действует только в режиме MANUAL.
bg_color Цвет фона неотображаемой области.		
vpss group crop info Сведения CROP группы модуля VPSS	grp_id	Номер GRP ID. Допустимый диапазон: [0, VPSS_MAX_GRP_NUM).
	crop_en	Включение функции CROP. Y: включено. N: выключено.
	coor_type	Тип координат. RATIO: относительные координаты; ABS: абсолютные координаты.
	x	Начальная координата по горизонтали. При относительных координатах допустимый диапазон [0, 999]; при абсолютных координатах допустимый диапазон [0, VPSS_MAX_IMAGE_WIDTH]。
	y	Начальная координата по вертикали. При относительных координатах допустимый диапазон [0, 999]; при абсолютных координатах допустимый диапазон [0, VPSS_MAX_IMAGE_WIDTH]。
	width	Ширина CROP RECT. При относительных координатах допустимый диапазон (0, 1000]; при абсолютных координатах допустимый диапазон (0, VPSS_MAX_IMAGE_WIDTH]。
	height	Высота CROP RECT. При относительных координатах допустимый диапазон (0, 1000]; при абсолютных координатах допустимый диапазон (0, VPSS_MAX_IMAGE_WIDTH]。
vpss chn crop info Сведения CROP канала модуля VPSS	grp_id	Номер GRP ID. Допустимый диапазон: [0, VPSS_MAX_GRP_NUM).
	crop_en	Включение функции CROP. Y: включено. N: выключено.
chn_id Номер канала ID. Допустимый диапазон: [0, VPSS_MAX_CHN_NUM).		
	coor_type	Тип координат. RATIO: относительные координаты; ABS: абсолютные координаты.

Раздел параметров	Имя параметра	Описание
	x	Начальная координата по горизонтали. При относительных координатах допустимый диапазон [0, 999]; при абсолютных координатах допустимый диапазон [0, VPSS_MAX_IMAGE_WIDTH]。
	y	Начальная координата по вертикали. При относительных координатах допустимый диапазон [0, 999]; при абсолютных координатах допустимый диапазон [0, VPSS_MAX_IMAGE_WIDTH]。
	width	Ширина CROP RECT. При относительных координатах допустимый диапазон (0, 1000]; при абсолютных координатах допустимый диапазон (0, VPSS_MAX_IMAGE_WIDTH]。
	height	Высота CROP RECT. При относительных координатах допустимый диапазон (0, 1000]; при абсолютных координатах допустимый диапазон (0, VPSS_MAX_IMAGE_WIDTH]。
vpss group pic queue Состояние очереди буферизованных изображений GRP модуля VPSS.	grp_id	Номер GRP ID. Допустимый диапазон: [0, VPSS_MAX_GRP_NUM).
delay Длина очереди Delay.		
	backup	Включение backup-кадра. Y: включено; N: выключено.
lft_in_cnt Число входных кадров группы, ожидающих обработки.		
lft_out_cnt Число выходных кадров группы, ожидающих получения.		
now_cst_us Время обработки последнего кадра.		
avg_cst_us Среднее время обработки последних 10 кадров.		
max_cst_us Максимальное время обработки изображения.		
run_cnt Число обработок изображения.		
vpss group pic info Сведения о текущем обрабатываемом входном изображении.	grp_id	Номер GRP ID. Допустимый диапазон: [0, VPSS_MAX_GRP_NUM).
width Фактическая ширина входного изображения.		
height Фактическая высота входного изображения.		
vir_w Виртуальная ширина входного изображения.		
vir_h Виртуальная высота входного изображения.		
pix_format Фактический формат пикселей входного изображения.		
dyn_range Фактический динамический диапазон входного изображения.		
	compress	Режим сжатия входного изображения: Y: сжатие; N: без сжатия.
vpss chn work status Рабочее состояние канала VPSS	grp_id	Номер GRP ID. Допустимый диапазон: [0, VPSS_MAX_GRP_NUM).
chn_id Номер канала ID. Допустимый диапазон: [0, VPSS_MAX_CHN_NUM).		

Раздел параметров	Имя параметра	Описание
get_frm_cnt Число кадров, полученных пользователем через RK_MPI_VPSS_GetChnFrame.		
get_frm_rate Частота кадров, получаемых пользователем через RK_MPI_VPSS_GetChnFrame.		
rel_frm_cnt Число кадров, возвращённых пользователем через RK_MPI_VPSS_ReleaseChnFrame.		
lft_in_cnt Число входных кадров канала, ожидающих обработки.		
lft_out_cnt Число выходных кадров канала, ожидающих получения.		
now_cst_us Время обработки последнего кадра канала.		
avg_cst_us Среднее время обработки последних 10 кадров канала.		
max_cst_us Максимальное время обработки изображения каналом.		
run_cnt Число обработок канала.		

Примечание

- Для отображения сведений о повороте канала VPSS необходимо:
  - успешно вызвать RK_MPI_VPSS_SetChnRotation или RK_MPI_VPSS_SetChnRotationEx;
  - задать ненулевой угол поворота.

6. RGN

Отладочная информация

```
[root@RK356X:/userdata/test]# ./dumpsys rgn
-------------------------------------------------------------------------------
DUMP OF SERVICE rgn:
---------------------- region status of overlay ----------------------------
hdl       type      used      pixel_format    width     height    mb             virt
clut_num

---------------------- region chn status of overlay ----------------------------
hdl       type      mod       dev       chn       is_show   x         y         fg_alpha
bg_alpha  layer

---------------------- region status of cover ----------------------------
hdl       type      used
0         1         N
1         1         N
2         1         N
3         1         N

---------------------- region chn status of cover ----------------------------
hdl       type      mod       dev       chn       is_show   x         y         width     height
   color     layer     coord_type
0         1         venc      0         0         true      0         0         256       256
  0xf800    1         ABS
1         1         venc      0         0         true      64        64        256       256
  0xf800    1         ABS
2         1         venc      0         0         true      128       128       256       256
  0xf800    1         ABS
3         1         venc      0         0         true      192       192       256       256
  0xf800    1         ABS

---------------------- region status of mosaic ----------------------------
hdl       type      used

---------------------- region chn status of mosaic ----------------------------
```

```
hdl       type      mod       dev       chn       is_show   x         y         width     height
   blk_size  layer

--------------------------------------------------------------------------------
END DUMP OF SERVICE rgn:
```

Анализ отладочной информации

Записывает сведения о текущих ресурсах областей.

Описание параметров

Раздел параметров	Имя параметра	Описание
region status of overlay Состояние overlay	hdl	Номер Handle RGN.
type Тип OVERLAY, значение 0.		
	used	Используется ли ресурс. N: не используется; Y: используется.
pixel_format Формат пикселей OVERLAY, см. PIXEL_FORMAT_E.		
width Ширина области OVERLAY.		
height Высота области OVERLAY.		
mb ID памяти MB холста OVERLAY.		
virt Виртуальный адрес памяти холста OVERLAY.		
	clut_num	Число цветов палитры OVERLAY. Допустимый диапазон [0, 255]. 0: используется внутренняя палитра по умолчанию; другое значение: используется пользовательская палитра с числом цветов clut_num.
region chn status of overlay Состояние отображения OVERLAY в канале RGN	hdl	Номер Handle RGN.
type Тип OVERLAY, значение 0.		
mod Модуль, к которому выполнен Attach.		
dev Номер устройства.		
chn Номер канала.		
	is_show	Отображается ли область в канале. N: скрыта. Y: отображается.
x Начальная координата X отображения в канале.		
y Начальная координата Y отображения в канале.		
fg_alpha Alpha переднего плана при отображении в канале.		
bg_alpha Alpha фона при отображении в канале.		
layer Слой отображения в канале.		
region status of cover Состояние cover	hdl	Номер Handle RGN.
type Тип COVER, значение 1.		
	used	Используется ли ресурс. N: не используется; Y: используется.
region chn status of cover Состояние отображения COVER в канале RGN	hdl	Номер Handle RGN.
type Тип COVER, значение 1.		
mod Модуль, к которому выполнен Attach.		
dev Номер устройства.		
chn Номер канала.		

Раздел параметров	Имя параметра	Описание
	is_show	Отображается ли область в канале. N: скрыта. Y: отображается.
x Начальная координата области COVER по горизонтали.		При относительных координатах допустимый диапазон [0, 999];
y Начальная координата области COVER по вертикали.		При относительных координатах допустимый диапазон [0, 999];
width Ширина области COVER.		При относительных координатах допустимый диапазон (0, 1000];
height Высота области COVER.		При относительных координатах допустимый диапазон (0, 1000];
color Цвет COVER.		
layer Слой отображения в канале.		
	coord_type	Тип координат. RATIO: относительные координаты; ABS: абсолютные координаты.
region status of mosaic Состояние mosaic	hdl	Номер Handle MOSAIC.
type Тип MOSAIC, значение 2.		
	used	Используется ли ресурс. N: не используется; Y: используется.
region chn status of mosaic Состояние отображения MOSAIC в канале	hdl	Номер Handle MOSAIC.
type Тип MOSAIC, значение 2.		
mod Модуль, к которому выполнен Attach.		
dev Номер устройства.		
chn Номер канала.		
	is_show	Отображается ли область в канале. N: скрыта. Y: отображается.
x Начальная координата области MOSAIC по горизонтали.		
y Начальная координата области MOSAIC по вертикали.		
width Ширина области MOSAIC.		
height Высота области MOSAIC.		
blk_size Размер блока MOSAIC. Допустимые значения {8, 16, 32}.		
layer Слой отображения в канале.		

7. VGS

Отладочная информация

```
dumpsys vgs
-------------------------------------------------------------------------------
DUMP OF SERVICE vgs:
---------------------- module params -----------------------------
g_max_job_num       g_max_task_num
 128                 200
---------------------- recent job info1 ----------------------------
seq_no    job_hdl   state     task_num  in_size   out_size  cost_time   hw_time
0         0         proced    1         345600    353280    1553        0
---------------------- recent job info2 ----------------------------
seq_no    scale     cover     mosaic    osd       line      rotate    crop
0         1         0         0         0         0         0         0
---------------------- recent job info3 ----------------------------
job_seq   task_seq  srcw      srch      src_vir_w src_vir_h srcformat       src_rect_x
src_rect_x  src_rect_w  src_rect_h
0         0         720       480       720       480       image:nv12      0           0
  720         480
job_seq   task_seq  dstw      dsth      dst_vir_w dst_vir_h dstformat       dst_rect_x
dst_rect_x  dst_rect_w  dst_rect_h
0         0         720       480       736       480       image:bgra5551  0           0
  720         480
---------------------- max waste time recent job info1 ----------------------------
seq_no    job_hdl   state     task_num  in_size   out_size  cost_time   hw_time
0         0         proced    1         345600    353280    1553        0
---------------------- max waste time recent job info2 ----------------------------
seq_no    scale     cover     mosaic    osd       line      rotate    crop
0         1         0         0         0         0         0         0
---------------------- max waste time recent job info3 ----------------------------
job_seq   task_seq  srcw      srch      src_vir_w src_vir_h srcformat       src_rect_x
src_rect_x  src_rect_w  src_rect_h
0         0         720       480       720       480       image:nv12      0           0
  720         480
job_seq   task_seq  dstw      dsth      dst_vir_w dst_vir_h dstformat       dst_rect_x
dst_rect_x  dst_rect_w  dst_rect_h
0         0         720       480       736       480       image:bgra5551  0           0
  720         480
---------------------- vgs job status ----------------------------
success   fail      cancel    all_job_num     free_num  begin_num busy_num  procing_num
1         0         0         128             128       0         0         0
---------------------- vgs task status ----------------------------
success   fail      cancel    all_task_num    free_num  busy_num
1         0         0         200             199       1

-------------------------------------------------------------------------------
END DUMP OF SERVICE vgs:
```

Анализ отладочной информации

Записывает последние завершённые задачи модуля VGS, последние задачи с максимальным временем обработки, накопленную статистику и другие сведения.

Описание параметров

Параметр		Описание
module para	g_max_job_num	Максимальное число job.
g_max_task_num Максимальное число task.		
recent job info1. Сведения о недавно завершённых job	seq_no	Порядковый номер вывода, диапазон [0, 7].
job_hdl Номер handle данного job.		
task_num Число task в данном job.		
state Состояние обработки job.		
in_size Суммарная площадь входных изображений всех task данного job, в пикселях.		
out_size Суммарная площадь выходных изображений всех task данного job, в пикселях.		
cost_time Время от отправки job (end_job) до успешного завершения, в us.		
hw_time Время аппаратной обработки job, в us.		
recent job info2. Сведения о недавно завершённых job	crop	Включение CROP (0: выключено, 1: включено).
cover Включение Cover (0: выключено, 1: включено).		
mosaic Включение Mosaic (0: выключено, 1: включено).		
osd Включение OSD (0: выключено, 1: включено).		
line Включение Line (0: выключено, 1: включено).		
rotate Включение поворота (0: выключено, 1: включено).		
recent job info3. Входные и выходные сведения об изображениях task, добавленных в недавно завершённый job.	job_seq	Порядковый номер вывода job, диапазон [0, 7].
task_seq Порядковый номер task, добавленного в job.		
srcw Фактическая ширина входного изображения task.		
srch Фактическая высота входного изображения task.		
src_vir_w Виртуальная ширина входного изображения task.		
src_vir_h Виртуальная высота входного изображения task.		
srcformat Формат входного изображения task.		
src_rect_x Координата X начальной точки обрабатываемой области входного изображения task.		
src_rect_y Координата Y начальной точки обрабатываемой области входного изображения task.		
src_rect_w Ширина обрабатываемой области входного изображения task.		
src_rect_h Высота обрабатываемой области входного изображения task.		
dstw Фактическая ширина выходного изображения task.		
dsth Фактическая высота выходного изображения task.		
dst_vir_w Виртуальная ширина выходного изображения task.		
dst_vir_h Виртуальная высота выходного изображения task.		
dstformat Формат выходного изображения task.		
dst_rect_x Координата X начальной точки обрабатываемой области выходного изображения task.		

Параметр		Описание
dst_rect_x Координата Y начальной точки обрабатываемой области выходного изображения task.		
dst_rect_w Ширина обрабатываемой области выходного изображения task.		
dst_rect_h Высота обрабатываемой области выходного изображения task.		
max waste time job info. Сведения о недавно выполненном job с максимальным временем обработки.	Все поля совпадают с полями recent job info1 recent job info.	Сведения о job с максимальным временем среди последних 500 задач. После превышения 500 задач максимальное значение сбрасывается.
max waste time job info2. Сведения о недавно выполненном job с максимальным временем. job с максимальным временем обработки.	Все поля совпадают с полями recent job info2 recent job info.	Сведения о job с максимальным временем среди последних 500 задач. После превышения 500 задач максимальное значение сбрасывается.
max waste time job info3. Сведения о недавно выполненном job с максимальным временем. job с максимальным временем обработки.	Все поля совпадают с полями recent job info3 recent job info.	Сведения о job с максимальным временем среди последних 500 задач. После превышения 500 задач максимальное значение сбрасывается.
vgs job status. Состояние задач VGS	success	Накопленное число успешно обработанных job.
fail Накопленное число job, обработанных с ошибкой.		
cancel Накопленное число отменённых пользователем job.		
all_job_num Общее число доступных задач VGS.		
free_num Число свободных job.		
begin_num Число созданных пользователем, но ещё не отправленных job.		
busy_num Число отправленных пользователем, но ещё не переданных аппаратуре задач.		
procing_num Число задач, обрабатываемых аппаратурой.		
vgs task status. Состояние task VGS	success	Накопленное число успешно обработанных task. Если job обработан успешно, все входящие в него task также считаются успешно обработанными. К числу успешно обработанных task добавляется число task в успешном job.
	fail	Накопленное число task, обработанных с ошибкой. Если job завершился с ошибкой, все входящие в него task также считаются ошибочными. К числу ошибочных task добавляется число task в ошибочном job.
	cancel	Накопленное число отменённых task. Если job отменён, все входящие в него task также отменяются. К числу отменённых task добавляется число task в отменённом job. в ошибочном job.
all_task_num Общее число Task VGS, обычно 200.		
free_num Число свободных task.		
busy_num Число task, добавленных в job.		

Команды отладки

- Запись данных

  Записывает входные и выходные данные обработки task в job VGS.

  Пример: включить запись. Данные сохраняются в каталоге tmp, максимум 10 кадров.

  ```
  ./dumpsys vgs record /tmp/ 10
  ```

8. ADEC

【Отладочная информация】

```
dumpsys adec
-------------------------------------------------------------------------------
DUMP OF SERVICE adec:
---------------------- adec chn attr ----------------------------
chn_id    codec_id  buf_cnt   mode      rate      channel   orig_send_cnt  send_cnt  get_cnt
put_cnt
0         mp3       4         packet    44100     2         3320         3316      3315   3315
-------------------------------------------------------------------------------
```

【Анализ отладочной информации】

Записывает конфигурацию атрибутов и состояние текущего аудиодекодирования.

【Описание параметров】

Раздел параметров	Имя параметра	Описание
Атрибуты и состояние канала аудиодекодирования	chn_id	Номер канала.
codec_id Тип протокола декодирования.		
buf_cnt Число буферов кадров.		
mode Декодирование по потоку или по кадрам.		
rate Частота дискретизации потока.		
channel Число каналов потока.		
orig_send_cnt Число кадров потока, отправленных передней частью в декодер.		
send_cnt Число аудиокадров, успешно отправленных в декодер.		
get_cnt Число аудиокадров, полученных пользователем.		
put_cnt Число аудиокадров, освобождённых пользователем.		

9. AENC

【Отладочная информация】

```
dumpsys aenc
-------------------------------------------------------------------------------
DUMP OF SERVICE aenc:
---------------------- aenc chn attr ----------------------------
chn_id    codec_id  buf_cnt   rate      channel   bit_with
0         flac      4         44100     2         16bit
---------------------- aenc chn status ----------------------------
chn_id    recv_frame   enc_ok    frame_err  get_stream     release_stream
0         1248         69        0             69             69
-------------------------------------------------------------------------------
END DUMP OF SERVICE aenc:
```

【Анализ отладочной информации】

Записывает конфигурацию атрибутов и состояние текущего аудиокодирования.

【Описание параметров】

Раздел параметров	Имя параметра	Описание
Атрибуты и состояние канала аудиокодирования	chn_id	Номер канала.
codec_id Тип протокола кодирования.		
buf_cnt Число буферов кадров.		
rate Частота дискретизации данных аудиокадра PCM.		
channel Число каналов данных аудиокадра PCM.		
bit_with Разрядность дискретизации данных аудиокадра PCM.		
Состояние канала аудиокодирования	chn_id	Номер канала.
recv_frame Число принятых кадров.		
enc_ok Число успешно закодированных аудиокадров.		
frame_err Число аудиокадров, кодирование которых завершилось ошибкой.		
get_stream Число получений пользователем аудиопотока.		
release_stream Число освобождений пользователем аудиопотока.		

10. AO

Отладочная информация

```
dumpsys ao
--------------------------------------------------------------------------------
DUMP OF SERVICE ao:
---------------------- ao dev attr -----------------------------
ao_dev    snd_rate  snd_channel    snd_bit_Width  data_rate data_channel   data_bit_width chn_cnt
  expand_flag    frm_num        frm_size
0             48000       2        16bit          44100     stereo         16bit         2
  0             4             1024
---------------------- ao dev extend status ----------------------------
ao_dev         track_mode     mute           volume
0              0              N              100
---------------------- ao chn attr -----------------------------
ao_dev    ao_chn    card_name        snd_open       state          resample_open  in_rate in_ch
  out_rate  out_ch
0         0         hw:0,0           open           start          Y              44100   2
  48000     2
---------------------- ao chn status -----------------------------
ao_dev              ao_chn              frm_len         frm_total_cnt       frm_total_len
0                   0                   1024                511                 523264
---------------------- ao node status -----------------------------
ao_dev   ao_chn   node    in_size    in_cnt     proc_cnt    write_size
0        0        0       2048       2          0           0
0        0        1       0          0          0           0
0        0        2       0          0          0           0
0        0        3       4460       4          508         565012
END DUMP OF SERVICE ao:
```

Анализ отладочной информации

Записывает конфигурацию атрибутов и состояние AO.

Описание параметров

Раздел параметров	Имя параметра	Описание
Атрибуты канала аудиовыхода	ao_dev	Номер устройства.
snd_rate Частота дискретизации открытой звуковой карты.		
snd_channel Число каналов открытой звуковой карты.		
snd_bit_Width Разрядность открытой звуковой карты.		
data_rate Частота дискретизации отправляемых данных.		Диапазон: [8k, 96k].
	data_channel	Число каналов отправляемых данных. mono: моно; stereo: стерео.
data_bit_width Разрядность дискретизации отправляемых данных.		Диапазон: [8bit, 16bit].
chn_cnt Пока не используется; можно задать значение по умолчанию 2.		
expand_flag Пока не используется; можно задать значение по умолчанию 0.		
frm_num Максимальное число buffer для отправляемых кадров.		
frm_size Размер buffer отправляемого кадра.		
Расширенные сведения об устройстве аудиовыхода	ao_dev	Номер устройства.
	track_mode	Режим каналов. 0：normal 1：both_left 2：both_right 3：exchange 4：mix 5：left_mute 6：right_mute 5：both_mute
	mute	Включена ли функция отключения звука. Y: включена. N: выключена.
volume Уровень громкости (0–100).		
Атрибуты канала аудиовыхода	ao_dev	Номер устройства.
ao_chn Номер канала.		
card_name Имя звуковой карты.		
	snd_open	Открыта ли звуковая карта: close: закрыта; open: открыта.
	state	Состояние канала. idle: состояние ожидания. pause: состояние паузы. start: рабочее состояние.
	resample_open	Включена ли передискретизация. Y: включена. N: выключена.
in_rate Исходная частота дискретизации для передискретизации.		
in_ch Исходное число каналов для передискретизации.		

Раздел параметров	Имя параметра	Описание
out_rate Целевая частота дискретизации.		
out_ch Целевое число каналов.		
Сведения о канале аудиовыхода	ao_dev	Номер устройства.
ao_chn Номер канала.		
frm_len Длина текущего кадра, отправляемого AO.		
frm_total_cnt Накопленное число кадров, отправленных AO.		
frm_total_len Накопленная длина кадров, отправленных AO.		
Сведения об узлах канала аудиовыхода	ao_dev	Номер устройства.
ao_chn Номер канала.		
node Номер узла.		
in_size Текущая длина буферизованных данных inputBuffer узла.		
in_cnt Текущее число inputBuffer узла.		
proc_cnt Накопленный счётчик потока функции process, только для узла playback.		
write_size Длина buffer, записанного в звуковую карту, только для узла playback.		

11. AI

Отладочная информация

```
dumpsys ai
-------------------------------------------------------------------------------
DUMP OF SERVICE ai:
---------------------- ai dev attr -----------------------------
ai_dev    snd_rate  snd_channel    snd_bit_Width  data_rate data_channel   data_bit_width chn_cnt
  expand_flag    frm_num        frm_size
0         48000      2             16bit          16000       stereo       16bit           2
     0            4                1024
---------------------- ai dev extend status ----------------------------
ai_dev    track_mode
0             0
---------------------- ai chn attr -----------------------------
ai_dev    ai_chn    card_name        snd_open       state       resample_open  in_rate   in_ch
  out_rate  out_ch
0         0         hw:1,0           open           start       Y              48000     2
  16000     2
---------------------- ai chn status -----------------------------
ai_dev       ai_chn        frm_len        frm_total_cnt    frm_total_len
0            0             1364           682              133250
---------------------- ai node status -----------------------------
ai_dev         ai_chn         node           in_size        in_cnt
0              0              0              0              0
0              0              1              4096           1
0              0              2              0              0
END DUMP OF SERVICE ai:
```

Анализ отладочной информации

Записывает конфигурацию атрибутов и состояние AI.

Описание параметров

Раздел параметров	Имя параметра	Описание
Атрибуты устройства аудиовхода	ai_dev	Номер устройства.
snd_rate Частота дискретизации открытой звуковой карты.		
snd_channel Число каналов открытой звуковой карты.		
snd_bit_Width Разрядность открытой звуковой карты.		
data_rate Частота дискретизации получаемых данных.		
	data_channel	Число каналов получаемых данных. mono: моно; stereo: стерео.
data_bit_width Разрядность дискретизации получаемых данных.		Диапазон: [8bit, 16bit].
chn_cnt Пока не используется; можно задать значение по умолчанию 2.		
expand_flag Пока не используется; можно задать значение по умолчанию 0.		
frm_num Пока не используется AI; можно задать значение по умолчанию 4.		
frm_size Заданная приложением длина получаемого кадра, в byte.		
Расширенные сведения об устройстве аудиовхода	ai_dev	Номер устройства.
	track_mode	Режим каналов. 0：normal 1：both_left 2：both_right 3：exchange 4：mix 5：left_mute 6：right_mute 5：both_mute
Атрибуты канала аудиовхода	ai_dev	Номер устройства.
ai_chn Номер канала.		
card_name Имя звуковой карты.		
	snd_open	Открыта ли звуковая карта: close: закрыта; open: открыта.
	state	Состояние канала. idle: состояние ожидания. pause: состояние паузы. start: рабочее состояние.
	resample_open	Включена ли передискретизация. Y: включена. N: выключена.
in_rate Исходная частота дискретизации для передискретизации.		
in_ch Исходное число каналов для передискретизации.		
out_rate Целевая частота дискретизации.		
out_ch Целевое число каналов.		
Сведения о канале аудиовхода	ai_dev	Номер устройства.

Раздел параметров	Имя параметра	Описание
ai_chn Номер канала.		
frm_len Длина текущего кадра, полученного AI.		
frm_total_cnt Накопленное число кадров, полученных AI.		
Сведения об узлах канала аудиовхода	ai_dev	Номер устройства.
ai_chn Номер канала.		
node Номер узла.		
in_size Текущая длина буферизованных данных inputBuffer узла.		
in_cnt Текущее число inputBuffer узла.		

12. VI

Отладочная информация

```
[root@RK356X:/]# dumpsys vi
-------------------------------------------------------------------------------
DUMP OF SERVICE vi:
---------------------- vi module_param ----------------------------
vi_max_chn_num
4

---------------------- vi chn attr ----------------------------
pipe  chn   width     height    max_width      max_height     compress_mode
0     0     1920      1080      0              0              0
memory_type    buf_type       pix_format     buf_count      buf_size       entity_name
4              0              image:nv12     3              3133440        rkispp_scale0

---------------------- vi chn query state ----------------------------
pipe chn width height enabled input_lost output_lost frame_id framerate vbfail freeze
0    0   1920  1080   1       0          1           233      25        0      0

---------------------- vi chn get buf state ----------------------------
pipe  chn   seq   pts          delay_us  poll_cnt  poll_fail_cnt get_cnt  release_cnt
0     0     233   4835459158   52942     233       0             233      233
qbuf_fail_cnt  commit_cnt  refs
0              3           1 1 1 0 0 0 0 0

---------------------- vi chn user picture attr ----------------------------
pipe chn  enable  mode  bg_color frame        width height pix_format refs seq   pts
0    0    Y       bgc   0x80     0x7f4023d2b0 1920  1080   image:nv12  3   2378  225580325

---------------------- vi chn connect info ----------------------------
pipe  chn   width  height  pix_format        connect
0     0     1920   1080    image:nv12        unknown

---------------------- vi chn edid info ----------------------------
pipe  chn  max_block  block_id  pad  edid_crc
0     1    2          0         0    a3
0     1    2          1         0    b5
---------------------- vi chn stream info ----------------------------
pipe  chn  codec_type
0     1    image:h265
-------------------------------------------------------------------------------
END DUMP OF SERVICE vi:
```

□Анализ отладочной информации□

Записывает конфигурацию атрибутов и состояние VI.

□Описание параметров□

Раздел параметров	Имя параметра	Описание
Модуль устройства видеовхода. Параметр	vi_max_chn_num	Максимальное число каналов, поддерживаемых устройством видеовхода.
Параметры канала видеовхода	pipe	Номер конвейера.
chn Номер канала.		
width Ширина изображения канала.		
height Высота изображения канала.		
max_width Максимальная ширина изображения канала, пока не используется.		
max_height Максимальная высота изображения канала, пока не используется.		
compress_mode Режим сжатия изображения канала: 0 — без сжатия; 1 — afbc.		
memory_type Тип памяти изображения канала: 0 — mmap; 1 — userptr; 2 — overlay; 3 — dma.		
buf_type Выделение памяти изображения канала: 0 — внутреннее; 1 — внешнее.		
pix_format Формат пикселей изображения канала.		
buf_count Число выделенных буферов изображения канала.		
buf_size Размер одного буфера изображения канала.		
entity_name Имя устройства канала.		
Состояние канала видеовхода	pipe	Номер конвейера.
chn Номер канала.		
width Ширина изображения канала.		
height Высота изображения канала.		
enabled Состояние включения канала.		
input_lost Число потерянных входных кадров канала.		
output_lost Число потерянных выходных кадров канала.		
frame_id Номер кадра выходного изображения канала.		
framerate Частота кадров канала.		
vbfail Число ошибок получения буфера каналом.		
freeze Включение замораживания видеовыхода канала: 0 — выключено; 1 — включено.		
Состояние buffer видеовхода	pipe	Номер конвейера.
chn Номер канала.		
seq Текущий порядковый номер seq buffer канала.		
pts PTS текущего buffer канала.		
delay_us Задержка текущего buffer канала, в us.		
poll_cnt Число запросов buffer каналом.		
poll_fail_cnt Число неудачных запросов buffer каналом. Постоянный рост означает, что buffer получить невозможно; возможно, драйвер работает некорректно.		
get_cnt Число получений buffer каналом.		
release_cnt Число освобождений buffer каналом.		

Раздел параметров	Имя параметра	Описание
qbuf_fail_cnt Число ошибок возврата buffer каналом.		
commit_cnt Число запрошенных buffer канала.		
refs Число ссылок на buffer канала; по умолчанию отображаются ссылки максимум для 8 buffer.		
Вставка изображения видеовхода. Сведения	pipe	Номер конвейера.
chn Номер канала.		
enable Включение функции вставки изображения: Y — включена; N — выключена.		
mode Режим вставки изображения: none — выключено; pic — вставка локального изображения; bgc — вставка фонового цвета.		
bg_color Значение фонового цвета, действует при вставке фонового цвета.		
frame Адрес указателя кадра вставляемого изображения.		
width Ширина кадра вставляемого изображения.		
height Высота кадра вставляемого изображения.		
pix_format Формат кадра вставляемого изображения.		
refs Число ссылок на кадр вставляемого изображения.		
seq Порядковый номер seq кадра вставляемого изображения.		
pts PTS кадра вставляемого изображения.		
Сведения о подключении канала	pipe	Номер конвейера.
chn Номер канала.		
width Ширина изображения, поступающего в канал.		
height Высота изображения, поступающего в канал.		
pix_format Формат изображения, поступающего в канал.		
connect Состояние подключения канала, HDMI RX.		unknown: неизвестно; connect: подключено; disconnect: не подключено.
Сведения EDID канала	pipe	Номер конвейера.
chn Номер канала.		
max_block Максимальное число block.		
block_id Текущий block ID.		
pad Значение pad.		
edid_crc Текущее значение CRC EDID блока.		
Сведения codec канала	pipe	Номер конвейера.
chn Номер канала.		
codec_type Заданный тип codec.		

13. VO

Отладочная информация

```
[root@RK356X:/]# dumpsys vo
--------------------------------------------------------------------------------
DUMP OF SERVICE vo:
-----------DEV CONFIG------------
DevId   DevEn   InfType InfSync       Vcnt
0       N       Unkown  Unkown        0
1       Y       EDP     1024x768p60   0
2       N       Unkown  Unkown        0
-------LAYER BIND CONFIG--------
DevId   Video   Gfx     Cursor
0       clu0    esm0    sm0
1       clu1    esm1    sm1
2       unkown  unkown  unkown
----------LAYER STATUS---------
LayId   LayEn   PixFmt  ImgW    ImgH    DispX   DispY   DispW   DispH   FrmRt   BufLens Comprs
ByPass VoDev
0       N       NV12    0       0       0       0       0       0       0       N       N
     -1
1       N       NV12    0       0       0       0       0       0       0       N       N
     -1
2       Y       BGR24   1024    768     0       0       1024    768     25      3       Y       N
     -1
3       N       NV12    0       0       0       0       0       0       0       N       N
     -1
4       N       NV12    0       0       0       0       0       0       0       N       N
     -1
5       Y       BGRA32  1024    768     0       0       1024    768     25      3       N       N
     -1
6       N       NV12    0       0       0       0       0       0       0       N       N
     -1
7       N       NV12    0       0       0       0       0       0       0       N       N
     -1
8       Y       BGR24   1920    1080    0       0       1920    1080    25      4       N       N
     0
9       Y       BGR24   1920    1080    0       0       1920    1080    25      4       N       N
     0
10      Y       BGR24   1920    1080    0       0       1920    1080    25      4       N       N
     0
11      Y       BGR24   1920    1080    0       0       1920    1080    25      4       N       N
     0
LAYER clu1 CHANNEL STATUS:
ChnId  Prio  X    Y    W    H   ChnFrt  FgAlpha BgAlpha KeyEn  Color Show  GDC  Pause  Step
Cache RevThr  RevCnt
0      0     960  540  960  540  0      128     0       N      0      Y     Y    N      N     4
2        34
127    127   0    0    960  540  0      255     0       N      0      Y     N    N      N     1
2        1
LAYER esm1 CHANNEL STATUS:
ChnId   Prio    X       Y       W       H       ChnFrt  FgAlpha BgAlpha Show    Pause   Step
Cache RevThr RevCnt
0       0       0       0       1024    768     25      128     0       Y       N       N     1
  3       83
1       1       0       0       1024    768     25      128     0       Y       N       N     1
  3       83
WBC STATUS:
WbcId   En      Src     W       H       Fmt     Comprs  Frt     Depth   SendCnt
0       N       Dev0    0       0       NV12    N       0       0       0
--------------------------------------------------------------------------------
```

Анализ отладочной информации

Записывает конфигурацию атрибутов и состояние VO.

Описание параметров

Раздел параметров	Имя параметра	Описание
Состояние устройства вывода изображения	DevId	Номер устройства вывода.
DevEn Состояние включения устройства вывода: Y — включено; N — выключено.		
InfType Тип интерфейса вывода.		
InfSync Синхронизация интерфейса вывода.		
Vcnt Строчное прерывание устройства вывода.		
Связь слоёв	DevId	Номер устройства вывода.
Video Имя видеослоя.		
Gfx Имя графического слоя.		
Cursor Имя слоя курсора.		
Состояние слоя	LayId	Номер устройства слоя.
LayEn Состояние включения слоя: Y — включён; N — выключен.		
PixFmt Формат пикселей слоя.		
ImgW Ширина холста слоя.		
ImgH Высота холста слоя.		
DispX Координата X верхнего левого угла области отображения слоя.		
DispY Координата Y верхнего левого угла области отображения слоя.		
DispW Ширина области отображения слоя.		
DispH Высота области отображения слоя.		
FrmRt Частота обновления слоя.		
BufLens Число буферов холста слоя.		
Comprs Состояние сжатия холста слоя: Y — сжатие; N — без сжатия.		
ByPass Включён ли сквозной режим слоя: Y — включён; N — выключен.		
VoDev ID устройства вывода, привязанного к слою; -1 означает отсутствие привязки.		
Состояние канала слоя	ChnId	Номер устройства канала.
Prio Приоритет канала.		
X Координата X верхнего левого угла области отображения канала.		
Y Координата Y верхнего левого угла области отображения канала.		
W Ширина области отображения канала.		
H Высота области отображения канала.		
ChnFrt Частота отображения кадров канала.		
FgAlpha Действует для данных канала в формате RGBA5551; значение Alpha при A=1.		
BgAlpha Действует для данных канала в формате RGBA5551; значение Alpha при A=0.		
KeyEn Включение ключевого цвета канала: Y — включён; N — выключен.		
Color Значение ключевого цвета канала в шестнадцатеричном виде.		
Show Видимость канала: Y — видим; N — скрыт.		

Раздел параметров	Имя параметра	Описание
GDC Включена ли коррекция искажений канала: Y — включена; N — выключена.		
Pause Состояние воспроизведения канала: Y — пауза; N — воспроизведение.		
Step Включён ли покадровый шаг канала: Y — включён; N — выключен.		
Cache Число кадров в кэше канала.		
RevThr Порог отображения видеовыхода канала.		
RevCnt Число кадров, принятых каналом; сбрасывается при отключении или включении канала.		
Состояние обратной записи	WbcId	Номер устройства обратной записи.
En Состояние включения обратной записи: Y — включена; N — выключена.		
Src Имя источника данных обратной записи.		
W Ширина данных обратной записи.		
H Высота данных обратной записи.		
Fmt Формат данных обратной записи.		
Frt Частота кадров данных обратной записи.		
Depth Число буферизованных кадров обратной записи.		
SendCnt Число отправленных кадров обратной записи.		

14. TDE

Отладочная информация

```
dumpsys tde
--------------------------------------------------------------------------------
DUMP OF SERVICE tde:
---------------------- module params ----------------------------
g_max_job_num       g_max_task_num
 128                 200
---------------------- recent job info1 ----------------------------
seq_no    job_hdl   state     task_num  in_size   out_size  cost_time   hw_time
0         0         proced    1         76800     518400    1160        0
---------------------- recent job info2 ----------------------------
seq_no    copy      fill      resize    bitblit   rotate
0         1         0         0         0         0
---------------------- recent job info3 ----------------------------
job_seq   task_seq  srcw      srch      src_vir_w src_vir_h srcformat     src_rect_x  src_rect_x
src_rect_w  src_rect_h
0         0         320       240       320       240       image:nv12    0           0
320         240
job_seq   task_seq  dstw      dsth      dst_vir_w dst_vir_h dstformat     dst_rect_x  dst_rect_x
dst_rect_w  dst_rect_h
0         0         320       240       960       540       image:nv12    56          0
320         240
---------------------- max waste time recent job info1 ----------------------------
seq_no    job_hdl   state     task_num  in_size   out_size  cost_time   hw_time
0         0         proced    1         76800     518400    1160        0
---------------------- max waste time recent job info2 ----------------------------
seq_no    copy      fill      resize    bitblit   rotate
0         1         0         0         0         0
---------------------- max waste time recent job info3 ----------------------------
```

```
job_seq   task_seq  srcw      srch      src_vir_w src_vir_h srcformat     src_rect_x  src_rect_x
src_rect_w  src_rect_h
0         0         320       240       320       240       image:nv12    0           0
320         240
job_seq   task_seq  dstw      dsth      dst_vir_w dst_vir_h dstformat     dst_rect_x  dst_rect_x
dst_rect_w  dst_rect_h
0         0         320       240       960       540       image:nv12    56          0
320         240
---------------------- tde job status ----------------------------
success   fail      cancel    all_job_num     free_num  begin_num busy_num  procing_num
1         0         0         128             128       0         0         0
---------------------- tde task status ----------------------------
success   fail      cancel    all_task_num    free_num  busy_num
1         0         0         200             199       1
-------------------------------------------------------------------------------
END DUMP OF SERVICE tde:
```

□Анализ отладочной информации□

Записывает последние завершённые задачи модуля TDE, последние задачи с максимальным временем обработки, накопленную статистику и другие сведения.

□Описание параметров□

Параметр		Описание
module para	g_max_job_num	Максимальное число job.
g_max_task_num Максимальное число task.		
recent job info1. Сведения о недавно завершённых job	seq_no	Порядковый номер вывода, диапазон [0, 7].
job_hdl Номер handle данного job.		
task_num Число task в данном job.		
state Состояние обработки job.		
in_size Суммарная площадь входных изображений всех task данного job, в пикселях.		
out_size Суммарная площадь выходных изображений всех task данного job, в пикселях.		
cost_time Время от отправки job (end_job) до успешного завершения, в us.		
hw_time Время аппаратной обработки job, в us.		
recent job info2. Сведения о недавно завершённых Сведения job	copy	Включение COPY (0: выключено, 1: включено).
fill Включение FILL (0: выключено, 1: включено).		
resize Включение Resize (0: выключено, 1: включено).		
bitblit Включение BITBLIT (0: выключено, 1: включено).		
rotate Включение поворота (0: выключено, 1: включено).		
recent job info3. Входные и выходные сведения об изображениях task, добавленных в недавно завершённый job.	job_seq	Порядковый номер вывода job, диапазон [0, 7].
task_seq Порядковый номер task, добавленного в job.		
srcw Фактическая ширина входного изображения task.		
srch Фактическая высота входного изображения task.		
src_vir_w Виртуальная ширина входного изображения task.		
src_vir_h Виртуальная высота входного изображения task.		
srcformat Формат входного изображения task.		
src_rect_x Координата X начальной точки обрабатываемой области входного изображения task.		
src_rect_y Координата Y начальной точки обрабатываемой области входного изображения task.		
src_rect_w Ширина обрабатываемой области входного изображения task.		
src_rect_h Высота обрабатываемой области входного изображения task.		
dstw Фактическая ширина выходного изображения task.		
dsth Фактическая высота выходного изображения task.		
dst_vir_w Виртуальная ширина выходного изображения task.		
dst_vir_h Виртуальная высота выходного изображения task.		
dstformat Формат выходного изображения task.		
dst_rect_x Координата X начальной точки обрабатываемой области выходного изображения task.		
dst_rect_x Координата Y начальной точки обрабатываемой области выходного изображения task.		

Параметр		Описание
dst_rect_w Ширина обрабатываемой области выходного изображения task.		
dst_rect_h Высота обрабатываемой области выходного изображения task.		
max waste time job info1. Сведения о недавно выполненном job с максимальным временем. job с максимальным временем обработки.	Все поля совпадают с полями recent job info1 recent job info.	Сведения о job с максимальным временем среди последних 500 задач. После превышения 500 задач максимальное значение сбрасывается.
max waste time job info2. Сведения о недавно выполненном job с максимальным временем. job с максимальным временем обработки.	Все поля совпадают с полями recent job info2 recent job info.	Сведения о job с максимальным временем среди последних 500 задач. После превышения 500 задач максимальное значение сбрасывается.
max waste time job info3. Сведения о недавно выполненном job с максимальным временем. job с максимальным временем обработки.	Все поля совпадают с полями recent job info3 recent job info.	Сведения о job с максимальным временем среди последних 500 задач. После превышения 500 задач максимальное значение сбрасывается.
tde job status. Состояние задач TDE	success	Накопленное число успешно обработанных job.
fail Накопленное число job, обработанных с ошибкой.		
cancel Накопленное число отменённых пользователем job.		
all_job_num Общее число доступных задач TDE.		
free_num Число свободных job.		
begin_num Число созданных пользователем, но ещё не отправленных job.		
busy_num Число отправленных пользователем, но ещё не переданных аппаратуре задач.		
procing_num Число задач, обрабатываемых аппаратурой.		
tde task status. Состояние task TDE	success	Накопленное число успешно обработанных task. Если job обработан успешно, все входящие в него task также считаются успешно обработанными. К числу успешно обработанных task добавляется число task в успешном job.
	fail	Накопленное число task, обработанных с ошибкой. Если job завершился с ошибкой, все входящие в него task также считаются ошибочными. К числу ошибочных task добавляется число task в ошибочном job.
	cancel	Накопленное число отменённых task. Если job отменён, все входящие в него task также отменяются. К числу отменённых task добавляется число task в отменённом job. в ошибочном job.
all_task_num Общее число Task TDE, обычно 200.		
free_num Число свободных task.		
busy_num Число task, добавленных в job.		

Команды отладки

- Запись данных

  Записывает входные и выходные данные обработки task в job TDE.

  Пример: включить запись. Данные сохраняются в каталоге tmp, максимум 10 кадров.

  ```
  ./dumpsys tde record /tmp/ 10
  ```

15. VDEC

Отладочная информация

```
[root@RK356X:/userdata]# ./dumpsys vdec
-------------------------------------------------------------------------------
DUMP OF SERVICE vdec:
---------------------- MODULE PARAM ----------------------------
vdec_max_chn_num    mb_src
64                  1
---------------------- CHN COMM ATTR & PARAM ----------------------------
id        type           width          height         vir_width      vir_height     disp_mode
   state
0         image:h265     640            360            640            384            1
   start
1         image:h264     720            576            768            592            1
   start
2         image:h264     720            576            768            592            1
   start
id        send_mode      send_timeout   set_user_pic   en_user_pic    attach_pool
0         frame          300            0              0              0
1         frame          300            0              0              0
2         frame          300            0              0              0
---------------------- CHN VIDEO ATTR & PARAMS ----------------------------
id        compress       en_dei         en_mv          pixfmt
0         Y              N              Y              N/A
1         Y              N              Y              N/A
2         Y              N              Y              N/A
---------------------- CHN STATE ----------------------------
id        send           send_ok        send_rate      max_input_cnt  left_input_cnt
left_input_size max_output_cnt left_output_cnt left_output_size err_status
0         8748           5094           25.00          10             8              37749
    8              0              0              0
1         8658           5094           25.11          10             10             48005
    8              0              0              0
2         8825           5095           25.00          10             10             46747
    8              0              0              0
---------------------- CHN DECODE BUFFER STATE ----------------------------
id        input_strm_cnt output_frm_cnt error_frm_cnt  unused_buf_cnt put_buf_cnt
0         7134           6061           2              0              6057
1         7134           6061           2              0              6057
2         7134           6061           2              0              6057
-------------------------------------------------------------------------------
END DUMP OF SERVICE vdec:
```

Анализ отладочной информации

Записывает конфигурацию атрибутов и состояние текущего видеодекодирования.

Описание параметров

Параметр		Описание
module param	vdec_max_chn_num	Максимальное число каналов декодирования, поддерживаемых VDEC.
	mb_src	Способ выделения памяти декодированных кадров. 1: модуль MODULE; 2: частный PRIVATE; 3: пользователь USER.
chn comm attr & params	id	Номер канала декодирования.
	type	Тип канала декодирования. image:h264; image:h265； image:jpeg;
width Ширина декодированного изображения.		
height Высота декодированного изображения.		
vir_width Виртуальная ширина декодированного изображения.		
vir_height Виртуальная высота декодированного изображения.		
dispMode Режим отображения (0: реального времени, 1: воспроизведения).		
state Состояние воспроизведения (start: канал запущен, stop: канал остановлен).		
send_mode Режим отправки потока канала декодирования. FRAME: по кадрам; STREAM: потоковая отправка.		
send_timeout Тайм-аут ожидания потока канала декодирования, ms; -1 — постоянная блокировка.		
set_user_pic Задано ли пользовательское изображение.		
en_user_pic Включено ли пользовательское изображение.		
attach_pool Используется ли внешний пул памяти.		
CHN VIDEO ATTR & PARAMS	id	Номер канала декодирования.
compress Режим сжатия (0: без сжатия, 1: сжатие).		
en_dei Включён ли deinterlace.		
en_mv Включён ли colmv.		
pixfmt Формат вывода JPEG; N/A для не-JPEG.		
CHN STATE	id	Номер канала декодирования.
send Число отправок потока.		
send_ok Число успешных отправок потока.		
send_rate Частота кадров отправляемого потока.		
max_input_buf_cnt Максимальное число входных буферов.		
left_input_buf_cnt Число неиспользованных входных buffer.		
left_input_size Размер неиспользованных входных buffer.		

Параметр		Описание
max_output_buf_cnt Максимальное число выходных буферов.		
left_output_buf_cnt Число оставшихся выходных кадров.		
left_output_size Размер оставшихся выходных кадров.		
err_status Код ошибки канала.		
CHN DECODE BUFFER STATE	id	Номер канала декодирования.
input_strm_cnt Число входных stream декодера.		
output_frm_cnt Число выходных frame декодера.		
error_frm_cnt Число ошибочных кадров декодера.		
unused_buf_cnt Число доступных buf декодера.		
put_buf_cnt Число buf, возвращённых декодеру.		

16. VENC

Отладочная информация

```
[root@RK356X:/userdata]# ./dumpsys venc
-------------------------------------------------------------------------------
DUMP OF SERVICE venc:
---------------------- venc module_param ----------------------------
venc_max_chn_num buf_cache  frm_buf_cyc  264_one_pkt  265_one_pkt  jpeg_one_pkt
16               0          0            multi        one          one

---------------------- venc chn attr ----------------------------
id  width   height  vir_w   vir_h   codec_type  pix_format  buf_count  buf_size  rc_mode
gop_mode  gop
0   1920    1080    1920    1080    8           image:nv12  5          3110400   H264CBR
NORMALP   60
vir_idr_len
0

---------------------- venc chn query stat ----------------------------
id        left_pics      left_strm_bytes     left_strm_frms      cur_packs      left_recv_pics
0         0              0                   0                   0              0

---------------------- venc chn rc info1 ----------------------------
id  start_qp  step_qp  max_qp  min_qp  max_i_qp  min_i_qp  delt_ip_qp  qfactor  qf_min  qf_max
0   26        8        51      10      46        24        2           0        0       0

---------------------- venc chn rc info2 ----------------------------
id  stat_time  set_min_bps set_avg_bps set_max_bps get_min_bps get_avg_bps get_max_bps
0   30         10000000    10240000    10240000    9085631     10546106     10761830

---------------------- venc chn roi info ----------------------------
id        index     is_intra  abs_qp    qp        x         y         width     height

---------------------- venc chn osd info ----------------------------
id  region  total_size  enable  inverse  start_x  start_y  width  height  offset

---------------------- venc chn dump status ----------------------------
id  in_fps    out_fps   seq     snap_set  attach  attach_cnt  attach_size fail_strm_cnt
0   24.05     23.98     207     10000     N       0           0           0
```

```
---------------------- venc chn param ----------------------------
id  crop_mode  src_x  src_y  src_w  src_h  dst_x  dst_y  dst_w  dst_y  fps_en  src_fps_set
dst_fps_set
0   none       0      0      0      0      0      0      0      0      0            0/0
0/0

---------------------- venc chn frame dump info1 -----------------------------
id  width  height vir_w   vir_h  hor_w ver_h  format     afbc_mode eos  pts delay_us  lost
0   1920   1080   1920    1080   1920  1080   image:nv12 0x0       0    0   10980     0

---------------------- venc chn frame dump info2 -----------------------------
id  rect_x  rect_y  rect_w  rect_h  send_idx  send_ok_idx get_idx   get_ok_idx  now_cst_us
avg_cst_us  max_cst_us  drop_cnt
0   0       0       0       0       60314     60314       60314     60313       4151        4154
       4375        0

---------------------- venc chn dump base config -----------------------------
id  rotation  mirror
0   0         none

---------------------- venc chn dump h264 config -----------------------------
id  intra_pred  tran_mode  chroma_qp  entropy  cabac_init  dblk_dis  dblk_a  dblk_b  full_range
0   0           0          -6         CABAC    0           0         0       0       Y

---------------------- venc chn dump h265 config -----------------------------
id  cb_qp  cr_qp  scaling_list  dblk_dis  dblk_a  dblk_b   sao_luma  sao_cr  pu_sis_en
full_range

---------------------- venc chn dump rc adv param ----------------------------
id  clear_stat
0   1

---------------------- venc chn dump super frm -----------------------------
id  super_frm_mode  rc_priority  Iframe_bits_thr  Pframe_bits_thr
0   0               0            0               0

---------------------- venc chn dump frm lost -----------------------------
id  frm_lost_open  frm_lost_bps_thr  frm_lost_mode  enc_frm_gaps
0   0              0                 0              0

---------------------- venc chn dump intra refresh -----------------------------
id  refresh_en  refresh_mode  refresh_num  req_i_qp
0   0           0             0            0

---------------------- venc chn dump hierarchical qp -----------------------------
id  hier_qp_en  hier_qp_delta       hier_frame_num
0   0           0    0    0    0    0    0    0    0

---------------------- venc chn dump de breath effect -----------------------------
id  de_breath_en  strength0  strength1
0   0             0          0

---------------------- venc chn dump ref param -----------------------------
id  base  enhance  en_preb
0   0      0        0

---------------------- venc chn dump slice split -----------------------------
id  enable  mode  size
0   Y       1     16

--------------------------------------------------------------------------------
END DUMP OF SERVICE venc:
```

Анализ отладочной информации

Записывает конфигурацию атрибутов и состояние текущего видеокодирования.

Параметр		Описание
venc module param	venc_max_chn_num	Максимальное число каналов кодирования, поддерживаемых VENC.
buf_cache Поддерживается ли получение потока через cache. Пока не используется.		
frm_buf_cyc Освобождается ли память кадров. Пока не используется.		
264_one_pkt Режим формирования кадров потока H264 (one: один пакет; multi: несколько пакетов).		
265_one_pkt Режим формирования кадров потока H265 (one: один пакет; multi: несколько пакетов).		
jpeg_one_pkt Режим формирования кадров потока MJPEG/JPEG (one: один пакет; multi: несколько пакетов).		
venc chn attr	id	Номер канала кодирования.
width Ширина кодируемого изображения.		
height Высота кодируемого изображения.		
vir_width Виртуальная ширина кодируемого изображения.		
vir_height Виртуальная высота кодируемого изображения.		
codec_type Тип формата кодируемого видео (H264/H265/JPEG/MJPEG).		
pix_format Формат кодируемого изображения.		
buf_count Максимальное число буферов выходных пакетов при кодировании.		
buf_size Размер одного выходного buf при кодировании.		
rc_mode Режим RC кодирования.		
gop_mode Режим GOP кодирования.		
gop Длина GOP кодирования.		
vir_idr_len Длина виртуального I-кадра кодирования.		
venc chn query stat	id	Номер канала кодирования.
left_pics Число изображений, ожидающих кодирования.		
left_strm_bytes Число оставшихся byte в buffer потока.		
left_strm_frms Число оставшихся кадров в buffer потока.		
cur_packs Число пакетов потока текущего кадра.		
left_recv_pics Число оставшихся кадров, ожидающих приёма.		
venc chn rc info1	id	Номер канала кодирования.
start_qp QP начального кодируемого кадра.		
step_qp Шаг QP кодирования.		
max_qp Максимальный QP кодирования.		
min_qp Минимальный QP кодирования.		
max_i_qp Максимальный QP I-кадра.		
min_i_qp Минимальный QP I-кадра.		
delt_ip_qp Разность между средним QP пяти P-кадров перед I-кадром и QP этого I-кадра.		
qfactor Параметр качества кодирования MJPEG/JPEG.		
qf_min Минимальный параметр качества MJPEG, действует при CBR/VBR.		

Параметр		Описание
qf_max Максимальный параметр качества MJPEG, действует при CBR/VBR.		
venc chn rc info2	id	Номер канала кодирования.
stat_time Период статистики битрейта кодирования, в секундах.		
set_min_bps Заданное минимальное значение битрейта, kbps.		
set_avg_bps Заданное среднее значение битрейта, kbps.		
set_max_bps Заданное максимальное значение битрейта, kbps.		
get_min_bps Полученное минимальное значение битрейта, kbps; обновляется раз в десять секунд.		
get_avg_bps Полученное среднее значение битрейта, kbps; обновляется раз в десять секунд.		
get_max_bps Полученное максимальное значение битрейта, kbps; обновляется раз в десять секунд.		
venc chn roi info	id	Номер канала кодирования.
index Индекс области ROI.		
is_intra Является ли кадр I-кадром.		
abs_qp Режим QP области ROI. N: относительный QP. Y: абсолютный QP.		
qp Значение QP.		
x Начальная точка области ROI по горизонтали.		
y Начальная точка области ROI по вертикали.		
width Ширина области ROI.		
height Высота области ROI.		
venc chn osd info	id	Номер канала кодирования.
region Номер области OSD, диапазон [0, 7].		
total_size Общий размер buf OSD.		
enable Включена ли OSD для области. N: выключена. Y: включена.		
inverse Инвертируется ли цвет OSD области. N: нет. Y: да.		
start_x Начальное смещение X области OSD.		
start_y Начальное смещение Y области OSD.		
width Ширина области OSD.		
height Высота области OSD.		
offset Смещение области OSD в общем buf.		
venc chn dump status	id	Номер канала кодирования.
in_fps Входная частота кадров кодирования.		
out_fps Выходная частота кадров кодирования.		
seq Порядковый номер выходного кодированного кадра.		
snap_set Заданное число выходных кодированных кадров.		
attach Является ли выходной buf attach pool. Y: да; N: нет.		
attach_cnt Число buf attach pool.		

Параметр		Описание
attach_size Размер buf attach pool.		
fail_strm_cnt Число кадров, для которых не удалось получить выходной поток. В многопакетном режиме ошибка возникает, когда output packet_cnt меньше числа пакетов		текущего кадра; такой кадр отбрасывается.
venc chn param	id	Номер канала кодирования.
crop_mode Режим обрезки кодирования.		none: обрезка выключена; crop_only: только обрезка; crop_scale: обрезка и масштабирование.
src_x crop_only: координата X начальной точки обрезки;		crop_scale: координата X начальной точки обрезки и масштабирования.
src_y crop_only: координата Y начальной точки обрезки;		crop_scale: координата Y начальной точки обрезки и масштабирования.
src_w crop_only: ширина обрезаемого изображения;		crop_scale: ширина изображения для обрезки и масштабирования.
src_h crop_only: высота обрезаемого изображения;		crop_scale: высота изображения для обрезки и масштабирования.
dst_x crop_scale: координата X целевой позиции.		
dst_y crop_scale: координата Y целевой позиции.		
dst_w crop_scale: ширина целевого изображения.		
dst_y crop_scale: высота целевого изображения.		
fps_en Включение управления частотой кадров кодирования; 0: выключено; 1: включено.		
src_fps_set Заданная входная частота кадров управления кодированием.		
dst_fps_set Заданная выходная частота кадров управления кодированием.		
venc chn frame dump info1	id	Номер канала кодирования.
width Фактическая ширина входного кадра кодирования.		
height Фактическая высота входного кадра кодирования.		
vir_w Фактическая ширина памяти входного кадра кодирования.		
vir_h Фактическая высота памяти входного кадра кодирования.		
hor_w Ширина памяти входного кадра после преобразования согласно формату.		
ver_h Высота памяти входного кадра после преобразования согласно формату.		
format Фактический формат входного кадра кодирования.		
afbc_mode Режим сжатия входного кадра кодирования.		0x0: без сжатия; 0x100000: AFBC V1; 0x200000: AFBC V2.
eos Признак окончания входного кадра кодирования. 1: конец; 0: не конец.		
pts PTS входного кадра кодирования.		
delay_us Задержка от входного кадра до выхода кодирования — полное время от первоначального назначения PTS этому кадру до завершения кодирования.		
lost Число потерянных входных кадров кодирования.		
venc chn frame dump info2	id	Номер канала кодирования.

Параметр		Описание
rect_x Смещение X области входного кадра кодирования.		
rect_y Смещение Y области входного кадра кодирования.		
rect_w Ширина W области входного кадра кодирования.		
rect_h Высота H области входного кадра кодирования.		
send_idx Число отправленных кадров на кодирование.		
send_ok_idx Число успешно отправленных кадров.		
get_idx Число запросов кодированных кадров.		
get_ok_idx Число успешно полученных кодированных кадров.		
now_cst_us Время кодирования последнего кадра, us.		
avg_cst_us Среднее время кодирования последних десяти кадров, us.		
max_cst_us Максимальное время кодирования, us.		
drop_cnt Общее число выходных кадров потока, отброшенных после сброса канала; после сброса не полученные данные кодирования отбрасываются.		
venc chn dump base config	id	Номер канала кодирования.
rotation Поворот пути кодирования: 0/90/180/270 градусов или illegal — недопустимый параметр.		
mirror Зеркальное отражение пути кодирования: horizontal — горизонтальное; vertical — вертикальное; both — горизонтальное и вертикальное; none — без отражения; illegal — недопустимый параметр.		
venc chn dump h264 config	id	Номер канала кодирования H264.
intra_pred Атрибут внутрикадрового предсказания;		0: выключено, предсказание без ограничений; 1: включён режим ограниченного предсказания.
tran_mode Атрибуты преобразования и квантования канала кодирования;		0: поддерживаются trans4x4 и trans8x8; 1: поддерживается trans4x4.
chroma_qp Смещение QP цветности.		
entropy Атрибут энтропийного кодирования; 0: CAVLC; 1: CABAC.		
cabac_init Номер таблицы инициализации, используемой при инициализации связанных переменных (0-2）。		
	dblk_dis	Определяет, отключается ли работа фильтра деблокинга кодирования на некоторых границах блоков полосы, и для каких границ фильтр отключается. 0: включён; 1: выключен; 2: выключен на границах slice.
dblk_a Смещение при обращении к таблицам alpha и tC0 фильтра деблокинга для макроблоков slice.		
dblk_b Смещение при обращении к таблице beta фильтра деблокинга для макроблоков slice.		
full_range Использует ли H264 полный цветовой диапазон. Y: да; N: нет.		
venc chn dump h265 config	id	Номер канала кодирования H265.
cb_qp Смещение концентрации синего компонента при кодировании.		
cr_qp Смещение концентрации красного компонента при кодировании.		

Параметр		Описание
scaling_list Включение таблицы преобразования и квантования канала. 0: выключена; 1: включена.		
	dblk_dis	Определяет, отключается ли работа фильтра деблокинга кодирования на некоторых границах блоков полосы, и для каких границ фильтр отключается. 0: включён; 1: выключен; 2: выключен на границах slice.
dblk_a Смещение при обращении к таблицам alpha и tC0 фильтра деблокинга для макроблоков slice.		
dblk_b Смещение при обращении к таблице beta фильтра деблокинга для макроблоков slice.		
sao_luma Применяется ли фильтр SAO к компоненту яркости текущего slice; 0: нет; 1: да.		
sao_cr Применяется ли фильтр SAO к компоненту цветности текущего slice; 0: нет; 1: да.		
pu_sis_en Условное использование билинейной интерполяции во внутреннем фильтре предсказания CVS; 0: не используется; 1: не используется.		
full_range Использует ли H265 полный цветовой диапазон. Y: да; N: нет.		
venc chn dump rc adv param	id	Номер канала кодирования.
clear_stat Очищать ли статистику управления битрейтом после задания нового битрейта канала; 0: не очищать; 1: очищать.		
venc chn dump super frm	id	Номер канала кодирования.
super_frm_mode Режим обработки сверхбольших кадров; 0: не обрабатывать; 1: отбросить; 2: перекодировать кадр.		
	rc_priority	Приоритет перекодирования сверхбольшого кадра; 0: при super_frm_mode = 0 значение 0 означает, что обработка сверхбольших кадров не включена; 1: приоритет целевого битрейта; 2: приоритет порога сверхбольшого кадра.
Iframe_bits_thr Порог сверхбольшого I-кадра в bit.		
Pframe_bits_thr Порог сверхбольшого P-кадра в bit.		
venc chn dump frm lost	id	Номер канала кодирования.
frm_lost_open Переключатель пропуска кадров; 0: выключен; 1: включён.		
frm_lost_bps_thr Порог пропуска кадров в процентах.		
frm_lost_mode Режим пропуска кадров: 0 — обычный пропуск при превышении порога мгновенным битрейтом; 1 — кодирование кадра pskip при превышении порога мгновенным битрейтом.		
enc_frm_gaps Максимальное допустимое число последовательных пропущенных кадров. 0 означает пропуск всех.		
venc chn dump intra refresh	id	Номер канала кодирования.
refresh_en Включение функции обновления I-slice; 0: выключена; 1: включена.		
refresh_mode Режим обновления I-макроблоков: по строкам или столбцам; 0: по строкам; 1: по столбцам.		
refresh_num Число строк или столбцов I-макроблоков, обновляемых за один раз.		
req_i_qp Значение QP I-кадра. Пока не используется.		
venc chn dump hierarchical qp	id	Номер канала кодирования.

Параметр		Описание
hier_qp_en Включение иерархии QP; 0: выключена; 1: включена.		
hier_qp_delta Разность QpDelta P-кадра уровня 0 относительно каждого уровня.		
hier_frame_num Число P-кадров каждого уровня.		
venc chn dump de breath effect	id	Номер канала кодирования.
de_breath_en Включение подавления эффекта дыхания; 0: выключено; 1: включено.		
strength0 Параметр 0 силы подавления эффекта дыхания. Пока не используется.		
strength1 Параметр 1 силы подавления эффекта дыхания.		
venc chn dump ref param	id	Номер канала кодирования.
base Период базового слоя расширенного пропуска кадров.		
enhance Период расширенного слоя расширенного пропуска кадров.		
en_preb Используются ли кадры базового слоя как опорные для других кадров базового слоя; 0: эквивалентно бесконечному base; 1: все кадры базового слоя ссылаются на IDR-кадр.		
venc chn dump slice split	id	Номер канала кодирования.
enable Включение разделения slice.		Y: да; N: нет.
mode Режим разделения slice.		0: по byte; 1: по MB или LCU.
	size	Размер разбиения slice. При split_mode = 0 — число байтов каждого slice; при split_mode = 1 — число MB или LCU каждого slice.

17. AVS

Отладочная информация

```
[root@RK3588:/]# dumpsys avs
--------------------------------------------------------------------------------
DUMP OF SERVICE avs:
---------------------- avs mod param ----------------------------
work_set_size  mb_source
68608          PRIVATE
---------------------- avs group attr ----------------------------
grp_id    mode           enable    pipe_num  is_sync   src_rate  dst_rate
0         BLEND          Y         6         Y         -1        -1
---------------------- avs lut attr ----------------------------
grp_id    lut_data_acc   lut_data_path
0         HIGH           NONE
---------------------- avs output attr ----------------------------
grp_id    proj_mode               center_x  center_y  fov_x     fov_y
0         EQUIRECTANGULAR         2048      2080      28000     9500

          ori_yaw   ori_pitch ori_roll  yaw       pitch         roll
0         0         0         0         0         0             0

          middel_lut_path     calib_path          mask_path           mesh_alpha_path
```

```
0         NONE               /usr/share/avs_calibNONE               /usr/share/avs_calib
                             /calib_file.pto                        /
---------------------- avs channel attr ----------------------------
grp_id    chn_id    enable    width     height    is_compress    dym_range depth     src_rate
dst_rate
0         0         Y         8176      2720      Y              0         0         -1        -1
---------------------- avs group work status -----------------------------
grp_id    now_cst_us  avg_cst_us  max_cst_us  now_itv_us  run_cnt     suc_run_cnt fail_run_cnt
0         20610       20460       25595       33333       185         185         0
---------------------- avs pipe work status ----------------------------
grp_id    pipe_id_0 pipe_id_1 pipe_id_2 pipe_id_3 pipe_id_4 pipe_id_5 pipe_id_6 pipe_id_7
0         318       318       319       318       318       318       0         0
          7         7         7         7         7         9         0         0
---------------------- avs pipe queue ----------------------------
grp_id    pipe_id_0 pipe_id_1 pipe_id_2 pipe_id_3 pipe_id_4 pipe_id_5 pipe_id_6 pipe_id_7
0


          Y(339)    Y(339)    Y(339)              Y(339)    Y(339)

------------------------------------------------------------------------------
END DUMP OF SERVICE avs:
```

Анализ отладочной информации

Записывает конфигурацию атрибутов и состояние текущей панорамной сшивки.

Описание параметров

Параметр		Описание
avs mod param	work_set_size	Размер рабочей области AVS, KB.
	mb_source	Источник буфера памяти канала AVS. COMMON: общий пул буферов памяти; PRIVATE: частный пул буферов памяти. USER: пользовательский пул буферов памяти.
avs group attr	grp_id	Номер Grp ID.
mode Рабочий режим AVS.		
	enable	Запущен ли Grp. N: остановлен; Y: запущен.
pipe_num Число Pipe.		
	is_sync	Синхронизируются ли изображения разных Pipe. N: остановлен; Y: запущен.
src_rate Исходная частота кадров Grp.		
dst_rate Целевая частота кадров Grp.		
avs lut attr	grp_id	Номер Grp ID.
	lut_data_acc	Точность таблицы поиска. HIGH: высокая; MEDIUM: средняя; LOW: низкая.
lut_data_path Каталог файла таблицы поиска.		
avs out attr	grp_id	Номер Grp ID.
proj_mode Режим выходной проекции.		
center_x Координата X центра проекции на выходном изображении.		
center_y Координата Y центра проекции на выходном изображении.		
fov_x Горизонтальный угол поля зрения.		
fov_y Вертикальный угол поля зрения.		
ori_yaw Исходный угол рыскания.		
ori_pitch Исходный угол тангажа.		
ori_roll Исходный угол крена.		
yaw Угол рыскания.		
pitch Угол тангажа.		
roll Угол крена.		
middel_lut_path Путь к промежуточной калибровочной таблице поиска.		
calib_path Путь к файлу калибровки.		
mask_path Путь к изображению маски.		
mesh_alpha_path Путь к выходным таблицам Mesh и Alpha.		
avs channel attr	grp_id	Номер Grp ID.

Параметр		Описание
chn_id Номер Chn ID.		
	enable	Запущен ли Chn. N: остановлен; Y: запущен.
Width Ширина канала.		
height Высота канала.		
	is_compress	Включено ли сжатие. N: остановлен; Y: запущен.
dym_range Динамический диапазон выходного изображения.		
depth Глубина очереди изображений канала, получаемых пользователем.		
src_rate Исходная частота кадров для управления частотой канала.		
dst_rate Целевая частота кадров для управления частотой канала.		
avs group work status attr	grp_id	Номер Grp ID.
now_cst_us Время выполнения последней завершённой задачи, us.		
avg_cst_us Среднее время выполнения последних 10 задач, ms.		
max_cst_us Время выполнения самой длительной задачи за всё время, us.		
now_itv_us Интервал между текущим и предыдущим выполнением задачи, us.		
run_cnt Общее число выполненных задач.		
suc_run_cnt Общее число успешно выполненных задач.		
fail_run_cnt Общее число задач, завершившихся ошибкой.		
avs pipe work status	grp_id	Номер Grp ID.
	Pipe0 Pipe1 Pipe2 ...	Число изображений, принятых соответствующим входным Pipe / число изображений, отброшенных соответствующим входным Pipe.
avs pipe queue	grp_id	Номер Grp ID.
	Pipe0 Pipe1 Pipe2 ...	Состояние очереди буферов соответствующего входного Pipe.

Команды отладки

- Запись данных

  При записи данных сохраняются все входные данные и выходные данные после сшивки; поддерживаются несжатые и сжатые AFBC данные. Например, при сшивке 6 входных потоков с одним выходом данные сохраняются в 7 файлах одного каталога. Следующие команды можно использовать только при нормальной работе программы.

  Пример команды

  Трижды записать данные AVS Grp 0 и сохранить их в каталоге /userdata/avs_record[grp][chn]_[n]/, где n — порядковый номер команды.

  ```
  ./dumpsys avs record 0 3 /userdata/
  ```

  Правила именования файлов

  pipe_input*[grp]*[pipe]*[w]x[h]*[syncID].bin

chn_output*/grp/*[chn]*/w/x/h/*[syncID].bin

- Проверка синхронизации кадров

  Проверяет, имеют ли входные изображения от VI одинаковые номера кадров при сшивке. Следующие команды необходимо использовать во время нормальной работы программы.

  - Включение проверки синхронизации кадров

    Пример команды

    Включить проверку синхронизации кадров AVS Grp 0.

    ```
    ./dumpsys avs open_sync_debug 0
    ```

  - Отключение проверки синхронизации кадров

    Пример команды

    Отключить проверку синхронизации кадров AVS Grp 0.

    ```
    ./dumpsys avs close_sync_debug 0
    ```

Пример вывода в последовательную консоль

```
AVS Input Frame Info:
PipeId:0, Seq:189, Y0-Y5: 00 c0 03 a0 12
PipeId:1, Seq:189, Y0-Y5: 00 c0 03 b0 98
PipeId:2, Seq:189, Y0-Y5: 00 c0 03 70 d4
PipeId:3, Seq:189, Y0-Y5: 00 c0 03 b0 58
PipeId:4, Seq:189, Y0-Y5: 00 c0 03 90 52
PipeId:5, Seq:189, Y0-Y5: 00 c0 03 b0 58
```

Вывод содержит номер Pipe, номер кадра и первые 5 byte кадра.

18. GDC

Отладочная информация

```
dumpsys gdc
--------------------------------------------------------------------------------
DUMP OF SERVICE gdc:
---------------------- gdc module params -----------------------------
g_max_job_num       g_max_task_num
 128                 200
---------------------- recent job info -----------------------------
seq_no  job_hdl  correct  correct_ex  q_src  q_src_a  q_pano  q_pano_a  state     task_num
in_size out_size  cost_time_us  hw_time_us
0       0        N        N           N      Y        N       N         proced    1         0
    0
4791          0
1       0        N        N           N      Y        N       N         proced    1         0
    0
5082          0
2       0        N        N           N      Y        N       N         proced    1         0
    0
4744          0
3       0        Y        N           N      N        N       N         proced    1
2073600  2073600  3061          0
4       0        N        N           N      Y        N       N         proced    1         0
    0
4783          0
5       0        N        N           N      Y        N       N         proced    1         0
    0
5096          0
```

```
6       0        N        N           N      Y        N       N         proced    1         0
    0
4755          0
7       0        Y        N           N      N        N       N         proced    1
2073600  2073600  3057          0
---------------------- max waste time recent job info -----------------------------
seq_no  job_hdl  correct  correct_ex  q_src  q_src_a  q_pano  q_pano_a  state     task_num
in_size out_size
0       0        N        N           N      Y        N       N         proced    1         0
    0
cost_time_us  hw_time_us
5559          0
---------------------- gdc latest task stat params ----------------------------
job  task  type          stat  auto_alloc  step_x  step_y
0    100   query_src_a   busy  Y           32      32

---------------------- gdc history task common params ----------------------------
job  task  type          img_w img_h max_cost_us  new_cost_us  new_process_cnt  all_process_cnt
0    100   correct       1920  1080  3624         3546         405              405
0    100   query_src_a   1920  1080  5658         5009         1215             1215

---------------------- gdc history correct common params ----------------------------
job  task  all_region  enable  lmf  en_bg_color  bg_color  hor_offset  ver_offset  trap_coef
fan_strength
0    100   4           Y       N    Y            0         33          5           0          0
mount_mode
wall

---------------------- gdc history correct region params ----------------------------
job  task  region  view_mode  in_rad  out_rad pan  tilt  h_zoom  v_zoom  out_x  out_y  out_w
out_h
0    100   0       none       0       442     180  180   4095    4095    0      0      960    540
0    100   1       normal     0       442     223  180   4095    4095    960    0      960    540
0    100   2       normal     0       442     180  250   4095    4095    0      540    960    540
0    100   3       normal     0       442     180  153   404     404     960    540    960    540

---------------------- gdc history query region params ----------------------------
job  task  query_type    point_num  pano_region
0    100   query_src_a   3000       0

---------------------- gdc job status ----------------------------
success   fail      cancel    all_job_num     free_num  begin_num busy_num  procing_num
1620      0         0         128             128       0         0         0
---------------------- gdc task status ----------------------------
success   fail      cancel    all_task_num    free_num  busy_num
1620      0         0         200             199       1

--------------------------------------------------------------------------------
END DUMP OF SERVICE gdc:
```

Анализ отладочной информации

Записывает последние завершённые задачи модуля GDC, последние задачи с максимальным временем обработки, накопленную статистику и другие сведения.

Описание параметров

Параметр		Описание
module para	g_max_job_num	Максимальное число job.
g_max_task_num Максимальное число task.		
recent job info. Сведения о недавно завершённом job	seq_no	Порядковый номер вывода, диапазон [0, 7].
job_hdl Номер handle данного job.		
correct Содержит ли job задачу коррекции «рыбьего глаза» (N: нет, Y: да).		
correct_ex Содержит ли job задачу коррекции «рыбьего глаза» в режиме XY (N: нет, Y: да).		
q_src Содержит ли job задачу получения координаты одной точки исходного изображения по скорректированному изображению (N: нет, Y: да).		
q_src_a Содержит ли job задачу получения координат нескольких точек исходного изображения по скорректированному изображению (N: нет, Y: да).		
да).		q_pano Содержит ли job задачу получения координаты одной точки панорамного изображения по скорректированному изображению (N: нет,
Y: да).		q_pano Содержит ли job задачу получения координаты одной точки панорамного изображения по скорректированному изображению (N: нет,
task_num Число task в данном job.		
state Состояние обработки job.		
in_size Суммарная площадь входных изображений всех task данного job, в пикселях.		
out_size Суммарная площадь выходных изображений всех task данного job, в пикселях.		
cost_time Время от отправки job (end_job) до успешного завершения, в us.		
hw_time Время аппаратной обработки job, в us.		
max waste time job info. Сведения о недавно выполненном job с максимальным временем обработки.	Все поля совпадают с полями Поля совпадают с полями recent job info.	Сведения о job с максимальным временем среди последних 200 задач. После превышения 200 задач максимальное значение сбрасывается.
gdc latest task stat params. Сведения о последней задаче	job	job id
	task	task id
	type	Тип задачи; none: отсутствует; correct: коррекция «рыбьего глаза»; correct_ex: коррекция «рыбьего глаза» в режиме XY; query_src: получение координаты одной точки исходного изображения по скорректированному изображению; query_src_a: получение координат нескольких точек исходного изображения; query_pano: получение координаты одной точки панорамного изображения; query_pano_a: получение координат нескольких точек панорамного изображения.
	stat	Состояние; idle: свободно; busy: занято.
auto_alloc Автоматически ли выделяется task ID (N: нет, Y: да).		
step_x Шаг по горизонтали X для коррекции «рыбьего глаза».		
step_y Шаг по вертикали Y для коррекции «рыбьего глаза».		

Параметр		Описание
gdc history task common params. Общие сведения об исторических задачах	job	job id
	task	task id
	type	Тип задачи; none: отсутствует; correct: коррекция «рыбьего глаза»; correct_ex: коррекция «рыбьего глаза» в режиме XY; query_src: получение координаты одной точки исходного изображения по скорректированному изображению; query_src_a: получение координат нескольких точек исходного изображения; query_pano: получение координаты одной точки панорамного изображения; query_pano_a: получение координат нескольких точек панорамного изображения.
img_w Ширина входного изображения.		
img_h Высота входного изображения.		
max_cost_us Максимальное время выполнения задачи, us.		
new_cost_us Время выполнения последней задачи, us.		
new_process_cnt Число изменений параметров задачи.		
all_process_cnt Число запусков задачи.		
gdc history correct common params. Общие сведения о коррекции «рыбьего глаза» для исторических задач.	job	job id
	task	task id
all_region Число областей коррекции одного изображения.		
enable Включено ли (N: нет, Y: да).		
lmf Используются ли заданные пользователем параметры LMF объектива «рыбий глаз» (N: нет, Y: да).		
en_bg_color Заполняется ли выходное изображение фоновым цветом (N: нет, Y: да).		
bg_color Фоновый цвет в формате RGB888.		
hor_offset Горизонтальное смещение центра объектива относительно центра SENSOR.		
ver_offset Вертикальное смещение центра объектива относительно центра SENSOR.		
trap_coef Коэффициент силы трапецеидальной коррекции; используется для коррекции угла тангажа при настенном монтаже.		
fan_strength Коэффициент силы веерной коррекции; действует только в режиме 180.		
mount_mode Режим установки объектива «рыбий глаз».		
gdc history correct region params. Сведения об области коррекции «рыбьего глаза» для исторических задач.	job	job id
	task	task id
region ID области коррекции.		
	view_mode	Режим коррекции данной области; 360_pano: режим панорамы 360°; 180_pano: режим панорамы 180°; normal: обычный режим коррекции; none: без коррекции, исходное изображение только масштабируется.

Параметр		Описание
in_rad В режиме панорамы 360° — внутренний радиус исходного изображения, соответствующий области коррекции; в других режимах не действует.		
out_rad В режиме панорамы 360° — внешний радиус исходного изображения, соответствующий области коррекции; в других режимах — видимый радиус области коррекции.		
pan Значение Pan параметров PTZ области коррекции.		
tilt Значение Tilt параметров PTZ области коррекции.		
h_zoom Значение горизонтального Zoom параметров PTZ области коррекции.		
v_zoom Значение вертикального Zoom параметров PTZ области коррекции.		
out_x Координата X выходной позиции области коррекции.		
out_y Координата Y выходной позиции области коррекции.		
out_w Ширина выходной области коррекции.		
out_h Высота выходной области коррекции.		
gdc history query region params. Сведения о запросах координат исторических задач.	job	job id
	task	task id
	query_type	Тип запроса; query_src: получение координаты одной точки исходного изображения по скорректированному изображению; query_src_a: получение координат нескольких точек исходного изображения; query_pano: получение координаты одной точки панорамного изображения; query_pano_a: получение координат нескольких точек панорамного изображения.
point_num Число запрашиваемых точек.		
pano_region ID панорамной области.		
gdc job status	success	Накопленное число успешно обработанных job.
fail Накопленное число job, обработанных с ошибкой.		
cancel Накопленное число отменённых пользователем job.		
all_job_num Общее число доступных задач GDC.		
free_num Число свободных job.		
begin_num Число созданных пользователем, но ещё не отправленных job.		
busy_num Число отправленных пользователем, но ещё не переданных аппаратуре задач.		
procing_num Число задач, обрабатываемых аппаратурой.		
gdc task status. Состояние task GDC	success	Накопленное число успешно обработанных task. Если job обработан успешно, все входящие в него Если job обработан успешно, все входящие в него task также считаются успешными. К числу успешно обработанных task добавляется число task в успешном job.
	fail	Накопленное число task, обработанных с ошибкой. Если job завершился с ошибкой, все входящие в него task также считаются ошибочными. К числу ошибочных task добавляется число task
	cancel	Накопленное число отменённых task. Если job отменён, все входящие в него task также отменяются. К числу отменённых task добавляется число task в отменённом job.
all_task_num Общее число Task GDC, обычно 200.		

Параметр		Описание
free_num Число свободных task.		
busy_num Число task, добавленных в job.		

19. ALL

Отладочная информация

```
dumpsys all
--------------------------------------------------------------------------------
DUMP OF SERVICE all:
rgn:
---------------------- region status of overlay ----------------------------
hdl       type      used      pixel_format   width     height    mb             virt

---------------------- region chn status of overlay ----------------------------
hdl       type      mod       dev       chn       is_show   x         y         fg_alpha
bg_alpha  layer

---------------------- region status of cover ----------------------------
hdl       type      used

---------------------- region chn status of cover ----------------------------
hdl       type      mod       dev       chn       is_show   x         y         width     height
   color     layer     coord_type

---------------------- region status of mosaic ----------------------------
hdl       type      used

---------------------- region chn status of mosaic ----------------------------
hdl       type      mod       dev       chn       is_show   x         y         width     height
   blk_size  layer


sys:
---------------------- bind relation table ----------------------------
src_mod   src_dev   src_chn   dst_mod   dst_dev   dst_chn   send_cnt
vdec      0         0         vo        0         0         182


venc:
---------------------- venc module_param ----------------------------
venc_max_chn_num
16

---------------------- venc chn attr ----------------------------
id        width     height    vir_w     vir_h     codec_typepix_format     stream_buf_count

---------------------- venc chn query stat ----------------------------
id        left_pics      left_strm_bytes     left_strm_frms      cur_packs      left_recv_pics

---------------------- venc chn roi info ----------------------------
id        index     is_intra  abs_qp    qp        x         y         width     height


vo:
-----------DEV CONFIG------------
DevId   DevEn   InfType InfSync
0       Y       HDMI    1920x1080p50
1       Y       EDP     1024x768p60
2       N       Unkown  Unkown
```

```
-------LAYER BIND CONFIG--------
DevId   Video   Gfx     Cursor
0       clu0    esm0    sm0
1       clu1    esm1    sm1
2       unkown  unkown  unkown
----------LAYER STATUS---------
LayId   LayEn   PixFmt  ImgW    ImgH    DispW   DispH   FrmRt   BufLens
0       Y       RGBA32  1920    1080    1920    1080    25      3
1       N       NV12    0       0       0       0       0       0
2       Y       RGBA32  1024    768     1024    768     25      3
3       N       NV12    0       0       0       0       0       0
4       N       NV12    0       0       0       0       0       0
5       N       NV12    0       0       0       0       0       0
6       N       NV12    0       0       0       0       0       0
7       N       NV12    0       0       0       0       0       0
LAYER clu0 CHANNEL STATUS:
ChnId   Prio    X       Y       W       H       ChnFrt  Show    Pause   Step    Cache
0       1       0       0       480     270     25      Y       N       N       8
LAYER clu1 CHANNEL STATUS:
ChnId   Prio    X       Y       W       H       ChnFrt  Show    Pause   Step    Cache
WBC STATUS:
WbcId   En      Src     W       H       Fmt     Frt     Depth
0       N       Dev0    0       0       NV12    0       0

vpss:
---------------------- vpss group attr ----------------------------
grp_id    max_w     max_h     pixel_format   dym_range src_rate  dst_rate       is_compress

---------------------- vpss channel attr ----------------------------
grp_id    chn_id    mode      width     height    dym_range src_rate  dst_rate  depth     align
  mirror    flip

---------------------- vpss group crop info ----------------------------
grp_id    crop_en   coor_type x         y         width     height

---------------------- vpss chn crop info ----------------------------
grp_id    chn_id    crop_en   coor_type x         y         width     height

---------------------- vpss group pic queue ----------------------------
grp_id    delay     backup

---------------------- vpss group pic info ----------------------------
grp_id    width     height    vir_w     vir_h     pix_format     dyn_range compress

---------------------- vpss chn output resolution ----------------------------
grp_id    chn_id    width     height    vir_w     vir_h     pix_format     dyn_range compress
send_ok   frm_rate

---------------------- vpss chn rotation attr ----------------------------
grp_id    chn_id    rotate


--------------------------------------------------------------------------------
END DUMP OF SERVICE all:
```

Анализ отладочной информации

Записывает конфигурацию и состояние всех зарегистрированных модулей; используется команда dumpsys или dumpsys all.

Описание параметров

Описание конкретных параметров см. в разделах соответствующих модулей.